Фаззинг: выявление уязвимостей методом грубой силы

Русский перевод практического руководства Майкла Саттона, Адама Грина и Педрама Амини по методам фаззинга, автоматизации тестирования и обнаружению уязвимостей в программах, форматах файлов, веб-приложениях и сетевых протоколах.

Больше крутых материалов в моём телеграм-канале @grogrigs

Часть I. Основы

Глава 1. Методологии обнаружения уязвимостей

> «Станет ли магистралей Интернета меньше?»
>
> - Джордж У. Буш, Конкорд, штат Нью-Гэмпшир, 29 января 2000 года

Спросите любого опытного исследователя безопасности, как он обнаруживает уязвимости, и вы, скорее всего, получите множество ответов. Почему? Существует целый ряд подходов, у каждого из которых есть свои преимущества и недостатки. Ни один подход нельзя назвать единственно правильным, и ни один отдельный метод не способен выявить все возможные уязвимости заданной цели. На высоком уровне выделяют три основных подхода к обнаружению уязвимостей безопасности: тестирование белого, чёрного и серого ящика. Различия между этими подходами определяются тем, к каким ресурсам вы как тестировщик имеете доступ. На одном краю спектра находится подход белого ящика, требующий полного доступа ко всем ресурсам. Сюда относятся исходный код, проектные спецификации и, возможно, даже сами программисты. На другом краю подход чёрного ящика практически не требует знания внутреннего устройства и по большей части ограничивается слепым тестированием. Хороший пример тестирования чёрного ящика - проверка на проникновение удалённого веб-приложения без доступа к исходному коду. Посередине находится тестирование серого ящика, определение которого зависит от того, кого вы спросите. В рамках этой книги тестирование серого ящика предполагает, например, наличие доступа к скомпилированным двоичным файлам и, возможно, к некоторой базовой документации.

В этой главе мы рассмотрим различные высокоуровневые и низкоуровневые подходы к обнаружению уязвимостей, начав с тестирования белого ящика, которое также могло встречаться вам под названиями тестирования прозрачного, стеклянного или полупрозрачного ящика. Затем мы определим тестирование чёрного и серого ящика, к которому относится фаззинг. Мы рассмотрим преимущества и недостатки каждого подхода, поскольку это даст необходимые исходные сведения для последующего изложения, где на протяжении всей оставшейся книги основное внимание будет уделяться исключительно фаззингу. Фаззинг - лишь один из подходов к обнаружению уязвимостей, и важно понимать, в каких случаях более практичными могут оказаться альтернативные подходы.

Тестирование белого ящика

Фаззинг как методология тестирования в основном относится к тестированию чёрного и серого ящика. Однако сначала мы рассмотрим распространённый альтернативный подход к обнаружению уязвимостей, применяемый разработчиками программного обеспечения и известный как тестирование белого ящика.

Анализ исходного кода

Анализ исходного кода можно выполнять вручную или с помощью средств автоматизации. Поскольку компьютерные программы обычно состоят из десятков или сотен тысяч строк кода, исключительно ручной анализ, как правило, непрактичен. Автоматизированные средства представляют собой бесценный ресурс, сокращающий утомительную работу по изучению отдельных строк кода, но они способны лишь выявить потенциально уязвимые или подозрительные участки. Чтобы определить, действительно ли обнаруженные проблемы реальны, необходим анализ, выполняемый человеком.

Средствам анализа исходного кода приходится преодолевать множество препятствий, чтобы выдавать полезные результаты. Подробное рассмотрение этих разнообразных и сложных проблем выходит за рамки данной книги, однако обратите внимание на следующий пример кода на языке C, который просто копирует слово `test` в десятибайтовый массив символов:

```
#include <string.h>
int main (int argc, char **argv)
{
char buffer[10];
strcpy(buffer, "test");
}
```

Теперь рассмотрим тот же пример кода, изменённый так, чтобы в массив символов можно было скопировать пользовательский ввод.

```
#include <string.h>
int main (int argc, char **argv)
{
char buffer[10];
strcpy(buffer, argv[1]);
}
```

Оба фрагмента кода используют функцию `strcpy()` для копирования данных в буфер, расположенный в стеке. Применять `strcpy()` при программировании на C/C++ обычно не рекомендуется, поскольку эта функция не проверяет границы буфера, в который копируются данные. В результате, если программист не позаботится о собственных проверках, может произойти переполнение, при котором данные окажутся за пределами предназначенного для них контейнера. Первый пример не вызовет переполнения буфера, поскольку размер строки `test` с учётом завершающего нулевого байта равен и всегда будет равен 5, а значит, он меньше десятибайтового буфера, в который выполняется копирование. Во втором случае переполнение буфера может произойти или не произойти в зависимости от размера аргумента командной строки, предоставленного пользователем. Ключевой вопрос здесь состоит в том, управляет ли пользователь входными данными, передаваемыми уязвимой функции. Простейший анализ кода пометил бы строку с `strcpy()` в обоих примерах как потенциально уязвимую. Однако, чтобы понять, действительно ли существует условие, пригодное для эксплуатации, требуется проследить значения входных данных. Это не означает, что анализ исходного кода не может быть ценным инструментом для исследователей безопасности. Анализ исходного кода следует проводить всякий раз, когда код доступен. Однако в зависимости от вашей роли и точки зрения доступ к сведениям такого уровня нередко отсутствует.

Люди часто ошибочно считают тестирование белого ящика более эффективным, чем тестирование чёрного ящика. Казалось бы, какой взгляд на программное обеспечение может быть лучше или полнее, чем доступ к исходному коду? Однако помните, что применительно к исходному коду вы не обязательно исполняете то, что видите. В процессе сборки программного обеспечения при преобразовании исходного кода в ассемблерный могут происходить радикальные изменения. По этой и другим причинам нельзя утверждать, что один подход к тестированию непременно лучше другого. Это просто разные подходы, которые, как правило, выявляют разные типы уязвимостей.

Для полного охвата необходимо сочетать несколько подходов.

Утечка исходного кода Microsoft

В подтверждение утверждения о том, что анализ исходного кода не превосходит тестирование чёрного ящика, рассмотрим событие, произошедшее в феврале 2004 года. Без какого-либо предупреждения в Интернете начали распространяться архивы кода, которые, по слухам, содержали крупные фрагменты исходного кода операционных систем Microsoft Windows NT 4.0 и Windows 2000. Позднее Microsoft подтвердила подлинность архивов. Многие корпорации были крайне обеспокоены тем, что эта утечка быстро приведёт к потоку уязвимостей в двух популярных операционных системах. Этого не произошло. Фактически по сей день с утекшим исходным кодом можно связать лишь несколько уязвимостей. В этот короткий список входит CVE-2004-0566, где описано целочисленное переполнение, которое могло быть вызвано при отображении файлов растровых изображений.[1] Интересно, что Microsoft оспорила это открытие, заявив, что компания уже обнаружила проблему в ходе внутреннего аудита.[2] Почему же мы не увидели потока уязвимостей? Разве доступ к исходному коду не должен раскрыть их все? Истина состоит в том, что анализ исходного кода, хотя он и является очень важной составляющей аудита безопасности приложения или операционной системы, тоже может быть труден из-за размера и сложности кода. Кроме того, методы анализа дизассемблированного кода способны охватить значительную часть той же области. В качестве примера взгляните на проекты TinyKRNL[3] и ReactOS,[4] каждый из которых стремится обеспечить совместимость с ядром и операционной системой Microsoft Windows. Разработчики этих проектов не имели доступа к исходному коду ядра Microsoft, однако смогли создать проекты, которые хотя бы в некоторой степени предоставляют совместимую среду Windows. При аудите операционной системы Windows у вас, скорее всего, также не будет доступа к её исходному коду, но исходный код этих проектов можно использовать как ориентир при интерпретации дизассемблированного кода Windows.

Инструменты и автоматизация

Средства анализа исходного кода обычно относятся к одной из трёх категорий: средства проверки во время компиляции, обозреватели исходного кода или автоматизированные средства аудита исходного кода. Средства проверки во время компиляции ищут уязвимости по мере компиляции исходного кода. Обычно они интегрированы с компиляторами, однако пытаются выявлять именно проблемы безопасности, а не неполадки в работе приложения. Параметр компилятора `/analyze` в Microsoft Visual C++ - пример средства проверки во время компиляции.[5] Microsoft также предоставляет PREfast for Drivers,[6] способный выявлять различные классы уязвимостей, характерные для разработки драйверов и способные остаться незамеченными компилятором.

Обозреватели исходного кода - это инструменты, предназначенные для помощи в ручном изучении и анализе исходного кода. Средства этого класса позволяют проверяющему выполнять расширенный поиск по коду, а также перечислять перекрёстные ссылки между различными местами и переходить по ним. Например, с помощью такого инструмента проверяющий может найти все вызовы `strcpy()`, пытаясь выявить потенциальные уязвимости переполнения буфера. Cscope[7] и Linux Cross-Reference[8] - популярные обозреватели исходного кода.

Автоматизированные средства аудита исходного кода предназначены для сканирования исходного кода и автоматического выявления потенциально проблемных участков. Как и большинство средств безопасности, автоматизированные средства аудита исходного кода существуют и в коммерческом, и в бесплатном исполнении. Более того, эти средства обычно ориентированы на конкретные языки программирования, поэтому, если целевая программа создана с использованием

разных языков, может потребоваться несколько инструментов. Среди коммерческих предложений существуют продукты таких поставщиков, как Fortify,[9] Coverity,[10] KlocWork,[11] GrammaTech[12] и других. В таблице 1.1 перечислены некоторые популярные бесплатные инструменты, языки, которые они проверяют, и поддерживаемые ими платформы.

Таблица 1.1. Бесплатные средства аудита исходного кода

Название	**Язык или языки**	**Платформа или платформы**	**Адрес загрузки**
RATS (Rough Auditing Tool for Security)	C, C++, Perl, PHP, Python	UNIX, Win32	http://www.fortifysoftware.com/security-resources/rats.jsp
ITS4	C, C++	UNIX, Win32	http://www.cigital.com/its4/
Splint	C	UNIX, Win32	http://lclint.cs.virginia.edu/
Flawfinder	C, C++	UNIX	http://www.dwheeler.com/flawfinder/
Jlint	Java	UNIX, Win32	http://jlint.sourceforge.net/
CodeSpy	Java	Java	http://www.owasp.org/software/labs/codespy.html

Важно помнить, что никакое средство автоматизации никогда не заменит навыков опытного исследователя безопасности. Такие средства всего лишь упорядочивают непомерно трудную задачу изучения тысяч строк исходного кода и помогают сберечь время и избежать разочарований. Отчёты, создаваемые подобными средствами, всё равно должен проверить опытный аналитик, чтобы выявить ложные срабатывания, а затем разработчики, чтобы фактически внести исправление. Рассмотрим, например, следующий образец вывода, созданного Rough Auditing Tool for Security (RATS) при обработке приведённого ранее в этой главе искусственного примера уязвимого кода. RATS указывает на две потенциальные проблемы безопасности. Он обращает внимание на использование буфера фиксированного размера и на потенциальную небезопасность `strcpy()`. Однако он не утверждает однозначно, что уязвимость существует. Средство лишь привлекает внимание пользователя к потенциально небезопасным участкам кода и полагается на него в решении вопроса о том, действительно ли код небезопасен.

```
Entries in perl database: 33
Entries in python database: 62
Entries in c database: 334
Entries in php database: 55
Analyzing userinput.c
userinput.c:4: High: fixed size local buffer
Extra care should be taken to ensure that character arrays that are allocated on the
stack are used safely. They are prime targets for buffer overflow attacks.
userinput.c:5: High: strcpy
Check to be sure that argument 2 passed to this function call will not copy more data
than can be handled, resulting in a buffer overflow.
Total lines analyzed: 7
Total time 0.000131 seconds
53435 lines per second
Entries in perl database: 33
Entries in python database: 62
Entries in c database: 334
Entries in php database: 55
```

```
Analyzing userinput.c
userinput.c:4: High: fixed size local buffer
Extra care should be taken to ensure that character arrays that are allocated
on the stack are used safely.  They are prime targets for buffer overflow
attacks.
userinput.c:5: High: strcpy
Check to be sure that argument 2 passed to this function call will not copy
more data than can be handled, resulting in a buffer overflow.
Total lines analyzed: 7
Total time 0.000794 seconds
8816 lines per second
```

Преимущества и недостатки

Как уже говорилось, единственного правильного подхода к обнаружению уязвимостей безопасности не существует. Как же выбрать подходящую методологию? Иногда решение принимают за нас обстоятельства. Например, тестирование белого ящика невозможно без доступа к исходному коду цели. Именно в таком положении находится большинство исследователей безопасности и конечных пользователей, особенно работающих с коммерческим программным обеспечением в среде Microsoft Windows. В чём состоят преимущества тестирования белого ящика?

• **Охват.** Благодаря доступности всего исходного кода его анализ обеспечивает полный охват. На предмет потенциальных уязвимостей можно проверить все возможные пути выполнения кода. Разумеется, это может приводить к ложным срабатываниям, поскольку некоторые пути могут быть недостижимы во время исполнения программы.

Анализ кода возможен не всегда. Даже когда его можно провести, его следует сочетать с другими методологиями обнаружения уязвимостей. У анализа исходного кода есть следующие недостатки.

• **Сложность.** Средства анализа исходного кода несовершенны и выдают ложные срабатывания. Поэтому создаваемые такими средствами отчёты об ошибках - ещё не конечный результат. Их всё равно должны проверять опытные программисты, чтобы выявить проблемы, представляющие собой настоящие уязвимости безопасности. Поскольку крупные программные проекты обычно содержат сотни тысяч строк кода, отчёты могут быть длинными, а их изучение - занимать значительное время.
• **Доступность.** Исходный код доступен не всегда. Хотя многие проекты UNIX имеют открытый исходный код и допускают его анализ, в среде Win32 такая ситуация встречается сравнительно редко, а при работе с коммерческим программным обеспечением, как правило, отсутствует вовсе. Без доступа к исходному коду эта форма тестирования белого ящика даже не может рассматриваться.

Тестирование чёрного ящика

Тестирование чёрного ящика подразумевает, что вам известно только то, что вы можете наблюдать. Вы как конечный пользователь управляете входными данными, поступающими в чёрный ящик, и можете наблюдать выходные данные, появляющиеся с другого его конца, но ничего не знаете о внутренней работе цели. Чаще всего такая ситуация встречается при удалённом доступе к веб-приложениям и веб-службам. Вы можете формировать входные данные в виде запросов на языке гипертекстовой разметки (HTML) или расширяемом языке разметки (XML) и наблюдать, соответственно, созданную веб-страницу либо возвращаемое значение, но при этом не иметь никакого представления о происходящем внутри.

В качестве другого примера представьте, что при покупке такого приложения, как Microsoft Office, вы обычно получаете предварительно скомпилированную двоичную программу, а не исходный код,

использованный для её сборки. От вашей точки зрения в этой ситуации будет зависеть оттенок цвета ящика в применяемом подходе к тестированию. Если вы не намерены использовать методы обратной разработки, то фактически будете тестировать программное обеспечение как чёрный ящик. В противном случае вы примените подход серого ящика, который рассматривается в следующем разделе.

Ручное тестирование

Если продолжить использовать веб-приложения в качестве примера, ручное тестирование может заключаться в исследовании иерархии веб-сайта с помощью обычного веб-браузера и утомительной подстановке потенциально опасных входных данных во все замеченные интересующие области. На самых ранних этапах аудита этот приём может применяться от случая к случаю: например, к различным параметрам добавляют одинарные кавычки в надежде обнаружить уязвимость внедрения SQL-кода.

Ручное тестирование приложений на наличие уязвимостей без помощи автоматизированных средств обычно не является практичным подходом, если только ваша компания не нанимает огромную орду стажёров. Одним из случаев, когда оно может принести большую пользу, является массовая проверка нескольких приложений на сходную уязвимость. Такая проверка использует ситуации, когда многочисленные программисты совершают одинаковые ошибки в разных проектах. Например, в определённом сервере облегчённого протокола доступа к каталогам (LDAP) может быть обнаружено переполнение буфера, а проверка того же вектора эксплуатации на других LDAP-серверах покажет, что они также уязвимы. Поскольку программы часто используют общий код, а одни и те же программисты работают над несколькими проектами, такое происходит нередко.

Массовая проверка

`CreateProcess()` - функция, предоставляемая программным интерфейсом приложений (API) Microsoft Windows. Как следует из названия, `CreateProcess()` запускает новый процесс и его основной поток.[13] Прототип функции приведён ниже:

```
BOOL CreateProcess(
LPCTSTR lpApplicationName,
LPTSTR lpCommandLine,
LPSECURITY_ATTRIBUTES lpProcessAttributes,
LPSECURITY_ATTRIBUTES lpThreadAttributes,
BOOL bInheritHandles,
DWORD dwCreationFlags,
LPVOID lpEnvironment,
LPCTSTR lpCurrentDirectory,
LPSTARTUPINFO lpStartupInfo,
LPPROCESS_INFORMATION lpProcessInformation
);
```

Известно и документировано, что, если параметру `lpApplicationName` задано значение `NULL`, запускаемым процессом становится первое значение параметра `lpCommandLine`, отделённое пробелами. Рассмотрим, например, следующий вызов `CreateProcess()`:

```
CreateProcess(
NULL,
"c:\program files\sub dir\program.exe",
...
);
```

В этой ситуации CreateProcess() будет по очереди пытаться запустить каждое из следующих значений, разделённых пробелами.

```
c:\program.exe
c:\program files\sub.exe
c:\program files\sub dir\program.exe
```

Попытки будут продолжаться, пока не будет найден исполняемый файл или не исчерпаются все варианты. Поэтому, если поместить исполняемый файл program.exe в каталог c:`, приложения с небезопасными вызовами CreateProcess() описанной структуры выполнят program.exe`. Это даёт злоумышленникам удобный приём, позволяющий попытаться добиться исполнения файла, когда права на исполнение недоступны.

В ноябре 2005 года было опубликовано уведомление о безопасности,[14] в котором указывалось несколько популярных приложений, использовавших небезопасные вызовы CreateProcess(). Эти находки стали результатом успешной и очень простой массовой проверки. Если вы хотите провести такую же проверку, скопируйте и переименуйте простое приложение, например notepad.exe, и поместите его в каталог c:`. Затем пользуйтесь компьютером как обычно. Если скопированное приложение неожиданно запустится, вы, вероятно, обнаружили небезопасный вызов CreateProcess()`.

Автоматизированное тестирование, или фаззинг

Фаззинг по большей части является методом полного перебора, и недостаток изящества он компенсирует простотой и эффективностью. В главе 2 «Что такое фаззинг?» мы дадим этому термину определение и подробно его разберём. Если выразиться кратко, фаззинг состоит в том, чтобы обрушить на цель всё что угодно и следить за результатами. Большинство программ рассчитано на работу с определёнными входными данными, но должно быть достаточно устойчивым, чтобы корректно восстанавливаться после получения аномальных входных данных. Рассмотрим простую веб-форму, показанную на рисунке 1.1.



Рисунок 1.1. Простая веб-форма

Разумно предположить, что поле Name должно получать строковое значение, а поле Age - целое число. Что случится, если пользователь случайно перепутает входные значения и введёт строку в поле возраста? Будет ли строковое значение автоматически преобразовано в целое число на основе ASCII-кодов символов? Появится ли сообщение об ошибке? Произойдёт ли аварийное

завершение приложения? Фаззинг стремится отвечать на эти вопросы посредством автоматизированного процесса. Исследователю не требуется знать внутреннее устройство приложения, поэтому он проводит тестирование чёрного ящика. Вы стоите в стороне, бросаете в цель камни и ждёте звона разбитого окна. В этом смысле фаззинг относится к категории чёрного ящика. Однако на протяжении книги мы покажем, как сделать фаззинг методом полного перебора более хирургически точным, чтобы камень всякий раз летел прямо в цель; поэтому фаззинг применим и в категории серого ящика.

Преимущества и недостатки

Тестирование чёрного ящика, хотя оно и не всегда является лучшим подходом, возможно всегда. К его преимуществам относятся следующие:

• **Доступность.** Тестирование чёрного ящика применимо всегда и может быть полезно даже тогда, когда исходный код доступен.
• **Воспроизводимость.** Поскольку тестирование чёрного ящика не делает никаких предположений о цели, тест, предназначенный, например, для одного сервера протокола передачи файлов (FTP), легко перепрофилировать для проверки любого другого FTP-сервера.
• **Простота.** В отличие от таких подходов, как обратная разработка кода (RCE), требующих специальных навыков, тестирование исключительно чёрного ящика на самом базовом уровне можно проводить без глубокого знания внутреннего устройства приложения. Однако в действительности, хотя простые уязвимости отказа в обслуживании можно обнаружить одним лишь применением автоматизированных средств тестирования, обычно требуются более глубокие знания, чтобы определить, можно ли превратить простое аварийное завершение приложения во что-нибудь более интересное, например в исполнение кода.

> **Применяет ли Microsoft фаззинг?**
>
> Ответ - да. Опубликованный Microsoft в марте 2005 года документ Trustworthy Computing Security Development Lifecycle[15] (SDL) ясно показывает, что Microsoft считает фаззинг ключевым средством выявления уязвимостей безопасности до выпуска программного обеспечения. SDL представлял собой инициативу по встраиванию безопасности в жизненный цикл разработки программного обеспечения, которая возлагала ответственность за безопасность на всех участников процесса разработки. В SDL фаззинг упоминается как класс средств тестирования безопасности, которые следует использовать на этапе реализации. В документе прямо говорится, что «значительное внимание к фазз-тестированию было добавлено в SDL сравнительно недавно, но полученные к настоящему времени результаты весьма обнадёживают».

Несмотря на доступность тестирования чёрного ящика, у него есть некоторые недостатки. К ним относятся следующие:

• **Охват.** Одна из самых серьёзных трудностей тестирования чёрного ящика - определить, когда следует прекратить проверку и насколько эффективной она была. Подробнее мы рассмотрим эту проблему в главе 23 «Отслеживание работы фаззера».
• **Интеллектуальность.** Тестирование чёрного ящика лучше всего подходит для ситуаций, в которых уязвимость вызывается одним вектором входных данных. Однако сложные атаки могут требовать нескольких векторов: одни переводят целевое приложение в уязвимое состояние, а другие запускают эксплуатацию. Подобные атаки требуют глубокого понимания внутренней логики приложения и обычно обнаруживаются только посредством ручного аудита исходного кода и RCE.

Тестирование серого ящика

Находясь между анализом белого и чёрного ящика, тестирование серого ящика мы определяем как аудит чёрного ящика, дополненный знаниями, которые даёт обратная разработка (RE), также известная как обратная разработка кода (RCE). Исходный код - бесценный ресурс, который сравнительно легко читать и который даёт глубокое понимание работы конкретных функций. Кроме того, он показывает, какие входные данные ожидает определённая функция и какие выходные данные она должна создавать. Но с отсутствием этого замечательного ресурса потеряно ещё не всё. Анализ скомпилированных ассемблерных инструкций помогает раскрыть сходную картину, хотя и требует больше усилий. Оценку безопасности на уровне ассемблерного, а не исходного кода обычно называют аудитом двоичного кода.

Аудит двоичного кода

RCE часто употребляют как синоним выражения «аудит двоичного кода», но в рамках этой книги мы рассматриваем RCE как подкатегорию, чтобы отличать её от полностью автоматизированных методов. Конечная цель RCE - определить внутренние функции скомпилированного двоичного приложения. Хотя преобразовать двоичный файл обратно в его первоначальное представление в виде исходного кода невозможно, последовательности ассемблерных инструкций можно подвергнуть обратной разработке и получить представление, находящееся между исходным кодом и машинным кодом, из которого состоят двоичные файлы. Обычно это промежуточное представление сочетает язык ассемблера и графическое отображение потока выполнения кода в приложении.

После преобразования двоичного файла в форму, читаемую человеком, код можно анализировать в поисках мест, потенциально содержащих уязвимости, во многом так же, как при анализе исходного кода. Как и в случае анализа исходного кода, обнаружение потенциально уязвимых участков кода не является конечным результатом. Необходимо также определить, может ли конечный пользователь повлиять на уязвимый участок. Следуя этой логике, аудит двоичного кода можно назвать методом «изнутри наружу»: исследователь сначала находит интересную строку глубоко в дизассемблированном листинге, а затем прослеживает путь назад, чтобы определить, можно ли эксплуатировать уязвимость.

Обратная разработка - хирургически точный метод, в котором применяются такие инструменты, как дизассемблеры, декомпиляторы и отладчики. Дизассемблеры преобразуют нечитаемый машинный код в один из вариантов языка ассемблера, пригодный для восприятия человеком. Существуют различные бесплатные дизассемблеры, но для серьёзной работы по RCE вам, скорее всего, стоит приобрести DataRescue Interactive Disassembler (IDA) Pro,[16] показанный на рисунке 1.2. IDA - коммерческий дизассемблер, работающий на платформах Windows, UNIX и MacOS и способный анализировать двоичные файлы для широкого спектра архитектур.

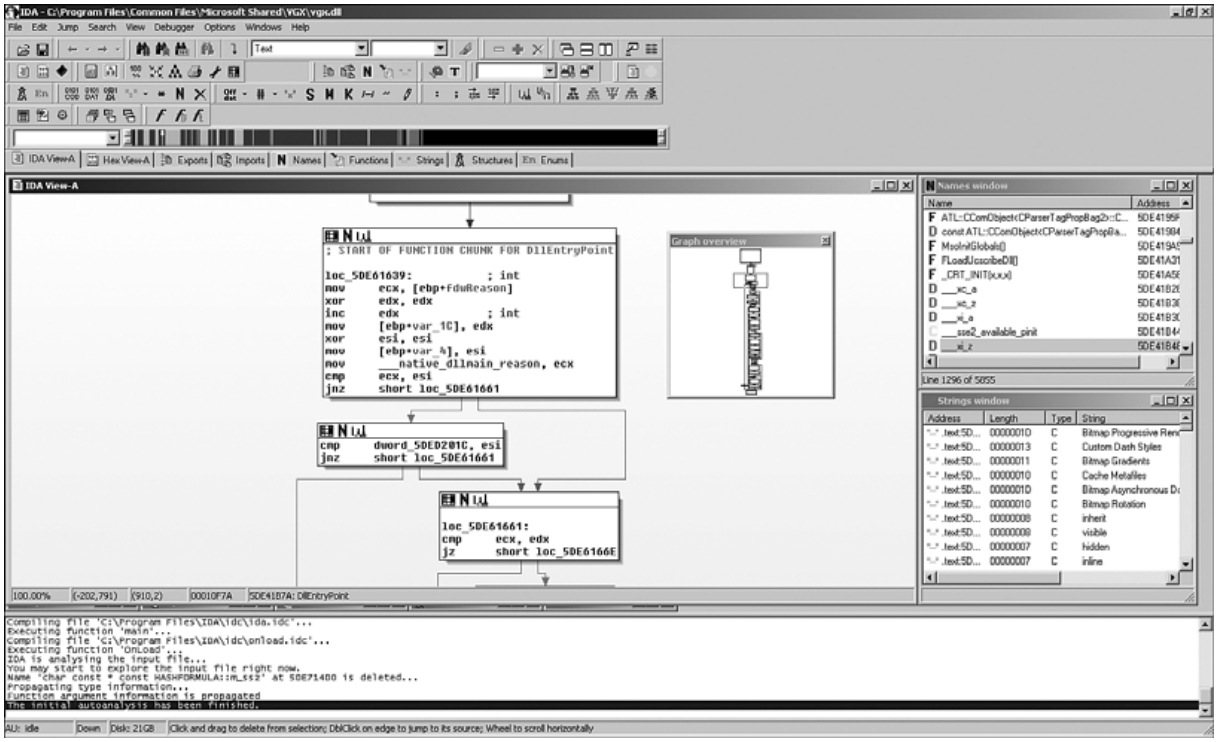


Рисунок 1.2. DataRescue IDA Pro

Подобно дизассемблеру, декомпилятор статически анализирует двоичный код и преобразует его в форму, читаемую человеком. Вместо прямого перевода в ассемблерные инструкции декомпилятор пытается получить конструкции языка более высокого уровня, такие как условия и циклы. Декомпиляторы не способны восстановить первоначальный исходный код, поскольку содержавшиеся в нём сведения - комментарии, имена переменных, имена функций и даже базовая структура - не сохраняются при компиляции. Декомпиляторы для языков, компилируемых в машинный код, например C и C++, обычно имеют существенные ограничения и носят преимущественно экспериментальный характер. Boomerang[17] - пример декомпилятора машинного кода. Декомпиляторы чаще встречаются для языков, компилируемых в некоторую промежуточную форму байт-кода, таких как C#, поскольку в скомпилированном коде сохраняется больше сведений и декомпиляция поэтому обычно успешнее.

В отличие от дизассемблеров и декомпиляторов, отладчики выполняют динамический анализ: открывают целевую программу или присоединяются к ней и следят за её исполнением. Отладчик может показывать содержимое регистров процессора и состояние памяти во время работы приложения. Среди популярных отладчиков для платформы Win32 - OllyDbg,[18] снимок экрана которого приведён на рисунке 1.3, и Microsoft WinDbg, произносимый как «wind bag».[19] WinDbg входит в пакет Debugging Tools for Windows[20] и доступен для бесплатной загрузки с сайта Microsoft. OllyDbg - условно-бесплатный отладчик, разработанный Олегом Ющуком и несколько более удобный для пользователя, чем WinDbg. Оба отладчика позволяют создавать собственные расширения, а для расширения возможностей OllyDbg доступно множество подключаемых модулей сторонних разработчиков.[21] Для среды UNIX существуют различные отладчики, из которых наиболее популярен и переносим GNU Project Debugger[22] (GDB). GDB - отладчик командной строки, входящий во многие дистрибутивы UNIX/Linux.

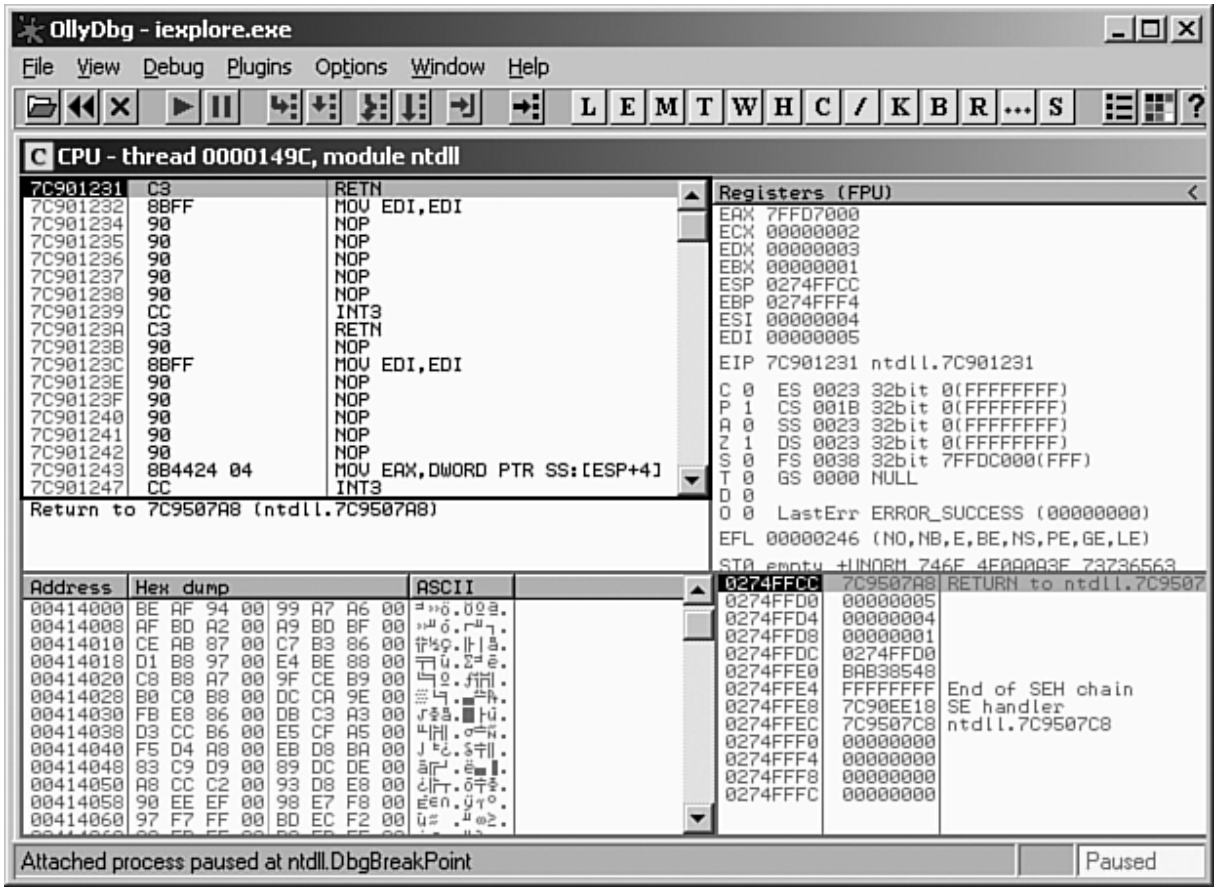


Рисунок 1.3. OllyDbg

Автоматизированный аудит двоичного кода

Появляется некоторое количество средств, пытающихся автоматизировать процесс RCE с целью выявления потенциальных уязвимостей безопасности в двоичных приложениях. Основные программы представлены как коммерческими, так и бесплатными продуктами и являются либо подключаемыми модулями IDA Pro, либо самостоятельными приложениями. Некоторые из них перечислены в таблице 1.2.

Таблица 1.2. Средства автоматизированного аудита двоичного кода

Название	Поставщик	Лицензия	Примечания
LogiScan	LogicLibrary	Коммерческая	LogicLibrary приобрела BugScan в сентябре 2004 года, после чего переименовала средство аудита двоичного кода и интегрировала его в решение управления Logidex SDA.

Название	Поставщик	Лицензия	Примечания
BugScam	Halvar Flake	Бесплатная	BugScam - набор сценариев IDC для IDA Pro, которые перечисляют вызовы функций в двоичном файле, чтобы выявить потенциально небезопасное применение различных библиотечных функций. Программа была написана главным образом как каламбур или пародия на BugScan.
Inspector	HB Gary	Коммерческая	Inspector - система управления RCE, объединяющая вывод различных средств RCE, таких как IDA Pro и OllyDbg.
SecurityReview	Veracode	Коммерческая	Продукт VeraCode интегрирует комплекс анализа двоичного кода непосредственно в среду разработки, подобно тому как работают комплексы анализа исходного кода наподобие Coverity. Анализ на уровне двоичного кода позволяет VeraCode избежать некоторых проблем, связанных с тем, что «вы исполняете не обязательно то, что видите».

Название	Поставщик	Лицензия	Примечания
BinAudit	SABRE Security	Коммерческая	На момент публикации BinAudit ещё не был выпущен. Однако, согласно веб-сайту SABRE Security, это подключаемый модуль для IDA Pro, предназначенный для выявления таких уязвимостей безопасности, как обращение к массиву за его границами, повреждение памяти при двойном освобождении, утечки памяти и так далее.

Преимущества и недостатки

Как уже отмечалось, тестирование серого ящика представляет собой своего рода гибридное решение, объединяющее традиционное тестирование чёрного ящика со знаниями, полученными благодаря RCE. Как и у других подходов, у тестирования серого ящика есть преимущества и недостатки. К преимуществам относятся следующие:

• **Доступность.** За исключением удалённых веб-служб и приложений, двоичные версии программного обеспечения доступны всегда.
• **Охват.** Сведения, полученные посредством анализа серого ящика, можно использовать для поддержки и совершенствования методов фаззинга, которые в ином случае относились бы исключительно к чёрному ящику.

Недостатки тестирования чёрного ящика включают следующее:

• **Сложность.** RCE представляет собой специализированный набор навыков, поэтому ресурсы для применения этого подхода могут отсутствовать.

Итоги

На самом высоком уровне методологии обнаружения уязвимостей можно разделить на тестирование белого, чёрного и серого ящика. Различие между этими тремя подходами определяется ресурсами, доступными тестировщику. При тестировании белого ящика доступны все возможные ресурсы, включая исходный код, тогда как при тестировании чёрного ящика доступ предоставляется лишь к входным данным и наблюдаемым результатам. Находящееся между ними тестирование серого ящика дополняет анализ чёрного ящика сведениями, полученными посредством RCE доступных двоичных файлов.

Тестирование белого ящика включает различные подходы к анализу исходного кода. Его можно выполнять вручную или с использованием автоматизированных средств, таких как средства проверки во время компиляции, обозреватели исходного кода или автоматизированные средства аудита исходного кода.

При тестировании чёрного ящика исходный код недоступен. Фаззинг посредством тестирования чёрного ящика считается слепым. Входные данные генерируются, а ответы отслеживаются, но для анализа недоступны никакие сведения о внутреннем состоянии цели. Фаззинг посредством тестирования серого ящика похож на подход чёрного ящика, но дополняется расширенными данными, полученными в ходе RCE. Фаззинг пытается многократно передавать приложению неожиданные входные данные, одновременно отслеживая возможные возникающие исключения. Оставшаяся часть этой книги посвящена фаззингу как подходу к обнаружению уязвимостей безопасности.

Сноски

[1] [назад] http://archives.neohapsis.com/archives/fulldisclosure/2004-02/0806.html
[2] [назад] http://news.zdnet.com/2100-1009_22-5160566.html
[3] [назад] http://www.tinykrnl.org/
[4] [назад] http://www.reactos.org/
[5] [назад] http://msdn2.microsoft.com/en-us/library/k3a3hzw7.aspx
[6] [назад] http://www.microsoft.com/whdc/devtools/tools/PREfast.mspx
[7] [назад] http://cscope.sourceforge.net/
[8] [назад] http://lxr.linux.no/
[9] [назад] http://www.fortifysoftware.com/
[10] [назад] http://www.coverity.com/
[11] [назад] http://www.klocwork.com/
[12] [назад] http://www.grammatech.com/
[13] [назад] http://msdn2.microsoft.com/en-us/library/ms682425.aspx
[14] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=340
[15] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnsecure/html/sdl.asp
[16] [назад] http://www.datarescue.com/
[17] [назад] http://boomerang.sourceforge.net/
[18] [назад] http://www.ollydbg.de/
[19] [назад] http://www.openrce.org/forums/posts/4
[20] [назад] http://www.microsoft.com/whdc/devtools/debugging/default.mspx
[21] [назад] http://www.openrce.org/downloads/browse/OllyDbg_Plugins
[22] [назад] http://www.gnu.org/software/gdb/gdb.html

Глава 2. Что такое фаззинг?

> «Они меня недонедооценили».
>
> - Джордж У. Буш, Бентонвилл, штат Арканзас, 6 ноября 2000 года

Термин «фаззинг» отсутствует в общеупотребительной лексике, имеет множество синонимов и для некоторых читателей может оказаться совершенно новым. Фаззинг - обширная область и увлекательный подход к анализу безопасности программного обеспечения. На протяжении книги мы подробно рассмотрим различные аспекты фаззинга и его цели. Но прежде в этой главе мы начнём с определения термина, изучим историю фаззинга, рассмотрим отдельные этапы полного аудита посредством фаззинга и в заключение разберём ограничения фазз-тестирования.

Определение фаззинга

Если вы поищете слово «фаззинг» в словаре, то вряд ли найдёте что-либо, что поможет определить этот термин в том смысле, в каком его используют исследователи безопасности. Первое публичное упоминание фаззинга появилось в исследовательском проекте Висконсинского университета в Мадисоне, и с тех пор термин стали применять для обозначения целой методологии тестирования программного обеспечения. В академическом мире фаззинг ближе всего к анализу граничных значений (BVA),[1] при котором рассматривают диапазон заведомо допустимых значений определённых входных данных и создают тестовые значения по обе стороны границы между заведомо допустимыми и недопустимыми значениями. Результаты BVA помогают убедиться, что обработка исключений надлежащим образом отфильтровывает нежелательные значения, пропуская при этом весь диапазон допустимых входных данных. Фаззинг похож на BVA, но при фаззинге нас интересуют не только граничные значения, но и любые входные данные, способные вызвать неопределённое или небезопасное поведение.

В рамках этой книги мы определяем фаззинг как метод обнаружения дефектов в программном обеспечении посредством передачи неожиданных входных данных и отслеживания исключений. Обычно это автоматизированный или полуавтоматизированный процесс, предполагающий многократное изменение данных и передачу их целевому программному обеспечению для обработки. Разумеется, это очень общее определение, однако оно передаёт основную идею фаззинга. Все фаззеры относятся к одной из двух категорий: фаззеры на основе мутаций изменяют существующие образцы данных для создания тестовых случаев, а фаззеры на основе генерации создают тестовые случаи с нуля, моделируя целевой протокол или формат файла. У обоих методов есть преимущества и недостатки. В главе 3 «Методы фаззинга и типы фаззеров» мы подробнее классифицируем методологии фаззинга и обсудим их достоинства и подводные камни.

Если вы ещё не знакомы с фаззингом, рассмотрите аналогию со взломом дома. Предположим, что нынешняя карьера у вас не сложилась и вы вступили на преступный путь. Применяя к проникновению в дом подход исключительно белого ящика, вы ещё до взлома получили бы полный доступ ко всей информации о доме. В неё могли бы входить чертежи, перечень изготовителей установленных замков, сведения о строительных материалах, использованных при возведении дома, и многое другое. Хотя такой подход даёт определённые преимущества, он не является ни безошибочным, ни лишённым недостатков. При этом методе вы статически анализируете проект дома, а не исследуете его во время исполнения, то есть в ходе самого проникновения. Допустим, в процессе изучения вы решите, что слабым местом является боковое окно гостиной, которое можно разбить и через него войти. Однако вы можете не предвидеть, что внутри вас ждёт разгневанный хозяин с дробовиком в руках. С другой стороны, применяя к проникновению метод исключительно чёрного ящика, вы могли бы под покровом темноты подойти к дому и тихо проверить все двери и окна, заглядывая внутрь, чтобы определить наилучшую точку входа.

Наконец, решив использовать для проникновения в дом фаззинг, вы могли бы отказаться от изучения чертежей и ручной проверки замков. Можно просто автоматизировать процесс, взяв ружьё и силой проложив себе путь в дом, - обнаружение уязвимостей методом полного перебора!

История фаззинга

Самое раннее известное нам упоминание фаззинга относится к 1989 году. Профессор Бартон Миллер, которого многие считают «отцом» фаззинга, и слушатели его курса по современным операционным системам разработали и применили примитивный фаззер для проверки устойчивости приложений UNIX.[2] Целью тестирования была не обязательно оценка безопасности системы, а скорее общее качество и надёжность кода. Хотя в исследовании вскользь упоминались соображения безопасности, при тестировании приложениям `setuid` не уделялось особого внимания. В 1995 году тестирование повторили на расширенном наборе утилит UNIX и операционных систем. Исследование 1995 года обнаружило общее повышение надёжности приложений, но всё же отметило «значительную частоту отказов».

Метод фазз-тестирования, применённый группой Миллера, был очень грубым. Если приложение аварийно завершалось или зависало, оно не проходило тест. Если этого не происходило, тест считался пройденным. Для этого в целевые приложения просто передавались случайные строки символов - подход исключительно чёрного ящика. Хотя он может показаться чрезмерно упрощённым, помните, что в то время сама идея фаззинга была сравнительно неизвестна.

Примерно в 1999 году в Университете Оулу началась работа над набором тестов PROTOS. Различные наборы тестов PROTOS разрабатывались посредством первоначального анализа спецификаций протоколов и последующего создания пакетов, которые либо нарушали спецификацию, либо, как предполагалось, могли обрабатываться конкретной реализацией протокола ненадлежащим образом. Предварительное создание таких наборов требовало значительных усилий, но после подготовки их можно было запускать на продуктах разных поставщиков. Это пример смешанного подхода белого и чёрного ящика, ставший важной вехой в развитии фаззинга благодаря большому числу дефектов, обнаруженных с его помощью.

В 2002 году Microsoft предоставила финансирование инициативе PROTOS,[3] а в 2003 году участники группы PROTOS основали Codenomicon - компанию, занимающуюся проектированием и выпуском коммерческих комплексов фазз-тестирования. Сегодня продукт по-прежнему основан на первоначальном наборе тестов Оулу, но, помимо прочего, включает графический пользовательский интерфейс, поддержку пользователей и обнаружение сбоев посредством функции проверки работоспособности.[4] Дополнительные сведения о Codenomicon и других коммерческих решениях для фаззинга представлены в главе 26 «Взгляд в будущее».

Когда в 2002 году PROTOS достиг зрелости, Дейв Эйтел выпустил фаззер с открытым исходным кодом под названием SPIKE[5] на условиях Стандартной общественной лицензии GNU (GPL). Фаззер Эйтела реализует блочный подход,[6] предназначенный для тестирования сетевых приложений. SPIKE использует более развитый подход, чем фаззер Миллера; в частности, он позволяет описывать блоки данных переменной длины. Кроме того, SPIKE способен не только генерировать случайные данные, но и содержит библиотеку значений, которые с большой вероятностью вызывают сбои в плохо написанных приложениях. В SPIKE также входят заранее определённые функции, способные создавать распространённые протоколы и форматы данных. Среди них Sun RPC и Microsoft RPC - две коммуникационные технологии, лежавшие в основе многих уязвимостей прошлого. Выпуск SPIKE стал важной вехой: это был первый общедоступный каркас, позволявший пользователям без особых усилий создавать собственные фаззеры. Этот каркас неоднократно упоминается на протяжении книги, в том числе в главе 21 «Каркасы фаззинга».

Примерно одновременно с первоначальным выпуском SPIKE Эйтел также выпустил локальный фаззер для UNIX под названием sharefuzz. В отличие от фаззера Миллера, sharefuzz нацелен на переменные среды, а не на аргументы командной строки. Sharefuzz также применяет полезный приём, облегчающий процесс фаззинга. Он использует разделяемые библиотеки для перехвата вызовов функций, возвращающих значения переменных среды, и вместо фактических значений

возвращает длинные строки, пытаясь выявить уязвимости переполнения буфера.

Большинство новшеств в области фаззинга после выпуска SPIKE появлялось в виде средств для отдельных конкретных классов фаззинга. В 2004 году Михал Залевский[7] (также известный как lcamtuf) привлёк внимание к фаззингу веб-браузеров выпуском mangleme[8] - сценария общего шлюзового интерфейса (CGI), предназначенного для непрерывного создания некорректных HTML-файлов, которые многократно обновлялись в целевом веб-браузере. Вскоре появились и другие фаззеры для веб-браузеров. Х. Д. Мур и Авив Рафф создали Hamachi[9] для фаззинга дефектов реализации динамического HTML (DHTML), а затем вместе с Мэттом Мёрфи и Тьерри Цоллером разработали CSSDIE[10] - фаззер, нацеленный на разбор каскадных таблиц стилей.

Фаззинг файлов вошёл в моду в 2004 году. В том году Microsoft выпустила бюллетень безопасности MS04-028, где подробно описывалось переполнение буфера в механизме обработки файлов JPEG.[11] Хотя это была не первая обнаруженная уязвимость формата файлов, она привлекла внимание из-за количества популярных приложений Microsoft, использовавших общий уязвимый код. Уязвимости форматов файлов также поставили уникальную задачу перед специалистами, отвечавшими за защиту сетей. Хотя в последующие годы появилось удивительно много сходных уязвимостей, просто запретить проникновение потенциально уязвимых типов файлов в корпоративную сеть было нереально. Изображения и мультимедийные файлы составляют значительную долю интернет-трафика. Насколько увлекательным был бы Интернет без изображений? Более того, большинство уязвимостей, которые вскоре обрушились на Microsoft, относились к файлам документов Microsoft Office - типам файлов, критически важным практически для всех организаций. Уязвимости форматов файлов оказались превосходной целью для фаззинга на основе мутаций, поскольку образцы легко доступны и их можно быстро изменять один за другим, одновременно отслеживая сбои целевого приложения. В 2005 году мы выступили на конференции Black Hat USA Briefings[12] и выпустили серию средств на основе как мутаций, так и генерации, предназначенных для фаззинга форматов файлов, включая FileFuzz, SPIKEfile и notSPIKEfile.[13]

В 2005 году компания Mu Security начала разработку аппаратного устройства фаззинга, предназначенного для изменения данных протокола по мере их прохождения по сети.[14] Появление этого коммерческого поставщика совпало с современной тенденцией, свидетельствующей о повышенном интересе к фазз-тестированию. Мы начинаем наблюдать рост доступности коммерческих решений для фаззинга наряду с развитием свободно доступных технологий фаззинга. Кроме того, теперь существует большое сообщество разработчиков и исследователей безопасности, интересующихся фаззингом, о чём свидетельствует создание списка рассылки Fuzzing,[15] который ведёт Гади Эврон. Лишь время покажет, какие захватывающие новшества нас ожидают.

Фаззинг ActiveX стал популярным направлением в 2006 году, когда Дэвид Зиммер выпустил COMRaider, а Х. Д. Мур опубликовал AxMan.[16] Оба средства были ориентированы на элементы управления ActiveX, экземпляры которых могли создаваться веб-приложениями при обращении через веб-браузер Microsoft Internet Explorer. Уязвимости в таких приложениях, допускающие удалённую эксплуатацию, представляют высокий риск из-за большой пользовательской базы. Как выяснилось, элементы управления ActiveX являются отличной целью фаззинга, поскольку содержат в самом элементе описания интерфейсов, прототипов функций и переменных-членов, что позволяет выполнять интеллектуальное автоматизированное тестирование. ActiveX и фаззинг браузеров в целом подробнее изучаются в главе 17 «Фаззинг веб-браузеров» и главе 18 «Фаззинг веб-браузеров: автоматизация».

В истории фаззинга есть много других вех и примечательных событий, но уже приведённых сведений достаточно, чтобы составить общую картину. Графическое представление краткой истории фаззинга дано на рисунке 2.1.

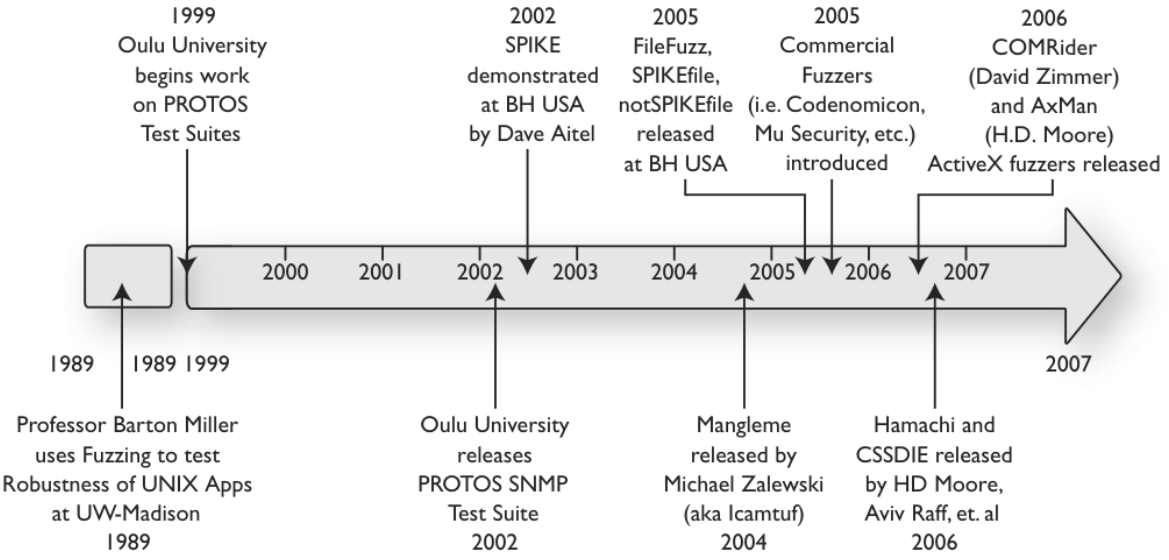


Рисунок 2.1. История фаззинга

Несмотря на достигнутый прогресс, фаззинг остаётся технологией в младенческом возрасте. Большинство существующих к настоящему времени средств представляет собой сравнительно небольшие проекты, которые поддерживают малые группы людей или даже один программист. Молодые начинающие компании вышли на коммерческий рынок лишь в последние несколько лет. Это не только придаёт фаззингу убедительность как самостоятельной области, но и указывает на то, что в ближайшие годы, по мере роста финансирования исследований, в фаззинге появится ещё много новшеств и будут достигнуты новые вехи.

В следующем разделе мы рассмотрим различные этапы проведения полного аудита посредством фаззинга.

Этапы фаззинга

В зависимости от различных факторов выбранный подход к фаззингу может существенно меняться. Единственно правильного подхода к фаззингу не существует. Всё полностью зависит от целевого приложения, навыков исследователя и формата фаззируемых данных. Однако независимо от того, что именно вы подвергаете фаззингу и какой подход выбираете, всегда применяются одни и те же основные этапы.

1. **Определить цель.** Невозможно выбрать средство или метод фаззинга, пока у нас нет конкретной цели. Если в ходе аудита безопасности вы подвергаете фаззингу приложение внутренней разработки, цель уже выбрана за вас. Если же вы проводите исследование уязвимостей с целью обнаружить их в сторонних приложениях, у вас будет определённая свобода. Выбирая цель, изучите историю поставщика с точки зрения ранее обнаруженных уязвимостей. Проверьте сайты-агрегаторы сведений об уязвимостях, такие как SecurityFocus[17] или Secunia.[18] Если в прошлом у поставщика было много уязвимостей, он, скорее всего, применяет некачественные методы программирования, которые в конечном итоге приведут к обнаружению новых уязвимостей. Помимо выбора приложения, может потребоваться нацелиться на конкретный файл или библиотеку внутри него. В таком случае стоит искать двоичные файлы, общие для нескольких приложений, поскольку уязвимости в подобных целях будут представлять более высокий риск из-за расширенной пользовательской базы.

2. **Определить входные данные.** Практически все уязвимости, пригодные для эксплуатации, возникают из-за того, что приложения принимают пользовательский ввод и обрабатывают эти данные, предварительно не очищая их и не применяя процедуры проверки. Перечисление векторов входных данных имеет решающее значение для успеха фаззинга. Неспособность найти потенциальные источники входных данных или ожидаемые входные значения может серьёзно ограничить тестирование. При поиске входных векторов мыслите нестандартно. Некоторые из них очевидны, другие гораздо менее заметны. В конечном счёте вектором входных данных следует считать всё, что отправляется клиентом цели. Сюда относятся заголовки, имена файлов, переменные среды, разделы реестра и так далее. Всё это следует считать входными векторами, а значит, потенциальными переменными фаззинга.
3. **Создать фаззированные данные.** После определения входных векторов необходимо создать данные для фаззинга. Решение о применении заранее заданных значений, изменении существующих данных или динамической генерации будет зависеть от цели и формата данных. Независимо от выбранного подхода этот процесс следует автоматизировать.
4. **Выполнить фаззированные данные.** Этот этап неразрывно связан с предыдущим; именно здесь фаззинг становится действием. Выполнение может заключаться в отправке пакета данных цели, открытии файла или запуске целевого процесса. И снова автоматизация имеет решающее значение. Без неё мы, строго говоря, не занимаемся фаззингом.
5. **Отслеживать исключения.** Жизненно важный, но часто упускаемый этап фаззинга - процесс отслеживания исключений или сбоев. Например, отправка целевому веб-серверу 10 000 фаззированных пакетов, в конце концов вызвавшая его отказ, бесполезна, если мы не способны точно определить пакет, ответственный за сбой. Отслеживание может принимать множество форм и зависит от целевого приложения и применяемого типа фаззинга.
6. **Определить возможность эксплуатации.** После выявления сбоя, в зависимости от целей аудита, может потребоваться установить, допускает ли обнаруженная ошибка дальнейшую эксплуатацию. Обычно это ручной процесс, требующий специальных знаний в области безопасности. Поэтому данный этап может выполнять не тот человек, который проводил первоначальный фаззинг.

Графическое представление этапов фаззинга приведено на рисунке 2.2.

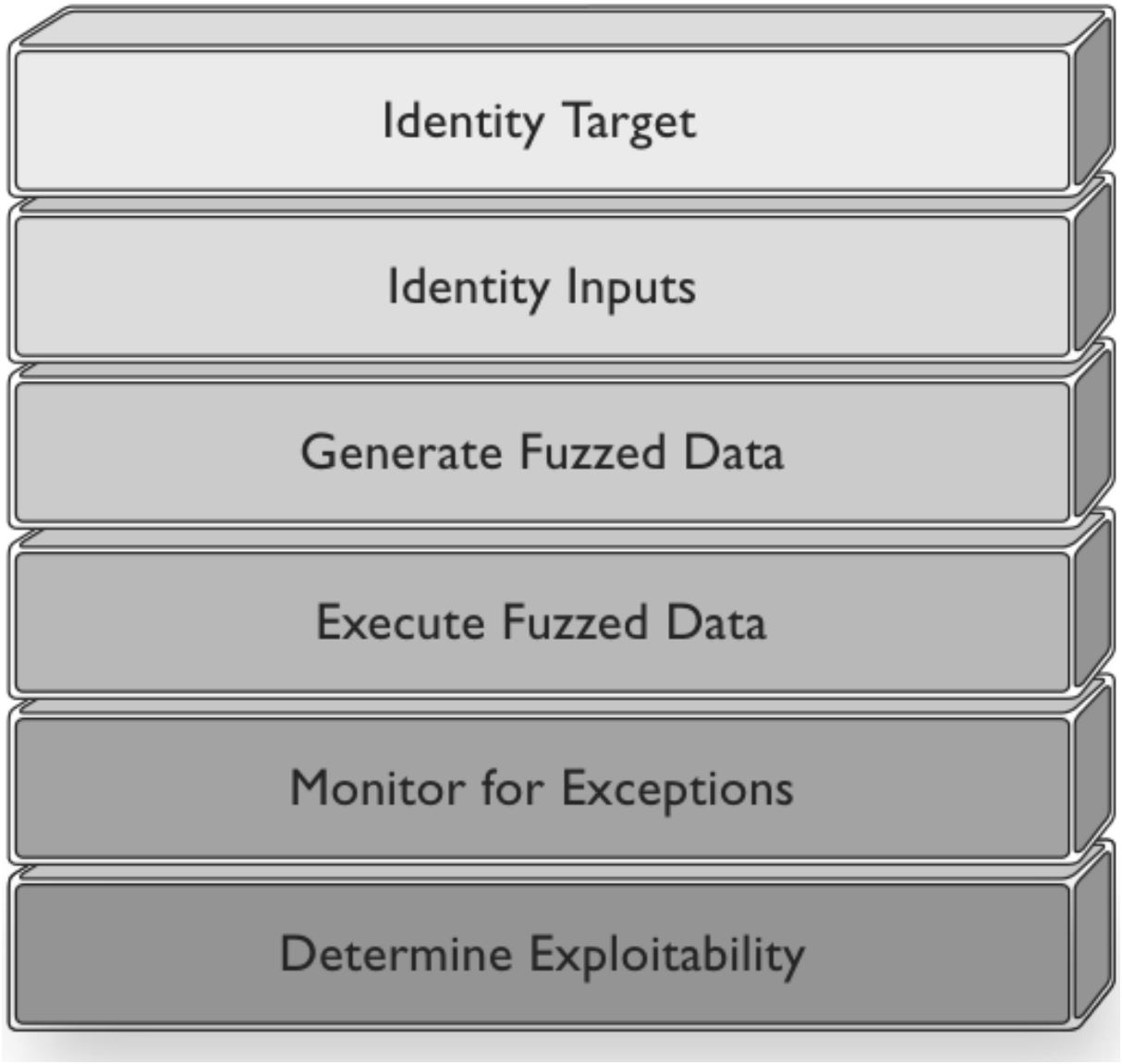


Рисунок 2.2. Этапы фаззинга

Независимо от проводимого типа фаззинга следует охватить все этапы, за возможным исключением определения возможности эксплуатации. Их порядок и внимание, уделяемое каждому из них, можно менять в зависимости от целей исследователя. При всей своей мощи фаззинг ни в коем случае не способен выявить 100 процентов ошибок в любой заданной программной цели. В следующем разделе мы опишем несколько классов ошибок, которые, скорее всего, останутся незамеченными.

Ограничения фаззинга и ожидания

По самой своей природе фаззинг хорошо подходит для обнаружения определённых типов слабых мест цели. В то же время он имеет конкретные ограничения в отношении типов обнаруживаемых уязвимостей. В этом разделе мы представим несколько классов уязвимостей, которые фаззер обычно не обнаруживает.

Недостатки контроля доступа

Некоторым приложениям для поддержки нескольких учётных записей или уровней пользователей требуются уровни привилегий. Рассмотрим, например, сетевую систему календарного планирования, доступную через веб-интерфейс. В приложении может быть определён администратор, управляющий тем, кому разрешено входить в систему. Может существовать особая группа пользователей, способных создавать календари. Все остальные пользователи могут иметь только права на чтение. Самая базовая форма контроля доступа гарантирует, что обычный пользователь не сможет выполнять административные задачи.

Фаззер может обнаружить дефект программы календарного планирования, позволяющий злоумышленнику получить полный контроль над системой. На низком уровне программные дефекты универсальны для разных целей, поэтому одну и ту же логику можно повсеместно применять для их обнаружения. В ходе тестирования фаззер также может успешно получить доступ к функциям, предназначенным только для администратора, используя лишь учётные данные обычного пользователя. Однако крайне маловероятно, что этот обход контроля доступа будет обнаружен. Почему? Учтите, что фаззер не понимает логику программы. Он никак не может знать, что административная область не должна быть доступна обычному пользователю. Реализовать в фаззере функции, учитывающие логику, возможно, но это может быть чрезвычайно сложно, а при тестировании других целей их, скорее всего, нельзя будет повторно применить без существенных изменений.

Неудачная логика проектирования

Фаззеры также не являются лучшими средствами выявления неудачной логики проектирования. Рассмотрим, например, уязвимость, обнаруженную в Veritas Backup Exec, которая позволяет злоумышленникам удалённо обращаться к реестру сервера Windows для создания, изменения или удаления разделов реестра.[19] Такой доступ может почти тривиальным образом привести к полной компрометации системы. Эта возможность существует благодаря прослушивающей службе, реализующей интерфейс удалённого вызова процедур (RPC) на конечной точке протокола управления передачей (TCP), которая не требует аутентификации, но принимает команды для манипулирования реестром. Это не было проблемой аутентификации, поскольку по проекту она вообще не требовалась. Скорее, это было неуместное проектное решение, вероятно возникшее из-за неспособности предположить, что злоумышленники захотят потратить время на расшифровку языка описания интерфейсов Microsoft (IDL), применённого для описания целевой процедуры, а затем создать специальную утилиту для взаимодействия с сервером.

Хотя фаззер может обнаружить некачественный разбор данных, передаваемых через RPC-соединение и вызывающих некоторый низкоуровневый сбой, он не способен определить, что предоставленная функция небезопасна. Позднее эта идея играет важную роль при рассмотрении фаззинга ActiveX в главе 18 «Фаззинг веб-браузеров: автоматизация», поскольку многие элементы управления просто предоставляют методы, позволяющие злоумышленнику, например, создавать и выполнять файлы. Обнаружению таких проектных ошибок уделяется особое внимание.

Потайные входы

Для фаззера, имеющего ограниченные сведения о структуре целевого приложения или не имеющего их вовсе, потайной вход ничем не отличается от любой другой логики цели, например экрана входа. И то и другое - всего лишь векторы входных данных, получающие учётные данные для аутентификации. Более того, если фаззеру не предоставлено достаточно сведений, чтобы распознавать успешные входы, он никак не сможет выявить успешную попытку входа с жёстко заданным паролем, даже если случайно его обнаружит. Например, во время фаззинга поля пароля фаззер обычно обнаруживает некорректные входные данные, вызвавшие аварийное завершение, но не случайно угаданный жёстко заданный пароль.

Повреждение памяти

Проблемы повреждения памяти часто приводят к аварийному завершению целевого процесса. Тогда проблему можно распознать по тем же признакам, что и отказ в обслуживании. Однако некоторые проблемы повреждения памяти корректно обрабатываются целевым приложением и никогда не будут распознаны простым фаззером. Рассмотрим, например, уязвимость строки формата, которая может остаться незамеченной, если к целевому процессу не присоединить отладчик. На уровне машинного кода уязвимость строки формата часто сводится к одной нарушающей правила инструкции. Например, на компьютере x86 под управлением Linux признаком атаки на строку формата с `%n` часто будет сбой на следующей инструкции:

```
mov    %ecx,(%eax)
```

Если фаззер передаёт процессу случайные данные, изредка добавляя запускающие обработку формата символы вроде `%n`, регистр `eax` во многих случаях будет содержать не доступный для записи адрес, а какое-либо мусорное слово из стека. Тогда инструкция вызовет сигнал нарушения сегментации `SIGSEGV`. Если в приложении есть обработчик сигнала `SIGSEGV`, он может просто завершить этот процесс и создать новый. В некоторых приложениях обработчик сигнала может даже попытаться продолжить исполнение в том же процессе без перезапуска. При уязвимости строки формата это действительно становится возможным, хотя крайне не рекомендуется, поскольку до возникновения сигнала `SIGSEGV` память могла фактически не быть повреждена. Но что, если наш фаззер этого не обнаружит? Процесс не завершается аварийно, а корректно обрабатывает сбой посредством обработчика сигнала, следовательно, проблемы безопасности нет, верно? Помните, что успешный эксплойт строки формата записывает данные в области памяти, а затем использует эти управляемые участки памяти, чтобы незаметно получить контроль над процессом. Обработчик сигнала не остановит точный эксплойт, поскольку точный эксплойт вообще не вызовет сбоя.

В зависимости от глубины отслеживания цели фаззер может обнаружить такие ситуации, а может и не обнаружить.

Многоэтапные уязвимости

Эксплуатация не всегда сводится к атаке одного слабого места. Сложные атаки часто предполагают последовательное использование нескольких уязвимостей для компрометации компьютера. Возможно, первоначальный дефект предоставляет непривилегированный доступ к компьютеру, а следующий позволяет повысить привилегии. Фаззинг может помочь выявить отдельные дефекты, но обычно не способен связать в цепочку ряд мелких уязвимостей или иных событий, которые сами по себе не представляют интереса, чтобы обнаружить многовекторную атаку.

Итоги

Хотя фаззинг существует уже некоторое время, за пределами сообщества исследователей безопасности эта методология всё ещё не получила широкого понимания. По этой причине вы, вероятно, встретите столько же уникальных определений фаззинга, сколько людей спросите. Теперь, когда мы определили фаззинг для целей этой книги, кратко изложили его историю и задали ожидания, пора углубиться в типы фаззеров и применяемые ими методы.

Сноски

[1] [назад] http://en.wikipedia.org/wiki/Boundary_value_analysis
[2] [назад] http://www.cs.wisc.edu/~bart/fuzz/
[3] [назад] http://www.ee.oulu.fi/research/ouspg/protos/index.html
[4] [назад] http://www.codenomicon.com/products/features.shtml
[5] [назад] http://immunityinc.com/resources-freesoftware.shtml
[6] [назад] http://www.immunityinc.com/downloads/advantages_of_block_based_analysis.html
[7] [назад] http://lcamtuf.coredump.cx/
[8] [назад] http://lcamtuf.coredump.cx/mangleme/mangle.cgi
[9] [назад] http://metasploit.com/users/hdm/tools/hamachi/hamachi.html
[10] [назад] http://metasploit.com/users/hdm/tools/see-ess-ess-die/cssdie.html
[11] [назад] http://www.microsoft.com/technet/security/bulletin/MS04-028.mspx
[12] [назад] http://www.blackhat.com/presentations/bh-usa-05/bh-us-05-sutton.pdf
[13] [назад] http://labs.idefense.com/software/fuzzing.php
[14] [назад] http://www.musecurity.com/products/overview.html
[15] [назад] http://www.whitestar.linuxbox.org/mailman/listinfo/fuzzing
[16] [назад] http://metasploit.com/users/hdm/tools/axman/
[17] [назад] http://www.securityfocus.com/
[18] [назад] http://secunia.com/
[19] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=269

Глава 3. Методы фаззинга и типы фаззеров

> «Слишком много хороших врачей уходят из профессии. Слишком многие акушеры-гинекологи не могут заниматься своей любовью с женщинами по всей нашей стране».
>
> - Джордж У. Буш, Поплар-Блафф, штат Миссури, 6 сентября 2004 года

Фаззинг определяет общий подход к обнаружению уязвимостей. Однако под общим названием фаззинга скрываются различные отдельные методы реализации этой методологии. В данной главе мы начнём разбирать фаззинг, рассмотрев эти особые методы. Мы также изучим разные типы фаззинга, которые применяют данные методы и используются против определённых классов целей. Часть II книги посвящена гораздо более подробному разбору каждого из этих типов

фаззеров.

Методы фаззинга

В предыдущей главе мы упомянули, что все фаззеры относятся к одной из двух категорий. Фаззеры на основе мутаций изменяют существующие образцы данных для создания тестовых случаев, а фаззеры на основе генерации создают тестовые случаи с нуля, моделируя целевой протокол или формат файла. В этом разделе мы дополнительно разделим эти две категории на подкатегории. Общепринятого списка категорий фаззинга не существует, но, поскольку это первая книга о фаззинге, мы выбрали следующие пять групп.

Заранее созданные тестовые случаи

Как говорилось в предыдущей главе, именно этот метод применяется в каркасе PROTOS. Разработка тестовых случаев начинается с изучения конкретной спецификации, чтобы понять все поддерживаемые структуры данных и допустимые диапазоны значений каждой из них. Затем создаются жёстко заданные пакеты или файлы, которые проверяют граничные условия либо полностью нарушают спецификацию. После этого тестовые случаи можно использовать для проверки точности реализации спецификации в целевых системах. Первоначальное создание тестовых случаев может потребовать значительной работы, однако обладает тем преимуществом, что их можно повторно использовать для единообразного тестирования нескольких реализаций одного протокола или формата файла.

Недостаток заранее созданных тестовых случаев состоит в том, что фазз-тестирование этим способом по своей природе ограничено. Поскольку случайная составляющая отсутствует, фаззинг завершается сразу после исчерпания списка тестовых случаев.

Случайные данные

Это наименее эффективный метод с большим отрывом, но его можно применить для быстрой первоначальной проверки, позволяющей определить, не является ли код цели совершенно ужасным. При случайном подходе в цель просто извергаются псевдослучайные данные в надежде на лучшее или на худшее - в зависимости от вашей точки зрения. Простейшим примером такого подхода служит неизменно забавная, но иногда эффективная команда:

```
while [ 1 ]; do cat /dev/urandom | nc -vv target port; done
```

Эта однострочная команда читает случайные данные из устройства Linux `urandom`, а затем передаёт их на целевой адрес и порт с помощью netcat. Цикл `while` обеспечивает продолжение процесса, пока пользователь его не прервёт.

Хотите верьте, хотите нет, но в прошлом с помощью этого метода обнаруживали уязвимости, затрагивавшие критически важное программное обеспечение. Какой позор! Трудность по-настоящему случайного фаззинга состоит в восстановлении причины исключения в случае аварийного завершения сервера. Проследить, каким образом 500 000 случайных байтов привели к сбою сервера, бывает мучительно трудно. Чтобы сузить область поиска, следует записывать отправляемый трафик анализатором пакетов. Кроме того, будьте готовы провести немало времени в отладчике и дизассемблере. Отладка разрушения стека таким методом может быть сложной, особенно потому, что стек вызовов будет повреждён. На рисунке 3.1 показан снимок экрана отладчика, присоединённого к корпоративной утилите наблюдения за системой и сетью, которая

только что аварийно завершилась под воздействием атаки случайным фаззингом. Можете определить причину сбоя? Вряд ли, если только вы не Нострадамус. Чтобы точно установить причину проблемы, требуется гораздо больше исследований. Обратите внимание, что некоторые данные скрыты ради защиты виновных.

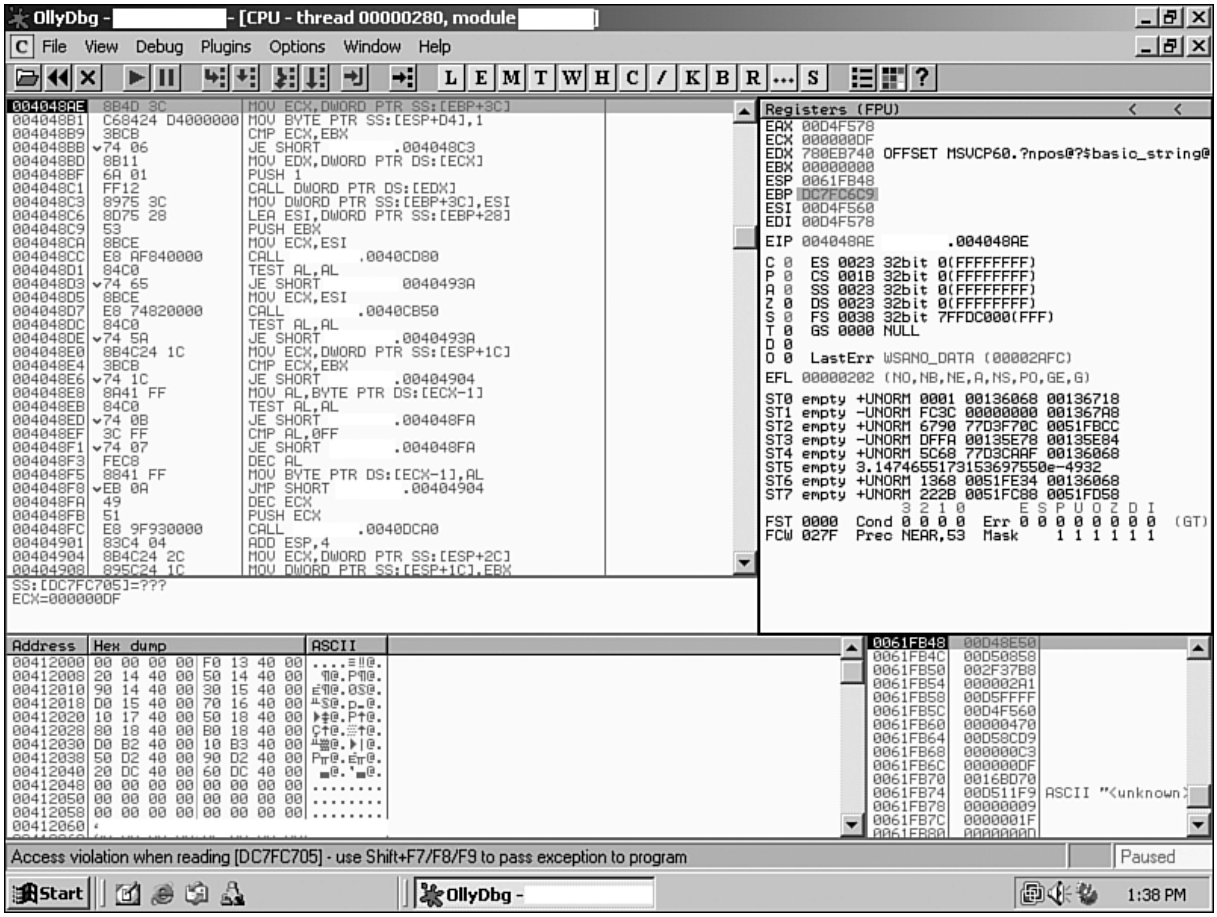


Рисунок 3.1. Что теперь? Дилемма фаззера на основе `/dev/urandom`

Ручное тестирование посредством мутаций протокола

Ручное тестирование протокола, пожалуй, ещё менее сложно, чем метод `/dev/urandom`. При ручном тестировании протокола никакой автоматизированный фаззер не участвует. Фактически фаззером является сам исследователь. Запустив целевое приложение, исследователь просто вводит неподходящие данные, пытаясь вызвать аварийное завершение сервера или иное нежелательное поведение. Это фаззинг для бедных в самом буквальном смысле, но в прошлом он, безусловно, успешно применялся. Преимущество этого подхода, разумеется, заключается в том, что во время аудита аналитик может опираться на прежний опыт и «внутреннее чутьё». Этот класс фаззинга чаще всего применяется к веб-приложениям.

Тестирование посредством мутаций, или полного перебора

Говоря здесь о полном переборе, мы имеем в виду фаззер, который начинает с допустимого образца протокола или формата данных и непрерывно искажает каждый отдельный байт, слово, двойное слово или строку внутри пакета данных либо файла. Это отличный первоначальный подход, поскольку он требует очень небольшого предварительного исследования, а реализация базового фаззера полного перебора сравнительно проста. От фаззера требуется лишь изменять данные и передавать их дальше. Разумеется, можно добавить множество дополнительных возможностей для обнаружения сбоев, журналирования и прочего, однако такое средство обычно удаётся создать за короткое время.

Этот подход несколько неэффективен, поскольку множество процессорных тактов тратится на данные, которые изначально невозможно интерпретировать. Однако этот недостаток отчасти компенсируется возможностью полностью автоматизировать процесс. Охват кода при подходе полного перебора зависит от набора проверяемых заведомо допустимых пакетов или файлов. Большинство спецификаций протоколов и определений форматов файлов сравнительно сложны, поэтому даже для приемлемого охвата потребуется множество образцов. Примерами фаззеров форматов файлов на основе полного перебора служат FileFuzz и notSPIKEfile для Windows и Linux соответственно.

Тестирование посредством автоматической генерации протокола

Тестирование посредством автоматической генерации протокола - более совершенный метод полного перебора. При этом подходе сначала требуется предварительное исследование, позволяющее понять и интерпретировать спецификацию протокола или определение формата файла. Однако вместо создания жёстко заданных тестовых случаев исследователь составляет грамматику, описывающую работу спецификации протокола. Тем самым он определяет части пакета или файла, которые должны оставаться неизменными, и части, представляющие собой переменные для фаззинга. Затем фаззер динамически разбирает эти шаблоны, создаёт данные фаззинга и отправляет получившийся пакет или файл цели. Успех этого подхода зависит от способности исследователя точно определить те части спецификации, которые с наибольшей вероятностью вызовут сбои в целевом приложении во время разбора. Примеры этого типа фаззинга - SPIKE и SPIKEfile. Оба средства принимают сценарии SPIKE с описанием целевого протокола или формата файла и применяют механизм фаззинга для создания искажённых данных. Недостаток подхода состоит во времени, необходимом для создания грамматики или определения.

Типы фаззеров

Мы рассмотрели разные методы, или подходы, фаззинга; теперь изучим некоторые конкретные типы фаззеров. Различные типы фаззеров определяются целью. Разные цели - разные звери, и для каждого типа подходит собственный класс фаззеров. В этом разделе мы кратко рассмотрим некоторые из этих типов. На протяжении оставшейся части книги мы подробно разберём каждый класс, включая как исходные сведения, так и конкретные примеры автоматизации и разработки.

Локальные фаззеры

В мире UNIX приложения `setuid` позволяют обычному пользователю временно получать повышенные привилегии. Это делает такие приложения очевидной целью фаззинга, поскольку любая уязвимость в приложении `setuid` позволит пользователю повысить привилегии на постоянной основе и выполнить код по своему выбору. У фаззинга приложений `setuid` есть две отдельные цели. Первая - фаззинг аргументов командной строки, сосредоточенный на передаче приложению `setuid` некорректных аргументов через командную строку. При втором варианте также передаются некорректные аргументы, но другим способом: через среду оболочки UNIX. Теперь рассмотрим фаззинг командной строки и переменных среды.

Фаззеры командной строки

При запуске приложение часто обрабатывает аргументы командной строки, переданные пользователем. Следующий пример, также использованный в главе 1 «Методологии обнаружения уязвимостей», демонстрирует простейшую форму переполнения стека параметром командной строки:

```
#include <string.h>
int main (int argc, char **argv)
{
char buffer[10];
strcpy(buffer, argv[1]);
}
```

Если бы эта программа действительно имела атрибут `setuid`, она представляла бы интерес для локального фаззера. Вопрос в том, как исследователи безопасности находят менее очевидные ошибки в приложениях `setuid`, исходный код которых недоступен. Вас не должно удивить, что самый простой ответ - фаззинг.

Для фаззинга командной строки полезны следующие средства:

• **clfuzz[1] от warl0ck.** Фаззер командной строки, проверяющий приложения на уязвимости строк формата и переполнения буфера.
• **iFUZZ[2] от Адама Грина.** Фаззер командной строки, проверяющий приложения на уязвимости строк формата и переполнения буфера. Включает параметры фаззинга нескольких разных аргументов, а также способен интеллектуально подвергать приложения фаззингу, если ему предоставлены сведения из справочного сообщения об использовании.

Фаззеры переменных среды

Другой тип локального фаззинга использует вектор переменных среды и также нацелен на приложения `setuid`. Рассмотрим следующее приложение, которое небезопасно использует значения из среды пользователя:

```
#include <string.h>
int main (int argc, char **argv)
{
char buffer[10];
strcpy(buffer, getenv("HOME"));
}
```

Существует несколько эффективных способов подвергнуть фаззингу использование переменных среды в приложении, но, несмотря на простоту процесса, автоматизированных средств немного. По-видимому, у многих исследователей безопасности есть собственные наспех собранные

сценарии для этой задачи, и отчасти именно поэтому общедоступных средств так мало. В большинстве случаев разработать локальный фаззер переменных среды не слишком сложно, и это часто не оправдывает публичного выпуска программы. Для фаззинга переменных среды полезны следующие средства:

• **Sharefuzz[3] от Дейва Эйтела.** Первый общедоступный фаззер переменных среды. Он перехватывает вызовы функции `getenv` и возвращает вредоносные данные.
• **iFUZZ[4] от Адама Грина.** Хотя iFUZZ главным образом является фаззером командной строки, пакет также включает базовые возможности фаззинга переменных среды. iFUZZ применяет тот же метод, что и Sharefuzz, но его несколько проще настраивать во время использования.

Локальный фаззинг и понятие `setuid` подробнее рассматриваются в главе 7 «Фаззинг переменных среды и аргументов» и главе 8 «Фаззинг переменных среды и аргументов: автоматизация».

Фаззеры форматов файлов

Множеству приложений, как клиентских, так и серверных, в какой-либо момент приходится иметь дело с файловым вводом и выводом. Например, антивирусным шлюзам часто требуется разбирать сжатые файлы, чтобы выяснить, что в них находится. Другой пример - офисный пакет, которому необходимо открыть документ. Приложения обоих типов могут быть подвержены уязвимостям, возникающим при разборе злонамеренно сформированных файлов.

Именно здесь нужен фаззинг форматов файлов. Фаззер формата файлов динамически создаёт различные некорректные файлы, которые затем открываются целевым приложением. Хотя конкретные методы фаззинга файлов не полностью совпадают с методами других типов фаззинга, общая идея остаётся той же. Для фаззинга форматов файлов полезны следующие средства:

• **FileFuzz[5] Майкла Саттона.** Средство фаззинга форматов файлов для Windows с графическим пользовательским интерфейсом (GUI).
• **notSPIKEfile и SPIKEfile[6] Адама Грина.** Средства фаззинга файлов для UNIX: первое не основано на SPIKE, а второе основано.
• **PAIMEIfilefuzz[7] Коди Пирса.** Ещё одно средство фаззинга форматов файлов для Windows с GUI, похожее на FileFuzz и построенное поверх каркаса обратной разработки PaiMei. PaiMei подробнее рассматривается далее в книге.

Фаззинг форматов файлов подробнее разбирается в главе 11 «Фаззинг форматов файлов», главе 12 «Фаззинг форматов файлов: автоматизация в UNIX» и главе 13 «Фаззинг форматов файлов: автоматизация в Windows».

Удалённые фаззеры

Удалённые фаззеры нацелены на программное обеспечение, прослушивающее сетевой интерфейс. Сетевые приложения, вероятно, являются наиболее часто выбираемыми целями фаззинга. С ростом Интернета практически у всех организаций появились общедоступные серверы, предоставляющие веб-страницы, электронную почту, разрешение имён системы доменных имён (DNS) и другие службы. Уязвимость в любой такой системе может предоставить злоумышленнику доступ к конфиденциальным данным или плацдарм для дальнейших атак на доверенные серверы.

Фаззеры сетевых протоколов

Фаззеры сетевых протоколов можно разделить на две основные категории: нацеленные на простые протоколы и предназначенные для сложных протоколов. Далее мы представим некоторые общие характеристики каждого типа.

Простые протоколы

Простые протоколы часто используют простую аутентификацию или не используют её вовсе. Зачастую они основаны на печатаемом тексте ASCII, а не на двоичных данных. Простой протокол не содержит полей длины или контрольной суммы. Кроме того, приложение обычно имеет немного состояний.

Пример простого протокола - FTP. В FTP весь обмен по каналу управления ведётся простым текстом ASCII. Для аутентификации требуются только имя пользователя и пароль в виде простого текста.

Сложные протоколы

Сложные протоколы обычно состоят из двоичных данных с отдельными читаемыми человеком строками ASCII. Для аутентификации может требоваться шифрование или какая-либо форма обфускации, а сам протокол может включать несколько сложных состояний.

Хороший пример сложного протокола - протокол удалённого вызова процедур Microsoft (MSRPC). MSRPC является двоичным протоколом, который перед передачей данных требует выполнить несколько шагов для установления канала связи. Протокол требует полей описания длины и фрагментации. В целом реализовывать этот протокол для фаззинга совсем не весело. Для фаззинга сетевых протоколов полезны следующие средства:

• **SPIKE[8] Дейва Эйтела.** SPIKE - первый публично выпущенный каркас фаззинга. Он включает заранее созданные сценарии фаззинга для нескольких популярных протоколов и также может использоваться как API.
• **Peach[9] Майкла Эддингтона.** Кроссплатформенный каркас фаззинга, написанный на Python. Он очень гибок и может применяться для фаззинга почти любой сетевой цели.

Фаззинг сетевых протоколов подробнее рассматривается в главе 14 «Фаззинг сетевых протоколов», главе 15 «Фаззинг сетевых протоколов: автоматизация в UNIX» и главе 16 «Фаззинг сетевых протоколов: автоматизация в Windows».

Фаззеры веб-приложений

Веб-приложения приобрели популярность как удобный способ доступа к внутренним службам, таким как электронная почта и оплата счетов. С появлением Web 2.0, что бы это на самом деле ни означало, традиционные настольные приложения, например текстовые процессоры, перемещаются в Интернет.[10]

При фаззинге веб-приложений исследователь прежде всего ищет характерные для них уязвимости, такие как внедрение SQL-кода, межсайтовый скриптинг (XSS) и прочие. Для этого требуются фаззеры, способные обмениваться данными по протоколу передачи гипертекста (HTTP) и сохранять ответы для дальнейшего анализа с целью выявления уязвимостей. Для фаззинга веб-приложений полезны следующие средства:

• **WebScarab[11] от OWASP.** Комплекс аудита веб-приложений с открытым исходным кодом и возможностями фаззинга.
• **SPI Fuzzer[12] от SPI Dynamics.** Коммерческий фаззер HTTP и веб-приложений, входящий в сканер уязвимостей WebInspect.
• **Codenomicon HTTP Test Tools[13] от Codenomicon.** Коммерческий набор тестов HTTP.

Фаззинг веб-приложений далее исследуется в главе 9 «Фаззинг веб-приложений и серверов» и главе 10 «Фаззинг веб-приложений и серверов: автоматизация».

Фаззеры веб-браузеров

Хотя технически фаззеры веб-браузеров представляют собой лишь особый тип фаззеров форматов файлов, мы считаем, что из-за популярности веб-приложений они заслуживают отдельного класса. Фаззеры веб-браузеров часто используют функции HTML для автоматизации процесса фаззинга. Например, утилита mangleme от lcamtuf, одно из первых общедоступных средств фаззинга браузеров, применяет тег `<META REFRESH>` для непрерывной автоматической загрузки тестовых случаев. Эта уникальная особенность веб-браузеров позволяет получить полностью автоматизированный фаззер клиентской стороны без сложных оболочек вокруг приложения. Для других фаззеров клиентской стороны такие оболочки обычно необходимы.

Фаззинг веб-браузеров не ограничивается разбором HTML: подходящих целей достаточно много. Например, средство See-Ess-Ess-Die нацелено на разбор каскадных таблиц стилей (CSS), а COM Raider - на объекты компонентной модели объектов (COM), которые можно загрузить в Microsoft Internet Explorer. К другим элементам, пригодным для фаззинга, относятся графика и заголовки ответов сервера. Для фаззинга веб-браузеров полезны следующие средства:

• **mangleme[14] от lcamtuf.** Первый публичный фаззер HTML. Это CGI-сценарий, многократно отправляющий браузеру искажённые данные HTML.
• **DOM-Hanoi[15] Х. Д. Мура и Авива Раффа.** Фаззер DHTML.
• **Hamachi[16] Х. Д. Мура и Авива Раффа.** Ещё один фаззер DHTML.
• **CSSDIE[17] Х. Д. Мура, Авива Раффа, Мэтта Мёрфи и Тьерри Цоллера.** Фаззер CSS.
• **COM Raider[18] Дэвида Зиммера.** Простой в использовании фаззер объектов COM, то есть элементов управления ActiveX, с графическим интерфейсом.

Фаззинг объектов COM и ActiveX подробно рассматривается в главе 17 «Фаззинг веб-браузеров» и главе 18 «Фаззинг веб-браузеров: автоматизация». Фаззинг CSS и графических файлов отдельно в контексте фаззинга веб-браузеров не рассматривается, но к нему применимы те же идеи, которые представлены в главах о фаззинге форматов файлов.

Фаззеры в памяти

Иногда во время тестирования определённые препятствия мешают быстрому и эффективному фаззингу. В таких случаях может пригодиться фаззинг в памяти. Основная идея проста, но разработка правильной реализации отнюдь не тривиальна. Один из подходов состоит в том, чтобы заморозить процесс, создать его снимок и быстро внедрять некорректные данные в одну из процедур разбора входных данных. После каждого тестового случая ранее созданный снимок восстанавливается и внедряются новые данные. Это повторяется до исчерпания всех тестовых случаев. Как и у любого другого метода фаззинга, у фаззинга в памяти есть преимущества и недостатки.

Преимущества включают следующее:

• **Скорость.** Не только отсутствуют требования к пропускной способности сети, но и можно исключить весь несущественный код, выполняемый между получением пакета из сети и его непосредственным разбором, тем самым повысив производительность тестирования.
• **Сокращение пути.** Иногда протокол использует собственные алгоритмы шифрования или сжатия либо содержит множество проверок контрольных сумм. Вместо того чтобы тратить время на создание фаззера, который способен всё это обрабатывать, фаззер в памяти может создать снимок в точке после распаковки, расшифрования или проверки контрольной суммы, тем самым избавив от значительной работы.

К недостаткам фаззинга в памяти относятся следующие:

• **Ложные срабатывания.** Поскольку необработанные данные внедряются в адресное пространство процесса, возможны случаи внедрения данных, которые никогда не могли бы поступить из внешнего источника.
• **Воспроизведение.** Хотя возникновение исключения может указывать на условие, пригодное для эксплуатации, исследователю всё равно придётся воссоздать исключение удалённо, вне процесса. Это может занять много времени.
• **Сложность.** Как мы увидим в главе 19 «Фаззинг в памяти» и главе 20 «Фаззинг в памяти: автоматизация», реализовать этот метод фаззинга чрезвычайно сложно.

Каркасы фаззинга

Каркасы фаззинга можно применять для фаззинга самых разных целей. Каркас фаззинга - просто универсальный фаззер или библиотека фаззера, упрощающая представление данных для многих типов целей. Некоторые уже упомянутые в этой главе фаззеры, включая SPIKE и Peach, на самом деле являются каркасами фаззинга.

Обычно каркас фаззинга включает библиотеку для создания фазз-строк, то есть значений, часто вызывающих проблемы в процедурах разбора. Также типичен набор процедур, упрощающих сетевой и дисковый ввод и вывод. Кроме того, каркасу фаззинга следует предоставлять некоторый сценарный язык, который можно использовать при создании конкретного фаззера. Каркасы фаззинга популярны, но отнюдь не являются исчерпывающим решением. У использования или проектирования каркаса есть как преимущества, так и недостатки. К преимуществам относятся следующие:

• **Повторное использование.** Если создать по-настоящему универсальный каркас фаззинга, его можно многократно применять при фаззинге разных целей.
• **Участие сообщества.** Значительно проще организовать совместную разработку большого проекта, который можно использовать множеством способов, чем привлечь большую группу разработчиков к маленькому проекту с очень узкой областью применения, как в случае фаззера конкретного протокола.

К недостаткам каркасов фаззинга относятся следующие:

• **Ограничения.** Даже при очень тщательном проектировании каркаса исследователь с большой вероятностью встретит цель, которая ему просто не подходит. Это может вызывать разочарование.

• **Сложность.** Иметь универсальный интерфейс фаззинга удобно, но даже простое обучение работе с каркасом может отнять много времени. Поэтому иногда выгоднее заново создать лишь те части механизма, которые нужны для конкретной цели, полностью отказавшись от каркаса.
• **Время разработки.** Проектирование чего-либо полностью универсального и пригодного для повторного использования требует гораздо больше работы, чем создание одноразового приложения для конкретного протокола. Эта работа почти всегда означает более длительную разработку.

Некоторые исследователи твёрдо убеждены, что каркасы - лучший подход для любого серьёзного фаззера, тогда как другие настаивают, что это не так и каждый фаззер следует создавать как одноразовое решение. Вам как читателю предстоит испытать оба подхода и решить самостоятельно.

Итоги

При проведении фазз-тестирования всегда следует оценивать пользу применения каждого метода, рассмотренного в этой главе. Часто наилучшие результаты даёт сочетание различных подходов, поскольку один метод может дополнять другой.

Подобно тому как существует множество разных целей, на которых может сосредоточиться фаззер, существует и множество классов фаззеров. По мере усвоения основных идей, лежащих в основе различных классов фаззеров, начинайте представлять, как вы могли бы реализовать некоторые из них. Это хорошо подготовит вас к критической оценке реализаций, подробно представленных далее в книге.

Сноски

[1] [назад] http://warl0ck.metaeye.org/cb/clfuzz.tar.gz
[2] [назад] http://fuzzing.org
[3] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[4] [назад] http://fuzzing.org
[5] [назад] http://fuzzing.org
[6] [назад] http://fuzzing.org
[7] [назад] https://www.openrce.org/downloads/details/208
[8] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[9] [назад] http://peachfuzz.sourceforge.net/
[10] [назад] http://www.google.com/a/
[11] [назад] http://www.owasp.org/index.php/Fuzzing_with_WebScarab
[12] [назад] http://www.spidynamics.com/products/webinspect/index.html
[13] [назад] http://www.codenomicon.com/products/internet/http/
[14] [назад] http://freshmeat.net/projects/mangleme/
[15] [назад] http://metasploit.com/users/hdm/tools/domhanoi/domhanoi.html
[16] [назад] http://metasploit.com/users/hdm/tools/hamachi/hamachi.html
[17] [назад] http://metasploit.com/users/hdm/tools/see-ess-ess-die/cssdie.html
[18] [назад] http://labs.idefense.com/software/fuzzing.php#more_comraider

Глава 4. Представление и анализ данных

> «Моя работа состоит в том, чтобы, типа, мыслить за пределами непосредственного».
>
> - Джордж У. Буш, Вашингтон, округ Колумбия, 21 апреля 2004 года

Компьютеры используют протоколы во всех аспектах внутреннего и внешнего обмена данными. Протоколы образуют основу структуры, необходимой для передачи и обработки данных. Чтобы успешно заниматься фаззингом, мы должны сначала понять протоколы, используемые нашими целями. Благодаря этому пониманию мы сможем нацеливаться на те части протоколов, которые с наибольшей вероятностью вызывают аномальные условия. Кроме того, чтобы обратиться к уязвимой части протокола, до начала фаззинга нам может потребоваться предоставить допустимые данные протокола. Знание как открытых, так и проприетарных протоколов неизбежно повысит эффективность обнаружения уязвимостей.

Что такое протоколы?

Протоколы необходимы для обеспечения обмена данными. Иногда они представляют собой определённые стандарты, а иногда - фактические стандарты, с которыми все в целом согласны. Устное общение служит примером протокола, использующего фактические стандарты. Хотя формальных документированных правил разговора не существует, обычно подразумевается, что, когда один человек говорит с другим, слушатель молчит, после чего получает возможность ответить. Точно так же нельзя просто подойти к другому человеку и начать извергать своё имя, адрес и прочие жизненно важные сведения, да в нынешнюю эпоху кражи личности этого и не следует делать. Вместо этого отдельным сведениям предшествуют метаданные. Например: «Меня зовут Джон Смит. Я живу по адресу Чамберс-стрит, 345». Чтобы осмысленно передавать данные, отправляющая и принимающая стороны должны договориться об определённых стандартах и правилах и понимать их. В вычислительной технике протоколы имеют решающее значение.

Компьютеры не обладают интеллектом. Они лишены такой роскоши, как интуиция. Поэтому для обмена и обработки данных жизненно необходимы строго определённые протоколы. Википедия определяет протокол как «соглашение или стандарт, управляющий соединением, обменом данными и передачей данных между двумя вычислительными конечными точками либо обеспечивающий их».[1] Эти конечные точки могут быть двумя отдельными компьютерами или двумя местами внутри одной машины. Например, при чтении данных из памяти компьютер должен обратиться к памяти на жёстком диске, переместить данные через шину данных в энергозависимую память, а затем передать их процессору. В каждой точке данные должны принимать определённую форму, чтобы отправляющая и принимающая конечные точки могли правильно их обработать. На самом низком уровне данные - не более чем набор битов. Только при восприятии в контексте этот набор битов обретает смысл. Если обе конечные точки не способны договориться о контексте, переданные данные бессмысленны.

Обмен между машинами также зависит от протоколов. Широко известный пример такого обмена - Интернет. Он представляет собой набор многочисленных многоуровневых протоколов. Домашний настольный компьютер в Соединённых Штатах может последовательно обернуть данные уровнями протоколов, а затем передать их через Интернет другому настольному компьютеру в Китае. Понимая те же протоколы, принимающий компьютер может снять оболочки и интерпретировать данные. В этом конкретном сценарии китайское правительство, вероятно, также перехватит переданные данные, снимет оболочки и самостоятельно их интерпретирует. Такой обмен возможен лишь потому, что обе участвующие машины понимают используемые протоколы. Между конечными точками находится множество маршрутизаторов, которые также должны понимать определённые аспекты протоколов, чтобы правильно направлять данные.

Хотя протоколы служат общей цели, они принимают разные формы. Некоторые протоколы рассчитаны на чтение человеком и представлены простым текстом. Другие представлены в двоичном виде - формате, не рекомендованном для восприятия человеком. Форматы файлов, используемые, например, для представления изображений GIF или электронных таблиц Microsoft Excel, являются двоичными протоколами.

Отдельные компоненты протокола обычно называют полями. Спецификация протокола определяет, как эти поля разделяются и упорядочиваются. Рассмотрим их подробнее.

Поля протокола

При проектировании протокола необходимо принять множество решений, но одно из важнейших касается того, как протокол будет разграничен или разделён на отдельные компоненты. И отправляющая, и принимающая машины должны знать, как интерпретировать отдельные элементы данных, и это определяется протоколом. Три типичных подхода к решению данной задачи - поля фиксированной длины, поля переменной длины и поля с разделителями. Простые текстовые протоколы часто разделяются такими символами, как возврат каретки. Когда клиент или сервер при разборе полученных данных встречает возврат каретки, это указывает на конец определённой команды. Поля с символьными разделителями также часто используются в форматах файлов, например в файлах значений, разделённых запятыми (CSV), которые применяются для двумерных массивов данных и могут читаться программами электронных таблиц, такими как Microsoft Excel. XML - ещё один пример типа файлов с символьными разделителями полей, однако вместо одного символа, обозначающего конец поля, XML использует для разграничения файла несколько символов. Например, элементы определяются открывающими и закрывающими угловыми скобками (< и >), а компоненты имени и значения атрибутов внутри элементов разделяются знаком равенства (=).

В полях фиксированной длины заранее задаётся определённое число байтов, используемых каждым полем. Такой подход часто применяется в заголовках сетевых протоколов, включая Ethernet, интернет-протокол (IP), протокол управления передачей (TCP) и протокол пользовательских дейтаграмм (UDP), поскольку эти заголовки требуют строго структурированных данных, необходимых при каждом использовании. Например, пакет Ethernet всегда начинается с шести байтов, определяющих целевой адрес управления доступом к среде (MAC), за которыми следуют ещё шесть байтов, определяющих исходный MAC-адрес.

> **Выбор полей с целью оптимизации**
>
> Ещё одно преимущество возможности предписать размер и положение определённых полей состоит в том, что автор спецификации может содействовать оптимизации процессора, разумно выбирая выравнивание по границам байтов. Например, многие поля заголовка IPv4 удобно выровнены по 32-битным границам. Как описано в разделе 3.41 руководства Intel по оптимизации, невыровненное чтение может дорого обходиться процессорам Pentium.
>
> Невыровненный доступ требует трёх тактов в семействе процессоров Pentium. На процессорах Pentium Pro и Pentium II невыровненный доступ, пересекающий границу строки кэша, требует от шести до девяти тактов. Расщепление блока кэша данных (DCU) - это обращение к памяти, пересекающее границу 32-байтовой строки. Невыровненные обращения, вызывающие расщепление DCU, приостанавливают процессоры Pentium Pro и Pentium II. Для наилучшей производительности убедитесь, что в структурах данных и массивах размером более 32 байт сама структура или элементы массива выровнены по 32-байтовой границе, а схемы доступа к структурам данных и элементам массивов не нарушают правил выравнивания.[2]
>
> И наоборот, некоторые архитектуры вычислений с сокращённым набором команд (RISC), например SPARC, при невыровненном обращении к памяти полностью откажут, выдав фатальный сигнал ошибки шины.

Когда данные менее структурированы, предпочтительными могут оказаться поля переменной длины. Такой подход часто применяется в мультимедийных файлах. Формат файла изображения или видео может быть весьма сложным и содержать множество необязательных заголовков и компонентов данных. Поля переменной длины дают разработчикам протокола гибкость и при этом позволяют создать эффективный протокол, экономно использующий память. Вместо выделения

для определённого элемента данных фиксированного числа байтов, которое может потребоваться не полностью, перед полем переменной длины обычно помещаются и идентификатор, указывающий тип поля, и значение размера, определяющее последующее количество байтов, образующих остаток поля.

Протоколы с простым текстом

Термин «протокол с простым текстом» обозначает протокол, в котором большинство передаваемых байтов данных находится в диапазоне печатаемых символов ASCII. Сюда входят все цифры, прописные и строчные буквы, такие символы, как знаки процента и доллара, а также возвраты каретки (`\r`, шестнадцатеричный байт `0x0D`), переводы строк (`\n`, шестнадцатеричный байт `0x0A`), табуляции (`\t`, шестнадцатеричный байт `0x09`) и пробелы (шестнадцатеричный байт `0x20`).

Протокол с простым текстом - это просто протокол, рассчитанный на чтение человеком. Такие протоколы обычно менее эффективны, чем двоичные аналоги, поскольку потребляют больше памяти, но существует множество ситуаций, когда протокол желательно сделать читаемым человеком. Канал управления протокола передачи файлов (FTP) - пример протокола с простым текстом. Канал передачи данных, напротив, способен передавать двоичные данные. FTP применяется для загрузки данных на удалённую машину и скачивания с неё. Благодаря тому, что управляющий трафик FTP читаем человеком, обмен можно вести вручную с помощью средств командной строки.

```
C:\>nc 192.168.1.2 21
220 Microsoft FTP Service
USER Administrator
331 Password required for Administrator.
PASS password
230 User Administrator logged in.
PWD
257 "/" is current directory.
QUIT
221
```

В предыдущем примере мы использовали популярное средство Netcat[3] для ручного подключения к серверу Microsoft FTP. Хотя существует множество клиентов FTP, Netcat позволяет полностью управлять запросами, отправляемыми серверу, и наглядно показать текстовую природу протокола. В этом примере весь полужирный текст обозначает запросы, отправляемые серверу, а обычный шрифт - ответы сервера. Мы ясно видим, что все запросы и ответы читаемы человеком. Благодаря этому очень легко понять, что именно происходит, и отладить любые возникающие проблемы.

Двоичные протоколы

Двоичные протоколы труднее расшифровывать человеку, поскольку вместо потока читаемого текста перед ним оказывается поток необработанных байтов. Без понимания протокола пакеты не будут особенно осмысленными. Для фаззинга понимание структуры протокола совершенно необходимо, если вы хотите выбирать в нём значимые места.

Чтобы лучше показать, как получить надлежащее понимание двоичного протокола для создания фаззера, мы изучим полнофункциональный протокол, который каждый день используют миллионы людей, обращающихся к AOL Instant Messenger (AIM), чтобы болтать с друзьями вместо работы. В частности, мы последовательно рассмотрим пакеты данных, отправляемые и получаемые во время сеанса входа.

Прежде чем погрузиться в особенности протокола AIM, обсудим несколько фундаментальных строительных блоков, применяемых при разработке двоичных протоколов. AIM - пример проприетарного протокола.[4] Хотя официальной документации нет, благодаря усилиям других людей по обратной разработке доступно множество сведений о структуре протокола. Официально протокол AIM называется OSCAR, то есть Open System for CommunicAtion in Realtime. Результаты обратной разработки позволили создать альтернативные клиенты, такие как GAIM[5] и Trillian.[6] Более того, некоторые анализаторы сетевых протоколов, например Wireshark, способны полностью декодировать структуру пакетов. Это важный момент. Если вы собираетесь создать фаззер для проприетарного протокола, никогда не начинайте с нуля, пытаясь выполнить обратную разработку протокола. Хотя в некоторых ситуациях это может быть необходимо, скорее всего, если подробности протокола или формата файла интересуют вас, они интересуют и других. Сначала обратитесь к своему доброму другу Google и выясните, какую работу другие уже сделали за вас. Для форматов файлов попробуйте Wotsit.org - превосходное собрание официальной и неофициальной документации для сотен проприетарных и открытых протоколов.

Теперь рассмотрим процесс аутентификации, или входа, AIM. Первый шаг в понимании протокола - захватить некоторое количество необработанных байтов и разложить их в более осмысленную структуру. В случае AIM нам повезло, поскольку Wireshark уже понимает структуру протокола. Мы пропустим несколько первоначальных пакетов установления связи и перейдём к моменту входа пользователя. На рисунке 4.1 показан вывод Wireshark для первоначального пакета, в котором имя пользователя клиента AIM отправляется серверу AIM. Видно, что протокол состоит из двух отдельных уровней. На верхнем уровне находится заголовок AOL Instant Messenger, определяющий тип отправляемого трафика. В данном случае заголовок указывает, что пакет содержит данные семейства AIM Signon (`0x0017`) и подсемейства Signon (`0x0006`).

На нижнем уровне под заголовком AIM Signon, Sign-on находятся собственно данные входа. На этом этапе отправляются только тип информации (`0x0001`), имя собеседника (`fuzzingiscool`) и длина имени собеседника в байтах (`13`). Это пример применения поля переменной длины. Обратите внимание, что непосредственно перед самим именем указана его длина. Как уже говорилось, двоичные протоколы часто используют блоки данных. Блок начинается со значения размера, указывающего длину данных, после чего содержит сами данные, то есть имя собеседника. Иногда значение размера включает байты, занятые самим значением размера, а иногда, как здесь, описывает только данные. Блоки данных - важная для фаззинга идея. Если вы создаёте фаззер, внедряющий данные в блоки, необходимо также соответствующим образом изменять размер блока, иначе принимающее приложение не сможет понять остаток пакета. Фаззеры сетевых протоколов, такие как SPIKE,[7] спроектированы с учётом этой идеи.

```
No.     Time        Source                Destination           Protocol Info
     15 3.225113    192.168.1.2           205.188.153.121       AIM Signon AIM Signon,
Sign-on Username: fuzzingiscool

Frame 15 (95 bytes on wire, 95 bytes captured)
Ethernet II, Src: Intel_a4:83:57 (00:0c:f1:a4:83:57), Dst: ZyxelCom_25:d5:72
(00:13:49:25:d5:72)
Internet Protocol, Src: 192.168.1.2 (192.168.1.2), Dst: 205.188.153.121
(205.188.153.121)
Transmission Control Protocol, Src Port: 4971 (4971), Dst Port: 5190 (5190), Seq: 11,
Ack: 11, Len: 41
AOL Instant Messenger
    Command Start: 0x2a
    Channel ID: SNAC Data (0x02)
    Sequence Number: 2
    Data Field Length: 35
    FNAC: Family: AIM Signon (0x0017), Subtype: Sign-on (0x0006)
AIM Signon, Sign-on
    Infotype: 0x0001
    Buddy: fuzzingiscool
        Buddyname len: 13
        Buddy Name: fuzzingiscool

0000  00 13 49 25 d5 72 00 0c f1 a4 83 57 08 00 45 00   ..I%.r.....W..E.
0010  00 51 87 c7 40 00 80 06 49 ff c0 a8 01 02 cd bc   .Q..@...I.......
0020  99 79 13 6b 14 46 1d 02 03 3b e2 5d 3c 37 50 18   .y.k.F...;.]<7P.
0030  ff f5 50 5c 00 00 2a 02 00 02 00 23 00 17 00 06   ..P\..*....#....
0040  00 00 00 00 00 00 00 01 00 0d 66 75 7a 7a 69 6e   ..........fuzzin
0050  67 69 73 63 6f 6f 6c 00 4b 00 00 00 5a 00 00      giscool.K...Z..
```

Рисунок 4.1. AIM Signon: отправка имени пользователя

В ответ на первоначальный запрос сервер, как показано на рисунке 4.2, предоставляет значение вызова (3740020309). Клиент AIM использует его для создания хеша пароля. Обратите внимание, что мы снова имеем дело с блоком данных, поскольку значению вызова предшествует его длина в байтах.

```
No.     Time        Source                Destination           Protocol Info
     17 3.253298    205.188.153.121       192.168.1.2           AIM Signon AIM Signon,
Sign-on Reply

Frame 17 (82 bytes on wire, 82 bytes captured)
Ethernet II, Src: ZyxelCom_25:d5:72 (00:13:49:25:d5:72), Dst: Intel_a4:83:57
(00:0c:f1:a4:83:57)
Internet Protocol, Src: 205.188.153.121 (205.188.153.121), Dst: 192.168.1.2
(192.168.1.2)
Transmission Control Protocol, Src Port: 5190 (5190), Dst Port: 4971 (4971), Seq: 11,
Ack: 52, Len: 28
AOL Instant Messenger
    Command Start: 0x2a
    Channel ID: SNAC Data (0x02)
    Sequence Number: 3005
    Data Field Length: 22
    FNAC: Family: AIM Signon (0x0017), Subtype: Sign-on Reply (0x0007)
AIM Signon, Sign-on Reply
    Signon challenge length: 10
    Signon challenge: 3740020309

0000  00 0c f1 a4 83 57 00 13 49 25 d5 72 08 00 45 20   .....W..I%.r..E
0010  00 44 aa 99 40 00 61 06 46 1a cd bc 99 79 c0 a8   .D..@.a.F....y..
0020  01 02 14 46 13 6b e2 5d 3c 37 1d 02 03 64 50 18   ...F.k.]<7...dP.
0030  40 00 b2 20 00 00 2a 02 0b bd 00 16 00 17 00 07   @.. ..*.........
0040  00 00 00 00 00 00 00 0a 33 37 34 30 30 32 30 33   ........37400203
0050  30 39                                            09
```

Рисунок 4.2. AIM Signon: ответ сервера

Получив вызов, клиент, как показано на рисунке 4.3, снова отвечает экранным именем, но на этот раз также включает хешированное значение пароля. Вместе с этими учётными данными для входа передаётся ряд сведений о клиенте, выполняющем вход, предположительно чтобы помочь серверу определить поддерживаемые клиентом функции. Эти значения передаются как пары «имя/значение» - ещё одна распространённая структура данных в двоичных протоколах. Примером может служить строка идентификатора клиента, то есть имя, передаваемая вместе со значением `AOL Instant Messenger, version 5.5.3591/WIN32`. И снова включается значение длины, описывающее размер поля значения.

```
No.     Time        Source                Destination           Protocol Info
     18 3.263960    192.168.1.2           205.188.153.121       AIM Signon AIM Signon, Logon

Frame 18 (215 bytes on wire, 215 bytes captured)
Ethernet II, Src: Intel_a4:83:57 (00:0c:f1:a4:83:57), Dst: ZyxelCom_25:d5:72
(00:13:49:25:d5:72)
Internet Protocol, Src: 192.168.1.2 (192.168.1.2), Dst: 205.188.153.121 (205.188.153.121)
Transmission Control Protocol, Src Port: 4971 (4971), Dst Port: 5190 (5190), Seq: 52, Ack: 39,
Len: 161
AOL Instant Messenger
    Command Start: 0x2a
    Channel ID: SNAC Data (0x02)
    Sequence Number: 3
    Data Field Length: 155
    FNAC: Family: AIM Signon (0x0017), Subtype: Logon (0x0002)
        Family: AIM Signon (0x0017)
        Subtype: Logon (0x0002)
        FNAC Flags: 0x0000
            .... .... .... ...0 = Followed By SNAC with related information: Not set
            0... .... .... .... = Contains Version of Family this SNAC is in: Not set
        FNAC ID: 0x00000000
AIM Signon, Logon
    TLV: Screen name
        Value ID: Screen name (0x0001)
        Length: 13
        Value: fuzzingiscool
    TLV: Password Hash (MD5)
        Value ID: Password Hash (MD5) (0x0025)
        Length: 16
        Value
    TLV: Unknown
        Value ID: Unknown (0x004c)
        Length: 0
        Value
    TLV: Client id string (name, version)
        Value ID: Client id string (name, version) (0x0003)
        Length: 45
        Value: AOL Instant Messenger, version 5.5.3591/WIN32
    TLV: Client id number
    TLV: Client major version
    TLV: Client minor version
    TLV: Client lesser version
    TLV: Client build number
    TLV: Client distribution number
    TLV: Client language
    TLV: Client country
    TLV: Use SSI

0000  00 13 49 25 d5 72 00 0c f1 a4 83 57 08 00 45 00   ..I%.r.....W..E.
0010  00 c9 87 c8 40 00 80 06 49 86 c0 a8 01 02 cd bc   ....@...I.......
0020  99 79 13 6b 14 46 1d 02 03 64 e2 5d 3c 53 50 18   .y.k.F...d.]<SP.
0030  ff d9 73 de 00 00 2a 02 00 03 00 9b 00 17 00 02   ..s...*.........
```

Рисунок 4.3. AIM Signon: отправка учётных данных для входа

Сетевые протоколы

И FTP, и AIM, подробно рассмотренные выше, являются примерами сетевых протоколов. Интернет целиком построен на сетевых протоколах, и недостатка в них нет. У нас есть протоколы передачи данных, маршрутизации, электронной почты, потокового мультимедиа, мгновенного обмена сообщениями и большего числа видов связи, чем можно пожелать. Как однажды сказал мудрец: «Самое замечательное в стандартах то, что их так много на выбор». Стандарты, представляющие собой сетевые протоколы, не исключение.

Как разрабатываются сетевые протоколы? Ответ в значительной степени зависит от того, является протокол открытым или проприетарным. Проприетарные протоколы могут разрабатываться закрытой группой внутри одной компании для использования конкретными продуктами, которые поддерживает и контролирует та же компания. В некотором смысле разработчики проприетарных протоколов обладают врождённым преимуществом, поскольку при согласовании стандарта им достаточно достичь единого мнения в небольшой группе разработчиков. Интернет-протоколы, напротив, по своей природе открыты и потому требуют согласия множества разнообразных групп. В целом интернет-протоколы разрабатываются и поддерживаются Инженерной группой Интернета (IETF).[8] В IETF действует длительный процесс публикации предлагаемых интернет-стандартов и получения отзывов о них. Он начинается с публикации запросов комментариев (RFC) - общедоступных документов, подробно описывающих протоколы и предназначенных для рецензирования коллегами. После надлежащего обсуждения и доработки IETF может принять RFC в качестве интернет-стандартов.

Форматы файлов

Подобно сетевым протоколам, форматы файлов описывают структуру данных и могут быть открытыми или проприетарными стандартами. Для иллюстрации рассмотрим документы Microsoft Office. Исторически Microsoft использовала проприетарные форматы в своих офисных приложениях, таких как Microsoft Office, Excel и PowerPoint. Конкурирующий открытый формат OpenDocument (ODF)[9] первоначально был создан OpenOffice.org, а позднее принят консорциумом Organization for the Advancement of Structured Information Standards (OASIS).[10] OASIS - отраслевая группа, поддерживаемая крупными поставщиками информационных технологий и работающая над разработкой и принятием стандартов электронного бизнеса.

В ответ на этот шаг OASIS в сентябре 2005 года Microsoft впервые отошла от проприетарного формата протоколов Microsoft Office,[11] объявив, что будущие версии Microsoft Office будут использовать новый формат Microsoft Office Open XML. Это открытый, но конкурирующий с ODF формат. Позднее Microsoft объявила о проекте Open XML Translator,[12] обещавшем средства преобразования между конкурирующими форматами XML.

Сравним открытый текстовый формат файла и проприетарный двоичный формат, сопоставив одинаковое содержимое, созданное в OpenOffice Writer и Microsoft Word 2003. В обоих случаях мы начнём с пустого документа, добавим слово `fuzzing`, используя шрифт по умолчанию, и сохраним файл. По умолчанию OpenOffice Writer сохраняет его как текстовый файл OpenDocument (`*.odt`). В шестнадцатеричном редакторе этот файл выглядит двоичным. Однако, если изменить расширение на `*.zip` и открыть его подходящей утилитой распаковки, станет видно, что это просто архив XML-файлов, каталогов, изображений и других текстовых файлов, как показывает следующий листинг каталога:

```
Directory of C:\Temp\fuzzing.odt

07/18/2006  12:07 AM    <DIR>          .
07/18/2006  12:07 AM    <DIR>          ..
```

```
07/18/2006  12:07 AM    <DIR>          Configurations2
07/18/2006  04:05 AM             2,633 content.xml
07/18/2006  12:07 AM    <DIR>          META-INF
07/18/2006  04:05 AM             1,149 meta.xml
07/18/2006  04:05 AM                39 mimetype
07/18/2006  04:05 AM             7,371 settings.xml
07/18/2006  04:05 AM             8,299 styles.xml
07/18/2006  12:07 AM    <DIR>          Thumbnails
               5 File(s)         19,491 bytes
               5 Dir(s)  31,203,430,400 bytes free
```

Все эти файлы описывают разные части содержимого или форматирования документа. Поскольку это XML-файлы, каждый из них можно прочитать в любом простом текстовом редакторе. Если взглянуть на файл `content.xml`, показанный далее, то после ряда элементов, описывающих пространства имён и параметры форматирования, ближе к концу файла можно ясно увидеть выделенное в оригинале полужирным исходное содержимое (`fuzzing`).

```
<?xml version="1.0" encoding="UTF-8"?>
<office:document-content
xmlns:office="urn:oasis:names:tc:opendocument:xmlns:office:1.0"
xmlns:style="urn:oasis:names:tc:opendocument:xmlns:style:1.0"
xmlns:text="urn:oasis:names:tc:opendocument:xmlns:text:1.0"
xmlns:table="urn:oasis:names:tc:opendocument:xmlns:table:1.0"
xmlns:draw="urn:oasis:names:tc:opendocument:xmlns:drawing:1.0"
xmlns:fo="urn:oasis:names:tc:opendocument:xmlns:xsl-fo-compatible:1.0"
xmlns:xlink="http://www.w3.org/1999/xlink"
xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:meta="urn:oasis:names:tc:opendocument:xmlns:meta:1.0"
xmlns:number="urn:oasis:names:tc:opendocument:xmlns:datastyle:1.0"
xmlns:svg="urn:oasis:names:tc:opendocument:xmlns:svg-compatible:1.0"
xmlns:chart="urn:oasis:names:tc:opendocument:xmlns:chart:1.0"
xmlns:dr3d="urn:oasis:names:tc:opendocument:xmlns:dr3d:1.0"
xmlns:math="http://www.w3.org/1998/Math/MathML"
xmlns:form="urn:oasis:names:tc:opendocument:xmlns:form:1.0"
xmlns:script="urn:oasis:names:tc:opendocument:xmlns:script:1.0"
xmlns:ooo="http://openoffice.org/2004/office"
xmlns:ooow="http://openoffice.org/2004/writer"
xmlns:oooc="http://openoffice.org/2004/calc"
xmlns:dom="http://www.w3.org/2001/xml-events"
xmlns:xforms="http://www.w3.org/2002/xforms"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
office:version="1.0">
<office:scripts/>
<office:font-face-decls>
<style:font-face style:name="Tahoma1" svg:font-family="Tahoma"/>
<style:font-face style:name="Times New Roman"
svg:font-family="&apos;Times New Roman&apos;"
style:font-family-generic="roman"
style:font-pitch="variable"/>
<style:font-face style:name="Arial"
svg:font-family="Arial"
style:font-family-generic="swiss"
style:font-pitch="variable"/>
<style:font-face style:name="Lucida Sans Unicode"
svg:font-family="&apos;Lucida Sans Unicode&apos;"
style:font-family-generic="system"
style:font-pitch="variable"/>
<style:font-face style:name="Tahoma"
svg:font-family="Tahoma"
style:font-family-generic="system"
style:font-pitch="variable"/>
</office:font-face-decls>
<office:automatic-styles/>
<office:body>
<office:text>
<office:forms form:automatic-focus="false"
form:apply-design-mode="false"/>
<text:sequence-decls>
<text:sequence-decl text:display-outline-level="0"
```

```
text:name="Illustration"/>
<text:sequence-decl text:display-outline-level="0"
text:name="Table"/>
<text:sequence-decl text:display-outline-level="0"
text:name="Text"/>
<text:sequence-decl text:display-outline-level="0"
text:name="Drawing"/>
</text:sequence-decls>
<text:p text:style-name="Standard">fuzzing</text:p>
</office:text>
</office:body>
</office:document-content>
```

Создав файл с тем же содержимым в Microsoft Word 2003, мы получим один двоичный документ Word (`*.doc`). Несмотря на двоичный формат, он фактически больше созданного ранее текстового файла OpenDocument: 20 Кбайт против 7 Кбайт. Открыв файл в шестнадцатеричном редакторе, вы заметите различные читаемые человеком строки вперемешку с байтами, похожими на бессмыслицу. Выбранные фрагменты ниже показывают исходное содержимое, авторов, перечисленных в свойствах документа, и другие строки, определяющие приложение, использованное для создания содержимого.

```
00000a00h: 66 75 7A 7A 69 6E 67 0D 00 00 00 00 00 00 00 00 ; fuzzing.........
…
000024d0h: 66 75 7A 7A 69 6E 67 00 1E 00 00 00 04 00 00 00 ; fuzzing.........
000024e0h: 00 00 00 00 1E 00 00 00 1C 00 00 00 4D 69 63 68 ; ............Mich
000024f0h: 61 65 6C 2C 20 41 64 61 6D 20 61 6E 64 20 50 65 ; ael, Adam and Pe
00002500h: 64 72 61 6D 00 00 00 00 1E 00 00 00 04 00 00 00 ; dram............
…
00002560h: 4D 69 63 72 6F 73 6F 66 74 20 4F 66 66 69 63 65 ; Microsoft Office
00002570h: 20 57 6F 72 64 00 00 00 40 00 00 00 00 00 00 00 ;  Word...@.......
```

Распространённые элементы протоколов

Зачем для фаззинга нужны эти исходные сведения? В зависимости от структуры формата файла или сетевого протокола вам придётся корректировать подход к фаззингу. Чем больше вы знаете о структуре данных, тем точнее сможете сосредоточить фаззинг на сегментах, с наибольшей вероятностью вызывающих аномальные условия. Рассмотрим некоторые распространённые элементы протоколов.

Пары «имя - значение»

Как в двоичных, так и в текстовых протоколах данные часто представлены парами «имя - значение», например `size=12`, но особенно характерно это для текстовых протоколов. В показанном ранее файле `content.xml` такие пары встречаются повсюду. Как правило, при работе с парами «имя - значение» потенциальные уязвимости вероятнее всего удастся выявить посредством фаззинга части значения.

Идентификаторы блоков

Идентификаторы блоков - это значения, определяющие тип данных, представленных в двоичном виде. За ними могут следовать данные переменной или фиксированной длины. В рассмотренном ранее примере AIM заголовок AIM Signon содержал заголовок блока типа информации (`0x0001`). Это значение определяло тип следовавших данных - в данном случае имя собеседника AOL. С помощью фаззинга можно выявлять другие, возможно недокументированные идентификаторы блоков, которые могут принимать дополнительные типы данных, также пригодные для фаззинга.

Размеры блоков

Размеры блоков были описаны ранее. Обычно блоки состоят из данных, например пар «имя - значение», которым предшествует один или несколько байтов, подробно определяющих тип поля и размер следующих данных переменной длины. При фаззинге попробуйте изменить размер так, чтобы он стал больше или меньше следующих данных, и проследите за результатами. Это распространённый источник переполнения и выхода за нижнюю границу буфера. И наоборот, при фаззинге данных внутри блока обязательно соответствующим образом изменяйте размер, чтобы приложение могло правильно определить данные.

Контрольные суммы

Некоторые форматы файлов встраивают контрольные суммы в разные части файла, помогая приложениям обнаруживать данные, которые по той или иной причине могли быть повреждены. Это не обязательно реализовано как мера безопасности, поскольку файлы могут повреждаться по разным причинам, однако механизм способен повлиять на результаты фаззинга: при неверной контрольной сумме приложения обычно прекращают обработку файла. Формат изображений PNG - пример типа файлов, использующего контрольные суммы. При встрече с контрольными суммами крайне важно, чтобы фаззер учитывал их, заново вычислял и перезаписывал соответствующие значения, тем самым позволяя целевому приложению правильно обработать файл.

Итоги

Хотя файлы и сетевой трафик можно подвергать фаззингу методом полного перебора, гораздо эффективнее выбирать только те части данных, которые с наибольшей вероятностью приводят к потенциально уязвимым ситуациям. Такие знания требуют некоторых предварительных усилий, но обычно эти усилия оправданны, особенно с учётом обилия документации как для открытых, так и для проприетарных протоколов. Опыт поможет определять лучшие целевые места внутри протокола, однако мы надеемся, что подробно рассмотренные в этой главе области помогут выделить некоторые наиболее распространённые слабые места, приводившие к уязвимостям в прошлом. В главе 22 «Автоматизированный разбор протоколов» мы рассмотрим некоторые автоматизированные методы исследования структуры протокола.

Сноски

[1] [назад] http://en.wikipedia.org/wiki/Protocol_%28computing%29
[2] [назад] http://download.intel.com/design/intarch/manuals/24281601.pdf
[3] [назад] http://www.vulnwatch.org/netcat/
[4] [назад] http://en.wikipedia.org/wiki/AOL_Instant_Messenger

[5] [назад] http://gaim.sourceforge.net/
[6] [назад] http://www.ceruleanstudios.com/
[7] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[8] [назад] http://www.ietf.org/
[9] [назад] http://en.wikipedia.org/wiki/OpenDocument
[10] [назад] http://www.oasis-open.org
[11] [назад] http://www.microsoft.com/office/preview/itpro/fileoverview.mspx
[12] [назад] http://sev.prnewswire.com/computer-electronics/20060705/SFTH05506072006-1.html

Глава 5. Требования к эффективному фаззингу

> «Вы учите ребёнка читать, и он или её сможет пройти проверку грамотности».
>
> - Джордж У. Буш, Таунсенд, штат Теннесси, 21 февраля 2001 года

В предыдущих главах мы представили различные классы фаззеров и разные подходы к фаззингу. В этой главе мы обсудим ряд советов и приёмов, способных сделать фаззинг более эффективным и производительным. До начала разработки фаззера следует учесть такие очевидные факторы, как воспроизводимость тестов и возможность повторного использования фаззера. Это поможет гарантировать, что будущая работа будет строиться поверх нынешних результатов, а не вместо них. В данной главе также рассматриваются дополнительные характеристики, повышающие сложность фаззера: состояние и глубина процесса, отслеживание и метрики, обнаружение ошибок и ограничения. Позднее, в части II книги, мы обсудим ряд целей фаззеров, а также создание автоматизированных средств, которые можно применять против каждой цели. По мере чтения этих глав помните об идеях, рассмотренных здесь: они будут играть роль всякий раз, когда мы проектируем новый фаззер, и особенно при создании каркаса фаззинга, обсуждаемого в главе 12 «Фаззинг форматов файлов: автоматизация в UNIX». Даже среди различных коммерческих фаззеров, продающихся сегодня, ни один не удовлетворяет всем требованиям, которые мы собираемся рассмотреть.

Воспроизводимость и документация

Очевидное требование к средству фаззинга - способность воспроизводить результаты как отдельных тестов, так и их последовательностей. Это крайне важно для передачи результатов тестирования другим людям или группам. Как специалист по фазз-тестированию, вы должны иметь возможность предоставить своему средству фаззинга список номеров вредоносных тестовых случаев, зная, что наблюдаемое поведение цели будет совершенно одинаковым при разных запусках. Рассмотрим следующую вымышленную ситуацию. Вы подвергаете фаззингу способность веб-сервера обрабатывать некорректные данные `POST` и обнаруживаете потенциально пригодное для эксплуатации повреждение памяти, когда отправленный пятидесятый тестовый случай вызывает аварийное завершение службы. Вы перезапускаете веб-демон и повторно передаёте последнюю вредоносную полезную нагрузку, но ничего не происходит. Была ли проблема случайностью? Разумеется, нет: компьютеры детерминированы и не имеют понятия о случайности. Проблема должна зависеть от некоторого сочетания входных данных. Возможно, более ранний пакет перевёл веб-сервер в состояние, которое позднее позволило пятидесятому тесту вызвать повреждение памяти. Без дальнейшего анализа мы этого не узнаем, а без возможности методично воспроизвести весь набор тестов не сможем сузить круг вариантов.

Документирование различных результатов тестирования также является полезным, если не обязательным, требованием на этапе обмена информацией. С учётом растущей тенденции передавать разработку за рубеж на аутсорсинг[1] специалист по тестированию безопасности часто

не может просто пройти по коридору и сесть рядом с разработчиком затронутого продукта. Аутсорсинг стал настолько популярен, что, как известно, им пользуются даже студенты, изучающие информатику.[2] Различные коммуникационные барьеры, включая часовой пояс, язык и средство связи, делают ещё более важным объединение как можно большего объёма сведений в ясной и краткой форме. Бремя ведения упорядоченной документации не должно полностью ложиться на ручной труд. Хорошее средство фаззинга создаёт и сохраняет журнальные сведения, которые легко разбирать и на которые легко ссылаться.

Подумайте, как отдельные обсуждаемые нами фаззеры обеспечивают воспроизводимость, журналирование и автоматизированное документирование. Подумайте, как можно улучшить их реализацию.

Повторное использование

В крупном масштабе, создавая средство фаззинга форматов файлов, мы не хотим переписывать его целиком всякий раз, когда требуется проверить новый формат. Мы можем создать некоторые повторно используемые функции, которые сэкономят нам время в будущем, если мы решим проверить другой формат. Продолжая наш пример, предположим, что мы решили создать средство фаззинга формата JPEG для поиска ошибок в Microsoft Paint. Заранее думая о том, что мы захотим повторно использовать часть своего труда, можно разделить набор средств на три компонента, показанных на рисунке 5.1.

Генератор файлов JPEG отвечает за создание бесконечной последовательности изменённых файлов JPEG. Интерфейс запуска отвечает за перебор созданных изображений и каждый раз запускает Microsoft Paint с соответствующими аргументами для загрузки следующего изображения. Наконец, механизм обнаружения ошибок отвечает за наблюдение за каждым экземпляром Microsoft Paint с целью выявления исключительных условий. Разделение на три компонента позволяет приспосабливать набор тестов к другим форматам файлов, изменяя только генератор.

В меньшем масштабе множество строительных блоков должно переноситься между нашими проектами фазз-тестирования. Рассмотрим, например, адрес электронной почты. Этот базовый формат строки встречается повсюду, в том числе в транзакциях простого протокола передачи почты (SMTP), на экранах входа и в протоколе инициирования сеансов (SIP) для передачи голоса по IP (VoIP):

Фрагмент транзакции SIP INVITE

```
49 4e 56 49 54 45 20 73 69 70 3a 72 6f 6f 74 40  INVITE sip:root@
6f 70 65 6e 72 63 65 2e 6f 72 67 20 53 49 50 2f  openrce.org SIP/
32 2e 30 0d 0a 56 69 61 3a 20 53 49 50 2f 32 2e  2.0..Via: SIP/2.
30 2f 55 44 50 20 70 61 6d 69 6e 69 4c 2e 75 6e  0/UDP voip.openr
```

В каждом случае это интересное поле для фаззинга, поскольку мы уверены, что оно будет разобрано и, возможно, разделено на различные компоненты, например пользователя и домен. Если мы собираемся потратить время на перечисление возможных вредоносных представлений адреса электронной почты, разве не было бы удобно повторно использовать их во всех фаззерах?

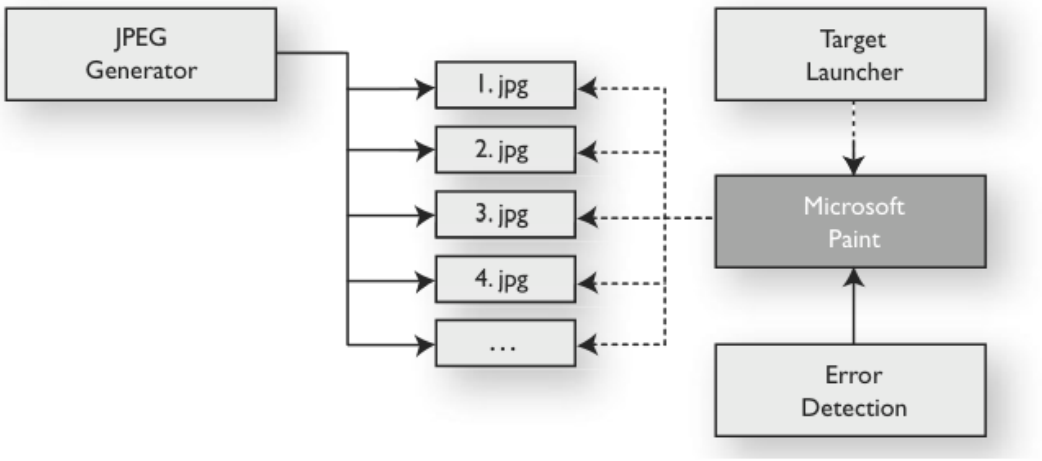


Рисунок 5.1. Разбиение и общая схема вымышленного фаззера форматов файлов

Подумайте, как можно абстрагировать отдельные обсуждаемые нами фаззеры или разделить их на модули, чтобы расширить возможности повторного использования.

Состояние и глубина процесса

Чтобы хорошо усвоить понятия состояния и глубины процесса, выберем пример, слишком хорошо знакомый большинству людей: банковские операции через банкомат. Рассмотрим простую диаграмму состояний на рисунке 5.2.

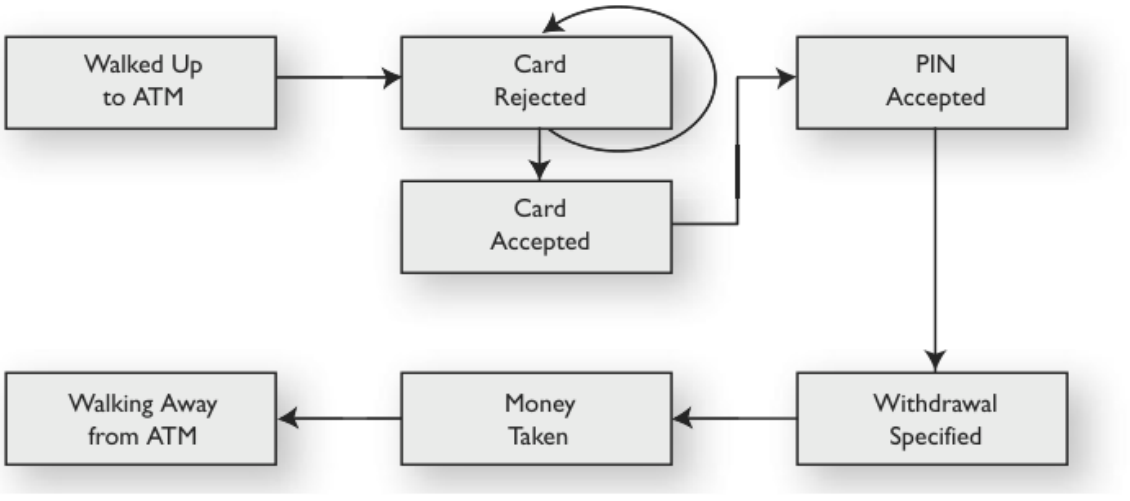


Рисунок 5.2. Искусственный пример диаграммы состояний банкомата

При обычной операции через банкомат вы подходите к машине, предельно скрытно убедившись, что за вами никто не следовал, вставляете карту, вводите PIN-код, проходите последовательность экранных меню, чтобы выбрать сумму для снятия, забираете деньги и завершаете операцию. Та же идея состояния и переходов между состояниями применима к программному обеспечению; через минуту мы приведём конкретный пример. Каждый шаг операции с банкоматом можно назвать состоянием. Мы определяем состояние процесса как конкретное состояние, в котором целевой процесс находится в данный момент. Действия, например вставка карты или выбор суммы снятия, переводят вас из одного состояния в другое. То, насколько далеко вы продвинулись в процессе,

называется глубиной процесса. Например, указание суммы снятия происходит на большей глубине, чем ввод PIN-кода.

В качестве примера, более тесно связанного с безопасностью, рассмотрим сервер безопасной оболочки (SSH). До подключения сервер находится в начальном состоянии. Во время аутентификации он находится в состоянии аутентификации. После успешной аутентификации пользователя сервер переходит в аутентифицированное состояние.

Глубина процесса - конкретная мера числа шагов «вперёд», необходимых для достижения определённого состояния. Продолжая пример SSH-сервера, рассмотрим диаграмму состояний на рисунке 5.3.

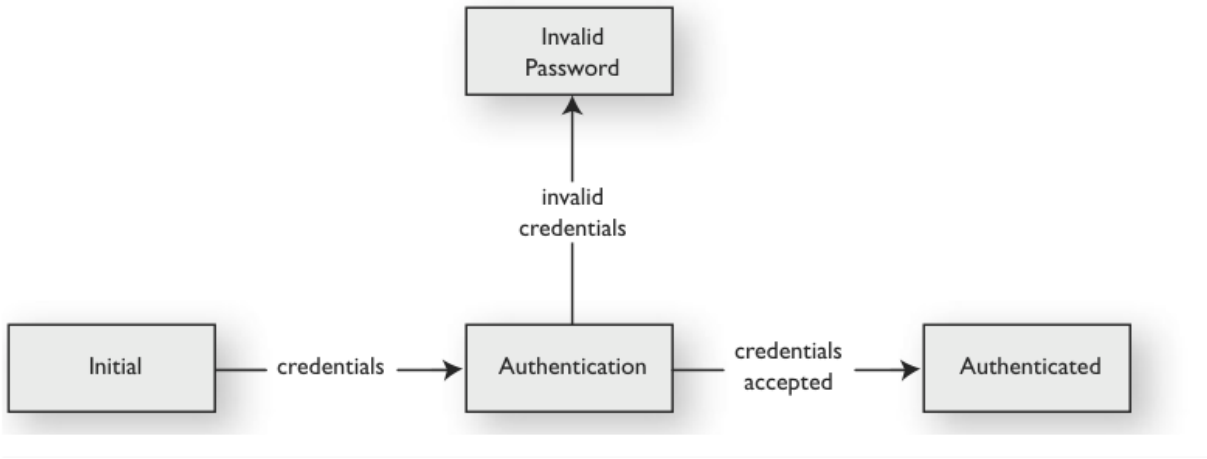


Рисунок 5.3. Диаграмма состояний SSH-сервера

Аутентифицированное состояние находится «глубже» в процессе, чем состояние аутентификации, поскольку аутентификация является обязательным промежуточным шагом для достижения аутентифицированного состояния. Понятия состояния и глубины процесса важны и способны существенно усложнить проектирование фаззера. Следующий пример демонстрирует такое усложнение. Чтобы подвергнуть фаззингу аргумент адреса электронной почты команды `MAIL FROM` на SMTP-сервере, необходимо подключиться к серверу и выдать команду `HELO` или `EHLO`. Как показано на рисунке 5.4, внутренняя реализация SMTP может обрабатывать команду `MAIL FROM` одной и той же функцией независимо от использованной команды инициализации.

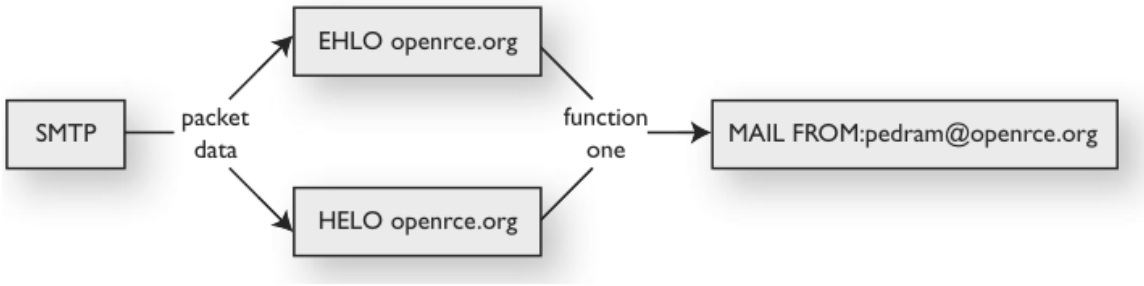


Рисунок 5.4. Диаграмма состояний примера SMTP 1

На рисунке 5.4 первая функция - единственная определённая функция обработки данных `MAIL FROM`. В другом варианте, показанном на рисунке 5.5, реализация SMTP может иметь две отдельные процедуры обработки данных `MAIL FROM` в зависимости от выбранной команды инициализации.

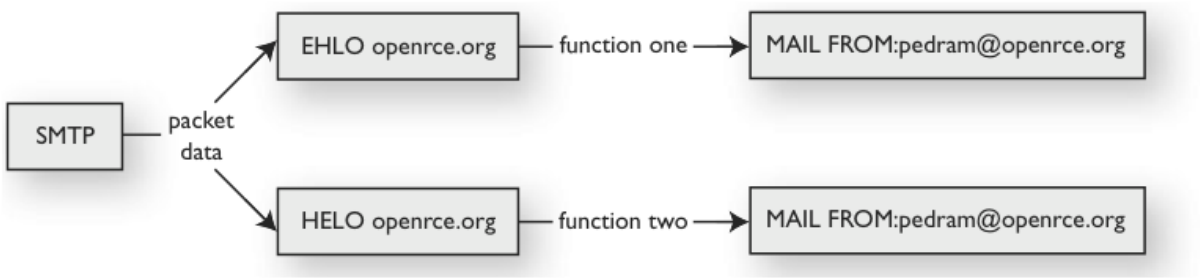


Рисунок 5.5. Диаграмма состояний примера SMTP 2

Это фактически пример из реального мира. 7 сентября 2006 года было опубликовано уведомление о безопасности,[3] подробно описывавшее удалённо эксплуатируемое переполнение стека в SMTP-сервере, входившем в Ipswitch Collaboration Suite. Переполнение происходит, когда при разборе адресов электронной почты между символами `@` и `:` передаются длинные строки. Однако уязвимая процедура разбора достижима только тогда, когда подключающийся клиент начинает обмен командой `EHLO`. При создании фаззеров помните о возможных подобных разветвлениях логики. Для полного охвата фаззеру придётся дважды перебрать все мутации адреса электронной почты: один раз через `EHLO` и один раз через `HELO`. Что произойдёт, если дальше по пути глубины процесса имеется ещё одно логическое разветвление? Число итераций, необходимых для полного охвата, начинает расти экспоненциально.

По мере обсуждения различных фаззеров в следующих главах подумайте, как можно обрабатывать различия в глубине процесса и где могут находиться разветвления логики.

Отслеживание, охват кода и метрики

Охват кода - термин, обозначающий объём состояний процесса, которых фаззер заставляет целевой процесс достичь и выполнить. На момент написания книги нам не было известно ни одной общедоступной или коммерческой технологии фаззинга, способной отслеживать и журналировать охват кода. Эта идея важна для всех аналитиков. Группы обеспечения качества (QA) могут использовать охват кода как метрику, определяющую степень уверенности в проведённом уровне тестирования. Например, если вы руководите QA веб-серверного продукта, то, вероятно, гораздо увереннее выпустите продукт с нулём отказов при 90-процентном охвате кода, чем с нулём отказов лишь при 25-процентном охвате. Исследователи уязвимостей могут извлечь пользу из анализа охвата кода, определяя изменения, необходимые для расширения охвата на более малоизвестные состояния цели, куда прежде могли не заглядывать другие. Эта важная идея подробно рассматривается в главе 23 «Отслеживание работы фаззера».

По мере обсуждения различных фаззеров в следующих главах подумайте о творческих подходах к определению охвата кода и о пользе, которую может принести такой анализ. При фаззинге люди всегда спрашивают: «Как начать?» Помните, что не менее важно спросить: «Когда остановиться?»

Обнаружение ошибок

Создание и передача потенциально вредоносного трафика составляет лишь половину битвы фаззинга. Вторая половина - точное определение момента возникновения ошибки. На момент написания большинство доступных фаззеров были «слепыми», то есть не имели представления о том, как цель реагирует на переданные тесты. Некоторые коммерческие решения чередуют вредоносные попытки с проверками `ping` или поддержания активности, используя их как контроль для определения работоспособности цели. Термин `ping` здесь употребляется в широком смысле и обозначает любую форму транзакции, которая должна вызвать заведомо правильный ответ. Существуют и другие решения, основанные на анализе журнального вывода. Это может заключаться в наблюдении за текстовыми журналами ASCII, которые ведут отдельные приложения, или в запросе записей системных журналов, таких как Windows Event Viewer, показанный на рисунке 5.6.

Преимущество этих подходов к обнаружению ошибок состоит в том, что они по большей части легко переносятся между платформами и архитектурами. Однако они сильно ограничены в отношении видов ошибок, которые способны обнаружить. Например, ни один из этих подходов не может выявить случай, когда в приложении Microsoft Windows возникает критическая ошибка, но её корректно обрабатывает процедура структурированной обработки исключений[4] (SEH).

Следующее поколение средств обнаружения ошибок использует облегчённые клиенты отладки, выявляющие возникновение исключительного условия в цели. Например, средство FileFuzz, которое мы обсуждаем в части II, включает специальный клиент отладки Microsoft Windows с открытым исходным кодом. Единственный отрицательный аспект использования средств этого типа состоит в необходимости разработать отдельное средство для каждой целевой платформы тестирования. Например, если вы хотите проверить три SMTP-сервера на Mac OS X, Microsoft Windows и Gentoo Linux, вам, вероятно, придётся разработать два или, возможно, три разных клиента наблюдения. Более того, в зависимости от цели создание клиента отладки может оказаться невозможным или несвоевременным. Например, при тестировании аппаратного телефона VoIP вам, возможно, придётся вернуться к контрольным тестам или наблюдению за журналом, поскольку аппаратные решения хуже поддаются отладке и могут требовать специальных средств.

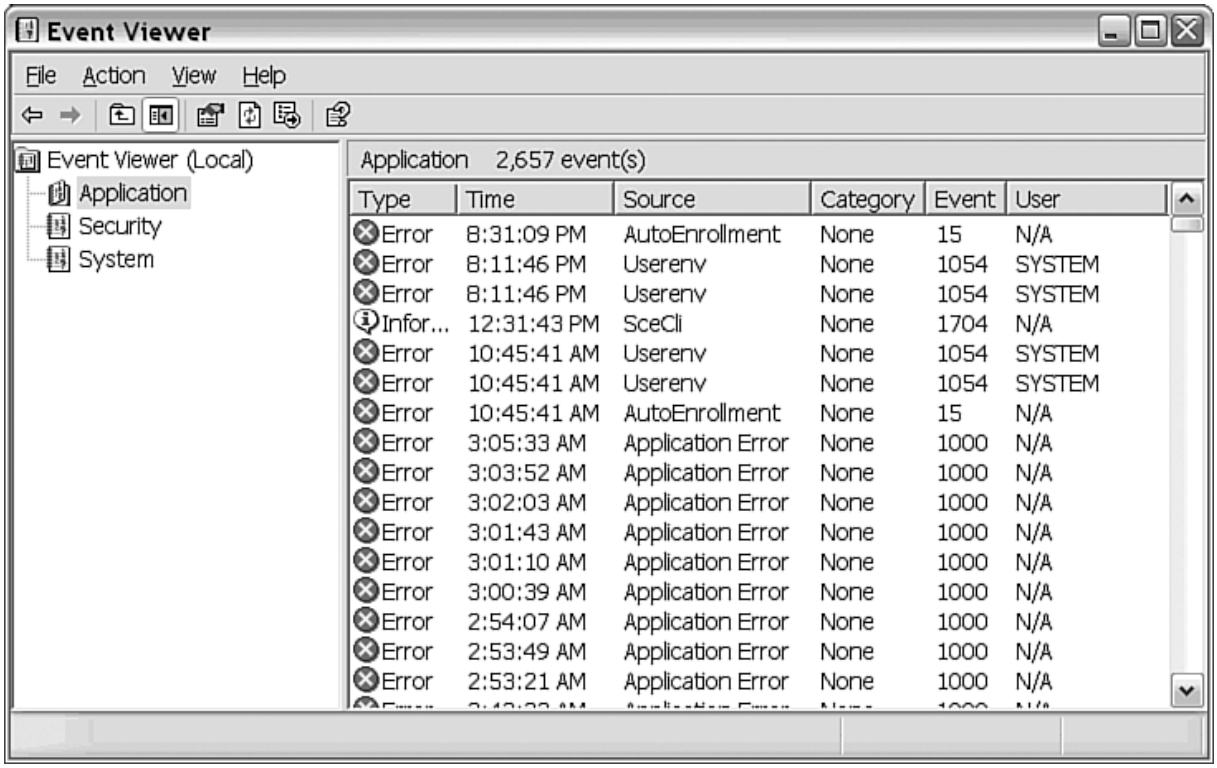


Рисунок 5.6. Пример журнала ошибок Microsoft Windows Event Viewer

Если заглянуть ещё дальше вперёд, панацея обнаружения ошибок находится в платформах динамического инструментирования и трансляции двоичного кода[5] (DBI), таких как Valgrind[6] и Dynamo Rio.[7] На подобных платформах становится возможным обнаруживать ошибки по мере их развития, а не после срабатывания. Если смотреть с высоты 50 000 футов, механизмы отладки на основе DBI способны очень точно анализировать и инструментировать целевое программное приложение на низком уровне. Такой контроль позволяет создавать проверки утечек памяти, переполнения и выхода за нижнюю границу буфера и так далее. Возвращаясь к примеру повреждения памяти, использованному при обсуждении воспроизводимости, облегчённый клиент отладки способен сообщить нам о срабатывании повреждения памяти. Как вы помните, в нашем примере мы описали сценарий, в котором службе был отправлен ряд пакетов, а при получении пятидесятого тестового случая она аварийно завершилась. На такой платформе, как Valgrind, мы могли бы обнаружить первоначальное повреждение памяти, возникшее во время какого-либо более раннего теста до срабатывания исключения. Этот подход способен сэкономить часы, а возможно и дни, настройки фаззинга и отслеживания ошибки.

По мере обсуждения различных фаззеров в следующих главах подумайте о разных подходах к наблюдению за целями и о том, какой из них может быть наиболее применим.

Ограничения ресурсов

Различные нетехнические факторы, такие как бюджет и сроки, могут накладывать ограничения на фазз-тестирование. Эти факторы необходимо учитывать на этапе проектирования и планирования. Например, вы можете оказаться в панике в последнюю минуту перед выпуском, потому что никто даже бегло не проверил безопасность продукта, в который вложено 50 миллионов долларов. Безопасность слишком часто становится запоздалой мыслью в жизненном цикле разработки программного обеспечения (SDLC), уступая добавлению новых функций и соблюдению производственных сроков. Если мы когда-нибудь надеемся создавать безопасное программное обеспечение, безопасность должна «запекаться внутрь», а не «наноситься кистью». Для этого требуются фундаментальные изменения SDLC, гарантирующие, что безопасность учитывается на каждом этапе разработки. При этом мы признаём, что программное обеспечение разрабатывается в реальном мире, а не в утопии, где ресурсы изобильны, а дефекты редки. Поэтому по мере чтения книги не менее важно мысленно классифицировать различные методы, применимые при ограниченных времени и финансах, наряду с мечтами об «идеальном» комплексе фаззинга. Подумайте также, где вы внедрили бы такие средства в SDLC и кто отвечал бы за эти процессы.

Итоги

Основная цель данной главы заключалась в представлении идей, побуждающих к творческим размышлениям о разработке полнофункционального фаззера. При разработке, сравнении и использовании технологий фаззинга благоразумно возвращаться к изложенным здесь идеям. Когда в последующих главах будут представлены и разобраны разные решения для фаззинга, подумайте об улучшениях, которые можно получить благодаря применению полезных требований, таких как метрики и усовершенствованное обнаружение ошибок.

Сноски

[1] [назад] http://www.outsourceworld.org/, http://money.cnn.com/2003/09/17/news/economy/outsourceworld/
[2] [назад] «Computer Science Students Outsource Homework», http://developers.slashdot.org/developers/06/01/19/0026203.shtml
[3] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-028.html
[4] [назад] http://msdn2.microsoft.com/en-us/library/ms680657.aspx
[5] [назад] http://en.wikipedia.org/wiki/Binary_translation
[6] [назад] Valgrind: http://valgrind.org/
[7] [назад] Dynamo RIO: http://www.cag.lcs.mit.edu/dynamorio

Часть II. Цели и автоматизация

Глава 6. Автоматизация и генерация данных

> «Наши враги изобретательны и находчивы, и мы тоже. Они никогда не прекращают думать о новых способах причинить вред нашей стране и нашему народу, и мы тоже».
>
> - Джордж У. Буш, Вашингтон, округ Колумбия, 5 августа 2004 года

Фаззинг целиком основан на автоматизации. Главное преимущество фаззинга перед альтернативными методологиями тестирования программного обеспечения - высокая доля автоматизации по сравнению с ручным трудом. Создание отдельных тестовых случаев - трудоёмкая и утомительная работа, то есть категория задач, идеально подходящая компьютеру. Основная компетенция фаззера заключается в способности создавать полезные данные, желательно при минимальном участии человека. Эта глава посвящена различным аспектам автоматизации, включая выбор языка, полезные строительные блоки и неизменно важное искусство выбора подходящих значений фаззинга при генерации данных для тщательного тестирования целевого программного обеспечения.

Ценность автоматизации

Хотя преимущества автоматизации могут показаться очевидными, для ясности мы рассмотрим их с двух точек зрения. Сначала подчеркнём ценность с точки зрения вычислительных возможностей человека, а затем обратимся к воспроизводимости. Во времена первых компьютеров вычислительное время было дорогим - фактически дороже времени человеческих вычислений, - поэтому программирование таких систем представляло собой очень утомительный ручной процесс. Программисты подавали ленту, переключали тумблеры и вручную вводили десятичные или двоичные машинные коды операций.

Со временем талант программистов становился всё ценнее, вычислительного времени становилось всё больше, и соотношение расходов изменилось. Сегодня эта тенденция продолжается, о чём свидетельствует растущая популярность языков программирования более высокого уровня, таких как Java, .NET и Python. Каждый из них жертвует вычислительным временем, чтобы предоставить программистам более удобную среду разработки и сократить время получения результата.

В связи с этим, хотя человек-аналитик вполне способен интерактивно подключиться к демону на основе сокетов и вручную вводить данные в надежде обнаружить дефекты программы, его время лучше потратить на другие задачи. Тот же довод убедителен при сравнении фаззинга с такими задачами ручного аудита, как анализ исходного или двоичного кода. Последние методы требуют времени высококвалифицированных аналитиков, тогда как первый, фаззинг, в большей или меньшей степени может проводить кто угодно. По меньшей мере автоматизацию следует использовать как первый этап, чтобы уменьшить объём работы, остающейся высококвалифицированным аналитикам, способным точно исследовать обнаруженные сбои и сосредоточиться на альтернативных методологиях.

Далее подчеркнём необходимость воспроизводимости. Её важность определяется двумя ключевыми факторами:

• Если мы способны создать воспроизводимый процесс тестирования одного FTP-сервера, тот же процесс легко применить к другим версиям, а также к совершенно иным FTP-серверам. В противном случае время напрасно тратилось бы на повторное проектирование и реализацию нового фаззера для каждого FTP-сервера.
• Если сбой цели был вызван нетривиальной последовательностью событий, мы должны иметь возможность воспроизвести всю последовательность, чтобы сузить её до конкретной подпоследовательности, ответственной за аномалию. Человек-аналитик, запускающий разные тестовые случаи по мере того, как вспоминает их, весьма далёк от научной воспроизводимости.

Короче говоря, сизифов труд генерации и повторной генерации данных и наблюдения за сбоями лучше поручить автоматизированной системе. Как и в большинстве вычислительных задач, применительно к фаззингу нам повезло: уже существует ряд средств и библиотек, которыми можно воспользоваться.

Полезные средства и библиотеки

Большинству разработчиков фаззеров приходится создавать средства с нуля, что подтверждается изобилием сценариев фаззеров, уже доступных широкой публике.[1] К счастью, на этапе проектирования и реализации фаззера вам могут помочь многие средства и библиотеки. В этом разделе перечислены некоторые из них в алфавитном порядке.

Ethereal[2] / Wireshark[3]

Wireshark, ответвление проекта Ethereal,[4] - популярный анализатор сетевого трафика и средство разбора протоколов с открытым исходным кодом. Хотя это не обязательно библиотека, поверх которой можно что-либо строить, средство, несомненно, пригодится и на этапе исследования, и при отладке создаваемого фаззера. Wireshark предоставляет множество средств разбора с открытым исходным кодом, которые также часто полезны как справочный материал. Захваченный трафик, для которого имеется средство разбора, отображается как последовательность пар «поле - значение», а не как блок необработанных байтов. Прежде чем погружаться в ручной анализ протокола, всегда благоразумно сначала выяснить, что о нём скажет Wireshark. Краткий список доступных средств разбора можно найти в репозитории Subversion Wireshark, а именно в каталоге `epan\dissectors`.[5]

libdasm[6] и libdisasm[7]

libdasm и libdisasm - свободно доступные библиотеки дизассемблера с открытым исходным кодом, которые можно встраивать в собственные средства для получения из двоичных потоков дизассемблированного кода в синтаксисе AT&T и Intel. libdasm написана на C, а libdisasm - на Perl. В libdasm также включён интерфейс Python. Хотя дизассемблер не нужен для генерации сетевого трафика, он важен при автоматизации обнаружения сбоев. Обе библиотеки используются на протяжении книги, в частности в главе 12 «Фаззинг форматов файлов: автоматизация в UNIX», главе 19 «Фаззинг в памяти», главе 20 «Фаззинг в памяти: автоматизация», главе 23 «Отслеживание работы фаззера» и главе 24 «Интеллектуальное обнаружение сбоев».

Libnet[8] / LibnetNT[9]

Libnet - свободно доступный высокоуровневый API с открытым исходным кодом для создания и внедрения низкоуровневых данных сетевых пакетов. Библиотека скрывает значительную часть сложности, обычно связанной с генерацией трафика IP и канального уровня, и одновременно обеспечивает переносимость между различными платформами. Если вы пишете фаззер сетевого стека, эта библиотека может вас заинтересовать.

LibPCAP[10]

LibPCAP и совместимая с Microsoft Windows библиотека WinPCAP[11] - свободно доступные высокоуровневые библиотеки с открытым исходным кодом, позволяющие легко создавать средства захвата и анализа сети как для UNIX, так и для Microsoft Windows. На этой библиотеке построено множество средств анализа сетевых протоколов, включая упомянутый выше Wireshark.

Metro Packet Library[12]

Metro Packet Library - библиотека C#, предоставляющая абстрагированный интерфейс для работы с IPv4, TCP, UDP и протоколом управляющих сообщений Интернета (ICMP). Она полезна при создании анализаторов пакетов и средств анализа сети. Библиотека подробнее обсуждается и используется в главе 16 «Фаззинг сетевых протоколов: автоматизация в Windows».

ptrace

Отладка на платформах UNIX по большей части выполняется с помощью системного вызова `ptrace()` (трассировка процесса). Процесс может использовать `ptrace()` для управления состоянием регистров, памятью и исполнением другого процесса, а также для перехвата создаваемых им сигналов. Этот механизм применяется при реализации средств, обсуждаемых в главе 8 «Фаззинг переменных среды и аргументов: автоматизация» и главе 12 «Фаззинг форматов файлов: автоматизация в UNIX».

Расширения Python

При создании фаззеров могут пригодиться различные расширения Python. Среди них Pcapy, Scapy и PyDbg. Pcapy[13] - расширение Python, взаимодействующее с LibPCAP/WinPCAP и позволяющее сценариям Python захватывать сетевой трафик. Scapy[14] - мощное и простое расширение для манипулирования пакетами, которое можно использовать интерактивно или как библиотеку. Scapy способно как создавать, так и декодировать широкий спектр протоколов. PyDbg[15] - полноценный 32-разрядный отладчик Microsoft Windows на чистом Python, позволяющий легко и изящно инструментировать процессы. Библиотека PyDbg является подмножеством более крупного каркаса обратной разработки PaiMei,[16] который используется далее в книге, в том числе в главах 19, 20, 23 и 24.

Некоторые из перечисленных здесь библиотек доступны для использования из нескольких языков. Другие имеют более узкие возможности. Ваши конкретные потребности и наличие библиотек, способных помочь в их удовлетворении, входят в число факторов, учитываемых при выборе языка разработки фаззера.

Выбор языка программирования

Многие считают выбор языка программирования фаззера предметом религиозного спора и яростно отстаивают свой вариант независимо от задачи, тогда как другие придерживаются философии «подходящий инструмент для подходящей работы». Мы склонны проповедовать вторую. На протяжении книги вы встретите исходный код на разных языках. При подготовке книги мы сознательно старались включить как можно больше пригодных для повторного использования примеров кода из реального мира. Независимо от личных предпочтений мы рекомендуем рассмотреть преимущества и недостатки каждого языка, прежде чем выбирать его для конкретной задачи.

На самом высоком уровне в языках программирования есть одно фундаментальное различие, которое приходится учитывать в самом начале разработки: интерпретируемые языки в противоположность компилируемым. Низкоуровневые компилируемые языки, такие как C и C++, предоставляют точный непосредственный доступ к нижележащим компонентам. Например, упомянутая выше библиотека Libnet имеет интерфейс на этом нижнем уровне. Интерпретируемые языки более высокого уровня, такие как Python и Ruby, обеспечивают более быструю разработку и возможность вносить изменения без повторной компиляции. Обычно существуют библиотеки, предоставляющие низкоуровневые функции этим языкам более высокого уровня. Поскольку природа фаззера требует гибкости, а не качества промышленного кода, фаззеры пишут на любых языках. От сценариев оболочки до Java, от PHP до C# - в зависимости от конкретной задачи и ваших знаний один язык может подходить лучше другого.

Генерация данных и эвристики фаззинга

Реализация способа генерации данных - лишь часть решения. Не менее важно определить, какие данные генерировать. Например, представим, что мы создаём фаззер для анализа устойчивости IMAP-сервера. Среди множества команд и конструкций IMAP, которые необходимо изучить, находится запрос продолжения команды (CCR), описанный в следующем фрагменте RFC 3501:[17]

> **7.5. Ответы сервера - запрос продолжения команды**
>
> Ответ с запросом продолжения команды обозначается маркером + вместо тега. Такая форма ответа указывает, что сервер готов принять продолжение команды от клиента. Остаток этого ответа представляет собой строку текста.
>
> Этот ответ используется в команде `AUTHENTICATE` для передачи данных сервера клиенту и запроса дополнительных данных клиента. Он также используется, если аргумент любой команды является литералом.
>
> Клиенту не разрешается отправлять октеты литерала, пока сервер не укажет, что они ожидаются. Это позволяет серверу обрабатывать команды и отклонять ошибки построчно. Остаток команды, включая `CRLF`, завершающий команду, следует за октетами литерала. Если имеются дополнительные аргументы команды, за октетами литерала следуют пробел и эти аргументы.

```
Example:    C: A001 LOGIN {11}
S: + Ready for additional command text
C: FRED FOOBAR {7}
S: + Ready for additional command text
C: fat man
S: A001 OK LOGIN completed
C: A044 BLURDYBLOOP {102856}
```

```
S: A044 BAD No such command as "BLURDYBLOOP"
...
```

Согласно RFC, любая команда, оканчивающаяся конструкцией {число}, выделенной в оригинале полужирным, указывает, что оставшиеся число байтов команды продолжаются на следующей строке. Это превосходная цель фаззинга, но какими значениями следует проверять поле? Можно ли проверить все возможные числовые значения? Целевой демон, вероятно, принимает значения вплоть до максимального 32-разрядного целого 0xFFFFFFFF (4 294 967 295). Если фаззер выполняет один тестовый случай в секунду, завершение проверки займёт более 136 лет! Даже если встряхнуть фаззер здоровой дозой кофеина и повысить пропускную способность до 100 тестов в секунду, для завершения потребуется почти 500 дней. При такой скорости протокол IMAP может уйти в отставку раньше, чем закончится тестирование. Ясно, что проверить весь диапазон невозможно и необходимо выбрать интеллектуальное подмножество, или выборку, возможных целых значений.

Намеренное включение потенциально опасных значений в список фазз-строк, или данных фаззинга, также называют эвристиками фаззинга. Рассмотрим несколько категорий типов данных, которые стоит включить в интеллектуальную библиотеку.

Целые значения

Два крайних граничных тестовых случая, 0 и 0xFFFFFFFF, очевидно подходят для нового улучшенного списка целочисленных тестов. Что ещё можно добавить? Возможно, переданное число используется как параметр размера процедуры выделения памяти. Нередко к указанному размеру добавляется место, например для заголовка, нижнего колонтитула или завершающего байта NULL:

```
int size = read_ccr_size(packet);
// резервируем место для завершителя NULL.
buffer = (char *) malloc(size + 1);
```

С другой стороны, перед выделением памяти указанное значение размера могут уменьшать. Это может происходить, если целевая программа знает, что не будет копировать все указанные данные в новый буфер. Помня, что целочисленные переполнения, то есть сложение с переходом за верхнюю границу диапазона 32-разрядного целого, и целочисленные выходы за нижнюю границу, то есть вычитание с переходом ниже 0, могут впоследствии привести к проблемам безопасности, благоразумно включить приграничные тестовые случаи: 0xFFFFFFFF-1, 0xFFFFFFFF-2, 0xFFFFFFFF-3, …, а также 1, 2, 3, 4 и так далее.

Аналогично к указанному размеру может применяться множитель. Рассмотрим, например, преобразование переданных данных в Unicode. Для этого указанный размер потребуется умножить на 2. Кроме того, можно добавить два байта, чтобы обеспечить завершение NULL. Например:

```
int size = read_ccr_size(packet);
// создаём место для преобразованного буфера Unicode
// и завершающего значения Unicode NULL (2 байта).
buffer = (char *) malloc((size * 2) + 2);
```

Чтобы вызвать целочисленное переполнение в этой и подобных ситуациях, в список также пришлось бы включить следующие приграничные тестовые случаи: 0xFFFFFFFF/2, 0xFFFFFFFF/2-1, 0xFFFFFFFF/2-2 и так далее. А как насчёт граничных случаев около результатов деления на 3? Или на 4? Раз уж мы этим занимаемся, почему бы не включить перечисленные выше приграничные случаи и для 16-разрядных целых (0xFFFF)? А для 8-разрядных (0xFF)? Добавим их в список вместе с некоторыми другими разумными вариантами. Теперь список включает:

• MAX32 - 16 <= MAX32 <= MAX32 + 16

- MAX32 / 2 - 16 <= MAX32 / 2 <= MAX32 / 2 + 16
- MAX32 / 3 - 16 <= MAX32 / 3 <= MAX32 / 3 + 16
- MAX32 / 4 - 16 <= MAX32 / 4 <= MAX32 / 4 + 16
- MAX16 - 16 <= MAX16 <= MAX16 + 16
- MAX16 / 2 - 16 <= MAX16 / 2 <= MAX16 / 2 + 16
- MAX16 / 3 - 16 <= MAX16 / 3 <= MAX16 / 3 + 16
- MAX16 / 4 - 16 <= MAX16 / 4 <= MAX16 / 4 + 16
- MAX8 - 16 <= MAX8 <= MAX8 + 16
- MAX8 / 2 - 16 <= MAX8 / 2 <= MAX8 / 2 + 16
- MAX8 / 3 - 16 <= MAX8 / 3 <= MAX8 / 3 + 16
- MAX8 / 4 - 16 <= MAX8 / 4 <= MAX8 / 4 + 16

Здесь MAX32 обозначает максимальное 32-разрядное целое (0xFFFFFFFF), MAX16 - максимальное 16-разрядное целое (0xFFFF), MAX8 - максимальное 8-разрядное целое (0xFF), а диапазон 16 был произвольно выбран как разумная величина. В зависимости от имеющегося времени и результатов вы можете увеличить этот диапазон. Если целевой протокол содержит сотни целочисленных полей, каждая дополнительная целочисленная эвристика увеличивает общее число тестовых случаев в сто раз.

Относитесь к выбранным целочисленным «эвристикам» с долей скептицизма. Эвристика - всего лишь изысканный способ сказать «обоснованная догадка». Более опытные пользователи могут исследовать целевой двоичный файл в дизассемблере и извлечь потенциально интересные целые значения, изучая перекрёстные ссылки на процедуры выделения памяти и копирования данных. Этот процесс тоже можно автоматизировать; данная идея кратко затрагивается в главе 22 «Автоматизированный разбор протоколов».

Необходимость интеллектуального набора данных

1 сентября 2005 года совместно с предоставленным соответствующим поставщиком исправлением было публично выпущено уведомление о безопасности «Переполнение кучи в запросе продолжения команды Novell NetMail IMAPD».[18] В уведомлении подробно описан дефект, позволяющий удалённым неаутентифицированным злоумышленникам выполнять произвольный код и тем самым полностью компрометировать нижележащую систему.

Описанный дефект находится в обработке CCR: заданное пользователем значение размера непосредственно передаётся как аргумент специальной оболочке выделения памяти MMalloc(), что показано в следующем фрагменте ассемблерного кода:

```
; ebx управляется злоумышленником
00402CA2 lea ecx, [ebx+1]
00402CA5 push ecx
00402CA6 call MMalloc
```

Перед выделением памяти процедура MMalloc() выполняет над полученным значением тривиальную математическую операцию. Злоумышленник может указать вредоносное число, которое приведёт к целочисленному переполнению и выделению небольшого блока памяти. Первоначальное, более крупное переданное значение позднее будет использовано во встроенной операции memcpy():

```
; память назначения выделена злоумышленником
00402D6E rep movsd
00402D70 mov ecx, edx
00402D72 and ecx, 3
00402D75 rep movsb
```

Эта последовательность инструкций скопирует предоставленные злоумышленником данные за границы выделенного блока кучи и произвольным образом перезапишет кучу, что в конечном итоге приведёт к полной компрометации.

Хотя исправление Novell успешно устранило этот конкретный вектор, оно не учло все возможные пути эксплуатации. К несчастью для Novell, демон IMAP преобразует читаемые человеком числовые формы в эквивалентные целые значения. Например, `-1` преобразуется в `0xFFFFFFFF`, `-2` - в `0xFFFFFFFE` и так далее. Поэтому следующий вредоносный запрос обрабатывается правильно:

```
x LOGIN {4294967295}
```

А этот - нет:

```
x LOGIN {-1}
```

Позднее этот дополнительный вектор эксплуатации обнаружил независимый исследователь, и 22 декабря 2006 года Novell выпустила исправление.[19]

Повторения строк

Мы рассмотрели диапазон разумно выбранных целых значений, которые стоит включить в данные фаззинга, но как насчёт строк? Начнём с древней классики[20] - «длинной строки»:

```
perl -e 'print "A"*5000'
```

При фаззинге строк следует включать и ряд дополнительных последовательностей символов. Сначала нужно добавить другие символы ASCII, например `B`. Это важно, в частности, при вызове переполнений кучи в Microsoft Windows из-за разного поведения, когда структуры кучи перезаписываются ASCII-значением `A` и когда значением `B`. Ещё одна причина важности этого приёма состоит в том, что, хотите верьте, хотите нет, авторы действительно видели промышленное программное обеспечение, специально ищущее и блокирующее длинные строки из символов `A`. Очевидно, некоторые поставщики осознали силу `AAAAAAAAAAAAAAAA`.

Разделители полей

Нужно также включить небуквенно-цифровые символы, в том числе пробелы и табуляции. Такие символы часто используются как разделители и ограничители полей. Случайное включение их в создаваемые фазз-строки повышает вероятность частично воспроизвести фаззируемый протокол и тем самым увеличить объём кода, подвергаемого фаззингу. Рассмотрим, например, сравнительно простой протокол HTTP и следующий список небуквенно-цифровых символов:

```
!@#$%^&*()-_=+{}|\;:'",<.>/?~`
```

Сколько из них вы узнаете как разделители полей HTTP? Для справки взгляните на следующий типичный код ответа HTTP-сервера:

```
HTTP/1.1 200 OK
Date: Sun, 01 Oct 2006 22:46:57 GMT
Server: Apache
X-Powered-By: PHP/5.1.4-pl0-gentoo
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, post-check=0, pre-check=0
Pragma: no-cache
```

```
Keep-Alive: timeout=15, max=93
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=ISO-8859-1
```

Первая закономерность, которую можно заметить, - множество строк, разделённых переводами строки, представленными на уровне байтов последовательностью `0x0d 0x0a`. В каждой строке дополнительно используются различные символы-разделители. Например, в первой строке видно, что для разделения кода ответа применяются пробел ( ), прямая косая черта (`/`) и точка (`.`). На последующих строках часто встречается двоеточие (`:`), отделяющее директивы, такие как `Content-Type`, `Server` и `Date`, от их значений. Дальнейшее изучение обнаруживает, что для разделения полей также используются запятая (`,`), знак равенства (`=`), точка с запятой (`;`) и дефис (`-`).

При генерации фазз-строк важно включать строки разной длины, разделённые различными разделителями полей, подобными показанным выше. Кроме того, важно увеличивать и длину разделителя. Рассмотрим, например, критическую уязвимость обработки заголовков Sendmail[21] 2003 года. В этом случае для возникновения пригодного к эксплуатации повреждения памяти требовалась длинная строка из символов `<>`. Рассмотрим также следующий искусственный фрагмент кода с уязвимостью разбора повторяющегося строкового разделителя:

```
void parse (char *inbuf)
{
char cpy[16];
char *cursor;
char *delim_index;
int   length = 0;

for (cursor=inbuf; *cursor; cursor++)
{
if (*cursor == ':')
delim_index = cursor;
else
length++;
}

// -2 для завершителя null и разделителя ':'
if (length < sizeof(cpy) - 2)
strcpy(cpy, inbuf);
}
```

Этот уязвимый анализатор обрабатывает строку, ожидая найти единственный экземпляр символа-разделителя, то есть двоеточия. Если разделитель найден, сохраняется указатель на его индекс в строке. В противном случае переменная `length` увеличивается на единицу. В конце цикла вычисленное значение `length` за вычетом единицы для завершающего `NULL` и ещё одной единицы для найденного разделителя сравнивается с размером целевого символьного буфера, чтобы убедиться в наличии достаточного места для последующего вызова `strcpy()`. Эта логика правильно обрабатывает ожидаемые строки вроде `name:pedram amini`. Для такой строки анализатор вычислит `length` как 16, определит, что места недостаточно, и пропустит `strcpy()`. А как насчёт строки `name::::::::::::::::::::::pedram`? Для неё вычисленное значение `length` равно 10, поэтому условная проверка проходит и выполняется `strcpy()`. Однако фактическая длина строки, используемой в `strcpy()`, равна 32, и будет вызвано переполнение стека.

Строки формата

Уязвимости строк формата - сравнительно простой для обнаружения класс ошибок, и создаваемые фаззером данные должны их учитывать. Уязвимость строки формата можно выявить любым маркером формата, например `%d`, отображающим десятичное число по основанию 10, или `%08x`, отображающим шестнадцатеричное число по основанию 16. Для фаззинга лучше выбрать маркеры `%s` или `%n`, либо оба. Они дают более высокую вероятность вызвать обнаружимую ошибку, например нарушение доступа к памяти. Большинство маркеров строки формата приводит к чтению памяти из стека и вряд ли вызовет сбой нижележащего уязвимого кода. Маркер `%s` вызывает больше операций чтения памяти, поскольку стек разыменовывается в поиске байта `NULL`, обозначающего завершение строки. В большинстве случаев маркер `%n` даёт наибольшую вероятность вызвать сбой. Список эвристик фаззинга должен включать длинные последовательности `%s%n`.

> **Ключ к эксплуатации уязвимостей строк формата**
>
> Все остальные маркеры строки формата, включая `%d`, `%x` и `%s`, вызывают чтение памяти из стека, а маркер `%n` уникален тем, что вызывает запись в память. Это ключевое требование для эксплуатации уязвимостей строк формата с целью исполнения кода. Другие внедрённые маркеры строки формата можно использовать для «утечки» потенциально важных сведений из нижележащего уязвимого программного обеспечения, но применение `%n` - единственный способ непосредственной записи в память через строку формата. Именно поэтому Microsoft решила реализовать средство включения и отключения поддержки маркера `%n` в семействе функций `printf`. За это отвечает API `_set_printf_count_output()`:[22] ненулевое значение включает поддержку `%n`, а нулевое отключает. Фактически `%n` настолько редко используется в промышленном коде, что по умолчанию его поддержка отключена.

Преобразование символов

Ещё одна тонкость, за которой следует следить, - преобразования и трансляции символов, особенно связанные с их расширением. Например, шестнадцатеричные значения `0xFE` и `0xFF` в UTF16 расширяются до четырёх символов. Недостаточный учёт такого расширения в коде разбора - распространённая область обнаружения уязвимостей.

Преобразования символов также могут быть реализованы неправильно, особенно при обработке редко встречающихся или используемых граничных случаев. Например, Microsoft Internet Explorer был затронут такой проблемой при преобразовании UTF-8 в Unicode.[23] Суть состояла в том, что при определении размера динамически выделяемой памяти для преобразованного буфера процедура преобразования неправильно учитывала пяти- и шестибайтовые символы UTF-8. Однако собственно процедура копирования данных правильно обрабатывала пяти- и шестибайтовые символы UTF-8, что приводило к переполнению буфера в куче. Список эвристик фаззинга должен включать эти и другие столь же вредоносные последовательности символов.

Обход каталогов

Уязвимости обхода каталогов затрагивают как сетевые демоны, так и веб-приложения. Распространено ошибочное убеждение, что они ограничены исключительно веб-приложениями. Уязвимости обхода каталогов действительно широко распространены в веб-приложениях, но этот вектор атаки может присутствовать и в проприетарных сетевых протоколах, и в других местах. Согласно статистике Mitre CVE за 2006 год, хотя со временем число уязвимостей обхода каталогов несколько уменьшилось, они остаются пятым по распространённости классом уязвимостей, обнаруживаемых в программных приложениях.[24] Эта статистика охватывает и веб-приложения, и традиционные клиент-серверные приложения. В Open Source Vulnerability Database (OSVDB) можно увидеть выборку множества уязвимостей обхода каталогов, поднимавших свои уродливые головы на протяжении многих лет.[25]

Рассмотрим, например, программу резервного копирования Computer Associates BrightStor ARCserve. BrightStor предоставляет специальный интерфейс журналирования по TCP через демон `caloggerd`. Хотя протокол не документирован, простой анализ пакетов показывает, что имя файла журнала фактически указывается внутри сетевого потока данных. Добавив перед переданным именем файла модификаторы обхода каталогов, злоумышленник может указать произвольный файл, в который будет записано произвольное журнальное сообщение. Поскольку демон журналирования работает с привилегиями суперпользователя, эту уязвимость можно тривиально эксплуатировать. Например, в системах UNIX в файл `/etc/passwd` можно добавить нового суперпользователя. На момент написания эта проблема оставалась уязвимостью нулевого дня, а её публичное раскрытие было запланировано на июль 2007 года. Список эвристик фаззинга должен включать такие модификаторы обхода каталогов, как `../../` и `....`.

Внедрение команд

Подобно уязвимостям обхода каталогов, уязвимости внедрения команд обычно связывают с веб-приложениями, а точнее со сценариями CGI. И снова распространено ошибочное мнение, будто этот класс ошибок присущ только веб-приложениям: они могут затрагивать сетевые демоны через документированные и проприетарные протоколы. Любая цель, веб-приложение или сетевой демон, передающая неотфильтрованные либо неправильно отфильтрованные пользовательские данные таким вызовам API, как `exec()` или `system()`, потенциально открывает уязвимость внедрения команд. Рассмотрим следующий упрощённый фрагмент кода Python:

```
directory = socket.recv(1024)
listing   = os.system("ls /" + directory)
socket.send(listing)
```

В обычных обстоятельствах сервер получает системный путь, определяет список файлов по этому пути и отправляет его клиенту. Однако из-за отсутствия фильтрации входных данных можно указать определённые символы, позволяющие выполнить дополнительные команды. В системах UNIX к ним относятся `&&`, `;` и `|`. Например, аргумент `var/lib ; rm -rf /` преобразуется в `ls /var/lib ; rm -rf /` - команду, способную доставить администратору затронутой системы значительные неприятности. Список эвристик фаззинга также должен включать эти символы.

Итоги

В начале главы мы обсудили необходимость автоматизации, а также привели краткий список и описание различных библиотек и утилит, способных облегчить разработку. Некоторые из этих библиотек подробнее объясняются и применяются при разработке специальных средств в частях II и III книги. Ключевой рассмотренной идеей были факторы выбора интеллектуальных и продуктивных значений фаззинга для чисел, строк и двоичных последовательностей. Эти сведения будут применяться и расширяться в последующих главах.

В следующих главах мы обсудим разнообразные цели фаззинга, включая веб-приложения, привилегированные приложения командной строки, сетевые службы и многое другое. При чтении помните об идеях, рассмотренных в данной главе. Обращайте внимание на различные библиотеки и средства, которыми можно воспользоваться при разработке описываемых фаззеров. Подумайте об интеллектуальных значениях фаззинга, обсуждённых здесь, и о том, какие будущие значения вы могли бы добавить в список.

Сноски

[1] [назад] http://www.threatmind.net/secwiki/FuzzingTools
[2] [назад] http://www.ethereal.com
[3] [назад] http://www.wireshark.org
[4] [назад] http://www.wireshark.org/faq.html#q1.2
[5] [назад] http://anonsvn.wireshark.org/wireshark/trunk/epan/dissectors/
[6] [назад] http://www.nologin.org/main.pl?action=codeView&codeId=49
[7] [назад] http://bastard.sourceforge.net/libdisasm.html
[8] [назад] http://www.packetfactory.net/libnet
[9] [назад] http://www.securityfocus.com/tools/1559
[10] [назад] http://www.tcpdump.org
[11] [назад] http://www.tcpdump.org/wpcap.html
[12] [назад] http://sourceforge.net/projects/dotmetro
[13] [назад] http://oss.coresecurity.com/projects/pcapy.html
[14] [назад] http://www.secdev.org/projects/scapy
[15] [назад] http://openrce.org/downloads/details/208/PaiMei
[16] [назад] http://openrce.org/downloads/details/208/PaiMei
[17] [назад] http://www.faqs.org/rfcs/rfc3501.html
[18] [назад] http://pedram.redhive.com/advisories/novell_netmail_imapd/
[19] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-053.html
[20] [назад] Если вы нам не верите, обратитесь к Google:
http://www.google.com/search?hl=en&q=%22perl+-e+%27print+%22A%22*%22
[21] [назад] http://xforce.iss.net/xforce/alerts/id/advise142
[22] [назад] http://msdn2.microsoft.com/en-us/library/ms175782.aspx
[23] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-017.html
[24] [назад] http://cwe.mitre.org/documents/vuln-trends.html#table1
[25] [назад] http://www.osvdb.com/searchdb.php?text=directory+traversal

Глава 7. Фаззинг переменных среды и аргументов

> «Вся эта внешняя политика немного раздражает».
>
> - Джордж У. Буш, цитируется по New York Daily News, 23 апреля 2002 года

Локальный фаззинг, пожалуй, является простейшим типом фаззинга. Хотя многие злоумышленники и исследователи получают более впечатляющие результаты, эксплуатируя удалённые уязвимости и уязвимости клиентской стороны, локальное повышение привилегий по-прежнему остаётся важной темой. Даже когда для получения доступа к целевой машине используется удалённая атака, локальные атаки часто служат вторичным вектором для получения необходимых привилегий.

Введение в локальный фаззинг

Пользователь может вводить переменные в программу двумя основными способами. Помимо очевидного стандартного устройства ввода, которым обычно является клавиатура, входными векторами служат аргументы командной строки и переменные среды процесса. Сначала мы представим аргументы командной строки как вектор фаззинга.

Аргументы командной строки

За исключением самых оберегаемых пользователей Windows, каждый когда-либо сталкивался с программой, которой требовались аргументы командной строки. Аргументы командной строки передаются программе и доступны через указатель `argv`, объявляемый в функции `main` программ на C. В `main` также передаётся переменная `argc`. Она содержит число аргументов, переданных программе, плюс единицу, поскольку имя программы в том виде, в каком она была вызвана, также считается аргументом. Рассмотрим несколько простых примеров.

```
int main(int argc,char *argv[])
{
int ix;
for (ix=0;ix<argc;ix++)
printf("argv[%d] == %s\n",ix,argv[ix]);
}
```

Если несколько раз запустить программу с разными аргументами, будут получены результаты, показанные на рисунке 7.1.

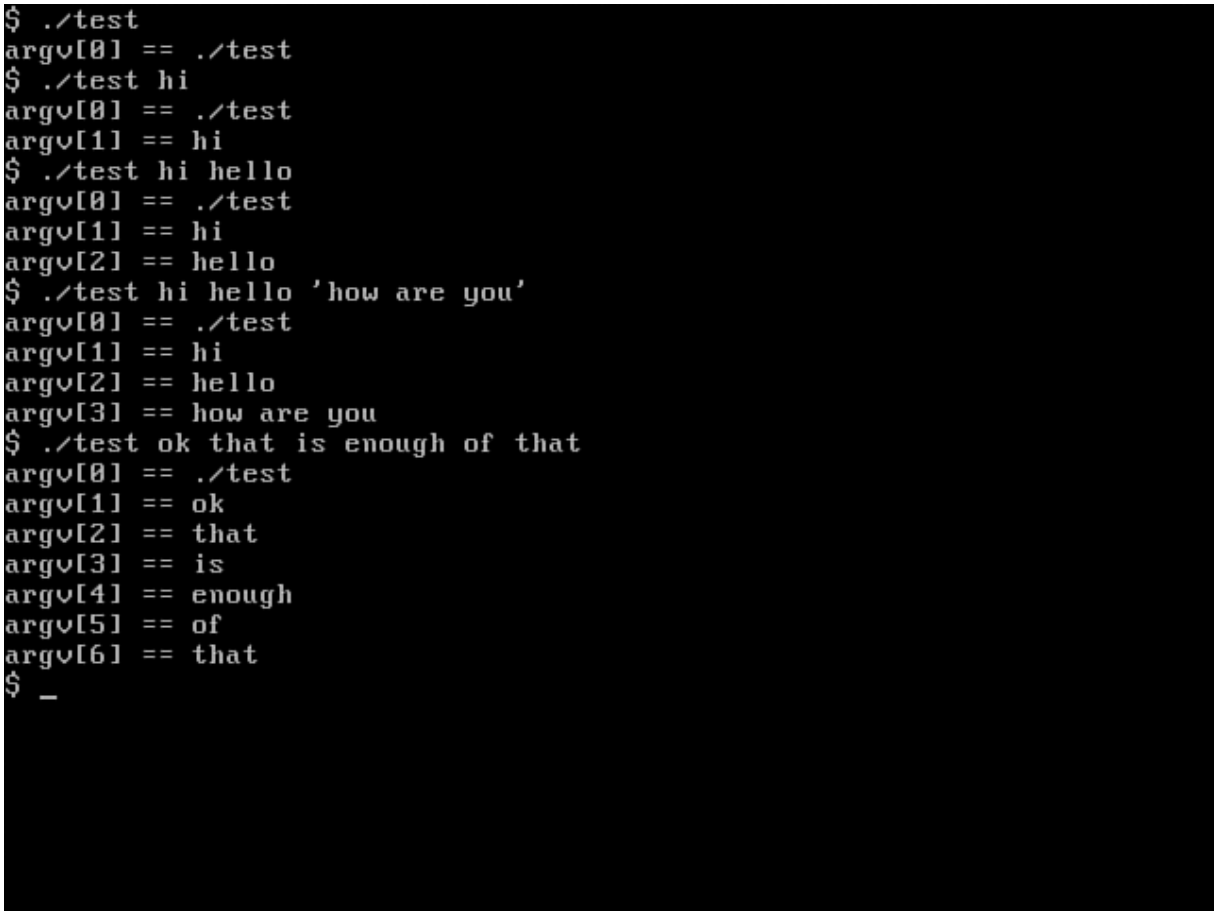


Рисунок 7.1. Демонстрация хранения аргументов командной строки

Переменные среды

Ещё один способ ввести переменные в процесс - использовать переменные среды. Каждый процесс содержит так называемую среду, состоящую из переменных среды. Переменные среды - глобальные значения, определяющие поведение приложений. Пользователь может задавать и отменять их, но обычно стандартные значения устанавливаются при установке пакета программного обеспечения или администратором. Большинство командных интерпретаторов заставляет все новые процессы наследовать текущую среду. Оболочка `command.com` - пример командного интерпретатора Windows. В системах UNIX обычно имеется несколько командных интерпретаторов, таких как `sh`, `csh`, `ksh` и `bash`.

К часто используемым переменным среды относятся `HOME`, `PATH`, `PS1` и `USER`. Эти значения содержат соответственно домашний каталог пользователя, текущий путь поиска исполняемых файлов, приглашение командной строки и текущее имя пользователя. Данные переменные достаточно стандартны, но многие другие распространённые переменные, в том числе создаваемые поставщиками программного обеспечения, используются только при работе их приложений. Когда приложению требуется узнать определённую переменную, оно просто вызывает функцию `getenv`, указав имя переменной в качестве аргумента. Хотя процессы Windows имеют среду так же, как приложения UNIX, мы в основном сосредоточимся на UNIX, поскольку в Windows отсутствует понятие приложений `setuid`, которые непривилегированный пользователь может запустить и которые получают привилегии во время исполнения. На рисунке 7.2 показано, как может выглядеть типичная среда UNIX. Текущую среду оболочки `bash` можно просмотреть командой `set`.

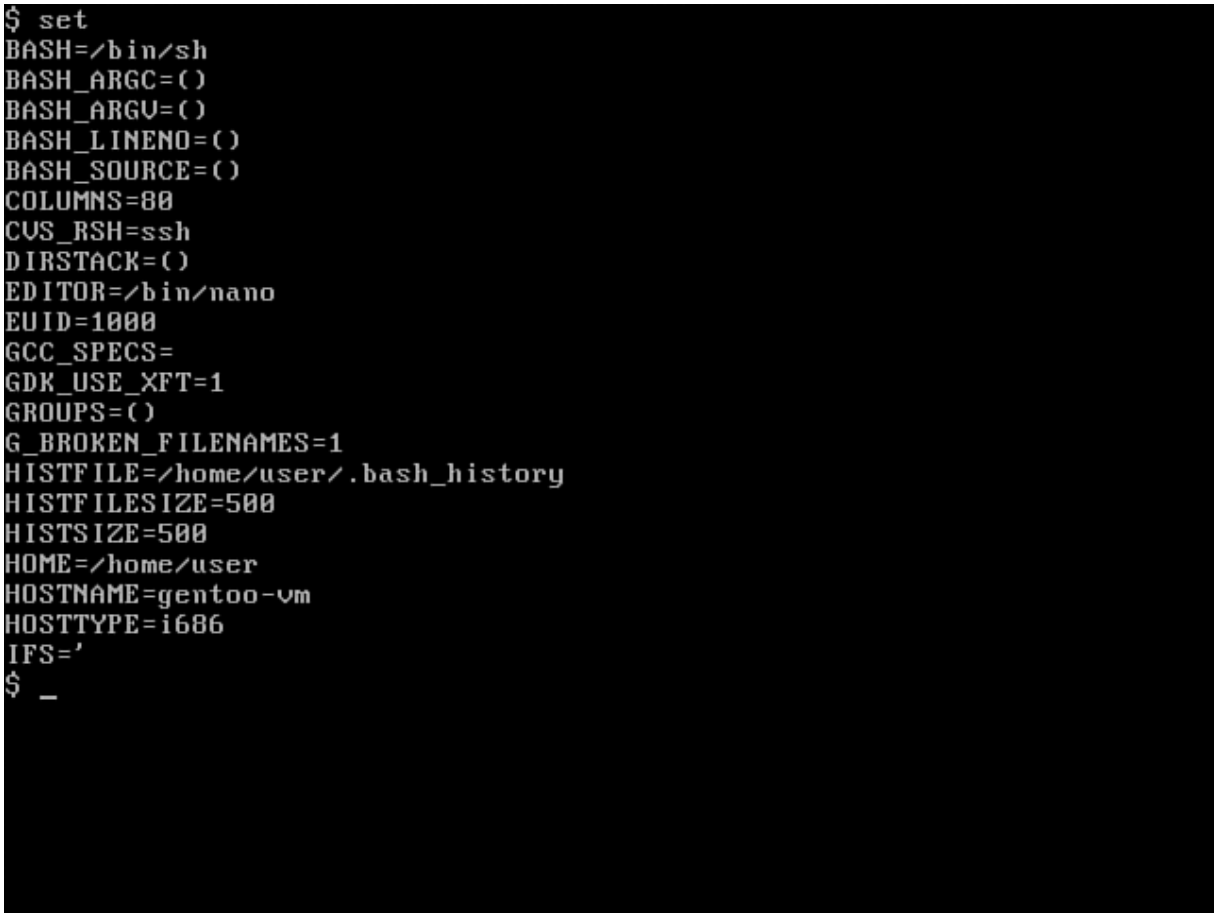


Рисунок 7.2. Пример некоторых переменных среды, используемых оболочкой bash

Пользователь может изменить любую переменную из этого списка с помощью команды `export`. Теперь, понимая использование аргументов командной строки и переменных среды, мы можем перейти к основным принципам их фаззинга.

Принципы локального фаззинга

Идея фаззинга переменных среды и командной строки проста: как отреагирует приложение, если переменная среды или параметр командной строки содержит неожиданное значение? Разумеется, нас интересуют только неправильно работающие привилегированные приложения. Причина в том, что локальный фаззинг требует локального доступа к машине. Поэтому простое аварийное завершение приложения имеет ограниченную ценность - вы проводили бы атаку отказа в обслуживании против самого себя. Если система используется несколькими пользователями, некоторый риск может существовать при обнаружении переполнения в переменной среды, вызывающего аварийное завершение системы или общего приложения. Однако больше всего нас интересует обнаружение переполнения буфера в привилегированном приложении, которое позволит пользователю с ограниченными правами повысить свои привилегии. Поиск привилегированных целей обсуждается далее в разделе «Поиск целей» этой главы.

Многие приложения рассчитаны на получение аргументов командной строки от пользователя при запуске. Затем приложение использует эти данные, чтобы определить требуемые действия. Прекрасный пример - приложение `su`, имеющееся почти во всех системах UNIX. Когда пользователи вызывают его без аргументов, предполагается, что они хотят пройти аутентификацию как пользователь `root`; если же первым аргументом указано другое имя, подразумевается, что они хотят переключиться на этого пользователя, а не на `root`.

Рассмотрим следующий код на C, упрощённо показывающий, как команда su может вести себя по-разному при разных аргументах:

```
int main(int argc,char *argv[])
{
[...]
if (argc >1)
become(argv[1]);
else
become("root");
[...]
}
```

Аргументы командной строки и переменные среды, по существу, всего лишь два разных вектора введения переменных в программу. Основная идея их фаззинга проста. Что произойдёт, если передать приложению через командную строку фиктивные данные? Приведёт ли такое поведение к риску безопасности?

Поиск целей

При локальном фаззинге в системе обычно имеется лишь несколько желательных двоичных целей. При исполнении эти программы обладают повышенными привилегиями. В системах на основе UNIX их легко распознать по установленным битам `setuid` или `setgid`.

Биты `setuid` и `setgid` указывают, что во время работы программа может получить повышенные привилегии. При наличии бита `setuid` процесс имеет привилегии владельца файла, а не запустившего его человека. При наличии бита `setgid` процесс имеет привилегии группы-владельца файла. Например, успешная эксплуатация программы с `setuid root` и `setgid staff` может предоставить оболочку с этими разрешениями.

Список двоичных файлов `setuid` тривиально строится с помощью команды `find`, стандартного средства UNIX и UNIX-подобных операционных систем. Следующей команды достаточно, чтобы вывести список всех двоичных файлов `setuid` в системе. Её следует выполнять от имени `root`, чтобы избежать ошибок чтения файловой системы:

```
find / -type f -perm -4000 -o -perm -2000
```

Команда `find` - очень мощное средство поиска в файловой системе строго определённых типов файлов, устройств и каталогов. В данном примере мы используем лишь несколько поддерживаемых ею параметров. Первый аргумент указывает, что поиск будет выполняться по всей системе, начиная с корневого каталога `/` и включая всё находящееся ниже. Параметр `type` сообщает `find`, что нас интересуют только файлы. Это означает, что символические ссылки, каталоги и устройства возвращаться не будут. Параметры `-perm` описывают интересующие нас разрешения. Параметр `-o` позволяет `find` использовать логическую операцию «или». Если у двоичного файла установлен бит `setgid` или `setuid`, выражение будет истинным и путь к файлу будет выведен. Итак, команда находит все обычные файлы, у которых установлен либо бит `setuid` (4), либо бит `setgid` (2). Ниже приведён пример вывода этой команды в стандартной установке Fedora Core 4:

```
[root@localhost /]# find / -type f -perm -4000 -o -perm -2000
/bin/traceroute6
/bin/traceroute
/bin/mount
/bin/su
/bin/ping6
/bin/ping
/bin/umount
/usr/bin/lppasswd
/usr/bin/gtali
```

```
/usr/bin/wall
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/glines
/usr/bin/gnibbles    ← все знают, что gnibbles совершенно необходима
/usr/bin/at             для работоспособной системы...
/usr/bin/gnotravex
/usr/bin/gnobots2
/usr/bin/sudo
/usr/bin/same-gnome
/usr/bin/gataxx
/usr/bin/rcp
/usr/bin/mahjongg
/usr/bin/iagno
/usr/bin/rlogin
/usr/bin/gnotski
/usr/bin/chage
/usr/bin/lockfile
/usr/bin/write
/usr/bin/gpasswd
/usr/bin/ssh-agent
/usr/bin/crontab
/usr/bin/gnomine
/usr/bin/sudoedit
/usr/bin/chfn
/usr/bin/slocate
/usr/bin/newgrp
/usr/bin/rsh
/usr/X11R6/bin/Xorg
/usr/lib/vte/gnome-pty-helper
/usr/libexec/openssh/ssh-keysign
/usr/sbin/userhelper
/usr/sbin/userisdnctl
/usr/sbin/sendmail.sendmail
/usr/sbin/usernetctl
/usr/sbin/lockdev
/usr/sbin/utempter
/sbin/pam_timestamp_check
/sbin/netreport
/sbin/unix_chkpwd
/sbin/pwdb_chkpwd
```

Объяснение разрешений файлов UNIX

В UNIX модель разрешений файлов допускает три основных типа доступа: чтение, запись и исполнение. Для каждого файла также существует три набора разрешений. Они относятся к пользователю, группе и тем, кто не относится ни к одной из этих категорий, то есть к остальным. В любой конкретной ситуации фактически применяется только один из наборов. Например, если вы владеете файлом, используется набор разрешений пользователя. Если вы не владеете файлом, но состоите в группе-владельце, применяются разрешения группы. Во всех остальных случаях применяются разрешения для остальных. Рассмотрим пример:

```
-r-x--x--- 2 dude staff  2048 Jan  2 2002  File
```

В этом примере файл принадлежит пользователю `dude`. Разрешения данного пользователя включают чтение и исполнение. Разумеется, поскольку пользователь владеет файлом, он может изменить эти разрешения любым образом.

Если другой участник группы `staff` попытается обратиться к файлу, он сможет лишь исполнить его, но не прочитать. Попытки чтения завершатся ошибкой недопустимых разрешений. Наконец, всем остальным пользователям будет отказано в доступе, поскольку у них нет прав чтения, записи или исполнения.

В UNIX существует особый способ описания абсолютных разрешений файлов. В этой системе разрешения представлены в восьмеричном виде. Иными словами, каждой комбинации разрешений соответствует значение от 0 до 7. Флаг чтения имеет восьмеричное значение 4, флаг записи - 2, а флаг исполнения - 1. Эти числа складываются для получения общего разрешения. Например, разрешения файла, допускающего чтение и запись для пользователя, группы и остальных пользователей, обозначаются как 666. Разрешения файла из примера, принадлежащего `dude`, представлены числом 510: пользователь (5) = чтение (4) + исполнение (1), группа (1) = исполнение (1), остальные (0) = отсутствие прав.

Четвёртый столбец представляет специальные флаги, такие как биты `setuid` и `setgid`. Бит `setuid` обозначается числом 4, а `setgid` - числом 2. Поэтому файл одновременно с `setuid` и `setgid` может иметь разрешения 6755. Если столбец специальных флагов опущен, подразумевается нулевое значение и, следовательно, отсутствие расширенных разрешений.

Методы локального фаззинга

Пользователь может легко задавать переменные среды и аргументы командной строки, а поскольку почти всегда это простые строки ASCII, против цели практично провести некоторое базовое ручное тестирование. Самая тривиальная проверка может состоять в задании переменной `HOME` длинной строкой и последующем запуске цели, чтобы увидеть результат. Это можно очень быстро сделать с помощью Perl, по умолчанию доступного в большинстве систем UNIX:

```
HOME=`perl -e 'print "X"x10000'` /usr/bin/target
```

Это крайне примитивный способ проверить, способно ли приложение обработать длинное значение `HOME`. Однако пример предполагает, что вы уже знаете приложение, использующее переменную `HOME`. А если вы не знаете, какие переменные применяет программа? Как их определить?

Перечисление переменных среды

Существует не менее двух автоматических способов определить переменные среды, используемые программой. Если система поддерживает предварительную загрузку библиотек, можно перехватить библиотечный вызов `getenv`. Новая функция `getenv`, которая выполняет стандартные действия `getenv` и одновременно записывает вызов в файл журнала, фактически зарегистрирует все переменные, запрошенные приложением. Расширенный вариант этого метода подробнее описан далее в разделе «Автоматизация фаззинга переменных среды».

Метод с GNU Debugger (GDB)

Другой доступный метод требует отладчика. С помощью GDB можно установить точку останова внутри функции `getenv` и вывести первый аргумент. Ниже приведён пример автоматизации этого процесса в Solaris 10 посредством сценарных возможностей GDB:

```
(08:55AM)[user@unknown:~]$gdb -q /usr/bin/id
(no debugging symbols found)...(gdb)
(gdb) break getenv
Function "getenv" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (getenv) pending.
```

```
(gdb) commands
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>silent
>x/s $i0
>cont
>end
(gdb) r
Starting program: /usr/bin/id
[...]
Breakpoint 2 at 0xff2c4610
Pending breakpoint "getenv" resolved
(no debugging symbols found)...
0xff0a9064:      "LIBCTF_DECOMPRESSOR"
0xff0a9078:      "LIBCTF_DEBUG"
0xff24b940:      "LIBPROC_DEBUG"
0xff351940:      "LC_ALL"
0xff351948:      "LANG"
0xff3518d8:      "LC_CTYPE"
0xff3518e4:      "LC_NUMERIC"
0xff3518f0:      "LC_TIME"
0xff3518f8:      "LC_COLLATE"
0xff351904:      "LC_MONETARY"
0xff351910:      "LC_MESSAGES"
uid=100(user) gid=1(other)
Program exited normally.
(gdb)
```

Если использованные в этом сеансе GDB команды вам незнакомы, их можно кратко описать так:

• Команда `break` устанавливает точку останова на заданной функции или адресе. Здесь она заставляет исполнение программы остановиться внутри вызова `getenv`.
• Команда `commands` задаёт действия, выполняемые при достижении точки останова. В данном случае мы предписываем работать без сообщений и вывести значение регистра `i0` как строку с помощью `x/s`. На SPARC регистр `i0` содержит первый аргумент вызываемой функции.
• Следующая команда просто продолжает исполнение, чтобы не приходилось вручную продолжать его после каждого останова. Затем для запуска программы используется сокращённая форма команды `run`.

Этот метод немедленно показывает список из 11 переменных среды, запрашиваемых программой `/usr/bin/id`. Обратите внимание, что метод должен работать во всех системах, однако потребуется изменить имя разыменовываемого регистра, поскольку разные архитектуры используют для регистров разные мнемоники. Например, на x86 можно выводить `$eax`, на SPARC - `$i0`, а на MIPS - `$a0`.

Теперь, рассмотрев переменные, которые будет использовать цель, мы изучим способы более автоматизированного их тестирования.

Автоматизация фаззинга переменных среды

Вспомните, что в предыдущем разделе «Перечисление переменных среды» мы кратко упомянули предварительную загрузку библиотек; она также полезна для автоматизированного фаззинга. Чтобы получить значение переменной среды, программа должна вызвать функцию `getenv`. Если перехватить `getenv` и возвращать длинные строки при всех её вызовах, даже не потребуется знать список используемых переменных: можно подвергнуть фаззингу каждую, перехватывая все вызовы `getenv`. Это очень полезно при быстрой проверке небезопасного использования переменных среды.

Следующая функция представляет собой тривиальную реализацию `getenv`. Она использует глобальную переменную `environ`, указывающую на начало среды. Код просто перебирает массив

environ и проверяет, находится ли запрошенное значение в этом массиве. Если находится, возвращается указатель на содержащееся значение; в противном случае возвращается NULL, указывающий, что переменная не задана:

```
extern char **environ;
char *getenv(char *variable)
{
int ix=0;
while (environ[ix])
{
if ( ! ( strncmp(string,environ[ix],strlen(string))) &&
(environ[ix][strlen(string)] == '=')  )
{
printf("%s\n",environ[ix]+strlen(string)+1);
return environ[ix]+strlen(string)+1;
}
ix++;
}
```

Предварительная загрузка библиотек

Следующая тема - предварительная загрузка библиотек. Она предоставляет простой способ перехватывать функции, используя компоновщик операционной системы, чтобы фактически заменять их функциями, предоставленными пользователем. Хотя конкретные детали различаются от системы к системе, общая идея одинакова. Обычно пользователь задаёт определённой переменной среды путь к скомпилированной им библиотеке. Затем библиотека загружается при исполнении программы. Если она содержит символы, дублирующие символы программы, вместо исходных используются библиотечные. Говоря о символах, мы прежде всего имеем в виду функции. Например, если пользователь создаст библиотеку с функцией strcpy и предварительно загрузит её при запуске двоичного файла, тот вызовет пользовательскую версию strcpy вместо системной. У этого есть множество законных применений, например обёртывание вызовов для профилирования и аудита. Есть и применения при поиске уязвимостей. Рассмотрим использование обёртывания или полной замены процедуры getenv, запрашивающей значения переменных среды.

Следующая функция представляет собой простую замену getenv, которую можно применять для обнаружения простых проблем с длинными строками. Предварительная загрузка библиотеки позволяет заставить эту функцию переопределить настоящую getenv:

```
#define BUFFSIZE 20000
char *getenv(char *variable)
{
char buff[BUFFSIZE];
memset(buff,'A',BUFFSIZE);
buff[BUFFSIZE-1] = 0x0;
return buff;
}
```

Легко заметить, что функция возвращает длинную строку при каждом запросе переменной. Массив environ вообще не используется, поскольку правильность возвращаемых значений нас не интересует.

Этот метод применяется в распространяемой по GPL утилите sharefuzz Дейва Эйтела, с помощью которой было обнаружено множество уязвимостей в приложениях setuid. Чтобы начать этот тривиальный фазз-тест, просто скомпилируйте код C в разделяемую библиотеку и воспользуйтесь функцией предварительной загрузки библиотек вашей операционной системы, если она имеется. В Linux это выполняется в два этапа:

```
gcc -shared -fPIC -o my_getenv.so my_getenv.c
LD_PRELOAD=./my_getenv.so /usr/bin/target
```

При исполнении /usr/bin/target все вызовы getenv будут использовать нашу изменённую функцию.

Обнаружение проблем

Теперь, когда вы знакомы с основными методами локального фаззинга, необходимо научиться распознавать интересное неправильное поведение цели; во многих случаях оно совершенно очевидно. Например, программа может аварийно завершиться и вывести Segmentation Fault или сообщение о другом фатальном сигнале.

Однако, поскольку наша конечная цель - автоматизация, нельзя полагаться на ручное распознавание сбоя пользователем. Для наших целей необходим надёжный программный способ. Есть как минимум два хороших варианта. Простейший - проверять код возврата приложения. В современных системах UNIX и Linux, если приложение завершилось из-за необработанного сигнала, код возврата оболочки равен 128 плюс номер сигнала. Например, ошибка сегментации заставит оболочку получить десятичный код возврата 139, поскольку SIGSEGV имеет значение 11. Если программа завершается на недопустимой инструкции, оболочка получит код 132, поскольку значение SIGILL равно 4. Логика проста: если код возврата приложения для оболочки равен 132 или 139, пометить событие как потенциально интересный сбой.

Также стоит учитывать сигнал прерывания. SIGABRT представляет интерес из-за введения более строгой проверки кучи в новых версиях glibc. Abort - сигнал, который можно вызвать в процессе для его завершения и создания дампа памяти. Хотя в определённом случае процесс может прерваться при повреждении кучи, существуют хитроумные способы обойти это.

Использовать код возврата оболочки разумно, если фаззинг выполняется наспех собранным сценарием оболочки. Однако при использовании полноценного фаззера, написанного, например, на C или другом языке, следует применять функции wait или waitpid. Общий способ локального фаззинга в таком варианте - простой fork с execve в дочернем процессе вместе с wait или waitpid в родительском. При правильном использовании легко определить аварийное завершение дочернего процесса, проверив состояние, возвращённое через wait или waitpid. В следующую главу включён упрощённый фрагмент iFUZZ, средства локального фаззинга, иллюстрирующий этот метод.

Если вас беспокоит перехват сигналов, которые приложение может обработать и которые поэтому останутся незамеченными предыдущим методом, помимо перехвата процедуры сигнала существует как минимум одна альтернатива. Придётся использовать отладочный API системы, чтобы присоединиться к процессу и перехватить получаемые им сигналы до вызова обработчика. В большинстве операционных систем UNIX для этого применяется ptrace. Общий метод здесь - fork с ptrace и execve в родительском процессе вместе с waitpid и ptrace внутри цикла дочернего процесса, чтобы непрерывно отслеживать исполнение процессов, перехватывать и передавать дальше любые возникающие сигналы. Каждый раз, когда waitpid возвращает управление в дочернем процессе, это означает, что программа получила сигнал или завершилась. Необходимо проверить возвращённое waitpid состояние, чтобы определить, что именно произошло. В большинстве случаев придётся также явно предписать приложению продолжить исполнение и передать сигнал. Это также делается посредством ptrace. В качестве справочных примеров общего метода можно использовать реализации SPIKEfile и notSPIKEfile. Эти два средства предназначены для фаззинга файлов и подробно объясняются в главе 12 «Фаззинг форматов файлов: автоматизация в UNIX». В следующей главе приведён фрагмент кода, демонстрирующий данный метод.

Во многих случаях метод ptrace избыточен для локального фаззинга. Очень немногие приложения UNIX с setuid активно используют обработчики таких сигналов, как SIGSEGV и SIGILL. Кроме того,

начиная применять `ptrace`, вы вводите код, который не обязательно совместим с разными операционными системами и архитектурами. Учитывайте это, если проектируете приложение, предназначенное для работы на многих платформах без изменений.

В следующей главе мы представим реализацию простого фаззера командной строки, спроектированного для компиляции и работы практически в любой системе UNIX с компилятором C. Средство также включает простой фаззер в виде разделяемой библиотеки для метода перехвата `getenv`.

Итоги

Хотя обнаружение локальных уязвимостей приносит гораздо меньше славы, хорошая ошибка повышения привилегий по-прежнему ценна. Мы заложили основу для демонстрации различных способов автоматизированного обнаружения уязвимостей этих типов, а в следующей главе реализуем некоторые методы, чтобы действительно найти несколько ошибок.

Глава 8. Фаззинг переменных среды и аргументов: автоматизация

> «Это оружие массового уничтожения должно же где-то быть!»
>
> - Джордж У. Буш, Вашингтон, округ Колумбия, 24 марта 2004 года

В этой главе представлена iFUZZ - программа, реализующая фаззинг локальных приложений. Основные цели здесь - аргументы командной строки и переменные среды в программах UNIX с `setuid`, обсуждавшиеся в главе 7 «Фаззинг переменных среды и аргументов». В данной главе мы рассмотрим возможности iFUZZ, объясним проектные решения и обсудим, как iFUZZ применялась для обнаружения многочисленных локальных уязвимостей в IBM AIX 5.3.

Возможности локального фаззера iFUZZ

iFUZZ содержит несколько возможностей, которые вы могли бы представить у локального фаззера. Среди них механизм автоматической обработки нескольких целевых двоичных файлов, возможность выводить запускающие файлы на языке C для лёгкого воспроизведения ошибок и несколько разных, реализованных в некоторой степени модульно методов фаззинга. Одна из наиболее удобных особенностей iFUZZ состоит в том, что она без изменений работает почти в любой операционной системе UNIX или UNIX-подобной системе. С её помощью находили уязвимости в IRIX, HP-UX, QNX, MacOS X и AIX. Сердце фаззера находится в отдельных модулях для каждого типа фаззинга:

• **Модули фаззера Argv.** Первые два модуля настолько похожи, что их можно объяснить вместе. Они подвергают фаззингу значения `argv[0]` и `argv[1]` исполняемого файла. Основной способ использования этих двух модулей прост: укажите каталог, полный целевых приложений, и запустите программу. Она будет пробовать строки разной длины, а также строки со спецификаторами формата. Фактически используемые строки зависят от базы данных фазз-строк, которую достаточно легко дополнить строками конечного пользователя.
• **Модули фаззинга одного или нескольких параметров.** Этот модуль всё ещё можно считать глупым фаззером: пользователь не предоставляет ему никаких сведений о цели, и модулю они безразличны. Он просто забрасывает цель строковыми значениями для каждого возможного параметра. При фаззинге одного параметра iFUZZ перебирает значения от `a` до `Z` и пытается

выполнять команды следующего вида:

```
./target -a FUZZSTRING
```

Значение FUZZSTRING - строка из внутренней базы данных фазз-строк iFUZZ. Это быстрый способ обнаружить простые проблемы, связанные с параметрами, но он не найдёт более сложных проблем, например требующих значений нескольких параметров.

• **Фаззер getopt.** Этому модулю требуются некоторые сведения от пользователя, а именно строка параметров, которую приложение передаёт getopt, чтобы фаззинг применялся только к заведомо используемым приложением параметрам. Хотя такой фаззинг может занимать очень много времени, он способен быть значительно тщательнее всех остальных типов. Подобный модуль также позволяет обнаруживать гораздо более сложные уязвимости. Рассмотрим, например, приложение, где переполнение буфера возникает только при включённых параметрах отладки и подробного вывода и переданном длинном аргументе -f. Ниже приведён пример вывода рассматриваемой вымышленной программы с информацией об использовании:

```
$ ./sample_program
Usage:
-f <file> Input filename
-o <file> Output filename
-v  Verbose output
-d  Debug mode
-s  Silent mode
```

По напечатанным сведениям об использовании можно определить, что строка getopt этого приложения, скорее всего, равна f:o:vds. Это означает, что параметры f и o принимают аргумент, а v, d и s являются простыми переключателями. Откуда это известно? Согласно странице руководства getopt:

> Аргумент options - строка, определяющая символы параметров, допустимых для программы. После символа параметра в строке может следовать двоеточие (:), указывающее, что параметр принимает обязательный аргумент. Если после символа параметра следуют два двоеточия (::), аргумент необязателен; это расширение GNU.

Если запустить iFUZZ в режиме фаззинга getopt со строкой f:o:vds, описанная нами уязвимость будет найдена очень скоро. Поскольку iFUZZ также случайным образом передаёт путь к существующему файлу как фазз-строку, можно обнаруживать даже уязвимости, требующие, чтобы один из параметров был допустимым файлом. Благодаря архитектуре iFUZZ проявлять здесь изобретательность несложно. Можно также добавить в базу другие допустимые строки: имена пользователей, имена узлов, допустимые дисплеи X и так далее. Возможности безграничны, и чем изобретательнее вы подходите к делу, тем тщательнее будет фаззинг.

В iFUZZ также входит простой предварительно загружаемый фаззер getenv. Он содержит массив переменных, которые требуется возвращать без изменений, и подвергает фаззингу все остальные. Это просто наспех переделанная sharefuzz Дейва Эйтела с небольшой дополнительной функцией, позволяющей при желании возвращать данные из настоящей среды. Фактически это не основной компонент iFUZZ, а скорее небольшая вспомогательная утилита.

Кроме того, включено простое средство перехвата getopt, которое можно предварительно загрузить, чтобы вывести строку параметров getopt из целевого двоичного файла. Хотя это полезно, на самом деле программа состоит всего из одной строки C и включена лишь для удобства. При использовании помните, что некоторые приложения применяют собственную функцию разбора параметров командной строки. Для них придётся вручную построить строку getopt на основе вывода функции справки об использовании.

Разработка

Теперь, когда возможности средства представлены, пора перейти к некоторым подробностям реализации. Чтобы полностью понять устройство iFUZZ, рекомендуем загрузить и изучить исходный код с fuzzing.org, но здесь мы выделим некоторые ключевые функции и рассмотрим конкретные проектные решения.

Подход к разработке

При создании iFUZZ ставилась цель сохранить модульность и, следовательно, расширяемость. Это позволило легко определить базовые функции приложения, не тратя с самого начала слишком много времени на каждый отдельный метод фаззинга. В действительности каждый модуль был добавлен позднее, уже после завершения основного механизма.

Первым модулем, разработанным после завершения базового механизма и вспомогательных функций, стал фаззер `argv[0]`. Набор тестов из нескольких двоичных файлов с известными уязвимостями переполнения и строк формата в `argv[0]`, а также нескольких безопасно написанных приложений использовался в разных операционных системах, включая QNX, Linux и AIX. Этот набор помог устранить мелкие ошибки и подтвердить точность результатов.

Как разработчик, вы не всегда будете знать о существующих уязвимых двоичных файлах, пригодных для набора тестов, поэтому можно создать собственные примеры уязвимых приложений. Например, простейший способ сделать приложение с уязвимостью строки формата в `argv[0]` выглядит так:

```
int main(int argc,char *argv[])
{
if (argc >1) printf(argv[1]);
exit(0);
}
```

Его можно скомпилировать и добавить к набору тестов вместе с несколькими заведомо безопасными двоичными файлами, чтобы выявлять ложноотрицательные и ложноположительные результаты.

> **Строки формата как вектор атаки**
>
> Уязвимости строк формата существуют уже некоторое время, однако когда-то считались сравнительно безвредными. В 1999 году после аудита безопасности proftpd 1.2.opre6 Тимм Твиллман опубликовал в списке рассылки BugTraq[1] подробности эксплойта, позволявшего злоумышленнику перезаписывать память через уязвимость строки формата. Причиной был вызов `snprintf()`, которому пользовательский ввод передавался без строки формата.[2]

Следующим реализованным модулем стал фаззер одного параметра. Хотя идея по-прежнему очень проста, при применении ко многим коммерческим системам UNIX она оказывается одной из самых эффективных. Проект был прост и не создавал настоящих трудностей. Обычный цикл перебирал весь алфавит, всякий раз используя выбранный символ как параметр программы. Аргумент этого параметра брался из базы данных фаззинга.

После реализации этих простых модулей стало ясно, что iFUZZ найдёт некоторые уязвимости, но не более трудные для вызова, например требующие нескольких параметров. Это привело к разработке следующих двух модулей: фаззеров нескольких параметров и `getopt`.

Архитектура фаззера нескольких параметров проста: это всего лишь фаззер одного параметра, вложенный в другой цикл фаззинга одного параметра. Максимальное число одновременно

используемых параметров можно указать в командной строке. Чем больше задано параметров, тем глубже становятся циклы.

Более эффективным и производительным вариантом фаззинга нескольких параметров стал фаззинг `getopt`. В основе это тот же фаззер нескольких параметров, но вместо неуклюжего перебора всех значений как параметров ему передаётся подмножество значений, о которых известно, что они значимы для приложения. Хотя технически это делает фаззинг менее тщательным, он гораздо быстрее завершается и часто значительно быстрее находит уязвимости.

Мы разработали два тривиальных метода перехвата исключений. Первый не обнаруживает обработанные сигналы, но, как упоминалось в предыдущей главе, очень немногие приложения UNIX обрабатывают интересующие нас сигналы.

Подход с fork, исполнением и ожиданием

Ниже приведена простая реализация подхода с `fork`, исполнением и ожиданием:

```
[...]
if ((pid = fork ()) != 0)
{
child = pid;
waitpid (pid, &status, 0);
if (WIFSIGNALED (status))
{
switch (WTERMSIG (status))
{
case SIGBUS:
case SIGILL:
case SIGABRT:
case SIGSEGV:
fprintf (stderr, "CRASH ON SIGNAL #%d\n",
WTERMSIG (status));
break;
default:
break;
}
}
}
else /* дочерний процесс */
{
execle ("/bin/program","program",NULL, environ);
perror ("execle");
}
[...]
```

Подход с fork, ptrace/исполнением и wait/ptrace

Следующий фрагмент кода C показывает, как можно создать процесс посредством `fork` и следить за интересными сигналами, даже если приложение обрабатывает их внутри. Этот сокращённый пример используется и в notSPIKEfile, и в SPIKEfile - двух средствах фаззинга файлов, обсуждаемых в главе 12 «Фаззинг форматов файлов: автоматизация в UNIX».

```
[...]
if ( !(pid = fork ()) )
{ /* дочерний процесс */
ptrace (PTRACE_TRACEME, 0, NULL, NULL);
execve (argv[0], argv, envp);
}
else
{ /* родительский процесс */
c_pid = pid;
```

```
monitor:
waitpid (pid, &status, 0);
if ( WIFEXITED (status) )
{ /* программа завершилась */
if ( !quiet )
printf ("Process %d exited with code %d\n", pid,WEXITSTATUS
(status));
return(ERR_OK);
}
else if ( WIFSIGNALED (status) )
{ /* программа завершилась из-за сигнала */
printf ("Process %d terminated by unhandled signal %d\n", pid, WTERMSIG
(status));
return(ERR_OK);
}
else if ( WIFSTOPPED (status) )
{ /* программа остановилась из-за сигнала */
if ( !quiet )
fprintf (stderr, "Process %d stopped due to signal %d (%s)
",pid,WSTOPSIG (status), F_signum2ascii (WSTOPSIG (status)));
}
switch ( WSTOPSIG (status) )
{ /* следующие сигналы обычно являются единственными интересующими нас */
case SIGILL:
case SIGBUS:
case SIGSEGV:
case SIGSYS:
printf("Program got interesting signal…\n");
if ( (ptrace (PTRACE_CONT, pid, NULL,
(WSTOPSIG (status) ==SIGTRAP) ? 0 : WSTOPSIG (status))) ==
-1 )
{
perror("ptrace");
}
ptrace(PTRACE_DETACH,pid,NULL,NULL);
fclose(fp);
return(ERR_CRASH); /* произошёл сбой */
}
/* передать сигнал дальше и продолжить трассировку */
if ( (ptrace (PTRACE_CONT, pid, NULL,
(WSTOPSIG (status) == SIGTRAP) ? 0 : WSTOPSIG (status))) == -1 )
{
perror("ptrace");
}
goto monitor;
}
return(ERR_OK);
}
```

Язык

Для разработки iFUZZ был выбран C. Хотя я мог бы придумать несколько ложных причин этого выбора, невозможно скрыть тот факт, что C использовался главным образом потому, что я ежедневно работаю на нём и чувствую себя с ним наиболее уверенно.

Однако у C явно есть преимущества помимо того, что это мой предпочтительный язык. Например, на большинстве машин UNIX, даже старых, уже установлены комплекты компиляторов C. Для таких языков, как Python или Ruby, это не обязательно верно. Perl также мог бы стать кандидатом, поскольку он довольно повсеместно распространён на машинах UNIX. Однако код Perl печально известен трудностью сопровождения.

Основное преимущество выбора для подобного проекта такого языка, как Python или Perl, состоит во времени разработки. Эти сценарные языки способны радикально его сократить.

В конечном счёте решение использовать C определили наше уверенное владение им и желание получить фаззер, который не был бы «поделкой» вроде всех меньших фаззеров, написанных нами на bash и других сценарных языках.

Практический пример

С помощью iFUZZ в IBM AIX 5.3 было выявлено более 50 локальных уязвимостей, каждая из которых имела разную степень полезности для специалиста по тестированию на проникновение или злоумышленника, хакера-взломщика. Многие из них обнаружить довольно легко: они скрывались лишь в `argv[0]` или `argv[1]`. Однако некоторые уязвимости оказались немного интереснее и «труднее» для поиска. Говоря «труднее», мы не подразумеваем, что их обнаружение требует каких-либо технических навыков. На самом деле понадобится лишь немного больше терпения при подготовке параметров командной строки iFUZZ и ожидании результатов. Следующие две уязвимости, находящиеся в одном двоичном файле с `setuid root`, демонстрируют мощь и эффективность iFUZZ как локального системного фаззера:

- `piomkpq -A ascii -p X -d X -D x -q LONGSTRING`
- `piomkpq -A ascii -p LONGSTRING -d X -D X -q`

Хотя целевой двоичный файл в этом случае не представляет значительной угрозы безопасности системы, поскольку для эксплуатации требуется доступ к группе `printq`, он выбран как пример из-за необычных условий срабатывания уязвимостей. Чтобы найти эти проблемы, мы прочитали страницу руководства двоичного файла и создали для iFUZZ строку `getopt`. Использовалась строка `a:A:d:D:p:q:Q:s:r:w:v:`.

Следующий вывод команды `ls` показывает расположение и разрешения двоичного файла:

```
-r-sr-x---   1 root     printq        32782 Dec 31 1969  /usr/lib/lpd/pio/etc/piomkpq*
```

После довольно продолжительной работы iFUZZ обнаружила в двоичном файле не менее двух интересных и, возможно, пригодных для эксплуатации сбоев. В приведённом выше представлении уязвимостей `LONGSTRING` означает строку приблизительно из 20 000 символов, а `X` - любую разумную строку, например саму строку `X`. Чтобы достичь уязвимого кода, значения `X` должны использоваться вместе с длинными строками.

Хотя возможность обнаружить подобные, более сложные для вызова уязвимости интересна, не менее увлекательно увидеть, сколько простых ошибок удастся найти за один проход по всем базовым модулям iFUZZ. Запустите iFUZZ в режиме `argv[0]`, `argv[1]` или одного параметра для двоичных файлов `setuid` в AIX 5.3, и вы обнаружите достаточно переполнений кучи, переполнений стека и проблем строк формата, чтобы занять себя на несколько дней. О многих уязвимостях сообщили IBM, и после обнаружения с помощью iFUZZ их исправили - как авторы, так и другие независимые исследователи, которые, скорее всего, применяли для поиска сходные идеи фаззинга.

Преимущества и возможности улучшения

Надеемся, после прочтения главы у вас появились некоторые идеи о полезных качествах iFUZZ. Не менее важно, чтобы вы выявили и некоторые её слабые места. Ниже приведены несколько наблюдений об iFUZZ, сделанных после разработки.

• Прежде всего, iFUZZ не учитывает возможность вызвать сбой системы, если не считать нескольких жёстко заданных задержек, предназначенных для снижения нагрузки. Было бы полезно анализировать нагрузку системы и определять, не нужно ли увеличить эту задержку, чтобы

предотвратить зависание системы, а также ложноположительные и ложноотрицательные результаты, возможные при высокой нагрузке.
• Желателен специальный параметр, определяющий приблизительную длину строки или абсолютный минимум, необходимый для вызова сбоя. Это достаточно тривиальная функция, но она экономила бы по несколько минут то тут, то там.
• Ещё одна отсутствующая в iFUZZ тривиальная функция - возможность автоматически находить в системе все двоичные файлы `setuid` и `setgid`. Это также сэкономило бы время.
• Возможность разбирать параметры заданного двоичного файла была бы превосходной, но реализовать её, вероятно, непросто. Если бы пользователь мог указать двоичный файл, а iFUZZ автоматически определяла необходимые приложению флаги и параметры, это обеспечило бы интеллектуальный и полный фаззинг при минимальном участии пользователя. Разумеется, реализация нетривиальна, поскольку приложения используют множество разных форматов. Даже после достаточно полной реализации функции новое приложение с новым стилем справки об использовании могло бы оказаться неразбираемым текущим кодом. Тем не менее вполне возможно сделать функцию расширяемой, чтобы пользователи добавляли новые форматы встречающихся им приложений.
• Мы обнаружили, что возможность генерации кода C реализуется тривиально, но иметь её бесценно. Если после сбоя создаётся программа на C, уязвимость можно почти мгновенно воспроизвести вручную. Сгенерированный код может даже использоваться как основа эксплойта. Это экономит время разработчика эксплойта, особенно при фаззинге слабой системы с множеством сбоев, когда время на написание каркасных программ C для вызова уязвимостей начинает накапливаться.

Итоги

При столь значительном внимании к удалённым уязвимостям, а в последнее время и к уязвимостям клиентской стороны, локальная безопасность системы остаётся областью, полной сравнительно легкодоступных целей. Реализация локального фаззера позволяет с очень небольшими затратами времени обнаружить и многие такие цели, и более сложные уязвимости, доказывая, что фаззинг способен приносить большое вознаграждение.

Сноски

[1] [назад] http://seclists.org/bugtraq/1999/Sep/0328.html
[2] [назад] http://en.wikipedia.org/wiki/Format_string_attack

Глава 9. Фаззинг веб-приложений и серверов

> «Я мастер низких ожиданий».
>
> - Джордж У. Буш, на борту Air Force One, 4 июня 2003 года

Теперь мы переходим от локального фаззинга к фаззингу в клиент-серверной архитектуре. В частности, рассмотрим фаззинг веб-приложений и веб-серверов. Как будет показано, фаззинг веб-приложения способен также выявить уязвимости нижележащего веб-сервера, но для простоты далее мы будем называть этот класс просто фаззингом веб-приложений. Хотя основные идеи остаются такими же, как в ранее обсуждавшемся сетевом фаззинге, необходимо внести несколько поправок. Во-первых, входные данные веб-приложений многочисленны и часто находятся в

неочевидных местах, поэтому потребуется заново определить, что считать входным вектором. Во-вторых, нужно скорректировать механизмы обнаружения, чтобы они могли перехватывать и интерпретировать сообщения об ошибках веб-приложений, которые способны раскрыть пригодное для эксплуатации условие. Наконец, чтобы сделать фаззинг веб-приложений практичным, необходимо построить архитектуру, компенсирующую потерю производительности из-за передачи входных данных по сети.

Что такое фаззинг веб-приложений?

Фаззинг веб-приложений - специализированная форма фаззинга сетевых протоколов. Тогда как фаззинг сетевых протоколов, рассматриваемый в главе 14 «Фаззинг сетевых протоколов», изменяет сетевые пакеты любого типа, фаззинг веб-приложений сосредоточен именно на пакетах, соответствующих спецификации HTTP. Мы выделили фаззинг веб-приложений в отдельную методологию из-за распространённости и важности веб-приложений в современном мире. Поставщики всё чаще переходят от традиционных программных продуктов, устанавливаемых непосредственно на локальном компьютере, к предоставлению программного обеспечения как услуги через Интернет. Веб-приложения могут размещаться как использующей их организацией, так и третьей стороной. При участии третьей стороны такую модель часто называют моделью поставщика услуг приложений (ASP).

Веб-приложения дают ряд преимуществ и поставщикам программного обеспечения, и конечным пользователям. Они обеспечивают поставщикам регулярный доход и позволяют мгновенно применять обновления, включая исправления безопасности. Конечным пользователям больше не нужно загружать и устанавливать исправления, поскольку приложение находится на центральном сервере. С точки зрения конечного пользователя это также означает меньшие накладные расходы на сопровождение приложения, так как им занимается ASP. Однако ценой становится необходимость доверить ASP поддержание надлежащей безопасности, защищающей ваши личные данные от любопытных глаз. Поэтому безопасность веб-приложений должна быть важнейшей заботой.

> **Microsoft Live**
>
> Microsoft делает крупную ставку на растущую важность веб-приложений. Хотя её хлебом с маслом традиционно были приложения с графическим интерфейсом, такие как Microsoft Office, в ноябре 2005 года председатель Microsoft Билл Гейтс объявил о запуске двух новых веб-приложений: Windows Live и Office Live.[1] Windows Live - набор служб для частных лиц, тогда как Office Live нацелен на малые предприятия. Microsoft будет получать доход от служб посредством рекламы и подписки. Комплекс служб Live впервые появился в ноябре 2002 года с запуском Xbox Live - сетевого сообщества и торговой площадки для игровой консоли Xbox.

Языки программирования и среды разработки веб-приложений также становятся всё удобнее для пользователя. В результате многие компании теперь создают собственные веб-приложения для внутреннего использования в корпоративных интрасетях и как средство упрощения торговли и общения с партнёрами и клиентами. Сегодня практически любую компанию можно считать разработчиком программного обеспечения независимо от её размера. Благодаря развитию технологий даже компания из одного человека способна создавать и размещать веб-приложения. Однако, несмотря на сравнительную простоту создания веб-приложения, создать безопасное веб-приложение нелегко. Если разработкой занимается компания, для которой создание программного обеспечения не является сильной стороной, приложение с меньшей вероятностью пройдёт строгую проверку безопасности перед развёртыванием.

К сожалению, простые в использовании и свободно доступные приложения для проверки безопасности веб-приложений не успевают за технологиями, предназначенными для первоначального создания этих приложений. Одна из целей книги - изменить такое положение. Ниже приведён обзор распространённых технологий, применяемых при подобной разработке.

Технологии веб-приложений

CGI

Общий шлюзовый интерфейс (CGI) - стандарт, первоначально разработанный Национальным центром суперкомпьютерных приложений (NCSA) для веб-сервера NCSA HTTPd в 1993 году. CGI определяет способ передачи данных между веб-клиентом и приложением, обрабатывающим запрос от имени веб-сервера.[2] Хотя с CGI можно использовать любой язык, чаще всего он реализуется на Perl.

PHP

Hypertext Preprocessor (PHP)[3] - популярный сценарный язык, часто применяемый для разработки веб-приложений. Широкая доступность интерпретаторов позволяет использовать PHP в большинстве операционных систем. Сценарии PHP можно встраивать непосредственно в страницы HTML для обработки тех частей веб-содержимого, которые требуют динамической генерации.

Flash

Flash первоначально разработала FutureWave Software, которую Macromedia приобрела в декабре 1996 года.[4] Flash несколько отличается от остальных упомянутых здесь технологий тем, что требует установки отдельного программного обеспечения клиентской стороны. Для большинства других названных технологий достаточно веб-браузера.

Macromedia удалось стимулировать распространение Macromedia Flash, сделав Macromedia Flash Player свободно доступным. Для реализации большинства интерактивных функций Flash применяет язык ActionScript. Хотя Flash прежде всего является технологией клиентской стороны, Macromedia Flash Remoting допускает взаимодействие между Flash Player и приложением веб-сервера.[5] Саму Macromedia в 2005 году приобрела Adobe Systems.[6]

JavaScript

Netscape создала JavaScript в 1995 году как средство добавления динамического содержимого на веб-страницы.[7] JavaScript можно применять как технологию клиентской или серверной стороны. На клиентской стороне JavaScript встраивается в HTML, передаваемый веб-браузеру, и интерпретируется непосредственно браузером. JavaScript также может использоваться как серверная технология и применяется веб-сервером при создании динамического содержимого. Другие серверные технологии, например Microsoft ASP.Net, рассматриваемая ниже, поддерживают включение JavaScript.

Java

Java была детищем Джеймса Гослинга из Sun Microsystems. Она начала жизнь под названием Oak и первоначально предназначалась для работы во встроенных системах. Позднее язык приспособили для веб-технологий, а название изменили на Java, когда выяснилось, что Oak уже зарегистрировано как товарный знак.[8] Java представляет собой нечто среднее между компилируемым и интерпретируемым языком. Исходный код Java компилируется в байт-код, который затем интерпретирует виртуальная машина Java, работающая на целевой платформе. Такой подход делает Java платформенно-независимой. Она часто используется в веб-браузерах как средство предоставления сложных интерактивных приложений.

ASP.Net

.Net - среда разработки, а не язык. Она состоит из среды Common Language Runtime (CLR), которую могут использовать многие языки, включая Visual Basic и C#. Microsoft представила .Net в 2002 году и рассчитывает, что она станет предпочтительной платформой разработки для многих приложений различных типов, включая веб-приложения. В этом отношении .Net похожа на Java: исходный код компилируется в промежуточный байт-код, известный как Common Intermediate Language (CIL), который затем интерпретируется виртуальной машиной.[9] Веб-приложения проектируются с помощью ASP .Net, не зависящей от языка. Приложения ASP.Net можно писать на любом совместимом с .Net языке.

Цели

Фаззинг веб-приложений способен выявить уязвимости в самом веб-приложении или в любом нижележащем компоненте его инфраструктуры, включая веб-сервер или сервер базы данных, с которым оно может быть интегрировано. Веб-приложения охватывают широкий спектр типов, но существуют распространённые категории приложений, которые часто предоставляются через Интернет. Ниже показана такая классификация и приведены примеры уязвимостей конкретных приложений, которые потенциально могли быть обнаружены методами фаззинга:

• **Веб-почта**
• Уязвимость межсайтового скриптинга Microsoft Outlook Web Access
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=261
• **Дискуссионные форумы**
• Уязвимость произвольного раскрытия файлов phpBB Group phpBB
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=204
• **Вики**
• Уязвимость внедрения команд через `tiki-user_preferences` в Tikiwiki
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=335
• **Веб-журналы**
• Исполнение произвольного кода PHP через переменную cookie `cache_lastpostdate` в WordPress
• http://www.osvdb.org/18672
• **Планирование ресурсов предприятия (ERP)**
• Разделение ответа HTTP через заголовок `sap-exiturl` в SAP Web Application Server
• http://www.osvdb.org/20714

• **Анализ журналов**
• Уязвимость удалённого исполнения команд в AWStats
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=185
• **Наблюдение за сетью**
• Уязвимость внедрения SQL-кода в IpSwitch WhatsUp Professional 2005 (SP1)
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=268
• Уязвимость удалённого включения файлов в Cacti нескольких поставщиков
• http://www.idefense.com/intelligence/vulnerabilities/display.php?id=265

Это никоим образом не полный перечень всех типов веб-приложений, но он должен дать представление о разновидностях приложений, хорошо подходящих для предоставления через Интернет, и показать несколько типов уязвимостей, которым они могут быть подвержены.

Методы

Прежде чем начать фаззинг веб-приложения, необходимо подготовить целевую среду, а затем выбрать входные векторы. Веб-приложения создают особые трудности в обоих отношениях. По своей архитектуре они могут быть развёрнуты на нескольких объединённых сетью машинах. Это обеспечивает масштабируемость, необходимую для развёртывания приложений в промышленной среде, но способно вызвать и нежелательные при фаззинге потери производительности. Кроме того, входные данные веб-приложения могут быть замаскированы множеством способов, но любые из них способны приводить к уязвимостям. Поэтому определять входные данные, пригодные для фаззинга, нужно достаточно широко.

Подготовка

В общем случае фаззинг требует способности быстро передавать цели множество входных данных. Цепочка событий для одного значения включает локальное создание входных данных, отправку целевому приложению, их обработку целью и последующее наблюдение за результатами. Таким образом, время работы фаззера определяется самым медленным звеном цепочки. При фаззинге локального приложения узким местом обычно становятся процессорные такты и время чтения и записи жёсткого диска. С учётом скорости современного компьютерного оборудования эти задержки минимальны, поэтому фаззинг является практичным подходом к исследованию уязвимостей.

При фаззинге веб-приложений узкое место обычно возникает из-за передачи сетевых пакетов от фаззера к цели. Рассмотрим процесс загрузки веб-страницы из удалённого местоположения. Когда вы просматриваете веб-страницу, скорость её отображения определяется скоростью трёх переменных: вашего компьютера, серверов, на которых находится страница, и интернет-соединения между ними. Вы управляете только первой из трёх переменных - локальной машиной. Поэтому при фаззинге веб-приложений важно ускорить сетевой трафик, устранив остальные две переменные. Когда возможно, вместо запуска целевого приложения на удалённом сервере разместите его локально, чтобы пакетам вообще не приходилось проходить через сеть. В большинство настольных операционных систем, таких как Windows XP[10] или Linux, встроены веб-серверы, поэтому обычно целевое приложение можно установить и настроить на локальной машине.

Приложение виртуальной машины (VM), такое как VMWare[11] или Microsoft Virtual Machine,[12] также может быть полезным средством при фаззинге веб-приложений. Целевое веб-приложение можно запустить в экземпляре VM, а фаззер - локально. Такая конфигурация даёт несколько преимуществ, которых нет при локальном запуске целевого приложения. Во-первых, объём потребляемых целью ресурсов лучше контролируется через VM. Это гарантирует, что фаззинг не поглотит все ресурсы машины. Кроме того, сетевой трафик, приводящий к сбоям системы и атакам отказа в обслуживании, не затронет локальную машину с фаззером. В конце концов, именно такие исключения мы пытаемся обнаружить, а сделать это трудно, если машина постоянно зависает.

Входные данные

До начала фаззинга нужно определить различные входные данные веб-приложения, поскольку именно они станут целями нашей работы. Это вызывает вопрос: что считать входными данными? Очевидно, что ими являются поля данных веб-формы, но как насчёт самого URL или файлов cookie, отправляемых веб-серверу? А заголовки веб-запроса? Ответ: всё перечисленное является входными данными. Для наших целей входными данными считается всё, что отправляется веб-серверу и интерпретируется им.

Через минуту мы разделим все возможные входные данные на категории, но сначала проследим процесс выполнения веб-запроса, чтобы лучше понять происходящее за кулисами. Большинство людей обращаются к веб-страницам с помощью браузера, например Microsoft Internet Explorer или Mozilla Firefox. При использовании браузера для получения содержимого веб-страницы достаточно ввести URL в адресную строку.

Однако веб-браузер скрывает значительную часть действий, фактически выполняемых при запросе страницы. Чтобы лучше понять весь процесс, запросим веб-страницу вручную. Для этого можно использовать приложение telnet, входящее в большинство современных операционных систем. Telnet подключается к выбранным узлу и порту, а затем отправляет и получает пакеты TCP. Чтобы запросить страницу, откройте окно консоли и выполните следующие команды:

```
telnet www.fuzzing.org 80[Return]
GET / HTTP/1.1[Return]
Host: www.fuzzing.org[Return]
[Return]
```

Разберём запрос. Сначала мы запускаем telnet и передаём ему два параметра: имя сервера (`fuzzing.org`) и порт (`80`) для подключения. По умолчанию Telnet подключается к порту TCP 23. Но поскольку запрос отправляется веб-серверу вручную, его нужно принудительно направить на порт TCP 80. Следующая строка представляет минимальный запрос, требуемый протоколом HTTP. Сначала мы сообщаем серверу используемый метод запроса (`GET`). Другие методы подробно описаны далее в главе. Затем передаём запрашиваемый путь или веб-страницу. В данном случае вместо конкретной страницы мы просим стандартную веб-страницу сервера (`/`). В той же строке веб-серверу сообщается желаемая версия протокола HTTP (`HTTP/1.1`). Следующий заголовок задаёт узел; в HTTP 1.0 он был необязательным, но в HTTP 1.1 стал обязательным.[13] После отправки запроса, обратите внимание, для завершения требуются два возврата каретки, сервер вернёт ответ, похожий на следующий:

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html
Set-Cookie: PREF=ID=56173d883ba96ae9:TM=1136763507:LM=1136763507:S=W43uFkQu1vexo
Pq-; expires=Sun, 17-Jan-2038 19:14:07 GMT; path=/; domain=.google.com
Server: GWS/2.1
Transfer-Encoding: chunked
Date: Sun, 08 Jan 2006 23:38:27 GMT
<html>
<head>
<meta http-equiv="content-type" content="text/html;charset=UTF-8">
<title>Google</title>
<style><!--
body,td,a,p,.h{font-family:arial,sans-serif;}
.h{font-size: 20px;}
.q{color:#0000cc;}
//-->
</style>
</head>
<body bgcolor=#ffffff text=#000000 link=#0000cc vlink=#551a8b
alink=#ff0000  topmargin=3 marginheight=3>
<center>
[snip]
```

```
<a href=http://www.google.com/intl/en/about.html>About Google</a>
<span id=hp style="behavior:url(#default#homepage)"></span>
</font><p><font size=-2>&copy;2006 Google</font></p></center>
</body>
</html>
```

Полученный ответ содержит исходный HTML-код веб-страницы, перед которым расположен ряд заголовков, предоставляющих веб-браузеру дополнительные сведения. Если сохранить HTML-часть ответа в файл и открыть её веб-браузером, вы увидите страницу, отображаемую при обычном переходе по URL. Однако некоторые изображения могут отсутствовать, если использовались относительные, а не абсолютные ссылки.

Мы упомянули, что предыдущий запрос представлял минимальный формат, необходимый для HTTP-запроса. Какие ещё входные данные можно предоставить веб-серверу? Чтобы ответить, сначала перехватим запрос, отправляемый браузером Internet Explorer. С помощью Ethereal удаётся обнаружить следующий запрос:

```
GET / HTTP/1.1
Accept: */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 1.1.4322;
.NET CLR 2.0.50727)
Host: www.google.com
Connection: Keep-Alive
Cookie:
PREF=ID=32a1c6fa8d9e9a7a:FF=4:LD=en:NR=10:TM=1130820854:LM=1135410309:S=b9I4GWDAtclpmX
BF
```

Что означают все дополнительные заголовки? Протокол HTTP определяет множество заголовков, и у каждого есть ряд допустимых значений. Полное описание HTTP/1.1 содержится в 176-страничном RFC 2616 «Hypertext Transfer Protocol - HTTP/1.1».[14] Вместо попытки пересказать весь документ рассмотрим встретившиеся выше заголовки:

• `Accept: */*`

Заголовок `Accept` определяет типы мультимедиа, которые можно использовать в ответе. В данном случае мы заявляем, что допустимы все типы (`*/*`). Однако ответы можно ограничить такими типами, как `text/html` или `image/jpeg`.

• `Accept-Language: en-us`

`Accept-Language` позволяет пользователю указать естественные языки, которые можно использовать в ответе. В нашем случае запрошен ответ на американском английском. Правильный формат языковых тегов определён в RFC 1766 «Tags for the Identification of Languages».[15]

• `Accept-Encoding: gzip, deflate`

И снова мы указываем допустимый формат ответа, но на этот раз определяем приемлемые схемы кодирования. Отправленный запрос указывает, что можно использовать алгоритмы кодирования gzip[16] или deflate.[17]

• `User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 1.1.4322; .NET CLR 2.0.50727)`

`User-Agent` определяет клиент, то есть веб-браузер, использованный для запроса. Для сервера это важно, поскольку позволяет приспосабливать ответы к разным функциям, поддерживаемым разными браузерами. Указанный здесь агент пользователя описывает Microsoft Internet Explorer 6.0 SP2 под управлением Windows XP SP2.

• `Host: www.google.com`

Этот заголовок определяет узел и порт, предоставляющие запрошенную веб-страницу. Если порт не включён, сервер предполагает стандартный порт веб-службы. Поле важно для сервера, поскольку один IP-адрес может обслуживать несколько имён узлов.

• Connection: Keep-Alive

Заголовок Connection позволяет клиенту указать различные желаемые параметры соединения. Постоянное соединение допускает несколько запросов без открытия отдельного соединения для каждого. Connection: close указывает, что соединение следует немедленно закрыть после отправки ответа.
• Cookie: PREF=ID=32b1c6fa8e9e9a7a:FF=4:LD=en:NR=10:TM=1130820854:LM=1135410309:S=b9I4GWDAtc2pmXBF

Файлы cookie можно хранить заданное время на локальном жёстком диске компьютера или в памяти; во втором случае они удаляются после завершения текущего сеанса. Cookie позволяет серверу узнавать отправителя запроса. Благодаря этому сайт может отслеживать привычки пользователя при просмотре и, возможно, настраивать для него веб-страницу. Если для определённого веб-сайта локально существует cookie, браузер отправит его серверу при выполнении запроса.

Теперь, проследив пример запроса, мы лучше подготовлены к определению разных входных данных для фаззинга веб-приложений. Как отмечалось, входными данными можно считать любую часть сведений, отправленных в запросе веб-серверу. На высоком уровне сюда входят метод запроса, унифицированный идентификатор ресурса (URI) запроса, версия протокола HTTP, все заголовки HTTP и данные POST. В следующих разделах мы разберём каждый компонент и определим подходящие переменные фаззинга:

```
[Method] [Request-URI] HTTP/[Major Version].[Minor Version]
[HTTP Headers]
[Post Data]
```

Метод

Для запроса веб-страницы чаще всего используются методы GET и POST. Они применяют пары «имя - значение», представляющие средство запроса конкретного содержимого веб-сервера. Считайте их переменными веб-запроса. Метод GET передаёт пары «имя - значение» веб-серверу внутри URI запроса. Например, http://www.google.com/search?as_q=security&num=10 отправит поисковой системе Google запрос, сообщающий, что мы хотим искать слово security (as_q=security) и получать по 10 результатов на странице (num=10). При использовании GET пары «имя - значение» добавляются к URI после символа ?, а разные пары разделяются символом &.

Пары «имя - значение» можно также передавать методом POST. При таком подходе они отправляются как заголовки HTTP после остальных стандартных заголовков. Преимущество POST состоит в возможности передавать значения любого размера. Хотя спецификация HTTP явно не ограничивает общую длину URI, веб-серверы и браузеры часто устанавливают предел. Если URI превышает ожидаемую длину, веб-сервер возвращает состояние 414 (Request-URI Too Long). Недостаток POST заключается в невозможности направить другого человека на динамически созданную страницу, просто переслав URI. Например, люди часто делятся картами и маршрутами, пересылая URI Google Maps, созданный после определённого поиска. Угадаете, куда приведёт следующая карта?

http://maps.google.com/maps?hl=en&q=1600+Pennsylvania+Ave&near=20500

Как уже говорилось, в запросе веб-сервера можно использовать ряд других методов. Вот краткое описание остальных допустимых методов:

• HEAD. Идентичен GET, но сервер возвращает только заголовки ответа без HTML-содержимого запрошенной веб-страницы.

• PUT. Позволяет пользователю загружать данные на веб-сервер. Хотя метод не поддерживается повсеместно, уязвимости обнаруживались, когда PUT поддерживался, но был неправильно реализован. Например, исправление безопасности из бюллетеня Microsoft MS05-006[18] без лишнего шума устранило именно такую уязвимость. Выяснилось, что неаутентифицированные пользователи могли загружать данные на сервер Microsoft SharePoint методом PUT.[19]
• DELETE. Позволяет пользователю запросить удаление ресурса с веб-сервера. И снова, хотя метод реализован не повсеместно, при тестировании следует убедиться, что он не поддерживается без ведома администратора. Неправильное предоставление этого метода может позволить злоумышленнику отказать законным пользователям в доступе, удалив ресурсы веб-сервера.
• TRACE. Позволяет клиенту отправить запрос, который затем возвращается тому же клиенту. Это полезно при отладке проблем соединения, поскольку можно увидеть структуру запроса, фактически дошедшего до сервера. В январе 2003 года Джеремия Гроссман из WhiteHat Security выпустил документ «Cross-Site Tracing (XST)»,[20] показавший, как сценарии клиентской стороны позволяют вредоносным веб-серверам получать значения cookie сторонних веб-серверов с поддержкой TRACE. И снова тестирование должно выявлять ситуации, когда TRACE поддерживается без ведома администратора.

• CONNECT. Зарезервирован для прокси-сервера, способного динамически переключаться в режим туннеля.
• OPTIONS. Позволяет клиентам запрашивать веб-сервер, чтобы определить поддерживаемые им стандартные и проприетарные методы. Сканеры уязвимостей могут использовать его для выяснения, реализованы ли потенциально уязвимые методы.

Например, с помощью OPTIONS можно было определить, работала ли на веб-сервере версия Internet Information Services (IIS), подверженная серьёзной уязвимости WebDAV - набора расширений HTTP, упрощающего публикацию веб-содержимого. Уязвимость подробно описана в MS03-007[21] (Unchecked Buffer in Windows Component Could Cause Server Compromise). При запросе OPTIONS * HTTP/1.0 сервер с поддержкой OPTIONS вернёт ответы, позволяющие определить, включён ли WebDAV.[22] Следующий ответ показывает, что WebDAV не включён, поскольку в заголовке Public нет ни одного расширения WebDAV:

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Mon, 17 Mar 2003 21:49:00 GMT
Public: OPTIONS, TRACE, GET, HEAD, POST
Content-Length: 0
```

Следующий ответ содержит целый ряд дополнительных заголовков с расширениями WebDAV. Наличие WebDAV на сервере Microsoft IIS 5.0 предполагает, что он может быть уязвим к MS03-007, если не установлены соответствующие исправления.

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Mon, 17 Mar 2003 21:49:00 GMT
Content-Length: 0
Accept-Ranges: bytes
DASL:
DAV: 1, 2
Public: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, POST, COPY, MOVE, MKCOL, PROPFIND,
PROPPATCH, LOCK, UNLOCK, SEARCH
Allow: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, POST, COPY, MOVE, MKCOL, PROPFIND,
PROPPATCH, LOCK, UNLOCK, SEARCH
Cache-Control: private
```

Request-URI

После метода клиент передаёт веб-серверу `request-URI`. Назначение `request-URI` - определить запрашиваемый ресурс, то есть веб-страницу. Он может иметь формат абсолютного URI, например `http://www.target.com/page.html`, или относительного URI (`/page.html`). Кроме того, вместо конкретного ресурса на сервере можно указать сам сервер. Для этого применяется символ *, обязательный для метода `OPTIONS`.

При фаззинге URI запроса можно подвергать фаззингу каждый его сегмент. Рассмотрим следующий относительный путь:

```
/dir/page.html?name1=value1&name2=value2
```

Его можно разделить на следующие компоненты:

```
/[path]/[page].[extension]?[name]=[value]&[name]=[value]
```

Каждый отдельный компонент можно и следует подвергать фаззингу. Некоторые нужно проверять известными значениями, приводившими к прежним уязвимостям, другим следует передавать случайные значения, чтобы определить, корректно ли сервер обрабатывает неожиданные запросы. Случайное содержимое должно также включать большие объёмы данных, позволяя установить, возникают ли переполнения буфера при разборе и интерпретации данных сервером. Рассмотрим каждый компонент отдельно:

• **Путь.** Самые распространённые уязвимости, обнаруживаемые при фаззинге компонента пути, - переполнения буфера и атаки обхода каталогов. Переполнение вероятнее всего выявить передачей больших объёмов данных, а обход каталогов - последовательными конструкциями `../`. Следует также применять разные схемы кодирования, пытаясь обойти процедуры проверки входных данных, которые могут выполняться до декодирования запроса.
• **Пример переполнения буфера.** Веб-сервер Macromedia JRun 4 до выпуска JRun 4 Updater 5 был подвержен переполнению стека при получении чрезмерно длинного пути.[23] Уязвимость срабатывает, когда длина пути превышает приблизительно 65 536 символов. Это показывает необходимость включать в фаззинг очень большие переменные.
• **Пример обхода каталогов.** Классический пример такой уязвимости предоставляет приложение 3Com Network Supervisor.[24] Оно включает веб-сервер, прослушивающий порт TCP 21700; было обнаружено, что в Network Supervisor 5.0.2 и более ранних версиях простой URL с последовательностями `../` позволял пользователю выйти за пределы корневого веб-каталога. Следует отметить, что подобные проблемы фактически являются уязвимостями сервера, а не приложения.
• **Страница.** Фаззинг распространённых имён страниц может выявить конфиденциальные страницы без надлежащей защиты или снова привести к уязвимостям переполнения буфера.
• **Пример переполнения буфера.** Microsoft IIS 4.0 был подвержен переполнению стека при чрезмерно длинных запросах к файлам с расширениями `.htr`, `.stm` и `.idc`. Уязвимость была подробно описана в бюллетене Microsoft MS99-019[25] и привела к появлению множества общедоступных эксплойтов.
• **Пример утечки информации.** В точке доступа 3Com OfficeConnect Wireless 11g имелась уязвимость, позволявшая обращаться к конфиденциальным веб-страницам административного интерфейса без соответствующих учётных данных.[26] Поэтому запрос страницы вроде `/main/config.bin` вместо приглашения входа раскрывал её содержимое, включая административные имя пользователя и пароль. Nikto[27] - пример приложения, которое ищет подобные уязвимости, многократно запрашивая у веб-сервера распространённые и, возможно, незащищённые страницы.
• **Расширение.** Как и локальные файлы, веб-страницы обычно имеют расширения, определяющие технологию их создания. Среди примеров: `*.html` (язык гипертекстовой разметки), `*.asp` (Active

Server Page) и `*.php` (Hypertext Preprocessor). В веб-серверах обнаруживались уязвимости при запросе страниц с неизвестными расширениями.

• **Имя.** Фаззинг распространённых компонентов имён может привести к обнаружению недокументированных переменных, которые можно передавать серверу. С другой стороны, неожиданные переменные способны нарушить работу приложения, если в нём отсутствует надлежащая обработка ошибок.

• **Значение.** Подход к фаззингу компонента значения зависит от типа переменной, ожидаемой веб-приложением. Например, если пара «имя - значение» в допустимом запросе выглядела как `length=50`, логично подвергать значение фаззингу чрезмерно большими или малыми числами. Что произойдёт, если переданная длина меньше фактической длины данных? Что случится при нулевом или даже отрицательном значении? Возможно, значение больше поддерживаемого приложением вызовет сбой или целочисленное переполнение. Ответы можно найти фаззингом. Если компонент ожидает строковое содержимое, попробуйте отправлять неожиданные строки и смотреть, корректно ли приложение их обрабатывает. И последнее, но не менее важное: заполняйте компонент всё большими объёмами данных, чтобы увидеть, возникнет ли переполнение буфера или сбой.

• **Разделитель.** Даже кажущиеся безобидными символы-разделители компонентов (`/`, `=`, `&`, `.`, `:` и прочие) должны стать целями фаззинга. Веб-сервер или приложение разбирают эти символы, и ненадлежащая обработка ошибок способна привести к пригодным для эксплуатации условиям, если сервер не справляется с неожиданными значениями.

Протокол

Числовыми переменными можно подвергать фаззингу версию HTTP, отправляя поддерживаемые и неподдерживаемые версии, что способно выявить уязвимости сервера. Самая новая стабильная версия протокола - HTTP/1.1. Версия HTTP разделяется на основную и дополнительную (`HTTP/[major].[minor]`).

Заголовки

Любые и все заголовки запроса можно и следует подвергать фаззингу. Они имеют следующий формат:

```
[Header name]: [Header value]
```

Следовательно, существуют три возможные переменные фаззинга: имя, значение и разделитель (`:`). Имена следует проверять известными допустимыми значениями, чтобы определить, поддерживает ли веб-приложение недокументированные заголовки. Список заголовков, соответствующих разным версиям HTTP, можно найти в следующих RFC:

• RFC 1945, «Hypertext Transfer Protocol - HTTP/1.0»[28]
• RFC 2616, «Hypertext Transfer Protocol - HTTP/1.1»[29]

Значения можно подвергать фаззингу, чтобы определить, способно ли приложение корректно обрабатывать неожиданные значения.

ПРИМЕР ПЕРЕПОЛНЕНИЯ КУЧИ

В январе 2006 года iDefense Labs опубликовала сведения об удалённо эксплуатируемой уязвимости переполнения кучи в Novell SUSE Linux Enterprise Server 9.[30] Уязвимость можно было вызвать, просто отправив запрос `POST` с отрицательным значением заголовка `Content-Length`. Ниже показан пример запроса, который вызывал переполнение:

```
POST / HTTP/1.0
Content-Length: -900

[Данные для перезаписи кучи]
```

Файлы cookie

Файлы cookie хранятся локально и передаются в виде заголовка HTTP, когда запрос отправляется веб-серверу, для которого ранее был сохранён файл cookie. Формат файла cookie выглядит следующим образом:

```
Cookie: [Name1]=[Value1]; [Name2]=[Value2] ...
```

И снова следует подвергать фаззингу имя, значение и разделители. Как и в случае с другими значениями, фаззинговые значения должны зависеть от типа переменной, которая передавалась бы в допустимом запросе.

Данные POST

Как упоминалось ранее, пары «имя - значение» можно передавать веб-серверу либо в URI запроса с помощью метода `GET`, либо в виде отдельных заголовков HTTP с помощью метода `POST`. Данные POST отправляются в следующем формате:

```
[Name1]=[Value1]&[Name2]=[Value2]
```

> **ПРИМЕР ПЕРЕПОЛНЕНИЯ БУФЕРА**
>
> Грег Макманус (Greg MacManus) из iDefense Labs обнаружил уязвимость переполнения буфера в веб-сервере, входившем в состав популярного беспроводного маршрутизатора Linksys WRT54G.[31] Он установил, что отправка странице `apply.cgi` запроса `POST` с длиной содержимого более 10 000 байт приводила к эксплуатируемому переполнению буфера. Хотя подобную уязвимость обычно было бы трудно эксплуатировать на встраиваемом веб-сервере, разработке демонстрационного кода эксплойта помогли открытый исходный код микропрограммы устройства и наличие модифицированной микропрограммы со средствами разработки.

Выявление входных данных

Теперь нам известны структура запросов к веб-приложениям и различные компоненты этих запросов, которые можно рассматривать как отдельные входные переменные и, следовательно, как потенциальные цели фаззинга. Следующий шаг - провести разведку и выявить все различные допустимые входные значения. Это можно сделать вручную или автоматически. Независимо от подхода мы хотим добиться как можно более полного покрытия. При изучении приложения нашей целью должно быть документирование всех следующих входных данных:

• веб-страницы;
• каталоги;
• методы, поддерживаемые страницами;
• веб-формы;
• пары «имя - значение»;
• скрытые поля;

• заголовки;
• файлы cookie.

Самый простой и наименее эффективный способ выявления перечисленных входных данных состоит в том, чтобы просто просматривать веб-сайт в браузере и изучать исходный код страниц. В исходном коде можно искать входные данные, включённые в веб-формы. Обязательно ищите скрытые поля (`input type="hidden"`), поскольку ленивые разработчики приложений иногда используют их как средство обеспечения безопасности за счёт сокрытия. Они могут полагать, что вы не станете проверять такие поля, поскольку на странице они не видны непосредственно. Очевидно, это крайне слабая мера контроля, так как скрытые поля всегда можно увидеть в исходном коде страницы.

При выявлении входных данных с помощью веб-браузера вы не сможете увидеть заголовки запросов и ответов. Например, вы не увидите структуру файлов cookie, передаваемых серверу приложений, поскольку они включены в заголовки HTTP и автоматически передаются браузером. Для просмотра необработанных запросов HTTP всегда можно воспользоваться анализатором сетевых протоколов, например Wireshark.[32]

Даже при наличии веб-браузера и анализатора трафика вручную выявлять все возможные входные данные веб-приложения становится непрактично, как только приложение разрастается более чем до нескольких веб-страниц. К счастью, обход веб-приложения можно автоматизировать с помощью веб-паука. Веб-паук, или веб-краулер, представляет собой приложение, которое выявляет все гиперссылки на веб-странице, переходит по ним, находит дополнительные гиперссылки и продолжает этот процесс, пока не посетит все возможные веб-страницы. Попутно оно может документировать важные для исследователя безопасности сведения, например упомянутые выше входные данные. К счастью, существует множество бесплатных веб-пауков и веб-пауков с открытым исходным кодом, обладающих превосходными возможностями.

Простой, но мощный веб-паук - утилита wget.[33] Хотя первоначально она была разработана для операционной системы UNIX, существуют и версии wget для платформы Win32.[34] Ещё один бесплатный веб-паук, который мы рекомендуем, входит в состав WebScarab.[35] Проект WebScarab представляет собой набор предоставляемых Open Web Application Security Project (OWASP) инструментов, полезных при анализе веб-приложений. В числе многочисленных утилит WebScarab есть веб-паук, который можно использовать для автоматического выявления URL во всём приложении. Другой полезный инструмент WebScarab - прокси-сервер, удобный при ручном аудите страниц. Если настроить веб-браузер на использование прокси WebScarab, он будет записывать все необработанные запросы и ответы во время просмотра приложения. В него даже входит самый простой веб-фаззер, но в следующей главе мы обсудим, как создать более мощный.

Уязвимости

Веб-приложения подвержены широкому спектру уязвимостей, каждую из которых можно выявить с помощью фаззинга. В следующем списке представлены основные классы уязвимостей:

• **Отказ в обслуживании (DoS).** Атаки DoS на веб-приложения представляют значительную угрозу. Хотя атака DoS не предоставит злоумышленнику доступа к цели, запрет доступа к сайту, который служит внешним порталом вашей корпорации и, возможно, источником дохода, способен причинить существенный финансовый ущерб.
• **Межсайтовый скриптинг (XSS).** По данным Mitre, в 2006 году на уязвимости XSS приходилось 21,5 процента новых обнаруженных уязвимостей.[36] Это делает уязвимости XSS самыми распространёнными не только среди веб-приложений, но и среди всех приложений. Когда-то уязвимости XSS считались скорее досадной неприятностью, чем угрозой безопасности, однако взрывной рост фишинговых атак всё изменил. Бреши XSS дают злоумышленнику возможность

управлять действиями на стороне клиента в веб-браузере и потому служат ценным инструментом для тех, кто проводит такие атаки.

• **Внедрение SQL.** Среди уязвимостей веб-приложений внедрение SQL является не только одной из самых распространённых, но и одной из самых критических слабостей. Статистика Mitre за 2006 год показывает, что внедрение SQL заняло второе место, составив 14 процентов новых уязвимостей. Распространённость объясняется как ростом числа динамических веб-сайтов на основе реляционных баз данных, так и небезопасными приёмами программирования SQL, которым по-прежнему учит большинство учебников. Некоторые ошибочно считают внедрение SQL лишь атакой, открывающей путь к чтению записей базы данных, а значит, исключительно риском для конфиденциальности. С появлением мощных хранимых процедур, поддерживаемых большинством реляционных баз данных, внедрение SQL представляет гораздо больший риск и может привести к полной компрометации доверенной внутренней системы.

• **Обход каталогов и слабое управление доступом.** Когда-то атаки обхода каталогов были относительно обычны, но теперь, к счастью, постепенно исчезают. С другой стороны, слабое управление доступом остаётся угрозой, поскольку разработчику совсем нетрудно забыть об ограниченной странице или каталоге и не применить надлежащие средства управления доступом. Это одна из многих причин, по которым веб-приложения следует постоянно проверять не только перед выпуском в рабочую среду, но и на протяжении всего срока их эксплуатации. Некогда безопасное веб-приложение может внезапно оказаться полностью открытым из-за ошибочного изменения конфигурации сервера, удалившего или ослабившего средства управления доступом.

• **Слабая аутентификация.** Схем аутентификации существует великое множество, и каждая может оказаться небезопасной при неправильной реализации. Слабые пароли уязвимы для атак методом полного перебора, а передача учётных данных открытым текстом делает их уязвимыми для перехвата в сети. Хотя при должной базовой осмотрительности такие ошибки обычно легко обнаружить, более трудная задача состоит в понимании бизнес-логики приложения и проверке того, не оставили ли разработчики по ошибке путь входа в приложение, позволяющий обойти средства аутентификации.

• **Слабое управление сеансами.** Поскольку HTTP является протоколом без сохранения состояния, необходим некоторый механизм управления состоянием, позволяющий различать одновременно работающих пользователей и не требовать ввода учётных данных для аутентификации на каждой странице. Не столь важно, передаются ли уникальные маркеры сеанса в файлах cookie, внутри URI или в самих данных страницы; гораздо важнее то, как они устроены и защищены. Файлы cookie должны быть достаточно случайными и иметь размер, делающий атаки методом полного перебора непрактичными. Кроме того, срок их действия должен истекать достаточно часто, чтобы защищать от атак повторного воспроизведения.

• **Переполнение буфера.** Атаки переполнения буфера встречаются в веб-приложениях реже, чем в настольных и серверных приложениях. Главным образом это заслуга языков, обычно используемых для разработки веб-приложений, например C# или Java: их средства управления памятью снижают риск переполнений буфера. Однако это не означает, что при фаззинге переполнения буфера следует игнорировать. Вполне возможно, что приложение передаёт предоставленные пользователем входные данные отдельному приложению, написанному на C или C++ и подверженному переполнению. Кроме того, при фаззинге помните, что у вас есть как минимум две цели: веб-приложение и веб-сервер. Веб-серверы, в свою очередь, часто пишутся на языках, подверженных переполнениям буфера.

• **Неправильно поддерживаемые методы HTTP.** Обычно веб-приложения обрабатывают запросы `GET` и `POST`. Однако, как обсуждалось ранее, существует множество других методов, соответствующих RFC или разработанных третьими сторонами. При неправильной реализации они могут позволить злоумышленнику изменять данные на сервере или получать ценную информацию для последующих атак. Поэтому фаззинг следует использовать для выявления всех поддерживаемых методов и определения того, правильно ли они реализованы.

• **Удалённое выполнение команд.** Веб-серверы могут просто передавать предоставленные пользователем входные данные другим приложениям или самой операционной системе. Если эти входные данные не проверяются надлежащим образом, злоумышленник может получить возможность непосредственно выполнять команды в целевой системе. Традиционно таким атакам особенно подвержены приложения на PHP и Perl.
• **Удалённое внедрение кода.** И снова эта уязвимость обычно поражает приложения PHP. Слишком часто встречается порочная практика программирования, при которой неочищенные пользовательские данные разрешается передавать методу вроде `include()` или `require()`, позволяющему включать код PHP из локальных или удалённых источников. Когда пользовательские данные безоговорочно передаются таким методам, злоумышленники могут внедрить в целевое приложение собственный код PHP. В статистике Mitre за 2006 год уязвимости включения файлов PHP заняли третье место, составив 9,5 процента новых уязвимостей.
• **Уязвимые библиотеки.** Разработчики часто ошибочно доверяют сторонним библиотекам, включаемым в приложение. Любой код, включённый в веб-приложение, независимо от того, написан ли он с нуля либо предварительно скомпилирован и получен от третьей стороны, потенциально уязвим и при проверке на дефекты безопасности должен подвергаться столь же тщательному анализу. Как минимум следует проверить архивы уязвимостей и убедиться, что включённые библиотеки не имеют известных уязвимостей. Помимо этого, их следует подвергнуть фаззингу и другим проверкам безопасности того же уровня, что и собственный код.
• **Разделение ответа HTTP.** Разделение ответа HTTP впервые получило широкую известность после публикации компанией Sanctum Inc. документа «Divide and Conquer».[37] Эта атака становится возможной, когда пользователь способен внедрить последовательность CRLF в заголовки ответа. В результате злоумышленник получает возможность сформировать ответ веб-сервера, что может привести к различным атакам, включая отравление кеша веб-прокси и браузера.
• **Подделка межсайтовых запросов (CSRF).** От атак CSRF трудно защититься, и они представляют растущую угрозу. Если у жертвы есть активный сеанс, уязвимость CSRF существует в том случае, когда злоумышленник может заставить жертву выполнить некоторое действие, просто убедив её щёлкнуть ссылку, например в сообщении электронной почты. Допустим, на банковском веб-сайте запрос на перевод средств выполняется отправкой веб-формы, содержащей сведения о счёте и переводимой сумме. Выполнить перевод может только владелец счёта, поскольку лишь он способен войти со своими учётными данными. Однако если этот человек уже вошёл в систему и, сам того не зная, щёлкнет ссылку, отправляющую те же данные веб-формы банковскому сайту, тот же перевод может произойти без ведома жертвы. Для защиты от атак CSRF веб-сайты нередко требуют от пользователя повторно пройти аутентификацию или выполнить некоторое ручное действие перед чувствительной операцией, например переводом средств. Веб-сайты также начинают применять в веб-формах однократные значения nonce, чтобы проверять источник формы.

Хотя это не полный перечень всех возможных уязвимостей веб-приложений, он показывает, что веб-приложения подвержены широкому спектру уязвимостей. Он также демонстрирует, что некоторые уязвимости, поражающие локальные приложения, влияют и на веб-приложения, однако многие атаки характерны именно для мира веб-приложений. Чтобы обнаруживать уязвимости веб-приложений с помощью фаззинга, входные данные необходимо передавать по HTTP, а для выявления ошибок нужно применять иные механизмы, чем при фаззинге локальных приложений. Более полный перечень угроз веб-приложениям можно найти в проекте Threat Classification организации Web Application Security Consortium.[38]

Обнаружение

Обнаружение исключений при фаззинге веб-приложений является одной из наиболее серьёзных задач этого класса фаззинга. Будет мало пользы, если мы оставим веб-фаззер работать на ночь, а затем обнаружим, что приложение аварийно завершилось из-за одного из более чем 10 000 полученных им запросов. Мы будем знать о существовании уязвимого состояния, но не сможем его воспроизвести. Поэтому важно анализировать следующие данные, которые можно использовать для выявления потенциально уязвимых состояний:

• **Коды состояния HTTP.** Отвечая на запрос, веб-сервер включает трёхзначный код, обозначающий состояние запроса. Полный список кодов состояния приведён в разделе 10 RFC 2616 «Hypertext Transfer Protocol - HTTP/1.1».[39] Эти коды могут подсказать, какие фаззинговые запросы требуют дальнейшего исследования. Например, последовательность ошибок `500 Internal Server Error` может указывать, что предыдущие фаззинговые запросы вызвали нарушение работы сервера. В свою очередь, ошибка `401 Unauthorized` говорит о том, что запрошенная страница существует, но защищена паролем.

• **Сообщения веб-сервера об ошибках.** Веб-приложение может быть спроектировано так, чтобы показывать сообщение об ошибке непосредственно в содержимом страницы. Такие сообщения можно выявить, разбирая регулярными выражениями HTML из ответа.

• **Разорванные соединения.** Если одно из фаззинговых входных значений заставит веб-сервер зависнуть или аварийно завершиться, последующим запросам, скорее всего, не удастся успешно подключиться к серверу. Поэтому инструменты фаззинга должны вести журналы, фиксирующие разрыв канала или неудавшееся подключение. Если при просмотре журналов обнаружена последовательность неудачных попыток подключения, виновника следует искать среди запросов, непосредственно предшествующих этим записям.

• **Файлы журналов.** Большинство веб-серверов можно настроить на регистрацию различных типов ошибок. И снова эти журналы могут подсказать, какие фаззинговые запросы вызывают проблемы, способные привести к эксплуатируемому состоянию. Трудность состоит в том, что записи журнала нельзя непосредственно связать с вызвавшим их запросом. Один из способов помочь сопоставить запросы с записями об ошибках - синхронизировать часы атакующей и целевой машин и сравнить временные метки запросов и записей журнала, хотя это лишь сузит перечень возможных виновников. Точности такого способа недостаточно, чтобы однозначно связать фаззинговый запрос с вызванной им ошибкой.

• **Журналы событий.** Журналы событий подобны журналам веб-сервера в том смысле, что они не связаны непосредственно с вызвавшими их запросами, но в некоторой степени могут быть соотнесены по временным меткам. Журналы событий используются операционными системами Microsoft Windows; получить к ним доступ можно с помощью приложения «Просмотр событий».

• **Отладчики.** Лучший способ выявлять обработанные и необработанные исключения - перед началом фаззинга подключить целевое приложение к отладчику. Обработка ошибок может скрывать явные признаки многих ошибок, вызванных фаззингом, но обычно их можно обнаружить с помощью отладчика. Важно искать как обработанные, так и необработанные исключения, поскольку ошибки определённых типов, например разыменование нулевого указателя и ошибки форматных строк, могут вызывать обработанные исключения, но при подходящих входных значениях способны превратиться в эксплуатируемые уязвимости. Как всегда, главная трудность состоит в определении того, какой именно запрос вызвал исключение. Однако эффективность отладчиков при фаззинге веб-приложений ограничена: они способны обнаруживать исключения сервера, но не исключения на уровне приложения.

Итоги

В этой главе мы определили фаззинг веб-приложений и веб-серверов и узнали, как с его помощью обнаруживать уязвимости в веб-приложениях. Для этого потребовалось глубоко разобраться в том, как веб-браузеры отправляют запросы и как веб-серверы на них отвечают. Благодаря этим знаниям мы смогли выявить все малозаметные входные данные, которыми конечный пользователь способен управлять и которые он может изменять. Именно эти входные данные можно подвергать фаззингу для обнаружения уязвимостей при надлежащем наблюдении, позволяющем выявить потенциальные исключения. В следующей главе мы создадим фаззер веб-приложений, чтобы автоматизировать процессы, обсуждавшиеся до сих пор.

Сноски

[1] [назад] http://news.com.com/2061-10805_3-6026895.html
[2] [назад] http://en.wikipedia.org/wiki/Common_Gateway_Interface
[3] [назад] http://www.php.net/
[4] [назад] http://en.wikipedia.org/wiki/Adobe_Flash
[5] [назад] http://www.macromedia.com/software/flashremoting/
[6] [назад] http://en.wikipedia.org/wiki/Macromedia
[7] [назад] http://en.wikipedia.org/wiki/Javascript
[8] [назад] http://en.wikipedia.org/wiki/Java_programming_language
[9] [назад] http://en.wikipedia.org/wiki/Microsoft_.Net
[10] [назад] http://www.microsoft.com/resources/documentation/windows/xp/all/proddocs/en-us/iiiisin2.mspx
[11] [назад] http://www.vmware.com
[12] [назад] http://www.microsoft.com/windows/virtualpc/default.mspx
[13] [назад] http://rfc.net/rfc2616.html#s14.23
[14] [назад] http://rfc.net/rfc2616.html
[15] [назад] http://rfc.net/rfc1766.html
[16] [назад] http://rfc.net/rfc1952.html
[17] [назад] http://rfc.net/rfc1951.html
[18] [назад] http://www.microsoft.com/technet/security/Bulletin/MS05-006.mspx
[19] [назад] http://support.microsoft.com/kb/887981
[20] [назад] http://www.cgisecurity.com/whitehat-mirror/WH-WhitePaper_XST_ebook.pdf
[21] [назад] http://www.microsoft.com/technet/security/bulletin/MS03-007.mspx
[22] [назад] http://www.klcconsulting.net/articles/webdav/webdav_vuln.htm
[23] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=360
[24] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=300
[25] [назад] http://www.microsoft.com/technet/security/bulletin/MS99-019.mspx
[26] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=188
[27] [назад] http://www.cirt.net/code/nikto.shtml
[28] [назад] http://rfc.net/rfc1945.html
[29] [назад] http://rfc.net/rfc2616.html
[30] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=371
[31] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=305
[32] [назад] http://wireshark.org/
[33] [назад] http://www.gnu.org/software/wget/
[34] [назад] http://gnuwin32.sourceforge.net/packages/wget.htm
[35] [назад] http://www.owasp.org/software/webscarab.html
[36] [назад] http://cwe.mitre.org/documents/vuln-trends.html#table1
[37] [назад] http://www.packetstormsecurity.org/papers/general/whitepaper_httpresponse.pdf
[38] [назад] http://www.webappsec.org/projects/threat/
[39] [назад] http://rfc.net/rfc2616.html

Глава 10. Фаззинг веб-приложений и серверов: автоматизация

> «Самое важное для нас - найти Усаму бен Ладена. Это наш приоритет номер один, и мы не успокоимся, пока его не найдём».
>
> Джордж У. Буш, Вашингтон, округ Колумбия, 13 сентября 2001 года

> «Я не знаю, где находится бен Ладен. Понятия не имею, и меня это на самом деле не волнует. Это не так важно. Это не наш приоритет».
>
> Джордж У. Буш, Вашингтон, округ Колумбия, 13 марта 2002 года

Теперь, когда мы обсудили, как можно подвергать фаззингу веб-приложения, пришло время проверить наши теории на практике. В этой главе мы применим знания из вводной главы, разработав WebFuzz, графический фаззер веб-приложений. Сначала мы спланируем архитектуру приложения и выявим все особые трудности, с которыми столкнёмся. Затем сможем перейти к выбору подходящей платформы разработки и приступить к созданию фаззера. Но и после завершения разработки наша работа ещё не закончится. Создавая инструмент обнаружения уязвимостей, никогда не следует считать разработку завершённой, пока вы не нашли с его помощью уязвимости. Вы ведь не стали бы строить автомобиль, не испытав его в поездке? Поэтому мы последовательно рассмотрим несколько классов известных уязвимостей веб-приложений, чтобы определить, сможет ли WebFuzz их обнаружить.

Фаззеры веб-приложений

Идея фаззинга веб-приложений не нова. Уже существуют различные фаззеры, и их список продолжает расти. Ниже перечислены некоторые популярные бесплатные и коммерческие фаззеры веб-приложений:

• **SPIKE Proxy.**[1] Разработанный Дейвом Эйтелом (Dave Aitel), SPIKE Proxy представляет собой браузерный веб-фаззер, написанный на Python. Он действует как прокси, перехватывая запросы веб-браузера, а затем позволяет запускать серию заранее определённых проверок целевого веб-сайта в попытке выявить различные уязвимости, например внедрение SQL, переполнения буфера и XSS. Благодаря открытой программной основе SPIKE Proxy можно дополнительно расширять для фаззинга различных целей. SPIKE Proxy не является исключительно фаззером; скорее, это сочетание сканера уязвимостей и фаззера. На рисунке 10.1 показан снимок экрана SPIKE Proxy.
• **WebScarab.**[2] Open Web Application Security Project (OWASP) предоставляет различные инструменты для проверки безопасности веб-приложений, включая WebScarab. Хотя WebScarab является скорее комплексным инструментом проверки безопасности веб-приложений, он содержит базовый фаззер для внедрения фаззинговых значений в параметры приложения.
• **SPI Fuzzer.**[3] SPI Fuzzer является компонентом SPI Toolkit, который, в свою очередь, входит в состав приложения WebInspect. WebInspect - коммерческий инструмент, разработанный SPI Dynamics и предназначенный для предоставления комплексного набора средств проверки веб-приложений.
• **Codenomicon HTTP Test Tools.**[4] Codenomicon выпускает коммерческие наборы фаззинговых тестов практически для любого мыслимого протокола, включая HTTP.
• **beSTORM.**[5] Подобно Codenomicon, компания Beyond Security построила бизнес на разработке коммерческих фаззеров. beSTORM - фаззер, способный работать с различными интернет-протоколами, включая HTTP.

Источником вдохновения для WebFuzz послужил коммерческий инструмент SPI Fuzzer. SPI Fuzzer - простой, но хорошо спроектированный графический фаззер веб-приложений, предоставляющий пользователю полный контроль над необработанными запросами HTTP, которые используются для фаззинга. Для разработки тестов, способных дать результаты, он требует базового знания протокола HTTP. Такие знания нужны и для интерпретации ответов, чтобы выявлять те из них, которые требуют дальнейшего исследования. На рисунке 10.2 показан SPI Fuzzer.

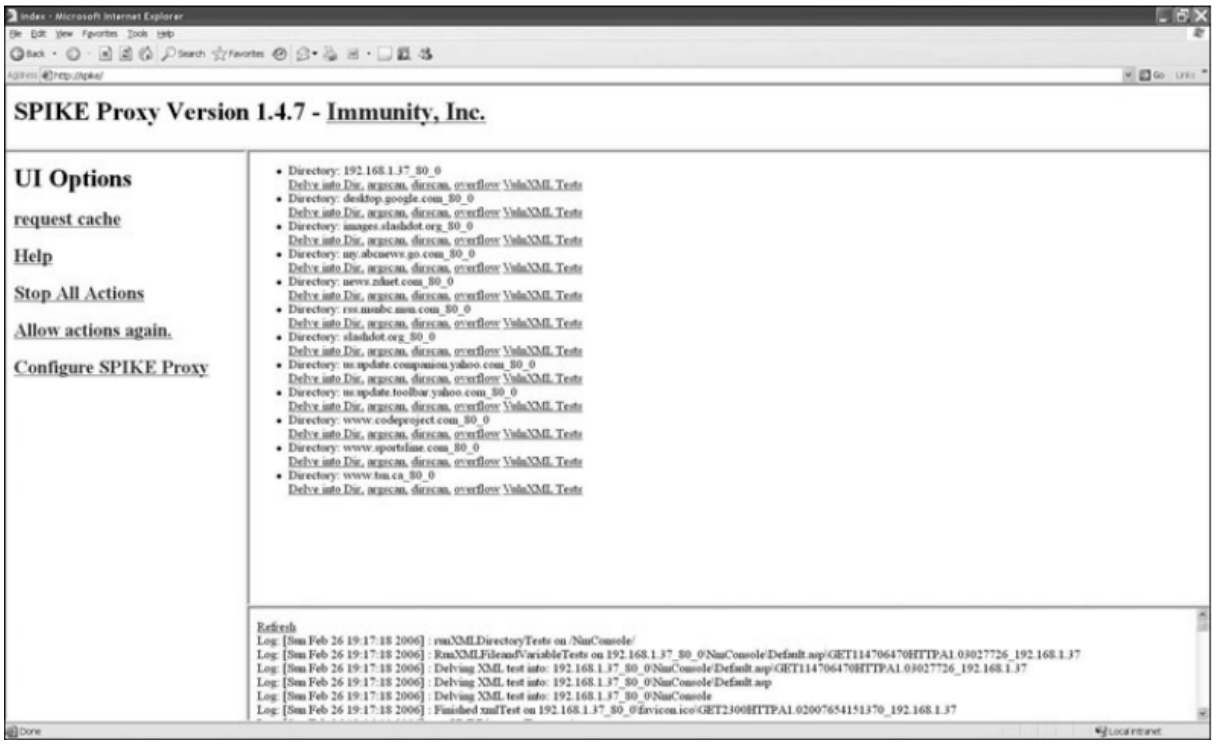


Рисунок 10.1. SPIKE Proxy

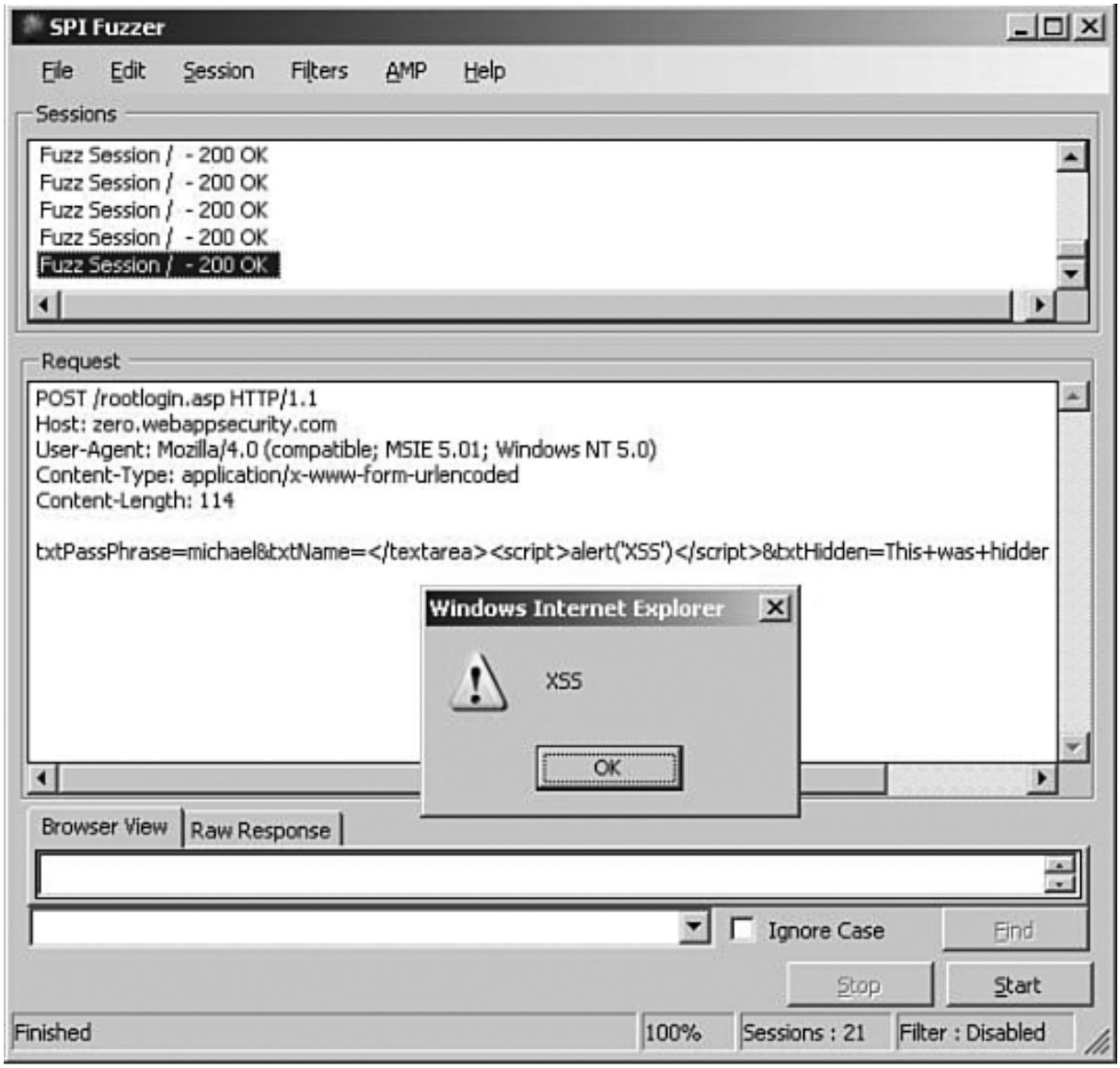


Рисунок 10.2. SPI Fuzzer

Главный недостаток SPI Fuzzer состоит в том, что он доступен лишь как компонент довольно дорогого коммерческого приложения. Опираясь на уроки SPI Fuzzer, мы создали в виде WebFuzz ограниченную, но открытую альтернативу, отвечающую нашим конкретным потребностям. Как и большинство фаззеров, WebFuzz не является инструментом безопасности, в котором достаточно щёлкнуть кнопку, уйти и предоставить ему выполнить за вас всю работу. Это всего лишь средство автоматизации того, что иначе пришлось бы делать вручную. Именно вам, конечному пользователю, предстоит применять его для разработки содержательных тестов и интерпретации результатов. Его также следует рассматривать как исходную точку, а не окончательное решение.

Подобно всем инструментам, разработанным для этой книги, WebFuzz является приложением с открытым исходным кодом. Следовательно, он предоставляет основу, которую можно и следует развивать. Мы призываем вас добавлять возможности и исправлять ошибки, но важнее всего то, что мы надеемся: вы поделитесь своими улучшениями со всем миром. В оставшейся части этой главы подробно описывается разработка WebFuzz и рассматриваются различные практические примеры, демонстрирующие его возможности и ограничения. WebFuzz можно загрузить с веб-сайта книги по адресу www.fuzzing.org.

Возможности

Прежде чем погружаться в создание собственного фаззера, сначала обдумаем то, что узнали в предыдущей главе о работе протокола HTTP, и на основе этих знаний определим возможности, необходимые нашему фаззеру.

Запросы

Начнём с самого начала. Нельзя подвергать веб-приложение фаззингу, если у вас нет способа отправлять ему запросы. Обычно при обмене данными с веб-сервером мы используем веб-браузер. В конце концов, он умеет говорить на нужном языке, HTTP, и берёт на себя все хлопотные подробности составления запроса HTTP. Однако при фаззинге нам нужны именно эти хлопотные подробности. Мы хотим иметь возможность засучить рукава и изменить каждый аспект запроса. Поэтому мы решили предоставить конечному пользователю необработанный запрос и разрешить подвергать фаззингу любую его часть.

На рисунке 10.3 показан базовый запрос WebFuzz. Запрос состоит из следующих полей:

• **Host.** Имя или IP-адрес целевой машины является обязательным полем. Нельзя подвергать веб-приложение фаззингу, не сообщив WebFuzz, куда отправлять запрос. Это поле не подлежит фаззингу.
• **Port.** Хотя по умолчанию веб-приложения работают на TCP-порту 80, с тем же успехом они могут работать на любом TCP-порту. Веб-приложения, предназначенные для предоставления консоли администрирования через веб-интерфейс, нередко работают на другом порту, чтобы не мешать основному веб-серверу. Как и поле имени узла, поле порта сообщает WebFuzz, куда отправлять запрос, и фаззингу не подлежит.
• **Timeout.** Поскольку мы намеренно отправляем нестандартные веб-запросы, целевое приложение нередко будет отвечать несвоевременно или не отвечать вовсе. Поэтому мы предусмотрели задаваемое пользователем значение времени ожидания, измеряемое в миллисекундах. При записи результата для запроса, время ожидания которого истекло, важно зафиксировать этот факт: он может означать, что наш запрос вывел цель из строя, создав потенциальную уязвимость DoS.
• **Request Headers.** Здесь начинается самое интересное. При использовании веб-браузера конечный пользователь управляет целевым узлом, портом и URI запроса, но не всеми разнообразными заголовками. Мы намеренно сохранили все компоненты запроса в одном редактируемом текстовом поле, поскольку хотим дать конечному пользователю возможность управлять каждым аспектом запроса. Запрос можно построить вручную, просто введя желаемый запрос в поле Request Headers. Если же вы предпочитаете решение с графическим интерфейсом, заголовки можно также собрать из списка стандартных заголовков в контекстном меню, как показано на рисунке 10.3. Наконец, если требуется базовый запрос веб-страницы по умолчанию, в контекстном меню можно выбрать Default Headers.

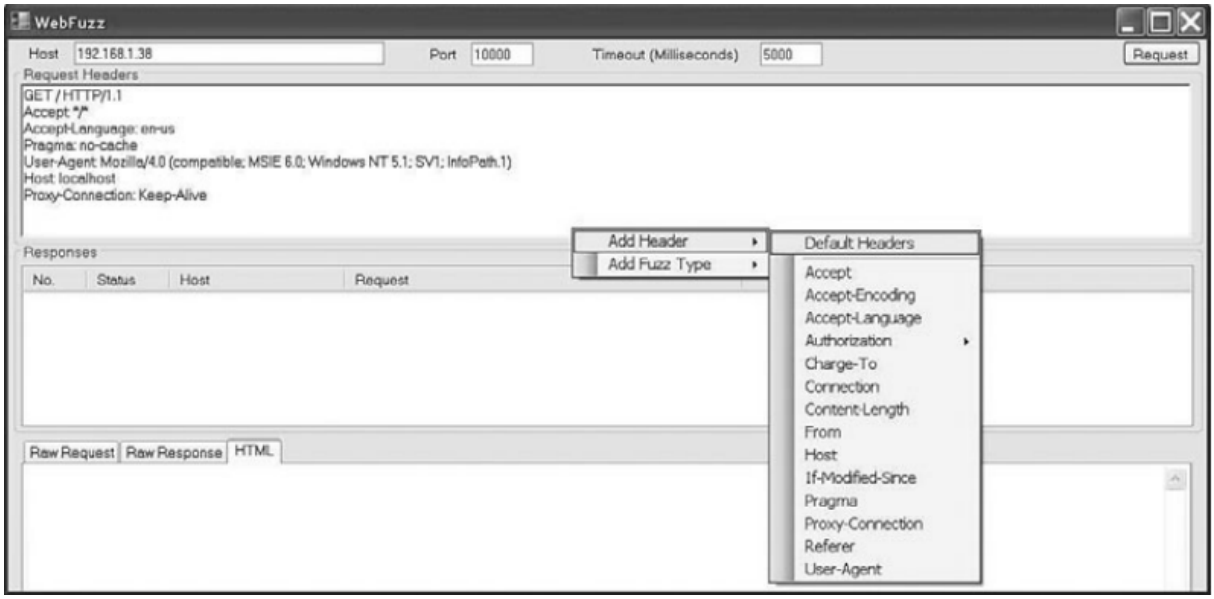


Рисунок 10.3. Запрос WebFuzz

Фаззинговые переменные

Фаззинговыми переменными мы называем участки запроса, которые будут заменяться фаззинговыми данными. Как уже говорилось, пользователь полностью управляет необработанным запросом, отправляемым веб-серверу. Поэтому фаззинговые переменные добавляются непосредственно в необработанный запрос и обозначаются именами переменных в квадратных скобках, например `[Overflow]`. При проектировании функций создания фаззинговых переменных мы решили разделить их на два основных типа: статические списки и генерируемые переменные.

Статические списки представляют собой фаззинговые переменные, получаемые из заранее определённого списка данных. Примером статического списка могут служить фаззинговые данные для выявления уязвимостей XSS. Составляется заранее определённый список различных входных значений, способных привести к уязвимостям XSS, например `<script>alert('XSS')</script>`, а затем данные построчно внедряются в запрос. Статические списки намеренно хранятся во внешних текстовых файлах ASCII, чтобы пользователи могли изменять переменные без повторной компиляции приложения.

Генерируемые фаззинговые переменные, напротив, создаются заранее определёнными алгоритмами, которые могут принимать данные от пользователя. Примером генерируемой фаззинговой переменной служит переменная Overflow. Для переполнения пользователь может задать используемый текст, его длину и число повторений. На рисунке 10.4 показано стандартное всплывающее окно, позволяющее изменять входные данные переменной переполнения.

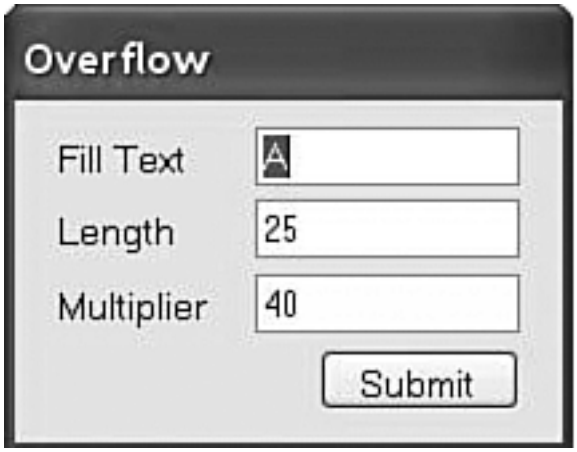


Рисунок 10.4. Фаззинговая переменная Overflow

Для повышения эффективности нам может понадобиться определить несколько фаззинговых переменных в одном запросе. Хотя иногда требуется динамически изменять две или более переменные одновременно, например значение длины вместе с описываемым им содержимым, ради простоты мы решили обрабатывать за один раз только одну переменную. Тем не менее в один запрос можно включить несколько фаззинговых переменных. WebFuzz обработает первую встреченную фаззинговую переменную, а остальные проигнорирует. Когда фаззинг первой переменной будет полностью завершён, она удаляется из необработанного запроса, чтобы WebFuzz мог перейти к последующим переменным. Следующий запрос WebFuzz демонстрирует запрос, предназначенный для выявления сразу нескольких распространённых уязвимостей.

```
[Methods] /file.php?var1=[XSS][SQL]&var2=[Format] HTTP/1.1
Accept: */*
Accept-Language: en-us
User-Agent: Mozilla/4.0
Host: [Overflow]
Proxy-Connection: Keep-Alive
```

Ответы

WebFuzz перехватывает и архивирует все полученные ответы в необработанном формате. Сохранение полного необработанного ответа позволяет гибко отображать ответы в различных форматах. В частности, мы даём пользователю возможность показывать необработанные результаты или просматривать HTML в элементе управления веб-браузера. На рисунке 10.5 показан необработанный ответ, а на рисунке 10.6 - те же данные в веб-браузере. Это важно, поскольку признаки существующих уязвимостей могут проявляться в различных формах. Например, заголовки могут содержать код состояния вроде `500 - Internal Error`, указывающий на возможный отказ в обслуживании. В другом случае сама веб-страница может показать пользовательское сообщение об ошибке, свидетельствующее о возможности внедрения SQL. Такую ошибку будет легче увидеть после интерпретации HTML браузером.

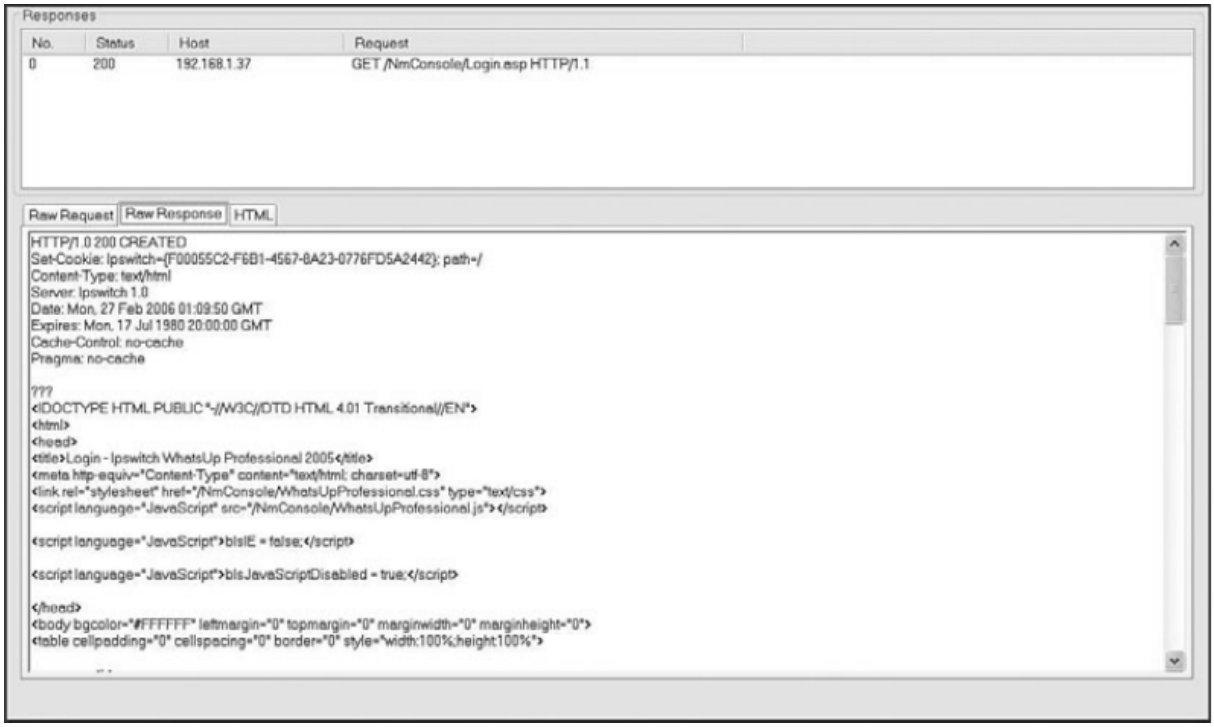


Рисунок 10.5. Необработанный ответ

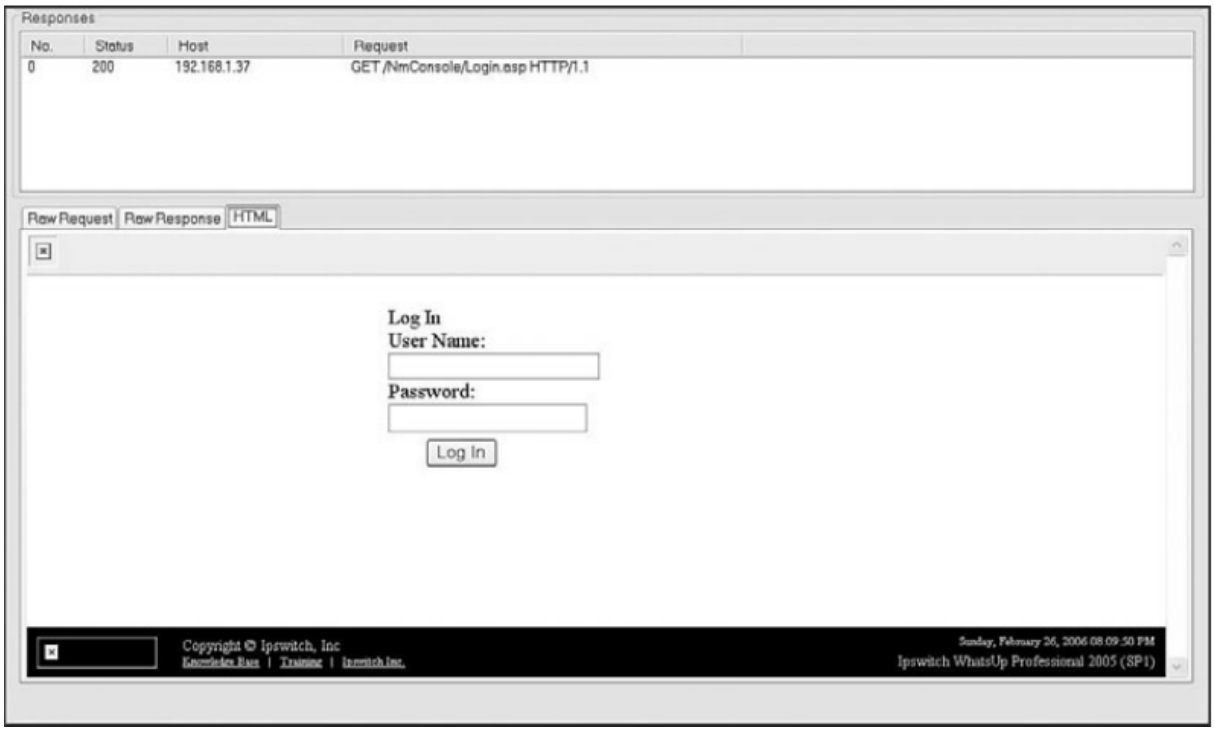


Рисунок 10.6. Ответ в формате HTML

Необходимые исходные сведения

HTTP ставит несколько особых задач как в отношении выбора наилучшего способа наблюдения за трафиком, так и, что ещё важнее, в отношении определения исключений при их возникновении. Теперь мы заглянем за кулисы и выясним, что происходит позади веб-браузера.

Выявление запросов

WebFuzz требует, чтобы пользователь построил необработанный запрос HTTP, но как выявить подходящие запросы для данного веб-приложения? Какие веб-страницы существуют? Какие переменные способны принимать эти страницы и как следует передавать переменные? В предыдущей главе мы обсуждали, как выявлять входные данные вручную или с помощью анализаторов трафика, веб-пауков и прокси. Прежде чем продолжить, мы хотим представить подключаемый модуль веб-браузера, который может пригодиться при работе с WebFuzz, когда требуется выявить необработанный запрос для отдельной веб-страницы. Проект LiveHTTPHeaders[6] предоставляет удобный инструмент для выявления необработанных запросов HTTP в веб-браузерах на основе Mozilla. На рисунке 10.7 показан использование LiveHTTPHeaders для отображения всех запросов и связанных с ними ответов на боковой панели Firefox. Приятное преимущество этого подхода состоит в том, что запрос можно перехватить, а затем вырезать и вставить непосредственно в WebFuzz, ускорив создание фаззингового запроса. Существуют и другие подключаемые модули браузеров, например Tamper Data[7] или Firebug,[8] которые также являются расширениями Firefox, а также Fiddler,[9] надстройка браузера Internet Explorer, однако благодаря простоте нашим любимым средством остаётся LiveHTTPHeaders.

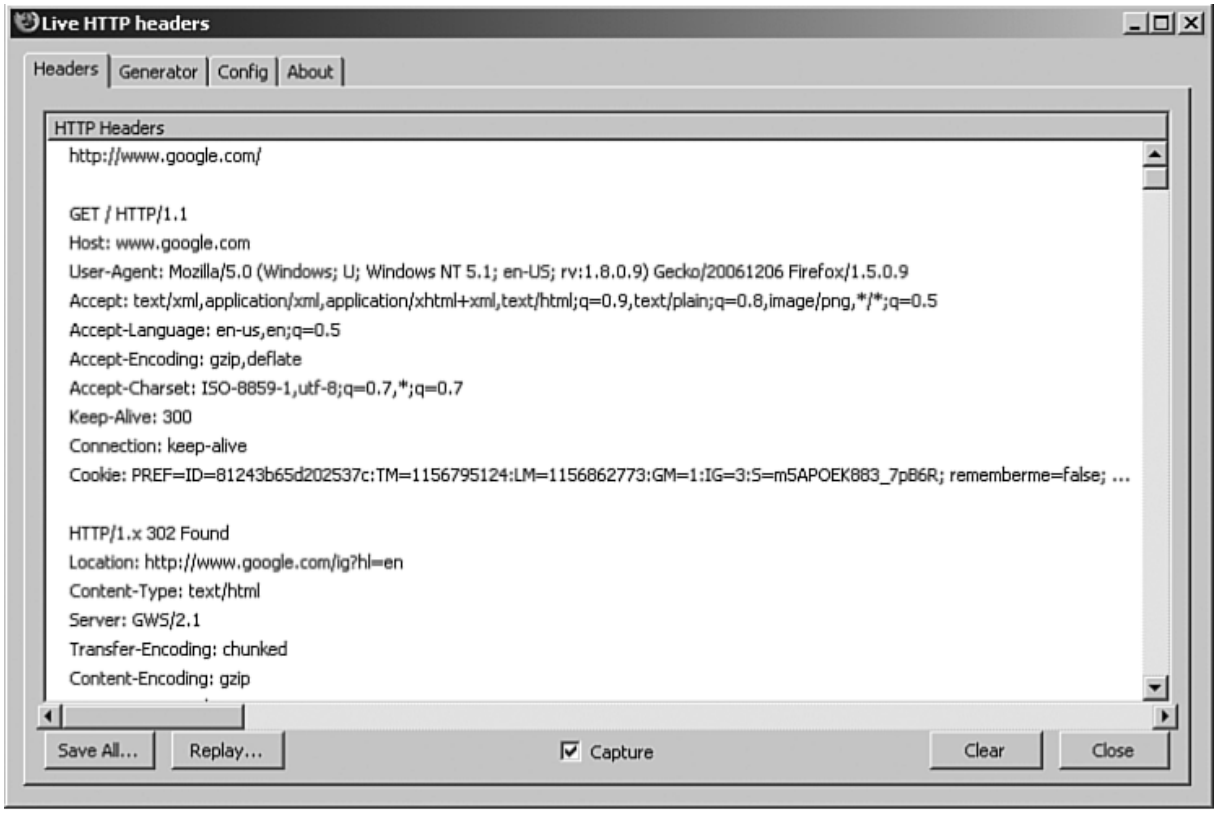


Рисунок 10.7. LiveHTTPHeaders

Обнаружение

Как обсуждалось ранее, ответ целевого приложения может содержать различные признаки воздействия первоначального фаззингового запроса. WebFuzz предназначен для предоставления пользователю данных обнаружения, но интерпретировать ответы должен сам пользователь.

Хотя вручную подробно просматривать все ответы веб-сервера нереалистично, мы надеемся, что отдельные части ответа укажут на возникновение аномального состояния. После этого пользователь сможет выявить связанный с ним запрос. Это возможно потому, что все ответы как в

необработанном формате, так и в формате HTML можно просматривать вместе со связанным запросом, просто выбрав соответствующую вкладку в окне Responses.

При работе WebFuzz на существование уязвимого состояния могут указывать следующие сведения:

• коды состояния HTML;
• сообщения об ошибках, встроенные в ответ;
• пользовательские входные данные, встроенные в ответ;
• снижение производительности;
• истечение времени ожидания запроса;
• сообщения WebFuzz об ошибках;
• обработанные или необработанные исключения.

Рассмотрим каждый из этих видов сведений отдельно, чтобы лучше понять, почему они могут помочь выявить уязвимости.

Коды состояния HTML

Мы уже упоминали, что коды состояния HTML представляют собой жизненно важные сведения, поскольку быстро и наглядно показывают успех или неудачу первоначального запроса. Поэтому WebFuzz разбирает необработанный ответ, чтобы выявить код состояния, который затем отдельно показывается в таблице со сведениями обо всех ответах. Благодаря этому пользователь может быстро выявить ответы, которые следует изучить подробнее.

Сообщения об ошибках, встроенные в ответ

Веб-серверы по своей конструкции могут включать сообщения об ошибках в динамически создаваемые веб-страницы. Особенно часто это происходит, если веб-сервер ошибочно запущен в рабочей среде с включёнными возможностями отладки. Классический пример слишком многословного сообщения об ошибке - сообщение аутентификации вроде «неверный пароль» вместо «неверное имя пользователя или пароль». При подборе данных на экране входа веб-приложения первое сообщение даст понять, что имя пользователя существует, но пароль неверен. Это сокращает число неизвестных переменных с двух, имени пользователя и пароля, до одной, пароля, и значительно повышает вероятность получить доступ. Сообщения приложения об ошибках могут также быть особенно полезны при выявлении атак внедрения SQL.

Пользовательские входные данные, встроенные в ответ

Когда динамически создаваемые веб-страницы содержат предоставленные пользователем данные, возможно существование уязвимостей XSS. Проектировщики веб-приложений должны фильтровать пользовательские данные, чтобы исключить такие атаки, однако неправильная фильтрация является очень распространённой проблемой. Поэтому обнаружение в ответе HTML данных, переданных WebFuzz, служит признаком того, что приложение следует проверить на уязвимости XSS.

Снижение производительности

Хотя полное аварийное завершение приложения позволяет легко заметить атаку DoS, обычно такие уязвимости проявляются менее явно. Нередко о возможной подверженности приложения атакам DoS свидетельствует снижение производительности. Один из способов выявить снижение производительности - истечение времени ожидания запроса, однако во время фаззинга следует также применять средства наблюдения за производительностью, чтобы обнаруживать такие проблемы, как чрезмерное использование процессора или памяти.

Истечение времени ожидания запроса

Как уже упоминалось, истечение времени ожидания запроса нельзя игнорировать, поскольку оно может указывать на временное или постоянное состояние DoS.

Сообщения WebFuzz об ошибках

WebFuzz имеет собственную обработку ошибок и показывает всплывающие сообщения, когда не удаётся выполнить определённые функции. Например, если целевой сервер отключился из-за предыдущего фаззингового запроса, WebFuzz может выдать сообщение о невозможности подключиться к цели. Это может означать, что произошла атака DoS.

Обработанные и необработанные исключения

При фаззинге веб-приложений можно обнаруживать уязвимости как в самом приложении, так и в веб-сервере, на котором оно работает. Поэтому важно также наблюдать за состоянием сервера. Хотя возвращаемые веб-сервером ответы позволяют судить о потенциальных уязвимостях, они не раскрывают полной картины. Вполне вероятно, что фаззинговые запросы вызовут обработанные и необработанные исключения, способные привести к эксплуатируемым состояниям при небольшом изменении входных данных. Поэтому во время фаззинга рекомендуется подключать к целевому веб-серверу отдельный отладчик для выявления таких исключений. Другие обсуждаемые в книге инструменты, например FileFuzz и COMRaider, содержат встроенные средства отладки. Однако при веб-фаззинге это не требуется, поскольку WebFuzz не нужно многократно запускать и завершать приложение. На этот раз мы отправляем серию фаззинговых запросов одному веб-приложению, которое продолжает работать и отвечать на все запросы, за исключением случая, когда входные данные вызывают состояние DoS.

Разработка

Итак, довольно теории; настало время самого интересного. Засучим рукава и создадим фаззер веб-приложений.

Подход

Проектируя WebFuzz, мы стремились создать удобный инструмент для фаззинга веб-приложений. Поскольку предполагается, что наша целевая аудитория достаточно хорошо понимает HTTP, создавать решение, управляемое одними щелчками мыши, не требовалось. Вместо этого мы хотели спроектировать инструмент, предоставляющий конечным пользователям максимальную гибкость в отношении структуры запроса. Кроме того, мы хотели выдавать данные ответов таким образом, чтобы упростить обнаружение потенциальных уязвимостей.

Выбор языка

Чтобы сделать WebFuzz удобным для пользователя, имело смысл создать приложение с графическим интерфейсом. C# был выбран в качестве языка разработки по двум основным причинам. Во-первых, C# позволял с минимальными усилиями спроектировать графический интерфейс достаточно профессионального вида. Во-вторых, C# предоставляет несколько классов, помогающих отправлять и принимать сетевой трафик. Недостаток использования C# состоит в том, что мы главным образом привязаны к платформе Windows. Однако при фаззинге веб-приложений наши цели не обязательно работают на той же машине, что и фаззер. Поэтому создание инструмента для Windows не ограничивает нас фаззингом только целей на основе Windows.

Архитектура

Невозможно разобрать каждую часть кода, не усыпив читателя. Однако основная функциональность нашего фаззера сосредоточена в нескольких базовых классах, и сейчас мы выделим ключевые возможности каждого из них. Помните, что полный исходный код всех представленных в книге приложений доступен на веб-сайте книги.

Класс TcpClient

C# предоставляет класс `WebClient`, включающий функции обработки запросов и ответов HTTP. Он инкапсулирует значительную часть кода, необходимого для создания и обработки требуемого сетевого трафика, и способен заметно ускорить разработку приложений. Он даже содержит функции для значительной части возможностей, необходимых WebFuzz, например для доступа к заголовкам в ответе HTTP. На несколько более низком уровне C# предоставляет классы `HttpWebRequest` и `HttpWebResponse`. Эти классы требуют немного больше усилий при написании кода, но зато открывают более развитые возможности, например использование прокси.

Какой же из этих удобных классов мы использовали в WebFuzz? Ответ: ни один. Вместо них мы выбрали класс `TcpClient`, предназначенный для TCP-трафика любого типа, а не только HTTP. Поэтому в нём отсутствуют готовые возможности других веб-классов. Зачем нам было так поступать? Неужели мы садисты и любим писать ненужные строки кода? Нет, это было всего лишь необходимым злом.

Одна из главных трудностей при написании фаззеров состоит в том, что вы пытаетесь делать вещи не так, как их положено делать. Поэтому стандартные классы и функции могут не отвечать вашим потребностям. Для наших целей необходим полный контроль над необработанным запросом HTTP, а различные веб-классы, к сожалению, не предоставляют столь детального управления. Рассмотрим, например, следующий код:

```
WebClient wclFuzz = new WebClient();
```

```
wclFuzz.Headers.Add("blah", "blah");
Stream data = wclFuzz.OpenRead("http://www.fuzzing.org");
StreamReader reader = new StreamReader(data);
data.Close();
reader.Close();
```

Этого простого примера кода вполне достаточно, чтобы отправить пользовательский веб-запрос с помощью класса `WebClient`. Мы создали базовый запрос `GET` и добавили лишь один пользовательский заголовок (`blah: blah`). Однако при перехвате фактически созданного трафика мы обнаруживаем, что был отправлен следующий запрос:

```
GET / HTTP/1.1
blah: blah
Host: www.fuzzing.org
Connection: Keep-Alive
```

Обратите внимание: в фактический запрос были добавлены два дополнительных заголовка, `Host` и `Connection`. Именно по этой причине мы не можем использовать классы, которые обычно были бы предпочтительны. Нам приходится пожертвовать простотой, перейти на более низкий уровень и получить полный контроль над процессом. В нашем случае для сетевого компонента WebFuzz мы будем использовать класс `TcpClient`.

Асинхронные сокеты

Сетевой обмен можно выполнять с помощью асинхронных или синхронных сокетов. Хотя асинхронные сокеты требуют немного дополнительной работы, их использование в WebFuzz было осознанным решением: этот выбор лучше учитывает ожидаемые сетевые проблемы, которые, скорее всего, возникнут при применении фаззера.

Синхронные сокеты являются блокирующими. Это означает, что при обработке запроса или ответа основной поток остановится и будет ждать завершения обмена, прежде чем продолжить работу. С помощью фаззера мы намеренно пытаемся вызвать аномальные состояния, некоторые из которых способны снизить производительность или полностью вывести целевое приложение из строя. Нам бы не хотелось, чтобы WebFuzz перестал отвечать, ожидая обмена, который может никогда не произойти. Асинхронные сокеты позволяют избежать этой проблемы, поскольку не являются блокирующими. Асинхронные сокеты запускают отдельный поток для обработки обмена; после его завершения этот поток вызывает функцию обратного вызова. Благодаря этому другие события могут продолжать выполняться без перерыва.

Разберём сетевой код WebFuzz, чтобы лучше понять идею асинхронных сокетов:

```
TcpClient client;
NetworkStream stream;
ClientState cs;
try
{
    client = new TcpClient();
    client.Connect(reqHost, Convert.ToInt32(tbxPort.Text));
    stream = client.GetStream();
    cs = new ClientState(stream, reqBytes);
}
catch (SocketException ex)
{
    MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    return;
}
catch (System.IO.IOException ex)
{
    MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
```

```
    return;
}
IAsyncResult result = stream.BeginWrite(cs.ByteBuffer, 0,
    cs.ByteBuffer.Length, new AsyncCallback(OnWriteComplete), cs);
result.AsyncWaitHandle.WaitOne();
```

После создания обычных объектов `TcpClient` и `NetworkStream` мы вызываем метод `BeginWrite()` потока. Метод `BeginWrite()` принимает следующие пять аргументов:[10]

• `byte[] array` - буфер, содержащий данные для записи в сетевой поток;
• `int offset` - позиция в буфере, с которой следует начать отправку данных;
• `int numBytes` - максимальное число записываемых байтов;
• `AsyncCallback userCallback` - метод обратного вызова, который будет вызван после завершения обмена;
• `object stateObject` - объект, позволяющий отличать данный запрос асинхронной записи от других подобных запросов.

Вызов `AsyncWaitHandle.WaitOne()` приводит к блокировке прослушивающего потока до успешной отправки запроса. В этот момент функция обратного вызова будет вызвана следующим образом:

```
public static void OnWriteComplete(IAsyncResult ar)
{
    try
    {
        ClientState cs = (ClientState)ar.AsyncState;
        cs.NetStream.EndWrite(ar);
    }
    catch (System.ObjectDisposedException ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

Завершив запись запроса в сетевой поток, мы можем получить результат от сервера:

```
try
{
    result = stream.BeginRead(cs.ByteBuffer, cs.TotalBytes,
        cs.ByteBuffer.Length - cs.TotalBytes,
        new AsyncCallback(OnReadComplete), cs);
}
catch (System.IO.IOException ex)
{

    MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    ReadDone.Close();
    return;
}
```

На этом этапе мы снова используем асинхронный сокет, но на сей раз он служит для получения ответа целевого приложения. Теперь мы вызываем метод `BeginRead()`, который принимает те же аргументы, что и метод `BeginWrite()`, однако в качестве метода обратного вызова на этот раз используется `OnReadComplete()`:

```
public void OnReadComplete(IAsyncResult ar)
{
    readTimeout.Elapsed += new ElapsedEventHandler(OnTimedEvent);
    readTimeout.Interval = Convert.ToInt32(tbxTimeout.Text);
    readTimeout.Enabled = true;
    ClientState cs = (ClientState)ar.AsyncState;
    int bytesRcvd;
    try
    {
        bytesRcvd = cs.NetStream.EndRead(ar);
    }
    catch (System.IO.IOException ex)
```

```
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        return;
    }
    catch (System.ObjectDisposedException ex)
    {
        return;
    }
    cs.AppendResponse(Encoding.ASCII.GetString(cs.ByteBuffer,
        cs.TotalBytes, bytesRcvd));
    cs.AddToTotalBytes(bytesRcvd);
    if (bytesRcvd != 0)
    {
        cs.NetStream.BeginRead(cs.ByteBuffer, cs.TotalBytes,
            cs.ByteBuffer.Length - cs.TotalBytes,
            new AsyncCallback(OnReadComplete), cs);
    }

    else
    {
        readTimeout.Enabled = false;
        if (ReadDone.Set() == false)
            ReadDone.Set();
    }
}
```

В начале `OnReadComplete()` мы создаём таймер (`readTimeout`), который вызовет `ReadDone.Set()`, если истечёт заданное пользователем время ожидания. Это позволяет не допустить бесконечного существования потока, если операция чтения не завершится, и предоставляет конечному пользователю возможность управлять длительностью ожидания. Затем мы добавляем полученный ответ в буфер. После этого необходимо решить, следует ли продолжать ожидание дальнейших данных. Для этого мы определяем, было ли получено больше нуля байтов. Если да, мы повторяем всё сначала, снова вызывая `BeginRead()`. В противном случае завершаем поток и продолжаем работу.

Создание запросов

Прежде чем отправлять запрос, необходимо сначала определить, что именно отправлять. Очевидно, это содержимое берётся из окна Request Headers, где пользователь создаёт запрос, однако каждую фаззинговую переменную `[XXX]` нужно заменить фактическими фаззинговыми данными. Процесс запускается, когда пользователь нажимает кнопку Request, внутри метода `btnRequest_Click()`:

```
if (rawRequest.Contains("[") != true || rawRequest.Contains("]") != true)
    rawRequest = "[None]" + rawRequest;
while (rawRequest.Contains("[") && rawRequest.Contains("]"))
{
    fuzz = rawRequest.Substring(rawRequest.IndexOf('[') + 1,
        (rawRequest.IndexOf(']') - rawRequest.IndexOf('[')) - 1);
```

При создании запросов мы начинаем цикл, который продолжает разбирать предоставленные пользователем данные до тех пор, пока в запросе встречаются фаззинговые переменные. Затем мы переходим к конструкции `switch`, определяющей действия для каждой фаззинговой переменной.

```
int arrayCount = 0;
int arrayEnd = 0;
Read fuzzText = null;
WebFuzz.Generate fuzzGenerate = null;
ArrayList fuzzArray = null;
string replaceString = "";
```

```
string[] fuzzVariables = { "SQL", "XSS", "Methods", "Overflow", "Traversal", "Format" };
switch (fuzz)
{
    case "SQL":
        fuzzText = new Read("sqlinjection.txt");
        fuzzArray = fuzzText.readFile();
        arrayEnd = fuzzArray.Count;
        replaceString = "[SQL]";
        break;
    case "XSS":
        fuzzText = new Read("xssinjection.txt");
        fuzzArray = fuzzText.readFile();
        arrayEnd = fuzzArray.Count;
        replaceString = "[XSS]";
        break;
    case "Methods":
        fuzzText = new Read("methods.txt");
        fuzzArray = fuzzText.readFile();
        arrayEnd = fuzzArray.Count;
        replaceString = "[Methods]";
        break;
    case "Overflow":
        fuzzGenerate = new WebFuzz.Overflow(overflowFill, overflowLength,
            overflowMultiplier);
        fuzzArray = fuzzGenerate.buildArray();
        arrayEnd = fuzzArray.Count;
        replaceString = "[Overflow]";
        break;
    case "Traversal":
        fuzzGenerate = new WebFuzz.Overflow("../", 1, 10);
        fuzzArray = fuzzGenerate.buildArray();
        arrayEnd = fuzzArray.Count;
        replaceString = "[Traversal]";
        break;
    case "Format":
        fuzzGenerate = new WebFuzz.Overflow("%n", 1, 10);
        fuzzArray = fuzzGenerate.buildArray();
        arrayEnd = fuzzArray.Count;
        replaceString = "[Format]";
        break;
    case "None":
        ArrayList nullValueArrayList = new ArrayList();
        nullValueArrayList.Add("");
        fuzzArray = nullValueArrayList;

        arrayEnd = fuzzArray.Count;
        replaceString = "[None]";
        break;
    default:
        arrayEnd = 1;
        break;
}
```

Фаззинговые переменные, поступающие из статического списка (`SQL`, `XSS` и `Methods`), создают новый объект класса `Read()` и передают конструктору имя текстового файла ASCII, содержащего фаззинговые переменные. Генерируемые переменные (`Overflow`, `Traversal` и `Format`), напротив, создают новый объект класса `Generate()` и передают строку для повторения, общий размер строки и число её повторений.

Получение ответов

По мере получения ответов WebFuzz архивирует запрос, необработанный ответ, ответ HTML, имя узла и путь, добавляя их в отдельные массивы строк. Кроме того, идентификационные сведения, включая код состояния, имя узла и запрос, добавляются в элемент управления `ListView`. Благодаря этому после завершения фаззинга можно просто щёлкнуть соответствующий ответ в элементе `ListView`, и полные сведения отобразятся в последовательности вкладок с элементами управления `RichTextBox` и `WebBrowser`.

```
rtbRequestRaw.Text = reqString;
rtbResponseRaw.Text = dataReceived;
wbrResponse.DocumentText = html;
string path = getPath(reqString);
lvwResponses.Items.Add(lvwResponses.Items.Count.ToString());
lvwResponses.Items[lvwResponses.Items.Count - 1].SubItems.Add(status);
lvwResponses.Items[lvwResponses.Items.Count - 1].SubItems.Add(reqHost);
lvwResponses.Items[lvwResponses.Items.Count - 1].SubItems.Add
    (requestString.Substring(0, requestString.IndexOf("\r\n")));
lvwResponses.Refresh();
requestsRaw[lvwResponses.Items.Count - 1] = reqString;
responsesRaw[lvwResponses.Items.Count - 1] = dataReceived;
responsesHtml[lvwResponses.Items.Count - 1] = html;
responsesHost[lvwResponses.Items.Count - 1] = reqHost;
responsesPath[lvwResponses.Items.Count - 1] = path;
```

Как уже упоминалось, WebFuzz не является управляемым одними щелчками мыши сканером уязвимостей. Это инструмент, предназначенный для того, чтобы знающий специалист мог подвергать фаззингу целевые части запроса HTTP. Полный исходный код и двоичные файлы инструмента доступны на www.fuzzing.org.

Практические примеры

Теперь, когда мы в общих чертах понимаем, как и почему был создан WebFuzz, пришло время перейти к более важным вещам. Пора выяснить, работает ли он на самом деле.

Обход каталогов

Уязвимость обхода каталогов существует, когда пользователь способен выйти за пределы корневого веб-каталога и получить доступ к файлам и папкам, не предназначенным для передачи через веб-приложение. Уязвимость этого типа создаёт риск для конфиденциальности, но в зависимости от файлов, к которым можно получить доступ, способна также перерасти в полную компрометацию системы. Рассмотрим, например, ситуацию, когда обход каталогов позволяет злоумышленнику получить файл паролей. Даже если файл зашифрован, после его получения может появиться возможность взломать пароли автономно, чтобы позднее злоумышленник вернулся и подключился к серверу с допустимыми учётными данными аутентификации.

Обычно обход выполняется отправкой последовательности символов `../`, позволяющей перейти в каталог более высокого уровня. Символы обхода также часто кодируют в URL, чтобы обойти базовые фильтры обнаружения. Если просмотр каталогов разрешён, этого может оказаться достаточно, однако обычно требуется также добавить имя существующего файла. При проверке атаки обхода каталогов рекомендуется добавлять имя файла, который по умолчанию существует на целевом сервере. Например, в системе Windows удачным выбором будут `boot.ini` или `win.ini`, поскольку это легко узнаваемые при обнаружении файлы ASCII, существующие во всех современных операционных системах Windows.

Начнём с простой атаки обхода каталогов в Trend Micro Control Manager,[11] вызванной неправильной проверкой входных данных в параметре `IMAGE` страницы `rptserver.asp`. Мы воспользуемся WebFuzz для отправки запроса `GET` странице `rptserver.asp`, но заменим допустимый параметр `IMAGE` фаззинговой переменной `[Traversal]`, за которой следует известный файл, в данном случае `win.ini`. По результатам на рисунке 10.8 видно, что обход каталогов действительно раскрывается уже после одной последовательности обхода.

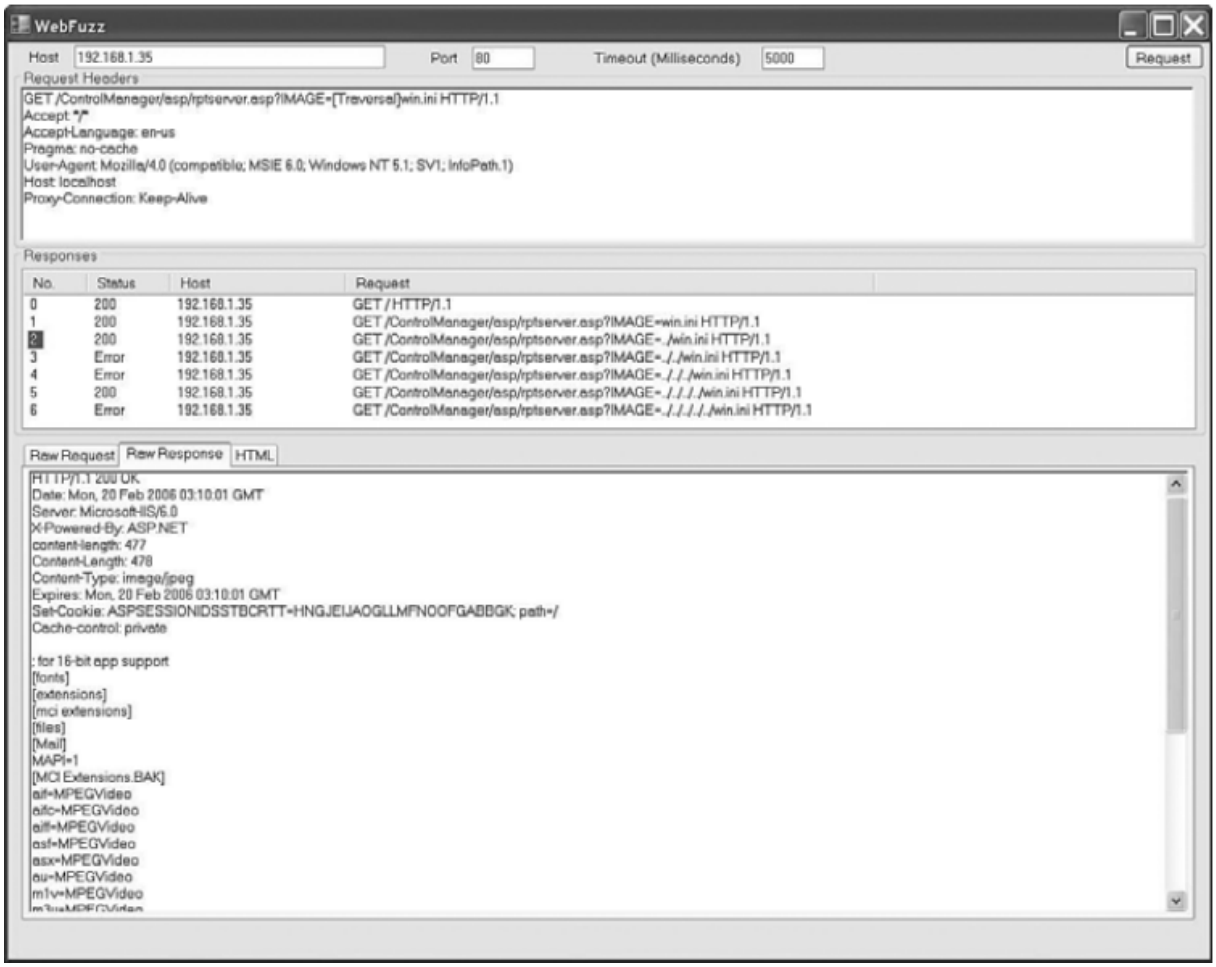


Рисунок 10.8. Обход каталогов в Trend Micro Control Manager

Попробуем другой пример, но на этот раз немного усложним задачу. Уязвимость в Ipswitch Imail Web Calendaring[12] показала нам, что иногда для обнаружения обхода каталогов необходимо также запросить то, чего не существует. В данном случае было обнаружено, что уязвимость обхода каталогов возникает при передаче последовательности обхода несуществующей странице JSP. И снова испытаем WebFuzz (см. рисунок 10.9).

Чтобы проверить эту конкретную уязвимость, мы передадим серверу метод `GET`, запрашивающий несуществующую веб-страницу `blah.jsp`, затем последовательность обхода и, наконец, распространённый файл `boot.ini`. Рисунок 10.9 показывает, что требуется несколько запросов, но после пяти последовательностей обхода... как вы уже догадались, уязвимость раскрывается.

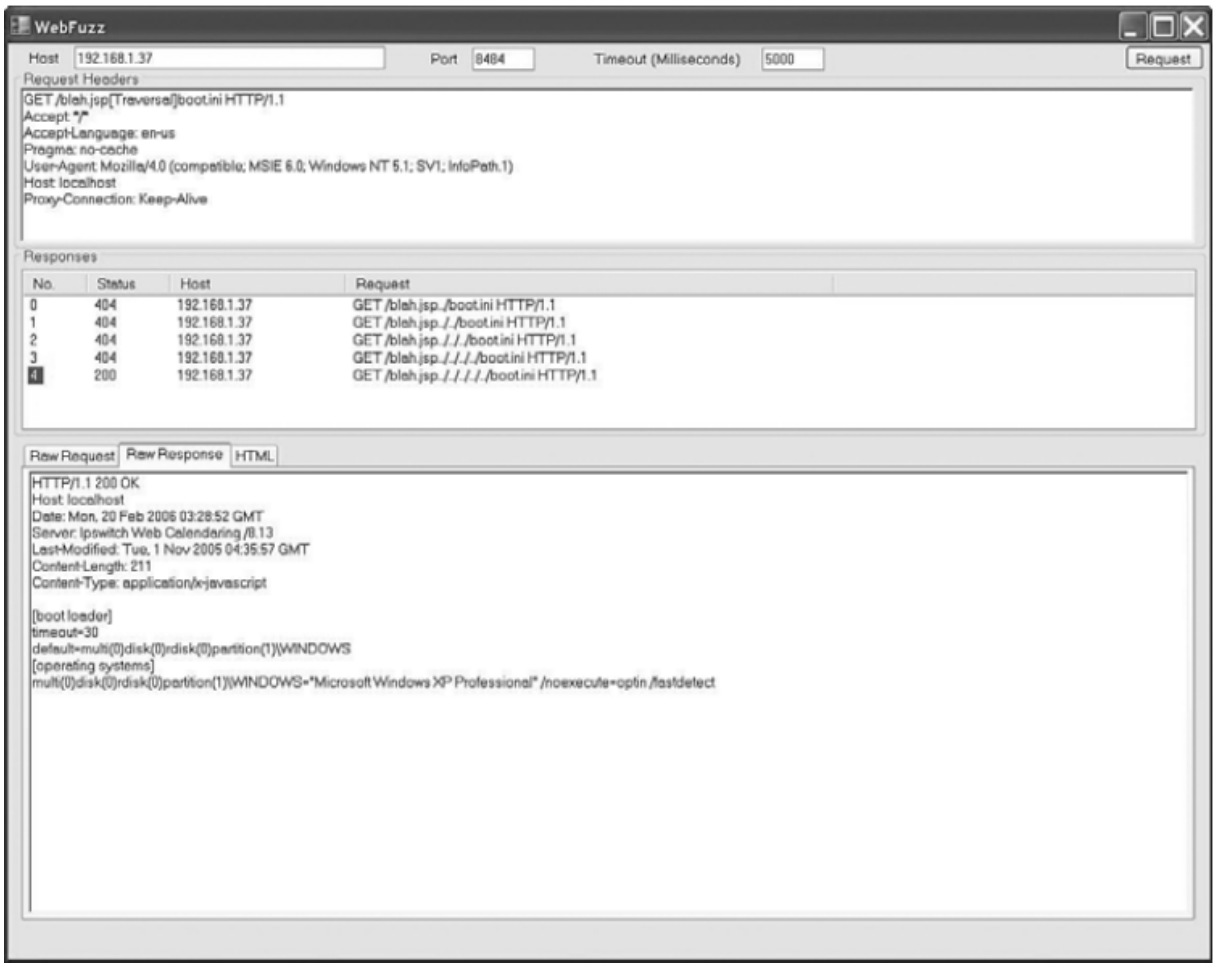


Рисунок 10.9. Обход каталогов в Ipswitch Imail Web Calendaring

Переполнение

Хотя в мире веб-приложений переполнения буфера относительно редки, они могут существовать в веб-приложениях и серверах так же, как в консольных приложениях и приложениях с графическим интерфейсом, и возникают по той же причине: предоставленные пользователем входные данные не фильтруются надлежащим образом, поэтому данные неожиданной длины переполняют буфер фиксированного размера. Переполнения особенно опасны, поскольку способны привести к выполнению произвольного кода. Однако при фаззинге наиболее вероятным признаком произошедшего переполнения будет аварийное завершение приложения после запроса, содержащего чрезмерно длинную строку. Подключив к цели отладчик во время фаззинга, можно определить, ограничивается ли переполнение атакой DoS или его удастся расширить до удалённого выполнения кода.

Чтобы показать применение WebFuzz для обнаружения уязвимостей переполнения, воспользуемся простым примером. Что может быть проще, чем переполнение в Simple Web Server компании PMSoftware?[13] В данном случае достаточно отправить серверу длинный запрос `GET`. Поэтому мы отправим следующий запрос, включающий фаззинговую переменную переполнения:

```
GET /[Overflow] HTTP/1.1
```

Сначала не происходит ничего, кроме ошибки `404 - Page Not Found`, но, как видно на рисунке 10.10, после запроса номер 10 WebFuzz больше не получает никаких ответов. Почему?

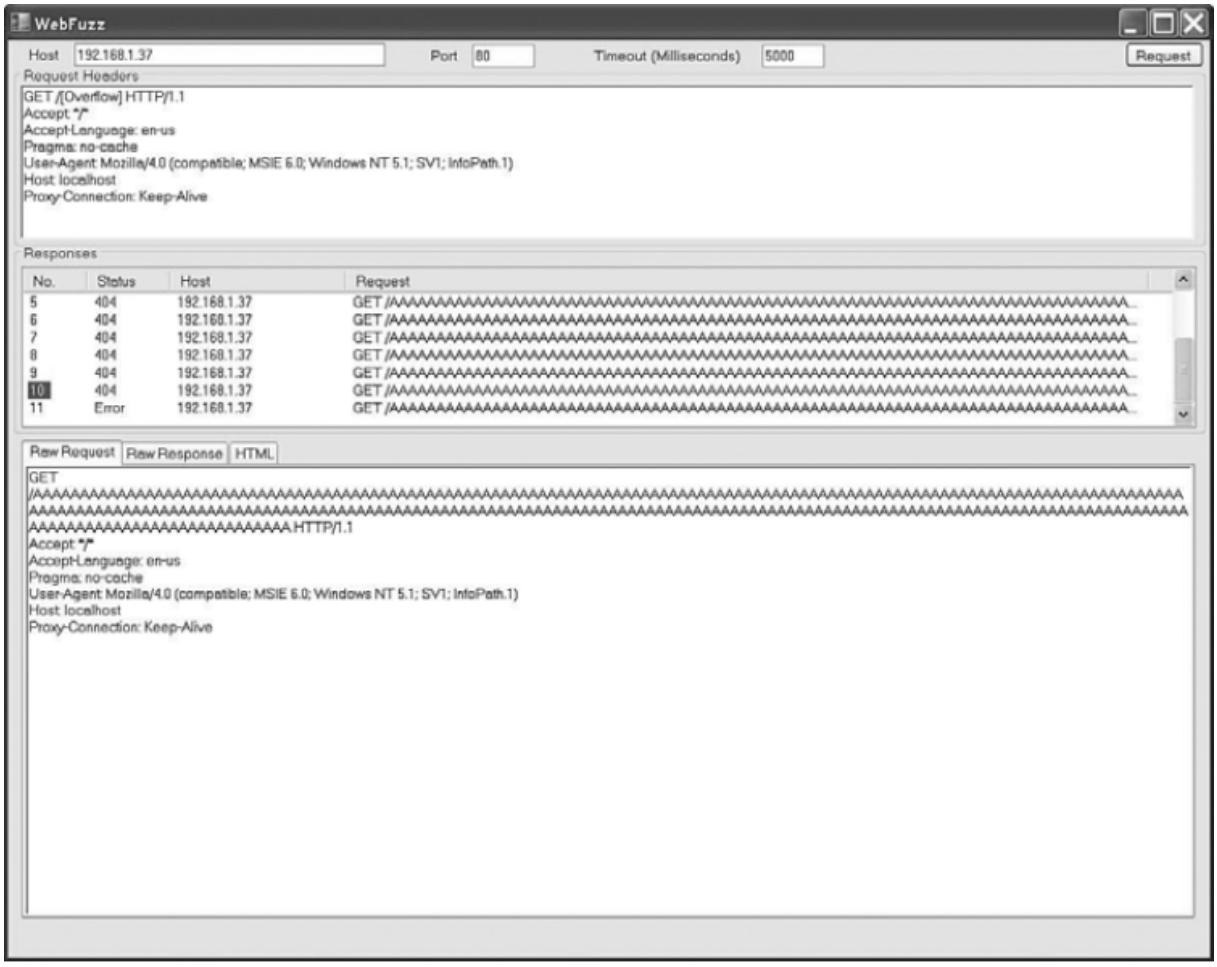


Рисунок 10.10. Переполнение буфера в Simple Web Server

Ответ вскоре становится очевиден, когда мы смотрим на сервер, где работал Simple Web Server: там появляется всплывающее сообщение, показанное на рисунке 10.11.

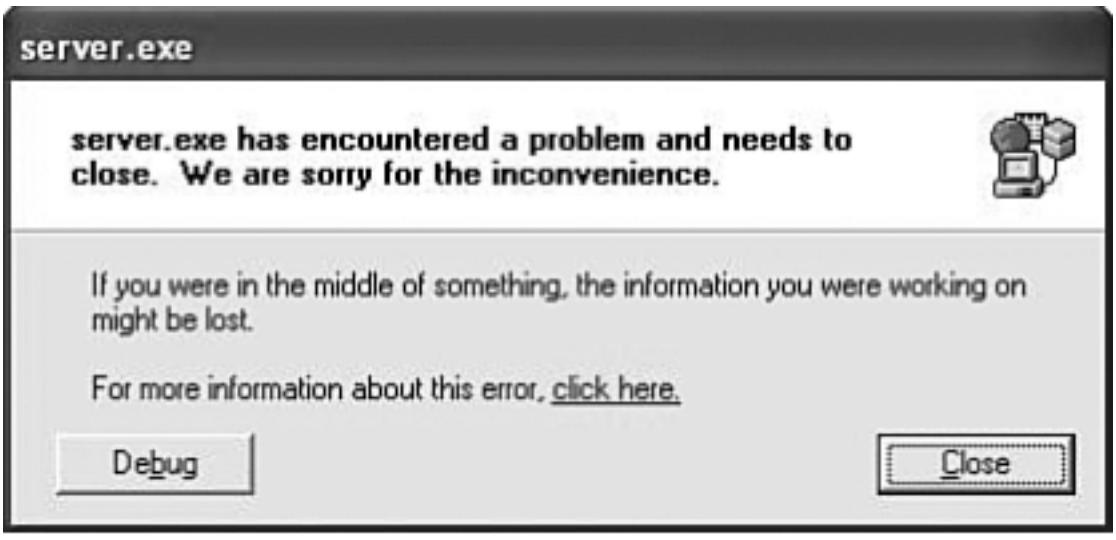


Рисунок 10.11. Сообщение Simple Web Server об ошибке

Это недобрый знак, по крайней мере для Simple Web Server. Вместе с закрытием сообщения об ошибке закрывается и приложение. Да, по меньшей мере мы получили атаку DoS. А есть ли также возможность выполнения кода? Ответ содержится на рисунке 10.12, где подключённый отладчик показывает, что регистр EIP находится под нашим управлением, а значит, выполнение кода будет

возможно.

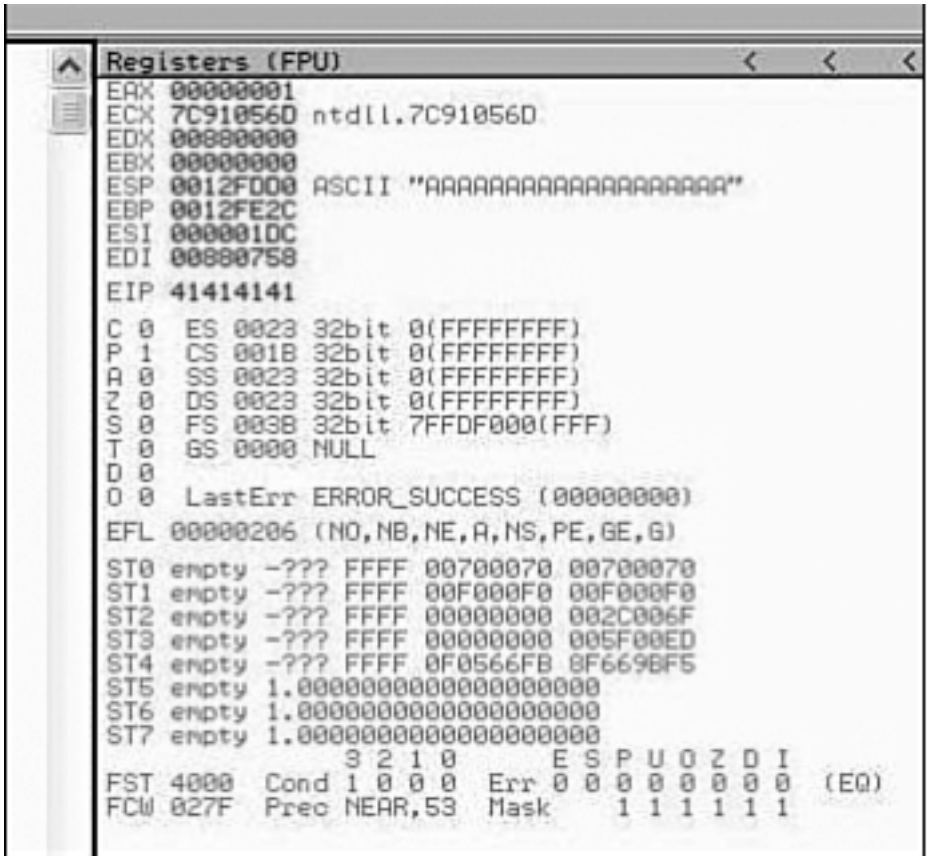


Рисунок 10.12. Переполнение в Simple Web Server

Если вы только знакомитесь с переполнениями буфера, не слишком воодушевляйтесь. Это наиболее благоприятный сценарий. Плод редко висит настолько низко.

Внедрение SQL

Атаки внедрения SQL происходят, когда предоставленные пользователем данные способны влиять на запросы SQL, поступающие внутренней реляционной базе данных. И снова виновником оказываются неправильно отфильтрованные пользовательские данные. При фаззинге сообщения приложения об ошибках служат веским признаком того, что внедрение SQL может быть возможно.

Атака внедрения SQL в поле имени пользователя на экране входа Ipswitch Whatsup Professional (SP1)[14] позволяла злоумышленникам полностью обходить защиту, изменяя пароль администратора. Но как её обнаружить? С помощью LiveHTTPHeaders легко увидеть, что параметры имени пользователя и пароля передаются странице `Login.asp` в запросе `POST`, как показано ниже:

```
POST /NmConsole/Login.asp HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.8.0.1) Gecko/20060111
Firefox/1.5.0.1
Accept:
text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/
png,*/*;q=0.5
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

```
Referer: http://localhost/NmConsole/Login.asp
Cookie: Ipswitch={A481461B-2EC6-40AE-B362-46B31959F6D1}
Content-Type: application/x-www-form-urlencoded
Content-Length: 81

bIsJavaScriptDisabled=false&sUserName=xxx&sPassword=yyy&btnLogIn=Log+In
```

Теперь, когда нам известен правильный формат запроса, можно составить следующий фаззинговый запрос:

```
POST /NmConsole/Login.asp HTTP/1.1
Host: localhost

bIsJavaScriptDisabled=false&sUserName=[SQL]&sPassword=&btnLogIn=Log+In
```

Стандартное сообщение об ошибке при неудавшейся попытке входа выглядит так: `There was an error while attempting to login: Invalid user name`. Однако при запуске WebFuzz мы обнаруживаем, что в некоторых ответах показываются другие сообщения об ошибках, например приведённое на рисунке 10.13. Из содержимого сообщения об ошибке ясно видно, что предоставленные пользователем данные, по-видимому, отправляются в базу данных.

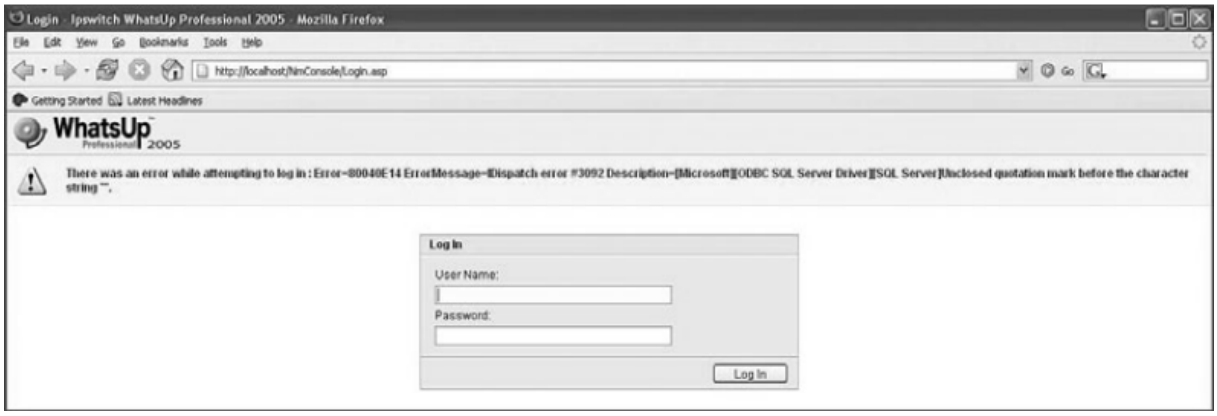


Рисунок 10.13. Сообщение Ipswitch Whatsup Professional (SP1) об ошибке

Однако это не простая атака внедрения SQL. Распространённые приёмы обхода аутентификации, например `' or 1=1`,[15] здесь не работают, поэтому нам придётся составить допустимый запрос `UPDATE`, чтобы изменить пароль администратора. На этом этапе для целевых атак внедрения SQL, специфичных для данной цели, нужны сведения о схеме базы данных, но где их получить? Быстрый поиск в Google даёт ответ. Во-первых, сама Ipswitch предоставляет схему для загрузки.[16]

Это замечательно, но составление содержательного запроса всё равно требует некоторых усилий, а мы заняты созданием фаззеров. Не беспокойтесь: Ipswitch любезно предоставляет искомый запрос. База знаний Ipswitch[17] фактически содержит следующую команду, сбрасывающую пароль пользователя Admin к значению по умолчанию:

```
osql -E -D WhatsUp -Q "UPDATE WebUser SET sPassword=DEFAULT WHERE sUserName='Admin'"
```

Ipswitch предоставила этот запрос для применения в инструменте командной строки, чтобы помочь администраторам, которые случайно лишили себя доступа к приложению, забыв пароль. Они не осознавали, что тот же запрос столь же хорошо работает и в нашей атаке внедрения SQL. Похоже, раздел вопросов и ответов базы знаний должен был выглядеть следующим образом:

> **Вопрос/проблема:** Я забыл пароль пользователя-администратора по умолчанию в веб-интерфейсе. Как его сбросить?
>
> **Ответ/решение:** Найдите уязвимость внедрения SQL и выполните следующую команду...

Межсайтовый скриптинг

XSS повсюду. По данным Mitre, в 2006 году XSS были связаны 21,5 процента всех новых уязвимостей,[18] и долго искать их не приходится. Более того, sla.ckers.org ведёт на своих форумах постоянно растущую «стену позора XSS»,[19] и огорчительно видеть, сколько крупных корпораций появляется в этом списке.

Как и большинство уязвимостей веб-приложений, XSS становится возможен из-за неправильной проверки входных данных. Уязвимое приложение принимает пользовательские данные и без всякой фильтрации встраивает их в динамическое содержимое страницы. Поэтому злоумышленник может внедрить на запрошенную страницу клиентский сценарий, например JavaScript. Это, в свою очередь, способно позволить злоумышленнику управлять содержимым отображаемой веб-страницы или выполнять действия от имени жертвы.

Для рассмотрения примера фаззинга XSS начнём с заведомо уязвимой веб-страницы. SPI Dynamics размещает по адресу http://zero.webappsecurity.com уязвимое веб-приложение, показанное на рисунке 10.14, для проверки WebInspect, своего сканера веб-приложений.



Рисунок 10.14. Приложение SPI Dynamics Free Bank

Стандартная страница входа содержит две веб-формы. Одна представляет собой форму входа, которая отправляет данные странице `login1.asp`, а другая отправляет данные странице `rootlogin.asp`. Именно вторая форма содержит уязвимость XSS. Среди полей ввода этой формы содержимое поля `txtName`, помеченного как Last Name, будет отражено на запрошенной странице `rootlogin.asp`. Поскольку пользовательские данные отображаются без какой-либо проверки,

приложение уязвимо для атаки XSS.

Распространённый способ проверить существование XSS состоит во вводе простого фрагмента кода JavaScript с функцией `alert`, который создаст всплывающее окно. Это быстрая и грубая проверка, дающая простой визуальный признак, который не оставляет сомнений в возможности внедрения клиентского JavaScript в целевую страницу. Поэтому для проверки страницы на XSS мы отправим следующий запрос:

```
POST /rootlogin.asp HTTP/1.1
Host: zero.webappsecurity.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.8.1.1) Gecko/20061204
Firefox/2.0.0.1
Accept:
text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/
png,*/*;q=0.5
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Referer: http://zero.webappsecurity.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 72

txtPassPhrase=first&txtName=<script>alert('Does fuzzing
work?')</script>&txtHidden=This+was+hidden+from+the+user
```

Как видно на рисунке 10.15, в результате на странице появляется всплывающее окно. Однако для фаззинга это непрактичный механизм обнаружения, поскольку потребовал бы от нас сидеть и наблюдать за результатами. Главное преимущество фаззинга состоит в возможности автоматизировать его, чтобы мы могли уйти, а по возвращении получить результаты.

К счастью, поскольку мы способны внедрять код в страницу, у нас есть множество вариантов. А что, если внедрять JavaScript, который при успешном выполнении будет «звонить домой»? Это сработало бы, но можно применить ещё более простой подход. Клиентский сценарий не обязан быть JavaScript: это может быть любой язык сценариев, который способен интерпретировать браузер. Как насчёт HTML? Это клиентский язык сценариев, и чёрные списки с меньшей вероятностью отфильтруют его, чем JavaScript, поэтому в одном отношении он обеспечивает более строгую проверку XSS. Тег HTML `IMG` предоставляет очень простую возможность «позвонить домой». Всё, что нужно сделать, - с помощью нашего фаззера внедрить тег HTML `IMG`, запрашивающий вымышленную страницу у локального веб-сервера. Когда фаззинг завершится, мы проверим журналы сервера, и если увидим запрос, вуаля! Мы будем знать, что цель уязвима для XSS. Давайте попробуем. Прежде всего необходимо добавить в WebFuzz подходящую фаззинговую переменную. Именно поэтому WebFuzz был спроектирован так, чтобы загружать фаззинговые переменные из обычных текстовых файлов. Добавлять новые переменные легко, и повторная компиляция приложения не требуется. Для проведения проверки мы добавим следующую строку в файл `xssinjection.txt`:

```
%3Cimg+src%3D%27http%3A%2F%2Flocalhost%2Fblah%27%3E
```

Это всего лишь закодированная в URL версия следующего запроса изображения, который пытается получить вымышленную страницу у нашего локального веб-сервера:

```
<img src='http://localhost/blah'>
```

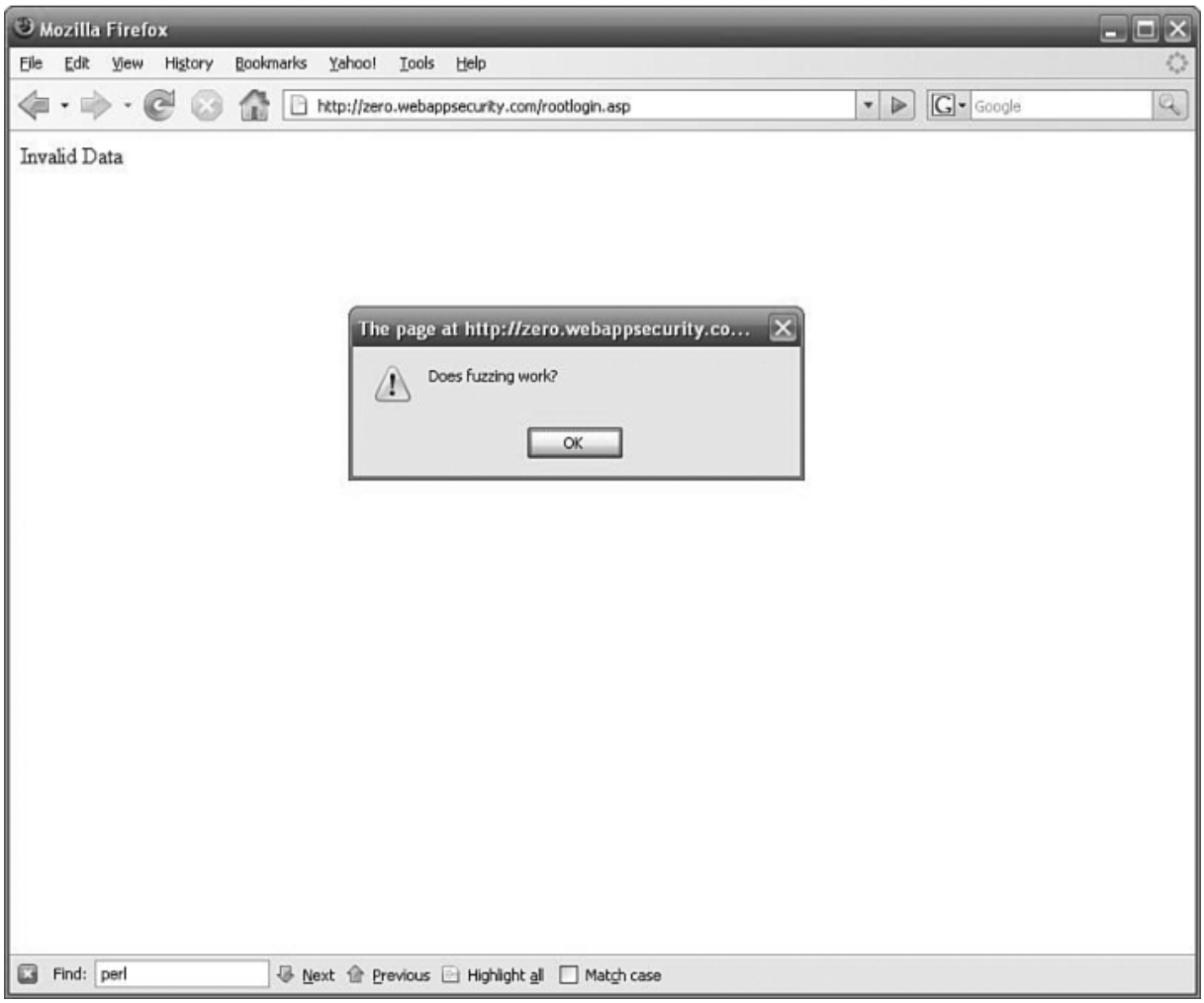


Рисунок 10.15. Атака межсайтового скриптинга в действии

Что мы увидим, проверив файлы журналов веб-сервера?

```
#Software: Microsoft Internet Information Services 5.1
#Version: 1.0
#Date: 2007-01-31 00:57:34
#Fields: time c-ip cs-method cs-uri-stem sc-status
00:57:34 127.0.0.1 GET /xss 404
```

Попался! Запись журнала `404 - Page Not Found` была создана, когда WebFuzz внедрил запрос изображения в страницу `rootlogin.asp`; это неопровержимо доказывает, что страница имеет уязвимость XSS.

Преимущества и возможности улучшения

Преимущества WebFuzz обусловлены уровнем контроля, предоставляемого конечному пользователю. А именно пользователь может полностью управлять каждым аспектом запроса. Однако ценой этого становится необходимость достаточно хорошо понимать HTTP, чтобы составить содержательный запрос, способный принести плоды. Одна из областей, где часть инструмента, отвечающую за запросы, можно улучшить, состоит в поддержке более сложных сочетаний фаззинговых переменных. Например, можно добавить зависимые переменные, чтобы одновременно вставлялось более одной переменной, причём значение одной определялось бы значением другой.

Остаётся также много возможностей для дальнейшей автоматизации WebFuzz с целью полного покрытия всего приложения. Значительно расширить её можно добавлением функций веб-паука, который первоначально запускался бы для выявления всех возможных запросов и их структуры. Перенос возможностей WebFuzz в расширение веб-браузера также был бы ценен, поскольку позволил бы подвергать фаззингу конкретные запросы по мере их появления при просмотре веб-сайтов. Наконец, мы видим возможность дополнительно автоматизировать обнаружение ответов, указывающих на потенциальные уязвимости. Например, механизм разбора мог бы проходить по необработанным ответам и искать признаки вроде предоставленных пользователем данных в ответе, способных указывать на существование уязвимостей XSS.

Что ж, мы сдвинули дело с мёртвой точки, но теперь мяч на вашей стороне. Улучшайте приложение и делитесь своими усовершенствованиями со всем миром, отправляя их нам, чтобы мы могли включить их в будущие версии.

Итоги

Веб-приложения ставят особые задачи, но определённо являются подходящими целями фаззинга. Надеемся, мы продемонстрировали ценность применения фаззинга для обнаружения уязвимостей в ваших веб-приложениях до их выпуска в рабочую среду. WebFuzz представляет собой лишь демонстрацию идеи, но даже в своём исходном состоянии способен обнаруживать уязвимости. Мы призываем вас взять исходный код WebFuzz и развить его в ещё более эффективный инструмент.

Сноски

[1] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[2] [назад] http://www.owasp.org/index.php/Category:OWASP_WebScarab_Project
[3] [назад] http://www.spidynamics.com/products/webinspect/toolkit.html
[4] [назад] http://www.codenomicon.com/products/internet/http/
[5] [назад] http://www.beyondsecurity.com/BeStorm_Info.htm
[6] [назад] http://livehttpheaders.mozdev.org/
[7] [назад] https://addons.mozilla.org/firefox/966/
[8] [назад] http://www.getfirebug.com/
[9] [назад] http://www.fiddlertool.com
[10] [назад] http://msdn.microsoft.com/library/en-us/cpref/html/frlrfsystemiofilestreamclassbeginwritetopic.asp
[11] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=352
[12] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=242
[13] [назад] http://secunia.com/advisories/15000/
[14] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=268
[15] [назад] http://www.securiteam.com/securityreviews/5DP0N1P76E.html
[16] [назад] http://www.ipswitch.com/support/whatsup_professional/guides/WhatsUpDBSchema.zip
[17] [назад] http://support.ipswitch.com/kb/WP-20041122-DM01.htm
[18] [назад] http://cwe.mitre.org/documents/vuln-trends.html#table1
[19] [назад] http://sla.ckers.org/forum/read.php?3,44

Глава 11. Фаззинг форматов файлов

> «Если бы это была диктатура, всё было бы чертовски проще, при условии, конечно, что диктатором был бы я».
>
> Джордж У. Буш, Вашингтон, округ Колумбия, 19 декабря 2000 года

Фаззинг форматов файлов - специализированный метод фаззинга с точно определёнными целями. Обычно этими целями служат клиентские приложения. Среди примеров - медиапроигрыватели,

веб-браузеры и пакеты офисных приложений. Однако целями могут быть и серверы, например антивирусные сканеры шлюзов, фильтры спама и даже обычные серверы электронной почты. Конечная цель фаззинга форматов файлов состоит в обнаружении эксплуатируемого дефекта в том, как приложение разбирает файл определённого типа.

В 2005 и 2006 годах было обнаружено впечатляющее число клиентских уязвимостей разбора форматов файлов, причём многие из них нашли злоумышленники: несколько эксплойтов нулевого дня обнаружили в реальных атаках ещё до обычной процедуры раскрытия уязвимостей. Исследовательская группа безопасности eEye превосходно описывает такие случаи в своём Zero-Day Tracker.[1] Ряд факторов указывает, что большинство этих находок было сделано с помощью фаззинга форматов файлов. Этот класс ошибок далеко не исчез, поэтому фаззинг форматов файлов остаётся очень интересной и «горячей» темой.

В этой главе мы представим различные подходы к фаззингу файлов, а также поговорим о разных способах, которыми определённые цели принимают входные данные. Наконец, мы продемонстрируем некоторые распространённые уязвимости, с которыми столкнётся файловый фаззер, и предложим способы обнаружения таких уязвимостей на практике. Разумеется, первый шаг - выбрать подходящую цель.

Цели

Как и при традиционных видах фаззинга, фаззинг форматов файлов позволяет находить множество различных типов уязвимостей. Существует и множество различных сценариев эксплуатации. Например, в некоторых ситуациях злоумышленнику придётся отправить пользователю вредоносный файл, который тот должен открыть вручную. В других ситуациях достаточно, чтобы пользователь перешёл на подконтрольную злоумышленнику веб-страницу. Наконец, иногда уязвимость можно вызвать, просто отправив вредоносное сообщение электронной почты через почтовый сервер или антивирусный шлюз. Последний сценарий имел место с уязвимостью Microsoft Exchange TNEF, упомянутой в таблице 11.1 вместе с другими примерами уязвимостей форматов файлов.

Таблица 11.1. Распространённые категории уязвимых приложений и примеры ранее обнаруженных уязвимостей форматов файлов

Категория приложения	**Название уязвимости**	**Бюллетень**
Пакеты офисных приложений	Переполнение буфера в библиотеке объектов гиперссылок Microsoft HLINK.DLL	http://www.tippingpoint.com/security/advisories/TSRT-06-10.html

Категория приложения	Название уязвимости	Бюллетень
Антивирусные сканеры	Переполнение буфера в анализаторе файлов CHM антивирусного ядра Kaspersky	http://www.idefense.com/intelligence/vulnerabilities/display.php?id=318
Медиапроигрыватели	Переполнение стека при разборе m3u в Winamp	http://www.idefense.com/intelligence/vulnerabilities/display.php?id=377
Веб-браузеры	Уязвимость Vector Markup Language, допускающая удалённое выполнение кода	http://www.microsoft.com/technet/security/Bulletin/MS06-055.mspx
Утилиты архивирования	Переполнение буфера при разборе MIME в WinZip	http://www.idefense.com/intelligence/vulnerabilities/display.php?id=76
Серверы электронной почты	Уязвимость декодирования TNEF в Microsoft Exchange	http://www.microsoft.com/technet/security/Bulletin/MS06-003.mspx

Вы обнаружите, что большинство целей относится к одной из этих категорий. Некоторые приложения благодаря своим дополнительным функциям попадают сразу в несколько категорий. Например, многие антивирусные сканеры включают также библиотеки распаковки файлов, позволяющие им выступать в роли утилит архивирования. Существуют даже сканеры содержимого, заявляющие, что анализируют файлы изображений на наличие порнографического содержания. Такие программы можно считать и средствами просмотра изображений![2] Нередко приложения совместно используют общие библиотеки, и тогда одна уязвимость может затрагивать несколько приложений. Рассмотрим, например, уязвимость, подробно описанную в бюллетене безопасности Microsoft MS06-055: она затрагивает как Internet Explorer, так и Outlook.

Методы

Фаззинг форматов файлов отличается от других видов фаззинга тем, что обычно целиком выполняется на одном узле. При фаззинге веб-приложений или сетевых протоколов у вас, скорее всего, будет не менее двух систем: целевая система и система, на которой работает фаззер. Повышение производительности благодаря возможности выполнять фаззинг на одной машине делает фаззинг форматов файлов особенно привлекательным подходом к обнаружению уязвимостей.

При сетевом фаззинге часто очевидно, когда в целевом приложении возникает интересное состояние. Во многих случаях сервер завершает работу или полностью аварийно прекращает выполнение и становится недоступен. При фаззинге форматов файлов, главным образом клиентских приложений, фаззер будет постоянно перезапускать и завершать целевое приложение, поэтому без надлежащего наблюдения фаззер может не распознать сбой. В этом отношении фаззинг форматов файлов сложнее сетевого фаззинга.

При фаззинге форматов файлов фаззеру обычно приходится при каждом запуске следить за исключениями в целевом приложении. Как правило, это делается с помощью отладочной библиотеки, которая динамически отслеживает обработанные и необработанные исключения в целевом приложении, записывая результаты для последующего просмотра. На самом общем уровне типичный файловый фаззер выполняет следующие действия:

1. Подготавливает тестовый пример путём мутации или генерации, о чём подробнее будет сказано далее.
2. Запускает целевое приложение и даёт ему команду загрузить тестовый пример.
3. Следит за сбоями целевого приложения, обычно с помощью отладчика.
4. Если обнаружен сбой, регистрирует находку. Если же за некоторый период сбой не обнаружен, вручную завершает целевое приложение.
5. Повторяет процесс.

Фаззинг форматов файлов можно реализовать как методами генерации, так и методами мутации. По нашему опыту оба метода очень эффективны, однако метод мутации, или «полного перебора», определённо проще реализовать. Метод генерации, или фаззинг «интеллектуальным полным перебором», требует больше времени на реализацию, но обнаруживает уязвимости, которые иначе не удалось бы найти более примитивным подходом полного перебора.

Фаззинг полным перебором, или на основе мутаций

При фаззинге методом полного перебора сначала нужно собрать несколько различных образцов целевого типа файлов. Чем больше разных файлов удастся найти, тем полнее будет проверка. Затем фаззер воздействует на эти файлы, создавая их мутации и передавая их анализатору целевого приложения. В зависимости от выбранного метода фаззера мутации могут принимать любую форму. Один из методов - побайтовая замена данных. Например, можно последовательно пройти по файлу и заменять каждый байт значением `0xff`. То же самое можно делать с несколькими байтами, например с диапазонами по два и четыре байта. Можно также вставлять данные в файл вместо простой перезаписи байтов. Такой метод полезен при проверке строковых значений. Однако, вставляя данные в файл, помните, что вы можете нарушить смещения внутри него. Это способно сильно сократить покрытие кода, поскольку некоторые анализаторы быстро обнаружат недопустимый файл и завершат работу.

Проверки контрольных сумм также могут помешать анализаторам полного перебора. Поскольку любое изменение байта делает контрольную сумму недействительной, весьма вероятно, что анализирующее приложение корректно завершит работу с сообщением об ошибке ещё до

достижения потенциально уязвимого участка кода. Для решения таких проблем можно либо перейти к интеллектуальному фаззингу, который обсуждается в следующем разделе, либо применить альтернативный подход и отключить проверки в целевой программе. Отключение программных проверок - нетривиальная задача, обычно требующая работы специалиста по обратной разработке.

Почему после реализации этот метод прост в применении? Всё просто. Конечному пользователю не нужно знать формат файла и принципы его работы. Если он может найти несколько файлов-образцов с помощью популярной поисковой системы или поиска по локальной системе, его исследовательская работа фактически закончена до тех пор, пока фаззер не обнаружит что-нибудь интересное.

У этого подхода к фаззингу есть несколько недостатков. Во-первых, он крайне неэффективен, поэтому фаззинг одного файла может занять некоторое время. Возьмём, например, простой документ Microsoft Word. Даже пустой документ будет иметь размер приблизительно 20 Кбайт. Для однократного фаззинга каждого байта потребуется создать и запустить 20 480 отдельных файлов. При двух секундах на файл работа займёт более 11 часов, и это лишь для проверки одного байтового значения. А как насчёт остальных 254 вариантов? Эту проблему можно отчасти обойти с помощью многопоточного фаззера, но она наглядно показывает неэффективность чистого фаззинга мутациями. Ещё один способ ускорить этот подход - сосредоточиться исключительно на областях файла, которые с большей вероятностью дадут желаемый результат, например на заголовках файла и полей.

Главный недостаток фаззинга полным перебором состоит в том, что почти всегда значительная часть функциональности останется непроверенной, если только вам каким-то образом не удалось собрать набор файлов-образцов, содержащий абсолютно все возможные возможности. Большинство форматов файлов очень сложны и допускают множество комбинаций. При измерении покрытия кода вы обнаружите, что несколько файлов-образцов проверяют приложение не так тщательно, как в случае, когда пользователь действительно понимает формат файла и вручную подготовил некоторые сведения о типе файла. Проблему полноты решает генерационный подход к фаззингу файлов, названный нами фаззингом интеллектуальным полным перебором.

Фаззинг интеллектуальным полным перебором, или на основе генерации

При фаззинге интеллектуальным полным перебором сначала придётся приложить усилия к исследованию спецификаций файла. Интеллектуальный фаззер всё равно остаётся механизмом фаззинга и потому всё ещё проводит атаку полным перебором. Однако он опирается на конфигурационные файлы пользователя, что делает процесс более интеллектуальным. Такие файлы обычно содержат метаданные, описывающие язык типов файлов. Представляйте эти шаблоны как списки структур данных, их взаимного расположения и возможных значений. На уровне реализации их можно представить во множестве различных форматов.

Если для проверки выбран формат файла без общедоступной документации, вам как исследователю придётся дополнительно изучить спецификацию формата, прежде чем создавать шаблон. Это может потребовать обратной разработки с вашей стороны, однако всегда начинайте с доброго друга Google: возможно, кто-то уже выполнил эту работу за вас. Несколько веб-сайтов, например Wotsit's Format,[3] служат превосходными архивами официальной и неофициальной документации по форматам файлов. Альтернативный, но дополняющий подход состоит в сравнении образцов файлов данного типа для выявления закономерностей и определения некоторых используемых типов данных. Помните, что эффективность интеллектуального фаззинга непосредственно связана с вашим пониманием формата файла и способностью описать его в общем виде используемому фаззеру. Позднее в книге, создавая SPIKEfile в главе 12 «Фаззинг

форматов файлов: автоматизация в UNIX», мы покажем пример реализации фаззера интеллектуального полного перебора.

После определения цели и метода следующий шаг - исследовать подходящие входные векторы для выбранной цели.

Входные данные

Выбрав целевое приложение, далее нужно перечислить поддерживаемые типы и расширения файлов, а также различные векторы, с помощью которых эти файлы можно передать анализатору. Следует также собрать и изучить доступные спецификации форматов. Даже если вы намерены провести лишь простую проверку полным перебором, всё равно полезно знать форматы файлов, которые рассматриваются как возможные кандидаты. Сосредоточиться на более сложных типах файлов может быть выгодно, поскольку правильный анализатор для них реализовать труднее, а значит, вероятность обнаружить уязвимость, возможно, возрастает.

Рассмотрим пример и посмотрим, как можно собрать входные данные. Утилита архивирования WinRAR[4] - популярная и свободно доступная программа. Простой способ узнать, какие файлы обрабатывает WinRAR, - просмотреть веб-сайт WinRAR. На главной странице WinRAR приведён список поддерживаемых типов файлов. В него входят zip, rar, tar, gz, ace, uue и несколько других.

Теперь, когда у вас есть список типов файлов, обрабатываемых WinRAR, необходимо выбрать цель. Иногда лучший способ выбрать цель - найти сведения о каждом типе файлов и остановиться на самом сложном. Здесь мы исходим из того, что сложность часто приводит к ошибкам программирования. Например, тип файла, использующий множество значений с метками длины и предоставленные пользователем смещения, может быть привлекательнее более простого типа со статическими смещениями и полями фиксированной длины. Разумеется, из этого правила существует множество исключений, как вы обнаружите, накопив некоторый опыт фаззинга. В идеале фаззер со временем проверит каждый возможный тип файла; какой тип выбран первым, не обязательно важно, однако всегда приятно обнаружить интересное поведение уже в первом наборе фаззинговых тестов конкретного приложения.

Уязвимости

При разборе искажённых файлов плохо написанное приложение может быть подвержено нескольким различным классам уязвимостей. В этом разделе рассматриваются следующие классы уязвимостей:

• отказ в обслуживании, то есть аварийное завершение или зависание;
• проблемы обработки целых чисел;
• простые переполнения стека или кучи;
• логические ошибки;
• форматные строки;
• состояния гонки.

Отказ в обслуживании

Хотя проблемы DoS не слишком интересны в клиентских приложениях, следует помнить, что целью могут быть и серверные приложения, которые должны оставаться доступными ради безопасности и производительности. Разумеется, к ним относятся серверы электронной почты и фильтры содержимого. По нашему опыту, наиболее распространёнными причинами проблем DoS в коде разбора файлов были чтение за пределами допустимой области, бесконечные циклы и разыменования указателя `NULL`.

Распространённая ошибка, приводящая к бесконечным циклам, - доверие значениям смещений в файлах, которые задают расположение других блоков внутри файла. Если приложение не проверяет, что это смещение направлено вперёд относительно текущего блока, может возникнуть бесконечный цикл, заставляющий приложение снова и снова обрабатывать один и тот же блок или блоки. В прошлом в ClamAV встречалось несколько подобных проблем.[5]

Проблемы обработки целых чисел

Целочисленные переполнения и проблемы знаковости очень распространены при разборе двоичных файлов. Некоторые из самых частых встречавшихся нам проблем похожи на следующий псевдокод:

```
[...]
[1] size            = read32_from_file();
[2] allocation_size = size+1;
[3] buffer          = malloc(allocation_size);
[4] for (ix = 0; ix < size; ix++)
[5]      buffer[ix] = read8_from_file();
[...]
```

Этот пример демонстрирует типичное целочисленное переполнение, приводящее к повреждению памяти. Если в файле для значения `size` указано максимальное беззнаковое 32-разрядное целое число (`0xFFFFFFFF`), то в строке [2] переменной `allocation_size` из-за циклического перехода целого числа присваивается ноль. В строке [3] код вызывает выделение области памяти нулевого размера. На этом этапе указатель `buffer` указывает на область памяти недостаточного размера. В строках [4] и [5] приложение выполняет цикл и копирует большой объём данных, ограниченный исходным значением `size`, в выделенный буфер, что приводит к повреждению памяти.

Эта конкретная ситуация не всегда будет эксплуатируемой. Возможность эксплуатации зависит от того, как приложение использует кучу. Простой перезаписи памяти в куче не всегда достаточно, чтобы получить управление приложением. Должна произойти некоторая операция, при которой будут использованы перезаписанные данные кучи. В ряде случаев подобные целочисленные переполнения вызывают сбой, не связанный с кучей, до использования памяти кучи.

Разумеется, это лишь один пример неправильного использования целых чисел при разборе двоичных данных. Мы наблюдали множество различных способов их неправильного применения, включая зачастую более простую ошибку сравнения знакового значения с беззнаковым. Следующий фрагмент кода демонстрирует логику уязвимости этого типа:

```
[0] #define MAX_ITEMS 512
[...]
[1] char buff[MAX_ITEMS]
[2] int size;
[...]
[3] size = read32_from_file();
[4] if (size > MAX_ITEMS)
[5]    { printf("Too many items\n"); return -1; }
[6] readx_from_file(size, buff);
[...]
```

```
/* readx_from_file: прочитать 'size' байт из файла в buff */
[7] void readx_from_file(unsigned int size, char *buff)
{
    [...]
}
```

Этот код допускает переполнение стека, если значение `size` является отрицательным числом. Причина в том, что в сравнении в строке [4] и `size`, определённая в строке [1], и `MAX_ITEMS`, определённая в строке [0], рассматриваются как знаковые числа, и, например, `-1` меньше 512. Позднее, когда `size` используется как граница копирования в функции из строки [7], это значение рассматривается как беззнаковое. Например, значение `-1` теперь интерпретируется как `4294967295`. Разумеется, возможность эксплуатации не гарантирована, но во многих случаях в зависимости от реализации функции `readx_from_file` уязвимость можно будет эксплуатировать, воздействуя на переменные и сохранённые регистры в стеке.

Простые переполнения стека и кучи

Эти проблемы хорошо изучены и многократно встречались в прошлом. Типичный сценарий выглядит так: выделяется буфер фиксированного размера в стеке или куче. Позднее при копировании в него данных из файла чрезмерного размера проверка границ не выполняется. В некоторых случаях проверку границ пытаются проводить, но делают это неправильно. При копировании память повреждается, что часто приводит к выполнению произвольного кода. Превосходным справочным источником с более подробными сведениями об этих классах уязвимостей служит книга «The Shellcoder's Handbook: Discovering and Exploiting Security Holes».[6]

Логические ошибки

В зависимости от устройства формата файла могут возникать эксплуатируемые логические ошибки. Хотя при исследовании уязвимостей форматов файлов мы лично логических ошибок не обнаруживали, идеальным примером этого класса служит уязвимость Microsoft WMF, исправленная в MS06-001.[7] Уязвимость не была вызвана обычным переполнением. Более того, она вообще не требовала повреждения памяти, но позволяла злоумышленнику непосредственно выполнять предоставленный пользователем позиционно-независимый код.

Форматные строки

Хотя уязвимости форматных строк по большей части вымерли, особенно в программах с открытым исходным кодом, упомянуть их стоит. Мы говорим «по большей части вымерли», потому что не все программисты могут быть столь же осведомлены о безопасности, как сотрудники US-CERT, которые для защиты программ рекомендуют «не использовать спецификатор форматной строки %n».[8]

Но если говорить серьёзно, по личному опыту мы действительно обнаруживали при фаззинге файлов несколько проблем, связанных с форматными строками. Некоторые из них были найдены в продуктах Adobe[9] и RealNetworks.[10] Значительная часть интереса при эксплуатации проблем форматных строк связана с возможностью использовать уязвимость для утечки памяти, помогающей эксплуатации. К сожалению, при клиентских атаках с использованием искажённых файлов такая возможность предоставляется редко.

Состояния гонки

Хотя обычно люди не думают, что уязвимости форматов файлов могут возникать из-за состояний гонки, несколько таких уязвимостей существовало, и, вероятно, впереди их гораздо больше. Основные цели для уязвимостей этого типа - сложные многопоточные приложения. Нам не хочется выделять какой-либо один продукт, но первым на ум приходит Microsoft Internet Explorer. Вероятно, уязвимости, вызванные использованием Internet Explorer неинициализированной памяти и памяти, которую использует другой поток, будут обнаруживаться и дальше.

Обнаружение

При фаззинге форматов файлов обычно создаётся множество экземпляров целевого приложения. Некоторые навсегда зависнут и потребуют принудительного завершения, некоторые аварийно завершатся, а некоторые самостоятельно закончат работу без ошибок. Задача состоит в том, чтобы определить, когда возникло обработанное или необработанное исключение и когда это исключение эксплуатируемо. Для получения дополнительных сведений о процессе фаззер может использовать несколько источников информации:

• **Журналы событий.** Журналы событий используются операционными системами Microsoft Windows, и доступ к ним можно получить через приложение «Просмотр событий». Для наших целей они не слишком полезны, поскольку при запуске сотен процессов за один сеанс фаззинга трудно сопоставить запись журнала событий с конкретным процессом.
• **Отладчики.** Лучший способ выявлять необработанные и обработанные исключения - подключить отладчик к целевому приложению до начала фаззинга. Обработка ошибок скрывает явные признаки многих ошибок, вызванных фаззингом, но обычно их можно обнаружить с помощью отладчика. Существуют более совершенные методы обнаружения сбоев, чем применение отладчика; некоторые из них затронуты в главе 24 «Интеллектуальное обнаружение сбоев».

• **Коды возврата.** Перехват и проверка кода возврата приложения, хотя и не всегда столь точны или информативны, как использование отладчика, могут служить быстрым и грубым способом определить причину завершения приложения. По крайней мере в UNIX по коду возврата можно определить, какой сигнал вызвал завершение приложения.
• **API отладки.** Вместо стороннего отладчика часто можно и полезно реализовать некоторые базовые отладочные возможности в самом фаззере. Например, о процессе нас будет интересовать лишь причина его завершения, текущая инструкция, состояние регистров и, возможно, значения в нескольких областях памяти, например по указателю стека или относительно некоторого регистра. Часто это реализуется без труда и позволяет сэкономить бесценное время при анализе возможности эксплуатации сбоев. В последующих главах мы исследуем этот вариант и представим простую многократно используемую основу для создания отладчика на платформе Microsoft Windows под названием PyDbg, которая входит в среду обратной разработки PaiMei.[11]

После того как установлено, что некоторый тестовый пример вызывает сбой, обязательно сохраните сведения, собранные выбранным способом наблюдения за сбоями, а также сам файл, вызвавший аварийное завершение. Важно сохранять в записях и метаданные тестового примера. Например, если фаззер проверял восьмое поле переменной в файле и использовал 42-е фаззинговое значение, файл можно назвать `file-8-42`. В некоторых случаях может понадобиться создать и сохранить файл дампа памяти. Это можно сделать, если фаззер перехватывает сигналы с помощью API отладки. Конкретные подробности реализации приведены в следующих двух главах.

Итоги

Хотя фаззинг форматов файлов является узко определённым методом фаззинга, существует множество целей и многочисленные векторы атак. Мы обсудили не только более традиционные клиентские уязвимости форматов файлов, но и некоторые сценарии «по-настоящему удалённых» атак, например на антивирусные шлюзы и почтовые серверы. По мере того как всё больше внимания уделяется предотвращению и обнаружению сетевых атак через TCP/IP, эксплойты форматов файлов по-прежнему остаются ценным оружием для проникновения во внутренние сегменты сети.

Теперь, когда вам известно, какие уязвимости вы ищете, какие приложения выбрали целями и как именно пытаетесь подвергнуть их фаззингу, настало время исследовать несколько реализаций инструментов фаззинга форматов файлов.

Сноски

[1] [назад] http://research.eeye.com/html/alerts/zeroday/
[2] [назад] http://www.clearswift.com/solutions/porn_filters.aspx
[3] [назад] http://www.wotsit.org
[4] [назад] http://www.rarlab.com
[5] [назад] http://idefense.com/intelligence/vulnerabilities/display.php?id=333
[6] [назад] ISBN-10: 0764544683
[7] [назад] http://www.microsoft.com/technet/security/Bulletin/MS06-001.mspx
[8] [назад] https://buildsecurityin.us-cert.gov/daisy/bsi/articles/knowledge/guidelines/340.html
[9] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=163
[10] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=311
[11] [назад] http://www.openrce.org/downloads/details/208

Глава 12. Фаззинг форматов файлов: автоматизация в UNIX

> «Я командующий, понимаете, мне не нужно объяснять, мне не нужно объяснять, почему я что-либо говорю. Вот что интересно в должности президента».
>
> Джордж У. Буш, цитируется по книге Боба Вудворда «Bush at War»

Уязвимости форматов файлов можно эксплуатировать как на стороне клиента, например в веб-браузерах и офисных пакетах, так и на стороне сервера, например в антивирусных шлюзах сканирования электронной почты. В отношении клиентской эксплуатации распространённость затронутого клиента напрямую связана с серьёзностью проблемы. Например, уязвимость разбора HTML, затрагивающая Microsoft Internet Explorer, относится к числу самых желанных уязвимостей форматов файлов с точки зрения серьёзности. И наоборот, клиентские уязвимости, ограниченные платформой UNIX, не столь интересны из-за меньшего охвата.

Тем не менее в этой главе мы представим два фаззера, notSPIKEfile и SPIKEfile, реализующих фаззинг форматов файлов соответственно на основе мутаций и генерации. Мы рассмотрим как возможности, так и недостатки обоих инструментов. Затем, глубже погрузившись в процесс разработки, представим подход, использованный при создании инструментов, и ключевые фрагменты кода. Мы также разберём некоторые базовые понятия UNIX, например интересные и неинтересные сигналы и процессы-зомби. Наконец, рассмотрим основы применения инструмента и объясним причины выбора использованного языка программирования.

notSPIKEfile и SPIKEfile

Два инструмента, разработанных для демонстрации фаззинга файлов в UNIX, называются SPIKEfile и notSPIKEfile. Как следует из названий, они соответственно основаны на SPIKE[1] и не основаны на SPIKE. Ниже кратко перечислены некоторые ключевые возможности этих реализаций:

• встроенный минимальный отладчик, способный обнаруживать обработанные и необработанные сигналы и создавать дампы памяти и состояния регистров;
• полностью автоматический фаззинг с задаваемой пользователем задержкой перед завершением целевого процесса;
• две отдельные базы эвристик фаззинга для двоичных и печатаемых текстовых типов данных ASCII;
• легко расширяемая база эвристик фаззинга, изначально заполненная значениями, которые, как известно из прошлого опыта, вызывают проблемы.

Чего не хватает?

В notSPIKEfile и SPIKEfile отсутствуют некоторые возможности, которые могли бы пригодиться пользователю этих инструментов. Ниже приведено краткое описание недостающих возможностей:

• **Переносимость.** Поскольку встроенный отладчик создавался для x86 и использует Linux `ptrace`, без доработок он не будет работать на других архитектурах или в других операционных системах. Однако добавить совместимость было бы довольно просто при наличии других библиотек отладки и дизассемблирования.
• **Интеллектуальное наблюдение за нагрузкой.** Хотя пользователь может указать, сколько процессов подвергать фаззингу одновременно, в настоящее время именно он должен полностью самостоятельно определять, не слишком ли высока нагрузка на систему.

Очевидно, в процессе разработки пришлось пойти на некоторые компромиссы, что и показывают недостающие возможности. Автор инструмента воспользуется старым как мир оправданием, заявив, что усовершенствования оставлены «в качестве упражнения для читателя». Рассмотрим процесс разработки этих инструментов.

Подход к разработке

Главная тема этой главы - не применение фаззера, а его реализация. В данном разделе описаны архитектура и подробности разработки фаззеров форматов файлов в Linux. В области разработки файловых фаззеров можно смело предположить, что фаззер будет выполняться в той же системе, что и цель. Наша архитектура включает три отдельных функциональных компонента.

Механизм обнаружения исключений

Эта часть кода отвечает за определение момента, когда цель демонстрирует некоторое неопределённое и потенциально небезопасное поведение. Как это сделать? Существуют два основных подхода. Первый, довольно простой, состоит лишь в наблюдении за всеми сигналами, получаемыми приложением. Это позволяет фаззеру обнаруживать, например, переполнения буфера, приводящие к недопустимому обращению к памяти. Разумеется, такой подход не обнаруживает поведение вроде внедрения метасимволов или логических дефектов. Например, рассмотрим приложение, которое передаёт частично предоставленное злоумышленником значение системной функции UNIX. Это позволило бы злоумышленнику выполнять произвольные программы с помощью метасимволов оболочки, но не вызвало бы никакого нарушения доступа к памяти. Хотя существуют уязвимости, позволяющие злоумышленнику нарушить безопасность узла вообще без повреждения памяти, мы решили, что такие ошибки нас не интересуют, поскольку реализация их обнаружения была бы мучительнее разработки, чем оправдывал бы результат.

Тем, кто хочет исследовать подобные логические ошибки, подойдёт перехват функций библиотеки C (`LIBC`) с наблюдением за небезопасной передачей предоставленных фаззером значений таким вызовам, как `open`, `creat`, `system` и прочим.

В нашей архитектуре мы решили просто обнаруживать сигналы через системный отладочный интерфейс `ptrace`. Хотя определить, что сигнал вызвал завершение приложения, совсем не трудно, если просто дождаться возврата приложения, для обнаружения сигналов, которые приложение обрабатывает внутренне, потребуется немного больше работы. Именно поэтому подход, использованный в механизме обнаружения исключений, в значительной мере опирается на системный отладочный интерфейс, в данном случае на системный вызов `ptrace`.

Составление отчётов об исключениях (обнаружение исключений)

Обнаружив исключение, хороший фаззер должен сообщить полезные сведения о том, что именно произошло. Как минимум следует сообщить возникший сигнал. В идеальном случае отчёт должен также включать дополнительные сведения, например вызвавшую ошибку инструкцию, состояние регистров процессора и дамп стека. Оба фаззера, реализованные в этой главе, способны создавать такие подробные отчёты. Для получения желаемых низкоуровневых сведений необходимо прибегнуть к помощи системного вызова `ptrace`. Если мы намерены сообщать пользователю инструкции, нужно также использовать библиотеку для их дизассемблирования. Библиотека должна уметь преобразовывать буфер произвольных данных в строковое представление инструкций x86. Мы выбрали libdisasm,[2] поскольку она хорошо работает, предоставляет простой интерфейс и, судя по всему, является предпочтительным выбором поисковой системы Google, которая поставила её выше всех остальных. Окончательно решило дело то, что libdisasm содержит примеры, которые мы можем скопировать и вставить, ещё немного облегчив себе жизнь.

Основной механизм фаззинга

Это сердце фаззера форматов файлов: данный компонент определяет, какие искажённые данные использовать и куда их вставлять. Код, выполняющий эту работу в notSPIKEfile, отличается от кода SPIKEfile, поскольку SPIKEfile использует для реализации этих возможностей уже существующий код SPIKE. Однако будьте уверены: они довольно похожи.

Как вы, несомненно, уже догадались, SPIKEfile использует тот же механизм фаззинга, что и SPIKE. Механизму фаззинга требуется шаблон, называемый сценарием SPIKE, который описывает формат файла. Затем SPIKE «интеллектуально» создаёт варианты этого формата, используя сочетания допустимых и недопустимых данных.

В notSPIKEfile механизм фаззинга значительно более ограничен. Пользователь должен предоставить фаззеру допустимый целевой файл. Затем механизм изменяет различные части этого файла, используя базу фаззинговых значений. Эти значения делятся на два типа, называемые двоичными и строковыми. Двоичные значения могут иметь любую длину и любое содержимое, но обычно применяются для представления размеров распространённых целочисленных полей. Строковые значения, как следует из названия, являются обычными строками. Это могут быть длинные или короткие строки, строки со спецификаторами формата и всевозможные другие исключительные значения, например пути к файлам, URL и любые иные особые строки, которые только можно придумать. Полный список значений этих типов приведён в исходном коде SPIKE и notSPIKEfile, а для начала таблица 12.1 предлагает краткий перечень и короткое объяснение нескольких эффективных фаззинговых строк. Этот список далеко не исчерпывающий, но должен дать представление о типах входных данных, которые необходимо учитывать.

Таблица 12.1. Некоторые распространённые фаззинговые строки и их назначение

Строка	**Назначение**
`"A" x 10000`	Длинная строка, способная вызвать переполнение буфера.
`"%n%n" x 5000`	Длинная строка со знаками процента, способная вызвать переполнение буфера или уязвимость форматной строки.
`HTTP:// + "A" x 10000`	Допустимый формат URL, способный вызвать переполнение буфера в коде разбора URL.
`"A" x 5000 + "@" + "A" x 5000`	Допустимый формат адреса электронной почты, способный вызвать переполнение буфера в коде разбора адресов электронной почты.
`0x20000000`, `0x40000000`, `0x80000000`, `0xffffffff`	Несколько из множества целых чисел, способных вызвать целочисленное переполнение. Здесь можно проявить большую изобретательность. Представьте код, выполняющий `malloc(user_count*sizeof(struct blah));`. Учитывайте также код, который может увеличивать или уменьшать целые числа без проверки переполнения или выхода ниже допустимого диапазона.
`"../" x 5000 + "AAAA"`	Способна вызвать переполнение в коде разбора пути или адреса URL.

Число фаззинговых строк практически не ограничено. Однако важно помнить: любой тип значения, который может требовать особого разбора или создавать исключительное состояние, должен быть хорошо представлен. Разница между пропуском и обнаружением ошибки вашим фаззером может сводиться к такой простой вещи, как добавление `.html` в конец большой фаззинговой строки.

В следующем разделе мы исследуем некоторые наиболее интересные и важные фрагменты кода SPIKEfile и notSPIKEfile.

Важные фрагменты кода

Оба рассматриваемых здесь инструмента фаззинга совместно используют значительную часть основной функциональности. Для начала выделим базовый метод создания и трассировки дочернего процесса. Следующий фрагмент написан на языке C и используется обоими фаззерами:

```
[...]
if (!(pid = fork()))
{ /* child */
    ptrace(PTRACE_TRACEME, 0, NULL, NULL);
    execve(argv[0], argv, envp);
}
else
{ /* parent */

    c_pid = pid;
monitor:
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
    { /* program exited */
        if (!quiet)
            printf("Process %d exited with code %d\n", pid,
                WEXITSTATUS(status));
        return(ERR_OK);
    }
    else if (WIFSIGNALED(status))
    { /* program ended because of a signal */
        printf("Process %d terminated by unhandled signal %d\n", pid,
            WTERMSIG(status));
        return(ERR_OK);
    }
    else if (WIFSTOPPED(status))
    { /* program stopped because of a signal */
        if (!quiet)
            fprintf(stderr, "Process %d stopped due to signal %d (%s) ",
                pid, WSTOPSIG(status), F_signum2ascii(WSTOPSIG(status)));
    }
    switch (WSTOPSIG(status))
    { /* the following signals are usually all we care about */
        case SIGILL:
        case SIGBUS:
        case SIGSEGV:
        case SIGSYS:
            printf("Program got interesting signal...\n");
            if ((ptrace(PTRACE_CONT, pid, NULL,
                (WSTOPSIG(status) == SIGTRAP) ? 0 : WSTOPSIG(status))) == -1)
            {
                perror("ptrace");
            }
            ptrace(PTRACE_DETACH, pid, NULL, NULL);
            fclose(fp);
            return(ERR_CRASH); /* it crashed */
    }
    /* передать сигнал дальше и продолжить трассировку */
    if ((ptrace(PTRACE_CONT, pid, NULL,
        (WSTOPSIG(status) == SIGTRAP) ? 0 : WSTOPSIG(status))) == -1)
    {
        perror("ptrace");
```

```
    }
    goto monitor;
}
return(ERR_OK);
}
```

Основной, или родительский, процесс начинает с создания нового процесса для цели. Новый, или дочерний, процесс с помощью вызова `ptrace` и запроса `PTRACE_TRACEME` указывает, что родитель будет его трассировать. Затем дочерний процесс продолжает выполнение цели, зная, что родитель, подобно всякому хорошему родителю, присмотрит за ним, если возникнет желание сделать что-нибудь неподобающее.

Поскольку дочерний процесс использовал запрос `PTRACE_TRACEME`, фаззер в роли родительского процесса способен получать все сигналы, предназначенные дочернему процессу. Родитель получает сигнал даже при любом вызове семейства функций `exec`. Цикл родителя прост, но эффективен. Фаззер циклически получает каждый сигнал, предназначенный дочернему процессу. Затем его поведение меняется в зависимости от сигнала и состояния дочернего процесса.

Например, если процесс остановлен, это означает, что программа не завершилась, но получила сигнал и ожидает решения родителя о разрешении продолжить работу. Если сигнал указывает на проблему повреждения памяти, фаззер передаёт его дочернему процессу, предполагая, что сигнал завершит процесс, после чего сообщает результаты. Если сигнал безвреден или попросту неинтересен, фаззер без каких-либо забот о наблюдении передаёт его приложению.

Фаззер также проверяет, действительно ли дочерний процесс завершился, что является другой, неинтересной ситуацией. А если программа аварийно завершилась, спросите вы? Разве это не должно очень нас интересовать? Поскольку мы перехватываем все интересные сигналы до фактического завершения приложения, нам известно, что программа завершилась либо естественным образом, либо из-за неинтересного сигнала. Это подчёркивает важность понимания того, какие сигналы интересны именно вам. Некоторые могут считать исключение операций с плавающей запятой интересным, если ищут уязвимости DoS. Других интересуют лишь настоящие проблемы повреждения памяти. Третьи могут искать сигналы прерывания, которые стали признаком повреждения кучи в новых версиях GLIBC. Было показано, что в некоторых обстоятельствах эти проверки повреждения кучи можно обойти и выполнить произвольный код.[3]

После знакомства с кодом обработки определённых сигналов может возникнуть вопрос, почему одни сигналы мы обрабатываем иначе, чем другие. Далее сигналы UNIX объясняются в контексте исследования уязвимостей.

Обычно интересные сигналы UNIX

В таблице 12.2 перечислены сигналы, которые исследователь уязвимостей может считать интересными во время фаззинга, и объяснено, почему они интересны.

Таблица 12.2. Интересные сигналы при фаззинге в UNIX

Имя интересного сигнала	Значение
`SIGSEGV`	Недопустимое обращение к памяти. Наиболее распространённый результат успешного фаззинга.

Имя интересного сигнала	Значение
SIGILL	Недопустимая инструкция. Возможное, хотя относительно редкое следствие повреждения памяти. Часто возникает, когда счётчик команд повреждается и попадает в середину данных или между инструкциями.
SIGSYS	Неверный системный вызов. Также возможное, но относительно редкое, фактически очень редкое, следствие повреждения памяти. Может возникнуть по тем же причинам, что и SIGILL.
SIGBUS	Ошибка шины. Часто вызвана той или иной формой повреждения памяти и возникает при неправильном обращении к памяти. Чаще встречается на машинах RISC из-за требований к выравниванию. В большинстве реализаций RISC невыровненные операции записи и загрузки создают SIGBUS.
SIGABRT	Создаётся вызовом функции abort. Часто представляет интерес, поскольку GLIBC вызывает abort, обнаружив повреждение кучи.

Не столь интересные сигналы UNIX

В отличие от таблицы 12.2, таблица 12.3 описывает сигналы, которые часто возникают при фаззинге, но обычно не интересуют исследователя уязвимостей.

Таблица 12.3. Неинтересные сигналы при фаззинге в UNIX

Имя неинтересного сигнала	Значение
SIGCHLD	Дочерний процесс завершился.
SIGKILL, SIGTERM	Процесс был завершён.
SIGFPE	Исключение операции с плавающей запятой, например деление на ноль.
SIGALRM	Истёк таймер.

Теперь, когда мы представили SIGCHLD, самое время обсудить обработку распространённого сценария, вызванного неправильной обработкой этого сигнала. В следующем разделе объясняется, что такое процесс-зомби и как правильно обрабатывать дочерние процессы, чтобы зомби не создавались.

Процессы-зомби

Процесс-зомби - это процесс, созданный родительским процессом и завершивший выполнение, то есть вышедший, но родитель которого не получил его состояние вызовом `wait` или `waitpid`. Когда это происходит, сведения о завершённом процессе сохраняются в ядре на неопределённый срок, пока родительский процесс их не запросит. В этот момент сведения освобождаются, и процесс действительно прекращает существование. Общий жизненный цикл процесса, созданного нашим фаззером, показан на рисунке 12.1.

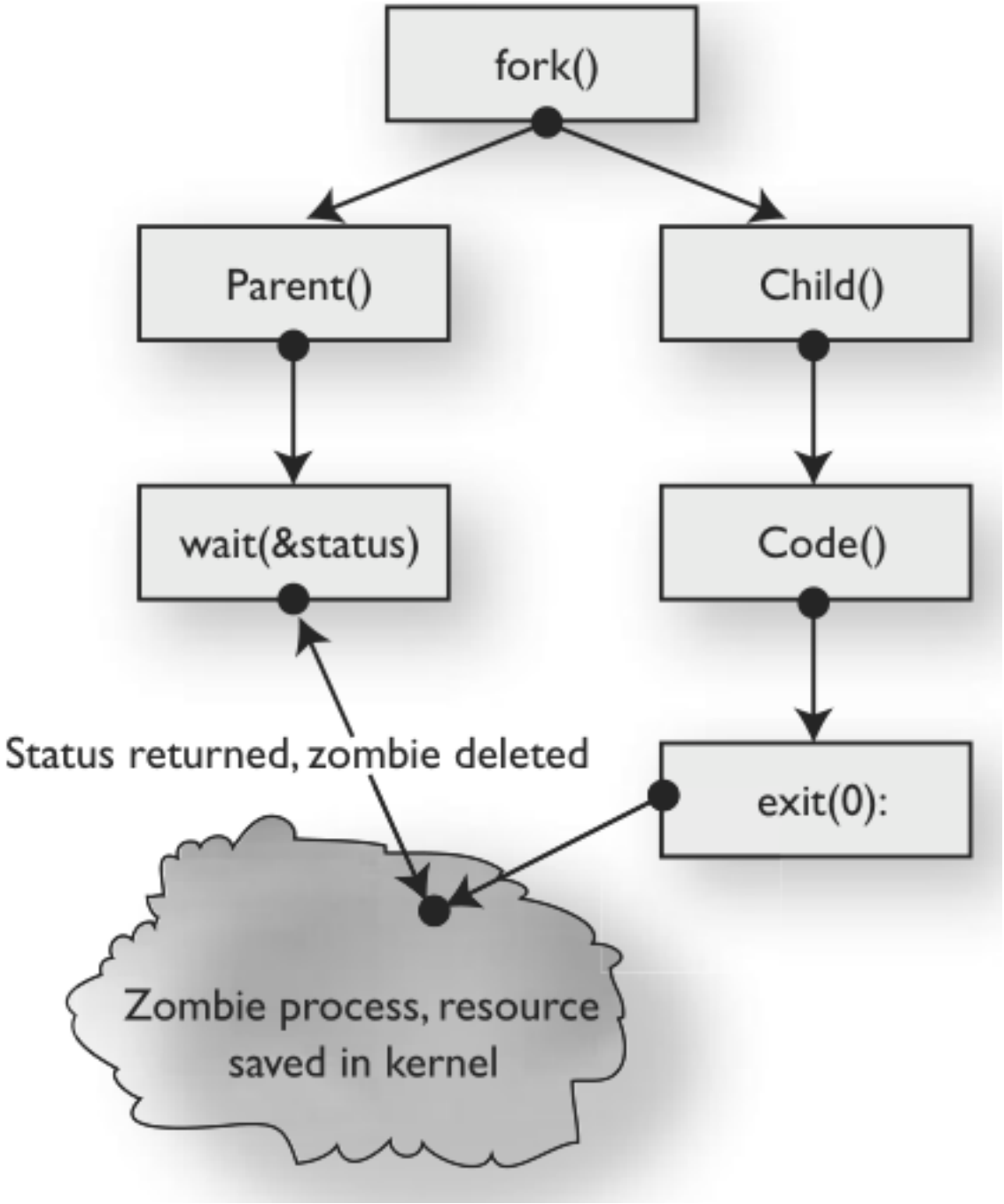


Рисунок 12.1. Жизненный цикл созданного процесса

При написании фаззера, создающего дочерние процессы с помощью `fork`, необходимо убедиться, что родитель получает сведения о завершении всех процессов через `wait` или `waitpid`. Если пропустить завершение процесса, возникнут процессы-зомби.

В ранних версиях notSPIKEfile существовали ошибки, из-за которых со временем число активных процессов постепенно уменьшалось, пока фаззинг не приходил в состояние взаимной блокировки. Например, предположим, что пользователь указал одновременно подвергать приложение фаззингу восемью процессами. Со временем число активных процессов постепенно сокращалось всего до одного. Причиной были две небрежные ошибки автора. Первоначальная архитектура полностью полагалась на сигнал `SIGCHLD`, отправляемый процессу при завершении дочернего процесса. Однако при такой архитектуре некоторые сигналы `SIGCHLD` пропускались. Кроме того, в нескольких местах автор по небрежности забывал уменьшить счётчик активных процессов после завершения дочерних процессов, что приводило к постепенному замедлению фаззинга. Упс!

После обнаружения эти ошибки оказалось совсем не трудно исправить. Все вызовы `wait` и `waitpid`, которые при блокировке могли создавать проблемы, были сделаны неблокирующими с помощью флага `WNOHANG`. После их возврата проверяется полученное состояние, чтобы определить, действительно ли процесс завершился. Если да, счётчик активных процессов всегда уменьшается.

В SPIKEfile нет возможности запускать одновременно несколько приложений, что значительно упростило архитектуру и реализацию. Нам не требовалось беспокоиться о пропущенных сигналах `SIGCHLD`, поскольку за один раз возвращался только один такой сигнал.

Поскольку SPIKEfile основан на SPIKE, нам потребовалось добавить к нему лишь один или два файла, чтобы он мог работать с файловым вводом и выводом вместо одного только сетевого ввода и вывода. Бегло изучив работу SPIKE с TCP/IP, было нетрудно на скорую руку добавить поддержку файлов. Ниже приведено простое дополнение `filestuff.c`:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "filestuff.h"
#include "spike.h"

extern struct spike *current_spike;

int
spike_fileopen(const char *file)
{
    int fd;
    if ((fd = open(file, O_CREAT | O_TRUNC | O_WRONLY,
        S_IRWXU | S_IRWXG | S_IRWXO)) == -1)
        perror("fileopen::open");
    return current_spike->fd = fd;
    current_spike->proto = 69; /* 69==file,0-68 are reserved by the ISO fuzzing standard */
}

int
spike_filewrite(uint32 size, unsigned char *inbuffer)
{
    if (write(current_spike->fd, inbuffer, size) != size)
    {
        perror("filewrite::write");

        return -1;
    }
    return 1;
}

void
spike_close_file()
{
    if (current_spike->fd != -1)
    {
        close(current_spike->fd);
        current_spike->fd = -1;
    }
```

```
}
```

После добавления этого файла в `Makefile` любой знакомый со SPIKE сможет столь же легко использовать в качестве конечной точки файлы. Если вас интересует, какие именно файлы были добавлены к SPIKE, эти сведения приведены в таблице 12.4.

Таблица 12.4. Список изменений SPIKE для создания SPIKEfile

Имя файла	Назначение
`filestuff.c`	Содержит процедуры открытия и записи файлов.
`util.c`	Содержит значительную часть кода, общего для notSPIKEfile и SPIKEfile. В него входят оболочки `ptrace`, основная функция `F_execmon` и некоторые другие полезные функции.
`generic_file_fuzz.c`	Основной исходный файл SPIKEfile. Содержит функцию `main`.
`include/filestuff.h`	Заголовочный файл для `filestuff.c`.
Libdisasm	Библиотека, используемая для дизассемблирования инструкций x86 при сбое.

Примечания по применению

Если вы собираетесь использовать SPIKEfile или notSPIKEfile с приложением, которое нельзя запустить непосредственно, придётся тем или иным способом обойти это ограничение. Хорошими примерами таких приложений служат Adobe Acrobat Reader и RealNetworks RealPlayer.

Обычно у подобных приложений фактически запускаемая программа является сценарием-оболочкой. Сценарий оболочки настраивает среду, чтобы настоящий двоичный файл мог правильно работать. Главным образом это делается для того, чтобы приложения могли включать собственные копии совместно используемых библиотек.

Включение собственных копий распространённых совместно используемых библиотек делает продукт более переносимым. Хотя это делается из любезности к пользователю и должно упростить ему работу, для наших целей оно несколько усложняет дело. Например, если указать фаззеру сценарий оболочки, мы подключимся не к фактически работающему двоичному файлу, а к экземпляру оболочки. Это лишит перехват сигналов всякого смысла. Ниже показано, как обойти эту проблему для Acrobat Reader и RealPlayer. Для всех других подобных приложений идея та же.

Adobe Acrobat

Для Acrobat сначала нужно просто запустить `acroread` с параметром `-DEBUG`. Вы окажетесь в оболочке с правильно настроенной средой, из которой можно непосредственно вызвать настоящий двоичный файл `acroread`, часто расположенный по пути `$PREFIX/Adobe/Acrobat7.0/Reader/intellinux/bin/acroread`. Хотя это не документировано и функция справки отсутствует, мы смогли определить эти сведения, просто прочитав сценарий оболочки `acroread`. Теперь двоичный файл можно без проблем подвергать фаззингу. Этот метод вместе с notSPIKEfile использовался для обнаружения уязвимости переполнения буфера UnixAppOpenFilePerform в Adobe Acrobat Reader.[4]

RealNetworks RealPlayer

Как показывает следующий вывод, команда `realplay` в действительности является сценарием оболочки. Мы также видим, что настоящее двоичное приложение называется `realplay.bin`.

```
user@host RealPlayer $ file realplay realplay.bin
realplay:     Bourne shell script text executable
realplay.bin: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/Linux
2.2.5, dynamically linked (uses shared libs), stripped
```

Для RealPlayer достаточно присвоить переменной среды оболочки `HELIX_PATH` путь установки RealPlayer. После этого можно непосредственно подвергать фаззингу двоичный файл `realplay.bin`, входящий в состав RealPlayer. Эти сведения были получены простым чтением файла сценария оболочки `realplay`. С помощью этого метода и notSPIKEfile была обнаружена уязвимость форматной строки формата RealPix в RealNetworks RealPlayer/HelixPlayer.[5]

Практический пример: уязвимость форматной строки RealPix в RealPlayer

Теперь мы опишем, как notSPIKEfile использовался для обнаружения уязвимости RealPlayer, исправленной в сентябре 2005 года. Первый шаг - произвольно выбрать формат файла, поддерживаемый RealPlayer. Разумеется, в данном случае был выбран формат RealPix. После долгих поисков в Google было собрано несколько файлов-образцов RealPix, которые использовались как исходные файлы для фаззинга notSPIKEfile. Ниже приведён один такой сокращённый пример:[6]

```
<imfl>
<head title="RealPix(tm) Sample Effects"
author="Jay Slagle"
copyright="(c)1998 RealNetworks, Inc."
timeformat="dd:hh:mm:ss.xyz"
duration="46"
bitrate="12000"
width="256"
height="256"
url="http://www.real.com"
aspect="true"/>
</imfl>
```

Это простейший пример минимального файла RealPlayer. Если загрузить его в RealPlayer, ничего не отобразится, поскольку он содержит только заголовок. Мы пропустим его через notSPIKEfile, чтобы проверить код разбора заголовка на ошибки. Для запуска фаззинга используется следующая команда:

```
user@host $ export HELIX_PATH=/opt/RealPlayer/
user@host $ ./notSPIKEfile -t 3 -d 1 -m 3 -r 0- -S -s SIGKILL -o FUZZY-sample1.rp \
sample1.rp "/opt/RealPlay/realplay.bin %FILENAME%"
```

```
[...]
user@host $
```

Параметром `-t` мы указываем инструменту разрешить каждому экземпляру RealPlayer работать три секунды. Параметром `-d` также предписываем ему ждать одну секунду между завершением бездействующего процесса и запуском нового. Параметром `-m` задаём одновременный запуск трёх экземпляров RealPlayer, а параметром `-r` указываем подвергать фаззингу весь файл, начиная с нулевого байта. Параметр `-S` включает режим фаззинга строк, а параметр `-s` задаёт сигнал `SIGKILL` для завершения бездействующих процессов. Наконец, мы сообщаем инструменту формат имён фаззинговых файлов, имя файла-образца `sample1.rp` и способ запуска RealPlayer, при котором тот разбирает наш файл. Всё готово! Вывод notSPIKEfile сообщает о нескольких сбоях, предполагая, что мы нашли некоторую уязвимость.

Просмотрев файлы в текущем каталоге, мы видим, что был создан файл `FUZZY-sample1.rp-0x28ab156b-dump.txt`. В нём находится подробный отчёт о состоянии процесса в момент сбоя. Мы также видим имя вызвавшего сбой файла. В данном случае он сохранён как `12288-FUZZY-sample1.rp`. Этот файл можно использовать для воспроизведения сбоя. При просмотре его содержимого мы получаем хорошую подсказку о возможной причине проблемы:

```
<imfl>
<head title="RealPix(tm) Sample Effects"
author="Jay Slagle"
copyright="(c)1998 RealNetworks, Inc."
timeformat="%n%n%n%n%n%n%n%n%n%n%ndd:hh:mm:ss.xyz"
duration="46"
bitrate="12000"
width="256"
height="256"
url="http://www.real.com"
aspect="true"/>
</imfl>
```

Из-за присутствия символов %n мы сразу подозреваем, что причиной сбоя является уязвимость форматной строки. Наши подозрения подтверждаются после запуска двоичного файла RealPlayer в GDB.

```
user@host ~/notSPIKEfile $ gdb -q /opt/RealPlayer/realplay.bin
Using host libthread_db library "/lib/tls/libthread_db.so.1".
(gdb) r 12288-FUZZY-sample1.rp
Starting program: /opt/RealPlayer/realplay.bin 12288-FUZZY-sample1.rp
Program received signal SIGSEGV, Segmentation fault.
0xb7e53e67 in vfprintf () from /lib/tls/libc.so.6
(gdb) x/i $pc
0xb7e53e67 <vfprintf+13719>:    mov    %ecx,(%eax)
```

Мы действительно обнаружили уязвимость форматной строки в RealPlayer при обработке параметра `timeformat`. Теперь создание эксплойта для этой уязвимости оставлено вам в качестве упражнения.

Язык

Эти инструменты были написаны на C по нескольким логичным причинам. Во-первых, подобно соли и перцу, C и Linux всегда хорошо сочетались и всегда будут хорошо сочетаться друг с другом. В каждом современном дистрибутиве Linux есть компилятор C и, вероятно, будет всегда, поскольку ядро Linux написано на C. Для предоставления нашему приложению интерфейса `ptrace` не требуются специальные библиотеки, и у нас есть вся свобода для выполнения необходимых задач.

Другая причина написания инструментов на C состоит в том, что SPIKE написан на C. Поскольку по меньшей мере один из инструментов активно использует код SPIKE и в идеале эти два

инструмента должны совместно использовать некоторый код, например обработку исключений и составление отчётов, было бы глупо реализовывать общую функциональность на двух разных языках.

Итоги

При наличии надёжных инструментов фаззинга форматов файлов обнаружение клиентских уязвимостей сводится лишь к выбору цели и терпению. Решите ли вы написать собственный фаззер или расширить чужой, затраченное время с лихвой окупится.

Сноски

[1] [назад] http://www.immunityinc.com/resources-freesoftware.shtml
[2] [назад] http://bastard.sourceforge.net/libdisasm.html
[3] [назад] http://www.packetstormsecurity.org/papers/attack/MallocMaleficarum.txt
[4] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=279
[5] [назад] http://www.idefense.com/intelligence/vulnerabilities/display.php?id=311
[6] [назад] http://service.real.com/help/library/guides/realpix/htmfiles/tags.htm

Глава 13. Фаззинг форматов файлов: автоматизация в Windows

> «В интересах нашей страны найти тех, кто хотел бы причинить нам вред, и убрать их с пути вреда».
>
> Джордж У. Буш, Вашингтон, округ Колумбия, 28 апреля 2005 года

В предыдущей главе мы рассмотрели автоматизацию фаззинга форматов файлов на платформе UNIX. Теперь перейдём к обнаружению уязвимостей форматов файлов в приложениях Windows. Хотя общая идея остаётся прежней, существуют важные различия, которые мы постараемся выделить. Прежде всего, программирование в Windows по своей природе располагает к созданию графических инструментов, поэтому мы отойдём от приложений командной строки из предыдущей главы и построим нарядный графический интерфейс для скрипт-кидди. Мы также уделим время выявлению подходящих целевых форматов файлов, что является важным решением при работе в среде Windows с учётом её сильной зависимости от приложений по умолчанию для конкретных типов файлов. Наконец, мы представим решение вечной задачи обнаружения эксплуатируемых состояний.

Уязвимости форматов файлов Windows

Хотя уязвимости форматов файлов могут затрагивать и серверы, чаще они поражают клиентские приложения. Появление уязвимостей форматов файлов обозначило тенденцию к росту важности клиентских уязвимостей. Со временем сетевые администраторы сосредоточили ресурсы на защите корпоративной сети от уязвимостей сетевого уровня. Одновременно поставщики программного обеспечения осознали угрозы серверных уязвимостей, и вместе эти усилия привели к сокращению числа критических серверных уязвимостей в популярных приложениях и операционных системах, которые в прошлом порождали быстро распространявшихся червей, причинявших значительный ущерб.

Однако о тенденциях на стороне клиента того же сказать нельзя. За последние несколько лет мы наблюдали рост числа клиентских уязвимостей, подходящих для целевых атак, а также фишинга и кражи личности.

Уязвимости форматов файлов создают для предприятий особый риск. Хотя такие уязвимости не способствуют появлению быстро распространяющихся червей или мгновенной компрометации сети, во многих отношениях от них труднее защититься. Интернет по своей конструкции поощряет обмен информацией. Веб наполнен фильмами, изображениями, музыкой и документами, которые делают его огромным источником информации и развлечений. Всё это содержимое требует открытого и свободного обмена файлами. Исторически мы не считали файлы вроде изображений или электронных таблиц вредоносными, поскольку сами они не являются исполняемыми. Однако, как обсуждалось ранее, при интерпретации уязвимым приложением они способны привести к эксплуатации. Как же защититься от этой угрозы? Следует ли сетевым администраторам блокировать всё содержимое на межсетевом экране? Веб был бы очень скучным местом, если бы состоял из одного текста. Неужели мы хотим вернуться во времена просмотра Веба текстовым браузером вроде Lynx? Конечно, нет, но нам необходимо осознавать угрозу уязвимостей форматов файлов.

Платформа Windows особенно сильно затронута уязвимостями форматов файлов из-за удобства для пользователя. Типы файлов связаны с приложениями, которые обрабатывают их по умолчанию. Благодаря этому пользователи могут посмотреть фильм или прочитать документ, просто дважды щёлкнув файл. Для просмотра или прослушивания даже не нужно знать, какие приложения способны воспроизводить конкретные типы файлов. Представьте риск уязвимости формата файлов, обнаруженной в приложениях, которые по умолчанию работают в операционной системе Windows. Она немедленно затронет миллионы конечных пользователей. В следующей врезке перечислены некоторые наиболее серьёзные уязвимости форматов файлов, повлиявшие на Microsoft Windows.

ЗНАЧИТЕЛЬНЫЕ УЯЗВИМОСТИ ФОРМАТОВ ФАЙЛОВ, ЗАТРАГИВАВШИЕ MICROSOFT WINDOWS

MS04-028. Переполнение буфера при обработке JPEG (GDI+) могло допустить выполнение кода

В сентябре 2004 года Microsoft выпустила бюллетень безопасности с подробным описанием уязвимости переполнения буфера, возникавшей, когда интерфейс графических устройств GDI+ отображал изображения JPEG, содержащие комментарии недопустимого размера. Комментариям в файлах JPEG предшествуют байты `0xFFFE`, за которыми следует двухбайтовый размер последующего комментария. Минимальный допустимый размер комментария равен двум байтам, поскольку размер комментария включает два байта самого значения размера. Было обнаружено, что нулевой или единичный размер приводит к эксплуатируемому переполнению кучи из-за целочисленного выхода ниже допустимого диапазона. Эта уязвимость затрагивала многочисленные приложения Windows, поскольку библиотека GDI+ (`gdiplus.dll`) используется многими приложениями. После выпуска бюллетеня безопасности быстро появился общедоступный код эксплойта.

MS05-009. Уязвимость обработки PNG могла допустить удалённое выполнение кода

Оказалось, что приложения многих поставщиков, включая Microsoft, уязвимы для переполнений буфера при отображении изображений Portable Network Graphics (PNG) с большими фрагментами `tRNS`, используемыми для прозрачности изображения. Уязвимость затрагивала Windows Messenger и MSN Messenger и заставила Microsoft блокировать доступ уязвимых клиентов к своей сети мгновенных сообщений до установки соответствующего исправления.

MS06-001. Уязвимость механизма отображения графики могла допустить удалённое выполнение кода

Во время праздничного сезона 2005 года начали появляться сообщения о веб-сайтах, размещавших вредоносные файлы Windows Meta File (WMF), которые могли привести к выполнению кода при просмотре страниц в Internet Explorer. Впоследствии из-за масштабов эксплуатации Microsoft была вынуждена в начале 2006 года выпустить внеплановое исправление. Оказалось, что уязвимость вызвана ошибкой проектирования. Файлы WMF содержат записи, позволяющие передавать аргументы определённым вызовам библиотеки Windows Graphics Device Interface (GDI). Один из вызовов, `Escape`, и его подкоманда `SETABORTPROC` фактически позволяли вызывать произвольный исполняемый код. Позднее сообщалось, что сведения об уязвимости были проданы на подпольном рынке за 4000 долларов.[1]

Аукцион уязвимости Excel на eBay

8 декабря 2005 года пользователь «fearwall» разместил на eBay аукцион, предлагавший сведения об уязвимости Microsoft Excel.[2] Объявление привлекло значительное внимание средств массовой информации и было быстро удалено eBay, которая сослалась на политику против содействия незаконной деятельности.[3]

FileFuzz был создан для автоматизации процесса выявления уязвимостей форматов файлов. У FileFuzz три цели. Во-первых, приложение должно быть интуитивно понятным и удобным. Во-вторых, оно должно автоматизировать как создание фаззинговых файлов, так и выполнение приложений, предназначенных для их интерпретации. В-третьих, оно должно включать отладочные возможности, обеспечивающие выявление обработанных и необработанных исключений. Для достижения этих целей мы решили спроектировать приложение на платформе Microsoft .NET. Это позволило создать графическую внешнюю часть на C#, тогда как некоторые внутренние компоненты, например отладчик, были написаны на C. Снимок графического интерфейса показан на рисунке 13.1.

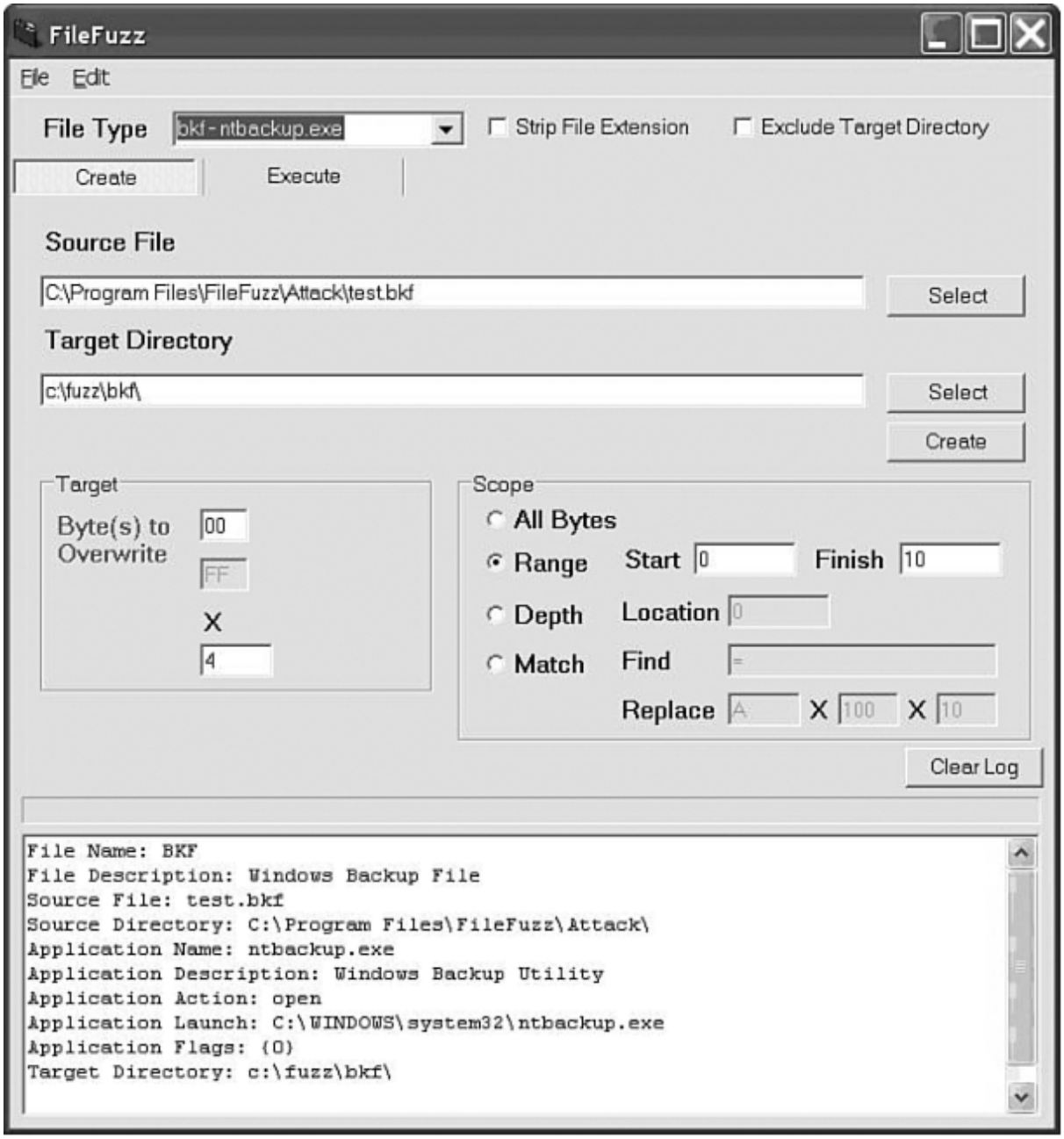


Рисунок 13.1. FileFuzz

Возможности

FileFuzz предназначен для выявления уязвимостей форматов файлов и использует для этого простой, но эффективный подход полного перебора. В двух словах, FileFuzz искажает правильно сформированные файлы, запускает их в приложениях, предназначенных для интерпретации таких файлов, и наблюдает, не возникнут ли проблемы. Возможно, это выглядит не слишком изящно, но работает. При разработке FileFuzz мы были поражены тем, насколько легко удавалось находить потенциально эксплуатируемые состояния в форматах файлов.

FileFuzz проходит три отдельных этапа. Сначала он создаёт файлы, которые предстоит подвергнуть фаззингу. Для этого он берёт допустимый файл, предоставленный пользователем, выполняет рассчитанную последовательность мутаций на основе заданных указаний и сохраняет полученные файлы. Затем изменённые файлы один за другим запускаются в целевом приложении.

Файлы запускаются последовательно, а получившийся процесс в конечном счёте завершается по истечении заданного пользователем времени. Наконец, встроенные средства отладки наблюдают за процессами и выявляют возможные обработанные и необработанные исключения. При обнаружении такие события записываются и сообщаются конечному пользователю. Далее каждый из этих этапов рассматривается подробнее.

Создание файлов

FileFuzz применяет к фаззингу подход полного перебора. Поэтому нам нужна возможность читать допустимый файл, перезаписывать определённые его сегменты и сохранять изменённый файл так, чтобы его могло прочитать приложение, отвечающее за интерпретацию файла. Этот процесс должен быть автоматизирован, поскольку повторяется сотни или тысячи раз.

FileFuzz допускает четыре отдельных подхода к мутации файлов: All Bytes, Range, Depth и Match. Их можно разделить на следующие категории:

• двоичные файлы:
• ширина:
• все байты;
• диапазон;
• глубина;
• текстовые файлы ASCII:
• совпадение.

Как видно, FileFuzz способен работать с двоичными и текстовыми форматами ASCII. В двоичных файлах мы используем два отдельных подхода: ширину и глубину. Чтобы объяснить разницу между ними, воспользуемся аналогией с бурением нефтяных скважин. Если вы ищете нефть на большой географической территории, нельзя просто сразу начать бурить. Сначала нужно различными методами выявить места, где с наибольшей вероятностью находится искомое сокровище. Возможно, вы изучаете карты, анализируете горные породы или используете георадар. Независимо от подхода, обнаружив интересные места, можно начать бурить разведочные скважины и выяснять, какие из них наиболее перспективны.

При фаззинге форматов файлов мы используем похожий подход. Сначала применяем ширину, чтобы найти интересные позиции, а затем глубину, чтобы определить, добрались ли мы до чёрного золота. Ширина означает покрытие всех байтов файла или определённого диапазона. Отдельные файлы создаются последовательным изменением значений байтов в диапазоне на заранее определённое значение до покрытия всего диапазона. После этого файлы запускаются один за другим, чтобы определить, приводят ли изменения к исключениям.

Иногда нам везёт, и идти дальше не приходится. Порой уже на этом этапе возникают явно интересные исключения и очевидно, что конечный пользователь управляет сбоем, поскольку значение внесённой мутации хорошо видно в регистрах. Но чаще всё не так просто. Сбой возникает, и его место может быть интересным, однако по значениям регистров непонятно, способен ли конечный пользователь каким-либо образом им управлять. Тогда мы переходим к глубине. Выявив интересные позиции байтов в файле, мы сосредоточиваемся на них и с помощью глубины перебираем все возможные значения байта в данной позиции. Анализируя полученные сбои, можно оценить степень управления сбоем. Если место исключения и значения регистров остаются одинаковыми независимо от переданных байтовых значений, мы не управляем исключением. Но если сбой возникает в разных местах или значения регистров постоянно меняются, становится ясно, что значения, используемые при мутации файла, дают нам по меньшей мере некоторое влияние на возникшее исключение.

FileFuzz поддерживает и текстовые файлы ASCII. Для этого конечному пользователю сначала разрешается выбрать строку, определяющую место перезаписи, а затем запрашиваются три отдельных значения для определения входных данных мутации файла. Конечный пользователь должен указать строковое значение, его длину и число повторений. Рассмотрим пример. Обычно текстовые файлы ASCII, например файлы `*.ini`, содержат пары «имя - значение» следующего формата:

```
name = value
```

Как правило, нам нужно перезаписать значение. Предположим, мы хотим последовательно перезаписывать значение символами `A` с шагом 10. В этом случае сначала зададим в поле Find символ `=`, поскольку он отмечает начало значения. Затем установим Replace в `A X 10 X 10`. Это создаст 10 изменённых файлов с перезаписанными значениями для каждого найденного символа `=`. Получившиеся 10 файлов будут содержать от 10 до 100 символов `A` на месте значения.

Выполнение приложения

После создания фаззинговых файлов их нужно запустить в целевом приложении. Например, если мы изменили файлы `*.doc`, затем потребуется открыть их в Microsoft Word. Для этого FileFuzz использует функцию `CreateProcess()` из Windows API, поэтому FileFuzz может запустить приложение, если мы способны определить, как то же приложение запускается из командной строки, включая необходимые передаваемые ему флаги. Позднее в этой главе подробно объясняется, как получить эти сведения.

При фаззинге форматов файлов приходится сотни или даже тысячи раз запускать одно и то же приложение. Если оставлять все ранее запущенные процессы работающими, доступная память быстро закончится. Поэтому этап выполнения завершается лишь после принудительного завершения процесса через заранее определённый интервал. Мы позволяем конечному пользователю управлять им с помощью поля Milliseconds на вкладке Execute, которое задаёт время работы процесса перед его принудительным завершением при необходимости.

Обнаружение исключений

Как упоминалось в предыдущих главах, обнаружение - ключевая составляющая фаззинга. Какой смысл запускать фаззер на ночь, а утром выяснить, что вам действительно удалось вызвать сбой приложения, но вы не представляете, какое из 10 000 входных значений стало причиной? Вы ничуть не ближе к ответу, чем до начала работы. Фаззинг форматов файлов предоставляет множество способов обнаружения исключений, но один выделяется среди остальных. Для начала можно использовать самый примитивный подход и просто наблюдать за фаззингом, выявляя окна ошибок или, возможно, сбой приложения либо системы. Если вы любите смотреть, как сохнет краска, этот подход для вас. Если хочется подняться на ступень выше, можно проверить файлы журналов и определить, возникла ли проблема. Это включает как журналы приложений, так и системные журналы, например поддерживаемые «Просмотром событий» Windows. Недостаток подхода состоит в том, что входные данные, фаззинговый файл, не связаны с результатом, событием журнала. Единственный способ сопоставить их - временные метки, а он в лучшем случае неточен.

Как и для большинства видов фаззинга, лучший подход к выявлению исключений - применение отладчика. Его преимущество в том, что он обнаруживает как обработанные, так и необработанные исключения. Windows обладает развитой обработкой исключений и часто способна восстановиться после чтения фаззинговых файлов. Однако такие ситуации важно выявлять, поскольку небольшое

изменение файла в соответствующей позиции может создать неустранимое или эксплуатируемое состояние.

Использовать отладчик при фаззинге форматов файлов не так просто, как при других видах фаззинга. В данном случае нельзя вручную подключить отладчик к целевому приложению и оставить фаззер работать. Причина в том, что фаззер постоянно запускает и завершает целевое приложение, одновременно завершая и отладчик. Поэтому нужно использовать API отладки и встроить отладчик непосредственно в фаззер. Тогда фаззер сможет запустить целевое приложение, самостоятельно подключить отладчик, а затем завершить приложение. Таким образом фаззер будет отслеживать исключения при каждом запуске целевого приложения.

Для FileFuzz мы создали `crash.exe`, фактически самостоятельный отладчик, который запускается графическим приложением и, в свою очередь, запускает целевое приложение. Это полностью самостоятельное приложение командной строки, а поскольку FileFuzz является проектом с открытым исходным кодом, вы можете использовать `crash.exe` в собственных проектах фаззинга.

Сохранённые аудиты

Аудит в FileFuzz можно провести вручную, выбрав цель фаззинга и указав, как следует создавать и хранить фаззинговые файлы. Другой вариант - использовать ранее сохранённый аудит, чтобы заполнить все необходимые поля и сразу начать фаззинг. В приложение входит несколько готовых аудитов, доступных в раскрывающемся меню File Types на главном экране. Однако конечные пользователи могут также создавать собственные сохранённые аудиты и добавлять их в раскрывающееся меню без повторной компиляции приложения. Это возможно потому, что меню динамически создаётся во время выполнения путём разбора файла `targets.xml`. Структура отдельного аудита показана в следующем примере:

```
<test>
<name>jpg - iexplore.exe</name>
<file>
<fileName>JPG</fileName>
<fileDescription>JPEG Image</fileDescription>
</file>
<source>
<sourceFile>gradient.jpg</sourceFile>
<sourceDir>C:\WINDOWS\Help\Tours\htmlTour\</sourceDir>
</source>
<app>
<appName>iexplore.exe</appName>
<appDescription>Internet Explorer</appDescription>
<appAction>open</appAction>
<appLaunch>"C:\Program Files\Internet Explorer\iexplore.exe"</appLaunch>
<appFlags>{0}</appFlags>

</app>
<target>
<targetDir>c:\fuzz\jpg\</targetDir>
</target>
</test>
```

Включение динамически создаваемого раскрывающегося меню было осознанным архитектурным решением. Хотя FileFuzz является приложением с открытым исходным кодом, предполагается, что большинство пользователей не обладает опытом программирования или интересом к расширению возможностей дополнительным написанием кода. Подход с динамическим меню позволяет большинству конечных пользователей расширять функциональность относительно удобным способом. Структура XML-файла показывает важный принцип: делайте приложения настолько интуитивно понятными и удобными, насколько возможно. Хотя документация является ключевой частью процесса разработки, не ожидайте, что пользователи первым делом обратятся к ней.

Пользователи ждут интуитивно понятных приложений и хотят начать работу сразу после установки.

Взгляните на приведённый выше XML-файл. Описательные теги XML понятны сами по себе. Конечный пользователь может добавить аудит, просто добавив и настроив дополнительный блок `<test>`. Хотя создание интуитивно понятных приложений не служит законным оправданием пренебрежению документацией, это важный принцип проектирования. Современные языки программирования позволяют создавать удобные приложения, беря на себя низкоуровневый код, например сетевой обмен или графическое отображение. Вложите сэкономленное время в создание приложения, обеспечивающего конечному пользователю более приятную работу. То, что вы проектируете приложение безопасности, ещё не означает, что для его использования должна требоваться докторская степень.

Необходимые исходные сведения

Фаззинг форматов файлов в Windows и UNIX опирается на одни и те же основы, но важность приложений по умолчанию в среде Windows может повышать риск этого класса уязвимостей. Поэтому перед созданием фаззера мы уделим некоторое время исследованию данного вопроса, чтобы лучше понять, как выявлять цели высокого риска.

Выявление целей

Microsoft Windows позволяет пользователям назначать приложения по умолчанию для конкретных типов файлов. Это повышает удобство операционной системы, позволяя автоматически запускать приложение после двойного щелчка файла. Этот важный момент следует помнить при выборе целевых форматов. Риск минимален, если переполнение буфера найдено в приложении, которое вряд ли когда-либо будет использоваться для просмотра данного типа файла, но он значителен, когда та же уязвимость обнаружена в приложении, по умолчанию открывающем широко распространяемый файл. Возьмём, например, переполнение в формате изображений JPEG. Как и для всех графических форматов, существует множество приложений, способных отобразить такое изображение, но в конкретной операционной системе с типом файла по умолчанию связано только одно. В Windows XP стандартное средство просмотра JPEG - Windows Picture and Fax Viewer. Поэтому переполнение, обнаруженное в Windows Picture and Fax Viewer, несёт гораздо больший риск, чем такое же переполнение в бесплатном средстве просмотра изображений, загруженном с Download.com. Почему? Если уязвимым оказывается стандартное приложение Windows, немедленно становятся уязвимыми миллионы машин.

Проводник Windows

Как выяснить, какие приложения отображают определённый тип файла в среде Windows? Простой подход - дважды щёлкнуть файл и посмотреть, какое приложение запустится. Для быстрой проверки это годится, но не помогает определить точные команды, выполненные для запуска приложения. Эти команды важны при фаззинге, поскольку нам нужен способ автоматизировать непрерывный запуск целевого приложения. Быстрые двойные щелчки не подходят, когда требуется подвергнуть фаззингу несколько тысяч файлов.

Проводник Windows - быстрый и простой способ выявить приложения, связанные с типами файлов, и аргументы командной строки, используемые для запуска приложения. Воспользуемся Проводником, чтобы доказать связь файлов JPEG с Windows Picture and Fax Viewer. Что ещё важнее, определим, как многократно запускать фаззинговые файлы JPEG в FileFuzz для выявления

уязвимостей.

Сначала необходимо выбрать в меню Проводника Tools > Folder Options. Затем нужно открыть вкладку File Types. Появится экран, показанный на рисунке 13.2.

Уже один этот экран является кладезем сведений. Просмотр списка Registered file types показывает почти все расширения файлов, связанные с конкретными приложениями. Эти типы файлов служат хорошими целями фаззинга, поскольку уязвимости стандартных приложений можно эксплуатировать, просто отправив файл жертве и убедив её дважды его щёлкнуть. Хотя это не совсем просто, спам доказал, что конечные пользователи и без того легко нажимают всё подряд, поэтому сценарий эксплуатации вполне реалистичен.

Благодаря надписи Opens with нам уже известно приложение, связанное с типом JPEG, но неизвестно, как именно операционная система запускает это приложение. К счастью, от жизненно важных сведений нас отделяет всего несколько щелчков мыши.

При обработке типов файлов Windows использует понятие действий. Действия позволяют открывать файл иным способом или другим приложением с уникальными параметрами командной строки. Для наших целей нужно узнать, как Windows открывает файлы JPEG, поэтому мы сосредоточимся на действии Open. Как показано на рисунке 13.3, выделение Open и нажатие кнопки Edit раскрывает ответ.

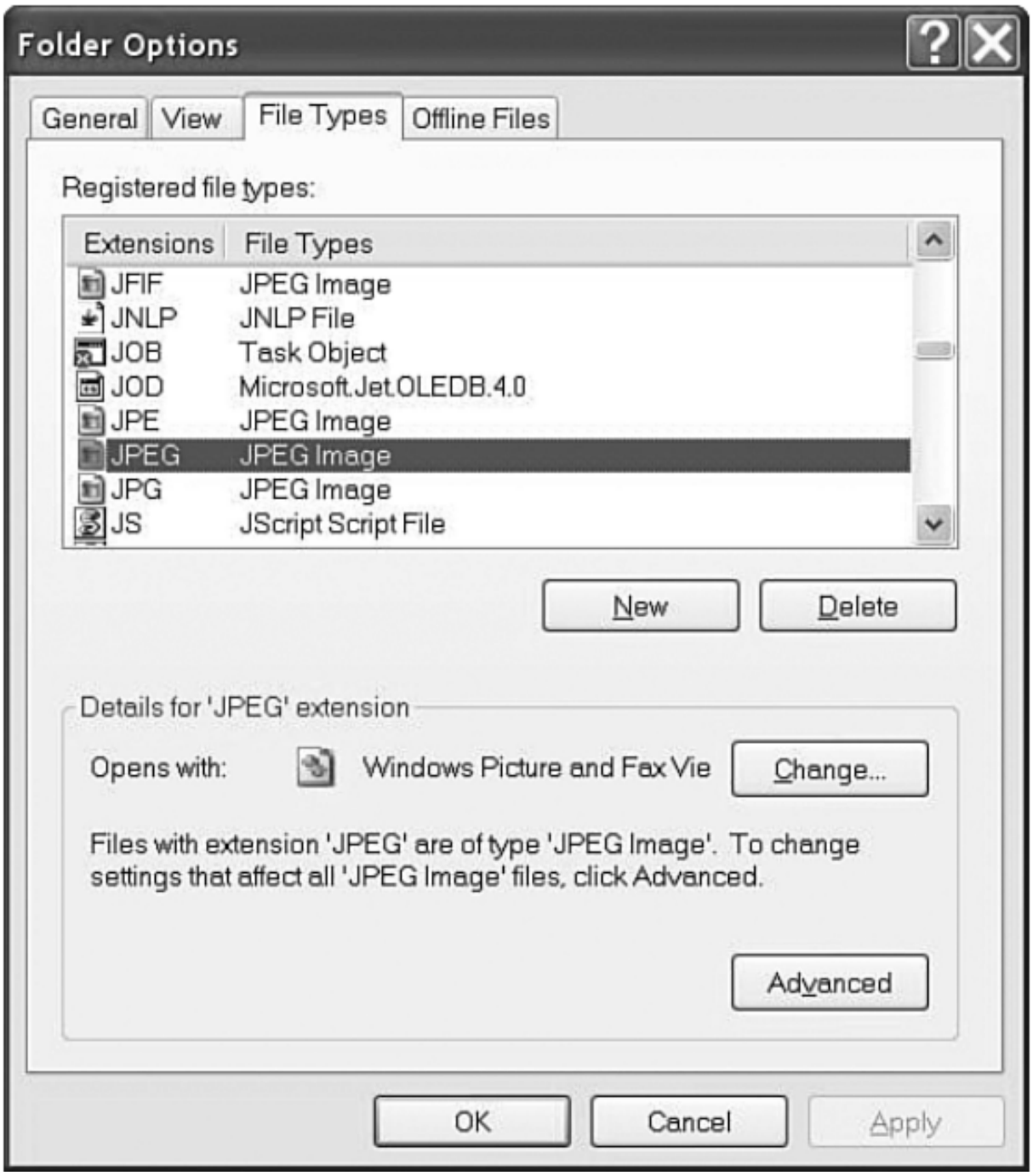


Рисунок 13.2. Диалоговое окно Folder Options Проводника Windows



Рисунок 13.3. Диалоговое окно Edit File Type Проводника Windows

Наконец-то! Windows раскрывает искомый секрет. В поле Application used to perform action, показанном на рисунке 13.4, видно, что Windows Picture and Fax Viewer вовсе не является исполняемым приложением. На самом деле это динамически подключаемая библиотека (DLL), запускаемая с помощью `rundll32.exe`. Полная команда запуска изображения в Windows Picture and Fax Viewer выглядит так:

```
rundll32.exe C:\WINDOWS\system32\shimgvw.dll,ImageView_Fullscreen %1
```

Windows Picture and Fax Viewer не только оказался не исполняемым файлом, как можно было ожидать, но мы также видим, что Windows требует аргумент `ImageView_Fullscreen`. Если выполнить эту строку в командной строке и заменить `%1` именем допустимого файла JPEG, файл действительно отобразится в Windows Picture and Fax Viewer, как и ожидалось. Это ключевой принцип. Если мы способны определить правильные аргументы командной строки для отображения файла данным приложением, то можем использовать FileFuzz для проверки уязвимостей. Теперь ту же команду можно скопировать в поля Application и Arguments на вкладке Execute в FileFuzz. Единственное необходимое изменение - заменить `%1`, обозначающий целевой файл, на `{0}`, поскольку именно такого формата ожидает FileFuzz.

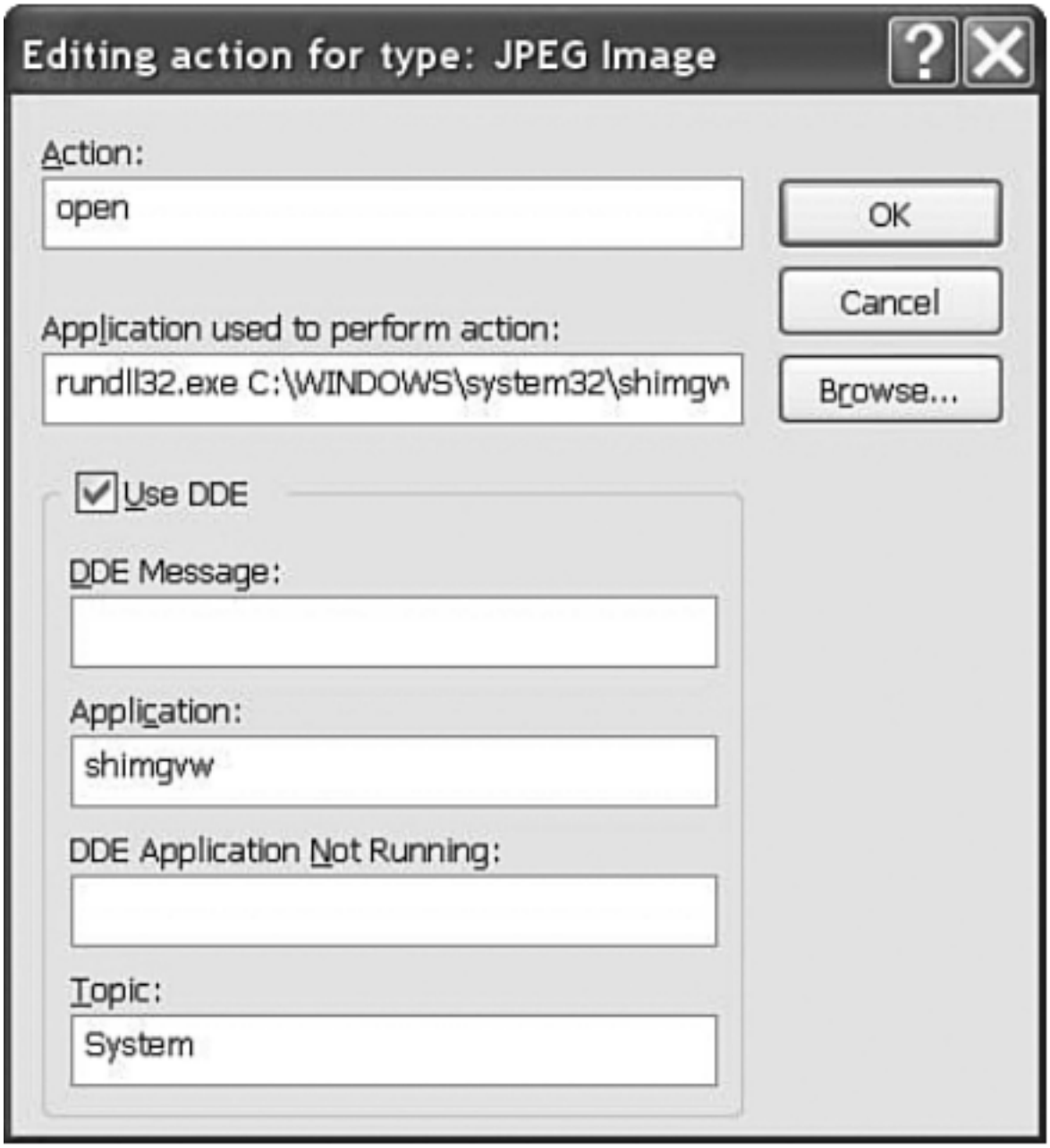


Рисунок 13.4. Диалоговое окно Editing Action for Type Проводника Windows

Небольшой совет: Windows может быть очень привередлива в отношении пробелов и кавычек в аргументах командной строки. Обязательно копируйте команду в точности, чтобы позднее избежать мучительной отладки.

Реестр Windows

Хотя Проводник Windows позволяет выявить связь между конкретным типом файла и приложением примерно в 90 процентах случаев, мы встречали ситуации, когда связь существовала, но в Проводнике не показывалась. Возьмём, например, тип файлов *.cbo. Файлы CBO используются приложением Microsoft Interactive Training, которое по умолчанию входит в некоторые дистрибутивы Windows XP, например поставлявшиеся со многими машинами Dell. Если у вас есть машина с Microsoft Interactive Training, вы заметите, что тип CBO отсутствует в списке File Types Проводника Windows, однако сам файл CBO показывается в Проводнике со значком карандаша и после двойного щелчка действительно запускает Microsoft Interactive Training. В чём дело? Как определить аргументы командной строки для запуска типа файла, не показанного в Проводнике? Для этого нужно обратиться к реестру Windows. Сначала проверьте значение поля по умолчанию ключа реестра \HKEY_CLASSES_ROOT.xxx, где xxx - соответствующее расширение файла. Это значение сообщает имя приложения, используемого для запуска файла. Затем найдите ключ реестра HKEY_CLASSES_ROOT`, соответствующий тому же описательному имени. В ключе ...\shell\open\command` находятся подробности аргументов командной строки для запуска приложения, связанного с загадочным расширением файла.

Разработка

Теперь, когда мы познакомились с особенностями фаззинга форматов файлов на платформе Windows, пришло время создать фаззер. Мы разберём ход мыслей при проектировании FileFuzz, а затем в завершение используем его для выявления широко обсуждавшейся уязвимости JPEG в GDI+, описанной в бюллетене безопасности Microsoft MS04-028.

Подход

При создании FileFuzz мы стремились разработать удобное графическое приложение, позволяющее конечному пользователю выполнять фаззинг без изучения ряда аргументов командной строки. Мы хотели получить максимально интуитивно понятное приложение, управляемое щелчками мыши. Кроме того, нашей целью было создать инструмент, специально учитывающий особенности фаззинга форматов файлов на платформе Windows, поэтому кроссплатформенность не являлась необходимой возможностью.

Выбор языка

Учитывая цели проектирования, мы снова выбрали для разработки платформу .NET. Она позволила с минимальными усилиями создать графическую внешнюю часть, освободив время для сосредоточения на функциональной стороне проекта. Графический интерфейс и все возможности создания файлов были реализованы на C#. Для отладочных возможностей мы перешли на C, поскольку он позволял проще и непосредственнее взаимодействовать с Windows API. Платформа .NET поддерживала такое архитектурное решение, поскольку разрешает проектам содержать несколько языков программирования при условии, что все они совместимы с .NET Framework.

Архитектура

Мы уже достаточно обсудили, как будем проектировать FileFuzz. Теперь пора разобрать конкретные части реализации. Описывать каждый участок кода непрактично, но мы выделим несколько наиболее важных сегментов. Для полного понимания архитектуры FileFuzz рекомендуем загрузить исходный код с www.fuzzing.org.

Создание файлов

FileFuzz должен был поддерживать любой формат файлов Windows, и мы поняли, что для двоичных и текстовых файлов ASCII потребуются разные подходы. Файл `Read.cs` содержит весь код чтения допустимых файлов, а `Write.cs` отвечает за создание фаззинговых файлов.

Чтение исходных файлов

FileFuzz использует подход полного перебора, поэтому мы начинаем с чтения заведомо исправных файлов. Данные допустимого файла считываются и сохраняются для последующей мутации при создании фаззинговых файлов. К счастью, .NET Framework делает такие задачи, как чтение файлов, относительно безболезненными. Для чтения и создания двоичных и текстовых файлов ASCII мы использовали разные подходы. В нашем случае для чтения двоичных файлов и сохранения данных в массиве байтов применялся класс `BinaryReader`. Чтение текстовых файлов ASCII очень похоже на чтение двоичных, однако для него мы использовали класс `StreamReader`. Кроме того, результат сохраняется в строковой переменной, а не в массиве байтов. Ниже показан конструктор класса `Read`.

```
private BinaryReader brSourceFile;
private StreamReader arSourceFile;
public byte [] sourceArray;
public string sourceString;

private int sourceCount;
private string sourceFile;

public Read(string fileName)
{
    sourceFile = fileName;
    sourceArray = null;
    sourceString = null;
    sourceCount = 0;
}
```

Переменная `sourceArray` будет хранить массив байтов считываемых двоичных файлов, а `sourceString` - содержимое текстового файла ASCII.

Запись фаззинговых файлов

Теперь, когда файлы прочитаны, нужно изменить их и сохранить получившиеся файлы для запуска в целевом приложении. Как уже упоминалось, FileFuzz делит создание файлов на следующие четыре типа в зависимости от подхода к мутации:

• все байты;
• диапазон;
• глубина;
• совпадение.

Все различные подходы обрабатываются в одном классе `Write`, но конструкторы перегружены для разных сценариев. Для двоичных типов файлов мы применяем класс `BinaryWriter`, записывающий новый массив байтов в файл, который будет использоваться для фаззинга целевого приложения на этапе выполнения. При создании текстовых файлов ASCII, напротив, используется класс `StreamWriter`, записывающий строковые переменные на диск.

Выполнение приложения

Код, отвечающий за начало процесса выполнения целевого приложения, находится в файле `Main.cs`, как показано в следующем сегменте. Однако при взгляде на него становится понятно, что он относительно прост, поскольку на самом деле отвечает не за запуск самого целевого приложения, а за запуск встроенного отладчика, который, в свою очередь, выполняет целевое приложение. Отладчик `crash.exe` подробно обсуждается далее.

Сначала мы создаём новый экземпляр класса `Process`. Затем внутри функции `executeApp()` запускаем цикл для каждого ранее созданного фаззингового файла. На каждом проходе цикла задаются атрибуты создаваемого процесса, включая имя выполняемого процесса. Как уже говорилось, независимо от объекта фаззинга это всегда будет `crash.exe`, поскольку приложение командной строки `crash.exe`, в свою очередь, запускает целевое приложение. На этом этапе управление передаётся `crash.exe`, а его результаты в конце концов возвращаются через стандартный вывод и стандартный поток ошибок и показываются в поле форматированного текста `rtbLog`, служащем главным окном вывода FileFuzz.

```
Process proc = new Process();

public Execute(int startFileInput, int finishFileInput,
    string targetDirectoryInput, string fileExtensionInput,
    int applicationTimerInput, string executeAppNameInput,
    string executeAppArgsInput)
{
    startFile = startFileInput;
    finishFile = finishFileInput;
    targetDirectory = targetDirectoryInput;
    fileExtension = fileExtensionInput;
    applicationTimer = applicationTimerInput;
    executeAppName = executeAppNameInput;
    executeAppArgs = executeAppArgsInput;
    procCount = startFile;
}

public void executeApp()
{
    bool exceptionFound = false;
    // Инициализировать индикатор выполнения
    if (this.pbrStart != null)
    {
        this.pbrStart(startFile, finishFile);
    }
    while (procCount <= finishFile)
    {
        proc.StartInfo.CreateNoWindow = true;
        proc.StartInfo.UseShellExecute = false;
        proc.StartInfo.RedirectStandardOutput = true;
        proc.StartInfo.RedirectStandardError = true;
        proc.StartInfo.FileName = "crash.exe";
        proc.StartInfo.Arguments = executeAppName + " " + applicationTimer + " " +
            String.Format(executeAppArgs, @targetDirectory + procCount.ToString() +
            fileExtension);
        proc.Start();
        // Обновить индикатор выполнения
        if (this.pbrUpdate != null)
```

```
        {
            this.pbrUpdate(procCount);
        }
        // Обновить счётчик
        if (this.tbxUpdate != null)
        {
            this.tbxUpdate(procCount);
        }
        proc.WaitForExit();
        // Записать стандартный вывод в журнал rich text box
        if (this.rtbLog != null && (proc.ExitCode == -1 || proc.ExitCode == 1))
        {
            this.rtbLog(proc.StandardOutput.ReadToEnd());
            this.rtbLog(proc.StandardError.ReadToEnd());
            exceptionFound = true;
        }
        procCount++;
    }
    // Очистить индикатор выполнения
    if (this.pbrStart != null)
    {
        this.pbrStart(0, 0);
    }
    // Очистить счётчик
    if (this.tbxUpdate != null)
    {
        this.tbxUpdate(0);
    }
    if (exceptionFound == false)
        this.rtbLog("No excpetions found\n\n");
    exceptionFound = false;
}
```

Обнаружение исключений

Как уже упоминалось, FileFuzz включает отладочные возможности в виде `crash.exe`, самостоятельного отладчика на C, использующего функции отладки, встроенные в Windows API. Он также применяет libdasm, открытую библиотеку, помогающую интерпретировать дизассемблированный код. Как видно в следующем сегменте, сначала выполняется проверка, что `crash.exe` получил не менее трёх аргументов. При использовании в FileFuzz этими аргументами являются имя и путь приложения, подвергаемого фаззингу, время ожидания до принудительного завершения целевого приложения и, наконец, дополнительные аргументы командной строки вместе с именем интерпретируемого фаззингового файла. Затем значение времени ожидания преобразуется из строки в целое число, а полная команда создаётся и сохраняется в массиве символов. После этого целевое приложение запускается командой `CreateProcess` с установленным флагом `DEBUG_PROCESS`.

```
if (argc < 4)
{
    fprintf(stderr, "[!] Usage: crash <path to app> <milliseconds> <arg1> "
        "[arg2 arg3 argn]\n\n");
    return -1;
}

// преобразовать время ожидания из строки в целое число.
if ((wait_time = atoi(argv[2])) == 0)
{
    fprintf(stderr, "[!] Milliseconds argument unrecognized: %s\n\n", argv[2]);
    return -1;
}

// создать строку командной строки для вызова CreateProcess().
strcpy(command_line, argv[1]);
for (i = 3; i < argc; i++)
```

```
{
    strcat(command_line, " ");
    strcat(command_line, argv[i]);
}

//
// запустить целевой процесс.
//
ret = CreateProcess(NULL,     // имя целевого файла.
    command_line,             // параметры командной строки.
    NULL,                     // атрибуты процесса.
    NULL,                     // атрибуты потока.
    FALSE,                    // дескрипторы не наследуются.
    DEBUG_PROCESS,            // отлаживать целевой процесс и все порождённые дочерние
    процессы.
    NULL,                     // использовать текущую среду.
    NULL,                     // использовать текущий рабочий каталог.
    &si,                      // указатель на структуру STARTUPINFO.
    &pi);                     // указатель на структуру PROCESS_INFORMATION.

printf("[*] %s\n", GetCommandLine());  // Вывести командную строку

if (!ret)
{
    fprintf(stderr, "[!] CreateProcess() failed: %d\n\n", GetLastError());
    return -1;
}
```

Теперь `crash.exe` может отслеживать и регистрировать исключения. В следующем сегменте видно, что до истечения времени ожидания мы следим за событиями отладки. При появлении события мы получаем дескриптор соответствующего потока и определяем тип возникшего исключения. С помощью конструкции `switch` ищем три интересующих типа исключений: нарушения доступа к памяти, ошибки деления на ноль и переполнения стека. Затем выводим важные отладочные сведения, которые помогут конечному пользователю решить, заслуживает ли исключение дальнейшего анализа. С помощью библиотеки libdasm мы декодируем место возникновения исключения, вызвавший его код операции и значения регистров в момент сбоя.

```
while (GetTickCount() - start_time < wait_time)
{
    if (WaitForDebugEvent(&dbg, 100))
    {
        // нас интересуют только события отладки.
        if (dbg.dwDebugEventCode != EXCEPTION_DEBUG_EVENT)
        {
            ContinueDebugEvent(dbg.dwProcessId, dbg.dwThreadId, DBG_CONTINUE);
            continue;
        }
        // получить дескриптор потока, вызвавшего нарушение.
        if ((thread = OpenThread(THREAD_ALL_ACCESS, FALSE, dbg.dwThreadId)) == NULL)
        {
            fprintf(stderr, "[!] OpenThread() failed: %d\n\n", GetLastError());
            return -1;
        }
        // получить контекст потока, вызвавшего нарушение.
        context.ContextFlags = CONTEXT_FULL;
        if (GetThreadContext(thread, &context) == 0)
        {
            fprintf(stderr, "[!] GetThreadContext() failed: %d\n\n", GetLastError());
            return -1;
        }

        // исследовать код исключения.
        switch (dbg.u.Exception.ExceptionRecord.ExceptionCode)
        {
            case EXCEPTION_ACCESS_VIOLATION:
                exception = TRUE;
                printf("[*] Access Violation\n");
                break;
            case EXCEPTION_INT_DIVIDE_BY_ZERO:
```

```
                exception = TRUE;
                printf("[*] Divide by Zero\n");
                break;
            case EXCEPTION_STACK_OVERFLOW:
                exception = TRUE;
                printf("[*] Stack Overflow\n");
                break;
            default:
                ContinueDebugEvent(dbg.dwProcessId, dbg.dwThreadId, DBG_CONTINUE);
        }
        // если возникло исключение, вывести дополнительные сведения.
        if (exception)
        {
            // открыть дескриптор целевого процесса.
            if ((process = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
                dbg.dwProcessId)) == NULL)
            {
                fprintf(stderr, "[!] OpenProcess() failed: %d\n\n", GetLastError());
                return -1;
            }
            // получить участок памяти по EIP для дизассемблирования.
            ReadProcessMemory(process, (void *)context.Eip, &inst_buf, 32, NULL);
            // декодировать инструкцию в строку.
            get_instruction(&inst, inst_buf, MODE_32);
            get_instruction_string(&inst, FORMAT_INTEL, 0,
                inst_string, sizeof(inst_string));
            // вывести исключение на экран.
            printf("[*] Exception caught at %08x %s\n", context.Eip, inst_string);
            printf("[*] EAX:%08x EBX:%08x ECX:%08x EDX:%08x\n",
                context.Eax, context.Ebx, context.Ecx, context.Edx);
            printf("[*] ESI:%08x EDI:%08x ESP:%08x EBP:%08x\n\n",
                context.Esi, context.Edi, context.Esp, context.Ebp);

            return 1;
        }
    }
}
```

Сведения об исключении, выявленные `crash.exe`, возвращаются графическому интерфейсу и показываются конечному пользователю. Будем надеяться, что эта информация даст ему быстрый визуальный признак, помогающий выявлять важные сбои. К сбоям, заслуживающим дальнейшего исследования, относятся исключения в инструкциях с кодами операций, которые позволили бы шелл-коду получить управление ходом выполнения кода, а также исключения, при которых регистры содержат управляемые пользователем или зависящие от него входные данные.

Практический пример

Теперь, когда мы разработали фаззер форматов файлов, проверим его на известной уязвимости, чтобы подтвердить правильность архитектуры. Во многом наш интерес к уязвимостям форматов файлов был вызван выпуском Microsoft бюллетеня безопасности MS04-028 «Buffer Overrun in JPEG Processing (GDI+) Could Allow Code Execution».[4] Этот бюллетень привлёк огромное внимание, поскольку наглядно показал, насколько разрушительными могут быть клиентские уязвимости. В ряде популярных клиентских приложений, устанавливавшихся по умолчанию и имевших огромную пользовательскую базу, существовало эксплуатируемое переполнение буфера. Результатом стало быстрое появление общедоступных эксплойтов, которые, хотя и зависели от некоторой социальной инженерии, подходили как для целевых атак, так и для неизбирательного фишинга и кражи личности.

Сама уязвимость находилась в библиотеке `gdiplus.dll`, которую использовало множество приложений, включая Microsoft Office, Internet Explorer и Проводник Windows. Формат JPEG позволяет встраивать комментарии в само изображение. Комментариям предшествует

последовательность байтов `0xFFFE`, за которой следует 16-разрядное слово, обозначающее общий размер комментария. Размер включает два байта, используемые для самого размера, а заголовок заканчивается собственно комментарием.

Смог бы FileFuzz выявить эту уязвимость? Давайте выясним. Начнём с допустимого файла изображения, а затем добавим комментарий вручную или с помощью редактора изображений. Если вы решите самостоятельно повторить этот пример, убедитесь, что используете уязвимую версию Windows. Наши результаты были получены в Windows XP SP1. Для тестового файла воспользуемся очень простым изображением, в нашем случае одним белым пикселем размером 1 x 1. Почему настолько простым? Как уже говорилось, фаззинг полным перебором неэффективен. Мы хотим сосредоточиться на заголовках изображения, а не на нём самом, поэтому используем изображение, которого практически нет. В результате получается файл размером всего 631 байт, и мы можем за разумное время выполнить полный перебор всех его байтов. Добавив комментарий `fuzz` и открыв файл в шестнадцатеричном редакторе, мы увидим следующую последовательность байтов:

```
0000009eh: FF FE 00 06 66 75 7A 7A                        ; ÿþ..fuzz
```

Разбор:

Байты	**Значение**
`FF FE`	Префикс комментария
`00 06`	Длина комментария в байтах
`66 75 7A 7A`	Значение ASCII строки `fuzz`

Прежде чем начинать фаззинг, нужно выяснить, какое приложение Windows XP по умолчанию отображает изображения JPEG. К счастью, ранее в этой главе мы уже определили, что за эту задачу отвечает Windows Picture and Fax Viewer, использующий для запуска изображения JPEG следующие аргументы командной строки (см. рисунок 13.5):

```
rundll32.exe C:\WINDOWS\system32\shimgvw.dll,ImageView_Fullscreen %1
```

Теперь можно запустить FileFuzz и начать фаззинг. В FileFuzz есть встроенный аудит файлов JPEG, доступный через раскрывающееся меню File Type, но для демонстрации возможностей FileFuzz начнём с нуля.

Откроем вкладку Create и зададим параметры для создания серии изменённых файлов JPEG на основе допустимого файла JPEG со встроенным комментарием, созданного ранее. На вкладке Create установим следующие значения:

• **Source file:** `C:\Program Files\FileFuzz\Attack\test.jpg`. Допустимый файл JPEG.
• **Target directory:** `C:\fuzz\jpg`. Целевой каталог для сохранения созданных фаззинговых файлов.
• **Byte(s) to overwrite:** `00 x 2`. Согласно подробным сведениям об уязвимости,[5] переполнение возникает, если установить длину в `0x00` или `0x01`. Комментарий не может иметь длину ноль или один байт, поскольку двухбайтовый размер входит в общую длину. Поэтому мы подвергнем файл фаззингу значением слова `0x0000` в надежде вызвать переполнение, перезаписав значение размера комментария.
• **Scope:** `Range = 150-170`. В созданном нами тестовом файле значение размера начинается с байта 160. Для надёжности подвергнем фаззингу диапазон байтов от 150 до 170.

Окончательные настройки показаны на рисунке 13.5.

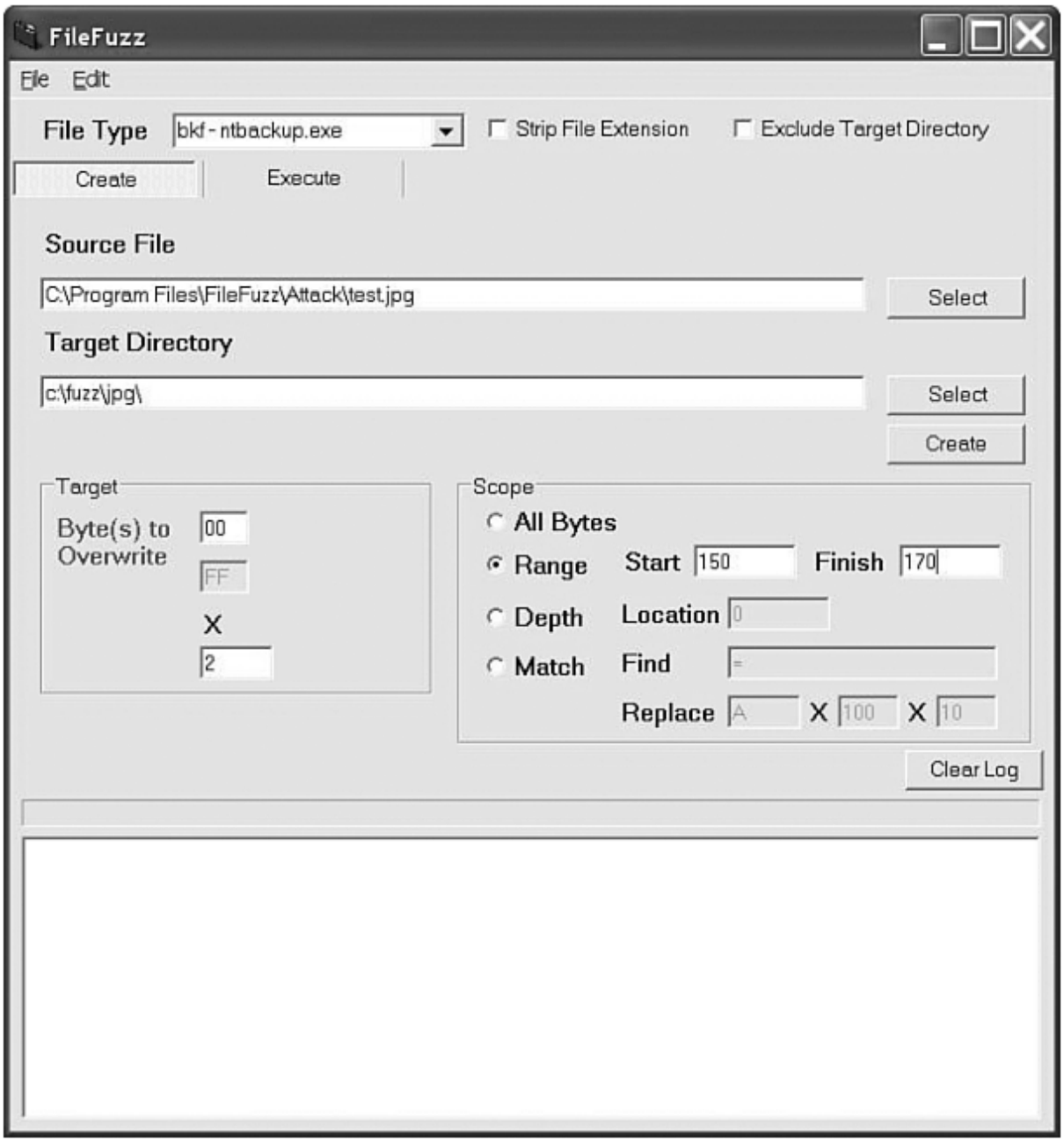


Рисунок 13.5. Настройки вкладки Create в FileFuzz для фаззинга JPEG

После задания всех параметров нажимаем Create, чтобы создать файлы. Теперь пора перейти на вкладку Execute. Здесь необходимо сообщить FileFuzz, как запускать Windows Picture and Fax Viewer. Значения на вкладке Execute будут следующими:

• **Application:** `rundll32.exe`. Поскольку Windows Picture and Fax Viewer фактически является DLL, используемое приложение - `run32.exe` (так в оригинале), которое запускает файл DLL.
• **Arguments:** `C:\WINDOWS\system32\shimgvw.dll,ImageView_Fullscreen {0}`. Передаваемые аргументы включают полный путь к Windows Picture and Fax Viewer (`shimgvw,dll`, так в оригинале), аргумент `ImageView_FullScreen` и `{0}`, служащий заполнителем имени открываемого фаззингового файла.
• **Start file:** `150`. Первый созданный файл.
• **Finish file:** `170`. Последний созданный файл.
• **Milliseconds:** `2000`. Время работы Windows Picture and Fax Viewer до принудительного закрытия.

Соответствующие настройки показаны на рисунке 13.6.

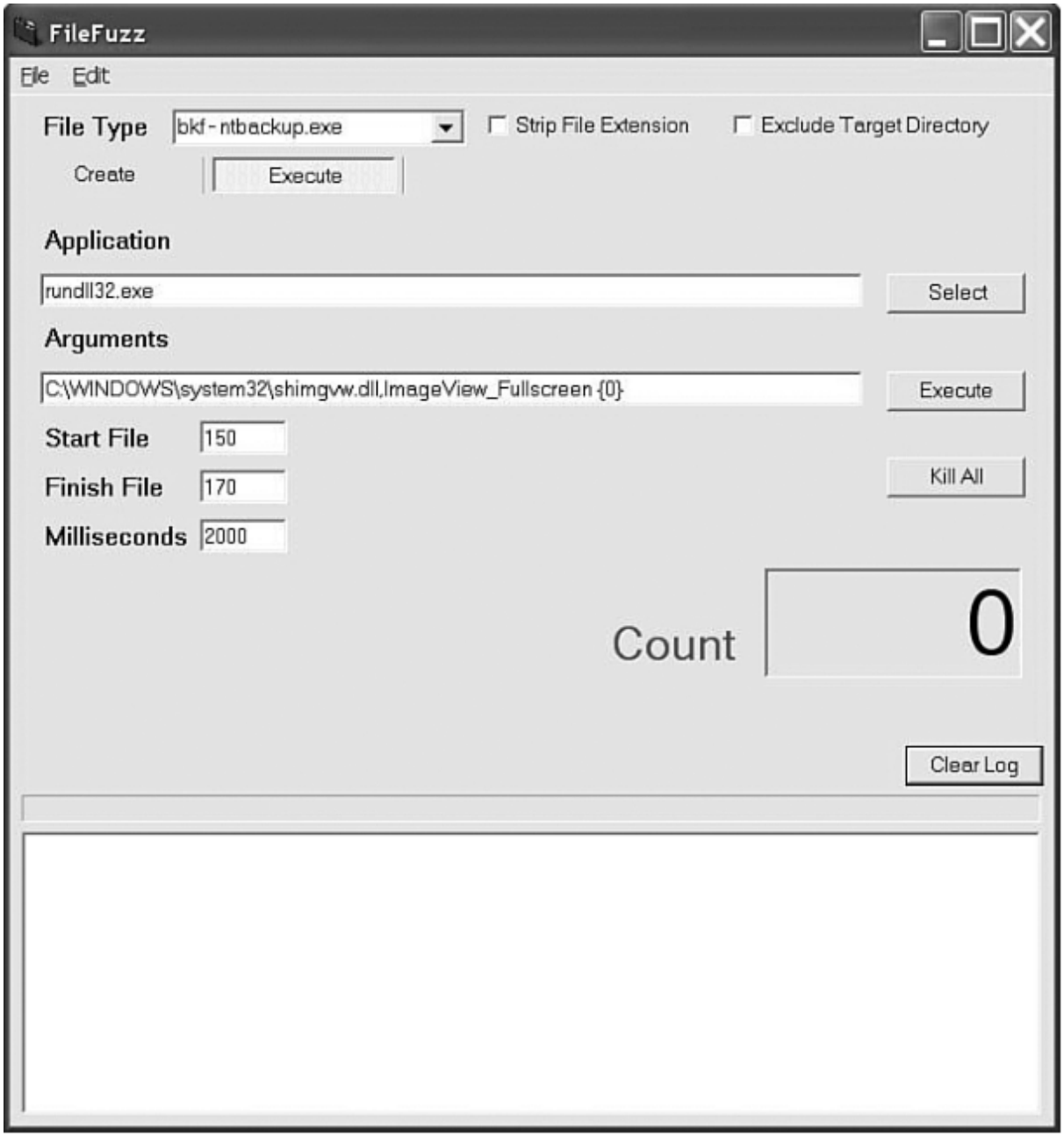


Рисунок 13.6. Настройки вкладки Execute в FileFuzz для фаззинга JPEG

Теперь всё готово. После нажатия Execute мы видим, как Windows Picture and Fax Viewer многократно запускается и закрывается. Это произойдёт 21 раз, пока открываются и интерпретируются наши 21 фаззинговый файл. Когда дым рассеивается, мы видим, что FileFuzz обнаружил несколько исключений. Однако интересующее нас исключение возникает при запуске `160.jpg`, как показано в следующем образце вывода. Оно интересно потому, что байт 160 является началом размера комментария JPEG, а в файле `160.jpg` исходное значение перезаписано значением `0x0000`.

```
[*] "crash.exe" rundll32.exe 2000
C:\WINDOWS\system32\shimgvw.dll,ImageView_Fullscreen c:\fuzz\jpg\160.jpg
[*] Access Violation
[*] Exception caught at 70e15599 rep movsd
[*] EAX:fffffffe EBX:00904560 ECX:3ffffe3c EDX:fffffffe
[*] ESI:0090b07e EDI:0090c000 ESP:00aaf428 EBP:00aaf43400
```

Преимущества и возможности улучшения

FileFuzz предоставляет базовые возможности для фаззинга форматов файлов методом полного перебора. Графический интерфейс и встроенные отладочные возможности позволяют быстро провести грубый аудит, используя в качестве отправной точки заведомо исправный файл. Тем не менее возможностей для улучшения предостаточно.

Прежде всего, можно разработать более комплексные средства аудита. В существующем виде одновременно выполняется только один аудит. Например, при использовании подхода ширины для двоичного типа файла за один раз можно выполнить лишь один проход по диапазону байтов с одним байтовым значением, например `0xFFFFFFFF`. Если затем нужно проверить другое значение, процесс приходится запускать снова с новым значением. Более полная функция аудита могла бы разрешать выбор нескольких значений или даже диапазонов значений, которые проверялись бы все. Возможно, можно добавить некоторую интеллектуальную функциональность, которая сначала применяла бы подход ширины, а затем автоматически переходила к глубине после выявления позиций байтов, вызвавших более определённого количества или типа исключений.

FileFuzz намеренно проектировался как фаззер полного перебора, поскольку фаззинг форматов файлов хорошо подходит для такого более простого подхода. Однако это не означает, что нельзя добавить возможности интеллектуального фаззинга. Возможно, в дополнение к существующей вкладке Create, которая тогда стала бы вкладкой Create - Brute Force, можно добавить новую вкладку Create - Intelligent. Она включала бы совершенно новый набор возможностей интеллектуального фаззинга, требующих от конечного пользователя разработать шаблон структуры конкретного типа файла вместо использования существующего файла как отправной точки. Такой подход потребовал бы от пользователя больше предварительных усилий, но позволил бы точнее выявлять конкретные области файла для фаззинга и способы их обработки. Шаблон, вероятно, создавался бы после изучения спецификаций целевого формата файла, однако FileFuzz мог бы также содержать встроенные шаблоны-образцы.

Интеллектуальная обработка исключений помогла бы отсеять множество исключений, которые вряд ли приведут к эксплуатируемым состояниям. Если при анализе вывода `crash.exe` применять встроенные правила, определённые исключения можно исключать на основе таких факторов, как коды операций по адресу сбоя, значения регистров или состояние стека. Вместо исключения определённых результатов, возможно, лучше было бы выделять те, которые выглядят более перспективно.

Короче говоря, возможностей для улучшения множество. Наша цель - лишь сдвинуть дело с мёртвой точки. Остальное зависит от вас.

Итоги

В последние пару лет уязвимости форматов файлов преследовали Microsoft. Возникали ли они в медиафайлах или документах Office, они появлялись целыми группами и использовались злоумышленниками. Это не означает, что Microsoft является единственным поставщиком, испытывающим трудности с этим классом уязвимостей, а лишь то, что она по-прежнему остаётся любимой целью охотников за ошибками. Будем надеяться, что недавнее внимание к уязвимостям форматов файлов заставит поставщиков программного обеспечения включить фаззинг в жизненный цикл разработки, чтобы обнаруживать такие уязвимости до выпуска в рабочую среду.

Сноски

[1] [назад] http://www.securityfocus.com/brief/126
[2] [назад] http://www.osvdb.org/blog/?p=71
[3] [назад] http://www.theregister.co.uk/2005/12/10/ebay_pulls_excel_vulnerability_auction/
[4] [назад] http://www.microsoft.com/technet/security/Bulletin/MS04-028.mspx
[5] [назад] http://www.securityfocus.com/archive/1/375204

Глава 14. Фаззинг сетевых протоколов

> «Я владею лесозаготовительной компанией? Для меня это новость. Дров не надо?»
>
> Джордж У. Буш, вторые президентские дебаты, Сент-Луис, штат Миссури, 8 октября 2004 года

Фаззинг родился в Висконсинском университете, когда утилитам UNIX командной строки с `setuid` начали передавать случайные аргументы. Несмотря на эту первоначальную связь, сегодня под термином «фаззинг» обычно понимают применение к сетевым протоколам, и на то есть веская причина. Фаззинг сетевых протоколов является наиболее интересным для исследователей безопасности способом передачи фаззинговых данных, поскольку обнаруженные уязвимости имеют наивысшую критичность. Удалённо эксплуатируемая уязвимость, для достижения которой не требуются допустимые учётные данные, а для эксплуатации - взаимодействие целевого пользователя, является вершиной открытий, если хотите, золотой медалью.

Клиентские уязвимости, например затрагивающие Microsoft Internet Explorer, обычно используются для создания ботнетов. Злоумышленники забрасывают широкую сеть в надежде поймать как можно больше рыбы. По большей части такой пойманной рыбой оказываются домашние настольные компьютеры с широкополосным подключением. Серверная уязвимость сетевого демона может быть столь же полезной для построения ботнетов, но применять её таким образом означает растрачивать потенциал. С точки зрения злоумышленника, получение контроля над программой вроде внутренней базы данных или корпоративного веб-сервера открывает прекрасную возможность кражи данных, а также предоставляет доверенную платформу для дальнейших атак. В этой главе мы представим фаззинг сетевых протоколов и некоторые уникальные свойства и трудности этой популярной категории фаззинга. По пути мы исследуем ряд уязвимостей сетевых протоколов.

Что такое фаззинг сетевых протоколов?

Как и в других видах фаззинга, при фаззинге сетевых протоколов необходимо выявить поверхность атаки, изменить или создать фаззинговые значения, вызывающие ошибки, передать эти значения цели и следить за сбоями цели. Проще говоря, если фаззер обменивается данными с целью через сокет того или иного типа, это фаззер сетевых протоколов.

Сокетный обмен при сетевом фаззинге имеет интересную особенность: он создаёт узкое место в пропускной способности тестирования. Сравнивать скорость передачи при фаззинге форматов файлов, аргументов командной строки и переменных среды со скоростью сети - всё равно что устраивать гонку Aston Martin DB9 с Geo Metro.

Существующие фаззеры сетевых протоколов обычно бывают двух видов. Одни являются универсальными основами, способными подвергать фаззингу множество протоколов. К этой категории относятся такие инструменты, как SPIKE[1] и ProtoFuzz, фаззер, который мы создадим с нуля в главе 16 «Фаззинг сетевых протоколов: автоматизация в Windows». SPIKE - самый

известный фаззер; подробнее он описан в следующей главе. Другой вид включает фаззеры сетевых протоколов, предназначенные для конкретного протокола. Примеры этой категории - ircfuzz,[2] dhcpfuzz[3] и Infigo FTPStress Fuzzer.[4] Уже по названию можно догадаться, для каких протоколов предназначен каждый из упомянутых фаззеров. Фаззеры конкретных протоколов обычно представляют собой небольшие целевые сценарии или приложения, тогда как основы, как правило, являются более крупными проектами разработки. В главе 21 «Основы фаззинга» мы обсудим преимущества создания и применения основ фаззинга по сравнению с отдельными специализированными инструментами. Теперь рассмотрим несколько примеров целей фаззинга сетевых протоколов.

MICROSOFT ПОДХВАТЫВАЕТ ВИРУС

Для большинства граждан мира, не страдающих параноидной шизофренией, новое тысячелетие стало временем праздника. Мы наслаждались крупнейшими в истории новогодними вечеринками, пережили страх перед проблемой 2000 года и увидели, что конец света не наступил. К несчастью для Microsoft, новое тысячелетие принесло начало нескольких лет, в течение которых во всех самых популярных продуктах Microsoft обнаруживались крупные серверные уязвимости. Оказалось, что для многих из них быстро разрабатывался и открыто публиковался код эксплойтов. Кроме того, ряд этих уязвимостей использовал быстро распространяющийся вредоносный код, причиняя существенный финансовый ущерб корпорациям по всему миру и становясь тревожным звонком для системных администраторов повсюду.

Рассмотрим несколько известных червей, затронувших Microsoft, и уязвимости, которые они эксплуатировали для распространения по всему миру. Репутационный ущерб от этих червей стал одним из непосредственных катализаторов создания Microsoft инициативы Trustworthy Computing, запущенной в 2002 году[5] и фундаментально изменившей подход Microsoft к безопасности на всём протяжении жизненного цикла разработки программ.

- **Code Red.** Переполнение буфера в расширении Internet Server Application Programming Interface (ISAPI) веб-сервера IIS было раскрыто 18 июня 2001 года.[6] Несмотря на доступность исправления почти месяц, 13 июля 2001 года был обнаружен червь, эксплуатировавший дефект для изменения веб-сайтов на уязвимых серверах сообщением `HELLO! Welcome to http://www.worm.com! Hacked By Chinese!`. После заражения червь ожидал от 20 до 27 дней, а затем пытался провести атаку DoS против различных фиксированных IP-адресов, включая IP-адрес `whitehouse.gov`.
- **Slammer.** Червь SQL Slammer эксплуатировал две отдельные уязвимости Microsoft SQL Server и Desktop Engine, описанные в бюллетенях безопасности Microsoft MS02-039[7] и MS02-061.[8] Когда он впервые появился 25 января 2003 года, ему потребовалось около десяти минут, чтобы заразить первые 75 000 жертв.[9] Эксплуатированное червём переполнение буфера (MS02-039) Microsoft раскрыла и исправила приблизительно за шесть месяцев до этого, но множество уязвимых серверов были и остаются доступными.
- **Blaster.** 11 августа 2003 года 18-летний Джеффри Ли Парсон (Jeffrey Lee Parson) поделился с миром своим творением: червём,[10] использовавшим переполнение буфера удалённого вызова процедур (RPC) DCOM в Windows XP и Windows 2000.[11] И снова исправления уже были доступны. Червь содержал возможность проведения распределённой атаки DoS путём SYN-флуда против `windowsupdate.com`. За свои усилия Парсон был вознаграждён 18 месяцами заключения, тремя годами освобождения под надзором и 100 часами общественных работ.[12]

Этот список далеко не исчерпывающий, но показывает некоторые наиболее значительные исторические серверные уязвимости Microsoft, породившие быстро распространявшихся червей. Любую из этих сетевых уязвимостей потенциально можно было обнаружить с

помощью сетевого фаззинга.

Цели

На выбор существуют тысячи целей, способных открыть удалённо эксплуатируемые уязвимости разбора сетевых протоколов. В таблице 14.1 приведена небольшая выборка известных сетевых уязвимостей, показывающая распространённые категории целей.

Таблица 14.1. Распространённые категории уязвимых приложений и примеры ранее обнаруженных уязвимостей форматов файлов

Категория приложения	Название уязвимости	Бюллетень
Почтовые серверы	Уязвимость удалённой обработки сигналов Sendmail	http://xforce.iss.net/xforce/alerts/id/216
Серверы баз данных	Уязвимость обхода аутентификации MySQL	http://archives.neohapsis.com/archives/vulnwatch/2004-q3/0001.html
Службы на основе RPC	Уязвимость переполнения буфера RPC DCOM	http://www.microsoft.com/technet/security/bulletin/MS03-026.mspx
Службы удалённого доступа	Уязвимость удалённого запроса OpenSSH	http://bvlive01.iss.net/issEn/delivery/xforce/alertdetail.jsp?oid=20584
Медиасерверы	Уязвимость `../` в `DESCRIBE` RealServer	http://www.service.real.com/help/faq/security/rootexploit082203.html
Серверы резервного копирования	Уязвимость переполнения буфера Message Engine в CA BrightStor ARCserve Backup	http://www.zerodayinitiative.com/advisories/ZDI-07-003.html

Эти шесть категорий образуют хороший набор пулов целей. Однако потенциальной целью может быть любое приложение или служба, принимающая входящие соединения. Сюда входят аппаратные устройства, сетевые принтеры, КПК, мобильные телефоны и так далее. Любую

программу, способную принимать сетевой трафик, можно проверять методами сетевого фаззинга.

Для более полного понимания типичных целей мы приведём примеры для каждого из семи уровней, определённых эталонной моделью взаимодействия открытых систем, моделью OSI,[13] показанной на рисунке 14.1. Хотя модель OSI никогда не была непосредственно реализована в одной сетевой технологии, её широко используют как точку отсчёта при разборе сетевых технологий, чтобы показать положение конкретной функциональности относительно других сетевых уровней. Теоретически фаззинг сетевых протоколов может быть направлен на любой из семи уровней OSI. На практике встречаются фаззеры всех уровней, кроме уровня 1, физического. При описании каждого уровня для наглядности приводится относящаяся к нему историческая уязвимость.

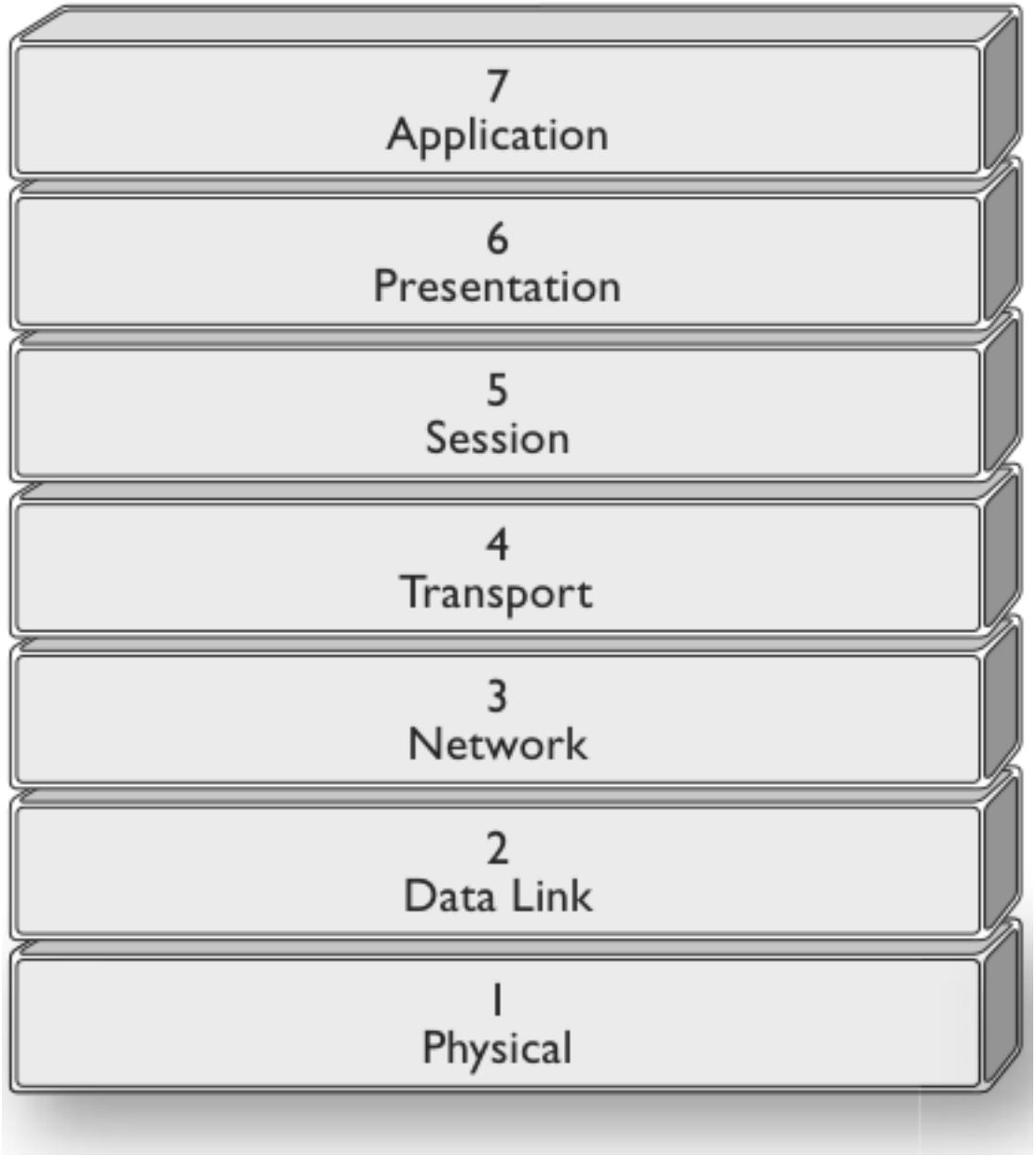


Рисунок 14.1. Модель взаимодействия открытых систем (OSI)

Уровень 2: канальный уровень

К важным технологиям канального уровня относятся кадры Ethernet и 802.11. Уязвимости уровня 2 интересны потому, что обработка этого низкоуровневого сетевого уровня часто реализована в ядре операционной системы. Пример недавней уязвимости уровня 2 был обозначен Mitre как CVE-2006-3507.[14] В этой уязвимости злоумышленник может скомпрометировать систему Mac OS с поддержкой беспроводной сети AirPort из-за нескольких переполнений буфера в стеке. Переполнения возникают в контексте ядра и, хотя эксплуатация нетривиальна, могут использоваться для полной компрометации затронутой системы. Интересное требование состоит в том, что злоумышленник должен находиться в зоне действия выбранной беспроводной сети. Если вы подключены к общедоступной беспроводной сети в кафе и ваша система Mac OS выходит из строя, возможно, разумно оглядеть помещение и взять в заложники для допроса ближайшего посетителя с ноутбуком.

APPLEGATE

На конференции Black Hat 2006 года в Лас-Вегасе исследователи безопасности Джон «Johnny Cache» Элч (Jon Ellch) и Дэвид Мейнор (David Maynor) вызвали серьёзные споры, опубликовав видеозапись, демонстрировавшую удалённую компрометацию Apple Macbook через предполагаемую уязвимость беспроводных драйверов.[15] Последовавшая шумиха в СМИ сопровождалась потоком взаимных обвинений всех участников, включая заявление Apple о том, что исследователи так и не предоставили достаточных сведений об уязвимости для проверки заявления. Верные поклонники Apple также вступили в бой, заявив о нечестности, поскольку, судя по всему, эксплуатировалась уязвимость стороннего драйвера, а не написанного разработчиками Apple.

В конце концов Apple выпустила ряд исправлений, включая CVE-2006-3507, но продолжает утверждать, что уязвимости были найдены внутренними аудитами, начатыми вследствие демонстрации на Black Hat. Мейнору и Элчу так и не приписали обнаружение дефектов. Единственное, что можно сказать наверняка об этой чрезмерно драматизированной цепочке событий: вероятно, мы никогда не узнаем правду.

Уровень 3: сетевой уровень

Уровень 3, сетевой уровень, включает IP и Internet Control Message Protocol (ICMP). Хотя наиболее распространённые реализации TCP/IP за долгие годы были хорошо проверены, уязвимости на этом уровне всё ещё встречаются. Стоит также упомянуть, что Windows Vista содержит полностью переписанный сетевой стек, а значит, может быть хорошей целью фаззинга. Недавняя уязвимость TCP/IP описана в MS06-032 «Vulnerability in TCP/IP Could Allow Remote Code Execution».[16] Она проявляется в ядре из-за неправильного разбора параметров маршрутизации от источника в IP версии 4. Неправильная обработка приводит к переполнению буфера, которое удалённые злоумышленники могут эксплуатировать для получения доступа на уровне ядра.

Уровень 4: транспортный уровень

На один уровень выше расположен уровень 4, транспортный, включающий TCP и UDP. Как уже упоминалось, большинство реализаций TCP/IP хорошо проверены, но в прошлом проблемы встречались и на этом уровне. Прекрасный пример уязвимости здесь - старая атака «winnuke», использовавшая внеполосные пакеты TCP.[17] Пожалуй, winnuke была самой простой из когда-либо существовавших удалённых атак DoS на ядро. Для удалённого аварийного завершения затронутой системы требовалось лишь передать пакет TCP с установленным указателем срочности. Этот указатель легко установить через любой API сокетов, просто указав, что пакет содержит внеполосные данные.

Уровень 5: сеансовый уровень

На сеансовом уровне, уровне 5 модели OSI, расположены два особенно проблемных протокола, оба реализующие удалённые вызовы процедур. Это DCE/RPC (MSRPC компании Microsoft) и ONC RPC, также известный как Sun RPC. Эти два протокола являются реализациями соответственно для систем Windows и UNIX. В прошлом через эти технологии было раскрыто множество критических уязвимостей. Одна из наиболее значительных была исправлена в бюллетене безопасности Microsoft MS04-011[18] и использовалась червём Sasser[19] для распространения в реальных системах. Уязвимость находилась в двоичном файле `lsass.exe`, который предоставлял конечные точки RPC, зарегистрированные по умолчанию во всех современных версиях операционной системы Windows. В частности, уязвимость возникала из-за переполнения буфера в функции `DsRolerUpgradeDownlevelServer` и могла эксплуатироваться удалённо.

Уровень 6: уровень представления

Хорошим примером технологии уровня 6, прекрасно подходящей для фаззинга, является XDR, eXternal Data Representation, используемое в Sun RPC. Среди обнаруженных в прошлом уязвимостей XDR особенно хорошим примером служит целочисленное переполнение `xdr_array`, найденное Нилом Мехтой (Neel Mehta).[20] В основе открытия лежало злоупотребление целочисленным переполнением. Если злоумышленник задавал большое число элементов массива, мог выделяться буфер недостаточного размера, в который затем выполнялась запись и который переполнялся. Злоумышленники могут использовать это повреждение памяти для компрометации нижележащей системы.

Уровень 7: прикладной уровень

Уровень 7, прикладной, является уровнем OSI, о котором чаще всего думают и который чаще всего выбирают целью при фаззинге сетевых протоколов. Здесь находятся распространённые протоколы FTP, SMTP, HTTP, DNS и множество других стандартизованных и проприетарных протоколов. Исторически на этом уровне обнаружено больше уязвимостей, чем на любом другом. Уровень 7 чаще всего подвергается фаззингу, поскольку для распространённых протоколов существует множество различных реализаций.

Оставшаяся часть главы главным образом посвящена уровню 7, но важно помнить, что программное обеспечение любого уровня может содержать ошибки реализации, и при тщательном аудите цели его никогда не следует игнорировать.

Методы

Доступные методы фаззинга по большей части совпадают с методами, представленными в главе 11 «Фаззинг форматов файлов». На высоком уровне к фаззингу форматов файлов и сетевому фаззингу можно подходить методом полного перебора либо интеллектуальным методом. Однако при применении к сетевому фаззингу эти подходы имеют несколько различий.

Фаззинг полным перебором, или на основе мутаций

В контексте фаззинга форматов файлов полный перебор требует от специалиста получить допустимые образцы целевого формата. Затем приложение-фаззер изменяет эти файлы различными способами и передаёт каждый тестовый пример целевому приложению. При сетевом фаззинге специалист обычно использует анализатор трафика, чтобы перехватить допустимый протокольный обмен статически заранее или динамически во время выполнения. После этого фаззер изменяет перехваченные данные и отправляет их цели. Разумеется, это не всегда так просто, как может показаться. Рассмотрим, например, любой протокол с базовой защитой от атак повторного воспроизведения. В подобных ситуациях простой фаззинг полным перебором будет неэффективен для всего, кроме кода инициализации сеанса, например процесса аутентификации.

Другой пример, в котором простой фаззер на основе мутаций потерпит неудачу, - попытка фаззинга протокола со встроенными контрольными суммами. Если поля контрольной суммы не обновляются фаззером динамически, целевая программа может отбросить переданные данные до любого глубокого разбора. В таких ситуациях тестовые примеры расходуются впустую. Фаззинг на основе мутаций хорошо работает с файлами, но для сетевых протоколов в целом лучше подходит следующий метод.

Фаззинг интеллектуальным полным перебором, или на основе генерации

При интеллектуальном фаззинге полным перебором сначала необходимо потратить усилия на изучение спецификаций протокола. Интеллектуальный фаззер остаётся механизмом фаззинга и потому всё ещё проводит атаку полным перебором. Однако он опирается на конфигурационные файлы пользователя, что делает процесс более интеллектуальным. Обычно эти файлы содержат метаданные, описывающие язык протокола.

При фаззинге простого сетевого протокола, например управляющего канала FTP, интеллектуальный фаззинг часто является лучшим решением. Простая грамматика, описывающая каждую команду протокола (`USER`, `PASS`, `CWD` и так далее), и описания типов данных каждого аргумента команд позволяют провести довольно полную проверку. Разработку фаззера на основе генерации можно вести поверх существующей основы, например PEACH,[21] либо с нуля. В последнем случае создаётся фаззер конкретного протокола, который по большей части нельзя перенастроить для других протоколов. При таком подходе любые соображения универсальной архитектуры отбрасываются, что упрощает проектирование и реализацию.

В открытом доступе существует несколько инструментов, направленных на один протокол, однако многие средства проверки остаются закрытыми, поскольку фактически могут рассматриваться как заряженное оружие. Иными словами, когда автор выпускает универсальный фаззер на основе мутаций или генерации, конечному пользователю всё ещё требуется самостоятельно провести некоторое исследование, чтобы найти уязвимости.

При наличии фаззера конкретного протокола нужна только цель. Если конечный пользователь запустит фаззер против той же цели, что и автор, будут обнаружены те же ошибки, после чего

инструмент потеряет значительную часть ценности для первоначального автора. Это не совсем верно, если можно найти новые, ещё не проверенные фаззером цели, использующие тот же протокол. Решение о публикации такого инструмента в конечном счёте сводится к личным убеждениям об этике и идеалах устранения ошибок и обмена исследованиями.

Мутационный фаззинг с изменённым клиентом

Одна из распространённых претензий к фаззингу состоит в том, что по мере усложнения разработанный фаззер приближается по сложности к созданию полноценного клиента. Так почему бы не пойти в обратную сторону? Этот особый класс фаззинга, ранее не упоминавшийся, обозначает метод, при котором исходный код клиентской части обмена «клиент - сервер», если он доступен, изменяется для создания фаззера. По сути, вместо реализации всего протокола в фаззере фаззер встраивается в приложение, уже говорящее на нужном языке. Преимущество состоит в доступе фаззера к существующим процедурам создания допустимых протокольных данных и сокращении усилий разработчика фаззера.

Хотя нам неизвестны общедоступные инструменты фаззинга, использующие этот подход, таким способом было создано несколько эксплойтов. Один из примеров - эксплойт OpenSSH `sshutuptheo`,[22] написанный GOBBLES для эксплуатации целочисленного переполнения до аутентификации в популярном сервере SSH.

Этот подход особенно выгоден для сложных протоколов вроде SSH. Однако разработчик должен учитывать все ограничения клиента, которые повлияют на покрытие кода. Например, если выбран SSH, а клиент поддерживает только SSH версии 1, особенности SSH версии 2 вообще не получат покрытия. Другой недостаток, разумеется, состоит в том, что для покрытия множества тестовых примеров допустимый клиент может потребовать значительных изменений.

Обнаружение сбоев

Обнаружение сбоев является критически важной частью процесса фаззинга, поэтому глава 24 «Интеллектуальное обнаружение сбоев» целиком посвящена его обсуждению. Хотя совершенные методы обнаружения сбоев будут рассмотрены лишь в той главе, познакомимся с некоторыми базовыми приёмами применительно к фаззингу сетевых протоколов. Сложность обнаружения сбоев при сетевом фаззинге полностью зависит от цели. Например, рассмотрим сетевой демон, который после любого сбоя аварийно завершается и перестаёт принимать соединения. Очевидно, с точки зрения простейшего обнаружения сбоев это превосходное поведение. Когда сервер прекращает работу, просто предположите, что его вывел из строя последний тестовый пример. Фаззер может сделать это полностью самостоятельно без помощи программного агента или ручной проверки целевого узла.

Рассмотрим другой сценарий, в котором целевое приложение способно обрабатывать сбои либо создаёт отдельный процесс и самостоятельно сохраняет прослушивающий сокет. Теоретически мы можем вызвать эксплуатируемую уязвимость и никогда об этом не узнать, если, разумеется, не применим некоторую форму более глубокого исследования целевого узла. Можно смело предположить, что во время большинства фаззинговых проверок у нас есть доступ к целевому узлу. Рассмотрим несколько базовых подходов к обнаружению сбоев на целевом узле.

Вручную, с помощью отладчика

При наличии локального доступа к машине самый простой способ следить за исключениями - подключить отладчик к процессу. Отладчик способен обнаружить возникновение исключения, позволяя пользователю определить дальнейшие действия. Для этой задачи подходят Ollydbg, Windbg, IDA и GDB. Проблемой становится сопоставление поведения с вызвавшим его тестовым примером или последовательностью тестов.

Автоматически, с помощью агента

Рассмотрим архитектуру, в которой ручная отладка заменена. Вместо отладочного приложения специалист по фаззингу пишет специальный для целевой платформы отладочный агент, работающий на цели. У агента две задачи. Первая - следить за исключениями целевого процесса. Вторая - обмениваться данными с фаззером в удалённой системе. Это позволяет легко сопоставлять данные с обнаруженными сбоями. Недостаток подхода в том, что разработчику придётся создавать агент для каждой проверяемой платформы. И снова эта идея подробнее разбирается в главе 24.

Другие источники

Хотя при сетевом фаззинге отладчики, скорее всего, будут самым ценным средством обнаружения исключений, не игнорируйте другие доступные признаки. Журналы приложения и операционной системы могут предоставить сведения о возникших проблемах. Как и при ручном подключении отладчика к приложению, трудность здесь состоит в сопоставлении проблемы с ответственным фаззинговым примером. Обязательно следите и за снижением производительности системы, которое может указывать на скрытые проблемы. Среди признаков могут быть рост загрузки процессора или исчерпание памяти. Тестовый пример, вызывающий бесконечный цикл, может никогда не породить исключение, но при этом фактически приводит к отказу в обслуживании. Смысл прост: хотя отладчики являются превосходными инструментами обнаружения уязвимостей, не следует игнорировать другие полезные признаки.

Итоги

Фаззинг сетевых протоколов, пожалуй, является самым известным и широко используемым способом передачи фаззинговых данных, и выполнять его можно множеством различных способов. Популярность обусловлена сочетанием факторов, не последним из которых является возможность обнаруживать удалённые уязвимости высокого риска, существующие до аутентификации. Кроме того, это зрелый вид фаззинга, поэтому исследователям безопасности доступен ряд общедоступных инструментов. Вооружившись знанием нескольких распространённых подходов к сетевому фаззингу, пора перейти к реализации сетевого фаззера.

Сноски

[1] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[2] [назад] http://www.digitaldwarf.be/products/ircfuzz.c
[3] [назад] http://www.digitaldwarf.be/products/dhcpfuzz.pl
[4] [назад] http://www.infigo.hr/en/in_focus/tools
[5] [назад] http://www.microsoft.com/mscorp/twc/2007review.mspx

[6] [назад] http://research.eeye.com/html/advisories/published/AD20010618.html
[7] [назад] http://www.microsoft.com/technet/security/bulletin/MS02-039.mspx
[8] [назад] http://www.microsoft.com/technet/security/bulletin/MS02-061.mspx
[9] [назад] http://en.wikipedia.org/wiki/SQL_slammer_worm
[10] [назад] http://en.wikipedia.org/wiki/Blaster_worm
[11] [назад] http://www.microsoft.com/technet/security/bulletin/MS03-026.mspx
[12] [назад] http://weblog.infoworld.com/techwatch/archives/001035.html
[13] [назад] http://en.wikipedia.org/wiki/Osi_model
[14] [назад] http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2006-3507
[15] [назад] http://blog.washingtonpost.com/securityfix/2006/08/hijacking_a_macbook_in_60_seco.html
[16] [назад] http://www.microsoft.com/technet/security/Bulletin/MS06-032.mspx
[17] [назад] http://support.microsoft.com/default.aspx?scid=kb;[LN];168747
[18] [назад] http://www.microsoft.com/technet/security/bulletin/MS04-011.mspx
[19] [назад] http://en.wikipedia.org/wiki/Sasser_worm
[20] [назад] http://bvlive01.iss.net/issEn/delivery/xforce/alertdetail.jsp?oid=20823
[21] [назад] http://peachfuzz.sourceforge.net/
[22] [назад] http://online.securityfocus.com/data/vulnerabilities/exploits/sshutuptheo.tar.gz

Глава 15. Фаззинг сетевых протоколов: автоматизация в UNIX

> «Думаю, мы согласны: прошлое закончилось».
>
> Джордж У. Буш о встрече с Джоном Маккейном, Dallas Morning News, 10 мая 2000 года

Хотя Microsoft Windows господствует в области настольных систем, системы UNIX по-прежнему занимают большую часть рынка предпочтительных серверных платформ. Например, веб-сервер Apache, в основном работающий в системах UNIX, согласно последнему исследованию NetCraft,[1] опережает Microsoft IIS почти на 30 процентных пунктов. Уязвимости популярных служб UNIX обладают высокой критичностью и далеко идущими последствиями. Основная часть Интернета работает на службах DNS, электронной почты и Веба на основе UNIX. Например, обнаружение дефекта в DNS-сервере Berkeley Internet Name Domain (BIND) может быть использовано для нарушения значительной части интернет-обмена. Хотя нельзя наверняка узнать, какая доля обнаруженных в прошлом уязвимостей была найдена с помощью фаззинга, определённо можно предположить, что многие из них обнаружили при содействии фаззера.

В этой главе мы не станем с нуля разрабатывать собственный фаззер на основе UNIX. Вместо этого представим и используем удобный интерфейс сценариев, предоставляемый основой фаззинга SPIKE, фаззером с открытым исходным кодом, неоднократно упоминавшимся в книге. Поскольку в этой главе мы не сосредоточены на фактическом программировании основы фаззинга, вместо этого от начала до конца проведём фаззинговую проверку приложения с закрытым исходным кодом с помощью SPIKE.

Фаззинг с помощью SPIKE

Чтобы продемонстрировать процесс фаззинга в SPIKE, полностью рассмотрим пример от начала до конца: выберем цель, исследуем целевой протокол, опишем его сценарием SPIKE, а затем спустим фаззер с поводка.

Выбор цели

Для нашего примера мы хотим выбрать цель со следующими желательными свойствами:

• целевая программа должна широко использоваться;
• целевую программу должно быть легко получить в виде демонстрационной или оценочной версии;

• целевая программа должна использовать общедоступный либо тривиальный для обратной разработки протокол.

Возможностей много, но для этого примера мы выберем Novell NetMail,[2] корпоративную систему электронной почты и календарей, реализующую несколько документированных протоколов и отвечающую всем нашим требованиям. В частности, целью будет NetMail Networked Messaging Application Protocol (NMAP). Этот NMAP не следует путать с Nmap,[3] повсеместно используемым средством сканирования и разведки сети. Что именно представляет собой NetMail NMAP? Мы не знали этого навскидку, но, к счастью, Novell охотно делится следующим определением:

> Аббревиатура NMAP означает Networked Messaging Application Protocol. Это текстовый IP-протокол, зарегистрированный Internet Assigned Numbers Authority (IANA) на порту 689 и используемый агентами NIMS для обмена данными. В сочетании с распределённой природой NDS eDirectory протокол позволяет агентам NIMS, работающим на разных серверах и даже платформах, действовать так, словно они находятся на одном сервере. Вместо замены сервера более мощным при росте спроса на службы обмена сообщениями NMAP позволяет добавлять дополнительные серверы в «кластер». Документация протокола NMAP в стиле RFC поставляется с каждой версией NIMS.[4]

NMAP, судя по всему, является разработанным Novell проприетарным текстовым протоколом поверх TCP. Выбор проприетарного протокола имеет преимущества и недостатки. С отрицательной стороны, созданный нами фаззер не будет использоваться за пределами этого поставщика. С положительной стороны, для собственного протокола нет проверенных временем и испытанных библиотек разбора. Поставщику пришлось написать анализатор протокола самостоятельно, код просматривало меньше людей, а значит, он может содержать больше уязвимостей.

Novell любезно предлагает на своём веб-сайте[5] бесплатные 90-дневные оценочные версии NetMail, поэтому целевую программу достаточно легко загрузить и установить в лабораторной среде. После установки мы обновляем её исправлениями, предоставленными Novell. Это критически важный шаг, гарантирующий, что мы не обнаружим повторно старые ошибки, если вообще что-либо найдём. Очень неприятно через неделю аудита понять, что вы пропустили этот шаг. Далее необходимо подробно изучить протокол NMAP.

Обратная разработка протокола

Прежде чем описывать протокол NMAP для SPIKE, нам, очевидно, нужно самим его понять. Существует ряд подходов к этому этапу. Возможно, самый очевидный - наблюдать за допустимым трафиком NMAP в лабораторной среде. На самом деле этот подход немного сложнее, чем можно предположить. При работе со сложными корпоративными программами иногда трудно создать именно тот трафик, который вы ищете. Другой подход - опереться на работу других. Никогда не повредит спросить Google, что известно окружающим о данном протоколе. Кроме того, всегда полезно проверить, существует ли декодер протокола в открытом анализаторе трафика Wireshark, ранее называвшемся Ethereal. Перейдите в репозиторий Subversion Wireshark, а именно в каталог `epan\dissectors`,[6] и посмотрите, удастся ли найти ваш протокол.

Ещё один, реже всего доступный способ - просто обратиться к самому демону NMAP и посмотреть, предоставит ли он какие-либо сведения. Для этого сначала нужно определить порты, по которым приложение обменивается данными с клиентами. Если предположить, что документация прямо не сообщила о привязке NMAP к TCP-порту 689, это можно выяснить вручную с помощью Microsoft TCPView,[7] ранее принадлежавшего Sysinternals. Простого запуска TCPView достаточно, чтобы сразу увидеть: `nmapd.exe` прослушивает TCP-порт 689. К демону можно подключиться базовым средством TCP-соединений, например netcat,[8] или даже командой Windows `telnet`. Сделав

смелое предположение, мы отправляем команду HELP и, к нашему приятному удивлению, получаем список команд, которые принимает демон.

Неплохо для нескольких минут работы. Развивая успех с собранными сведениями, загружаем двоичный файл nmapd.exe в любимый дизассемблер IDA Pro. Нажимаем Shift+F12 для просмотра базы строк, показанной на рисунке 15.1. Затем находим то, что похоже на ответ справки и начинается с текста ASCII 1000. Теперь сортируем базу строк по содержимому, а не по адресу, и прокручиваем до строк, начинающихся с 1000. Здесь мы находим все команды, поддерживаемые сервером NMAP, и их ожидаемый синтаксис.

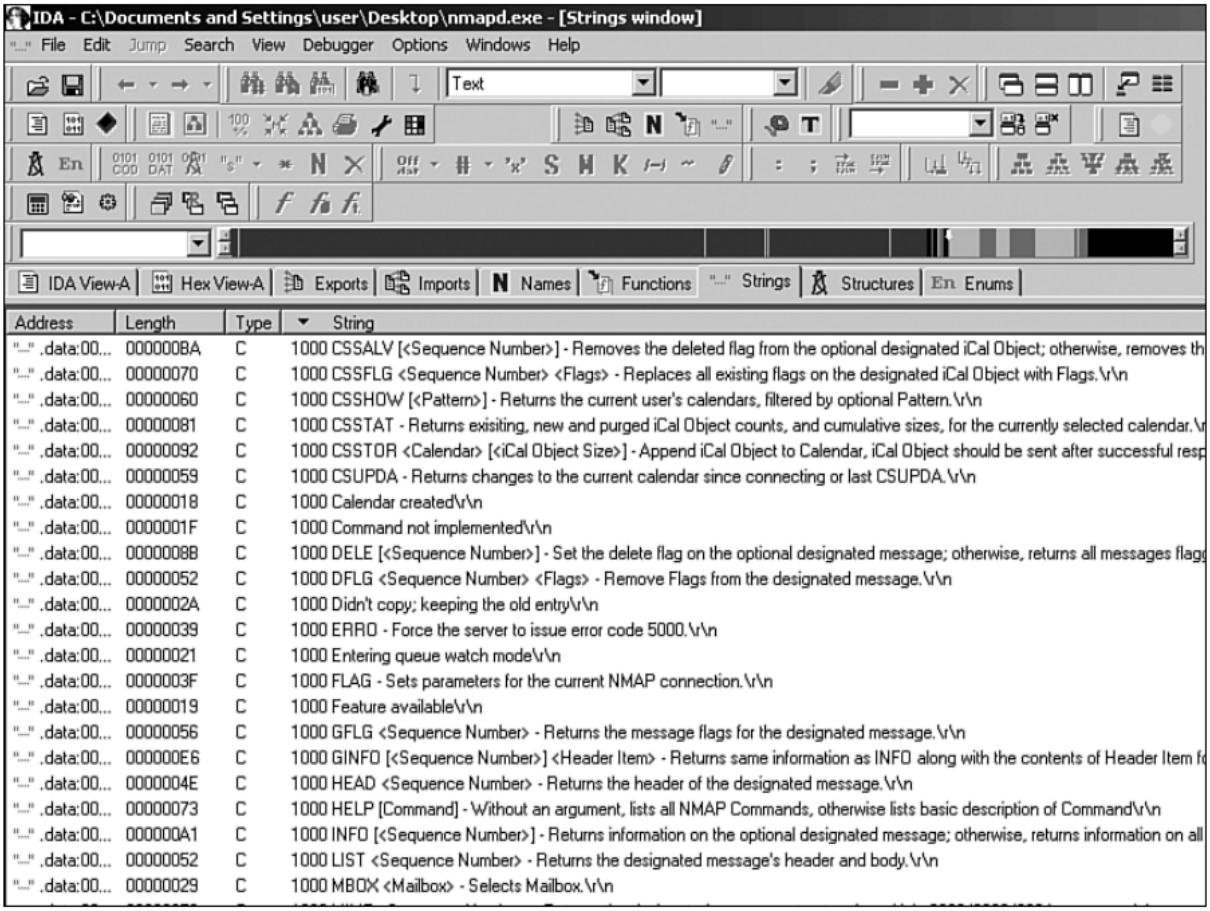


Рисунок 15.1. Список строк, найденных в исполняемом файле nmapd.exe

После изучения различных команд и описанного синтаксиса обозначения в выводе справки становятся понятны. Хотя аргументы некоторых команд не соответствуют этому методу, по большей части используется следующий прототип:

• <argument> - обязательный аргумент; если он не указан, команда завершится ошибкой;
• [argument] - необязательный аргумент;
• {CONSTANT1|CONSTANT2|CONSTANT3} - обязательный аргумент, который для успешного выполнения команды должен быть выбран из набора постоянных строк; возможные значения разделены вертикальной чертой |.

Кроме того, любая строка вне угловых скобок должна рассматриваться как буквальная, а внутри угловых скобок - как переменная. Те же правила применяются к командам с вложенным описанием синтаксиса. Например, команда PASS описана так:

```
PASS {SYS | USER <Username>} <Password>
```

Прототип указывает, что после команды всегда следует буквальная строка SYS или USER. Поскольку переменная Username находится в угловых скобках после буквальной строки USER, она

обязательна, но только при выборе USER. Если выбрано SYS, аргумент Username не требуется. Наконец, переменная Password всегда обязательна как последний аргумент команды. Протокол команды PASS, вероятно, является самым сложным в этом протоколе, что не слишком впечатляет. Как оказалось, мы выбрали очень дружелюбный протокол для фаззинга.

Поскольку пример команды связан с аутентификацией, можно сразу предположить, что демон требует её по меньшей мере для части функциональности. Для специалиста по фаззингу это может быть интересно или неинтересно в зависимости от области проверки. Например, небольшая группа исследователей, спешащая уложиться в срок и проверяющая лишь наиболее критические состояния приложения, существующие до аутентификации, скорее всего, проигнорирует команды после аутентификации. Однако для тщательной проверки всех состояний программы фаззер должен обеспечить правильную аутентификацию после исчерпания состояния до аутентификации.

Предположив, что нас интересуют состояния после аутентификации, необходимо определить способ успешного входа. Мы видим несколько вариантов. Пользователь может войти, отдельно отправив команды USER, а затем PASS, как в протоколе FTP. Другой вариант - указать имя пользователя внутри команды PASS. Третий тип аутентификации позволяет другим агентам NMAP проходить проверку с директивой SYS вместо USER в команде PASS.

Потратив некоторое время на изучение других команд и их удобно подробных описаний, найденных с помощью IDA, мы начинаем понимать работу протокола. На самом базовом уровне каждая команда начинается с одной строки ASCII, которую мы называем глаголом. Он сообщает демону, какое действие мы запрашиваем. Следующий компонент каждой команды - пробел-разделитель. После пробела каждой команде требуется либо символ новой строки, если глагол не принимает аргументов, либо специфические для глагола аргументы. Формат аргументов можно установить по строкам, найденным в IDA.

Вооружившись этими базовыми сведениями о протоколе NMAP, теперь можно приступить к созданию фаззера NMAP в SPIKE.

SPIKE 101

В главе 21 «Основы фаззинга» SPIKE кратко рассматривается среди других доступных основ фаззинга, но здесь мы изучим его подробно. Поскольку для фаззинга протокола будет применяться SPIKE, важно познакомиться с его механизмом фаззинга и универсальным сценарным построчным фаззером TCP.

Механизм фаззинга

SPIKE работает, последовательно перебирая переменные и фаззинговые строки. Представляйте переменные как поля протокола: имена пользователей, пароли, команды и так далее. Эти переменные и их расположение в потоке данных специфичны для каждой цели. Фаззинговые строки являются библиотекой различных строк и двоичных данных, способных вызвать сбой; элементы библиотеки тщательно выбраны на основе прошлого опыта с конкретными последовательностями, вызывавшими проблемы в других программах. Например, рассмотрим самую простую фаззинговую строку: строку ASCII из 64 000 последовательных символов A. Она может заменить переменную имени пользователя в протоколе, чтобы вызвать переполнение буфера до аутентификации. Помните, что термин фаззинговая «строка» несколько вводит в заблуждение. Фаззинговая строка может иметь любой тип данных, даже быть, например, массивом двоичных данных в кодировке XDR.

Универсальный построчный фаззер TCP

Создаваемый здесь фаззер известен как универсальный построчный фаззер TCP. Код приложения находится в файле `line_send_tcp.c` исходного пакета SPIKE. Этот фаззер представляет собой очень простую, но мощную оболочку SPIKE. Он обрабатывает сценарий SPIKE и подвергает фаззингу каждую его переменную, заново подключаясь к узлу при каждой новой попытке. Переменные проверяются в порядке их расположения в фаззинговом сценарии.

Это означает, что в большинстве протоколов с аутентификацией SPIKE сначала проверяет связанные с ней сведения.

Реализация языка сценариев позволяет автору сценария непосредственно вызывать функции API SPIKE. Чтобы дать базовое представление о написании сценария, перечислим некоторые функции, которые вам, скорее всего, придётся вызывать:

• `s_string(char * instring)` добавляет в SPIKE фиксированную строку. Она никогда не изменяется.
• `s_string_variable(unsigned char *variable)` добавляет в SPIKE строковую переменную. При обработке соответствующей переменной эта строка заменяется фаззинговыми строками.
• `s_binary(char * instring)` добавляет в SPIKE двоичные данные. Они никогда не изменяются.
• `s_xdr_string(unsigned char *astring)` добавляет в SPIKE строку в стиле XDR. Она включает четырёхбайтовую метку длины и дополняется нулями до размера, кратного четырём. Строка никогда не изменяется.
• `s_int_variable(int defaultvalue, int type)` добавляет в SPIKE целое число.

В вызовах `s_int_variable()` значение `type` может быть одним из следующих:

• **Binary Big Endian:** целое с наиболее значимым байтом первым (MSB), 4 байта;
• **ASCII:** знаковое десятичное число в формате ASCII;
• **One byte:** однобайтовое целое;
• **Binary Little Endian Half Word:** целое с наименее значимым байтом первым (LSB), 2 байта;
• **Binary Big Endian Half Word:** целое MSB, 2 байта;
• **Zero X ASCII Hex:** шестнадцатеричное число ASCII с префиксом `0x`;
• **ASCII Hex:** шестнадцатеричное число в формате ASCII;
• **ASCII Unsigned:** беззнаковое десятичное число в формате ASCII;
• **Intel Endian Word:** целое LSB, 4 байта.

Поскольку при использовании SPIKE как узла сценариев препроцессор C не применяется, нам нужно знать числовые значения типов. В следующем фрагменте из `SPIKE/SPIKE/include/listener.h` показаны их определения:

```
#define BINARYBIGENDIAN 1
#define ASCII           2
#define ONEBYTE         3
#define BINARYLITTLEENDIANHALFWORD 4
#define BINARYBIGENDIANHALFWORD 5

#define ZEROXASCIIHEX 6
#define ASCIIHEX      7
#define ASCIIUNSIGNED 8
#define INTELENDIANWORD 9
```

Обсуждённых возможностей SPIKE достаточно для разработки фаззера NMAP. Однако рассмотрим главный вклад SPIKE в область фаззинга: блочное представление протокола.

Блочное моделирование протокола

Хотя мы выбрали для работы простой текстовый протокол, SPIKE поддерживает и более сложные протоколы благодаря блочному фаззингу. Оно позволяет динамически создавать пакеты с допустимыми длинами полей. Рассмотрим, например, протокол со структурой пакета, в которой перед полем имени пользователя стоит длина указанного имени. Нам хотелось бы, чтобы фаззер автоматически обновлял поле длины при фаззинге имени пользователя. Это гарантирует правильную передачу фаззинговых строк анализатору имени. Функции `s_block_start()` и `s_block_end()` позволяют моделировать такие протоколы и объявлены следующим образом:

```
int s_block_start(char *blockname)
int s_block_end(char *blockname)
```

Для эффективного использования достаточно поместить их до и после измеряемого поля. Затем там, где фактическую длину нужно вставить в поток данных, применяется одна из нескольких функций размера блока. Они похожи на рассмотренные ранее целочисленные поля и имеют подробные имена. Обратите внимание: некоторые размеры блоков являются переменными, а значит, будет выполнен цикл фаззинга с недопустимыми размерами. Решение о том, подвергать ли фаззингу поля длины, остаётся за вами. Ниже приведён почти полный список различных типов размеров блоков SPIKE.

```
s_blocksize_signed_string_variable(char * instring, int size)
s_blocksize_unsigned_string_variable(char * instring, int size)
s_blocksize_asciihex_variable(char * blockname)
s_binary_block_size_word_bigendian(char *blockname)
s_binary_block_size_word_bigendian_variable(char *blockname)
s_binary_block_size_halfword_bigendian(char * blockname)
s_binary_block_size_halfword_bigendian_variable(char *blockname)
s_binary_block_size_byte(char * blockname)

s_binary_block_size_byte_variable(char * blockname)
s_binary_block_size_byte_plus(char * blockname, long plus)
s_binary_block_size_word_bigendian_plussome(char *blockname, long some)
s_binary_block_size_intel_halfword(char *blockname)
s_binary_block_size_intel_halfword_variable(char *blockname)
s_binary_block_size_intel_halfword_plus_variable(char *blockname,long plus)
s_binary_block_size_intel_halfword_plus(char *blockname,long plus)
s_binary_block_size_byte_mult(char * blockname, float mult)
s_binary_block_size_halfword_bigendian_mult(char * blockname, float mult)
s_binary_block_size_word_bigendian_mult(char *blockname, float mult)
s_binary_block_size_intel_word(char *blockname)
s_binary_block_size_intel_word_variable(char *blockname)
s_binary_block_size_intel_word_plus(char *blockname,long some)
s_binary_block_size_word_intel_mult_plus(char *blockname, long some, float mult)
s_binary_block_size_intel_halfword_mult(char *blockname,float mult)
s_blocksize_unsigned_string_variable(char * instring, int size)
s_blocksize_asciihex_variable(char * blockname)
```

Дополнительные возможности SPIKE

SPIKE является не просто фаззером, а полноценной основой фаззинга с огромным API полезных функций, способных упростить создание собственных фаззеров. Он также включает большую коллекцию фаззеров и сценариев фаззинга для конкретных протоколов и приложений. Помимо механизма SPIKE и кода обмена данными, он предоставляет следующее.

Фаззеры конкретных протоколов

SPIKE включает небольшую коллекцию готовых фаззеров конкретных протоколов:

• фаззер HTTP;
• фаззер Microsoft RPC;
• фаззер X11;
• фаззер Citrix;
• фаззер Sun RPC.

По большей части эти фаззеры полезны лишь как примеры применения SPIKE. Причина в том, что они существуют уже некоторое время и запускались практически против каждой цели, которая может вас заинтересовать.

Сценарии фаззинга конкретных протоколов

В состав SPIKE также входят сценарии, подключаемые к одному из многочисленных универсальных фаззеров SPIKE:

• CIFS;
• FTP;
• H.323;
• IMAP;
• Oracle;
• Microsoft SQL;
• PPTP;
• SMTP;
• SSL;
• POP3.

Универсальные фаззеры на основе сценариев

Как уже упоминалось, в SPIKE есть несколько универсальных фаззеров, принимающих сценарии на вход:

• фаззер прослушивания TCP, то есть клиентской стороны;
• фаззеры отправки TCP/UDP;
• фаззер отправки TCP со строковой буферизацией.

Написание сценария фаззера NMAP в SPIKE

Возвращаясь к цели NetMail, следующий пример сценария SPIKE построен исключительно на знаниях, полученных из сообщения `HELP` и списков строк IDA Pro. Этот фаззинговый сценарий начинает с фаззинга самой команды аутентификации, разыскивая возможные ошибки до аутентификации. Если заданы правильные имя пользователя и пароль, сценарий продолжает фаззинг команд после аутентификации.

```
s_string_variable("PASS");
s_string(" ");
s_string_variable("USER");
s_string(" ");
s_string_variable("devel_user");
s_string(" ");
s_string_variable("secretpassword");
s_string("\r\n");
s_string("QCREA ");
s_string_variable("test");
```

```
s_string("\r\n");
s_string("CREA ");
s_string_variable("inbox");
s_string("\r\n");
s_string("MBOX ");
s_string_variable("test");
s_string("\r\n");
s_string("LIST ");
s_string_variable("0");
s_string("\r\n");
s_string("GINFO ");
s_string_variable("0");
s_string(" ");
s_string_variable("test");
s_string("\r\n");
s_string("SEARCH BODY ");
s_string_variable("test");
s_string("\r\n");
s_string("DFLG ");
s_string_variable("0");
s_string(" ");
s_string_variable("SEEN");
s_string("\r\n");
s_string("CSCREA ");
s_string_variable("test");
s_string("\r\n");
s_string("CSOPEN ");
s_string_variable("test");
s_string("\r\n");

s_string("CSFIND ");
s_string_variable("0");
s_string(" ");
s_string_variable("0");
s_string(" ");
s_string_variable("0");
s_string("\r\n");
s_string("BRAW ");
s_string_variable("0");
s_string(" ");
s_string_variable("0");
s_string(" ");
s_string_variable("0");
s_string("\r\n");
```

Чтобы продолжить, мы подключаем выбранный отладчик к процессу NMAP в целевой системе и запускаем разработанный сценарий через SPIKE. Для выполнения сценария `nmap.spk` используется универсальный построчный фаззер TCP SPIKE со следующими параметрами командной строки:

```
./line_send_tcp 192.168.1.2 689 nmap.spk 0 0
```

Всего через несколько мгновений после запуска фаззер обнаруживает эксплуатируемое переполнение стека! Демон NMAP в уязвимом состоянии показан в отладчике OllyDbg на рисунке 15.2.

В верхней правой области рисунка 15.2 показаны значения регистров в момент сбоя. Регистры EBP, указатель кадра стека, EBX, ESI, EDI и, что важнее всего, EIP, указатель инструкций, перезаписаны шестнадцатеричным значением `0x41`, то есть символом ASCII `A`. В нижней правой области показан кадр стека, который, как хорошо видно, перезаписан длинной строкой символов `A`. В верхней левой области обычно находится список выполняемых инструкций, но сейчас она пуста, поскольку указатель инструкций имеет адрес `0x41414141`. По этому адресу не отображена ни одна страница памяти, поэтому в окне дизассемблирования нечего показывать. Далее необходимо точно определить, какая пара «команда - аргумент» вызвала сбой, чтобы воспроизвести его и, возможно, даже написать эксплойт.

Сделать это можно несколькими способами. Один из самых простых использует встроенную особенность SPIKE. SPIKE часто аварийно завершается, если не может подключиться к цели. По последнему выводу SPIKE перед аварийным завершением можно определить, какой тестовый пример вызвал сбой NetMail NMAP. Однако поскольку мы подключены отладчиком, целевой процесс не завершится, а зависнет, поэтому «особенность» аварийного завершения SPIKE не проявится.

Для воспроизведения мы перезапускаем процесс, разрешаем ему работать без наблюдения, то есть без подключённого отладчика, и рассчитываем, что SPIKE аварийно завершится вместе с побеждённым демоном NMAP. Снова запускаем проверку и ждём, опять наблюдая сбой NMAP, а затем сбой SPIKE со следующим выводом:

```
-snip-
Fuzzing Variable 5:1
Read first line
Variablesize= 5004
Fuzzing Variable 5:2
Couldn't tcp connect to target
Segmentation fault
-snip-
```

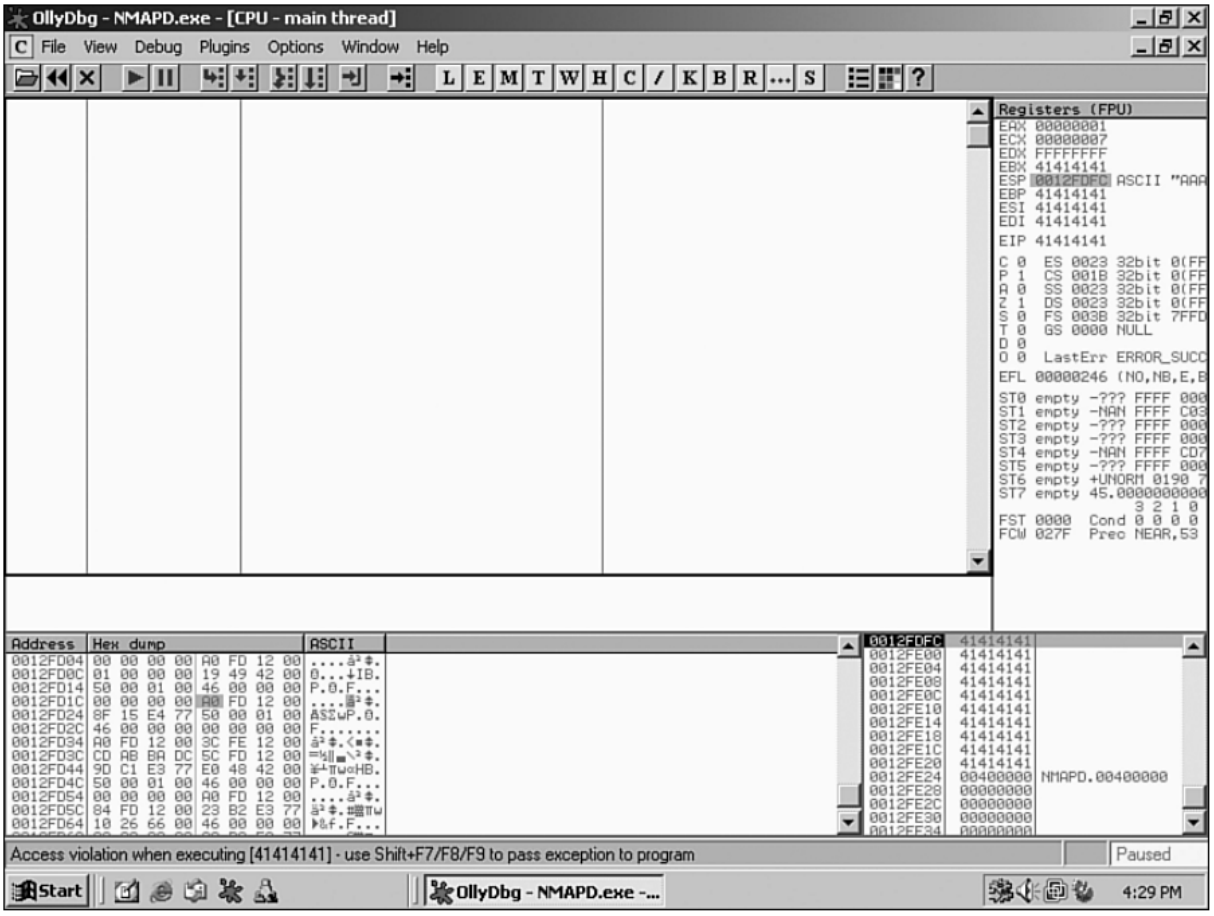


Рисунок 15.2. Эксплуатируемое переполнение стека в nmapd.exe

Этот подход определённо является самым примитивным способом отследить неверный тестовый пример, вызвавший сбой NMAP. Несколько более научный метод мог бы состоять в наблюдении за всем сетевым трафиком с помощью анализатора. NMAP аварийно завершится и перестанет отвечать на запросы. SPIKE, напротив, продолжит передавать тестовые примеры. Найдя последнюю передачу, которая получила ответ, можно определить тестовый пример, вероятно вызвавший сбой.

Вернёмся к примитивному способу обратного отслеживания. Последняя успешная передача SPIKE указана строкой `Fuzzing Variable 5:1`. Она сообщает, что последнее успешное соединение относилось к фаззинговой переменной 5. Последней отправленной процессу фаззинговой строкой была строка 1. Чтобы определить, какая переменная является фаззинговой переменной 5, достаточно загрузить сценарий SPIKE и с нуля подсчитать строки, содержащие слово `variable`. Мы приходим к аргументу команды `CREA`, доступной только после аутентификации. Теперь необходимо определить, какое значение использовалось для фаззинговой строки 1, аргумента этой команды `CREA`.

Существует несколько способов. Один - использовать анализатор трафика, как упоминалось ранее. Другой доступный подход - добавить вызов функции `printf()` в программу фаззинга `line_send_tcp.c`, приказав приложению выводить текущую фаззинговую строку, а затем просто повторно запустить фаззер. Оба способа показывают, что вызывающая ошибку строка имеет простой вид `CREA <longstring>`. Мы мгновенно воспроизводим этот сбой после аутентификации. Пользователю достаточно войти и отправить вредоносный аргумент команды `CREA`. Это было не так трудно: небольшие затраты времени дали удалённую уязвимость. Невольно задаёшься вопросом, почему больше поставщиков не применяет такую проверку до того, как поставить знак одобрения на свою программу. Если мы хотим продолжить фаззинг NMAP, можно просто удалить из сценария SPIKE часть, относящуюся к команде `CREA`, чтобы не продолжать выводить цель из строя уже известной проблемой.

Итоги

Обдумывая написание собственного фаззера, всегда важно сначала оценить существующие фаззеры и основы фаззинга. Для простой цели вроде протокола NMAP разработка фаззера с нуля дала бы мало преимуществ. Всего за несколько часов можно создать набор сценариев SPIKE, эффективно проверяющих код демона NMAP. Однако помните, что качество результатов будет соответствовать времени, вложенному в разработку фаззера. В данном случае мы ограничились абсолютным минимумом. Мы решили войти, выполнить несколько командных запросов и выйти. В NMAP можно найти десятки других функций и добавить их в сценарии для дальнейшей проверки.

Сноски

[1] [назад] http://news.netcraft.com/archives/2007/02/23/march_2007_web_server_survey.html
[2] [назад] http://www.novell.com/products/netmail/
[3] [назад] http://insecure.org/nmap/
[4] [назад] http://support.novell.com/techcenter/articles/ana20000303.html
[5] [назад] http://download.novell.com/index.jsp
[6] [назад] http://anonsvn.wireshark.org/wireshark/trunk/
[7] [назад] http://www.microsoft.com/technet/sysinternals/Networking/TcpView.mspx
[8] [назад] http://www.vulnwatch.org/netcat/

Глава 16. Фаззинг сетевых протоколов: автоматизация в Windows

> «Я не могу представить, чтобы кто-то вроде Усамы бен Ладена понимал радость Хануки».
>
> Джордж У. Буш, церемония зажжения меноры в Белом доме, Вашингтон, округ Колумбия, 10 декабря 2001 года

Хотя системы UNIX могут господствовать в серверных, во всём мире установлено больше экземпляров операционной системы Microsoft Windows, что делает её столь же желанной целью атак. Уязвимости настольных систем Windows часто используются для создания множества существующих сегодня ботнетов. В качестве демонстрации мощи сетевой уязвимости Windows рассмотрим червя Slammer,[1] эксплуатирующего переполнение буфера Microsoft SQL Server. Уязвимость была исправлена в бюллетене безопасности Microsoft MS02-039[2] 24 июля 2002 года, а червь Slammer появился 25 января 2003 года. У червя практически нет полезной нагрузки: он просто использует заражённый узел для поиска и заражения других машин.[3] Несмотря на отсутствие полезной нагрузки, агрессивное сканирование создало достаточно трафика, чтобы нарушить работу Интернета, обработку кредитных карт, а в некоторых случаях и доступность сетей мобильной связи. Особенно интересно, что даже через четыре года червь Slammer по-прежнему входит в пятёрку наиболее часто наблюдаемых событий, создающих трафик.[4] [5]

Очевидно, открытая сетевая уязвимость Windows имеет далеко идущие последствия.

В предыдущей главе мы использовали существующую основу фаззинга SPIKE, чтобы в среде UNIX создать фаззер протокола, направленный на демон Novell NetMail NMAP. В этой главе применим другой подход и от начала до конца рассмотрим создание простого, удобного фаззера для Windows с графическим интерфейсом. Хотя окончательный продукт под названием ProtoFuzz будет предоставлять лишь базовые возможности, он служит хорошей платформой для расширения и даёт другой взгляд на создание фаззеров. Начнём с обсуждения желаемого набора возможностей.

Возможности

Прежде чем погружаться в разработку, нужно сделать шаг назад и обдумать необходимые и желательные возможности. На самом базовом уровне фаззер протокола просто передаёт цели изменённые пакеты. Если фаззер умеет создавать и отправлять пакеты данных, он отвечает требованиям. Однако было бы неплохо также вооружить ProtoFuzz способностью понимать структуру пакетов, которые мы хотим подвергать фаззингу. Расширим эти базовые требования.

Структура пакета

Прежде чем фаззер сможет отправить пакет данных, он должен понимать, как его построить. Существующие фаззеры применяют один из трёх основных подходов к сборке пакетов:

• **Составленные наборы тестов.** И PROTOS Test Suite,[6] и его коммерческий родственник Codenomicon[7] жёстко задают структуру пакетов данных для фаззинга. Создание таких наборов - трудоёмкая задача, поскольку требует анализа спецификации протокола и последующей разработки, возможно, тысяч жёстко заданных тестовых примеров.
• **Генерационные фаззеры.** Как мы видели в предыдущей главе, такие инструменты, как SPIKE, требуют от пользователя построить шаблон, описывающий структуру пакета данных. Затем фаззер отвечает за создание и передачу отдельных тестовых примеров во время выполнения.
• **Мутационные фаззеры.** Вместо построения пакета с нуля можно начать с заведомо исправного пакета, а затем последовательно изменять его части. Хотя каждый байт пакета можно изменять полным перебором, для фаззинга протокола это обычно не подходит: такой подход неэффективен и создаёт множество пакетов, не соответствующих сетевым протоколам и, возможно, даже не маршрутизируемых к цели. Подход можно немного исправить, позаимствовав приём шаблона протокола. Фаззер начинает с заведомо исправного пакета, а затем пользователь создаёт из него шаблон, обозначая области данных, выбранные для фаззинга. Именно этот подход будет использовать ProtoFuzz.

Ни один подход не превосходит остальные во всех случаях; каждый лучше подходит для определённых ситуаций. Мутация пакетов была выбрана для ProtoFuzz из-за простоты. Начало с заведомо исправного пакета, который можно перехватить в наблюдаемом сетевом трафике, позволяет пользователю почти немедленно приступить к фаззингу без чрезмерной предварительной работы по исследованию протокола и созданию генерационных шаблонов.

Перехват данных

Поскольку подходом фаззинга выбрана мутация пакетов, разумно дать ProtoFuzz возможность перехватывать сетевые данные в неразборчивом режиме, подобно анализатору трафика. Мы встроим в ProtoFuzz анализатор сетевых протоколов, чтобы он мог перехватывать трафик к целевому приложению и от него, а затем выбирать отдельные пакеты для фаззинга. Для реализации этой возможности воспользуемся существующей библиотекой перехвата пакетов, подробно описанной в разделе «Разработка».

Разбор данных

Помимо перехвата данных, хотя это и не строго необходимо, мы хотим, чтобы ProtoFuzz представлял содержимое перехваченных пакетов в удобном для восприятия виде. Это поможет пользователю выявлять области пакета, подходящие для фаззинга. Данные сетевого пакета являются всего лишь последовательностью байтов по определённому шаблону. Для отображения данных в удобочитаемом виде нужна библиотека перехвата, способная понимать структуру пакета и разделять её на различные заголовки протоколов и сегменты данных, составляющие поток. Большинство знакомо с форматом представления Wireshark, поэтому возьмём его за основу. На рисунке 16.1 показан простой пакет TCP, перехваченный Wireshark.

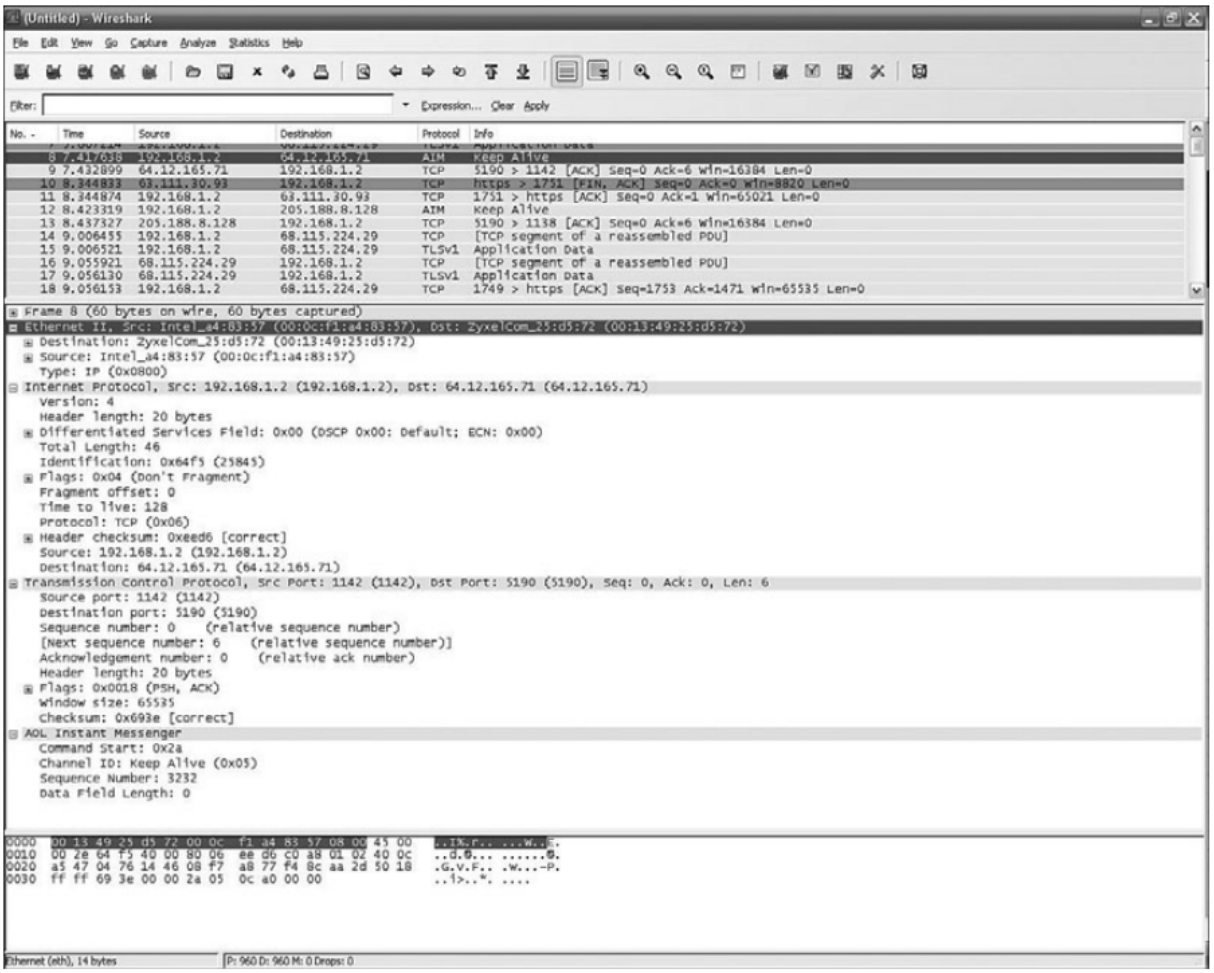


Рисунок 16.1. Wireshark разбирает пакет поддержания соединения AIM

На этом снимке экрана Wireshark данные разделены на три области. Верхняя содержит список отдельных перехваченных пакетов. Выбор одного из них загружает содержимое пакета в две нижние области. Средняя показывает разбор пакета на отдельные поля. В данном случае видно, что Wireshark знаком с протоколом AOL Instant Messenger, поскольку успешно декодировал весь сегмент данных пакета TCP. Наконец, в нижней области показаны необработанные шестнадцатеричные байты и байты ASCII выбранного пакета.

Фаззинговые переменные

После наблюдения и перехвата сетевых данных мы хотим позволить пользователю обозначать их области для мутации при фаззинге. Для этого применим очень простой формат, заключающий части шестнадцатеричного представления пакета в открывающие и закрывающие теги. Разные теги будут обозначать разные типы фаззинговых переменных времени выполнения. Простой формат позволит пользователю визуально выявлять фаззинговые переменные, а ProtoFuzz - при разборе структуры пакета определять области для замены фаззинговыми данными. Будут использоваться следующие теги:

• `[XX]`**, полный перебор.** Байты в квадратных скобках подвергаются фаззингу всеми возможными байтовыми значениями. Поэтому один байт проверяется 256 раз, а значение слова, два байта, 65 536 раз.

• `<XX>`**, строки.** Байты также можно подвергать фаззингу шестнадцатеричными представлениями заранее определённых строк переменной длины из управляемого пользователем текстового файла

`Strings.txt`. Такой фаззинг обычно выполняется в областях данных пакетов, а не в полях заголовков, которым требуется определённая структура.

Следующий шаблон показывает пакет TCP с фаззинговыми переменными полного перебора и строк:

```
00 0C F1 A4 83 57 00 13 49 25 D5 72 08 00 45 00 00 28<B0 3B>00 00 FE 06
89 40 C0 A8 01 01 C0 A8 01 02 08 A6 0B 35 14 9E E1 9F 9F 33 69 E5 50 11
10 00 09 4E 00 00 01 00 5E 00 00[16]
```

Отправка данных

Библиотеки протоколов обычно применяют один из двух подходов к отправке данных. Во-первых, они могут иметь ряд функций, позволяющих задавать определённые сведения, например целевые IP- и MAC-адреса. Однако основная часть структуры пакета строится автоматически по соответствующему RFC. Пример такого класса .NET Framework - `HttpRequest`. Он разрешает определять такие свойства, как метод и URL, но большинство заголовков HTTP и все заголовки Ethernet, TCP и IP создаются автоматически. Альтернативный подход - создать необработанный пакет данных, в котором отдельные байты задаются явно, а программист должен сам обеспечить соответствие пакета структуре из надлежащего RFC. Хотя второй подход требует от исследователя больше работы, более детальный контроль позволит подвергать фаззингу как заголовки протокола, так и сегменты данных.

Необходимые исходные сведения

Прежде чем обсуждать разработку ProtoFuzz, необходимо выделить несколько важных исходных сведений.

Обнаружение

Как и при любом виде фаззинга, обнаружение сбоев целевого приложения критически важно для выявления уязвимостей. Идеального способа нет, но некоторые подходы определённо лучше других. Одно из препятствий при фаззинге протоколов состоит в том, что в отличие, например, от фаззинга файлов фаззер и целевое приложение, скорее всего, находятся в двух разных системах. При фаззинге сетевых протоколов подключение отладчика к целевому приложению является хорошей отправной точкой и позволяет точно выявлять обработанные и необработанные исключения, которые иначе могли бы остаться незамеченными. При использовании отладчика всё равно остаётся задача сопоставления фаззинговых пакетов с создаваемыми ими исключениями. Хотя это не безошибочное решение, обходной путь состоит в отправке некоторой проверки после каждого пакета, чтобы убедиться в доступности цели. Например, можно отправить цели запрос ping и перед следующим фаззинговым пакетом убедиться, что получен ответ. Это далеко не идеально, поскольку могло возникнуть исключение, не влияющее на способность цели отвечать на ping. Однако проверку можно настроить для конкретной цели.

Снижение производительности

Помимо наблюдения за сбоями, можно следить за снижением производительности целевого приложения. Рассмотрим, например, искажённый пакет данных, вызывающий бесконечный цикл в логике целевого приложения. Фактический сбой в такой ситуации не создаётся, но возникает состояние DoS. Запуск средств наблюдения за производительностью на целевой машине, например оснасток Performance Logs and Alerts или System Monitor для Microsoft Management Console, может помочь выявить подобные ситуации.

Истечение времени ожидания запросов и неожиданные ответы

Не каждый переданный фаззинговый пакет вызовет ответ. Однако для пакетов, которые должны его вызывать, важно следить за ответом, чтобы убедиться, что целевое приложение по-прежнему работает правильно. Можно пойти дальше и также разбирать содержимое пакета ответа. Тогда удастся выявлять не только отсутствие ответов, но и ответы с неожиданными данными.

Драйвер протокола

Многие библиотеки перехвата пакетов требуют специального драйвера протокола. Выбранная для ProtoFuzz библиотека Metro Packet Library[8] включает `ndisprot.inf`, пример драйвера протокола Network Driver Interface Specification (NDIS), предоставленный Microsoft в составе Driver Development Kit. Фактически NDIS является API сетевых адаптеров, предоставляющим приложениям возможность отправлять и принимать необработанные пакеты Ethernet. Перед использованием ProtoFuzz драйвер необходимо вручную установить и запустить, что можно сделать из командной строки командой `net start ndisprot`. Один из недостатков библиотеки, требующей такого специального драйвера, состоит в том, что драйвер может поддерживать не все типы сетевых адаптеров. Например, драйвер Metro не работает с беспроводным адаптером.

Разработка

Уже существует ряд прекрасно работающих фаззеров протоколов. Если мы собираемся создать новый, чтобы избежать очередного проекта «и я тоже», он должен предлагать нечто отсутствующее у остальных. Поскольку многие существующие фаззеры работают из командной строки, мы стремимся выделить ProtoFuzz следующими качествами:

• **Интуитивность.** Использование ProtoFuzz не должно требовать долгого обучения. Конечный пользователь должен разобраться в основах инструмента без борьбы с руководством и запоминания запутанных параметров командной строки.
• **Простота применения.** ProtoFuzz должен быть готов к работе сразу после запуска. Вместо требования создавать громоздкие шаблоны структуры пакета данных мы воспользуемся структурой ранее перехваченных пакетов для построения фаззинговых шаблонов.
• **Доступ ко всем уровням протокола.** Некоторые сетевые фаззеры сосредоточены на данных пакета, а не на самих заголовках протокола. ProtoFuzz должен уметь подвергать фаззингу любую часть пакета: от заголовка Ethernet до данных TCP/UDP.

Создавая ProtoFuzz, мы определённо не стремимся заменить существующие инструменты. Это попытка построить базовую платформу сетевого фаззинга из среды Windows, которая послужит как учебным средством, так и открытым проектом, доступным для расширения всем заинтересованным.

Выбор языка

Как и в случае с FileFuzz, нашим инструментом фаззинга форматов файлов в Windows, для создания этого сетевого фаззера с графическим интерфейсом мы обратились к C# и Microsoft .NET Framework. Платформа .NET берёт на себя значительную часть тяжёлой работы при разработке графических приложений, позволяя сосредоточиться на бизнес-логике приложения или, в нашем случае, на логике разрушения бизнеса. Приложения .NET требуют предварительной установки .NET Framework. Однако рост распространённости приложений на .NET превращает это в небольшое неудобство, поскольку в большинстве систем библиотека уже установлена.

Библиотека перехвата пакетов

Одно из ключевых решений при создании фаззера сетевых протоколов - выбор подходящей библиотеки перехвата пакетов. Эта библиотека отвечает за три основных компонента: перехват, разбор и передачу пакетов. Хотя такую функциональность определённо можно создать с нуля, обычно это неразумно, поскольку уже существует ряд надёжных открытых библиотек перехвата пакетов.

Выбор разработки ProtoFuzz на платформе Windows автоматически ограничивает варианты. Обычно предпочтительной библиотекой для Microsoft Windows была бы WinPcap,[9] превосходная библиотека перехвата пакетов, появившаяся, когда Пьеро Виано (Piero Viano) перенёс libpcap в Windows в рамках своей дипломной работы. WinPcap уже много лет развивается как открытый проект и потому превратился в сильную кодовую базу с поддержкой большинства основных протоколов. Более того, она является обязательным компонентом многих коммерческих и открытых приложений, которым требуется работа с пакетами данных, например Wireshark, ранее Ethereal, и Core Impact.[10] На веб-сайте WinPcap перечислено более 100 инструментов, использующих библиотеку для перехвата пакетов, и это лишь те, о которых известно разработчикам!

Хотя WinPcap рассматривалась для ProtoFuzz, выбор C# в качестве языка разработки осложнил работу с популярной библиотекой. WinPcap написана на C; предпринималось несколько попыток написать оболочки, упрощающие применение WinPcap из приложения C#, но нам ещё не встречался комплексный проект, включающий все возможности WinPcap. Для проекта также рассматривалась PacketX,[11] библиотека классов COM, предоставляющая оболочку WinPcap, но в конечном счёте от неё отказались, поскольку она является коммерческой, а при написании книги мы стремились по возможности использовать свободно распространяемые библиотеки.

После долгих поисков мы с радостью наткнулись на Metro Packet Library. Хотя она не столь мощна, как WinPcap, библиотека целиком написана на C#, поставляется с хорошими учебными материалами и документацией и более чем подходила для базового фаззера, который мы хотели создать. Единственная слабость Metro - ограниченные возможности разбора. В ней есть классы для распознавания всех высокоуровневых заголовков пакетов: Ethernet, TCP, UDP, ICMP, IPv4 и Address Resolution Protocol (ARP), но пока нет классов, способных понимать данные пакетов ниже этих заголовков. С учётом скромных целей ProtoFuzz это снова не представляло серьёзной проблемы, а поскольку Metro является открытым проектом, мы всегда можем расширить существующие классы, разработав необходимую функциональность.

Архитектура

Довольно теории; пора программировать. Как всегда, весь исходный код ProtoFuzz доступен на веб-сайте www.fuzzing.org. Мы не станем разбирать все части кода, но в следующих разделах выделим некоторые наиболее важные сегменты.

Сетевой адаптер

Мы хотим, чтобы ProtoFuzz мог перехватывать трафик, используемый для создания фаззинговых шаблонов. Для этого пользователь сначала должен выбрать сетевой адаптер, который будет переведён в неразборчивый режим прослушивания. Вместо требования вручную узнать и указать имя или идентификатор адаптера мы хотим предоставить простое раскрывающееся меню, где можно выбрать любой активный сетевой адаптер системы. К счастью, Metro предоставляет классы, делающие этот процесс относительно простым, как показано в следующем фрагменте:

```
private const string DRIVER_NAME = @"\\.\ndisprot";
NdisProtocolDriverInterface driver = new NdisProtocolDriverInterface();
try
{
    driver.OpenDevice(DRIVER_NAME);
}
catch (SystemException ex)
{
    string error = ex.Message;

    error += "\n";
    error += "Please ensure that you have correctly installed the " +
        DRIVER_NAME + " device driver.";
    error += "Also, make sure it has been started.";
    error += "You can start the driver by typing \"net start " +
        DRIVER_NAME.Substring(DRIVER_NAME.LastIndexOf("\\") + 1) +
        "\" at a command prompt.";
    error += "To stop it again, type \"net stop " +
        DRIVER_NAME.Substring(DRIVER_NAME.LastIndexOf("\\") + 1) +
        "\" in a command prompt.";
    error += "\n";
    error += "Press 'OK' to continue...";
    MessageBox.Show(error, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    return;
}
foreach (NetworkAdapter adapter in driver.Adapters)
{
    cbxAdapters.Items.Add(adapter.AdapterName);
    if (cbxAdapters.Items.Count > 0)
        cbxAdapters.SelectedIndex = 0;
}
```

Сначала мы создаём новый объект `NdisProtocolDriverInterface`, а затем вызываем функцию `OpenDevice()` с сетевым адаптером `ndisprot`, который пользователь уже должен был вручную установить. Если адаптер недоступен, будет перехвачено `SystemException`, а пользователь получит указание установить и запустить адаптер. После успешного завершения у нас появляется массив `NetworkAdapter` со сведениями обо всех доступных сетевых адаптерах. Затем простым циклом `foreach` мы перебираем массив и добавляем свойство `AdapterName` в элемент управления с полем со списком.

Перехват данных

Открыв адаптер, теперь можно перевести его в неразборчивый режим и начать перехват трафика:

```
try
{
    maxPackets = Convert.ToInt32(tbxPackets.Text);
    capturedPackets = new byte[maxPackets][];
    driver.BindAdapter(driver.Adapters[cbxAdapters.SelectedIndex]);

    ThreadStart packets = new ThreadStart(capturePacket);
    captureThread = new Thread(packets);
    captureThread.Start();
}
catch (IndexOutOfRangeException ex)
{
    MessageBox.Show(ex.Message +
        "\nYou must select a valid network adapter.",
        "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Сначала мы создаём двумерный массив `capturedPackets`, содержащий массив перехваченных пакетов, каждый из которых, в свою очередь, является массивом байтов. Затем вызываем функцию `BindAdapter()`, связывающую выбранный сетевой адаптер с ранее созданным `NdisProtocolDriverInterface` (`driver`). После этого вызываем `capturePacket` в отдельном потоке. Важно запустить отдельный поток, чтобы графический интерфейс не завис во время перехвата пакетов.

```
private void capturePacket()
{
    while (packetCount < maxPackets)
    {
        byte[] packet = driver.RecievePacket();
        capturedPackets[packetCount] = packet;
        packetCount++;
    }
}
```

Чтобы получить массив байтов перехваченного пакета, достаточно вызвать функцию `ReceivePacket()`. После этого остаётся лишь добавить пакет в массив `capturedPackets`.

Разбор данных

Разбор данных выполняется в два этапа, поскольку перехваченные пакеты отображаются двумя разными способами. Следуя компоновке Wireshark, после выбора перехваченного пакета его краткое содержимое показывается в элементе управления `TreeView`, а все необработанные байты одновременно выводятся в `RichTextBox`. Пользователь получает понятный общий обзор, но сохраняет управление отдельными байтами для фаззинга. Мы попытались связать два окна, выделяя соответствующие необработанные байты красным текстом при выборе строки в `TreeView`. Полная компоновка ProtoFuzz с перехваченным пакетом показана на рисунке 16.2.

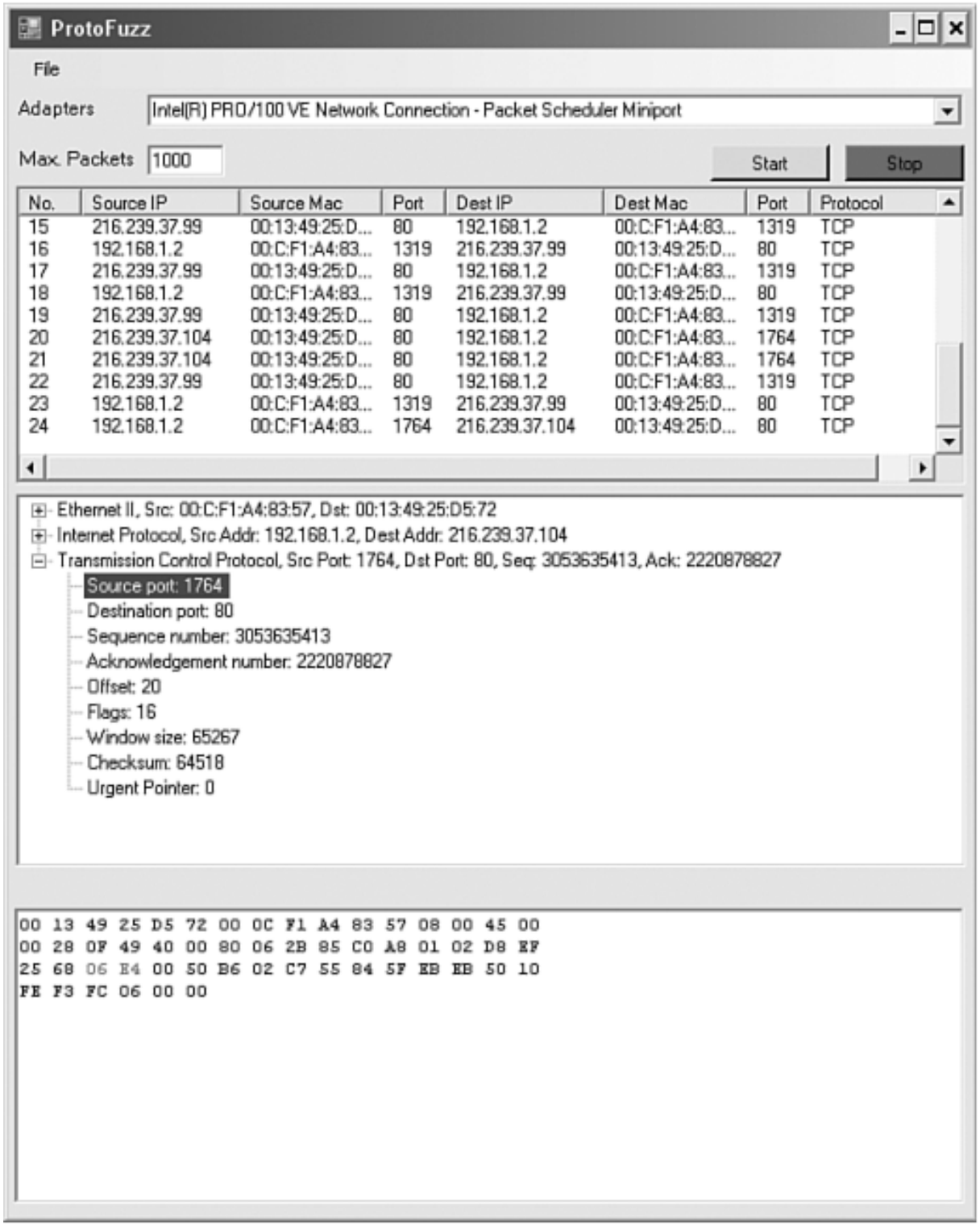


Рисунок 16.2. ProtoFuzz отображает перехваченный пакет

Сводное представление перехваченного пакета находится в средней области `TreeView`. Код создания `TreeView` содержится в функции `packetTvwDecode()`, где есть отдельные блоки для разбора заголовков Ethernet, TCP, UDP, IP, ARP и ICMP. Для простоты следующий сегмент показывает разбор только заголовков Ethernet:

```
Ethernet802_3 ethernet = new Ethernet802_3(capPacket);
strSourceMacAddress = ethernet.SourceMACAddress.ToString();
strDestMacAddress = ethernet.DestinationMACAddress.ToString();
strEthernet = "Ethernet II, Src: " + strSourceMacAddress +
    ", Dst: " + strDestMacAddress;
```

```
strSrcMac = "Source: " + strSourceMacAddress;
strDstMac = "Destination: " + strDestMacAddress;
strEthernetType = "Type: " + ethernet.NetworkProtocol.ToString();
strData = "Data: " + ethernet.Data.ToString();

TreeNode nodeEthernet = tvwDecode.Nodes.Add(strEthernet);
TreeNode nodeEthernetDstMac = nodeEthernet.Nodes.Add(strDstMac);
TreeNode nodeEthernetSrcMac = nodeEthernet.Nodes.Add(strSrcMac);
TreeNode nodeType = nodeEthernet.Nodes.Add(strEthernetType);
TreeNode nodeData = nodeEthernet.Nodes.Add(strData);
```

Как видно, класс `Ethernet802_3` создаётся с массивом байтов из `capPacket`, переданным конструктору, а каждый узел, показанный в `TreeView`, является простым свойством класса. Следующий сегмент представляет базовый цикл, отображающий необработанные байты пакета в виде сетки. Цикл выводит по 16 байтов в строке.

```
static string PrintData(byte [] packet)
{
    string sData = null;
    int nPosition = 0, nColumns = 16;
    for (int i = 0; i < packet.Length; i++)
    {
        if (nPosition >= nColumns)
        {
            nPosition = 1;
            sData += "\n";
        }
        else
            nPosition++;
        byte nByte = (byte) packet.GetValue(i);
        if (nByte < 16)
            sData += "0";
        sData += nByte.ToString("X", oCulture.NumberFormat) + " ";
    }
    sData += "\n";
    return (sData);
}
```

Фаззинговые переменные

Фаззинговые переменные представлены байтами, заключёнными либо в квадратные скобки (`[]`) для полного перебора, либо в угловые (`<>`) для строкового фаззинга. При отправке пакета код просто читает необработанные пакеты и ищет парные скобки. При обнаружении таких переменных исходные данные заменяются фаззинговыми перед отправкой пакета.

Шестнадцатеричное кодирование и декодирование

Последняя задача - кодирование и декодирование шестнадцатеричных представлений байтов, используемых для отображения содержимого перехваченных пакетов. .NET Framework предоставляет функцию `ToString("X")` для преобразования массива байтов в шестнадцатеричную строку, но не имеет подобной возможности преобразования строки в массив байтов.[12] Поэтому был добавлен класс `HexEncoding`, значительная часть которого заимствована из статьи «Converting Hexadecimal String to/from Byte Array in C#» на www.codeproject.com.

Практический пример

Теперь, когда сетевой фаззер создан, пора убедиться, что он работает. ProtoFuzz является однократным инструментом: он принимает пакет, многократно изменяет его, а затем отправляет цели. Он не способен передать последовательность начальных пакетов, предназначенных, например, для перевода цели в уязвимое состояние до отправки пакета, вызывающего сбой.

Рассмотрим переполнение буфера в сервере SMTP, возникающее в команде `RCPT TO`, как показано ниже:

```
220 smtp.example.com ESMTP
HELO mail.heaven.org
250 smtp.example.com Hello smtp.example.com
MAIL FROM:god@heaven.org
250 2.1.0 god@heaven.org... Sender ok
RCPT TO:[Ax1000]
```

В этом примере серверу SMTP потребуется отправить несколько запросов. Нужно установить первоначальное соединение TCP, а затем отправить по одному запросу для команд `HELO`, `MAIL FROM` и `RCPT TO`, выделенных в оригинале полужирным. Начальные запросы необходимы, чтобы перевести сервер в состояние готовности принять команду `RCPT TO` и длинную строку, которая в конечном счёте вызовет переполнение. Такая уязвимость лучше подходит для инструмента вроде SPIKE, позволяющего создавать сценарии, определяющие структуру пакетов и отправляющие несколько запросов.

ProtoFuzz лучше подходит для ситуации, когда требуется просто перехватить передаваемый пакет, выбрать определённую область данных и немедленно начать фаззинг. Возможно, вы имеете дело с проприетарным протоколом и не располагаете достаточными сведениями для создания сценария, подробно описывающего структуру протокола. Либо вы просто хотите провести быструю проверку, не тратя время на описание протокола для фаззера.

Zero Day Initiative (ZDI) компании TippingPoint раскрыла подходящую для наших целей уязвимость переполнения стека в Mercury LoadRunner компании Hewlett-Packard.[13] Переполнение возникает в агенте, привязывающемся к TCP-порту 54345 и ожидающем входящих соединений. Перед приёмом пакетов агент не требует никакой аутентификации, поэтому мы получаем искомую однократную уязвимость. Кроме того, агент использует проприетарный протокол, а бюллетень безопасности содержит недостаточно подробностей для создания демонстрационного эксплойта с нуля. Он сообщает лишь, что переполнение вызывается чрезмерно длинным значением поля `server_ip_name`. Поэтому мы воспользуемся ProtoFuzz, чтобы перехватить допустимый трафик, выявить поле `server_ip_name` и соответствующим образом изменить пакет.

Установив и правильно настроив Mercury LoadRunner, можно начать поиск операции, которая может содержать уязвимость. При прослушивании трафика мы выявляем следующий пакет, содержащий сочетание двоичных данных и читаемого текста ASCII, а также явно искомое поле `server_ip_name`.

```
0070  2b 5b b6 00 00 05 b2 00  00 00 07 00 00 00 12 6d   +[...... .......m
0080  65 72 63 75 72 79 32 3b  31 33 30 34 3b 31 33 30   ercury2; 1304;130
0090  30 00 00 00 00 05 88 28  2d 73 65 72 76 65 72 5f   0......( -server_
00a0  69 70 5f 6e 61 6d 65 3d                           ip_name=
```

Перехватив пакет, мы выделяем значение поля `server_ip_name`, щёлкаем правой кнопкой мыши и выбираем Fuzz > Strings. Выбранные байты будут заключены в угловые скобки (<>). При разборе перехваченного пакета ProtoFuzz заменит обозначенные байты каждым значением из файла `Strings.txt`. Поэтому для вызова переполнения потребуется поэкспериментировать, добавляя в файл всё более длинные строки. Для каждой строки `Strings.txt` будет отправлен отдельный фаззинговый пакет.

Во время отправки фаззинговых данных цели важно следить за исключениями. Приложение `Magentproc.exe` привязывается к TCP-порту 54345, поэтому мы подключим к этому процессу отладчик и начнём фаззинг. После вызова сбоя вывод OllyDbg выглядит следующим образом:

```
Registers
EAX 00000000
ECX 41414141
EDX 00C20658
EBX 00E263BC
ESP 00DCE7F0
EBP 41414141
ESI 00E2549C
EDI 00661221 two_way_.00661221
EIP 41414141

Stack
00DCE7F0
00000000
00DCE7F4
41414141
00DCE7F8
41414141
00DCE7FC
41414141
00DCE800
41414141
00DCE804
41414141
00DCE808
41414141
00DCE80C
41414141
00DCE810
41414141
00DCE814
41414141
```

Подобно практическому примеру NetMail из предыдущей главы, это явный пример удалённо эксплуатируемого переполнения стека. Тот факт, что служба не требует аутентификации, определённо повышает риск этой уязвимости в популярном инструменте контроля качества.

Преимущества и возможности улучшения

В настоящее время ProtoFuzz не способен обрабатывать блоки данных, в которых последовательности байтов предшествует размер данных. В такой ситуации для сохранения допустимости пакета фаззинг байтов данных требует параллельного обновления размера блока. Эта возможность значительно расширила бы функциональность ProtoFuzz и была бы довольно проста в реализации.

Сейчас ProtoFuzz также не предоставляет функции проверки доступности целевого узла во время фаззинга. Как уже упоминалось ранее в главе, после фаззингового пакета можно отправлять проверочные пакеты, чтобы определить состояние целевого приложения. Если цель не отвечает, фаззинг можно остановить, поскольку дальнейшие тестовые примеры отправлялись бы в пустоту. Эта возможность также помогла бы конечному пользователю сопоставлять фаззинговые пакеты, вызвавшие аномальное поведение цели. ProtoFuzz также выиграл бы от расширения для обработки сложных пар «запрос - ответ». Вместо простой отправки фаззинговых пакетов ProtoFuzz мог бы до фаззинга переводить цель в определённое состояние, сначала передавая заведомо исправные пакеты и ожидая надлежащих ответов, а также разбирая и интерпретируя их содержимое.

Обнаружение сбоев можно дополнительно улучшить, разработав удалённый агент на целевой машине, который следил бы за состоянием целевого приложения. Затем агент мог бы обмениваться данными с ProtoFuzz и останавливать фаззинг при обнаружении сбоя. Это помогло бы непосредственно выявить пакет, вызвавший сбой.

Итоги

ProtoFuzz предстоит выполнить длинный список задач разработки, прежде чем он сможет претендовать на звание мощного фаззера. Однако он выполняет своё назначение, демонстрируя базовую основу фаззинга сетевых протоколов, которую легко расширить для конкретных целей. Ключевым фактором при создании сетевого фаззера был поиск библиотеки, способной выполнять перехват, разбор и передачу необработанных пакетов. Избегайте библиотек, не позволяющих собирать необработанные пакеты, если только вы не создаёте фаззер конкретных сетевых протоколов, которому не нужен доступ ко всем заголовкам. Для наших целей библиотека Metro предоставила ProtoFuzz прочную основу, а .NET Framework позволила создать интуитивно понятную графическую внешнюю часть. Мы призываем вас развивать ProtoFuzz и направлять его в разные стороны.

Сноски

[1] [назад] http://www.cert.org/advisories/CA-2003-04.html
[2] [назад] http://www.microsoft.com/technet/security/bulletin/MS02-039.mspx
[3] [назад] http://pedram.openrce.org/__research/slammer/slammer.txt
[4] [назад] http://isc.sans.org/portreport.html?sort=targets
[5] [назад] http://atlas.arbor.net/
[6] [назад] http://www.ee.oulu.fi/research/ouspg/protos/
[7] [назад] http://www.codenomicon.com/products/
[8] [назад] http://sourceforge.net/projects/dotmetro/
[9] [назад] http://www.winpcap.org/
[10] [назад] http://www.coresecurity.com/products/coreimpact/index.php
[11] [назад] http://www.beesync.com/packetx/index.html
[12] [назад] http://www.codeproject.com/csharp/hexencoding.asp
[13] [назад] http://www.zerodayinitiative.com/advisories/ZDI-07-007.html

Глава 17. Фаззинг веб-браузеров

> «Одна из обнаруженных мною общих закономерностей состоит в том, что ожидания поднимаются выше ожидаемого».
>
> Джордж У. Буш, Лос-Анджелес, 27 сентября 2000 года

Клиентские уязвимости быстро оказались в центре внимания, поскольку злоумышленники активно используют их для фишинговых атак, кражи личности и создания крупных сетей ботов, ботнетов. Уязвимости веб-браузеров создают богатую целями среду для таких атак, поскольку слабость популярного браузера подвергает опасности миллионы ничего не подозревающих жертв. Клиентские атаки всегда требуют некоторой социальной инженерии: злоумышленник должен сначала заставить потенциальную жертву посетить вредоносную веб-страницу. Часто этот процесс облегчается спамом или использованием дополнительной уязвимости популярного веб-сайта. Добавьте к этому тот факт, что отдельные пользователи Интернета обычно обладают минимальными знаниями компьютерной безопасности, и станет неудивительно, что многие злоумышленники переключают внимание на клиентские уязвимости.

Во многих отношениях 2006 год стал временем выхода ошибок браузеров на передний план. Были обнаружены уязвимости всех популярных веб-браузеров, включая Microsoft Internet Explorer и Mozilla Firefox. Дефекты находили в разборе искажённых тегов HTML, JavaScript, собственных файлов изображений JPG, GIF и PNG, CSS и элементов управления ActiveX. Были обнаружены десятки критических уязвимостей ActiveX, затрагивавших как стандартные установки операционной системы Microsoft Windows, так и приложения сторонних поставщиков. Кроме того, нашли сотни некритических уязвимостей ActiveX. Хотя исследователи безопасности и раньше проявляли интерес к аудиту ActiveX и COM, в 2006 году этот интерес явно достиг пика. Важным фактором резкого роста числа уязвимостей элементов управления ActiveX стала общедоступность новых удобных инструментов их аудита. В этой главе мы рассмотрим применение фаззинга для обнаружения уязвимостей веб-браузеров. Если история служит ориентиром, предстоит найти ещё множество ошибок браузеров.

Что такое фаззинг веб-браузеров?

Первоначально веб-браузеры создавались исключительно для просмотра Веба и разбора страниц HTML, но развились в компьютерный эквивалент швейцарского армейского ножа. Современные браузеры умеют обрабатывать динамический HTML, таблицы стилей, несколько языков сценариев, апплеты Java, каналы Really Simple Syndication (RSS), соединения FTP и многое другое. Доступно множество расширений и подключаемых модулей, превращающих веб-браузер в очень гибкое приложение. Такая гибкость позволяет компаниям вроде Google переносить в Веб ряд распространённых настольных приложений. К несчастью для конечного пользователя, дополнительная функциональность увеличивает и поверхность атаки. Неудивительно, что по мере добавления в веб-браузеры всё новых возможностей обнаруживается всё больше уязвимостей.

МЕСЯЦ ОШИБОК БРАУЗЕРОВ

В июле 2006 года Х. Д. Мур (H.D. Moore) объявил «Месяц ошибок браузеров» (Month of Browser Bugs, MoBB).[1] В течение июля Х. Д. каждый день публиковал новую браузерную уязвимость и к концу месяца раскрыл сведения о 31 уязвимости. Хотя большинство затрагивало Microsoft Internet Explorer, были представлены все основные веб-браузеры, включая Safari, Mozilla, Opera и Konqueror.[2] Большинство уязвимостей приводило лишь к отказу в обслуживании из-за аварийного завершения браузера вследствие таких проблем, как разыменование нулевого указателя. Хотя серьёзность этих уязвимостей была минимальна, незадолго до MoBB Skywing и skape, участник проекта Metasploit вместе с Х. Д. Муром, раскрыли подробности дефекта проектирования в цепочке необработанных фильтров исключений Internet Explorer, который при определённых условиях мог использовать иначе неэксплуатируемые уязвимости для полного выполнения кода.[3] Это потенциально повышало риск некоторых уязвимостей, опубликованных во время MoBB. Хотя некоторые не соглашались с раскрытием подробностей уязвимостей до появления исправлений, оно определённо открыло людям глаза на распространённость уязвимостей веб-браузеров и создаваемый ими риск.

Цели

Каждый раз, когда выходит исследование, показывающее, что Firefox безопаснее Internet Explorer, появляется новый и столь же предвзятый отчёт с противоположным результатом. С исследовательской точки зрения вопрос не имеет значения. Все веб-браузеры содержат серьёзные уязвимости, а фаззинг является эффективным способом их выявления. Как и для любой программы, чем популярнее приложение, тем больше людей непосредственно затронет уязвимость. Хотя статистика постоянно меняется, независимо от источника вы, скорее всего, обнаружите, что львиная доля конечных пользователей принадлежит Internet Explorer, а Firefox уверенно занимает второе место. Естественно, популярность Internet Explorer объясняется его включением по умолчанию в операционную систему, работающую на большинстве персональных компьютеров мира. Несмотря на это, Firefox продолжает составлять Internet Explorer серьёзную конкуренцию. Другие веб-браузеры, например Netscape, Opera, Safari и Konqueror, обычно имеют небольшую общую долю пользователей и вместе занимают далёкое третье место.

Методы

При фаззинге веб-браузеров необходимо сделать два выбора. Сначала определить подход к проведению фаззинга, а затем выбрать часть браузера, на которой сосредоточиться. Учитывая постоянно растущую функциональность веб-браузеров, получение полного покрытия кода может быть чрезвычайно трудной задачей. Для комплексного аудита рассчитывайте использовать несколько инструментов и подходов.

Подходы

Для фаззинга веб-браузеров существует несколько базовых подходов, и конкретный выбор зависит от проверяемой части браузера. В целом доступны следующие варианты:

• **Обновление страниц HTML.** В этом сценарии фаззинговые веб-страницы создаются веб-сервером и содержат средство регулярного обновления. Это можно реализовать разными способами, например тегом `META` с `HTTP-EQUIV` для обновления или клиентским сценарием. При обновлении выдаётся новая версия страницы с новым фаззинговым содержимым. Например, Mangleme,[4] утилита фаззинга для обнаружения дефектов разбора HTML в веб-браузерах, включает следующую строку в часть `HEAD` каждой фаззинговой страницы:

```
<META HTTP-EQUIV="Refresh" content="0;URL=mangle.cgi">
```

Этот тег немедленно обновляет страницу mangleme и перенаправляет на `mangle.cgi`, где веб-браузер ожидает новую искажённую страницу. Похожую задачу выполняет следующий клиентский JavaScript, обновляющий страницу через 2000 миллисекунд:

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!--
var time = null
function move() {
    window.location = 'http://localhost/fuzz'
}
//-->
</SCRIPT>
</HEAD>
<BODY ONLOAD="timer=setTimeout('move()',2000)">
[Page Content]
</BODY>
```

```
</HTML>
```

• **Загрузка страниц.** Фаззинг можно выполнять исключительно на стороне клиента, непосредственно загружая фаззинговую страницу в веб-браузер. С помощью параметров командной строки целевому браузеру можно приказать открыть фаззинговую страницу, находящуюся на локальной машине. Например, следующая команда непосредственно откроет файл `fuzz.html` в Internet Explorer:

```
C:\>"C:\Program Files\Internet Explorer\iexplore.exe" C:\temp\fuzz.html
```

• **Выбор отдельных объектов браузера целью.** В некоторых ситуациях браузерный фаззинг можно проводить вообще без запуска браузера. Это возможно, когда нашей целью является объект браузера, а браузер служит лишь средством доступа к нему. Например, можно подвергать фаззингу элемент управления ActiveX, к которому способен удалённо обратиться вредоносный веб-сайт, если объект помечен как «безопасный для сценариев». В такой ситуации для определения уязвимости не требуется запускать элемент ActiveX из браузера. COMRaider,[5] созданный Дэвидом Циммером (David Zimmer), использует этот подход к элементам ActiveX.

Входные данные

Как и для других целей, фаззинг веб-браузеров требует выявить различные типы пользовательских входных данных. И снова необходимо выйти за пределы предвзятых представлений о том, что считается вводом. В случае веб-браузера злоумышленником является веб-сервер, доставляющий содержимое, которое затем интерпретирует браузер. Поэтому входными данными является всё доставленное сервером, а не только код HTML.

Заголовки HTML

Первая часть данных, получаемая браузером, - заголовки HTML. Хотя браузер не отображает их непосредственно, заголовки содержат важные сведения, способные влиять на поведение веб-браузера и отображение последующего содержимого.

Пример такой уязвимости - CVE-2003-0113,[6] вошедшая в бюллетень безопасности Microsoft MS03-015.[7] Йоуко Пюннёнен (Jouko Pynnönen) обнаружил, что из-за неправильной проверки границ в `urlmon.dll` длинные значения заголовков `Content-type` и `Content-encoding`[8] могут привести к переполнению буфера в стеке. Было установлено, что этому типу атаки подвержены многочисленные версии Internet Explorer 5.x и 6.x.

Теги HTML

Содержимое самой страницы доставляется браузеру как последовательность тегов HTML. Технически браузер способен отображать различные языки разметки, например HTML, XML и XHTML, но для наших целей будем собирательно называть содержимое веб-страницы просто HTML.

По мере развития и расширения функциональности HTML становился всё более сложным стандартом. Добавьте к этому множество проприетарных тегов конкретных веб-браузеров и несколько языков разметки, которые способны интерпретировать браузеры, и вы увидите множество возможностей для исключений при проектировании браузера. Эти ошибки программирования, в свою очередь, часто можно выявить фаззингом.

HTML имеет относительно простую структуру. Существует несколько стандартов HTML, и для обозначения применяемого стандарта правильно сформированная веб-страница должна начинаться с объявления типа документа (Document Type Declaration, DTD). Например, следующее DTD сообщает браузеру, что страница должна соответствовать HTML 4.01:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
```

Когда мы думаем об HTML, обычно вспоминаем теги, определяющие содержимое документа. Эти теги называются элементами и обычно состоят из открывающего и закрывающего тегов. Помимо самих тегов элементы могут иметь атрибуты и содержимое. Следующий элемент HTML демонстрирует элемент шрифта:

```
<font color="red">Fuzz</font>
```

В этом элементе `color` является атрибутом элемента `font`, а слово `Fuzz` - его содержимым. Хотя это самые основы, важно отметить, что все компоненты кода HTML являются законными целями фаззинга. Веб-браузер предназначен для разбора и интерпретации кода HTML, а если код имеет неожиданный формат, разработчик мог не предусмотреть надлежащую обработку ошибок аномального фрагмента HTML.

Mangleme был создан для обнаружения уязвимостей в интерпретации веб-браузерами искажённых тегов HTML и привёл к обнаружению CVE-2004-1050,[9] исправленной в бюллетене безопасности Microsoft MS04-040.[10] Нед и Беренд-Ян Вевер (Ned and Berend-Jan Wever) с помощью mangleme обнаружили, что чрезмерно длинные атрибуты `SRC` или `NAME` в элементах `IFRAME`, `FRAME` и `EMBED` способны привести к переполнению буфера в куче. Впоследствии они опубликовали полный общедоступный код эксплойта[11] для этой проблемы. Другие утилиты фаззинга структуры HTML в веб-браузерах включают DOM-Hanoi[12] и Hamachi,[13] разработанные Х. Д. Муром и Авивом Раффом (Aviv Raff).

Теги XML

XML - универсальный язык разметки, производный от Standardized General Markup Language (SGML) и широко используемый для передачи данных по HTTP. Были разработаны стандартизованные реализации XML, определяющие тысячи форматов данных, например RSS, Application Vulnerability Description Language (AVDL), Scalable Vector Graphics (SVG) и так далее. Как и HTML, современные веб-браузеры способны разбирать и интерпретировать элементы XML и связанные с ними атрибуты и содержимое. Поэтому неожиданные данные, встреченные браузером при интерпретации элементов, могут привести к необработанному исключению. Vector Markup Language (VML) - спецификация XML для определения векторной графики. В библиотеках Internet Explorer и Outlook были обнаружены уязвимости, вызванные неспособностью обрабатывать определённые искажённые теги VML. 19 сентября 2006 года Microsoft выпустила рекомендацию по безопасности 925568,[14] подробно описав переполнение буфера в стеке механизма отображения векторной графики (`vgx.dll`), способное привести к удалённому выполнению кода. Рекомендация появилась после свидетельств активной эксплуатации, заставивших Microsoft семь дней спустя выпустить внеплановое исправление.[15] Позднее в той же библиотеке было раскрыто целочисленное переполнение, исправленное в январе 2007 года.[16]

Элементы управления ActiveX

Элементы управления ActiveX представляют собой проприетарную технологию Microsoft на основе COM, предоставляющую многократно используемые программные компоненты.[17] Веб-разработчики часто встраивают элементы ActiveX для расширенной функциональности, обычно недоступной веб-приложению. Главный недостаток ActiveX состоит в том, что проприетарная природа ограничивает применение браузером Internet Explorer и операционной системой Windows. Тем не менее элементы ActiveX широко используются на многих веб-сайтах. Они обладают огромными возможностями и после признания доверенными получают полный доступ к операционной системе, поэтому их никогда не следует запускать, если они не поступили из доверенного источника.

Многочисленные уязвимости обнаруживались в элементах ActiveX, входящих в операционную систему Windows или сторонние пакеты программ. Если такие элементы помечены как «безопасные для инициализации» и «безопасные для сценариев», к ним может обратиться удалённый веб-сервер и использовать для компрометации целевой машины.[18] Возникает опасная ситуация: пользователь может первоначально установить элемент ActiveX из доверенного источника, но если позднее в нём обнаружится уязвимость, например переполнение буфера, тот же доверенный элемент сможет использовать вредоносный веб-сайт.

COMRaider - превосходный управляемый мастером инструмент фаззинга для выявления потенциально уязвимых элементов ActiveX в целевой системе. Кроме того, он умеет фильтровать проверяемые элементы, выбирая лишь те, к которым может обратиться вредоносный веб-сервер, поскольку это резко повышает риск уязвимости.

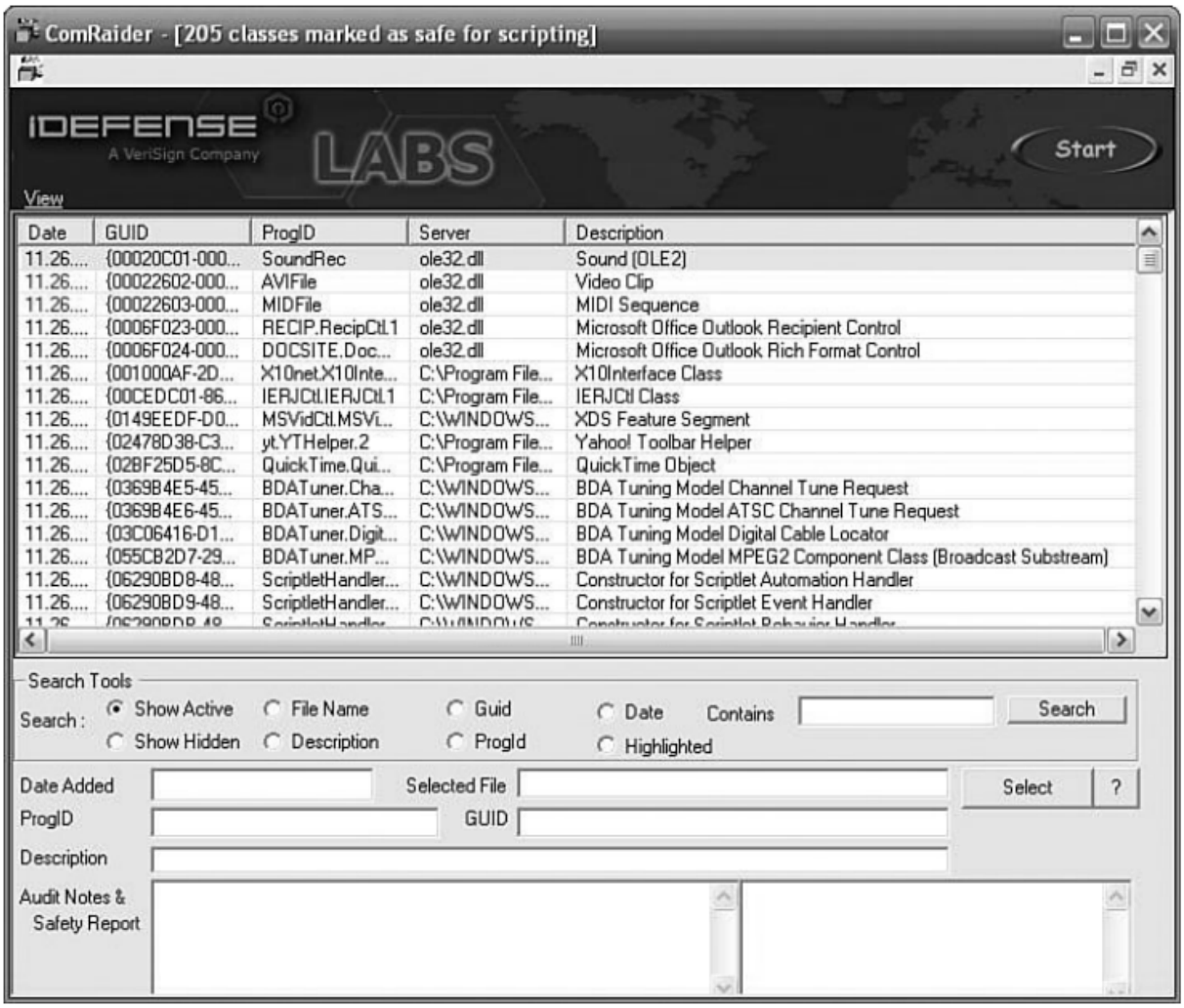


Рисунок 17.1. COMRaider

COMRaider полезен для полного аудита всех элементов ActiveX в системе. Сначала он выявляет все элементы ActiveX и связанные методы и входные данные, а затем последовательно подвергает фаззингу все выбранные элементы. COMRaider имеет встроенный отладчик и потому способен обнаруживать обработанные и необработанные исключения. Он также включает возможности распределённого аудита, позволяя группе совместно использовать результаты прошлых проверок. На рисунке 17.1 показан список, созданный COMRaider после полного аудита системы для всех элементов ActiveX, помеченных как безопасные для сценариев.

AxMan[19] - ещё один свободно доступный фаззер ActiveX. AxMan был создан Х. Д. Муром и использовался для обнаружения большинства уязвимостей ActiveX, опубликованных во время «Месяца ошибок браузеров» в июле 2006 года.[20]

Каскадные таблицы стилей

CSSDIE[21] - фаззер CSS, созданный Х. Д. Муром, Мэттом Мёрфи (Matt Murphy), Авивом Раффом и Тьерри Цоллером (Thierry Zoller) и использованный во время «Месяца ошибок браузеров» для выявления проблемы повреждения памяти браузера Opera.[22] Было обнаружено, что присваивание свойству CSS `background` элемента DHTML чрезмерно длинного URL приводит к клиентскому отказу в обслуживании с аварийным завершением браузера.

Другие проблемы CSS приводили к разнообразным уязвимостям, например CVE-2005-4089, исправленной в MS06-021,[23] которая позволяла злоумышленнику обходить междоменные ограничения Internet Explorer. Это было возможно потому, что сайты могли использовать директиву `@import` для загрузки файлов из других доменов, не являвшихся правильными файлами CSS. Матан Гиллон (Matan Gillon) из hacker.co.il продемонстрировал использование уязвимости для просмотра результатов Google Desktop Search (GDS) другого человека.[24] Хотя атака больше не работает из-за изменений Google, она стала превосходным примером сочетания простой уязвимости Internet Explorer с возможностями другого приложения, в данном случае GDS, для относительно сложной атаки. В CSS формат элемента определяется внутри фигурных скобок, а свойство и значение разделяются двоеточием. Например, тег ссылки можно определить как `a {color: white}`. Когда директивой `@import` импортировался файл, не являвшийся CSS, Internet Explorer пытался интерпретировать его как CSS, и всё содержимое после открывающей фигурной скобки можно было затем получить через свойство `cssText`. Матан смог использовать атаку для кражи уникального ключа GDS посетителя веб-страницы: сначала он импортировал страницу Google News как файл CSS с запросом `}{`. Результаты запроса попадали в свойство `cssText`, откуда можно было получить уникальный ключ GDS пользователя. После этого злоумышленник мог другой директивой `@import` импортировать результаты из локального GDS пользователя и на этот раз просматривать локальный индекс GDS.

Клиентские сценарии

Веб-браузеры используют разные языки клиентских сценариев для создания динамического содержимого веб-страниц. Хотя наиболее распространён JavaScript, существуют и другие, например VBScript, JScript[25] и ECMAScript.[26] Мир языков клиентских сценариев кровосмесителен: многие языки производны друг от друга. Например, ECMAScript является лишь стандартизованной версией JavaScript, а JScript - просто реализация JavaScript компании Microsoft. Эти языки предоставляют веб-сайтам мощные возможности, но также породили множество проблем безопасности.

Эксплойты с клиентскими языками сценариев обычно не эксплуатируют уязвимости самих механизмов сценариев, а лишь используют язык для обращения к другому уязвимому компоненту, например элементу ActiveX. Однако это не означает, что уязвимостей механизмов сценариев не существует. Проблемы повреждения памяти относительно обычны; пример - нулевая ссылка, вызываемая в Internet Explorer перебором собственной функции с помощью JavaScript.[27] Во время «Месяца ошибок браузеров» эта уязвимость демонстрировалась одной строкой JavaScript:

```
for (var i in window.alert) { var a = 1; }
```

Уязвимость механизма JavaScript Firefox, обнаруженная Azafran, приводила к утечке информации, позволявшей вредоносному веб-сайту удалённо получать содержимое произвольной памяти кучи.[28] Уязвимость была вызвана обработкой лямбда-выражений функцией `replace()`. Хотя злоумышленник не управлял получаемой памятью, в ней могли находиться конфиденциальные сведения, например пароли.

> **ЗАПОЛНЕНИЕ КУЧИ**
>
> Обычно клиентские языки сценариев используются не как вектор атаки, а для облегчения эксплуатации отдельного уязвимого компонента браузера. Например, эксплуатация браузерных уязвимостей повреждения памяти или висячих указателей обычно выполняется при содействии клиентского языка сценариев. JavaScript можно применять для различных задач эксплуатации, и он часто используется при переполнениях кучи. Это очевидный выбор благодаря широкой распространённости и кроссплатформенности.

Представьте, что вы обнаружили ошибку, приводящую к последовательности разыменований под вашим управлением. Предположим, в следующем примере вы полностью управляете регистром EAX:

```
MOV EAX, [EAX]
CALL [EAX+4]
```

Большинство значений EAX заставят эту пару инструкций вызвать нарушение доступа и аварийно завершить браузер. Для успешного перенаправления потока управления переданный адрес должен быть допустимым указателем на указатель на допустимый код, а такое в обычном адресном пространстве процесса трудно найти легко и надёжно. Однако с помощью JavaScript можно последовательно выделять крупные блоки данных в куче и преобразовать пространство памяти в нечто более дружелюбное к злоумышленнику. Поэтому клиентский JavaScript часто используется для заполнения кучи цепочкой NOP и шелл-кодом, повышая вероятность попадания в шелл-код при переходе в кучу. NOP означает No OPeration в ассемблерном коде и является операцией или последовательностью операций, фактически ничего не делающих. После передачи выполнения цепочке NOP оно продолжает проходить по цепочке, не меняя регистры и другие области памяти. Беренд-Ян Вевер, известный как SkyLined, разработал популярный метод распыления кучи в коде Internet Exploiter,[29] использовавшем значение `0x0D`. Выбор важен, поскольку значение может иметь двойной смысл. Во-первых, оно представляет пятибайтовую NOP-подобную инструкцию, эквивалентную `OR EAX, 0D0D0D0D`. Во-вторых, оно может быть самоссылочным указателем на допустимый адрес памяти. Интерпретация зависит от контекста ссылки.

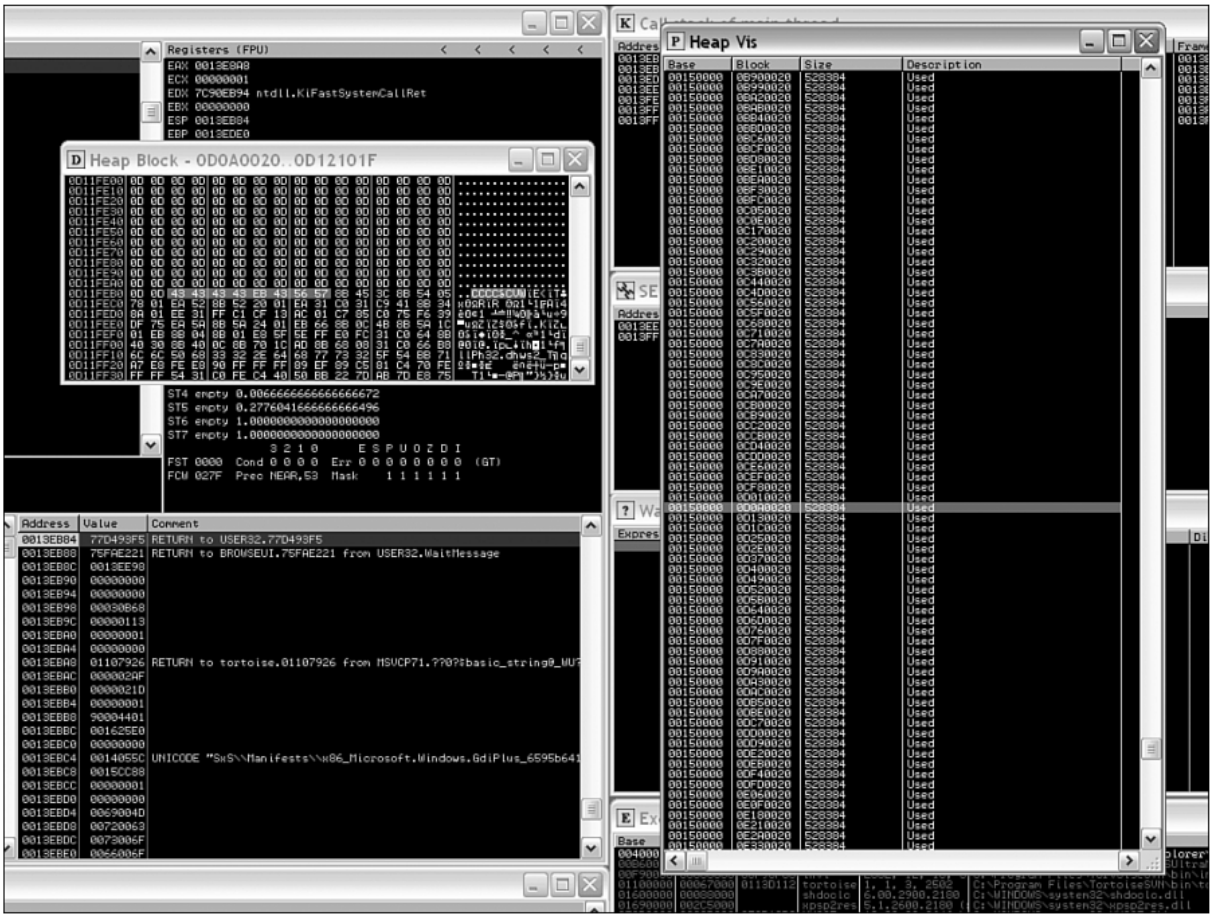


Рисунок 17.2. Заполнение кучи SkyLined в действии в Heap Vis для OllyDbg

Рассмотрим снимок подключаемого модуля OllyDbg Heap Vis,[30] показанный на рисунке 17.2.[31] Он демонстрирует, как модуль можно использовать для исследования и наглядного представления состояния памяти Internet Explorer во время эксплуатации с заполнением кучи.

Верхняя правая область снимка с подписью Heap Vis показывает список выделенных блоков кучи. Выделенная запись соответствует блоку размером приблизительно 500 Кбайт, начинающемуся по адресу `0x0D0A0020`. Это блок кучи, содержащий адрес `0x0D0D0D0D`. Верхняя левая область с подписью `Heap Block - 0D0A0020..0D12101F` показывает содержимое выделенного блока, а именно конец повторения строки `0x0D` и начало встроенного шелл-кода. Вернёмся к примеру с двумя строками разыменования и предположим, что присваиваем EAX значение `0x0D0D0D0D`. Первое разыменование `MOV EAX, [EAX]` оставляет то же значение EAX, поскольку все байты по целевому адресу равны `0x0D`. Следующая инструкция `CALL [EAX+4]` передаёт управление адресу `0x0D0D0D11`, расположенному посреди длинной последовательности `0x0D`. Процессор выполняет её как эквивалентные NOP инструкции до достижения шелл-кода, как показано на снимке.

Выбор значения `0x0D0D0D0D` для распыления кучи использует тот факт, что заполнение кучи обычно растёт от младших адресов к старшим. Выбор адреса, начинающегося с нуля, сокращает время заполнения до достижения целевого адреса. Более высокое значение вроде `0x44444444` потребует больше времени и повысит вероятность того, что целевой пользователь завершит браузер до успешной эксплуатации. Для дополнительной наглядности рассмотрим два «плохих» варианта: `0x01010101`, преобразующийся в `ADD`

`[ECX], EAX`, и `0x0A0A0A0A`, преобразующийся в `OR CL, [EDX]`. Первый, скорее всего, потерпит неудачу из-за недопустимого адреса записи в ECX, второй - из-за недопустимого адреса чтения в EDX. Другой хороший пример - `0x05050505`, преобразующийся в `ADD EAX, 0x05050505`. Такая последовательность встречается в некоторых общедоступных эксплойтах.

Flash

Хотя Adobe Flash Player является сторонним дополнением веб-браузера, а не встроенной функцией, он получил настолько широкое распространение, что сегодня в большинстве браузеров установлена та или иная версия Flash Player. Двоичные файлы Flash обычно имеют расширение `.swf` и могут открываться самостоятельно, но чаще встраиваются как объекты веб-страниц, загружаемые браузером и затем запускаемые во Flash Player. Из-за двоичной природы файлов Flash при фаззинге можно применить один из двух альтернативных подходов. Заведомо исправный файл `.swf` можно подвергнуть фаззингу, как описано в главах 11 «Фаззинг форматов файлов», 12 «Фаззинг форматов файлов: автоматизация в UNIX» и 13 «Фаззинг форматов файлов: автоматизация в Windows», где подробно рассматриваются методы фаззинга форматов файлов. В ноябре 2005 года eEye раскрыла подробности уязвимости доступа к памяти в Macromedia Flash 6 и 7. Хотя неизвестно, как именно она была обнаружена, вероятным подходом представляется фаззинг.

Другой вариант - программировать файлы Flash с помощью ActionScript, языка сценариев Flash для управления данными и содержимым во время выполнения.[32] Затем фаззинг можно использовать для мутации различных методов ActionScript до компиляции двоичных файлов Flash. В октябре 2006 года Rapid7 выпустила бюллетень безопасности с описанием того, как метод `XML.addRequestHeader()` позволял добавлять произвольные заголовки HTTP в запрос объекта Flash.[33] Это, в свою очередь, можно было использовать для выполнения произвольных запросов HTTP методом разделения запросов HTTP.

URL

Иногда к уязвимостям приводит сам URL. После выпуска MS06-042[34] eEye обнаружила, что недавно выпущенные исправления этого бюллетеня также внесли уязвимость переполнения кучи. Было установлено, что при передаче Internet Explorer чрезмерно длинного URL и указании целевым веб-сайтом кодирования GZIP или deflate могло возникнуть переполнение, когда вызов `lstrcpynA()` пытался скопировать URL длиной до 2084 байтов в буфер размером 260 байтов.[35] Для атаки с этой уязвимостью требовалось лишь заставить пользователя уязвимой версии Internet Explorer щёлкнуть предоставленный URL.

Уязвимости

Хотя для успешной клиентской атаки требуется по меньшей мере некоторая социальная инженерия, такие уязвимости всё равно создают значительный риск из-за возможных типов атак.

• **Отказ в обслуживании.** Многие уязвимости веб-браузеров приводят лишь к атаке DoS, вызывающей аварийное завершение браузера или прекращение ответа. Обычно причиной является бесконечный цикл или повреждение памяти, которое нельзя эксплуатировать дальше. В общей картине клиентская атака DoS относительно незначительна. Хотя необходимость перезапускать веб-браузер после каждого сбоя может раздражать, постоянного ущерба она не

причиняет. В отличие от серверной атаки DoS, у клиентской атаки только одна жертва.

• **Переполнение буфера.** Переполнения буфера являются относительно распространёнными уязвимостями веб-браузеров. Их способен вызвать практически любой из ранее упомянутых входных векторов, и они особенно опасны, поскольку могут привести к выполнению кода.
• **Удалённое выполнение команд.** Уязвимости выполнения команд обычно используют существующую функциональность, которая не предназначалась для удалённого выполнения кода, но всё же допускает его. Например, Альберт Пучсеч Галисия (Albert Puigsech Galicia) обнаружил, что злоумышленник мог внедрять команды FTP непосредственно в URI FTP, заставляя Internet Explorer 6.x и более ранние версии выполнять их после простого убеждения пользователя щёлкнуть ссылку.[36] Эту уязвимость можно было, например, использовать для загрузки файлов на компьютер пользователя. Позднее выяснилось, что та же уязвимость могла заставлять браузер отправлять сообщения электронной почты. Microsoft исправила её в MS06-042.
• **Обход междоменных ограничений.** Веб-браузеры имеют средства, запрещающие одному веб-сайту обращаться к содержимому другого. Это важное ограничение, иначе любой сайт мог бы, например, получать файлы cookie других сайтов, часто содержащие идентификаторы сеанса. Появлялись многочисленные уязвимости, позволявшие сайту вырваться из доменного ограничения. Упомянутая ранее уязвимость GDS является одним из примеров.
• **Обход зон безопасности.** Internet Explorer применяет безопасность в соответствии с зоной происхождения содержимого. Документы из Интернета обычно считаются недоверенными и подвергаются большим ограничениям. Напротив, локально открытые файлы считаются доверенными и получают больше привилегий. В феврале 2005 года Йоуко Пюннёнен выпустил бюллетень, подробно описывающий применение особым образом закодированных URL для обмана Internet Explorer, заставлявшего интерпретировать удалённые файлы как открытые из локальной зоны.[37] Это, в свою очередь, могло позволить злоумышленнику включить в загруженный файл вредоносный сценарий, выполняющий предоставленный им эксплойт. Microsoft исправила уязвимость в бюллетене безопасности MS05-014.[38]
• **Подмена адресной строки.** Фишинг стал серьёзной проблемой, поскольку всё больше преступников пытаются получить личные сведения, например номера кредитных карт, у ничего не подозревающих посетителей Веба. Хотя большинство фишинговых атак просто использует социальную инженерию, некоторые более сложные атаки используют уязвимости веб-браузеров, чтобы фишинговый сайт выглядел законным. Уязвимости, позволяющие подменить адресную строку, ценны для фишеров, поскольку позволяют поддельной веб-странице выглядеть размещённой на законном сайте. К несчастью, несколько таких уязвимостей существовало во всех основных веб-браузерах.

Обнаружение

При фаззинге веб-браузеров важно параллельно пытаться выявлять неочевидные ошибки. Единственного места для поиска ошибок нет. Вместо этого полезно исследовать несколько альтернативных источников.

• **Журналы событий.** Если браузер подвергается фаззингу в среде Windows, не забывайте о «Просмотре событий». Internet Explorer 7.0 фактически добавил в него отдельный журнал Internet Explorer. Если целью является более ранняя версия Internet Explorer или другой браузер, записи могут находиться в журнале приложений. Хотя этот источник далеко не исчерпывающий, его легко проверить, и он может содержать полезные данные.
• **Средства наблюдения за производительностью.** Повреждение памяти или бесконечные циклы, скорее всего, приведут к снижению производительности целевого браузера. Средства наблюдения помогают выявлять такие состояния. Однако обязательно используйте для фаззинга отдельную машину, чтобы снижение производительности не было вызвано другими факторами.

• **Отладчики.** Безусловно, самым полезным средством обнаружения при фаззинге браузера будет сторонний отладчик, подключённый к выбранному браузеру. Он позволит выявлять обработанные и необработанные исключения, а также поможет определить, вероятно ли эксплуатируемы проблемы повреждения памяти.

Итоги

Хотя когда-то клиентские уязвимости упускали из виду, появление фишинговых атак заставляет внимательнее рассматривать создаваемый ими риск. Единственный уязвимый браузер в корпоративной сети может стать шлюзом для злоумышленника. Клиентские атаки требуют некоторой социальной инженерии, но это относительно небольшое препятствие. Уязвимости веб-браузеров также являются превосходными векторами целевых атак. В этой главе мы представили ряд существующих утилит фаззинга веб-браузеров. Далее мы используем полученные знания для создания собственного фаззера веб-браузеров.

Сноски

[1] [назад] http://browserfun.blogspot.com/
[2] [назад] http://osvdb.org/blog/?p=127
[3] [назад] http://www.uninformed.org/?v=4&a=5&t=sumry
[4] [назад] http://freshmeat.net/projects/mangleme/
[5] [назад] http://labs.idefense.com/software/fuzzing.php#more_COMRaider
[6] [назад] http://cve.mitre.org/cgi-bin/cvename.cgi?name=CAN-2003-0113
[7] [назад] http://www.microsoft.com/technet/security/bulletin/MS03-015.mspx
[8] [назад] http://downloads.securityfocus.com/vulnerabilities/exploits/urlmon-ex.pl
[9] [назад] http://cve.mitre.org/cgi-bin/cvename.cgi?name=CAN-2004-1050
[10] [назад] http://www.microsoft.com/technet/security/bulletin/ms04-040.mspx
[11] [назад] http://downloads.securityfocus.com/vulnerabilities/exploits/InternetExploiter.txt
[12] [назад] http://metasploit.com/users/hdm/tools/domhanoi/domhanoi.html
[13] [назад] http://metasploit.com/users/hdm/tools/hamachi/hamachi.html
[14] [назад] http://www.microsoft.com/technet/security/advisory/925568.mspx
[15] [назад] http://www.microsoft.com/technet/security/Bulletin/MS06-055.mspx
[16] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=462
[17] [назад] http://en.wikipedia.org/wiki/ActiveX_Control
[18] [назад] http://msdn.microsoft.com/workshop/components/activex/safety.asp
[19] [назад] http://metasploit.com/users/hdm/tools/axman/
[20] [назад] http://browserfun.blogspot.com/2006/08/axman-activex-fuzzer.html
[21] [назад] http://metasploit.com/users/hdm/tools/see-ess-ess-die/cssdie.html
[22] [назад] http://browserfun.blogspot.com/2006/07/mobb-26-opera-css-background.html
[23] [назад] http://www.microsoft.com/technet/security/Bulletin/MS06-021.mspx
[24] [назад] http://www.hacker.co.il/security/ie/css_import.html
[25] [назад] http://en.wikipedia.org/wiki/Jscript
[26] [назад] http://en.wikipedia.org/wiki/Ecmascript
[27] [назад] http://browserfun.blogspot.com/2006/07/mobb-25-native-function-iterator.html
[28] [назад] http://www.mozilla.org/security/announce/2005/mfsa2005-33.html
[29] [назад] http://www.milw0rm.com/exploits/930
[30] [назад] http://www.openrce.org/downloads/details/1
[31] [назад] http://pedram.openrce.org/images/olly_heap_vis/skylined_ie_heap_fill.gif
[32] [назад] http://en.wikipedia.org/wiki/Actionscript
[33] [назад] http://www.rapid7.com/advisories/R7-0026.jsp
[34] [назад] http://www.microsoft.com/technet/security/bulletin/ms06-042.mspx
[35] [назад] http://research.eeye.com/html/advisories/published/AD20060824.html
[36] [назад] http://osvdb.org/displayvuln.php?osvdb_id=12299
[37] [назад] http://jouko.iki.fi/adv/zonespoof.html
[38] [назад] http://www.microsoft.com/technet/security/bulletin/ms05-014.mspx

Глава 18. Фаззинг веб-браузеров: автоматизация

> «Природный газ является полушарным. Мне нравится называть его полушарным по природе, потому что это продукт, который мы можем найти в наших районах».
>
> Джордж У. Буш, Вашингтон, округ Колумбия, 20 декабря 2000 года

В главе 17, «Фаззинг веб-браузеров», мы обсудили ряд аспектов веб-браузеров вместе с их «пригодностью для фаззинга». Возросший интерес к фаззингу браузеров привёл к созданию ряда инструментов фаззинга и ещё большего числа уязвимостей, затрагивающих самые популярные браузеры, используемые сегодня, такие как Mozilla Firefox и Microsoft Internet Explorer. В этой главе мы рассмотрим требования к созданию фаззера ActiveX. Хотя эксплуатация ограничена Internet Explorer и уже существует несколько фаззеров ActiveX, среди прочих векторов тестирования для дальнейшего обсуждения был выбран именно этот, потому что он наиболее интересен и сложен. Ограничение Internet Explorer тоже не является совершенно обескураживающим, поскольку браузер Microsoft всё ещё удерживает большую часть рынка пользователей Веба. Эта глава начинается с краткого изложения истории технологии ActiveX, а затем сразу переходит к разработке инструмента фаззинга ActiveX.

Предыстория Component Object Model

Microsoft COM представляет собой амбициозную программную технологию, впервые представленную в начале 1990-х годов и нацеленную на предоставление универсального протокола взаимодействия программного обеспечения.

Стандартизованное взаимодействие клиент-сервер позволяет программам, написанным на различных языках, поддерживающих COM, обмениваться данными друг с другом как локально, в одной системе, так и удалённо, между разными системами. Сегодня COM широко используется и имеет насыщенную событиями историю, породившую на своём пути целый ряд акронимов, а также путаницу.

История в двух словах

Самого раннего предка COM можно проследить до Dynamic Data Exchange (DDE), технологии, до сих пор используемой в операционной системе Windows. DDE можно увидеть в действии в расширениях файлов оболочки, программе просмотра ClipBook (NetDDE) и Microsoft Hearts (также NetDDE). В 1991 году Microsoft представила Object Linking and Embedding (OLE). В то время как DDE ограничена чистым обменом данными, OLE способна встраивать типы документов друг в друга. Взаимодействие клиента и сервера OLE происходит внутри системных библиотек посредством таблиц виртуальных функций (VTBL).

За появлением COM последовала OLE, а позднее выпуск OLE 2, нового поколения OLE, построенного поверх COM вместо VTBL. В 1996 году эта технология была переименована в ActiveX. Позднее в том же году Microsoft выпустила Distributed COM (DCOM) как расширение COM и ответ на Common Object Request Broker Architecture[1] (CORBA). Distributed Computing Environment/Remote Procedure Calls[2] (DCE/RPC) является механизмом RPC, лежащим в основе DCOM. Добавление DCOM ещё больше расширило гибкость COM, поскольку позволяет разработчикам программного обеспечения предоставлять функциональность через такие среды, как Интернет, не предоставляя доступа к лежащему в основе коду.

Самым недавним событием в истории COM стало появление COM+, выпущенной вместе с операционной системой Windows 2000. COM+ предоставляет дополнительное преимущество «ферм компонентов», управляемых с помощью Microsoft Transaction Server, входящего в комплект Windows 2000. Подобно DCOM, компоненты COM+ также могут быть распределёнными и дополнительно допускают повторное использование без необходимости выгружать компонент из памяти.

Объекты и интерфейсы

Архитектура COM определяет как объекты, так и интерфейсы. Объект, такой как элемент управления ActiveX, описывает свою функциональность, определяя и реализуя интерфейс. Затем программное обеспечение может запросить объект COM, чтобы определить, какие интерфейсы и функциональность он может предоставлять. Каждому объекту COM назначается уникальный 128-битный идентификатор, известный как идентификатор класса (CLSID). Кроме того, каждому интерфейсу COM также назначается уникальный 128-битный идентификатор, известный как идентификатор интерфейса (IID). Примерами интерфейсов COM являются `IStream`, `IDispatch` и `IObjectSafety`. Эти и все прочие интерфейсы являются производными от базового интерфейса, известного как `IUnknown`.

Помимо CLSID, объекты при необходимости могут указывать для идентификации идентификатор программы (ProgID). ProgID представляет собой удобочитаемую строку, которой гораздо удобнее ссылаться на объект. Например, рассмотрим следующие псевдонимы:

• `000208D5-0000-0000-C000-000000000046`
• `Excel.Application`

И CLSID, и ProgID определяются в реестре и могут использоваться вместо друг друга. Однако будьте осторожны: уникальность ProgID не гарантируется.

ActiveX

ActiveX является единственной технологией COM, относящейся к рассматриваемой нами теме, фаззингу веб-браузеров. Подобно апплетам Java, элементы управления ActiveX имеют более широкий доступ к операционной системе и используются для разработки множества расширений, которые в ином случае недоступны браузеру. Компоненты ActiveX широко распространены, их можно найти как в составе многих программных пакетов, так и распространяемыми непосредственно через Веб. Примерами продуктов и служб, которые могут полагаться на технологию ActiveX, являются сетевые антивирусные сканеры, веб-конференции и телефония, программы мгновенного обмена сообщениями, сетевые игры и многое другое.

Microsoft Internet Explorer является единственным браузером, который изначально поддерживает ActiveX и реализует стандартную объектную модель документа (DOM) для обработки создания экземпляров, параметров и вызова методов. Для справки и демонстрации ниже приведены следующие примеры.

Чистая DOM

```
<object classid = “clsid:F08DF954-8592-11D1-B16A-00C0F0283628”
id      = “Slider1”
width   = “100”
height  = “50”>
<param name=”BorderStyle”  value=”1” />
<param name=”MousePointer” value=”0” />
<param name=”Enabled”      value=”1” />
<param name=”Min”          value=”0” />
<param name=”Max”          value=”10” />
</object>

Slider1.method(arg, arg, arg)
```

Устаревший Embed

```
<embed type   = “application/x-oleobject”
name   = “foo”
align  = “baseline”
border = “0”
width  = “200”
height = “300”
clsid  = “{8E27C92B-1264-101C-8A2F-040224009C02}”>

foo.method(arg, arg, arg)
```

Javascript / Jscript

```
<script type=”javascript”>
function call_function()
{
obj = new ActiveXObject(‘AcroPDF.PDF’);
obj.property = “value”;
obj.method(arg, arg, arg);
}
call_function();
</script>
```

Visual Basic

```
<object classid = ‘clsid:38EE5CEE-4B62-11D3-854F-00A0C9C898E7’
id      = ‘target’ >
</object>
<script language=’vbscript’>
‘Wscript.echo typename(target)
‘Sub SelectAndActivateButton ( ByVal lButton As Long )
arg1=2147483647
target.property = arg1
target.method arg1
</script>
```

Помимо загрузки элементов управления непосредственно в браузере, элемент управления ActiveX можно загрузить и взаимодействовать с ним напрямую как со стандартным объектом COM. Непосредственная загрузка элемента управления может быть выгодна для нашего фаззера, поскольку устраняет промежуточные шаги создания и загрузки браузерного кода в Internet Explorer.

Программирование Microsoft COM и её внутреннее устройство представляют собой обширную тему, которой посвящены целые книги. Дополнительные сведения о COM можно найти на

веб-сайте Microsoft COM[3], где приведён обзор высокого уровня, и в статье MSDN «The Component Object Model: A Technical Overview»[4], где приведён обзор низкого уровня.

В следующем разделе мы сразу погрузимся в разработку фаззера ActiveX, по ходу дела раскрывая дополнительные относящиеся к делу подробности технологии COM.

Разработка фаззера

Для разработки фаззера ActiveX подходит ряд языков. Это очевидно из того, что при реализации различных существующих сегодня фаззеров ActiveX используется множество языков программирования. COMRaider[5] в основном построен на Visual Basic с небольшими вкраплениями C++. AxMan[6] разработан как гибрид C++, JavaScript и HTML. Мы строим свой фаззер по тому же общему подходу, который применён в обоих этих фаззерах:

• перечислить все загружаемые элементы управления ActiveX;
• перечислить доступные методы и свойства каждого элемента управления ActiveX;
• разобрать библиотеку типов, чтобы определить типы параметров методов;
• сгенерировать тестовые случаи фаззинга;
• выполнить тестовые случаи фаззинга, одновременно отслеживая аномалии.

В отличие от наших предшественников, мы реализовали свой фаззер целиком на одном языке, Python. Поначалу выбор языка может показаться вам странным и даже неосуществимым. Однако разработка на Python действительно возможна, поскольку современные дистрибутивы включают ряд модулей, предоставляющих доступ к лежащему в основе Windows API. Среди различных модулей, на которые мы полагаемся при взаимодействии с COM, находятся `win32api`, `win32com`, `pythoncom` и `win32con`. Если вас интересуют дополнительные подробности программирования на Python в среде Windows, обратитесь к книге с подходящим названием «Python Programming on Win32»[7], написанной Марком Хаммондом, непосредственным разработчиком ряда модулей, которые мы используем, чтобы связать Python с миром COM. На рисунках 18.1 и 18.2 приведены снимки экранов обозревателей COM и библиотек типов PythonWin соответственно, показывающие высокоуровневый доступ к низкоуровневой информации COM.

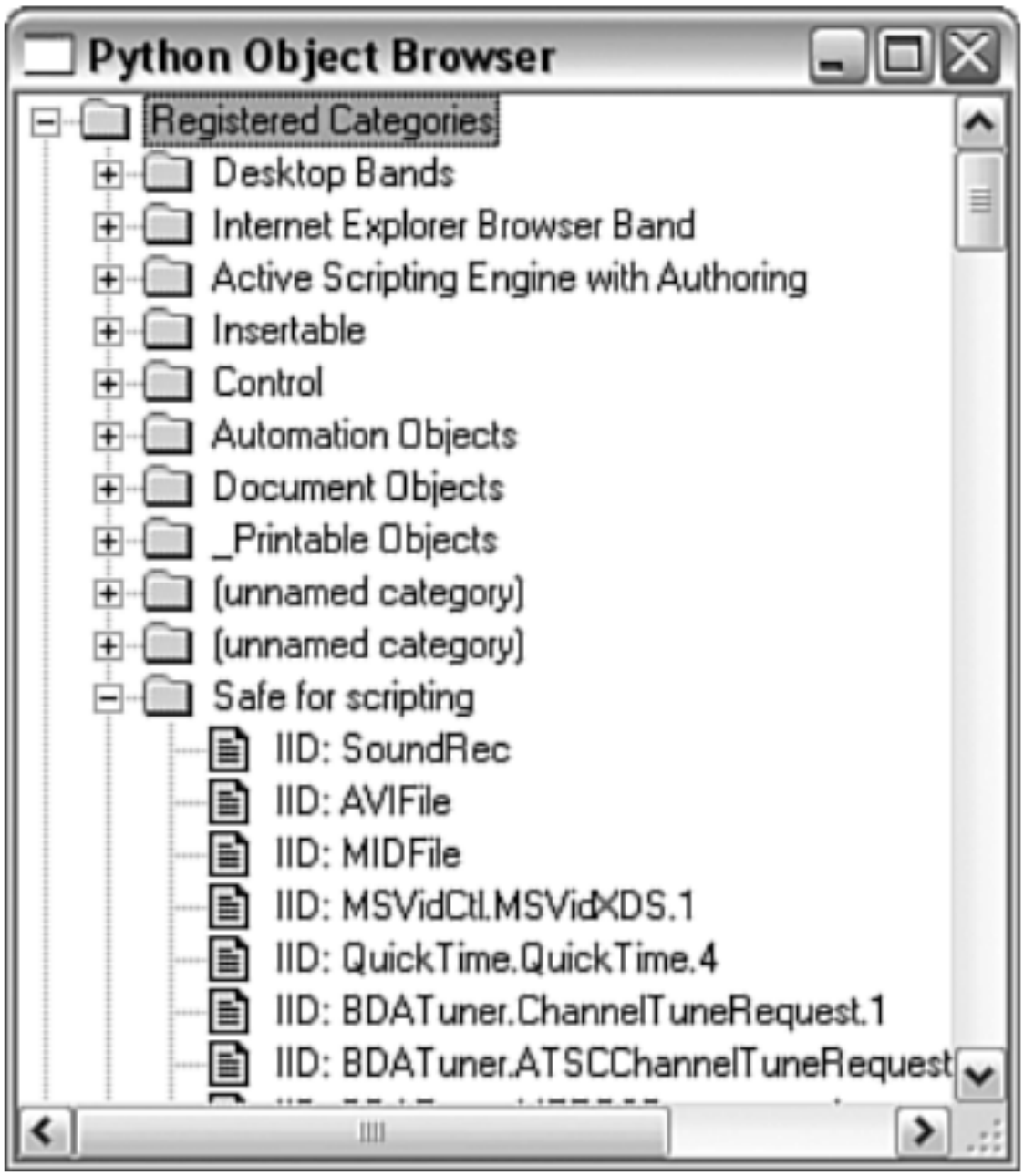


Рисунок 18.1. Обозреватель COM в PythonWin

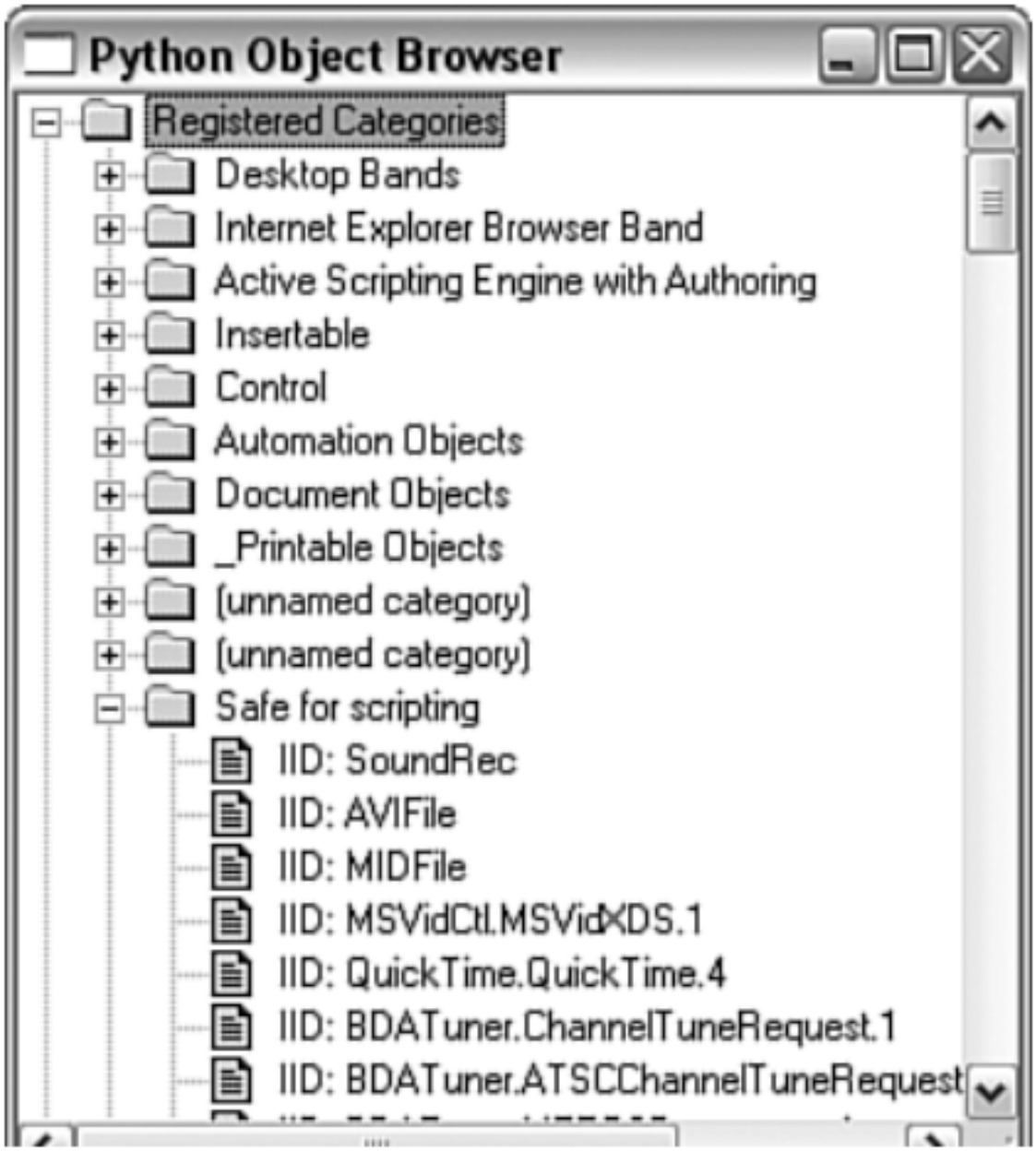


Рисунок 18.2. Обозреватель библиотек типов в PythonWin

В качестве программного примера рассмотрим следующий фрагмент, который создаст экземпляр Microsoft Excel и сделает его видимым, установив логическое свойство `Visible`:

```
import win32com.client
xl = win32com.client.Dispatch("Excel.Application")
xl.Visible = 1
```

Далее мы сразу перейдём к дополнительным примерам, рассмотрев, как перечислить все загружаемые элементы управления ActiveX. Фрагменты кода, представленные в оставшейся части этой главы, являются выдержками из полнофункционального фаззера COM, доступного для загрузки с официального веб-сайта этой книги (http://www.fuzzing.org).

Перечисление загружаемых элементов управления ActiveX

Наша первая задача разработки заключается в перечислении всех объектов COM, доступных в целевой системе. Полный список объектов COM опубликован в реестре Windows[8] в разделе `HKEY_LOCAL_MACHINE` (`HKLM`) и подразделе `SOFTWARE\Classes`. Получить доступ к этому разделу можно с помощью стандартного API доступа к реестру Windows:[9]

```
import win32api, win32con
import pythoncom, win32com.client
from win32com.axscript import axscript

try:
    classes_key = win32api.RegOpenKey( \
        win32con.HKEY_LOCAL_MACHINE,   \
        “SOFTWARE\\Classes”)
except win32api.error:
    print “Problem opening key HKLM\\SOFTWARE\\Classes”
```

Первые три строки этого фрагмента отвечают за импорт функциональности, необходимой для доступа к реестру, а также для взаимодействия с объектами COM. После открытия раздел перебирается в поисках допустимых CLSID классов. Если CLSID найден, запись сохраняется в списке, который используется позднее.

```
skey_index = 0
clsid_list = []

while True:
    try:
        skey = win32api.RegEnumKey(classes_key, skey_index)
    except win32api.error:
        print “End of keys”
        break

    progid = skey

    try:
        skey = win32api.RegOpenKey(win32con.HKEY_LOCAL_MACHINE, \
            “SOFTWARE\\Classes\\%s\\CLSID” % progid)
    except win32api.error:
        print “Couldn’t get CLSID key...skipping”
        skey_index += 1
        continue

    try:
        clsid = win32api.RegQueryValueEx(skey, None)[0]
    except win32api.error:
        print “Couldn’t get CLSID value...skipping”
        skey_index += 1
        continue

    clsid_list.append((progid, clsid))
    skey_index += 1
```

В совокупности эти фрагменты создают список кортежей, описывающих каждый доступный в системе объект COM. Тестирование безопасности объектов COM обычно ограничено элементами управления, доступными Internet Explorer. Причина в том, что самым популярным методом эксплуатации ActiveX с большим отрывом является размещение злоумышленником вредоносного веб-сайта и принуждение пользователей, у которых установлен уязвимый компонент, посетить эту страницу. Поскольку в Internet Explorer доступны лишь некоторые из этих элементов управления, наша вторая задача заключается в удалении недоступных из списка. Internet Explorer загрузит элемент управления ActiveX без предупреждения, если выполнен любой из следующих трёх критериев:[10]

• в реестре Windows элемент управления помечен как «Safe for Scripting»;
• в реестре Windows элемент управления помечен как «Safe for Initialization»;

• элемент управления реализует COM-интерфейс IObjectSafety.

Реестр Windows содержит раздел Component Categories, в котором перечислены подразделы для каждой категории, реализованной установленными компонентами. Две нужные нам категории называются CATID_SafeForScripting и CATID_SafeForInitializing. Следующие процедуры определят, помечен ли заданный CLSID в реестре как доступный в Internet Explorer:

```
def is_safe_for_scripting (clsid):
    try:
        key = win32api.RegOpenKey(win32con.HKEY_CLASSES_ROOT, \
            "CLSID\\%s\\Implemented Categories" % clsid)
    except win32api.error:
        return False

    skey_index = 0

    while True:
        try:
            skey = win32api.RegEnumKey(key, skey_index)
        except:
            break

        # CATID_SafeForScripting
        if skey == "{7DD95801-9882-11CF-9FA9-00AA006C42C4}":
            return True

        skey_index += 1

    return False

def is_safe_for_init (clsid):
    try:
        key = win32api.RegOpenKey(win32con.HKEY_CLASSES_ROOT, \
            "CLSID\\%s\\Implemented Categories" % clsid)
    except win32api.error:
        return False

    skey_index = 0

    while True:
        try:
            skey = win32api.RegEnumKey(key, skey_index)
        except:
            break

        # CATID_SafeForInitializing
        if skey == "{7DD95802-9882-11CF-9FA9-00AA006C42C4}":
            return True

        skey_index += 1

    return False
```

Помимо реестра, элемент управления ActiveX может пометить себя безопасным для Internet Explorer, реализовав интерфейс IObjectSafety. Чтобы определить, реализует ли данный элемент управления ActiveX интерфейс IObjectSafety, сначала необходимо создать его экземпляр, а затем выполнить запрос. Следующая процедура определит, реализует ли данный элемент управления интерфейс IObjectSafety и, следовательно, доступен ли он Internet Explorer:

```
def is_iobject_safety (clsid):
    try:
        unknown = pythoncom.CoCreateInstance(clsid, \
            None,                                   \
            pythoncom.CLSCTX_INPROC_SERVER,         \
            pythoncom.IID_IUnknown)
    except:
        return False

    try:
```

```
            objsafe = unknown.QueryInterface(axscript.IID_IObjectSafety)
        except:
            return False

        return True
```

Чтобы завершить формирование списка загружаемых элементов управления ActiveX, мы должны учесть последнее обстоятельство. Microsoft предоставляет механизм установки бита блокировки,[11] то есть предотвращения загрузки отдельных CLSID в Internet Explorer, с помощью раздела реестра `HKLM\Software\Microsoft\Internet Explorer\ActiveX Compatibility\<CLSID of ActiveX Control>`. Каждый CLSID, для которого в этом расположении реестра существует запись, необходимо удалить из нашего списка. Следующую функцию можно использовать, чтобы определить, был ли заданный CLSID отключён таким способом:

```
def is_kill_bitted (clsid):
    try:
        key = win32api.RegOpenKey(win32con.HKEY_LOCAL_MACHINE, \
            “SOFTWARE\\Microsoft\\Internet Explorer”           \
            “\\ActiveX Compatibility\\%s” % clsid)
    except win32api.error:
        return False

    try:
        (compat_flags, typ) = win32api.RegQueryValueEx(key, \
            “Compatibility Flags”)
    except win32api.error:
        return False

    if typ != win32con.REG_DWORD:
        return False

    if compat_flags & 0x400:
        return True
    else:
        return False

    return False
```

После того как список элементов управления ActiveX составлен и дважды проверен с целью определить, какие из них могут вести себя непослушно, нам необходимо рассмотреть предоставляемые свойства и методы.

Свойства, методы, параметры и типы

Возможность систематически создавать список целевых элементов управления удобна, но именно здесь наш фаззинг ActiveX становится по-настоящему изящным. Описание каждого свойства и метода, предоставляемого объектом COM, встроено непосредственно в него. Более того, описаны также типы как свойств, так и параметров методов. Эта возможность COM настолько же восхищает нас, разработчиков фаззеров, насколько и разработчиков программного обеспечения, которые впервые о ней слышат.

Возможность программно перечислить поверхность атаки элемента управления ActiveX позволяет нам создавать интеллектуальные фаззеры, знающие, какие данные может ожидать конкретный метод. Благодаря этому мы не тратим время на передачу строки, когда ожидается целое число.

В мире COM данные передаются через структуру, известную как `VARIANT`. Структура данных `VARIANT` поддерживает ряд типов данных, включая целые числа, числа с плавающей точкой, строки, даты, логические значения, другие объекты COM и любой массив из них. PythonCOM предоставляет уровень абстракции, скрывающий от нас многие из этих деталей. В таблице 18.1 показано соответствие между некоторыми собственными типами Python и их эквивалентами

VARIANT.

Таблица 18.1. Преобразование VARIANT в PythonCOM

Тип объекта Python	Тип VARIANT
Integer	VT_I4
String	VT_BSTR
Float	VT_R8
None	VT_NLL
True/False	VT_BOOL

Модуль pythoncom предоставляет процедуру LoadTypeLib() для разбора библиотеки типов непосредственно из двоичного объекта COM. Всё, что нам необходимо знать о свойствах и методах объекта COM, доступно через загруженную библиотеку типов. В качестве примера пройдём по библиотеке типов элемента управления Adobe Acrobat PDF, ранее показанного на рисунке 18.2. Этот элемент управления ActiveX входит в комплект Adobe Acrobat Reader и доступен Internet Explorer, поскольку помечен как безопасный и для сценариев, и для инициализации. Следующий фрагмент демонстрирует загрузку библиотеки типов для этого кода и дополнительно применяет небольшую хитрость Python, чтобы создать отображение имён VARIANT:

```
adobe = r"C:\Program Files\Common Files" \
        r"\Adobe\Acrobat\ActiveX\AcroPDF.dll"

tlb = pythoncom.LoadTypeLib(adobe)

VTS = {}
for vt in [x for x in pythoncom.__dict__.keys() if x.count("VT_")]:
    VTS[eval("pythoncom.%s"%vt)] = vt
```

Сгенерированное отображение имён VARIANT используется исключительно в демонстрационных целях, как мы увидим позднее. Библиотека типов может определять несколько типов, а их количество для перебора можно получить вызовом GetTypeInfoCount(). В нашем примере присутствуют три типа, показанные в первом столбце рисунка 18.2. Следующий пример показывает, как перебрать различные типы и вывести их имена:

```
for pos in xrange(tlb.GetTypeInfoCount()):
    name = tlb.GetDocumentation(pos)[0]
    print name
```

В элементе управления Acrobat определены три типа. Давайте подробнее рассмотрим выделенный на рисунке 18.2 тип IAcroAXDocShim. Как и большинство видов счёта в программировании, отсчёт позиций начинается с нуля, а значит, индекс нужного нам типа равен двум, а не трём. В следующем блоке кода сведения о типе и атрибуты извлекаются из ранее определённой библиотеки типов, а затем используются для перечисления свойств этого конкретного типа:

```
info = tlb.GetTypeInfo(2)
attr = info.GetTypeAttr()

print "properties:"
for i in xrange(attr.cVars):
    id    = info.GetVarDesc(i)[0]
    names = info.GetNames(id)
    print "\t", names[0]
```

Переменная атрибута cVars указывает число свойств, или переменных, определённых в типе. Это число используется, чтобы перебрать и вывести имя каждого свойства. Перечислить методы,

параметры и типы параметров столь же просто, как видно из следующей выдержки кода:

```
print “methods:”
for i in xrange(attr.cFuncs):
    desc  = info.GetFuncDesc(i)

    if desc.wFuncFlags:
        continue

    id    = desc.memid
    names = info.GetNames(id)

    print “\t%s()” % names[0]

    i = 0
    for name in names[1:]:
        print “\t%s, %s” % (name, VTS[desc.args[i][0]])
        i += 1
```

В этом случае переменная атрибута `cFuncs` указывает число методов, определённых в этом типе. Методы перечисляются с пропуском тех, для которых установлен `wFuncFlags`. Этот флаг указывает, что метод ограничен, то есть недоступен, и потому не является хорошим кандидатом для фаззинга. Процедура `GetNames()` возвращает имя метода, а также имена каждого из его параметров. Имя метода выводится, после чего выполняется обход оставшейся части списка, `names[1:]`, для доступа к параметрам. Описание функции, первоначально возвращённое вызовом `GetFuncDesc()`, содержит список типов `VARIANT` для каждого параметра, или аргумента. Тип `VARIANT` представлен целым числом, которое мы преобразуем в имя посредством индексирования ранее сгенерированного отображения `VARIANT`.

Ниже приведён конечный результат выполнения этого сценария против этого сценария на интерфейсе `IAcroAXDocShim` элемента управления ActiveX Adobe Acrobat PDF:

```
properties:
methods:
src()
LoadFile()
        fileName, VT_BSTR
setShowToolbar()
        On, VT_BOOL
gotoFirstPage()
gotoLastPage()
gotoNextPage()
gotoPreviousPage()
setCurrentPage()
        n, VT_I4
goForwardStack()
goBackwardStack()
setPageMode()
        pageMode, VT_BSTR
setLayoutMode()
        layoutMode, VT_BSTR
setNamedDest()
        namedDest, VT_BSTR
Print()
printWithDialog()
setZoom()
        percent, VT_R4
setZoomScroll()
        percent, VT_R4
        left, VT_R4
        top, VT_R4
setView()
        viewMode, VT_BSTR
setViewScroll()
        viewMode, VT_BSTR
        offset, VT_R4

setViewRect()
```

```
        left, VT_R4
        top, VT_R4
        width, VT_R4
        height, VT_R4
printPages()
        from, VT_I4
        to, VT_I4
printPagesFit()
        from, VT_I4
        to, VT_I4
        shrinkToFit, VT_BOOL
printAll()
printAllFit()
        shrinkToFit, VT_BOOL
setShowScrollbars()
        On, VT_BOOL
GetVersions()
setCurrentHightlight()
        a, VT_I4
        b, VT_I4
        c, VT_I4
        d, VT_I4
setCurrentHighlight()
        a, VT_I4
        b, VT_I4
        c, VT_I4
        d, VT_I4
postMessage()
        strArray, VT_VARIANT
messageHandler()
```

В выведенных результатах видно, что в этом типе определено 39 методов, или функций, и ни одного свойства. Список параметров и типов каждого метода был успешно перечислен и отображён. Эти сведения критически важны при разработке интеллектуального фаззера, их следует использовать при создании тестовых случаев, чтобы каждый параметр фаззился соответствующим образом. Например, рассмотрим фаззинг короткого целого (`VT_I2`) в сравнении с длинным целым (`VT_I4`). Вместо того чтобы считать оба этих типа обобщённым целым, можно сэкономить драгоценное время, фаззя короткое целое в допустимом диапазоне от 0 до `0xFFFF` (65 535), а не в допустимом диапазоне полноразмерного целого, длинного, который простирается от 0 до `0xFFFFFFFF` (4 294 967 295).

На этом этапе мы рассмотрели необходимые шаги для перечисления каждого свойства, метода и параметра каждого доступного Internet Explorer элемента управления ActiveX в заданной системе. Следующий шаг заключается в выборе подходящих эвристик фаззинга и начале тестирования.

Фаззинг и мониторинг

В главе 6, «Автоматизация и генерация данных», рассматривалась важность выбора осмысленных фаззинговых значений строк и целых чисел. Выбранные эвристики фаззинга были предназначены для максимального повышения вероятности вызвать ошибку. Большинство фаззеров и тестовых случаев, описанных в этой книге, фактически предназначены для выявления низкоуровневых дефектов, таких как переполнения буфера, а не логических проблем, таких как доступ к ограниченным ресурсам. Модификаторы обхода каталогов являются примером эвристики, нацеленной на обнаружение таких уязвимостей безопасности. При поиске нарушений поведения во время фаззинга элементов управления ActiveX требуется дополнительная осмотрительность, поскольку исследователи часто обнаруживают элементы управления, которые попросту никогда не должны были быть доступны через Internet Explorer.

Рассмотрим, например, уязвимость раскрытия небезопасного метода элемента управления ActiveX WinZip FileView.[12] В данном случае элемент управления ActiveX с ProgID

WZFILEVIEW.FileViewCtrl.61 распространялся как безопасный для сценариев, позволяя вредоносному веб-сайту загружать элемент управления в браузер и использовать предоставляемую им функциональность. В частности, методы ExeCmdForAllSelected и ExeCmdForFolder позволяют вызывающей стороне копировать, перемещать, удалять и исполнять файлы из произвольных расположений, включая сетевые общие ресурсы, каталоги FTP и веб-каталоги. Это предоставляет злоумышленникам тривиальный вектор для загрузки и запуска произвольного исполняемого файла без какого-либо предупреждения посредством простого размещения вредоносного веб-сайта. WinZip выпустила исправление этой проблемы, удалив параметр безопасности для сценариев и дополнительно установив бит блокировки для конкретного элемента управления.

Помимо ранее использованных типичных эвристик фаззинга, в фаззинговые данные следует включить допустимые пути к файлам, команды и URL, чтобы помочь выявить ошибки, подобные уязвимости WinZip FileView. Разумеется, монитор фаззинга должен содержать необходимые возможности для определения того, был ли успешно получен доступ к какому-либо из предоставленных ресурсов. Позднее эта концепция будет разобрана подробнее.

После создания подходящего списка фаззинговых данных следующий шаг заключается в создании ряда тестовых случаев. Один подход к выполнению этой задачи состоит в построении страницы HTML, которая встраивает целевой элемент управления ActiveX любым из ранее перечисленных методов, а затем вызывает целевой метод с фаззинговыми параметрами. Для каждого теста можно создать отдельный файл, после чего загружать отдельные тестовые случаи в Internet Explorer. В качестве альтернативы целевой элемент управления можно загрузить и тестировать непосредственно. Продолжая предыдущие примеры, следующий фрагмент Python создаст экземпляр элемента управления ActiveX Adobe Acrobat PDF и обратится к двум его предоставляемым методам:

```
adobe = win32com.client.Dispatch("AcroPDF.PDF.1")
print adobe.GetVersions()
adobe.LoadFile("c:\\test.pdf")
```

Первая строка этого фрагмента отвечает собственно за создание экземпляра объекта COM Adobe, доступного через переменную с именем adobe. После создания взаимодействовать с объектом можно совершенно естественным образом. Вторая строка выводит результаты вызова процедуры GetVersions(), а третья заставляет элемент управления загрузить файл PDF в средство просмотра Acrobat Reader. Расширить этот пример для фаззинга других методов, параметров и элементов управления совсем несложно.

Оставшейся частью головоломки является функция мониторинга фаззера. Помимо модулей COM для Python, мы используем библиотеку обратной разработки PaiMei[13] для реализации фаззингового монитора на основе отладчика. Эта библиотека широко используется в последующих главах, где рассматривается более подробно. По существу, создаётся облегчённый отладчик, оборачивающий выполнение целевых элементов управления ActiveX. В случае обнаружения низкоуровневой уязвимости, такой как переполнение буфера, задействуется отладчик и записываются подробности дефекта. Библиотека PaiMei также предоставляет функциональность перехвата вызовов API. Эта возможность используется для мониторинга поведенческих проблем. Наблюдая за аргументами вызовов библиотек Microsoft, таких как CreateFile()[14] и CreateProcess(),[15] мы можем определить, получил ли целевой элемент управления ActiveX успешный доступ к ресурсу, к которому не должен был обращаться. Рассмотрение подробностей функциональности перехвата API оставлено читателю в качестве упражнения.

В качестве последнего замечания стоит исследовать имена предоставляемых свойств и методов с помощью простых поисков по регулярным выражениям. Если вам встретятся методы с такими именами, как GetURL(), DownloadFile(), Execute() и так далее, возможно, вы захотите сразу уделить им личное внимание.

Итоги

В этой главе мы исследовали историю технологии COM компании Microsoft и описали требования, при которых элемент управления ActiveX доступен из веб-браузера Internet Explorer. Были рассмотрены необходимые интерфейсы COM для Python, а также конкретные шаги, необходимые для перечисления доступных элементов управления ActiveX и их свойств, методов, параметров и типов параметров. Рассмотренные в этой главе фрагменты кода являются выдержками из полнофункционального фаззера COM, доступного для загрузки с официального веб-сайта этой книги (http://www.fuzzing.org).

Сноски

[1] [назад] http://en.wikipedia.org/wiki/Corba
[2] [назад] http://en.wikipedia.org/wiki/DCE/RPC
[3] [назад] http://www.microsoft.com/com/default.mspx
[4] [назад] http://msdn2.microsoft.com/en-us/library/ms809980.aspx
[5] [назад] https://labs.idefense.com/software/fuzzing.php#more_comraider
[6] [назад] http://metasploit.com/users/hdm/tools/axman/
[7] [назад] http://www.oreilly.com/catalog/pythonwin32/
[8] [назад] http://en.wikipedia.org/wiki/Windows_registry
[9] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/sysinfo/base/registry_functions.asp
[10] [назад] http://msdn.microsoft.com/workshop/components/activex/safety.asp
[11] [назад] http://support.microsoft.com/kb/240797
[12] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-040.html
[13] [назад] https://www.openrce.org/downloads/details/208/PaiMei
[14] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/fileio/fs/createfile.asp
[15] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dllproc/base/createprocess.asp

Глава 19. Фаззинг в памяти

> «Он белый».
>
> Джордж У. Буш, после того как ребёнок в Британии спросил его, как выглядит Белый дом, 19 июля 2001 года

В этой главе мы вводим понятие фаззинга в памяти, нового подхода к фаззингу, который почти не привлёк публичного внимания и для которого ещё не было открыто выпущено полнофункциональных инструментов для подтверждения концепции. По существу, этот метод нацелен на переход фаззинга от знакомой модели клиент-сервер, или клиент-цель, к модели, состоящей только из цели и целиком находящейся в памяти. Хотя технически это более сложный подход, требующий глубоких низкоуровневых знаний языка ассемблера, организации памяти процесса и инструментирования процессов, у этого метода есть ряд преимуществ, которые мы опишем до перехода к каким-либо конкретным подробностям реализации.

В предыдущих главах мы предпринимали целенаправленные усилия, чтобы представить непредвзятый взгляд и с точки зрения UNIX, и с точки зрения Windows. Из-за сложного охвата этого метода фаззинга в данной главе мы ограничиваемся одной платформой. В качестве платформы, на которой сосредоточим внимание, мы выбрали Microsoft Windows по ряду причин. Во-первых, этот метод фаззинга лучше применять к целям с закрытым исходным кодом, поскольку двоичные дистрибутивы программного обеспечения популярнее на платформе Windows, чем в UNIX. Во-вторых, Windows предоставляет богатые и мощные API отладки для инструментирования

процессов. Это не значит, что API отладки недоступны на платформах UNIX, но, по нашему мнению, они просто не столь надёжны. Разумеется, обсуждаемые здесь общие идеи и подходы можно адаптировать для применения на различных платформах.

Фаззинг в памяти: что и зачем?

Каждый обсуждавшийся до сих пор метод фаззинга требовал создания и передачи данных по ожидаемым каналам. В главе 11, «Фаззинг форматов файлов», главе 12, «Фаззинг форматов файлов: автоматизация в UNIX», и главе 13, «Фаззинг форматов файлов: автоматизация в Windows», изменённые файлы загружались непосредственно приложением. В главе 14, «Фаззинг сетевых протоколов», главе 15, «Фаззинг сетевых протоколов: автоматизация в UNIX», и главе 16, «Фаззинг сетевых протоколов: автоматизация в Windows», фаззинговые данные передавались целевым службам через сетевые сокеты. В обеих ситуациях от нас требуется представить цели целый файл или полный протокол, даже если нас интересует фаззинг лишь конкретного поля. Кроме того, в обеих ситуациях мы в значительной мере не знали и не заботились о том, какой лежащий в основе код исполнялся в ответ на наши входные данные.

Фаззинг в памяти является совершенно иным зверем. Вместо того чтобы сосредотачиваться на конкретном протоколе или формате файла, мы сосредотачиваемся на фактическом лежащем в основе коде. Мы переносим внимание с входных данных на функции и отдельные инструкции ассемблера, отвечающие за разбор входных данных. Таким образом, мы пропускаем ожидаемый канал передачи данных и вместо этого сосредотачиваемся либо на изменении, либо на создании данных в памяти целевого приложения.

Когда это полезно? Рассмотрим ситуацию, когда вы фаззите сетевой демон с закрытым исходным кодом, реализующий сложную схему шифрования или обфускации пакетов. Чтобы успешно фаззить такую цель, сначала необходимо полностью выполнить обратную разработку схемы обфускации и воспроизвести её. Это первоклассный кандидат для фаззинга в памяти. Вместо ненужного сосредоточения на инкапсуляции протокола мы заглядываем в пространство памяти целевого приложения в поисках интересных точек, например после деобфускации полученных сетевых данных. В качестве другого примера рассмотрим ситуацию, когда мы хотим проверить надёжность конкретной функции. Допустим, благодаря некоторым усилиям по обратной разработке мы точно установили процедуру разбора электронной почты, которая принимает единственный строковый аргумент, адрес электронной почты, и некоторым образом разбирает его. Мы могли бы создавать и передавать сетевые пакеты или тестовые файлы, содержащие изменённые адреса электронной почты, но с помощью фаззинга в памяти можем сосредоточить усилия, фаззя целевую процедуру напрямую.

Эти понятия дополнительно проясняются на протяжении оставшейся части этой главы, а также следующей. Однако сначала нам необходимо рассмотреть некоторые справочные сведения.

Необходимые справочные сведения

Прежде чем продолжить, давайте пройдём ускоренный курс по модели памяти Microsoft Windows, кратко обсудив общую организацию и свойства пространства памяти типичного процесса Windows. В этом разделе мы затронем ряд сложных и подробных предметных областей, которые рекомендуем вам изучить глубже.

Поскольку часть этого материала обычно бывает довольно сухой, при желании можете пролистать до обзора и перейти прямо к схеме в конце.

Начиная с Windows 95, операционная система Windows построена на плоской модели памяти, которая на 32-битной платформе предоставляет в общей сложности 4 Гбайт адресуемой памяти. По умолчанию диапазон адресов в 4 Гбайт делится пополам. Нижняя половина (0x00000000-0x7FFFFFFF) зарезервирована для пространства пользователя, а верхняя (0x80000000-0xFFFFFFFF) зарезервирована для пространства ядра. Это разделение при необходимости можно изменить на 3:1 с помощью параметра /3GB в boot.ini[1]: 3 Гбайт для пространства пользователя и 1 Гбайт для ядра, чтобы повысить производительность приложений с интенсивным использованием памяти, таких как база данных Oracle. Плоская модель памяти, в отличие от сегментированной, обычно использует один сегмент памяти для обращения к программному коду и данным. Сегментированная модель памяти, напротив, использует несколько сегментов для обращения к различным расположениям памяти. Главные преимущества плоской модели памяти над сегментированной заключаются в резком приросте производительности и снижении сложности с точки зрения программиста, поскольку выбирать и переключать сегменты не требуется.

Ещё больше упрощая мир для программистов, операционная система Windows управляет памятью посредством виртуальной адресации. По существу, модель виртуальной памяти предоставляет каждому работающему процессу собственное виртуальное адресное пространство объёмом 4 Гбайт. Эта задача выполняется посредством преобразования виртуальных адресов в физические при содействии блока управления памятью (MMU). Очевидно, не очень многие системы физически способны предоставить 4 Гбайт памяти каждому работающему процессу. Вместо этого пространство виртуальной памяти построено на понятии страничной организации. Страница представляет собой непрерывный блок памяти типичного для Windows размера 4096 (0x1000) байт. Используемые в данный момент страницы виртуальной памяти хранятся в ОЗУ, основном хранилище. Неиспользуемые страницы памяти могут быть выгружены на диск, вторичное хранилище, а позднее восстановлены в ОЗУ, когда снова понадобятся.

Другим важным понятием в управлении памятью Windows является защита памяти. Атрибуты защиты памяти применяются с наименьшей гранулярностью к страницам памяти, назначить атрибуты защиты только части страницы невозможно. Среди доступных атрибутов защиты, которые могут нас интересовать, находятся следующие:[2]

• PAGE_EXECUTE. Страница памяти является исполняемой, попытки чтения из неё или записи в неё приводят к нарушению доступа.
• PAGE_EXECUTE_READ. Страница памяти является исполняемой и доступной для чтения, попытки записи в неё приводят к нарушению доступа.
• PAGE_EXECUTE_READWRITE. Страница памяти полностью доступна. Она является исполняемой, из неё можно читать и в неё можно записывать.

• PAGE_NOACCESS. Страница памяти недоступна. Любая попытка исполнить данные со страницы памяти, прочитать их или записать приводит к нарушению доступа.
• PAGE_READONLY. Страница памяти доступна только для чтения. Любая попытка записи на страницу памяти приводит к нарушению доступа. Если лежащая в основе архитектура поддерживает различение чтения и исполнения, что бывает не всегда, то любая попытка исполнить данные со страницы памяти также приводит к нарушению доступа.
• PAGE_READWRITE. Страница памяти доступна для чтения и записи. Как и в случае PAGE_READONLY, если лежащая в основе архитектура поддерживает различение чтения и исполнения, то любая попытка исполнить данные со страницы памяти приводит к нарушению доступа.

Для наших целей нас может интересовать только модификатор PAGE_GUARD, который, согласно MSDN,[3] при любой попытке обратиться к защищённой странице вызывает STATUS_GUARD_PAGE_VIOLATION, а затем удаляет защиту барьером. По существу, PAGE_GUARD действует как одноразовый сигнализатор доступа. Эту возможность мы могли бы захотеть использовать для мониторинга доступа к определённым областям памяти. В качестве

альтернативы можно также использовать атрибут PAGE_NOACCESS.

Знакомство со страницами памяти, их организацией и атрибутами защиты важно для фаззинга в памяти, как мы увидим в следующей главе. Управление памятью является сложной темой, которая подробно обсуждается во многих других публикациях.[4] Мы рекомендуем вам обратиться к этим работам. Однако для наших целей вам необходимо знать лишь следующее:

• Каждый процесс Windows «видит» собственное виртуальное адресное пространство объёмом 4 Гбайт.
• Для пространства пользователя доступны только нижние 2 Гбайт от 0x00000000 до 0x7FFFFFFF, верхние 2 Гбайт зарезервированы как пространство ядра.
• Виртуальное адресное пространство процесса неявно защищено от доступа других процессов.
• Адресное пространство объёмом 4 Гбайт разбито на отдельные страницы типичного размера 4096 байт (0x1000).
• С наименьшей гранулярностью атрибуты защиты памяти применяются к отдельным страницам памяти.
• Модификатор защиты памяти PAGE_GUARD можно использовать как одноразовый сигнализатор доступа к странице.

Изучите рисунок 19.1, чтобы лучше понять, где в виртуальном адресном пространстве типичного процесса Windows можно найти конкретные элементы. Рисунок 19.1 также послужит полезной справкой в следующей главе, когда мы будем обсуждать конкретные подробности разработки инструмента автоматизации фаззинга в памяти. После рисунка приведено краткое описание различных перечисленных элементов на случай, если вы с ними не знакомы.

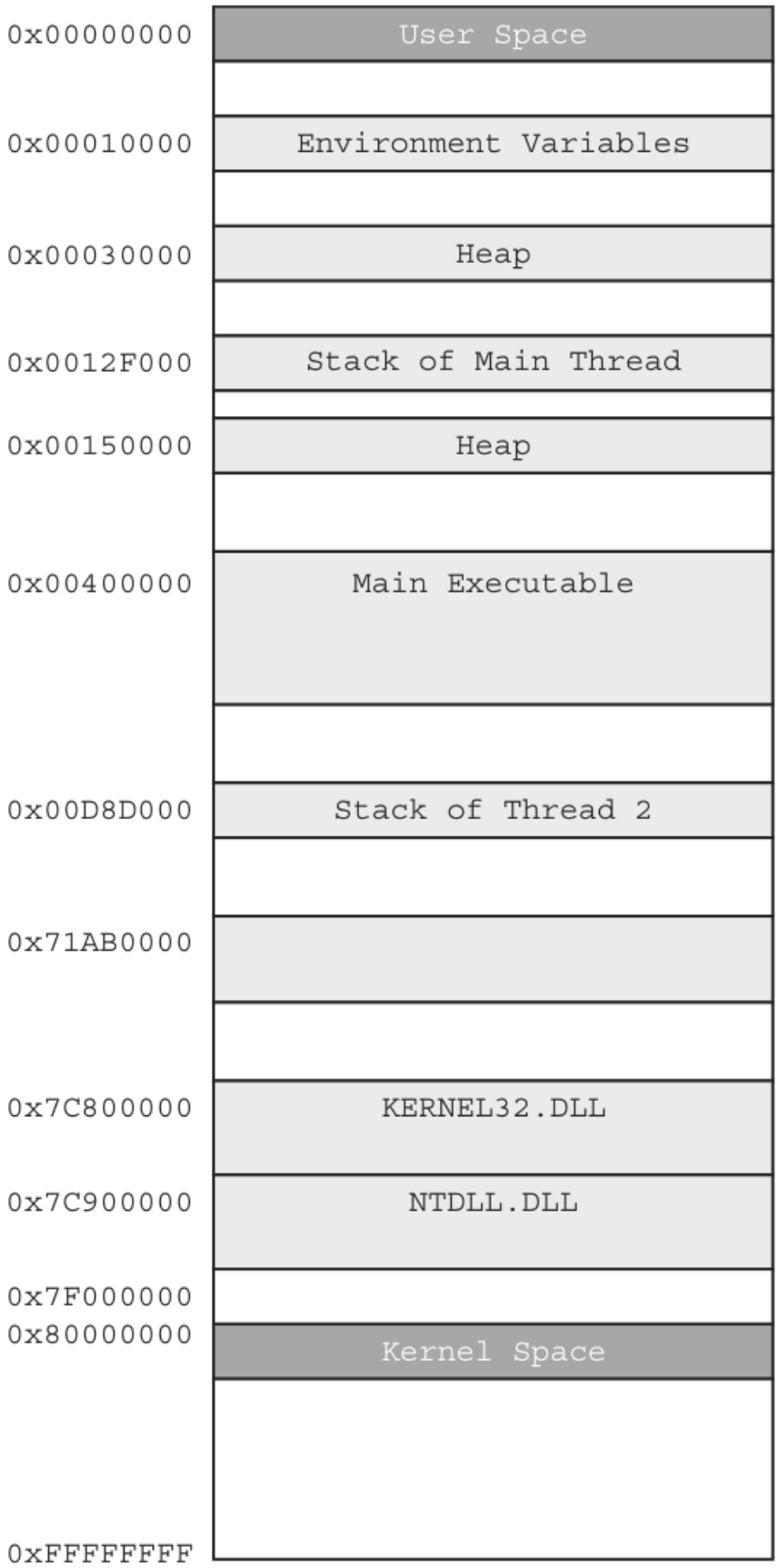


Рисунок 19.1. Типичная организация памяти Windows, без соблюдения масштаба

Давайте кратко опишем различные элементы, показанные на рисунке 19.1, для тех, кто с ними не знаком. Начиная с самого нижнего диапазона адресов, по адресу `0x00010000` мы находим

переменные среды процесса. Вспомните из главы 7, «Фаззинг переменных среды и аргументов», что переменные среды представляют собой общесистемные глобальные переменные, используемые для определения определённых вариантов поведения приложений. Выше в диапазоне адресов мы находим два диапазона памяти, выделенных куче, по адресам `0x00030000` и `0x00150000`. Куча представляет собой пул памяти, из которого происходят динамические выделения памяти посредством таких вызовов, как `malloc()` и `HeapAlloc()`. Обратите внимание, что у каждого заданного процесса может быть более одной кучи, как и показывает схема. Между двумя диапазонами кучи по адресу `0x0012F000` мы находим стек основного потока. Стек представляет собой структуру данных типа «последним пришёл, первым вышел», которая используется для отслеживания цепочек вызовов функций и локальных переменных. Обратите внимание, что каждый поток внутри процесса имеет собственный стек. На нашей примерной схеме стек потока 2 расположен по адресу `0x00D8D000`. По адресу `0x00400000` находится область памяти нашего основного исполняемого файла, файла `.exe`, который мы запустили, чтобы загрузить процесс в память. Ближе к верхней границе пользовательского адресного пространства мы находим несколько системных DLL, `kernel32.dll` и `ntdll.dll`. DLL представляют собой реализацию общих библиотек Microsoft, общего кода, используемого разными приложениями и экспортируемого через единый файл. И исполняемые файлы, и DLL используют формат файла Portable Executable (PE). Обратите внимание, что показанные на рисунке 19.1 адреса не будут одинаковыми в разных процессах и выбраны здесь исключительно для примера.

И снова мы затронули ряд сложных тем, заслуживающих собственных книг. Мы рекомендуем вам обратиться к более углублённым материалам. Однако для наших целей мы должны были вооружиться достаточными сведениями, чтобы погрузиться в рассматриваемую тему.

Нет, правда, что такое фаззинг в памяти?

В этой книге мы описываем и реализуем два подхода к фаззингу в памяти. В обоих случаях мы перемещаем фаззер снаружи цели внутрь самой цели. Этот процесс лучше всего объяснить и понять визуально. Для начала рассмотрим рисунок 19.2, изображающий простую схему потока управления вымышленной, но типичной сетевой цели.

Цикл внутри основного потока нашего примерного целевого приложения ожидает новых клиентских соединений. При получении соединения порождается новый поток для обработки клиентского запроса, который принимается посредством одного или нескольких вызовов `recv()`. Затем собранные данные передаются через некоторую процедуру демаршалинга или обработки. Процедура `unmarshal()` может отвечать за распаковку или расшифровку протокола, но фактически не разбирает отдельные поля в потоке данных. Обработанные данные, в свою очередь, передаются основной процедуре разбора `parse()`, которая построена поверх ряда других процедур и библиотечных вызовов. Процедура `parse()` обрабатывает различные отдельные поля в потоке данных, выполняя соответствующие запрошенные действия, прежде чем наконец вернуться в цикл для получения дальнейших инструкций от клиента.

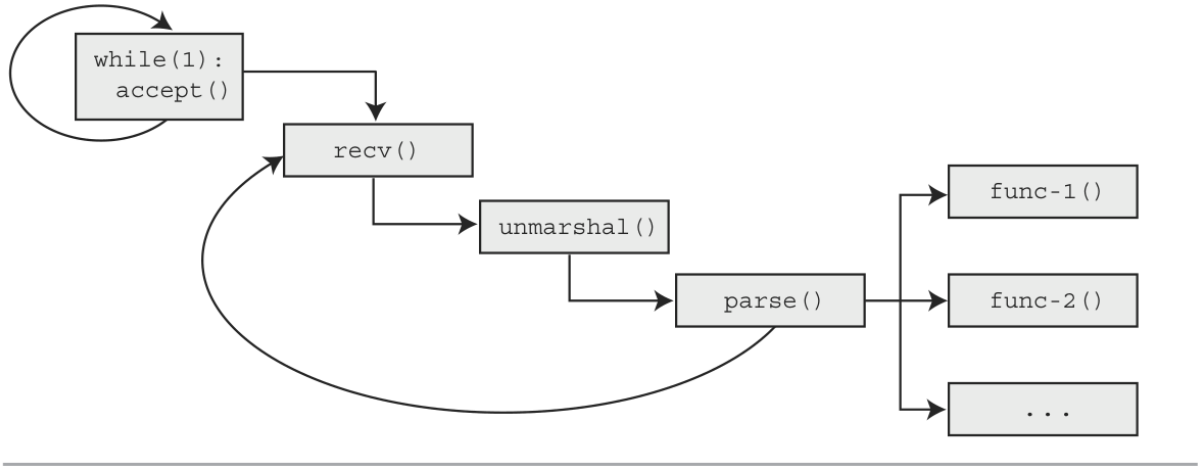


Рисунок 19.2. Пример схемы потока управления

Если дана такая ситуация, что, если вместо передачи фаззинговых данных по сети мы могли бы целиком пропустить сетевые этапы? Такая изоляция кода позволяет нам работать непосредственно с процедурами, в которых мы, вероятнее всего, найдём ошибки, например с процедурой разбора. Затем мы можем внедрять дефекты в процедуру и отслеживать результат этих изменений, одновременно резко повышая эффективность процесса фаззинга, поскольку всё выполняется внутри памяти. Посредством некоторой формы инструментирования процесса мы хотим «подключиться» к нашей цели непосредственно перед процедурами разбора, изменить различные принимаемые ими входные данные и позволить процедурам пройти свой путь. Очевидно, поскольку предмет этой книги представляет собой фаззинг, нам, разумеется, нужна возможность циклически повторять эти шаги, автоматически изменяя входные данные на каждой итерации и отслеживая результаты.

Цели

Вероятно, самым трудоёмким требованием при создании фаззера сетевого протокола или формата файла является разбор самих протокола и формата файла. Предпринимались попытки автоматического разбора протоколов, заимствующие методы из области биоинформатики, темы, которой мы касаемся в главе 22, «Автоматизированный разбор протокола». Однако по большей части разбор протокола является ручным и утомительным процессом.

При работе со стандартными документированными протоколами, такими как SMTP, POP и HTTP, например, вы получаете преимущество обращения к многочисленным источникам, таким как RFC, клиенты с открытым исходным кодом и, возможно, даже ранее написанные фаззеры. Кроме того, учитывая широкое использование этих протоколов различными приложениями, возможно, стоит реализовать протокол в своём фаззере, поскольку он будет многократно использоваться повторно. Однако вы часто будете оказываться в ситуации, когда целевой протокол недокументирован, сложен или является проприетарным. В таких случаях, когда разбор протокола может потребовать значительного времени и ресурсов, фаззинг в памяти способен послужить жизнеспособной альтернативой, экономящей время.

Что, если цель взаимодействует посредством тривиального протокола, инкапсулированного внутри открытого стандарта шифрования? Что ещё хуже, что, если протокол инкапсулирован внутри проприетарной схемы шифрования или обфускации? Чрезвычайно популярный комплекс связи Skype[5] является прекрасным примером трудной для аудита цели. Группа безопасности EADS/CRC приложила огромные усилия,[6] чтобы пробиться сквозь уровни обфускации и выявить критический дефект безопасности Skype (`SKYPE-SB/2005-003`).[7] Возможно, вам удастся обойти проблему, если

цель предоставляет незашифрованный интерфейс, но что, если это не так? Зашифрованные протоколы являются одним из главных препятствий, останавливающих фаззинг. Фаззинг в памяти также может позволить вам избежать кропотливого процесса обратной разработки процедуры шифрования.

Перед тем как применить этот подход к фаззингу своей цели, необходимо учесть множество факторов, первый и наиболее существенный из которых заключается в том, что фаззинг в памяти требует обратной разработки цели для обнаружения оптимальных точек «подключения», поэтому порог входа может оказаться непомерным. В целом фаззинг в памяти лучше всего направлять на цели с закрытым исходным кодом, работающие на платформе, которую поддерживают ваши инструменты фаззинга в памяти.

Метод: вставка цикла мутации

Первый рассматриваемый нами метод фаззинга в памяти называется вставкой цикла мутации (mutation loop insertion, MLI). MLI требует, чтобы мы сначала вручную, посредством обратной разработки, определили начало и конец процедуры `parse()`. После их определения наш клиент MLI может вставить процедуру `mutate()` в пространство памяти цели. Процедура мутации отвечает за преобразование данных, обрабатываемых процедурой разбора, и может быть реализована несколькими способами, которые мы конкретно рассмотрим в следующей главе. Далее наш клиент MLI вставляет безусловные переходы с конца процедуры разбора на начало процедуры мутации и с конца процедуры мутации на начало процедуры разбора. После того как всё это будет сделано, схема потока управления нашей цели будет выглядеть так, как показано на рисунке 19.3.

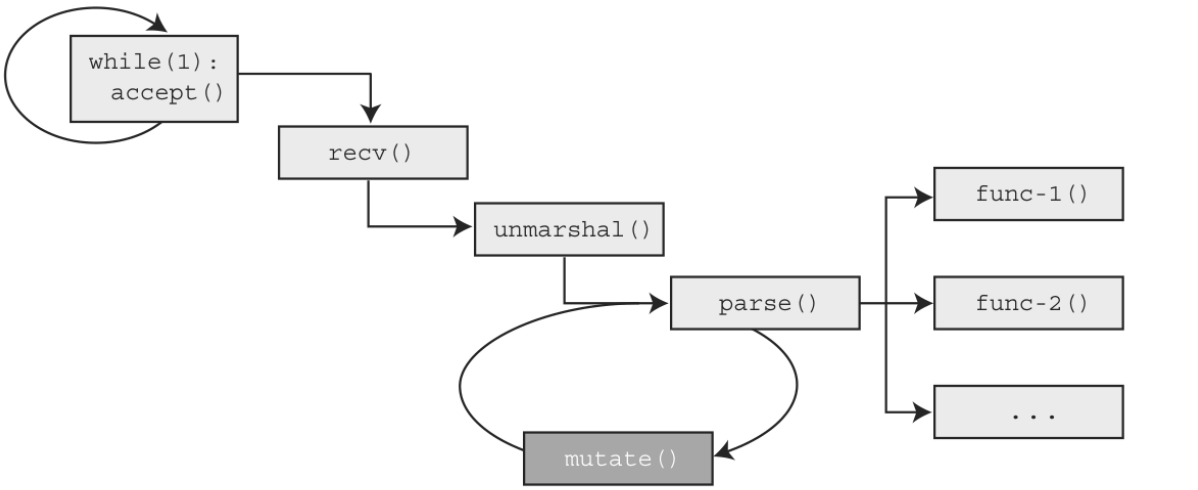


Рисунок 19.3. Визуальное представление вставки цикла мутации

Итак, что происходит теперь? Мы создали самодостаточный цикл мутации данных вокруг кода разбора нашей цели. Больше не требуется удалённо подключаться к цели и отправлять пакеты данных. Естественно, это даёт значительную экономию времени. Каждая итерация цикла будет передавать процедурам разбора разные данные, потенциально вызывающие дефекты, как предписывает `mutate()`.

Само собой разумеется, что наш искусственный пример чрезмерно упрощён. Хотя это сделано намеренно, чтобы обеспечить удобоваримое введение, важно отметить, что фаззинг в памяти носит в высшей степени экспериментальный характер. Мы углубимся в эти подробности в главе 20, «Фаззинг в памяти: автоматизация», когда будем строить функциональный фаззер.

Метод: мутация с восстановлением снимка

Второй метод фаззинга в памяти мы называем мутацией с восстановлением снимка (snapshot restoration mutation, SRM). Как и в случае MLI, мы хотим обойти ориентированные на сеть части нашей цели и атаковать центральную точку, в данном случае процедуру `parse()`. Также, как и MLI, SRM требует определить начало и конец кода разбора. Отметив эти места, наш клиент SRM сделает снимок целевого процесса, когда будет достигнуто начало кода разбора. После завершения разбора наш клиент SRM восстановит снимок процесса, изменит исходные данные и повторно исполнит код разбора. Изменённая схема потока управления нашей цели будет выглядеть так, как показано на рисунке 19.4.

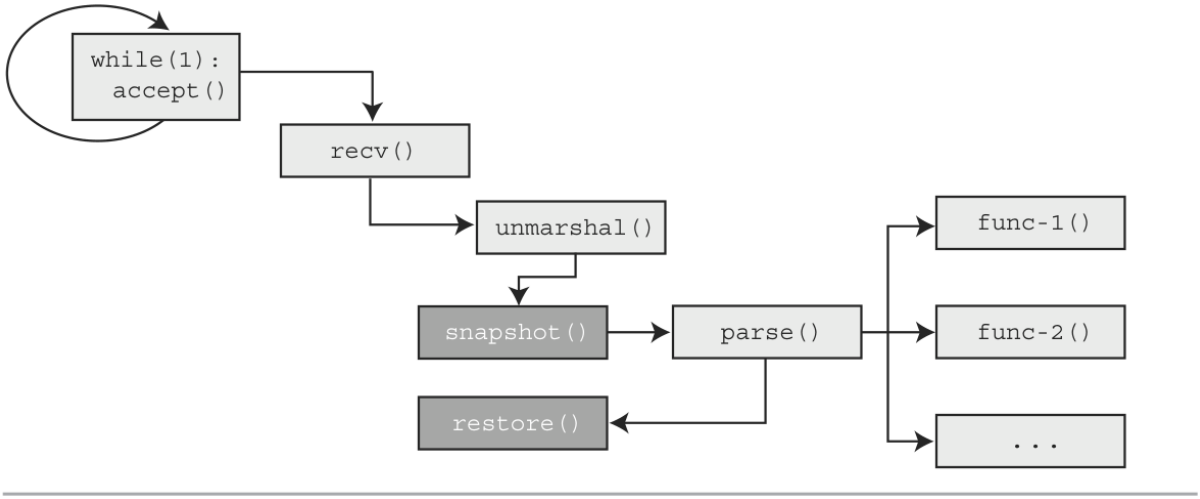


Рисунок 19.4. Визуальное представление мутации с восстановлением снимка

Мы снова создали самодостаточный цикл мутации данных вокруг кода разбора нашей цели. В главе 20 мы рассмотрим реализацию функционального инструмента для подтверждения концепции, чтобы применить оба описанных метода на практике. Построенный нами фаззер прототипного качества поможет эмпирически определить конкретные плюсы и минусы каждого подхода. Однако сначала давайте подробнее обсудим одно из главных преимуществ фаззинга в памяти.

Скорость тестирования и глубина процесса

С тестированием сетевых протоколов в традиционной модели клиент-цель связана неизбежная задержка. Тестовые случаи необходимо по отдельности передавать по сети и, в зависимости от сложности фаззера, ответ цели необходимо получать обратно и обрабатывать. Эта задержка дополнительно усугубляется при фаззинге глубоких состояний протокола. Рассмотрим, например, протокол почты POP, обычно предоставляющий службу через TCP-порт 110. Допустимый обмен для получения почты может выглядеть следующим образом, при этом ввод пользователя выделен полужирным:

```
$ nc mail.example.com 110
+OK Hello there.
user pedram@openrce.org
+OK Password required.
pass xxxxxxxxxxx
+OK logged in.
list

+OK
1 1673
2 19194
```

```
3 10187
... [output truncated]...
.
retr 1
+OK 1673 octets follow.
Return-Path: <ralph@openrce.org>
Delivered-To: pedram@openrce.org
… [output truncated]…
retr AAAAAAAAAAAAAAAAAAAAAA
-ERR Invalid message number. Exiting.
```

Чтобы наш сетевой фаззер нацелился на аргумент команды `RETR`, ему придётся проходить необходимые состояния процесса, заново подключаясь к почтовому серверу и повторно проходя аутентификацию для каждого тестового случая. Вместо этого мы могли бы сосредоточиться исключительно на разборе аргумента `RETR`, проводя тестирование только на желаемой глубине процесса с помощью одного из ранее описанных методов фаззинга в памяти. Вспомните, что глубина и состояние процесса объяснялись в главе 4, «Представление и анализ данных».

Обнаружение дефектов

Одним из преимуществ фаззинга в памяти является обнаружение вызванных фаззером дефектов. Поскольку мы уже работаем с нашей целью на столь низком уровне, по большей части добавить эту возможность легко. При обоих подходах, MLI и SRM, наш фаззер, если он реализован как отладчик, а так и будет, может отслеживать исключительные условия во время каждой изменённой итерации. Место дефекта можно сохранить вместе со сведениями о контексте времени исполнения, которые позднее исследователь сможет изучить, чтобы определить точное расположение ошибки. Следует отметить, что цели, использующие методы противодействия отладке, такие как ранее упомянутый Skype, будут мешать выбранному нами методу реализации.

Самой большой трудностью при обнаружении дефектов станет отсеивание ложных срабатываний. Во многом подобно принципу неопределённости Гейзенберга,[8] сам акт наблюдения за нашим целевым процессом под микроскопом фаззинга в памяти заставляет его вести себя иначе. Поскольку мы напрямую изменяем состояние целевого процесса способами, которые совершенно определённо не были задуманы изначально, существует большой простор для возникновения несущественных дефектов. Рассмотрим рисунок 19.5.

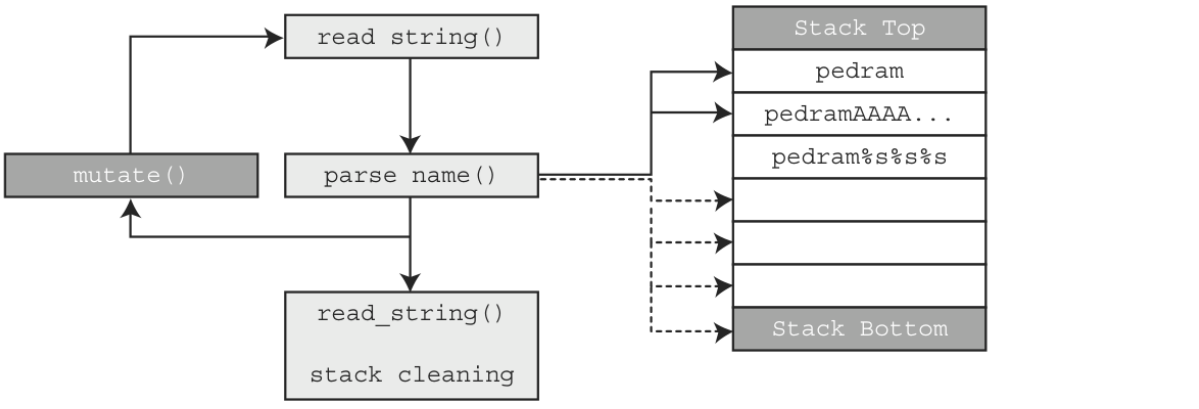


Рисунок 19.5. Исчерпание стека при вставке цикла мутации

Мы применили MLI, чтобы охватить циклом `parse_name()` и `read_string()`. На каждой итерации входные данные `parse_name()` изменяются в попытке вызвать дефект. Однако, поскольку мы подключаемся после выполнения выделений в стеке и непосредственно перед коррекцией стека, на каждой итерации цикла мы непрерывно теряем пространство стека. В конечном счёте мы

исчерпаем доступное пространство стека и тем самым вызовем неустранимый дефект, целиком порождённый нашим процессом фаззинга, а не уже существовавший в цели.

Итоги

Фаззинг в памяти является занятием не для слабонервных. Прежде чем можно будет провести какое-либо тестирование, квалифицированный исследователь должен выполнить обратную разработку цели, чтобы точно определить точки вставки и инструментирования. Исследователям также требуется определить, действительно ли проблемы, обнаруженные посредством фаззинга в памяти, достижимы с помощью пользовательских входных данных при обычных обстоятельствах. В большинстве случаев такой вещи, как «немедленная» находка, не будет, скорее будет потенциальное входное значение, которое следует рассмотреть. Эти недостатки перевешиваются многочисленными преимуществами, предоставляемыми методом.

Насколько нам известно, понятие фаззинга в памяти впервые представил публике Грег Хоглунд из HBGary LLC на конференциях по безопасности BlackHat Federal[9] и Blackhat USA[10] в 2003 году. HBGary предлагает коммерческое решение для фаззинга в памяти под названием Inspector.[11] В своей презентации Хоглунд намекает на часть своей проприетарной технологии при упоминании снимков памяти процесса. Снимки памяти являются одним из методов, которые мы подробно обсуждаем при создании инструмента автоматизации.

В следующей главе мы также пошагово рассмотрим реализацию и использование специализированного каркаса фаззинга в памяти. Насколько нам известно, это первый код с открытым исходным кодом для подтверждения концепции, взявшийся за эту задачу. По мере усовершенствования каркаса обновления будут публиковаться на веб-сайте книги по адресу http://www.fuzzing.org.

Сноски

[1] [назад] http://support.microsoft.com/kb/q291988/
[2] [назад] http://msdn2.microsoft.com/en-us/library/aa366786.aspx
[3] [назад] http://msdn2.microsoft.com/en-us/library/aa366786.aspx
[4] [назад] *Microsoft Windows Internals*, Fourth Edition, Mark E. Russinovich, David A. Solomon; *Undocumented Windows 2000 Secrets: A Programmer's Cookbook*, Sven Schreiber; *Undocumented Windows NT*, Prasad Dabak, Sandeep Phadke, Milind Borate.
[5] [назад] http://www.skype.com
[6] [назад] http://www.ossir.org/windows/supports/2005/2005-11-07/EADS-CCR_Fabrice_Skype.pdf
[7] [назад] http://www.skype.com/security/skype-sb-2005-03.html
[8] [назад] http://en.wikipedia.org/wiki/Heisenberg_principle
[9] [назад] http://www.blackhat.com/presentations/bh-federal-03/bh-fed-03-hoglund.pdf
[10] [назад] http://www.blackhat.com/presentations/bh-usa-03/bh-us-03-hoglund.pdf
[11] [назад] http://hbgary.com/technology.shtml

Глава 20. Фаззинг в памяти: автоматизация

> «Я слышал, в Интернетах ходят слухи, что у нас будет призыв».
>
> Джордж У. Буш, вторые президентские дебаты, Сент-Луис, штат Миссури, 8 октября 2004 года

В предыдущей главе мы ввели понятие фаззинга в памяти. Хотя ранее мы пытались сохранять равновесие между освещением платформ Windows и UNIX, из-за сложности рассматриваемой

темы для этого подхода к фаззингу мы решили сосредоточиться исключительно на платформе Windows. Точнее, мы рассматриваем 32-битную платформу Windows x86. В этой главе мы подробно обсуждаем пошаговые инструкции по созданию фаззера в памяти для подтверждения концепции. Мы перечислим желаемый набор возможностей, рассмотрим свой подход и обоснуем выбор языка. Наконец, после рассмотрения практического примера, мы завершим главу обзором внедрённых нами преимуществ и областей для улучшения. Весь обсуждаемый в этой главе код будет иметь открытый исходный код и находиться в открытом доступе по адресу http://www.fuzzing.org. Мы приветствуем от читателей как вклады в функциональность, так и сообщения об ошибках.

В качестве предварительного предупреждения обратите внимание: хотя мы затрагиваем и объясняем различные низкоуровневые аспекты архитектуры x86 и платформы Windows, мы не рассматриваем их глубоко. Это вышло бы далеко за рамки этой работы. Положительным моментом является то, что различные понятия, описанные нами в этой главе, и разрабатываемый код используются в последующих главах, таких как глава 24, «Интеллектуальное обнаружение дефектов».

Необходимый набор возможностей

Вспомните из главы 19, «Фаззинг в памяти», что мы представили два подхода к фаззингу в памяти, оба из которых хотим включить в нашу реализацию для подтверждения концепции: MLI и SRM. Для MLI нам нужна возможность изменять инструкцию с целью создания цикла, «подключаться» к цели в начале цикла и изменять целевой буфер в этой точке подключения. Для SRM нам понадобится возможность подключаться к целевому процессу в двух конкретных точках, делать копию пространства памяти цели в одной точке подключения, а затем восстанавливать пространство памяти цели в другой. Для дальнейшего обращения на рисунках 20.1 и 20.2 наши требования изображены наглядно: полужирный текст обозначает изменения, которые нам необходимо внести в целевой процесс, а тёмные блоки обозначают функциональность, которую должен добавить наш фаззер в памяти.

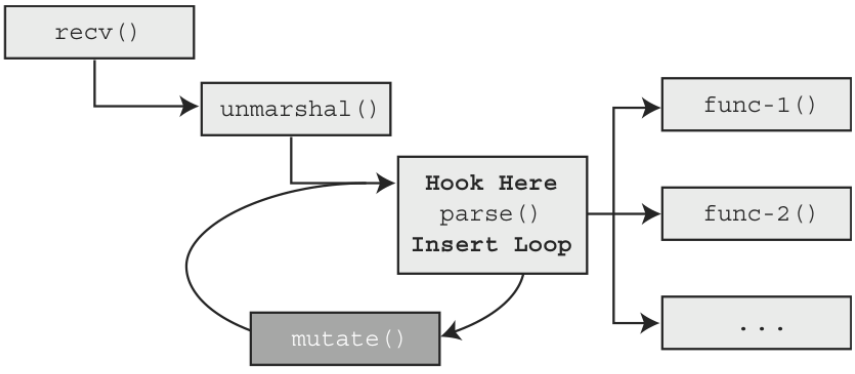


Рисунок 20.1. Требования к изменениям при вставке цикла мутации

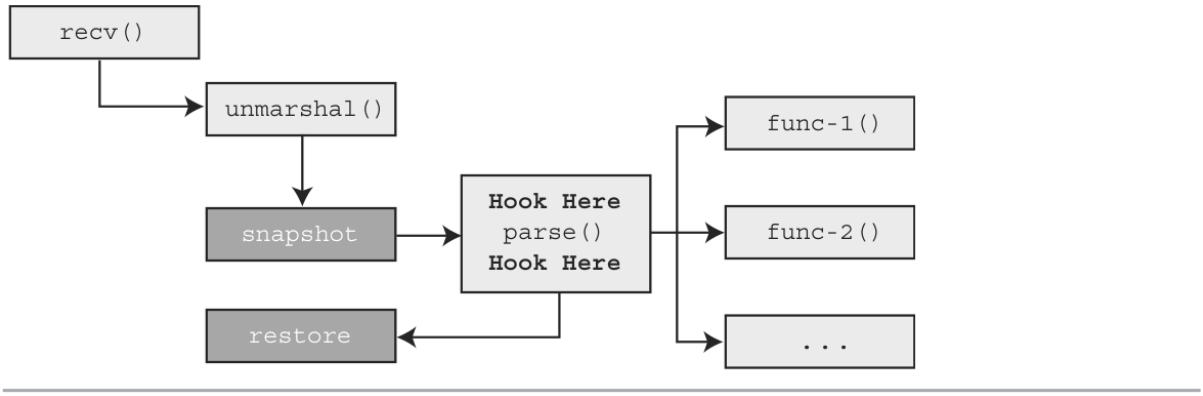


Рисунок 20.2. Требования к изменениям при мутации с восстановлением снимка

Для реализации перечисленных требований к инструментированию процесса мы напишем специализированный отладчик Windows. Создание нашего фаззера в памяти в виде отладчика, к счастью, удовлетворит все наши требования. Термин «инструментировать» мы определяем широко, охватывая ряд возможностей. Нам необходимо уметь произвольным образом читать из пространства памяти целевого процесса и записывать в него. Нам также необходимо уметь изменять контекст любого потока внутри целевого процесса. Контекст потока[1] содержит различные зависящие от процессора данные регистров, такие как текущий указатель инструкций (EIP), указатель стека (ESP) и указатель кадра (EBP), а также остальные регистры общего назначения (EAX, EBX, ECX, EDX, ESI и EDI). Управление контекстом потока позволит нам изменять состояние исполнения, что понадобится в процессе восстановления SRM. К счастью, как мы увидим позднее, операционная система Windows предоставляет мощный API, который удовлетворит все наши потребности.

ПРИМЕЧАНИЕ

Поскольку оба метода сходны по своей природе, при разработке и автоматизации мы решили сосредоточиться именно на SRM. В качестве упражнения мы рекомендуем читателям развить примеры из этой главы и реализовать подход MLI.

SRM обладает некоторыми явными преимуществами над MLI. Например, в SRM состояние глобальных переменных восстанавливается, а в MLI нет. Кроме того, при MLI необходимо очень осторожно выбирать точки инструментирования. Неосмотрительное решение в этом вопросе может внести утечку памяти, препятствующую успешному фаззингу.

Выбор языка

Прежде чем переходить к справочным сведениям о конкретных функциях и структурах API отладки Windows, которые мы будем использовать, давайте определим язык разработки, чтобы представлять справочные фрагменты кода на том же языке. Хотя с учётом наших требований наиболее естественным выбором языка сразу могут показаться C или C++, мы решили, что было бы идеально написать инструмент на интерпретируемом языке. Разработка на интерпретируемом языке, таком как Perl, Python или Ruby, имеет ряд преимуществ. Для нас величайшее преимущество заключается в скорости разработки, а для вас, читателя, она повысит удобочитаемость кода. В конечном счёте для этой задачи мы выбрали Python после обнаружения модуля Python ctypes,[2] разработанного Томасом Хеллером. Модуль ctypes предоставляет нам интерфейс к Windows API, а также возможность легко создавать сложные типы данных C и манипулировать ими непосредственно в Python. Например, следующий фрагмент демонстрирует, насколько легко вызвать функцию `GetCurrentProcessId()`, экспортируемую `kernel32.dll`:

```
from ctypes import *

# создать удобный псевдоним для kernel32.
kernel32 = windll.kernel32

# определить идентификатор текущего процесса.
current_pid = kernel32.GetCurrentProcessId()

print “The current process ID is %d” % current_pid
```

Создавать и передавать типы данных C столь же легко. Модуль ctypes предоставляет все необходимые основные примитивы в виде собственных классов Python, как показано в таблице 20.1.

Таблица 20.1. Совместимые с C типы данных ctypes

Тип ctypes	Тип C	Тип Python
c_char	Char	character
c_int	Int	Integer
c_long	Long	Integer
c_ulong	unsigned long	Long
c_char_p	Char *	string or None
c_void_p	Void *	integer or None

* Полный список доступных примитивов можно найти по адресу http://starship.python.net/crew/theller/ctypes/tutorial.html

Всем доступным типам при создании экземпляра можно передать необязательное начальное значение. В качестве альтернативы значение можно установить, присвоив атрибут `value`. Передать значение по ссылке можно без труда с помощью вспомогательной функции `byref()`. Важно помнить, что типы указателей, такие как `c_char_p` и `c_void_p`, неизменяемы. Для создания блока изменяемой памяти используйте вспомогательную функцию `create_string_buffer()`. Для доступа к изменяемому блоку или его изменения используйте атрибут `raw`. Следующий фрагмент демонстрирует эти положения при вызове `ReadProcessMemory()`:

```
read_buf = create_string_buffer(512)
count    = c_ulong(0)

kernel32.ReadProcessMemory(h_process, \
    0xDEADBEEF, \
    read_buf,   \
    512,        \
    byref(count))

print “Successfully read %d bytes: ” % count.value
print read_buf.raw
```

API `ReadProcessMemory()` принимает дескриптор процесса, из пространства памяти которого мы хотим читать, адрес, с которого следует читать, указатель на буфер для хранения прочитанных данных, число байт, которые мы хотим прочитать, и, наконец, указатель на целое число для хранения фактического числа байт, которые удалось прочитать. Раз уж мы говорим о чтении памяти процесса, следует также обсудить запись в память процесса, продемонстрированную в следующей выдержке кода:

```
c_data = c_char_p(data)
```

```
length = len(data)
count  = c_ulong(0)

kernel32. WriteProcessMemory(h_process, \
    0xC0CAC01A, \
    c_data,     \
    length,     \
    by_ref(count))

print “Sucessfully wrote %d bytes: ” % count.value
```

Функция `WriteProcessMemory()` имеет формат, сходный с её родственной функцией `ReadProcessMemory()`. Ей требуются дескриптор процесса, в пространство памяти которого мы хотим записать, адрес для записи, указатель на буфер, содержащий данные, которые мы хотим записать, число байт, которые мы хотим записать, и, наконец, указатель на целое число для хранения фактического числа байт, которые удалось записать. Поскольку возможность читать и записывать память процесса нужна нам для выполнения ряда требований, позднее в этой главе вы увидите эти процедуры в действии.

Теперь у нас есть строительные блоки, необходимые для погружения в разработку, поэтому давайте кратко рассмотрим, что нам нужно знать об API для создания отладчика.

API отладки Windows

В главе 19 мы кратко обсудили организацию памяти и компоненты типичного процесса Windows. Прежде чем продолжить, давайте рассмотрим некоторые справочные сведения об API отладки Windows, который мы будем использовать или злоупотреблять им, в зависимости от вашей точки зрения.

Начиная с Windows NT, операционная система предоставляет мощный набор функций и структур API, позволяющий разработчикам сравнительно легко создавать событийно-ориентированный отладчик. Основные компоненты API отладки можно разделить на следующие три категории: функции,[3] события[4] и структуры.[5] Рассматривая конкретные подробности реализации различных требований, мы затронем все три элемента. Первая задача, которую нам необходимо выполнить, заключается в запуске цели под управлением нашего отладчика. Сделать это можно двумя способами. Мы можем либо загрузить целевой процесс под управлением нашего отладчика, либо позволить целевому процессу запуститься самостоятельно, а затем присоединить к нему отладчик. Для загрузки процесса под управлением отладчика можно использовать следующий фрагмент кода:

```
pi = PROCESS_INFORMATION()
si = STARTUPINFO()
si.cb = sizeof(si);

kernel32.CreateProcessA(path_to_file,
    command_line,   \
    0,              \
    0,              \
    0,              \
    DEBUG_PROCESS,  \
    0,              \
    0,              \
    byref(si),      \
    byref(pi))

print “Started process with pid %d” pi.dwProcessId
```

Обратите внимание на добавленную к `CreateProcess` букву `A`: API Windows часто экспортируются как в версии Unicode, так и в версии ANSI. В таких случаях версия API без добавленной буквы служит простой обёрткой. Однако для наших целей при работе через ctypes необходимо вызывать

версию с добавленной буквой. Определить, экспортируется ли конкретный API и в ANSI, и в Unicode, можно простой проверкой страницы MSDN. Например, страница MSDN о `CreateProcess`[6] сообщает ближе к концу следующее: «Реализовано как `CreateProcessW` (Unicode) и `CreateProcessA` (ANSI)». Структуры `PROCESS_INFORMATION` и `STARTUP_INFO` передаются API `CreateProcess` по ссылке и, в свою очередь, заполняются сведениями, которые понадобятся нам позднее, такими как идентификатор созданного процесса (`pi.dwProcessId`) и дескриптор созданного процесса (`pi.hProcess`). В качестве альтернативы, если бы мы хотели присоединиться к уже работающему процессу, мы вызвали бы `DebugActiveProcess()`:

```
# подключиться к процессу с указанным идентификатором.
kernel32. DebugActiveProcess(pid)

# разрешить отключение в системах, которые его поддерживают.
try:
    kernel32.DebugSetProcessKillOnExit(True)
except:
    pass
```

Обратите внимание, что ни один вызов API в этом фрагменте не требует добавленной буквы `A` или `W`. Процедура `DebugActiveProcess()` присоединяет наш отладчик к процессу с указанным идентификатором. До вызова `DebugActiveProcess()` нам, возможно, придётся повысить уровень привилегий, но об этом позаботимся позднее. Процедура `DebugSetProcessKillOnExit()`[7] доступна начиная с Windows XP и позволяет нам завершить работу отладчика, не уничтожая отлаживаемый процесс, к которому мы присоединены. Мы оборачиваем вызов конструкцией `try/except`, чтобы наш отладчик не завершился с ошибкой, если запущен на платформе, не поддерживающей запрошенный API, например Windows 2000. После передачи целевого процесса под управление нашего отладчика нам необходимо реализовать цикл обработки событий отладки. Цикл событий отладки можно представить как стереотипную любопытную, и пожилую, соседку в городе, Агнес. Агнес сидит у окна и наблюдает за всем происходящим по соседству. Хотя Агнес видит всё, большинство происходящего недостаточно интересно, чтобы побудить её позвонить друзьям. Однако изредка происходит нечто интересное. Ребёнок соседей загонит кошку на дерево, упадёт и сломает руку. Агнес немедленно позвонит в полицию. Подобно Агнес, наш цикл событий отладки будет видеть множество событий. Наша задача заключается в том, чтобы указать, какие события нас интересуют и в отношении каких мы хотим предпринять дополнительные действия. Ниже приведён каркас типичного цикла событий отладки:

```
debugger_active = True
dbg             = DEBUG_EVENT()
continue_status = DBG_CONTINUE

while debugger_active:
    ret = kernel32. WaitForDebugEvent(byref(dbg), 100)

    # если событие отладки не возникло, продолжить.
    if not ret:
        continue

    event_code = dbg.dwDebugEventCode

    if event_code == CREATE_PROCESS_DEBUG_EVENT:
        # создан новый процесс

    if event_code == CREATE_THREAD_DEBUG_EVENT:
        # создан новый поток

    if event_code == EXIT_PROCESS_DEBUG_EVENT:
        # процесс завершился

    if event_code == EXIT_THREAD_DEBUG_EVENT:
        # поток завершился

    if event_code == LOAD_DLL_DEBUG_EVENT:
```

```
        # загружена новая DLL

    if event_code == UNLOAD_DLL_DEBUG_EVENT:
        # DLL выгружена

    if event_code == EXCEPTION_DEBUG_EVENT:
        # перехвачено исключение

    # продолжить обработку
    kernel32.ContinueDebugEvent(dbg.dwProcessId, \
        dbg.dwThreadId, \
        continue_status)
```

Цикл обработки событий отладки основан главным образом на вызове `WaitForDebugEvent()`,[8] который принимает указатель на структуру `DEBUG_EVENT` в качестве первого аргумента и число миллисекунд ожидания возникновения события отладки в отлаживаемом процессе в качестве второго. Если событие отладки происходит, структура `DEBUG_EVENT` будет содержать тип события отладки в атрибуте `dwDebugEventCode`. Мы исследуем эту переменную, чтобы определить, было ли событие отладки вызвано созданием или завершением процесса, созданием или завершением потока, загрузкой или выгрузкой DLL либо событием исключения отладки. Если происходит событие исключения отладки, конкретную причину исключения можно определить, исследовав атрибут структуры `DEBUG_EVENT u.Exception.ExceptionRecord.ExceptionCode`. В MSDN перечислен ряд возможных кодов исключений,[9] но для наших целей нас прежде всего интересуют следующие:

• `EXCEPTION_ACCESS_VIOLATION`. Нарушение доступа произошло из-за попытки чтения из недопустимого адреса памяти или записи по нему.
• `EXCEPTION_BREAKPOINT`. Исключение было вызвано встреченной точкой останова.
• `EXCEPTION_SINGLE_STEP`. Ловушка одного шага включена и исполнена одна инструкция.
• `EXCEPTION_STACK_OVERFLOW`. Поток-нарушитель исчерпал пространство стека. Обычно это показатель неконтролируемой рекурсии, и, как правило, последствие ограничивается исключительно отказом в обслуживании.

Мы можем перемежать обработку различных событий и исключений любой желаемой логикой. Завершив обработку сообщённого события, мы позволяем потоку-нарушителю продолжить работу вызовом `ContinueDebugEvent()`.

Собираем всё вместе

На этом этапе мы обсудили основы организации памяти Windows, составили список требований, выбрали язык разработки, рассмотрели основы модуля ctypes и немного поэкспериментировали с основами API отладки Windows. Остаётся несколько нерешённых вопросов:

• Как реализовать нашу потребность «подключаться» к целевому процессу в конкретных точках?
• Как обрабатывать снимки процесса и восстановления?
• Как находить и изменять целевое пространство памяти?
• Как выбирать точки подключения?

Как реализовать нашу потребность «подключаться» к целевому процессу в конкретных точках?

Как мы уже упоминали в этой главе, при выбранном подходе подключение к процессу можно реализовать с помощью точек останова отладчика. На заданной платформе поддерживаются два типа точек останова: аппаратные и программные. Совместимые с 80x86 процессоры поддерживают до четырёх аппаратных точек останова. Каждую из них можно настроить на срабатывание при чтении, записи или исполнении любого диапазона длиной один, два или четыре байта. Для установки аппаратных точек останова необходимо изменить контекст целевого процесса, манипулируя отладочными регистрами с DR0 по DR3 и DR7. Первые четыре регистра содержат адрес аппаратной точки останова. Регистр DR7 содержит флаги, указывающие, какие точки останова активны, для какого диапазона и какого типа доступа, чтения, записи или исполнения. Аппаратные точки останова неинтрузивны и не изменяют код. Программные точки останова, напротив, должны изменять целевой процесс и реализуются однобайтной инструкцией `INT3`, представленной в шестнадцатеричном виде как `0xCC`.

Для выполнения нашей задачи можно было бы использовать как аппаратные, так и программные точки останова. Поскольку в какой-то момент нам может потребоваться более четырёх точек подключения, мы решаем использовать программные точки останова. Чтобы хорошо понять процесс, давайте пройдём необходимые шаги установки программной точки останова в вымышленной программе по адресу `0xDEADBEEF`. Сначала наш отладчик должен прочитать и сохранить исходный байт по целевому адресу с помощью упомянутого ранее вызова API `ReadProcessMemory`, как показано на рисунке 20.3.

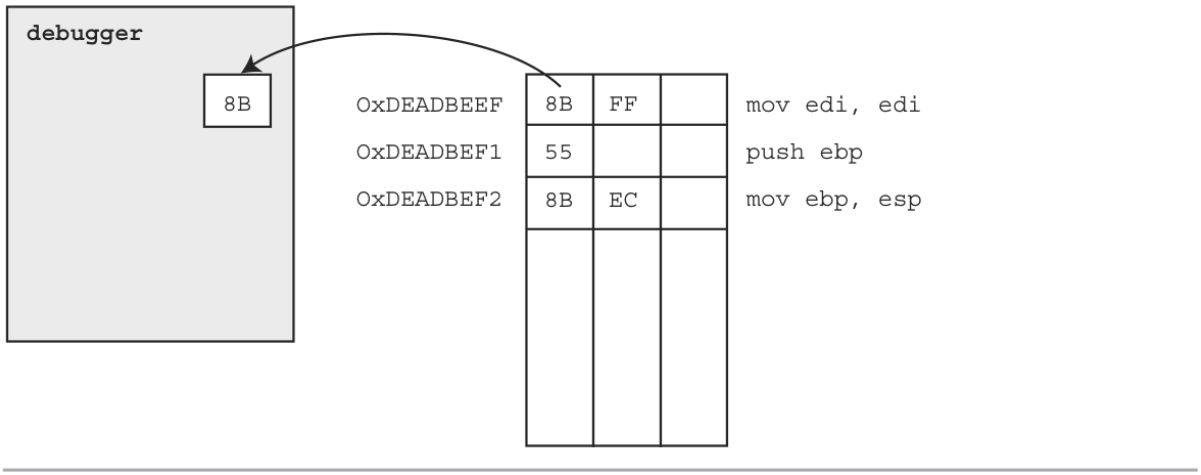


Рисунок 20.3. Сохранение исходного байта по адресу точки останова

Обратите внимание, что первая инструкция по целевому адресу фактически состоит из двух байт. Следующий шаг заключается в записи инструкции `INT3` по целевому адресу с помощью также упомянутого ранее API `WriteProcessMemory`, как показано на рисунке 20.4.

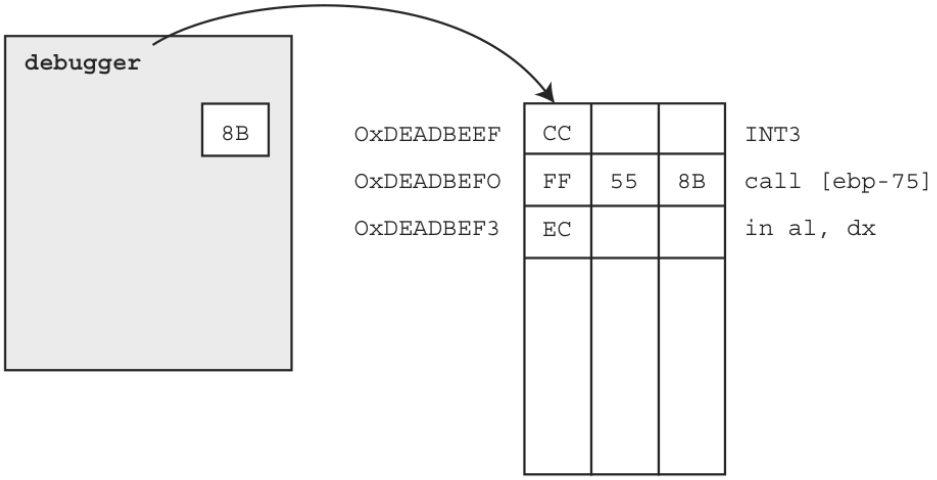


Рисунок 20.4. Запись кода операции INT3

Но что случилось с нашими прежними инструкциями? Вставленный `0xCC` дизассемблируется как однобайтная инструкция `INT3`. Второй байт инструкции `mov edi, edi` (`0xFF`), объединённый с байтом, ранее представлявшим `push ebp` (`0x55`), и первым байтом инструкции `mov ebp, esp` (`0x8B`), дизассемблируются как инструкция `call [ebp-75]`. Оставшийся байт `0xEC` дизассемблируется как однобайтная инструкция `in al, dx`. Теперь, когда исполнение достигнет адреса `0xDEADBEEF`, инструкция `INT3` вызовет событие отладки `EXCEPTION_DEBUG_EVENT` с кодом исключения `EXCEPTION_BREAKPOINT`, которое наш отладчик перехватит в цикле событий отладки, помните Агнес? Состояние процесса в этот момент показано на рисунке 20.5.

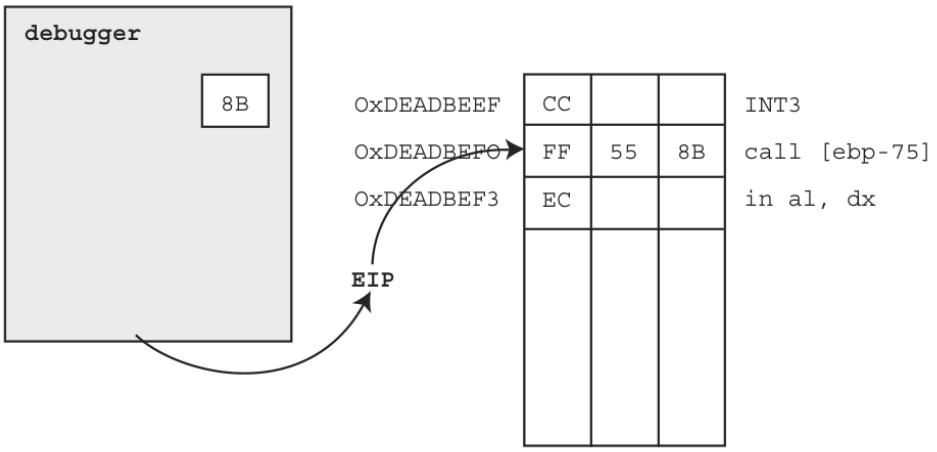


Рисунок 20.5. Перехвачено EXCEPTION_BREAKPOINT

Итак, теперь мы успешно вставили и перехватили программную точку останова, но это лишь половина дела. Обратите внимание, что наши исходные инструкции всё ещё отсутствуют. Кроме того, указатель инструкций, EIP, который сообщает ЦП, откуда извлечь, декодировать и исполнить следующую инструкцию, находится по адресу `0xDEADBEF0`, а не `0xDEADBEEF`. Причина в том, что единственная инструкция `INT 3`, вставленная нами по адресу `0xDEADBEEF`, была успешно исполнена, в результате чего EIP обновился до `0xDEADBEEF+1`. Прежде чем продолжить, нам придётся исправить значение EIP и восстановить исходный байт по адресу `0xDEADBEEF`, как показано на рисунке 20.6.

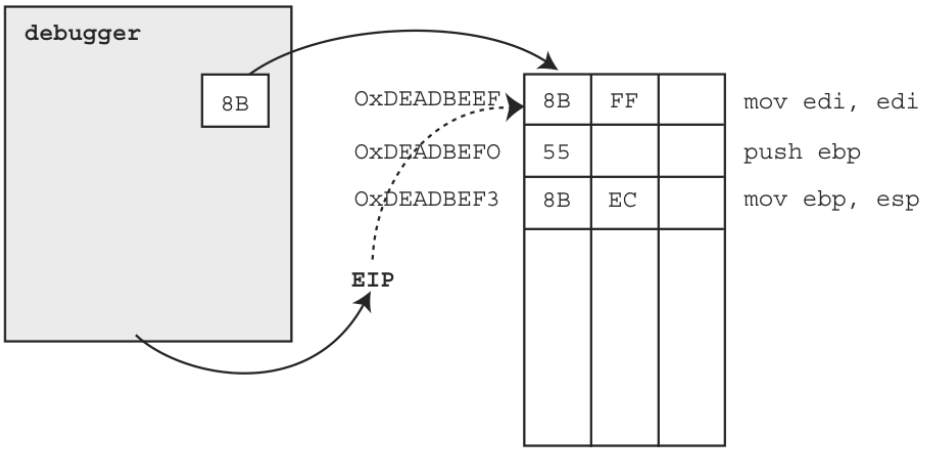


Рисунок 20.6. Корректировка EIP

Восстановление байта по адресу `0xDEADBEEF` является уже знакомой нам задачей. Однако изменение значения указателя инструкций, регистра EIP, это совсем другая история. Ранее в этой главе мы упоминали, что контекст потока содержит различные зависящие от процессора данные регистров, такие как указатель инструкций EIP, который сейчас нас интересует. Получить контекст любого заданного потока можно вызовом API `GetThreadContext()`,[10] передав ему дескриптор текущего потока и указатель на структуру `CONTEXT`. Затем мы можем изменить содержимое структуры `CONTEXT` и вызвать API `SetThreadContext()`,[11] снова передав дескриптор текущего потока для изменения контекста:

```
context = CONTEXT()
context.ContextFlags = CONTEXT_FULL
kernel32.GetThreadContext(h_thread, byref(context))
context.Eip -= 1
kernel32.SetThreadContext(h_thread, byref(context))
```

Теперь исходный контекст исполнения восстановлен, и мы готовы позволить процессу продолжиться.

Как обрабатывать снимки процесса и восстановления?

Чтобы ответить на этот вопрос, сначала следует рассмотреть другой: что изменяется во время работы процесса? Очень многое. Создаются и уничтожаются новые потоки. Открываются и закрываются дескрипторы файлов, сокетов, окон и других элементов. Память выделяется и освобождается, из неё читают и в неё записывают. Различные регистры в контекстах отдельных потоков крайне изменчивы и постоянно меняются. Мы могли бы выполнить задачу с помощью технологии виртуальных машин, такой как VMWare,[12] которая позволяет делать и восстанавливать полные снимки системы. Однако этот процесс мучительно медленен и требует взаимодействия между гостевой системой виртуальной машины и некоторой формой посредника на узле виртуальной машины. Вместо этого мы заимствуем ранее обсуждавшийся метод[13] и «жульничаем», занимаясь только контекстами потоков и изменениями памяти. Процесс создания снимка будет состоять из двух шагов.

На первом шаге мы сохраняем контекст каждого потока внутри целевого процесса. Мы уже видели, насколько легко получать и задавать контексты отдельных потоков. Теперь этот код необходимо обернуть логикой, отвечающей за перечисление системных потоков, которые принадлежат целевому процессу. Для этого мы полагаемся на Tool Help Functions.[14] Сначала получаем список всех системных потоков, указав флаг `TH32CS_SNAPTHREAD`:

```
thread_entry = THREADENTRY32()
```

```
contexts     = []

snapshot = kernel32.CreateToolhelp32Snapshot( \
    TH32CS_SNAPTHREAD,                        \
    0)
```

Далее мы получаем из списка первую запись потока. Однако до этого обязательное требование API `Thread32First()` состоит в инициализации переменной `dwSize` внутри структуры записи потока. API `Thread32First()` мы передаём ранее сделанный снимок и указатель на структуру записи потока:

```
thread_entry.dwSize = sizeof(thread_entry)

success = kernel32.Thread32First( \
    snapshot,                     \
    byref(thread_entry))
```

Наконец, перебираем потоки в поисках принадлежащих идентификатору процесса (`pid`) нашей цели. Если такие есть, получаем дескриптор потока с помощью API `OpenThread()`, получаем контекст, как прежде, и добавляем его в список:

```
while success:
    if thread_entry.th32OwnerProcessID == pid:
        context = CONTEXT()
        context.ContextFlags = CONTEXT_FULL

        h_thread = kernel32.OpenThread( \
            THREAD_ALL_ACCESS,          \
            None,                       \
            thread_id)

        kernel32.GetThreadContext( \
            h_thread,              \
            byref(context))

        contexts.append(context)
        kernel32.CloseHandle(h_thread)

    success = kernel32.Thread32Next( \
        snapshot,                    \
        byref(thread_entry))
```

Сохранив контекст каждого из потоков, принадлежащих нашему процессу, позднее мы можем восстановить снимок процесса, снова перебрав все системные потоки и восстановив сохранённый контекст всех ранее замеченных потоков.

На втором шаге мы сохраняем содержимое каждого изменяемого блока памяти. Вспомните из предыдущей главы, что каждый процесс на 32-битной платформе Windows x86 «видит» собственное пространство памяти объёмом 4 Гбайт. Из этих 4 Гбайт обычно нижняя половина зарезервирована для использования нашим процессом (`0x0000000-0x7FFFFFFF`). Это пространство памяти далее разделено на отдельные страницы, обычно размером 4096 байт. Наконец, разрешения памяти с наименьшей гранулярностью применяются к каждой отдельной странице. Вместо хранения содержимого каждой отдельно используемой страницы памяти мы экономим время и ресурсы, ограничивая снимок страницами, которые считаем изменяемыми. Мы игнорируем все страницы с ограничительными разрешениями, предотвращающими запись. Сюда входят страницы, помеченные как:

- `PAGE_READONLY`
- `PAGE_EXECUTE_READ`
- `PAGE_GUARD`
- `PAGE_NOACCESS`

Мы также хотим игнорировать страницы, принадлежащие исполняемым образам, поскольку они вряд ли изменятся. Для прохода по всем доступным страницам памяти требуется простой цикл

вокруг процедуры API `VirtualQueryEx()`,[15] предоставляющей сведения о страницах в указанном диапазоне виртуальных адресов:

```
cursor        = 0
memory_blocks = []
read_buf      = create_string_buffer(length)
count         = c_ulong(0)
mbi           = MEMORY_BASIC_INFORMATION()

while cursor < 0xFFFFFFFF:
    save_block = True

    bytes_read = kernel32.VirtualQueryEx( \
        h_process,                        \
        cursor,                           \
        byref(mbi),                       \
        sizeof(mbi))

    if bytes_read < sizeof(mbi):
        break
```

Если вызов `VirtualQueryEx()` завершается неудачей, мы предполагаем, что доступное пользовательское пространство исчерпано, и выходим из цикла чтения. Для каждого обнаруженного в цикле блока памяти мы проверяем подходящие разрешения страницы:

```
    if mbi.State != MEM_COMMIT or \
       mbi.Type  == MEM_IMAGE:
        save_block = False

    if mbi.Protect & PAGE_READONLY:
        save_block = False

    if mbi.Protect & PAGE_EXECUTE_READ:
        save_block = False

    if mbi.Protect & PAGE_GUARD:
        save_block = False

    if mbi.Protect & PAGE_NOACCESS:
        save_block = False
```

Если нам встречается блок памяти, который мы хотим включить в снимок, мы читаем его содержимое с помощью API `ReadProcessMemory()`[16] и сохраняем необработанные данные, а также сведения о памяти в списке снимка. Наконец, увеличиваем курсор сканирования памяти и продолжаем:

```
    if save_block:
        kernel32.ReadProcessMemory( \
            h_process,              \

            mbi.BaseAddress,        \
            read_buf,               \
            mbi.RegionSize,         \
            byref(count))

        memory_blocks.append((mbi, read_buf.raw))

    cursor += mbi.RegionSize
```

Возможно, вы сразу заметили изъян в нашей методологии. Что, если во время создания снимка заданная страница помечена как `PAGE_READONLY`, но позднее изменяется на доступную для записи и модифицируется? Хорошее замечание, ответ состоит в том, что мы упускаем такие случаи. Мы никогда не обещали, что подход будет совершенным! Фактически воспользуемся этой возможностью, чтобы снова подчеркнуть экспериментальный характер предмета. Для любознательных читателей одним потенциальным решением этого недостатка является «подключение» к различным функциям, изменяющим разрешения памяти, и изменение нашего

мониторинга на основании наблюдаемых изменений.

В совокупности два наших шага предоставляют необходимые элементы для создания и восстановления снимков конкретного процесса.

Как выбирать точки подключения?

Здесь мы переключаемся с науки на искусство, поскольку окончательного подхода к выбору точек подключения не существует. Решение потребует опыта бывалого специалиста по обратной разработке. По существу, мы ищем начало и конец кода, отвечающего за разбор управляемых пользователем данных. Если у вас нет предварительных знаний о целевом программном обеспечении, трассировка с помощью отладчика является хорошим началом для сужения диапазона выбора. Трассировка отладчиком подробнее рассматривается в главе 23, «Отслеживание фаззера».

Чтобы хорошо это почувствовать, позднее мы пошагово рассмотрим пример.

Как находить и изменять целевое пространство памяти?

Последний шаг в нашем длинном контрольном списке предварительных условий состоит в выборе областей памяти для изменения. И снова этот процесс может быть скорее искусством, чем наукой, а трассировка отладчиком способна помочь в поиске. Однако в общем случае нам следовало выбрать исходную точку подключения так, чтобы указатель на целевые данные или область памяти существовал в непосредственной близости.

PyDbg, ваш новый лучший друг

Как вы, несомненно, уже поняли, написание отладчика требует немалой работы. К счастью, всё рассмотренное нами до этого момента, и даже больше, авторы уже написали для вас в виде удобного класса Python под названием PyDbg.[17] Почему мы не упомянули об этом раньше, можете раздражённо спросить вы себя? Что ж, мы вам солгали. Основы важно понимать, и, скорее всего, вы не обратили бы внимания, сообщи мы вам о коротком пути с самого начала.

Инструментирование процессов через API отладки Windows с PyDbg выполняется проще простого. С PyDbg вы без труда можете делать следующее:

• читать, записывать и запрашивать память;
• перечислять процессы, присоединяться к ним, отсоединяться и завершать их;
• перечислять, приостанавливать и возобновлять потоки;
• устанавливать, удалять и обрабатывать точки останова;
• создавать и восстанавливать снимки состояния процесса, то есть SR из SRM;
• разрешать адреса функций;
• и многое другое...

Рассмотрим следующий простой пример, где мы создаём экземпляр объекта PyDbg, присоединяемся к целевому процессу с PID 123 и входим в цикл отладки:

```
from pydbg import *
from pydbg.defines import *

dbg = new pydbg()
dbg.attach(123)
dbg.debug_event_loop()
```

Это не слишком увлекательно, поэтому добавим немного функциональности. Развивая предыдущий пример, в следующем фрагменте мы устанавливаем точку останова на вызове API Winsock `recv()` и регистрируем обработчик обратного вызова, вызываемый при каждом срабатывании точки останова:

```
from pydbg import *
from pydbg.defines import *

ws2_recv = None

def handler_bp (pydbg, dbg, context):
    global ws2_recv

    exception_address = \
        dbg.u.Exception.ExceptionRecord.ExceptionAddress

    if exception_address == ws2_recv:
        print “ws2.recv() called!”

    return DBG_CONTINUE

dbg = new pydbg()
dbg.set_callback(EXCEPTION_BREAKPOINT, handler_bp)
dbg.attach(123)

ws2_recv = dbg.func_resolve(“ws2_32”, “recv”)
dbg.bp_set(ws2_recv)

dbg.debug_event_loop()
```

Полужирный текст в предыдущем фрагменте выделяет наши дополнения. Первое видимое дополнение представляет собой определение функции `handler_bp()`, принимающей три аргумента. Первый аргумент получает наш экземпляр PyDbg. Второй получает структуру `DEBUG_EVENT`[18] из цикла событий отладки и содержит разнообразные сведения о только что произошедшем событии отладки. Третий аргумент получает контекст потока, в котором произошло событие отладки. Наш обработчик точки останова просто проверяет, совпадает ли адрес, по которому произошло исключение, с адресом API Winsock `recv()`. Если да, выводится сообщение. Обработчик точки останова возвращает `DBG_CONTINUE`, сообщая PyDbg, что обработка исключения завершена и PyDbg следует позволить целевому процессу продолжить исполнение. Возвращаясь к основной части сценария отладчика, вы увидите добавление вызова процедуры PyDbg `set_callback()`. Эта процедура используется для регистрации функции обратного вызова, посредством которой PyDbg обрабатывает указанное событие или исключение отладки. В данном случае нас интересовал вызов `handler_bp()` при каждом срабатывании точки останова. Наконец, мы видим добавление вызовов `func_resolve()` и `bp_set()`. Первый разрешает адрес API `recv()` внутри модуля Windows `ws2_32.dll` и сохраняет его в глобальной переменной. Второй устанавливает точку останова по разрешённому адресу. После присоединения к целевому процессу любые вызовы API Winsock `recv()` приведут к тому, что отладчик отобразит сообщение «ws2.recv() called» и продолжит исполнение обычным образом. И снова это не слишком увлекательно, но теперь мы можем сделать ещё один скачок вперёд и создать свой первый фаззер в памяти для подтверждения концепции.

Искусственный пример

Рассмотрев множество справочных и предварительных сведений, мы наконец должны создать первоначальное подтверждение концепции, чтобы показать вам, что вся эта теоретическая чепуха действительно может быть осуществима. На веб-сайте fuzzing.org вы найдёте `fuzz_client.exe` и `fuzz_server.exe`, включая исходный код обоих. Пока не изучайте исходный код. Чтобы воспроизвести более реалистичный сценарий, предположим, что требуется обратная разработка. Эта пара клиент-сервер является очень простой целью. После запуска сервер привязывается к TCP-порту 11427 и ожидает клиентского соединения. Клиент подключается и отправляет серверу данные, которые позднее разбираются. Как они разбираются? Мы точно не знаем, да сейчас нас это и не волнует, поскольку наша цель состоит в фаззинге цели, а не в изучении исходного или двоичного кода.

Сначала запустим сервер:

```
$ ./fuzz_server.exe
Listening and waiting for client to connect...
```

Далее запускаем клиент, принимающий два аргумента. Первый аргумент является IP-адресом сервера. Второй представляет собой данные для передачи серверу на разбор:

```
$ ./fuzz_client.exe 192.168.197.1 'sending some data'
connecting....
sending...
sent...
```

Клиент успешно подключился к серверу по адресу `192.168.197.1` и передал строку «sending some data». На стороне сервера мы видим следующие дополнительные сообщения:

```
client connected.
received 17 bytes.
parsing: sending some data
exiting...
```

Сервер успешно получил переданные нами 17 байт, разобрал их, а затем завершил работу. Однако при исследовании пакетов, проходящих по сети, мы не можем найти свои данные. Пакет, который должен содержать наши данные, появляется сразу после TCP-рукопожатия из трёх этапов, но вместо этого содержит байты, выделенные на снимке экрана Ethereal[19] на рисунке 20.7.

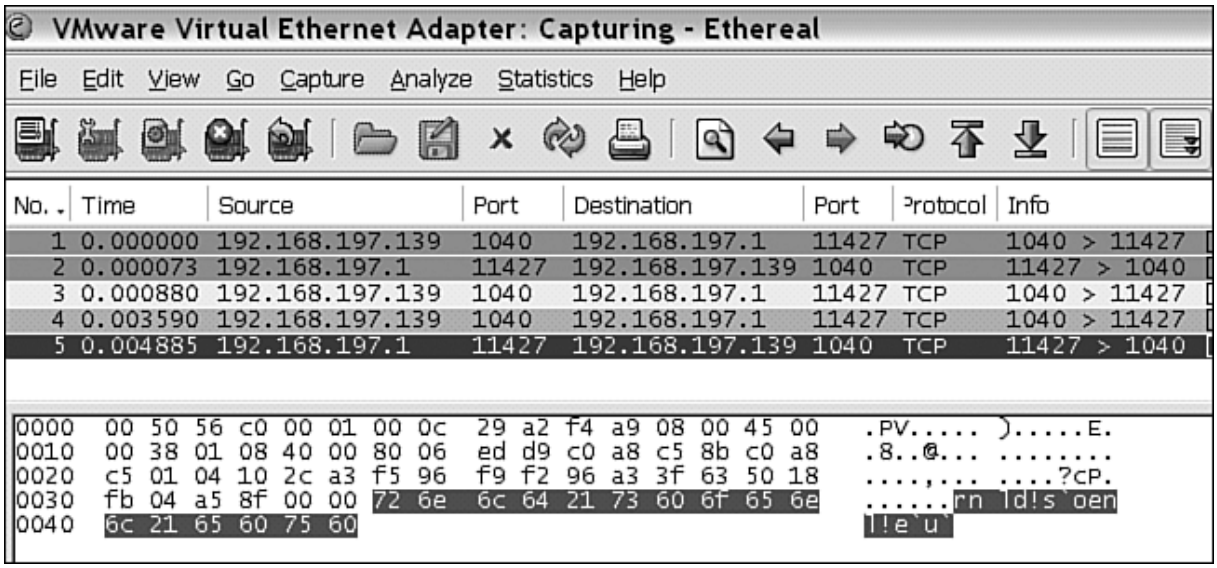


Рисунок 20.7. Захват данных клиент-сервер в Ethereal

Проект сетевого анализатора Ethereal перешёл в Wireshark и доступен для загрузки по адресу http://www.wireshark.org.

Клиент, должно быть, исказил, зашифровал, сжал или иным образом обфусцировал данные пакета до их записи в сеть. На стороне сервера он должен был деобфусцировать данные пакета до их разбора, поскольку в журнале выходных сообщений мы видим правильную строку. Фаззинг нашего примерного сервера традиционным способом требует обратной разработки процедуры обфускации. После раскрытия секретов метода обфускации мы сможем создавать и отправлять произвольные данные. Когда позднее вы увидите решение, то поймёте, что в данном случае сделать это было бы совсем несложно. Однако ради соблюдения условий нашего примера мы предполагаем, что обратная разработка метода обфускации потребует значительных ресурсов. Это прекрасный пример того, когда фаззинг в памяти может оказаться ценным, что неудивительно, ведь пример в конце концов искусственный. Мы избежим необходимости расшифровывать процедуру обфускации, вместо этого «подключившись» к `fuzz_server.exe` после того, как он уже деобфусцировал переданные данные.

Для выполнения задачи нам необходимо точно определить два места внутри `fuzz_server.exe`. Первое является точкой снимка. В какой момент исполнения мы хотим сохранить состояние целевого процесса? Второе является точкой восстановления. В какой момент исполнения мы хотим перемотать состояние процесса назад, изменить входные данные и продолжить инструментированный цикл исполнения? Ответ на оба вопроса требует небольшого отслеживания входных данных с помощью отладчика. Мы воспользуемся OllyDbg,[20] свободно доступным мощным отладчиком пользовательского режима Windows.

Описание использования и функциональности OllyDbg почти заслуживает отдельной книги, поэтому мы предполагаем, что вы уже с ним знакомы. Первым делом необходимо определить, где `fuzz_server.exe` получает данные. Мы знаем, что это происходит по TCP, поэтому загрузим `fuzz_server.exe` в OllyDbg и установим точку останова на API `recv()` из `WS2_32.dll`. Для этого откроем список модулей, выберем `WS2_32.dll` и нажмём Ctrl + N, чтобы открыть список имён внутри модуля, как показано на рисунке 20.8. Затем прокрутим список до `recv()` и нажмём F2 для включения точки останова.

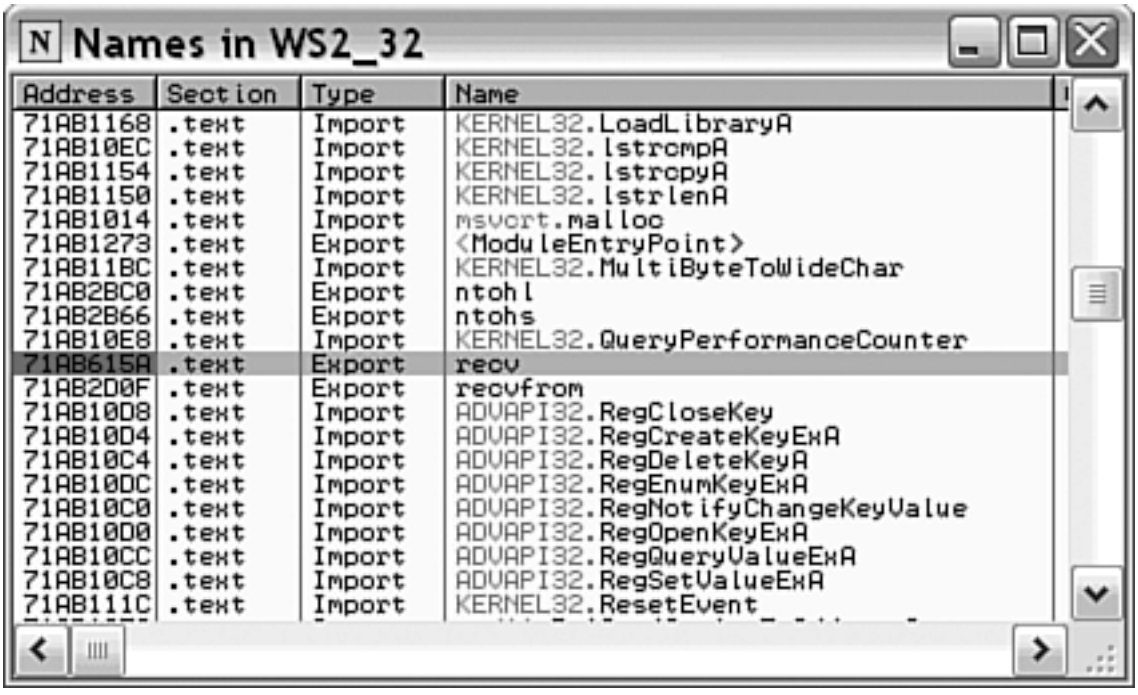


Рисунок 20.8. Точка останова OllyDbg на WS2_32.recv()

Установив точку останова, нажимаем F9 для продолжения исполнения, а затем запускаем `fuzz_client`, как и прежде. Сразу после отправки данных OllyDbg приостанавливает `fuzz_server`, поскольку установленная нами точка останова срабатывает. Затем нажимаем Alt + F9, чтобы

выполнить команду «Execute til user code». Теперь виден вызов из `fuzz_server` в `WS2_32`. Несколько раз нажимаем F8 и переступаем через вызов `printf()`, отображающий сообщение сервера о количестве полученных байт. Далее, как показано на рисунке 20.9, мы видим вызов безымянной подпрограммы внутри `fuzz_server` по адресу `0x0040100F`. Исследование первого аргумента этой функции в окне дампа OllyDbg показывает, что это указатель на обфусцированную форму наших данных, которую мы видели в Ethereal. Может ли процедура по адресу `0x0040100F` отвечать за деобфускацию данных пакета?

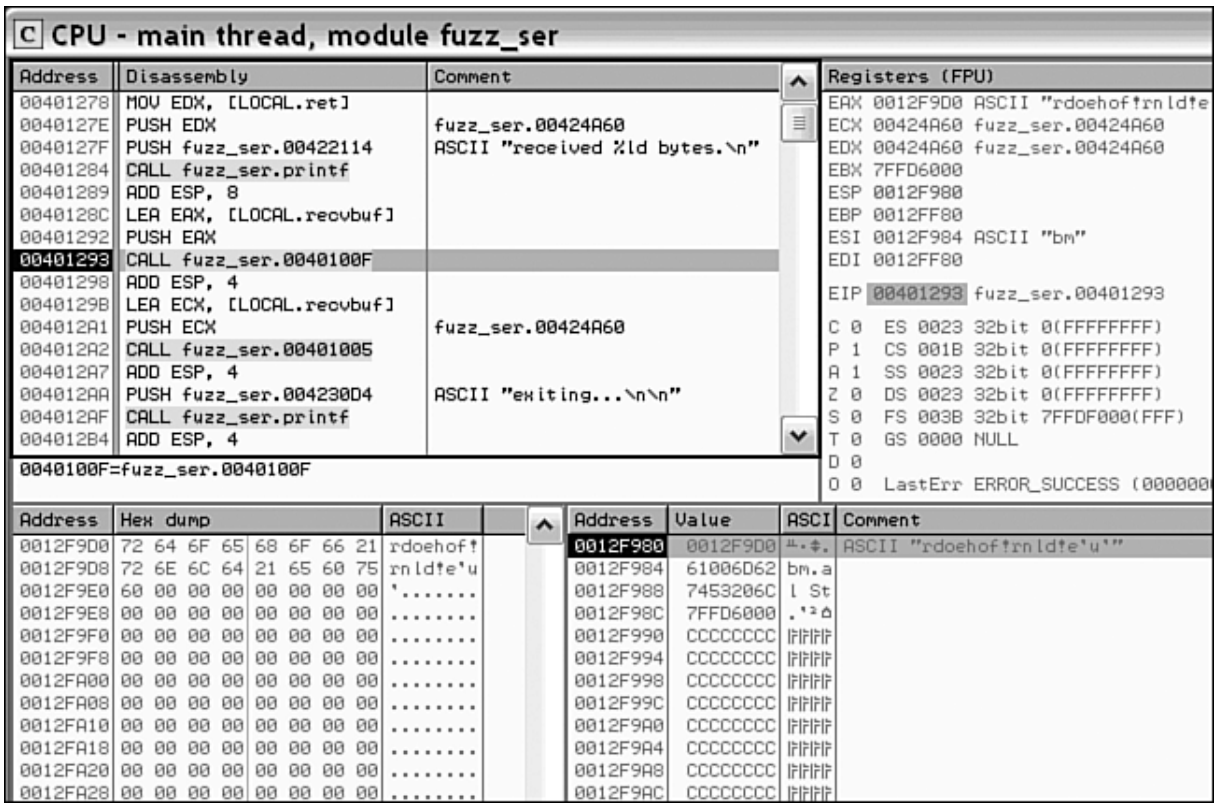


Рисунок 20.9. OllyDbg непосредственно перед demangle

Есть один простой способ это выяснить: позволить ей исполниться и посмотреть, что произойдёт. Снова нажимаем F8, чтобы переступить через вызов. Мы сразу видим, как на рисунке 20.10, что обфусцированные данные в окне дампа преобразились.

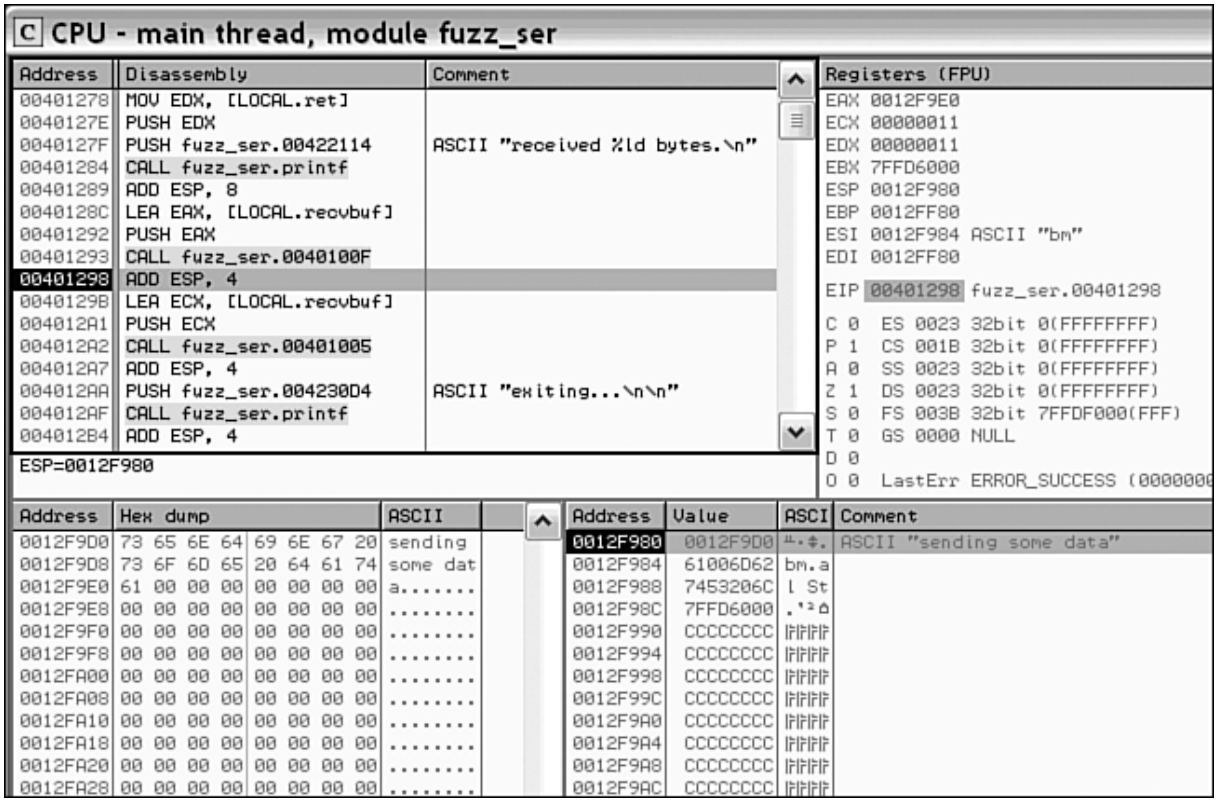


Рисунок 20.10. OllyDbg непосредственно после demangle

Отлично! Итак, мы знаем, что точка снимка должна находиться где-то после этой процедуры. Просматривая код ниже, видим вызов безымянной процедуры по адресу `0x00401005`, а затем вызов `printf()`. Видно, что строка «exiting...» передаётся как аргумент `printf()`, и, вспоминая ранее наблюдавшееся поведение `fuzz_server`, мы знаем, что вскоре он завершится. Процедура по адресу `0x00401005`, должно быть, является нашей процедурой разбора. Вход в неё клавишей F7 показывает безусловный переход по адресу `0x00401450`, истинное начало предполагаемой процедуры разбора, показанное на рисунке 20.11.

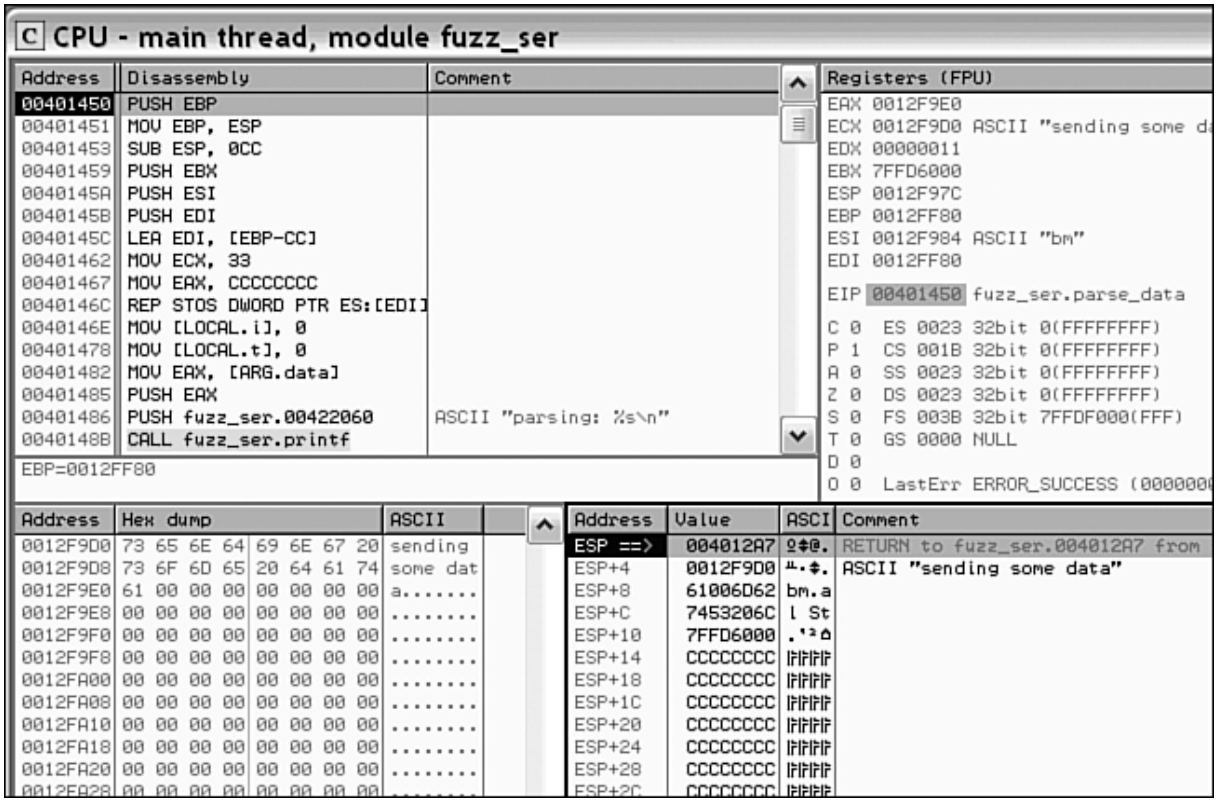


Рисунок 20.11. OllyDbg в начале процедуры parse

Обратите внимание, что деобфусцированная строка «sending some data» появляется как первый аргумент процедуры разбора по адресу `ESP+4`. Это выглядит хорошим местом для подключения снимка. Мы можем сохранить полное состояние процесса в начале процедуры разбора, а затем в некоторой точке после её завершения восстановить состояние процесса. После восстановления процесса можно изменить содержимое разбираемых данных, продолжить исполнение и повторять процесс бесконечно. Но сначала потребуется найти точку восстановления. Нажимаем Ctrl + F9, чтобы выполнить «Execute til return», затем F7 или F8, чтобы попасть по адресу возврата, где, точно как на рисунке 20.12, снова видим вызов `printf()` со строкой «exiting...». Выберем точку восстановления после вызова `printf()`, по адресу `0x004012b7`, чтобы увидеть, как `fuzz_server` выводит «exiting...» до восстановления. Мы делаем это потому, что нам приятно, что `fuzz_server` хочет завершиться, но мы ему не позволяем.

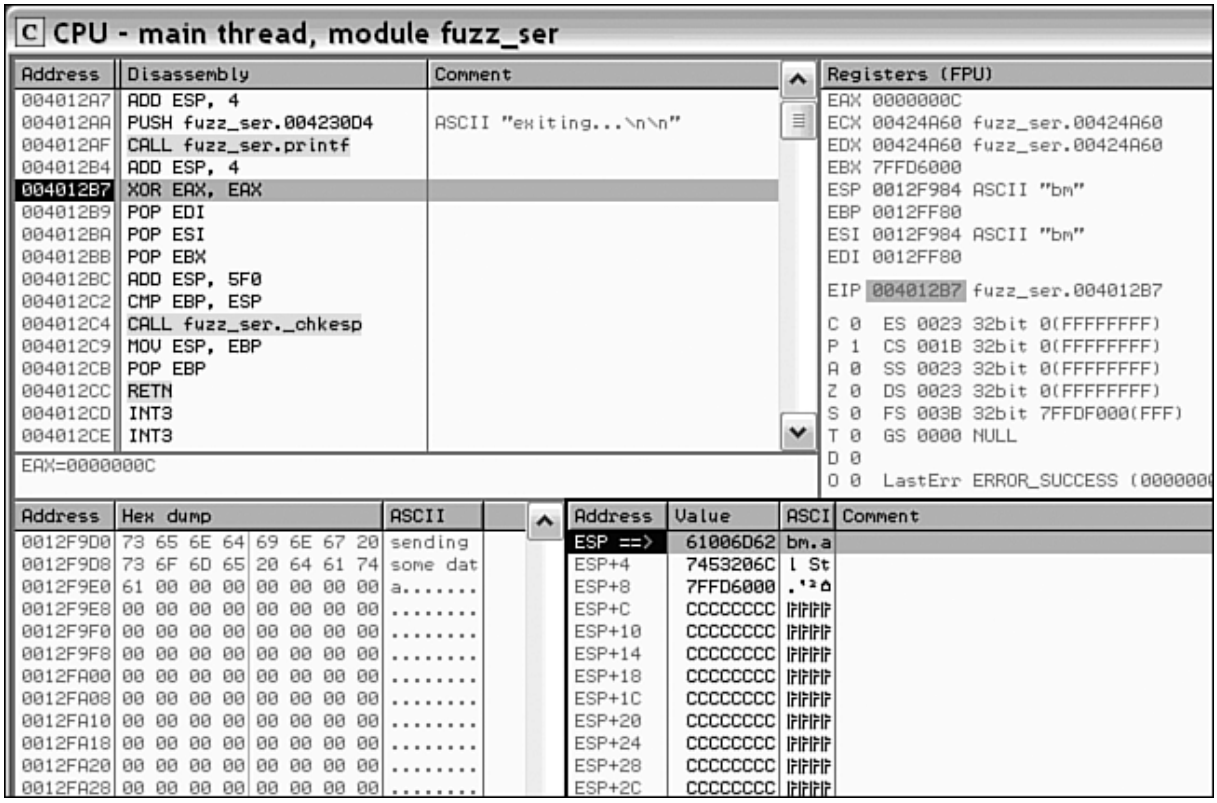


Рисунок 20.12. Точка восстановления OllyDbg

Мы знаем, что процедура по адресу `0x0040100F` отвечает за декодирование. Мы знаем, что процедура по адресу `0x00401450` отвечает за разбор декодированных данных. В качестве точки снимка и мутации мы выбрали начало анализатора. Несколько произвольно в качестве точки восстановления мы выбрали `0x004012b7`, после вызова `printf("exiting...")`. Теперь у нас есть все необходимые части, чтобы приступить к написанию кода, а на рисунке 20.13 представлена концептуальная схема того, что мы собираемся делать.

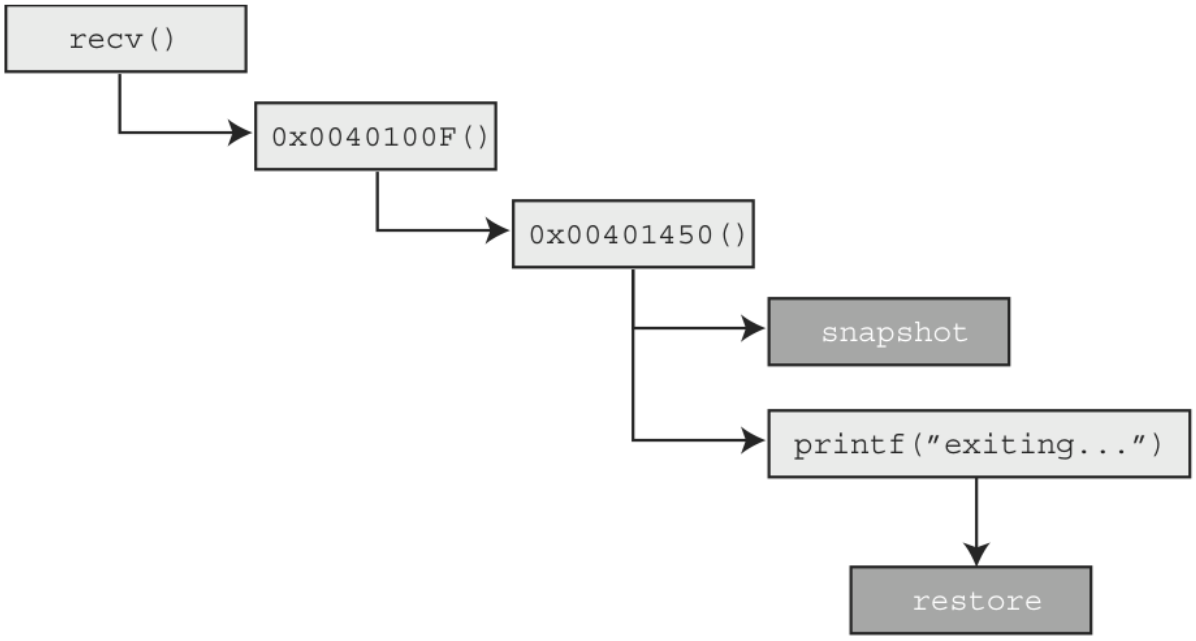


Рисунок 20.13. Концептуальная схема

Реализация фаззера в памяти с помощью PyDbg требует ненамного больше работы, чем мы уже видели. Начинаем с необходимых импортов и определения глобальных переменных для хранения таких сведений, как выбранные точки снимка и восстановления. Затем переходим к стандартному каркасу PyDbg: создаём экземпляр объекта PyDbg, регистрируем обратные вызовы, самую существенную часть, которую определим позднее, находим целевой процесс, присоединяемся к нему, устанавливаем точки останова в точках снимка и восстановления и, наконец, входим в цикл событий отладки:

```
from pydbg import *
from pydbg.defines import *

import time
import random

snapshot_hook  = 0x00401450
restore_hook   = 0x004012B7
snapshot_taken = False
hit_count      = 0
address        = 0

dbg = pydbg()
dbg.set_callback(EXCEPTION_BREAKPOINT,handle_bp)
dbg.set_callback(EXCEPTION_ACCESS_VIOLATION,handle_av)

found_target = False

for (pid, proc_name) in dbg.enumerate_processes():
    if proc_name.lower() == “fuzz_server.exe”:
        found_target = True
        break

if found_target:
    dbg.attach(pid)
    dbg.bp_set(snapshot_hook)
    dbg.bp_set(restore_hook)

    print “entering debug event loop”
    dbg.debug_event_loop()
else:
    print “target not found.”
```

Из двух определённых обратных вызовов обработчик нарушения доступа понять легче, поэтому сначала рассмотрим его. Мы зарегистрировали этот обратный вызов для обнаружения потенциально эксплуатируемой ситуации. Это довольно простой для понимания блок кода, который легко можно перепрофилировать в других приложениях PyDbg. Процедура начинает с получения из записи исключения некоторых полезных сведений, таких как адрес инструкции, вызвавшей исключение, флаг, сообщающий, произошло ли нарушение из-за чтения или записи, и адрес памяти, вызвавший исключение. Предпринимается попытка получить дизассемблированное представление инструкции-нарушителя и вывести сообщение о характере исключения. Наконец, до завершения отлаживаемого процесса предпринимается попытка вывести текущий контекст исполнения цели в момент исключения. Контекст исполнения содержит различные значения регистров, содержимое данных, на которые они указывают, если это указатель, и переменное число разыменований стека, в данном случае мы указали пять. Дополнительные сведения о конкретике API PyDbg см. в подробной документации по адресу http://pedram.redhive.com/PaiMei/docs/PyDbg/.

```
def handle_av (pydbg, dbg, context):
    exception_record  = dbg.u.Exception.ExceptionRecord
    exception_address = exception_record.ExceptionAddress
    write_violation   = exception_record.ExceptionInformation[0]
    violation_address = exception_record.ExceptionInformation[1]

    try:
        disasm = pydbg.disasm(exception_address)
```

```
    except:
        disasm = "[UNRESOLVED]"
        pass

    print "*** ACCESS VIOLATION @%08x %s ***" % \
        (exception_address, disasm)

    if write_violation:
        print "write violation on",
    else:
        print "read violation on",

    print "%08x" % violation_address

    try:
        print pydbg.dump_context(context, 5, False)
    except:
        pass

    print "terminating debuggee"
    pydbg.terminate_process()
```

В качестве альтернативы мы можем перехватывать нарушения доступа в другом отладчике, таком как OllyDbg. Для этого выбранный отладчик должен быть настроен в операционной системе как оперативный отладчик (just-in-time, JIT).[21] Тогда тело обработчика нарушения доступа можно заменить следующим:

```
def handle_av (pydbg, dbg, context):
    pydbg.detach()
    return DBG_CONTINUE
```

Когда нарушение доступа действительно произойдёт, появится знакомое диалоговое окно, показанное на рисунке 20.14.



Рисунок 20.14. Fuzz server аварийно завершается

Нажатие кнопки Debug откроет зарегистрированный JIT-отладчик для более тщательного рассмотрения того, что именно пошло не так внутри цели. Последний компонент нашего фаззера в памяти представляет собой сердце приложения, обработчик точки останова. Обработчик точки останова будет вызываться каждый раз, когда исполнение `fuzz_server` достигает точек снимка и восстановления, на которых мы ранее установили точки останова. Это будет самый запутанный блок кода PyDbg из всех увиденных, поэтому давайте разберём его по частям. В начале функции мы определяем глобальные переменные, к которым будем обращаться, и получаем адрес, по которому произошла точка останова, в переменную `exception_address`.

```
def handle_bp (pydbg, dbg, context):
    global snapshot_hook, restore_hook
```

```
global snapshot_taken, hit_count, address

exception_address = \
    dbg.u.Exception.ExceptionRecord.ExceptionAddress
```

Затем мы проверяем, находимся ли в точке подключения снимка. Если да, увеличиваем переменную `hit_count` и выводим сообщение о текущем расположении.

```
if exception_address == snapshot_hook:
    hit_count += 1
    print “snapshot hook hit #%d\n” % hit_count
```

Далее проверяем логический флаг `snapshot_taken`. Если снимок `fuzz_server` ранее не делался, делаем его сейчас вызовом процедуры PyDbg `process_snapshot()`. В информационных целях оборачиваем операцию таймером и устанавливаем флаг в `True`.

```
    # если снимок процесса ещё не создан,
    # создать его сейчас.
    if not snapshot_taken:
        start = time.time()
        print “taking process snapshot...”,
        pydbg.process_snapshot()
        end = time.time() - start
        print “done. took %.03f seconds\n” % end
        snapshot_taken = True
```

Помните, что мы всё ещё находимся внутри блока `if`, где адрес исключения равен точке подключения. Следующий блок кода занимается мутацией данных. Условная проверка `hit_count` гарантирует, что мы не изменяем аргумент до завершения первой итерации процедуры разбора с исходными данными. Это не обязательное требование. Если был найден ранее выделенный адрес, откуда он берётся, станет ясно через мгновение, мы освобождаем его с помощью удобной обёртки PyDbg `virtual_free()`.

```
    if hit_count >= 1:
        if address:
            print “freeing last chunk at”,
            print “%08x” % address

            pydbg.virtual_free( \
                address,        \
                1000,           \
                MEM_DECOMMIT)
```

Оставаясь внутри блока `if hit_count >= 1`, мы выделяем участок памяти в пространстве процесса `fuzz_server` с помощью удобной обёртки PyDbg `virtual_alloc()`. Именно для этого выделения мы только что видели освобождение. Зачем вообще выделять память? Потому что вместо изменения на месте исходных данных, переданных процедуре разбора, нам проще поместить изменённые данные в другое место пространства процесса `fuzz_server`, а затем просто изменить указатель с исходных данных на наш мутантный блок. Здесь следует отметить одно предостережение: потенциальные повреждения стека могут выглядеть как повреждения кучи, поскольку уязвимый буфер потенциально перемещается из стека.

```
        print “allocating memory for mutation”

        address = pydbg.virtual_alloc( \
            None,                      \
            1000,                      \
            MEM_COMMIT,                \
            PAGE_READWRITE)

        print “allocation at %08x\n” % address
```

Мы предполагаем, что сервер может разбирать только данные ASCII, и используем простой «алгоритм» генерации данных для заполнения выделенного мутантного блока фаззинговыми данными. Начав с длинной строки символов A, выбираем случайный индекс в строке и вставляем

случайный символ ASCII. Всё достаточно просто:

```
print “generating mutant...”,

fuzz = “A” * 750
random_index = random.randint(0, 750)

mutant  = fuzz[0:random_index]
mutant += chr(random.randint(32,126))
mutant += fuzz[random_index:]
mutant += “\x00”

print “done.\n”
```

Далее мы записываем фаззинговые данные в ранее выделенный участок памяти с помощью удобной процедуры PyDbg `write_process_memory()`:

```
print “writing mutant to target memory”
pydbg.write_process_memory(address, mutant)
print
```

Наконец, мы изменяем указатель аргумента функции так, чтобы он указывал на только что выделенный блок памяти. Вспомните из рисунка 20.11, что указатель на буфер, содержащий наши исходные деобфусцированные данные, находится по положительному смещению четыре относительно текущего указателя стека. Затем продолжаем исполнение.

```
print “modifying function argument”

pydbg.write_process_memory( \
    context.Esp + 4,        \
    pydbg.flip_endian(address))

print
print “continuing execution...\n”
```

В оставшейся части определения обработчика обратного вызова точки останова имеется последний блок `if`, обрабатывающий восстановление снимка при достижении точки подключения восстановления. Это выполняется простым вызовом процедуры API PyDbg `process_restore()`. И снова в информационных целях оборачиваем блок таймером:

```
if exception_address == restore_hook:
    start = time.time()
    print “restoring process snapshot...”,
    pydbg.process_restore()
    end = time.time() - start
    print “done. took %.03f seconds\n” % end

    pydbg.bp_set(restore_hook)

return DBG_CONTINUE
```

Вот и всё... мы готовы испытать этого плохиша, поэтому запускаем сервер:

```
$ ./fuzz_server.exe
Listening and waiting for client to connect...
```

Запускаем фаззер в памяти:

```
$ ./chapter_20_srm_poc.py
entering debug event loop
```

И запускаем клиент:

```
$ ./fuzz_client.exe 192.168.197.1 ‘sending some data’
connecting....
sending...
sent...
```

Как только клиент передаёт полезную нагрузку, достигается точка подключения снимка и наш фаззер в памяти вступает в действие:

```
snapshot / mutate hook point hit #1
taking process snapshot... done. took 0.015 seconds
```

Создаётся снимок, после чего исполнение продолжается. Разбор завершается, `fuzz_server` выводит свои сообщения и затем хочет завершиться:

```
received 17 bytes.
parsing: sending some data
exiting...
```

Однако прежде чем он успевает завершиться, достигается точка подключения восстановления и наш фаззер в памяти снова вступает в действие:

```
restoring process snapshot... done. took 0.000 seconds
```

Состояние процесса `fuzz_server` успешно перемотано назад до точки снимка, и исполнение продолжается. На этот раз, поскольку `hit_count` теперь больше 1, выполняется блок мутации внутри обработчика точки останова:

```
snapshot / mutate hook point hit #2
allocating chunk of memory to hold mutation
memory allocated at 003c0000
generating mutant... done.
writing mutant into target memory space
modifying function argument to point to mutant
continuing execution...
```

Память была выделена, фаззинговый мутант создан и записан в `fuzz_server`, а указатель аргумента изменён. Когда исполнение продолжается, выходные сообщения `fuzz_server` подтверждают успех нашей первой мутации с восстановлением снимка:

```
parsing:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA)AAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

На 265-й итерации нашего фаззера SRM для подтверждения концепции в примерной цели была обнаружена явно эксплуатируемая уязвимость. Нарушение доступа по адресу `41414141` означает, что процесс пытался прочитать и исполнить инструкцию с виртуального адреса `0x41414141`, но не смог, поскольку эта область памяти недоступна для чтения. `0x41`, если вы ещё не догадались, является шестнадцатеричным представлением символа ASCII A. Данные нашего фаззера вызвали переполнение и перезаписали адрес возврата в стеке. Когда функция, где произошло переполнение, вернула исполнение вызывающей стороне, возникло нарушение доступа, которое обнаружил наш фаззер. Злоумышленники могут легко эксплуатировать эту уязвимость, но не забывайте: для атаки на сервер процедуру обфускации придётся расшифровать, поскольку в реальном мире вы, скорее всего, не сможете просто приказать клиенту отправить произвольные данные. Анализ исходного или двоичного кода для выявления точной природы и причины этой уязвимости оставлен читателю в качестве упражнения.

Итоги

Мы прошли долгий путь, обсуждая преимущественно теоретический и новый подход к фаззингу. Теория интересна, и разбор применения теории стоит прочтения. Однако мы рекомендуем читателям загрузить примерные файлы и самостоятельно выполнить примерный тест. Неинтерактивная среда книги не воздаёт этому примеру должного.

Надеемся, содержимое последних двух глав навело вас на размышления и, возможно, даже когда-нибудь окажется применимым к конкретной проблеме, с которой вы столкнётесь. Платформа PyDbg и примерные приложения имеют открытый исходный код, с ними можно свободно экспериментировать. Просто загрузите их с http://www.fuzzing.org и наслаждайтесь. Как и в случае всех наших проектов, мы надеемся, что усовершенствования, сообщения об ошибках, исправления и примеры использования будут передаваться нам, помогая сохранять инструменты свежими и динамичными.

Хотя в этой главе мы немного отклонились от главной темы книги, полученные в ней знания об отладке помогут вам в целом при попытках автоматизировать обнаружение исключений. Мы вернёмся к этой теме подробнее в главе 24, где обсудим расширенное обнаружение исключений.

Сноски

[1] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/context_str.asp
[2] [назад] http://starship.python.net/crew/theller/ctypes/
[3] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/debugging_functions.asp
[4] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/debugging_events.asp
[5] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/debugging_structures.asp
[6] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dllproc/base/createprocess.asp
[7] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/debugsetprocesskillonexit.asp
[8] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/waitfordebugevent.asp
[9] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/exception_record_str.asp
[10] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/getthreadcontext.asp
[11] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/setthreadcontext.asp
[12] [назад] http://www.vmware.com
[13] [назад] Greg Hoglund, *Runtime Decompilation*, BlackHat Proceedings
[14] [назад] http://msdn2.microsoft.com/en-us/library/ms686832.aspx
[15] [назад] http://msdn2.microsoft.com/en-us/library/aa366907.aspx
[16] [назад] http://msdn2.microsoft.com/en-us/library/ms680553.aspx
[17] [назад] http://openrce.org/downloads/details/208/PaiMei
[18] [назад] http://msdn2.microsoft.com/en-us/library/ms686832.aspx
[19] [назад] http://www.ethereal.com/, Ethereal: A Network Protocol Analyzer.
[20] [назад] http://www.ollydbg.de
[21] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/debugging_terminology.asp

Часть III. Продвинутые технологии фаззинга

Глава 21. Каркасы фаззинга

> «В Теннесси есть старая поговорка, я знаю, она есть в Техасе, вероятно, и в Теннесси, которая гласит: обмани меня один раз, позор... позор тебе. Обмани меня... тебя снова не обмануть».
>
> Джордж У. Буш, Нашвилл, штат Теннесси, 17 сентября 2002 года

Существует ряд доступных специализированных утилит фаззинга, нацеленных на многие распространённые и документированные сетевые протоколы и форматы файлов. Эти фаззеры исчерпывающе перебирают заданный протокол и могут повсеместно использоваться для стрессового тестирования разнообразных приложений, поддерживающих этот протокол. Например, один и тот же специализированный фаззер SMTP можно было бы использовать против различных программ передачи электронной почты, таких как Microsoft Exchange, Sendmail, qmail и так далее. Другие «глупые» фаззеры применяют более общий подход, допускающий фаззинг произвольных протоколов и форматов файлов, и выполняют простые мутации без учёта протокола, такие как инверсия битов и перестановка байтов.

Хотя эти фаззеры эффективны против широкого спектра распространённых приложений, нам часто требуются более индивидуализированный и тщательный фаззинг проприетарных и ранее не тестировавшихся протоколов. Именно здесь каркасы фаззинга становятся чрезвычайно полезными.

В этой главе мы исследуем ряд доступных сегодня каркасов фаззинга с открытым исходным кодом, включая SPIKE, неизменно популярный каркас, название которого стало общеизвестным, в зависимости от степени увлечения технологиями в вашем доме. Мы также рассмотрим некоторых захватывающих новичков в этой области, таких как Autodafé и GPF. После разбора существующих технологий мы увидим, почему, несмотря на мощь многих каркасов фаззинга общего назначения, время от времени нам всё же потребуется создавать фаззер с нуля.

Позднее мы проиллюстрируем это положение реальным примером проблемы фаззинга и разработкой решения. Наконец, мы представим новый каркас, разработанный авторами, и исследуем достигнутые в этой работе усовершенствования.

Что такое каркас фаззинга?

Некоторые доступные сегодня каркасы фаззинга разработаны на C, другие на Python или Ruby. Одни предлагают функциональность на своём родном языке, тогда как другие используют специализированный язык. Например, каркас фаззинга Peach предоставляет конструкции на Python, а dfuz реализует собственный набор объектов фаззинга, оба этих каркаса подробнее обсуждаются далее в главе. Одни абстрагируют генерацию данных, другие нет. Одни объектно-ориентированы и хорошо документированы, другими по большей части способен пользоваться лишь их создатель. Однако общая цель всех каркасов фаззинга одинакова: предоставить разработчикам фаззеров быструю, гибкую, повторно используемую и однородную среду разработки.

Хороший каркас фаззинга должен абстрагировать и сводить к минимуму ряд утомительных задач. Для помощи на первых этапах моделирования протокола некоторые каркасы включают утилиты преобразования перехваченного сетевого трафика в формат, понятный каркасу. Это позволяет исследователю импортировать крупные фрагменты эмпирических данных и сосредоточить усилия на более подходящей человеку задаче, такой как определение границ полей протокола.

Автоматическое вычисление длины является безусловной необходимостью для полноценного каркаса. Многие протоколы реализованы с использованием синтаксиса в стиле TLV (Type, Length, Value), сходного со стандартом ASN.1.[1] Рассмотрим пример, в котором первый байт передаваемых данных определяет тип следующих данных: `0x01` для обычного текста и `0x02` для необработанных двоичных данных. Следующие два байта определяют длину следующих за ними данных. Наконец, оставшиеся байты определяют значение, то есть конкретные данные сообщения, как показано здесь:



Пример структуры Type, Length, Value.

При фаззинге поля Value этого протокола необходимо вычислять и обновлять двухбайтное поле длины в каждом тестовом случае. В противном случае мы рискуем тем, что тестовые случаи будут немедленно отброшены при обнаружении нарушения спецификаций протокола.

Вычисление циклического избыточного кода (Cyclic Redundancy Check, CRC)[2] и других алгоритмов контрольных сумм является ещё одной задачей, которую должен включать полезный каркас.

Значения CRC часто встраиваются и в спецификации файлов, и в спецификации протоколов для выявления потенциально повреждённых данных. Например, файлы изображений PNG используют значения CRC, позволяющие программам не обрабатывать изображение, если полученное значение CRC не совпадает с вычисленным. Хотя это важная возможность для безопасности и функциональности, она воспрепятствует фаззингу, если CRC не обновлять правильно при мутации протокола. В качестве более экстремального примера рассмотрим спецификацию Distributed Network Protocol (DNP3),[3] используемую во взаимодействиях систем диспетчерского управления и сбора данных (SCADA). Потоки данных по отдельности нарезаются на фрагменты по 250 байт, и перед каждым фрагментом помещается контрольная сумма CRC-16! Наконец, учтите, что IP-адреса клиента, сервера или обоих часто встречаются внутри передаваемых данных и что оба адреса могут часто меняться в ходе фаззингового теста. Было бы удобно, если бы каркас предлагал метод автоматического определения и включения этих значений в создаваемые фаззинговые тестовые случаи.

Большинство, если не все, каркасы предоставляют методы генерации псевдослучайных данных. Хороший каркас сделает дополнительный шаг, включив серьёзный список эвристик атаки. Эвристика атаки является не более чем сохранённой последовательностью данных, которая, как ранее было известно, вызывала дефект программы. Последовательности строки формата (%n%n%n%n) и обхода каталогов (`../../../`) являются распространёнными примерами простых эвристик атаки. Циклический перебор конечного списка таких тестовых случаев до перехода к генерации случайных данных во многих сценариях сэкономит время и является стоящей инвестицией.

Обнаружение дефектов играет важную роль в фаззинге и подробно обсуждается в главе 24, «Интеллектуальное обнаружение дефектов». На самом простом уровне фаззер может обнаружить возможный отказ цели, если она не способна принять новое соединение. Более развитое обнаружение дефектов обычно реализуется с помощью отладчика. Продвинутый каркас фаззинга должен позволять фаззеру напрямую взаимодействовать с отладчиком, присоединённым к

целевому приложению, либо даже включать специализированную технологию отладчика целиком.

В зависимости от ваших личных предпочтений может существовать длинный перечень мелких возможностей, способных значительно улучшить опыт разработки фаззера. Например, некоторые каркасы включают поддержку разбора широкого спектра форматированных данных. При копировании и вставке необработанных байтов в сценарий фаззинга, например, было бы удобно иметь возможность вставлять шестнадцатеричные байты в любом из следующих форматов: `0x41 0x42`, `\x41 \x42`, `4142` и так далее.

Метрикам фаззинга, см. главу 23, «Отслеживание фаззера», до настоящего времени также уделялось мало внимания. Продвинутый каркас фаззинга может включать интерфейс для взаимодействия с инструментом сбора метрик, таким как монитор покрытия кода.

Наконец, идеальный каркас фаззинга предоставит средства, максимально повышающие повторное использование кода, делая разработанные компоненты легкодоступными для будущих проектов. При правильной реализации эта концепция позволяет фаззеру развиваться и становиться «умнее» по мере использования.

Помните об этих понятиях, пока мы исследуем ряд каркасов фаззинга, прежде чем углубиться в проектирование и создание как специализированного для задачи, так и универсального специализированного каркаса.

Существующие каркасы

В этом разделе мы разбираем ряд каркасов фаззинга, чтобы понять, что уже существует. Мы не охватываем каждый доступный каркас фаззинга, а исследуем выборку каркасов, представляющих диапазон различных методологий. Следующие каркасы перечислены в общем порядке зрелости и богатства возможностей, начиная с самого примитивного.

Antiparser[4]

Написанный на Python antiparser представляет собой API, предназначенный для помощи в создании случайных данных специально для построения фаззеров. Этот каркас можно использовать для разработки фаззеров, работающих на нескольких платформах, поскольку он зависит исключительно от наличия интерпретатора Python. Использование каркаса довольно прямолинейно. Сначала необходимо создать экземпляр класса `antiparser`, этот класс служит контейнером. Далее antiparser предоставляет ряд типов фаззинга, экземпляры которых можно создавать и добавлять в контейнер. К доступным типам фаззинга относятся следующие:

• `apChar()`: восьмибитный символ C.
• `apCString()`: строка в стиле C, то есть массив символов, завершающийся нулевым байтом.
• `apKeywords()`: список значений, каждое из которых связано с разделителем, блоком данных и ограничителем.
• `apLong()`: 32-битное целое C.
• `apShort()`: 16-битное целое C.
• `apString()`: строка свободной формы.

Из доступных типов данных наиболее интересен `apKeywords()`. С помощью этого класса вы определяете список ключевых слов, блок данных, разделитель между ключевыми словами и данными и, наконец, необязательный ограничитель блока данных. Класс создаёт данные в формате `[ключевое слово] [разделитель] [блок данных] [ограничитель]`.

antiparser распространяет пример сценария evilftpclient.py, использующий тип данных apKeywords(). Давайте исследуем части сценария, чтобы лучше понять, как выглядит разработка на этом каркасе. Следующая выдержка кода Python показывает относящиеся к делу части evilftpclient.py, отвечающие за тестирование демона FTP на уязвимости строки формата при разборе аргументов команд FTP. Например, в этой выдержке не показана функциональность аутентификации с целевым демоном FTP. Полный листинг см. в исходном коде.

```
from antiparser import *

CMDLIST = [‘ABOR’, ‘ALLO’, ‘APPE’, ‘CDUP’, ‘XCUP’, ‘CWD’,
           ‘XCWD’, ‘DELE’, ‘HELP’, ‘LIST’, ‘MKD’,  ‘XMKD’,
           ‘MACB’, ‘MODE’, ‘MTMD’, ‘NLST’, ‘NOOP’, ‘PASS’,
           ‘PASV’, ‘PORT’, ‘PWD’,  ‘XPWD’, ‘QUIT’, ‘REIN’,
           ‘RETR’, ‘RMD’,  ‘XRMD’, ‘REST’, ‘RNFR’, ‘RNTO’,
           ‘SITE’, ‘SIZE’, ‘STAT’, ‘STOR’, ‘STRU’, ‘STOU’,
           ‘SYST’, ‘TYPE’, ‘USER’]

SEPARATOR  = “ ”
TERMINATOR = “\r\n”

for cmd in CMDLIST:
    ap = antiparser()

    cmdkw = apKeywords()
    cmdkw.setKeywords([cmd])
    cmdkw.setSeparator(SEPARATOR)
    cmdkw.setTerminator(TERMINATOR)
    cmdkw.setContent(r”%n%n%n%n%n%n%n%n%n%n%n%n%n%n”)
    cmdkw.setMode(‘incremental’)
    cmdkw.setMaxSize(65536)

    ap.append(cmdkw)

    sock = apSocket()
    sock.connect(HOST, PORT)

    # вывести баннер FTP-демона
    print sock.recv(1024)

    # отправить фаззинговый тест
    sock.sendTCP(ap.getPayload())

    # вывести ответ FTP-демона
    print sock.recv(1024)

    sock.close()
```

Код начинается с импорта всех доступных типов данных и классов из каркаса antiparser, определения списка команд FTP, символа-разделителя аргумента команды, пробела, и последовательности завершения команды, возврата каретки, за которым следует новая строка. Затем каждая перечисленная команда FTP перебирается и тестируется отдельно, начиная с создания экземпляра нового класса-контейнера antiparser. Далее вступает в действие тип данных apKeywords(). В качестве ключевых слов, в данном случае ключевого слова, указывается список, определяющий один элемент, текущую тестируемую команду. Затем определяются соответствующие символы разделения команды и аргумента и завершения команды. Содержимое данных созданного объекта apKeyword() устанавливается в последовательность токенов строки формата. Если целевой сервер FTP предоставляет уязвимость строки формата при разборе аргументов команд, это содержимое данных наверняка её вызовет.

Следующие два вызова, setMode(‘incremental’) и setMaxSize(65536), указывают, что при перестановке блок данных должен последовательно расти до максимального значения 65 336. Однако в данном конкретном случае эти два вызова несущественны, поскольку фаззер не перебирает множество тестовых случаев или перестановок вызовом ap.permute(). Вместо этого каждая команда тестируется с единственным блоком данных.

Оставшиеся строки кода в основном говорят сами за себя. Единственный тип данных, `apKeywords()`, добавляется в контейнер antiparser, и создаётся сокет. После установления соединения тестовый случай создаётся вызовом `ap.getPayload()` и передаётся посредством `sock.sendTCP()`.

Очевидно, что antiparser имеет ряд ограничений. Примерный фаззер FTP можно легко воспроизвести на чистом Python без помощи каркаса. Соотношение кода, специфичного для каркаса, и обобщённого кода при разработке фаззеров на antiparser значительно ниже, чем у других каркасов. В каркасе также отсутствуют многие желательные средства автоматизации, перечисленные в предыдущем разделе, например возможность автоматически вычислять и представлять распространённый формат протокола TLV. В качестве последнего замечания, документация этого каркаса скудна и, к сожалению, по состоянию на версию 2.0, выпущенную в августе 2005 года, доступен лишь один пример. Хотя этот каркас прост и предоставляет некоторые преимущества при создании простых фаззеров, для выполнения сложных задач он непригоден.

Dfuz[5]

Написанный на C Диего Бауче, Dfuz является активно сопровождаемым и часто обновляемым каркасом фаззинга. Этот каркас использовался для выявления различных уязвимостей, затрагивающих таких поставщиков, как Microsoft, Ipswitch и RealNetworks. Исходный код открыт и доступен для загрузки, но ограничительная лицензия запрещает любое дублирование или изменение без явного разрешения автора. В зависимости от ваших потребностей эта ограниченная лицензия может отвратить вас от использования каркаса. Мотивирующим фактором такой жёсткой лицензии, по-видимому, является недовольство автора качеством собственного кода, согласно файлу README. Возможно, стоит попробовать связаться с ним напрямую, если вы хотите повторно использовать части его кода. Dfuz был спроектирован для работы в UNIX/Linux и предоставляет специализированный язык для разработки новых фаззеров. Этот каркас не является самым продвинутым из разобранных в данной главе. Однако простой и интуитивный проект делает его хорошим практическим примером проектирования каркаса, поэтому давайте исследуем его подробно.

К основным компонентам Dfuz относятся данные, функции, списки, параметры, протоколы и переменные. Эти различные компоненты используются для определения набора правил, которые механизм фаззинга позднее может разобрать для создания и передачи данных. Для определения переменных внутри файлов правил используется следующий знакомый и простой синтаксис:

```
var my_variable = my_data
var ref_other   = “1234”,$my_variable,0x00
```

Переменные определяются простым префиксом `var`, а ссылаться на них из других мест можно, добавляя к имени переменной знак доллара, `$`, как в Perl или PHP. Создание фаззера целиком самодостаточно. Это означает, что, в отличие, например, от antiparser, построение фаззера поверх этого каркаса выполняется полностью на его специализированном языке сценариев.

Dfuz определяет различные функции для выполнения часто необходимых задач. Функции легко распознать, поскольку перед их именами стоит знак процента (%). Среди определённых функций находятся следующие:

• `%attach()`: ожидать нажатия клавиши Enter или Return. Это полезно для приостановки фаззера ради выполнения другой задачи. Например, если ваша цель фаззинга порождает новый поток для обработки входящих соединений и вы хотите присоединить отладчик к этому новому потоку, вставьте вызов `%attach()` после установления первоначального соединения, затем найдите целевой поток и присоединитесь к нему.

• %length() или %calc(): вычислить и вставить размер переданного аргумента в двоичном формате. Например, %length(“AAAA”) вставит в двоичный поток двоичное значение 0x04. Размер вывода по умолчанию для этих функций равен 32 битам, но его можно изменить на 8 бит вызовом %length:uint8() или на 16 бит вызовом %length:uint16().
• %put:<size>(number): вставить указанное число в двоичный поток. Размер можно указать как byte, word или dword, являющиеся псевдонимами uint8, uint16 и uint32 соответственно.
• %random:<size>(): создать и вставить случайное двоичное значение указанного размера. Подобно %put(), размер можно указать как byte, word или dword, являющиеся псевдонимами uint8, uint16 и uint32 соответственно.
• %random:data(<length>,<type>): создать и вставить случайные данные. length указывает число создаваемых байтов. type указывает вид создаваемых случайных данных и может иметь значение ascii, alphanum, plain или graph.
• %dec2str(num): преобразовать десятичное число в строку и вставить её в двоичный поток. Например, %dec2str(123) создаёт 123.
• %fry(): случайным образом изменить ранее определённые данные. Например, правило “AAAA”,%fry() приведёт к замене случайного числа символов строки “AAAA” случайным байтовым значением.
• %str2bin(): разбирает различные шестнадцатеричные представления строк в их необработанное значение. Например, 4141, 41 41 и 41-41 будут преобразованы в AA.

Данные можно представлять несколькими способами. Специализированный язык сценариев поддерживает синтаксис для указания строк, необработанных байтов, адресов, повторений данных и простых циклов данных. Несколько определений данных можно связать в простой список посредством разделителя-запятой. Следующие примеры демонстрируют различные способы определения данных, дополнительные сведения см. в документации:

```
var my_variable1 = “a string”
var my_variable2 = 0x41,|0xdeadbeef|,[Px50],[\x41*200],100
```

Списки объявляются ключевым словом list, за которым следуют имя списка, ключевое слово begin, список значений данных, разделённых новыми строками, и, наконец, завершающее ключевое слово end. Список можно использовать для определения и индексирования последовательности данных. Например:

```
list my_list:
begin
some_data
more_data
even_more_data
end
```

Подобно переменным, на списки можно ссылаться из других мест, добавляя к имени списка знак доллара ($). Синтаксисом, сходным с другими языками сценариев, такими как Perl и PHP, элемент списка можно индексировать посредством квадратных скобок: $my_list[1]. Поддерживаются также случайные индексы внутри списка посредством ключевого слова rand: $my_list[rand].

Существует ряд параметров управления поведением всего механизма, включая следующие:

• keep_connecting: продолжать фаззинг цели, даже если соединение установить невозможно.
• big_endian: изменить порядок байтов при генерации данных на от старшего к младшему, по умолчанию используется порядок от младшего к старшему.
• little_endian: изменить порядок байтов при генерации данных на от младшего к старшему, он используется по умолчанию.
• tcp: указать, что соединения сокетов следует устанавливать по TCP.
• udp: указать, что соединения сокетов следует устанавливать по UDP.
• client_side: указать, что механизм будет фаззить сервер и тем самым действовать как клиент.

• server_side: указать, что механизм будет фаззить клиент и тем самым действовать как сервер, ожидающий соединения.
• use_stdout: создавать данные в стандартный вывод, консоль, а не подключённому через сокет партнёру. Этот параметр необходимо сочетать со значением узла “stdout”.

Чтобы облегчить бремя воспроизведения часто фаззимых протоколов, Dfuz умеет эмулировать протоколы FTP, POP3, Telnet и Server Message Block (SMB). Эта функциональность предоставляется посредством таких функций, как ftp:user(), ftp:pass(), ftp:mkd(), pop3:user(), pop3:pass(), pop3:dele(), telnet:user(), telnet:pass(), smb:setup() и так далее. Полный список см. в документации Dfuz.

Эти основные компоненты необходимо объединить с некоторыми дополнительными директивами для создания файлов правил. В качестве простого, но полного примера рассмотрим следующий файл правил, входящий в комплект каркаса, для фаззинга сервера FTP:

```
port=21/tcp

peer write: @ftp:user(“user”)
peer read
peer write: @ftp:pass(“pass”)
peer read
peer write: “CWD /”, %random:data(1024,alphanum), 0x0a
peer read
peer write: @ftp:quit()
peer read

repeat=1024
wait=1

# Без параметров
```

Первая директива указывает, что механизм должен подключиться по TCP-порту 21. Поскольку параметры не указаны, по умолчанию он ведёт себя как клиент. Директивы peer read и peer write указывают механизму, когда соответственно следует читать данные из цели фаззинга и записывать их в неё. В данном конкретном файле правил функциональность протокола FTP используется для аутентификации с целевым сервером FTP. Далее команда Change Working Directory (CWD) вручную строится и передаётся серверу. Команде CWD передаётся 1024 случайных байта буквенно-цифровых данных, за которыми следует завершающая новая строка (0x0a). Наконец, соединение закрывается. Последняя директива repeat указывает, что блок чтения и записи партнёра следует выполнить 1024 раза. Для каждого тестового случая Dfuz установит аутентифицированное соединение с сервером FTP, выдаст команду CWD со случайной буквенно-цифровой строкой длиной 1024 байта в качестве аргумента и разорвёт соединение.

Dfuz является простым и мощным каркасом фаззинга, который можно использовать для воспроизведения и фаззинга многих протоколов и форматов файлов. Сочетание поддержки stdout, стандартного вывода, с некоторыми простыми сценариями командной строки также может превратить этот каркас в фаззер форматов файлов, переменных среды или аргументов командной строки. Dfuz сравнительно легко освоить, а разработка на нём быстра. Тот факт, что разработка фаззеров целиком выполняется на его собственном языке сценариев, является обоюдоострым мечом. Положительная сторона состоит в том, что непрограммисты могут описывать и фаззить протоколы на этом каркасе, отрицательная в том, что опытные программисты не могут использовать врождённую мощь и возможности зрелого языка программирования. Dfuz способствует некоторому повторному использованию кода, но далеко не в такой степени, как другие каркасы, например Peach. Важной возможностью, которой сейчас не хватает, является наличие осмысленного набора эвристик атаки. В целом Dfuz является интересным практическим примером хорошо спроектированного каркаса фаззинга и хорошим инструментом, который стоит держать про запас.

SPIKE[6]

SPIKE, написанный Дейвом Айтелом, вероятно, является самым широко используемым и известным каркасом фаззинга. SPIKE написан на C и предоставляет API для быстрой и эффективной разработки фаззеров сетевых протоколов. SPIKE имеет открытый исходный код и выпущен под гибкой лицензией GNU General Public License (GPL).[7] Такое благоприятное лицензирование позволило создать SPIKEfile, перепрофилированную версию каркаса, предназначенную специально для фаззинга форматов файлов, см. главу 12, «Фаззинг форматов файлов: автоматизация в UNIX». SPIKE использует новый метод представления и последующего фаззинга сетевых протоколов. Структуры данных протокола разбиваются и представляются в виде блоков, также называемых SPIKE, содержащих и двоичные данные, и размер блока. Блочное представление протокола позволяет абстрагированно строить различные уровни протокола с автоматическим вычислением размеров.

Чтобы лучше понять блочную концепцию, рассмотрим следующий простой пример из технического документа «The Advantages of Block-Based Protocol Analysis for Security Testing»:[8]

```
s_block_size_binary_bigendian_word(“somepacketdata”);
s_block_start(“somepacketdata”)
s_binary(“01020304”);
s_block_end(“somepacketdata”);
```

Этот базовый сценарий SPIKE, сценарии SPIKE пишутся на C, определяет блок с именем `somepacketdata`, помещает в него четыре байта `0x01020304` и добавляет перед блоком его длину. В данном случае длина блока будет вычислена как 4 и сохранена как слово в порядке от старшего байта к младшему. Обратите внимание, что перед большей частью API SPIKE стоит префикс `s_` или `spike_`. API `s_binary()` используется для добавления двоичных данных в блок и весьма либерален к формату аргумента, позволяя обрабатывать широкий спектр скопированных и вставленных входных данных, таких как строка `4141 \x41 0x41 41 00 41 00`. Несмотря на простоту, этот пример демонстрирует основы и общий подход к построению SPIKE. Поскольку SPIKE позволяет встраивать блоки внутрь других блоков, произвольно сложные протоколы легко разбиваются на мельчайшие атомы. Расширим предыдущий пример:

```
s_block_size_binary_bigendian_word(“somepacketdata”);
s_block_start(“somepacketdata”)
s_binary(“01020304”);
s_blocksize_halfword_bigendian(“innerdata”);
s_block_start(“innerdata”);
s_binary(“00 01”);
s_binary_bigendian_word_variable(0x02);
s_string_variable(“SELECT”);
s_block_end(“innerdata”);
s_block_end(“somepacketdata”);
```

В этом примере определены два блока, `somepacketdata` и `innerdata`. Второй содержится внутри первого, и перед каждым отдельным блоком помещено значение размера. Новый определённый блок `innerdata` начинается со статического двухбайтного значения (`0x0001`), за которым следуют четырёхбайтное переменное целое со значением по умолчанию `0x02` и, наконец, строковая переменная со значением по умолчанию `SELECT`. API `s_binary_bigendian_word_variable()` и `s_string_variable()` будут перебирать предопределённые наборы соответственно целых и строковых переменных, эвристик атаки, которые, как ранее было известно, выявляли уязвимости безопасности. SPIKE начнёт с перебора возможных мутаций переменной-слова, после чего перейдёт к мутации строковой переменной. Истинная мощь этого каркаса состоит в том, что SPIKE автоматически обновляет значения каждого поля размера по мере выполнения различных мутаций. Чтобы исследовать или расширить текущий список фаззинговых переменных, загляните в `SPIKE/src/spike.c`. Версия 2.9 каркаса содержит список почти из 700 эвристик, вызывающих ошибки.

Используя базовые понятия, продемонстрированные в предыдущем примере, вы начинаете видеть, как в этом каркасе можно моделировать произвольно сложные протоколы. Существует ряд дополнительных API и примеров. Дополнительные сведения см. в документации SPIKE. Придерживаясь нашего постоянного примера, следующая выдержка кода взята из фаззера FTP, распространяемого со SPIKE. Это не лучшая демонстрация возможностей SPIKE, поскольку блоки фактически не определяются, но она помогает сравнивать подобное с подобным.

```
s_string("HOST ");
s_string_variable("10.20.30.40");
s_string("\r\n");
s_string_variable("USER");
s_string(" v);
s_string_variable("bob");
s_string("\r\n");
s_string("PASS ");
s_string_variable("bob");
s_string("\r\n");

s_string("SITE ");
s_string_variable("SEDV");
s_string("\r\n");
s_string("ACCT ");
s_string_variable("bob");
s_string("\r\n");
s_string("CWD ");
s_string_variable(".");
s_string("\r\n");
s_string("SMNT ");
s_string_variable(".");
s_string("\r\n");
s_string("PORT ");
s_string_variable("1");
s_string(",");
s_string_variable("2");
s_string(",");
s_string_variable("3");
s_string(",");
s_string_variable("4");
s_string(",");
s_string_variable("5");
s_string(",");
s_string_variable("6");
s_string("\r\n");
```

Документация SPIKE появляется от случая к случаю, а распространяемый пакет содержит множество устаревших компонентов, способных вызвать путаницу. Однако доступен ряд рабочих примеров, служащих превосходными справочными материалами для знакомства с этим мощным каркасом фаззинга. Отсутствие полной документации и неорганизованность дистрибутивного пакета побудили некоторых исследователей предположить, что SPIKE намеренно сломан в ряде областей, чтобы помешать другим обнаружить уязвимости, в частном порядке найденные автором. Истинность этого утверждения остаётся непроверенной.

В зависимости от ваших индивидуальных потребностей одним из главных недостатков каркаса SPIKE является отсутствие поддержки Microsoft Windows, поскольку SPIKE был предназначен для работы в среде UNIX, хотя существуют противоречивые сообщения о запуске SPIKE на платформе Windows через Cygwin.[9] Другим фактором, который следует учесть, является то, что даже незначительные изменения каркаса, такие как добавление новых фаззинговых строк, требуют повторной компиляции. В качестве последнего негативного замечания, повторное использование кода между разработанными фаззерами требует ручного копирования и вставки. Новые элементы, такие как фаззер адресов электронной почты, невозможно просто определить, а позднее глобально использовать по ссылке во всём каркасе.

В целом SPIKE доказал свою эффективность и использовался как автором, так и другими людьми для обнаружения множества широко известных уязвимостей. SPIKE также включает такие утилиты, как прокси, позволяющий исследователю отслеживать и фаззить взаимодействия между браузером и веб-приложением. Способность SPIKE вызывать дефекты сыграла значительную роль в утверждении ценности фаззинга в целом. Блочный подход к фаззингу приобрёл популярность, что видно из принятия этого метода рядом каркасов фаззинга после первоначального открытого выпуска SPIKE.

Peach[10]

Peach, выпущенный IOACTIVE, является кроссплатформенным каркасом фаззинга, написанным на Python и впервые выпущенным в 2004 году. Peach имеет открытый исходный код и открытую лицензию. По сравнению с другими доступными каркасами фаззинга архитектура Peach, вероятно, является самой гибкой и больше всего способствует повторному использованию кода. Кроме того, по мнению автора, у него самое интересное название, peach, персик, покрыт пушком, поняли? Каркас предоставляет ряд основных компонентов для построения новых фаззеров, включая генераторы, преобразователи, протоколы, издатели и группы.

Генераторы отвечают за создание данных от простых строк до сложных многоуровневых двоичных сообщений. Генераторы можно соединять в цепочки, упрощая создание сложных типов данных. Абстрагирование генерации данных в отдельный объект позволяет легко повторно использовать код в реализованных фаззерах. Рассмотрим, например, что во время аудита сервера SMTP был разработан генератор адресов электронной почты. Этот генератор можно прозрачно повторно использовать в другом фаззере, требующем создания адресов электронной почты.

Преобразователи изменяют данные определённым образом. Примерами преобразователей могут быть кодировщик base64, gzip и кодирование HTML. Преобразователи также можно соединять в цепочки и связывать с генератором. Например, созданный адрес электронной почты можно передать через преобразователь URL-кодирования, а затем снова через преобразователь gzip. Абстрагирование преобразования данных в отдельный объект позволяет легко повторно использовать код в реализованных фаззерах. После реализации заданного преобразования его можно прозрачно повторно использовать во всех разрабатываемых в будущем фаззерах.

Издатели реализуют форму транспорта созданных данных через протокол. Примерами являются файловые издатели и издатели TCP. И снова абстрагирование этой концепции в отдельный объект способствует повторному использованию кода. Хотя в текущей версии Peach это невозможно, конечная цель издателей заключается в возможности прозрачного взаимодействия с любым издателем. Рассмотрим, например, что вы создаёте генератор изображений GIF. Этот генератор должен уметь публиковать данные в файл или отправлять их в веб-форму простой заменой указанного издателя.

Группы содержат один или несколько генераторов и являются механизмом перебора значений, которые способен создавать генератор. Peach включает несколько стандартных реализаций групп. Дополнительный компонент, объект Script, представляет собой простую абстракцию для сокращения объёма избыточного кода, необходимого для реализации циклического перебора данных посредством вызовов `group.next()` и `protocol.step()`.

В качестве полного, но простого примера рассмотрим следующий фаззер Peach, предназначенный для перебора пароля пользователя FTP по файлу словаря:

```
from Peach              import *
from Peach.Transformers import *
from Peach.Generators   import *
from Peach.Protocols    import *
from Peach.Publishers   import *
```

```
loginGroup = group.Group()
loginBlock = block.Block()

loginBlock.setGenerators((
    static.Static(“USER username\r\nPASS ”),
    dictionary.Dictionary(loginGroup, “dict.txt”),
    static.Static(“\r\nQUIT\r\n”)
))

loginProt = null.NullStdout(ftp.BasicFtp(‘127.0.0.1’, 21), loginBlock)

script.Script(loginProt, loginGroup, 0.25).go()
```

Фаззер начинает с импорта различных компонентов каркаса Peach. Далее создаются новые блок и группа. Блок определяется для передачи имени пользователя, а затем команды пароля. Следующий элемент блока импортирует словарь потенциальных паролей. Именно этот элемент блока будет перебираться во время фаззинга. Последний элемент блока завершает команду пароля и выдаёт команду FTP quit для отключения от сервера. Определяется новый протокол, расширяющий уже доступный протокол FTP. Наконец, создаётся объект сценария, организующий цикл соединений и итераций по словарю. Первая мысль, которая может прийти в голову после просмотра этого сценария, заключается в том, что взаимодействие с каркасом не очень интуитивно. Это справедливая критика и, вероятно, самый большой недостаток Peach.

Разработка первого фаззера в Peach определённо займёт у вас больше времени, чем разработка первого фаззера в Autodafé или Dfuz.

Архитектура Peach позволяет исследователю сосредоточиться на отдельных подкомпонентах заданного протокола, позднее связав их для создания полного фаззера. Этот подход к разработке фаззеров, хотя, вероятно, и не столь быстр, как блочный подход, определённо больше всех прочих каркасов фаззинга способствует повторному использованию кода. Например, если для тестирования антивирусного решения необходимо разработать преобразователь gzip, он становится доступным в библиотеке для прозрачного последующего использования при тестировании способности сервера HTTP обрабатывать сжатые данные. Это прекрасная грань Peach. Чем больше вы его используете, тем умнее он становится. Благодаря реализации целиком на Python фаззер Peach можно запускать в любой среде с подходящей установкой Python. Кроме того, используя существующие интерфейсы, такие как Python, Microsoft COM[11] или пакеты Microsoft .NET, Peach позволяет напрямую фаззить элементы управления ActiveX и управляемый код. Также предоставлены примеры прямого фаззинга DLL Microsoft Windows и встраивания Peach в код C/C++ для создания инструментированных клиентов и серверов.

Peach активно разрабатывается, и на момент публикации последней доступной версией является 0.5, выпущенная в апреле 2006 года. Хотя теоретически Peach весьма продвинут, к сожалению, он не имеет подробной документации и не используется широко. Отсутствие справочных материалов приводит к существенной кривой обучения, которая может отвратить вас от рассмотрения этого каркаса. Автор Peach представил несколько новых идей и создал прочное основание для расширения. В качестве последнего замечания, был объявлен перенос каркаса Peach на Ruby, хотя на момент написания дополнительные подробности отсутствовали.

General Purpose Fuzzer[12]

General Purpose Fuzzer (GPF), выпущенный Джаредом ДеМоттом из Applied Security, назван как игра слов с широко известным термином general protection fault. GPF активно сопровождается, доступен с открытым исходным кодом под лицензией GPL и разработан для работы на платформе UNIX. Как следует из названия, GPF спроектирован как обобщённый фаззер; в отличие от SPIKE, он способен создавать бесконечное число мутаций. Это не значит, что генерационные фаззеры превосходят эвристические, поскольку обе методологии имеют плюсы и минусы. Главное преимущество GPF над другими каркасами, перечисленными в этом разделе, заключается в низком пороге запуска работающего фаззера. GPF предоставляет функциональность посредством ряда режимов, включая PureFuzz, Convert, GPF, основной режим, Pattern Fuzz и SuperGPF.

PureFuzz является простым в использовании чисто случайным фаззером, похожим на подключение `/dev/urandom` к сокету. Хотя создаваемое пространство входных данных неинтеллектуально и бесконечно, этот метод ранее обнаруживал уязвимости даже в распространённом корпоративном программном обеспечении. Главное преимущество PureFuzz над сочетанием netcat и `/dev/urandom` состоит в том, что PureFuzz предоставляет параметр начального значения, позволяющий воспроизводить псевдослучайный поток. Кроме того, если PureFuzz достигает успеха, конкретные пакеты, отвечающие за возникновение исключения, можно точно определить посредством параметра диапазона.

> **СЛУЧАЙНОСТЬ ТОЖЕ МОЖЕТ БЫТЬ ЭФФЕКТИВНОЙ!**
>
> Многие предполагают, что такие фаззеры, как PureFuzz из GPF, создающие чисто случайные данные, слишком примитивны, чтобы когда-либо быть полезными. Чтобы развеять это распространённое заблуждение, рассмотрим следующий реальный пример, касающийся решения резервного копирования Computer Associates BrightStor ARCserve Backup. В августе 2005 года в агенте, отвечавшем за обработку резервного копирования Microsoft SQL Server, была обнаружена тривиально эксплуатируемая уязвимость переполнения буфера в стеке.[13] Для её вызова требуется лишь передать более 3168 байт затронутому демону, прослушивающему TCP-порт 6070.
>
> Эту уязвимость легко обнаружить случайным фаззингом с минимальной подготовкой и без анализа протокола. Считайте этот пример доказательством того, что запустить фаззер вроде PureFuzz, пока продолжается ручная работа по построению интеллектуального фаззера, определённо может быть полезно.

Convert является утилитой GPF, способной преобразовывать файлы libpcap, например созданные Ethereal[14] и Wireshark,[15] в файл GPF. Эта утилита снимает часть утомительной работы на начальных этапах моделирования протокола, преобразуя двоичный формат pcap, захват пакетов, в удобочитаемый и готовый к изменению текстовый формат.

В основном режиме GPF передаются файл GPF и различные параметры командной строки для управления разнообразными базовыми атаками на протокол. Перехваченный трафик воспроизводится повторно, а его части изменяются различными способами. Среди мутаций находятся вставка постепенно увеличивающихся строковых последовательностей и токенов строки формата, циклический перебор байтов и случайные мутации.

Этот режим фаззинга требует интенсивного ручного труда, поскольку основная часть его логики строится на чутье аналитика.

Pattern Fuzz (PF) является самым примечательным режимом GPF благодаря способности автоматически разбивать на токены и фаззить обнаруженные части протоколов с обычным текстом. PF исследует целевые протоколы на распространённые границы ASCII и ограничители полей и автоматически фаззит их согласно набору внутренних правил. Внутренние правила определяются в

коде C с названием tokAids. Механизм мутации ASCII определён как tokAid (`normal_ascii`), существуют и некоторые другие, например DNS. Для точного и интеллектуального моделирования и фаззинга специализированного двоичного протокола необходимо написать и скомпилировать tokAid.

SuperGPF представляет собой обёртку GPF в виде сценария Perl, написанную для ситуаций, когда для фаззинга выбрана конкретная конечная точка сокета, но исследователь не имеет ни малейшего представления, с чего начать. SuperGPF объединяет захват GPF с текстовым файлом, содержащим допустимые команды протокола, и создаёт тысячи новых файлов захвата. Затем сценарий запускает несколько экземпляров GPF в различных режимах фаззинга, обрушивая на цель широкий спектр созданных данных. SuperGPF ограничен фаззингом только протоколов ASCII.

И снова мы приводим пример сценария GPF для сравнения с ранее показанными фаззерами FTP:

```
Source:S Size:20 Data:220 (vsFTPd 1.1.3)
Source:C Size:12 Data:USER fuzzy
Source:S Size:34 Data:331 Please specify the password.
Source:C Size:12 Data:PASS wuzzy
Source:S Size:33 Data:230 Login successful. Have fun.
Source:C Size:6 Data:QUIT
Source:S Size:14 Data:221 Goodbye.
```

Самым большим препятствием при работе с GPF является его сложность. Кривая обучения, необходимая для начала работы с GPF, значительна. Моделирование проприетарных двоичных протоколов с помощью tokAids не столь просто, как другие альтернативы, например блочный подход SPIKE, и, кроме того, требует компиляции для использования. Наконец, сильная зависимость от параметров командной строки приводит к таким громоздким командам, как следующая:

```
GPF ftp.gpf client localhost 21 ? TCP 8973987234 100000 0 + 6 6 100 100 5000 43 finish
0 3 auto none -G b
```

В целом GPF является ценным инструментом благодаря и гибкости, и расширяемости. Различные режимы позволяют исследователю быстро начать фаззинг, одновременно подготавливая вторую, более интеллектуальную волну атак. Возможность автоматически обрабатывать и фаззить протоколы ASCII мощна и выделяется среди других каркасов. На момент написания автор GPF разрабатывает новый режим фаззинга, использующий основы эволюционных вычислений для автоматического разбора и интеллектуального фаззинга неизвестных протоколов. Этот продвинутый подход к фаззингу подробнее обсуждается в главе 22, «Автоматизированный разбор протокола».

Autodafé[16]

Проще всего Autodafé можно описать как следующее поколение SPIKE, которое можно использовать для фаззинга как сетевых протоколов, так и форматов файлов. Autodafé выпущен Мартином Вуаньу под гибкой GNU GPL и предназначен для работы на платформах UNIX. Подобно SPIKE, основной механизм фаззинга Autodafé применяет блочный подход к моделированию протоколов. Следующая выдержка из технического документа «Autodafé, an Act of Software Torture»[17] демонстрирует блочный язык Autodafé и покажется знакомой пользователям SPIKE:

```
string(“dummy”);            /* define a constant string */
string_uni(“dummy”);        /* define a constant unicode string */
hex(0x0a 0a \x0a);          /* define a hexadecimal constant value */
block_begin(“block”);       /* define the beginning of a block */
block_end(“block”);         /* define the end of a block */
block_size_b32(“block”);    /* 32 bits big-endian size of a block */
block_size_l32(“block”);    /* 32 bits little-endian size of a block */
block_size_b16(“block”);    /* 16 bits big-endian size of a block */
block_size_l16(“block”);    /* 16 bits little-endian size of a block */
```

```
block_size_8(“block”);      /* 8 bits size of a block */
block_size_8(“block”);      /* 8 bits size of a block */
block_size_hex_string(“a”); /* hexadecimal string size of a block
block_size_dec_string(“b”); /* decimal string size of a block */
block_crc32_b(“block”);     /* crc32 of a block in big-endian */
block_crc32_l(“block”);     /* crc32 of a block in little-endian */
send(“block”);              /* send the block */
recv(“block”);              /* receive the block */
fuzz_string(“dummy”);       /* fuzz the string “dummy” */
fuzz_string_uni(“dummy”);   /* fuzz the unicode string “dummy” */
fuzz_hex(0xff ff \xff);     /* fuzz the hexadecimal value */
```

Эта функциональность обманчиво проста, поскольку с её помощью можно представить большинство форматов двоичных и текстовых протоколов.

Главная цель каркаса Autodafé заключается в сокращении размера и сложности общего пространства фаззинговых входных данных, чтобы эффективнее сосредоточиться на областях протокола, которые, вероятно, приведут к обнаружению уязвимостей безопасности. Вычислить пространство входных данных, или сложность, для полного фаззингового аудита просто. Рассмотрим следующий простой сценарий Autodafé:

```
fuzz_string(“GET”);
string(“ /”);
fuzz_string(“index.html”);
string(“ HTTP/1.1”);
hex(0d 0a);
```

После запуска против целевого веб-сервера этот сценарий сначала переберёт ряд мутаций команд HTTP, а затем ряд мутаций аргумента команды. Предположим, что Autodafé содержит библиотеку замен фаззинговых строк из 500 записей. Общее число тестовых случаев, необходимое для завершения этого аудита, равно 500, умноженным на число переменных для фаззинга, всего 1000 тестовых случаев. В большинстве реальных ситуаций библиотека замен будет как минимум вдвое больше, а переменных для фаззинга могут быть сотни. Autodafé применяет интересный метод под названием Markers Technique, чтобы назначить каждой фаззинговой переменной вес. Autodafé определяет маркер как данные, строковые или числовые, которыми может управлять пользователь, или фаззер. Применённые веса используются для определения порядка обработки фаззинговых переменных с сосредоточением на тех, которые вероятнее приведут к обнаружению уязвимостей безопасности.

Для выполнения этой задачи Autodafé включает компонент отладчика под названием adbg. Технологии отладки традиционно использовались рядом с фаззерами и даже включались в конкретные инструменты фаззинга, такие как FileFuzz, см. главу 13, «Фаззинг форматов файлов: автоматизация в Windows», но Autodafé является первым каркасом фаззинга, явно включающим отладчик. Компонент отладчика используется Autodafé для установки точек останова на распространённых опасных API, таких как `strcpy()`, известный как источник переполнений буфера, и `fprintf()`, известный как источник уязвимостей строки формата. Фаззер передаёт тестовые случаи одновременно цели и отладчику. Затем отладчик отслеживает вызовы опасных API в поисках строк, происходящих из фаззера.

Каждая фаззинговая переменная считается маркером. Вес отдельного маркера, обнаруженного проходящим через опасный API, увеличивается. Маркеры, не касающиеся опасных API, не фаззятся во время первого прохода. Маркерам с большими весами отдаётся приоритет, и они фаззятся первыми. Если отладчик обнаруживает нарушение доступа, фаззер автоматически получает сведения, а ответственный тестовый случай записывается. За счёт расстановки приоритетов фаззинговых переменных и игнорирования тех, которые никогда не проходят через опасный API, общее пространство входных данных можно резко сократить.

Autodafé включает несколько дополнительных инструментов, помогающих быстро и эффективно разрабатывать фаззеры. Первый инструмент, PDML2AD, может разбирать экспортированные

Ethereal и Wireshark XML-файлы пакетных данных (PDML) в блочный язык Autodafé. Если ваш целевой протокол входит в число более чем 750 протоколов, распознаваемых этими популярными сетевыми анализаторами, основную часть утомительного блочного моделирования можно выполнить автоматически. Даже если целевой протокол не распознан, PDML2AD всё равно способен предоставить несколько сокращений пути, поскольку автоматически обнаружит поля обычного текста и создаст соответствующие вызовы `hex()`, `string()` и так далее. Второй инструмент, TXT2AD, представляет собой простой сценарий оболочки, преобразующий текстовый файл в сценарий Autodafé. Третий и последний инструмент, ADC, является компилятором Autodafé. ADC полезен при разработке сложных сценариев Autodafé, поскольку способен обнаруживать распространённые ошибки, такие как неправильные имена функций и незакрытые блоки.

Autodafé является хорошо продуманным продвинутым каркасом фаззинга, развивающим фундамент, заложенный SPIKE. Autodafé разделяет многие плюсы и минусы SPIKE. Самой впечатляющей возможностью этого каркаса является компонент отладки, выделяющийся среди остальных каркасов. И снова, в зависимости от индивидуальных потребностей, отсутствие поддержки Microsoft Windows может сразу исключить этот каркас из рассмотрения. Изменения каркаса требуют повторной компиляции, и, опять же, повторное использование кода между сценариями фаззинга не настолько прозрачно и легко, каким могло бы быть.

Практический пример специализированного фаззера: Shockwave Flash

В предыдущем разделе обсуждалась хорошая выборка доступных сейчас каркасов фаззинга. Более полный перечень см. на сопутствующем веб-сайте книги по адресу http://www.fuzzing.org. Мы рекомендуем вам загрузить отдельные пакеты каркасов и исследовать полную документацию и различные примеры. Ни один каркас не выделяется как лучший в своём классе, способный справиться с любой поставленной задачей лучше остальных. Вместо этого конкретный каркас может быть более применим в зависимости от конкретной цели, задач, сроков, бюджета и других факторов. Знакомство с несколькими каркасами предоставит группе тестирования гибкость для применения лучшего инструмента к каждой заданной работе.

Хотя большинство протоколов и форматов файлов можно легко описать как минимум в одном общедоступном каркасе фаззинга, будут случаи, когда не применим ни один из них. При аудите специализированного программного обеспечения или протокола решение создать новый фаззер для конкретной цели может дать лучшие результаты и сэкономить время на протяжении всего аудита. В этом разделе мы обсуждаем формат файла Adobe Macromedia Shockwave Flash[18] (SWF) как реальную цель фаззинга и приводим части специализированного тестового решения.

Уязвимости безопасности в Shockwave Flash обычно имеют высокое воздействие, поскольку какая-либо версия Flash Player установлена почти на каждом настольном компьютере планеты. Фактически современные операционные системы Microsoft Windows по умолчанию поставляются с функциональной версией Flash Player. Разбор каждой подробности полномасштабного аудита SWF далеко выходит за рамки этой главы, поэтому обсуждаются лишь самые относящиеся к делу и интересные части. Дополнительные сведения, актуальные результаты и листинги кода[19] см. на официальном веб-сайте книги по адресу http://www.fuzzing.org.

Несколько факторов делают SWF прекрасным кандидатом для специализированного решения фаззинга. Во-первых, структура формата файла SWF полностью документирована в справочнике разработчика Adobe под названием «Macromedia Flash (SWF) and Flash Video (FLV) File Format Specification».[20] При написании этой работы использовалась версия документа 8. Если вы не проводите аудит формата файла с открытым исходным кодом или разработанного внутри организации, столь подробная документация обычно недоступна, что может ограничить тестирование более общими подходами. В данном конкретном случае мы определённо

воспользуемся каждой подробностью, предоставленной Adobe и Macromedia. Исследование спецификации показывает, что формат файла SWF в основном разбирается как поток битов, а не с типичной гранулярностью на уровне байтов, с которой разбирается большинство файлов и протоколов. Поскольку SWF обычно передаются через Веб, вероятной мотивацией выбора разбора на уровне битов является минимизация размера. Это добавляет нашему фаззеру уровень сложности и даёт дополнительную причину создать специализированное решение, поскольку ни один из исследованных в этой главе каркасов фаззинга не поддерживает битовые типы.

Моделирование файлов SWF

Первая веха, которую необходимо преодолеть для успешного фаззинга формата файла SWF, заключается в способности нашего фаззера и изменять, и создавать тестовые случаи. SWF состоит из заголовка, за которым следует ряд тегов, напоминающих следующую высокоуровневую структуру:

```
[Header]
    <magic>
    <version>
    <length>
    <rect>
        [nbits]
        [xmin]

        [xmax]
        [ymin]
        [ymax]
    <framerate>
    <framecount>
[FileAttributes Tag]
[Tag]
    <header>
    <data>
        <datatypes>
        <structs>
        ...
[Tag]
    <header>
    <data>
        <datatypes>
        <structs>
        ...
...
[ShowFrame Tag]
[End Tag]
```

Различные поля разбиваются следующим образом:

• **magic.** Состоит из трёх байтов и для любого допустимого файла SWF должно иметь значение “FWS”.
• **version.** Состоит из одного байта, представляющего числовую версию Flash, создавшую файл.
• **length.** Четырёхбайтное значение, указывающее размер всего файла SWF.
• **rect.** Это поле разделено на пять компонентов:
• **nbits.** Имеет ширину пять бит и указывает длину в битах каждого из следующих четырёх полей.
• **xmin, xmax, ymin и ymax.** Эти поля представляют экранные координаты, по которым следует отрисовать содержимое Flash. Обратите внимание, что эти поля представляют не пиксели, а twips, единицу измерения Flash, равную 1/20 «логического пикселя».

В поле `rect` мы начинаем видеть сложность этого формата файла. Если значение поля `nbits` равно 3, общая длина части `rect` заголовка равна 5 + 3 + 3 + 3 + 3 = 17 бит. Если значение поля `nbits` равно 4, общая длина части `rect` заголовка равна 5 + 4 + 4 + 4 + 4 = 21 бит. Представить эту

структуру с помощью одного из доступных каркасов фаззинга непросто. Последние два поля заголовка представляют скорость воспроизведения кадров и общее число кадров в файле SWF.

За заголовком следует ряд тегов, определяющих содержимое и поведение файла Flash. Первый тег, FileAttributes, был представлен во Flash версии 8 и является обязательным тегом, служащим двум целям. Во-первых, тег указывает, содержит ли файл SWF различные свойства, такие как заголовок и описание. Эти сведения не используются внутри, а предоставляются для справки внешним процессам, таким как поисковые системы. Во-вторых, тег указывает, должен ли Flash Player предоставить файлу SWF доступ к локальным или сетевым файлам. Каждый файл SWF имеет переменное число оставшихся тегов, относящихся к одной из двух категорий: теги определения и управления. Теги определения задают такое содержимое, как фигуры, кнопки и звук. Теги управления определяют такие понятия, как размещение и движение. Flash Player обрабатывает теги до встречи тега ShowFrame, в этот момент содержимое отрисовывается на экране. Файл SWF завершается обязательным тегом End.

Существует ряд доступных тегов на выбор, каждый состоит из двухбайтного заголовка тега, за которым следуют данные. В зависимости от длины данных тег можно определить как длинный или короткий. Если длина данных меньше 63 бит, тег считается коротким и определяется так: `[десятибитный идентификатор тега] [шестибитная длина данных] [данные]`. Если блок данных больше, тег считается длинным и определяется с дополнительным четырёхбайтным полем в заголовке, указывающим длину данных: `[десятибитный идентификатор тега] [111111] [четырёхбайтная длина данных] [данные]`. И снова мы видим, что сложность формата SWF плохо поддаётся моделированию в любом из вышеупомянутых каркасов фаззинга.

Дальше всё становится только хуже. Каждый тег имеет собственный набор полей. Некоторые содержат необработанные данные, другие состоят из базовых типов SWF с названием структуры (`struct` или `structs`). Каждая структура состоит из набора определённых полей, которые могут включать и необработанные данные, и даже другие структуры! И всё по-прежнему становится лишь хуже. Группы полей как внутри тегов, так и внутри структур могут быть определены или не определены в зависимости от значения другого поля внутри того же тега или структуры. Даже объяснение может запутать, поэтому давайте начнём строить снизу вверх, а затем покажем несколько примеров, чтобы всё объяснить лучше.

Для начала определяем на Python класс битового поля для представления произвольных числовых полей переменной битовой длины. Мы расширяем этот класс битового поля для определения байтов, символов, слов, коротких целых, двойных слов, длинных целых, и четверных слов, чисел двойной точности, поскольку их легко определить как 8-, 16-, 32- и 64-битные экземпляры класса битового поля. Эти базовые структуры данных доступны в пакете Sulley, который не следует путать с каркасом фаззинга Sulley, обсуждаемым в следующем разделе:

```
BIG_ENDIAN    = “>”
LITTLE_ENDIAN = “<”

class bit_field (object):
    def __init__ (self, width, value=0, max_num=None):
        assert(type(value) is int or long)

        self.width   = width
        self.max_num = max_num
        self.value   = value
        self.endian  = LITTLE_ENDIAN
        self.static  = False
        self.s_index = 0

        if self.max_num == None:
            self.max_num = self.to_decimal(“1” * width)

    def flatten (self):
        ‘’’
```

```
    @rtype:  Raw Bytes
    @return: Raw byte representation
    '''

    # дополнить поток битов до следующей границы байта.
    bit_stream = ""
    if self.width % 8 == 0:
        bit_stream += self.to_binary()
    else:
        bit_stream  = "0" * (8 - (self.width % 8))
        bit_stream += self.to_binary()

    flattened = ""

    # преобразовать поток битов из строки битов в необработанные байты.
    for i in xrange(len(bit_stream) / 8):
        chunk = bit_stream[8*i:8*i+8]
        flattened += struct.pack("B", self.to_decimal(chunk))

    # при необходимости изменить порядок байтов необработанных данных.
    if self.endian == LITTLE_ENDIAN:
        flattened = list(flattened)
        flattened.reverse()
        flattened = "".join(flattened)

    return flattened

def to_binary (self, number=None, bit_count=None):
    '''

    @type number:     Integer
    @param number:    (Opt., def=self.value) Number to convert
    @type bit_count:  Integer
    @param bit_count: (Opt., def=self.width) Width of bit string

    @rtype:  String
    @return: Bit string
    '''

    if number == None:
        number = self.value
    if bit_count == None:
        bit_count = self.width

    return "".join(map(lambda x:str((number >> x) & 1), \
        range(bit_count -1, -1, -1)))

def to_decimal (self, binary):
    '''
    Convert a binary string into a decimal number and return.
    '''

    return int(binary, 2)

def randomize (self):
    '''
    Randomize the value of this bitfield.
    '''

    self.value = random.randint(0, self.max_num)

def smart (self):
    '''
    Step the value of this bitfield through a list of smart values.
    '''

    smart_cases = \
    [
        0,
        self.max_num,
        self.max_num / 2,
```

```
            self.max_num / 4,
            # и так далее...
        ]

        self.value    = smart_cases[self.s_index]
        self.s_index += 1

class byte (bit_field):
    def __init__ (self, value=0, max_num=None):
        if type(value) not in [int, long]:
            value = struct.unpack(endian + “B”, value)[0]

        bit_field.__init__(self, 8, value, max_num)

class word (bit_field):
    def __init__ (self, value=0, max_num=None:
        if type(value) not in [int, long]:
            value = struct.unpack(endian + “H”, value)[0]

        bit_field.__init__(self, 16, value, max_num)

class dword (bit_field):
    def __init__ (self, value=0, max_num=None):
        if type(value) not in [int, long]:
            value = struct.unpack(endian + “L”, value)[0]

        bit_field.__init__(self, 32, value, max_num)

class qword (bit_field):
    def __init__ (self, value=0, max_num=None):
        if type(value) not in [int, long]:
            value = struct.unpack(endian + “Q”, value)[0]

        bit_field.__init__(self, 64, value, max_num)

# псевдонимы классов
bits   = bit_field
char   = byte
short  = word
long   = dword
double = qword
```

Базовый класс `bit_field` определяет ширину поля (`width`), максимальное число, которое способно представлять поле (`max_num`), значение поля (`value`), порядок битов поля (`endian`), флаг, указывающий, можно ли изменять значение поля (`static`), и, наконец, используемый внутри индекс (`s_index`). Класс `bit_field` дополнительно определяет ряд полезных функций:

• Процедура `flatten()` преобразует и возвращает ограниченную байтами последовательность необработанных байтов из поля.
• Процедура `to_binary()` может преобразовать числовое значение в строку битов.
• Процедура `to_decimal()` выполняет обратное действие, преобразуя строку битов в числовое значение.
• Процедура `randomize()` обновляет значение поля до случайного значения в допустимом диапазоне поля.
• Процедура `smart()` обновляет значение поля по последовательности «умных» границ, при этом показана лишь выдержка из этих чисел.

При построении сложного типа из строительного блока `bit_field` последние две процедуры можно вызывать для мутации созданной структуры данных. Затем структуру данных можно обойти и записать в файл посредством рекурсивного вызова процедур `flatten()` каждого отдельного компонента.

Используя эти базовые типы, теперь мы можем начать определять более простые структуры SWF, такие как `RECT` и `RGB`. Следующие выдержки кода показывают определения этих классов, наследующих от не показанного базового класса, определение базового класса см. по адресу

http://www.fuzzing.org:

```
class RECT (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.fields = \
        [
            (“Nbits”, sulley.numbers.bits(5, value=31, static=True)),
            (“Xmin” , sulley.numbers.bits(31)),
            (“Xmax” , sulley.numbers.bits(31)),
            (“Ymin” , sulley.numbers.bits(31)),
            (“Ymax” , sulley.numbers.bits(31)),
        ]

class RGB (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.fields = \
        [

            (“Red”  , sulley.numbers.byte()),
            (“Green”, sulley.numbers.byte()),
            (“Blue” , sulley.numbers.byte()),
        ]
```

Эта выдержка также показывает простое сокращение, обнаруженное и согласованное во время разработки. Для упрощения фаззинга все поля переменной ширины определяются с максимальной длиной. Чтобы представлять структуры с зависимыми полями, из класса `bit_field` был выведен новый примитив `dependent_bit_field`, как показано здесь:

```
class dependent_bit_field (sulley.numbers.bit_field):
    def __init__ (self, width, value=0, max_num=None, static=False, \
                  parent=None, dep=None, vals=[]):

        self.parent = parent
        self.dep    = dep
        self.vals   = vals

        sulley.numbers.bit_field.__init__(self, width, value, \
                                          max_num, static)

    def flatten (self):
        # если существует зависимость для сведения, включающая эту
        # структуру, проверить её.
        if self.parent:
            #                               VVV - значение объекта
            if self.parent.fields[self.dep][1].value not in self.vals:
                # зависимость не выполнена, не включать этот объект.
                return “”

        return sulley.numbers.bit_field.flatten(self)
```

Этот расширенный класс задаёт индекс поля и значение, которые следует исследовать до создания и возврата данных. Если соответствующее поле, на которое ссылается `dep`, не содержит значения из списка `vals`, данные не возвращаются. Структура `MATRIX` демонстрирует использование нового определённого примитива:

```
class MATRIX (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.fields = \
        [
            (“HasScale”     , sulley.numbers.bits(1)),

            (“NScaleBits”   , dependent_bits(5, 31, parent=self, \
                                             dep=0, vals=[1])),
```

```
        ("ScaleX"       , dependent_bits(31, parent=self, \
                                         dep=0, vals=[1])),
        ("ScaleY"       , dependent_bits(31, parent=self, \
                                         dep=0, vals=[1])),
        ("HasRotate"    , sulley.numbers.bits(1)),
        ("NRotateBits"  , dependent_bits(5, 31, parent=self, \
                                         dep=4, vals=[1])),
        ("skew1"        , dependent_bits(31, parent=self, \
                                         dep=0, vals=[1])),
        ("skew2"        , dependent_bits(31, parent=self, \
                                         dep=0, vals=[1])),
        ("NTranslateBits" , sulley.numbers.bits(5, value=31)),
        ("TranslateX"     , sulley.numbers.bits(31)),
        ("TranslateY"     , sulley.numbers.bits(31)),
    ]
```

Из этой выдержки видно, что поле `NScaleBits` внутри структуры `MATRIX` определено как поле шириной пять бит со значением по умолчанию 31, включаемое в структуру только в том случае, если значение поля с индексом 0 (`HasScale`) равно 1. Поля `ScaleX`, `ScaleY`, `skew1` и `skew2` также зависят от поля `HasScale`. Иными словами, если `HasScale` равно 1, эти поля допустимы. В противном случае они не должны быть определены. Подобным образом поле `NRotateBits` зависит от значения поля с индексом 4 (`HasRotate`). На момент написания в этой нотации было точно определено более 200 структур SWF.[21]

Обладая всеми необходимыми примитивами и структурами, теперь мы можем начать определять целые теги. Сначала определяется базовый класс, от которого выводятся все теги:

```
class base (structs.base):
    def __init__ (self, parent=None, dep=None, vals=[]):
        self.tag_id = None

        (structs.base).__init__(self, parent, dep, vals)

    def flatten (self):
        bit_stream = structs.base.flatten(self)

        # дополнить поток битов до следующей границы байта.

        if len(bit_stream) % 8 != 0:
            bit_stream  = "0" * (8-(len(bit_stream)%8)) + bit_stream

        raw = ""

        # преобразовать поток битов из строки битов в необработанные байты.
        for i in xrange(len(bit_stream) / 8):
            chunk = bit_stream[8*i:8*i+8]
            raw += pack("B", self.to_decimal(chunk))

        raw_length = len(raw)

        if raw_length >= 63:
            # длинный (заголовок записи состоит из word + dword)
            record_header = self.tag_id
            record_header <<= 6
            record_header |= 0x3f

            flattened = pack('H', record_header)

            record_header <<= 32
            record_header |= raw_length

            flattened += pack('Q', record_header)
            flattened += raw
        else:
            # короткий (record_header является word)
            record_header = self.tag_id
            record_header <<= 6
            record_header |= raw_length
            flattened = pack('H', record_header)
```

```
            flattened += raw

        return flattened
```

Процедура `flatten()` базового класса тега автоматически вычисляет длину части данных и создаёт правильный короткий или длинный заголовок. На момент написания на этом каркасе было точно определено более 50 тегов SWF. Ниже приведено несколько примеров простой и средней сложности:

```
class PlaceObject (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.tag_id = 4

        self.fields = \
        [
            (“CharacterId”    , sulley.numbers.word(value=0x01)),
            (“Depth”          , sulley.numbers.word()),
            (“Matrix”         , structs.MATRIX()),
            (“ColorTransform” , structs.CXFORM()),
        ]

class DefineBitsLossless (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.tag_id = 20
        self.fields = \
        [
            (“CharacterId”          , sulley.numbers.word()),
            (“BitmapFormat”         , sulley.numbers.byte()),
            (“BitmapWidth”          , sulley.numbers.word()),
            (“BitmapHeight”         , sulley.numbers.word()),
            (“BitmapColorTableSize” , structs.dependent_byte( \
                parent=self, dep=1, vals=[3])),
            (“ZlibBitmapData”       , structs.COLORMAPDATA( \
                parent=self, dep=1, vals=[3])),
            (“ZlibBitMapData_a”     , structs.BITMAPDATA( \
                parent=self, dep=1, vals=[4, 5])),
        ]

class DefineMorphShape (base):
    def __init__ (self, *args, **kwargs):
        base.__init__(self, *args, **kwargs)

        self.tag_id = 46
        self.fields = \
        [
            (“CharacterId”      , sulley.numbers.word()),
            (“StartBounds”      , structs.RECT()),
            (“EndBounds”        , structs.RECT()),
            (“Offset”           , sulley.numbers.word()),
            (“MorphFillStyles”  , structs.MORPHFILLSTYLE()),
            (“MorphLineStyles”  , structs.MORPHLINESTYLES()),
            (“StartEdges”       , structs.SHAPE()),
            (“EndEdges”         , structs.SHAPE()),
        ]
```

Представлено много сведений, поэтому давайте подведём итог. Чтобы правильно смоделировать произвольный SWF, мы начали с самого базового примитива `bit_field`. Из него вывели примитивы для байтов, слов, двойных и четверных слов. Также из `bit_field` мы вывели `dependent_bit_field`, из которого далее вывели зависимые байты, слова, двойные и четверные слова. Эти типы вместе с новым базовым классом образуют основу структур SWF. Базовый класс тегов выводится из базового класса структур и вместе со структурами и примитивами образует теги SWF. Дополнительное пояснение даёт связь, изображённая на рисунке 21.1.

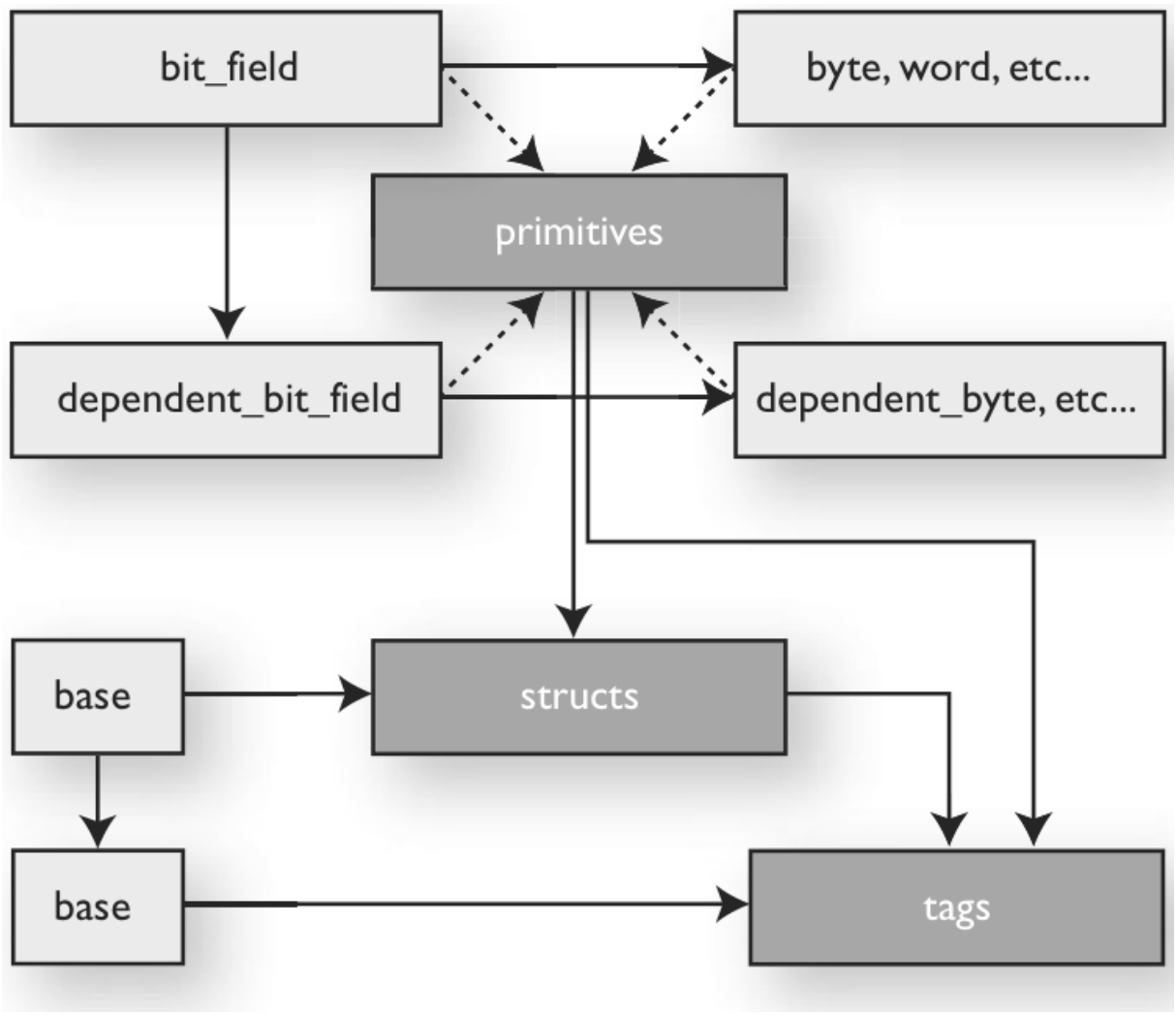


Рисунок 21.1. Связь компонентов фаззера SWF

Для полного моделирования SWF необходимы и некоторые другие строительные блоки, например строковые примитивы. Их определения см. в исходном коде. Имея все эти классы, мы строим глобальный контейнер SWF и начинаем создавать экземпляры тегов и добавлять их в него. Полученную структуру данных затем можно легко обходить и изменять вручную или автоматически с помощью процедур `randomize()` или `smart()` каждого отдельного компонента. Наконец, структуру можно записать в файл, снова обойдя сложную структуру данных и объединив результаты процедуры `flatten()`. Такой проект позволяет прозрачно обрабатывать особые случаи внутри отдельных тегов или структур.

Генерация допустимых данных

Вскоре после преодоления первой трудности, успешного представления базовых структур SWF, мы встречаем следующую. Спецификация SWF гласит, что теги могут зависеть только от ранее определённых тегов. При фаззинге тега с несколькими зависимостями сначала необходимо успешно разобрать все зависимости, иначе целевой тег никогда не будет достигнут. Угадывание допустимых значений полей является кропотливым процессом, но существует хитрая альтернатива. Используя Google SOAP API,[22] мы можем искать в Вебе файлы SWF по ключевому слову `filetype:swf`. Пишется простой сценарий, последовательно выполняющий поиск `filetype:swf`, `b filetype:swf` и так далее до `z filetype:swf`. Сценарий разбирает возвращённые результаты, ищет и загружает обнаруженные файлы SWF. Файлы именуются по хешу MD5, чтобы предотвратить хранение дубликатов.

Паук собрал приблизительно 10 000 уникальных файлов SWF, занимающих более 3 Гбайт. Исследование поля версии в заголовке SWF позволяет получить статистическую выборку распределения, показанную в таблице 21.1.

Таблица 21.1. Распределение версий собранных пауком Flash SWF

Версия SWF	**% от общего числа**
Flash 8	< 1%
Flash 7	~ 2%
Flash 6	~ 11%
Flash 5	~ 55%
Flash 4	~ 28%
Flash 1 - Flash 3	~ 3%

Эти результаты интересны. К сожалению, из этой статистики нельзя сделать выводов о распределении активных Flash Player. Затем пишется другой сценарий, разбирающий каждый файл, извлекающий и сохраняющий отдельные теги. Теперь это хранилище допустимых тегов можно использовать для помощи в создании тестовых случаев SWF.

Среда фаззинга

Каркас обратной разработки PaiMei[23] используется для облегчения создания экземпляров отдельных тестовых случаев и их мониторинга. PaiMei представляет собой набор классов на чистом Python, предназначенных для помощи в разработке инструментов автоматизации обратной разработки. Использование каркаса подробнее описано в главе 23. Фактический процесс тестирования сходен с применяемым обобщённым PaiMei FileFuzz и выглядит следующим образом:

1. Загрузить тестовый случай SWF в Flash Player под управлением сценарного отладчика PaiMei PyDbg.
2. Запустить проигрыватель и отслеживать исполнение в течение пяти секунд. Задержка в пять секунд выбрана произвольно и предполагает, что если при разборе файла SWF должна произойти ошибка, она произойдёт в этот промежуток времени.

3. Если за это время обнаружена ошибка, сохранить тестовый случай и относящиеся к делу сведения отладки.
4. Если за это время ошибка не обнаружена, закрыть экземпляр Flash Player.
5. Перейти к следующему тестовому случаю.

Для более эффективного тестирования можно изменить частоту кадров в заголовке SWF со значения по умолчанию `0x000C` на максимальное `0xFFFF`. Тем самым мы позволяем обработать более крупные части файла SWF за пять секунд, отведённых каждому тестовому случаю. Метод просмотра файлов SWF по умолчанию заключается в их загрузке в веб-браузер, такой как Microsoft Internet Explorer или Mozilla Firefox. Хотя этот подход приемлем в данном конкретном случае, альтернативным методом, оказывающимся удобным, особенно для фаззинга ActionScript,[24] является использование `SAFlashPlayer.exe`. Этот небольшой автономный проигрыватель распространяется с Macromedia Studio.[25]

Методологии тестирования

Самый общий подход к фаззингу файлов SWF заключается в инверсии битов. Этот элементарный метод тестирования встречался в предыдущих главах на уровне байтов. Применение этого метода для SWF на более мелком уровне битов разумно, поскольку один байт может охватывать более одного поля. Делая шаг дальше инверсии битов, специализированный сценарий разбора тегов SWF можно использовать для изменения порядка тегов внутри отдельного файла SWF и перестановки тегов между двумя разными SWF. Наконец, самый тщательный подход к фаззингу SWF заключается в отдельном стрессовом тестировании каждого поля внутри каждой структуры и каждого тега. К ранее созданному хранилищу тегов и структур можно обращаться, чтобы по частям собирать базовый SWF для мутации.

При всех трёх подходах созданные файлы одинаковым образом передаются через среду фаззинга. В следующем разделе мы исследуем новый каркас фаззинга, представленный и выпущенный авторами вместе с этой книгой.

Sulley: каркас фаззинга

Sulley является каркасом разработки фаззеров и фаззингового тестирования, состоящим из нескольких расширяемых компонентов. Sulley, по нашему скромному мнению, превосходит возможности большинства ранее опубликованных технологий фаззинга, как коммерческих, так и общедоступных. Цель каркаса состоит в упрощении не только представления данных, но также их передачи и мониторинга цели. Sulley ласково назван в честь существа из «Корпорации монстров»[26], потому что, ну, он пушистый. Последнюю версию Sulley можно загрузить по адресу http://www.fuzzing.org/sulley.

Современные фаззеры по большей части сосредоточены исключительно на генерации данных. Sulley не только впечатляюще генерирует данные, но пошёл ещё дальше и включает многие другие важные аспекты, которые должен предоставлять современный фаззер. Sulley наблюдает за сетью и методично ведёт записи. Sulley инструментирует и отслеживает состояние цели и способен возвращать её в исправное состояние несколькими способами. Sulley обнаруживает, отслеживает и классифицирует выявленные дефекты. Sulley может фаззить параллельно, значительно повышая скорость тестирования. Sulley способен автоматически определить, какая уникальная последовательность тестовых случаев вызывает дефекты. Sulley делает всё это и многое другое автоматически и без присмотра.

Общее использование Sulley разбивается на следующие этапы:

1. **Представление данных.** Это первый шаг при использовании любого фаззера. Запустите цель и немного воздействуйте на несколько интерфейсов, одновременно перехватывая пакеты. Разбейте протокол на отдельные запросы и представьте их в Sulley в виде блоков.
2. **Сеанс.** Свяжите разработанные запросы, чтобы образовать сеанс, подключите различные доступные агенты мониторинга Sulley, сокет, отладчик и так далее, и начните фаззинг.
3. **Посмертный анализ.** Просмотрите созданные данные и результаты мониторинга. Воспроизведите отдельные тестовые случаи.

Загрузив последнюю версию пакета Sulley с http://www.fuzzing.org, распакуйте его в выбранный каталог. Структура каталогов сравнительно сложна, поэтому давайте посмотрим, как всё организовано.

Структура каталогов Sulley

В структуре каталогов Sulley есть определённая логика. Сохранение структуры гарантирует, что всё останется организованным, пока вы расширяете фаззер элементами Lego, запросами и утилитами. Следующая иерархия описывает то, что вам необходимо знать о структуре каталогов:

• `archived_fuzzies`: каталог свободной формы, организованный по имени цели фаззинга, для хранения архивных фаззеров и данных, созданных сеансами фаззинга.
• `trend_server_protect_5168`: этот завершённый фаззинг упоминается в пошаговом руководстве далее в документе.
• `trillian_jabber`: ещё один завершённый фаззинг, упоминаемый в документации.
• `audits`: записанные PCAP, корзины аварий, покрытие кода и графики анализа для активных сеансов фаззинга следует сохранять в этом каталоге. После завершения записанные данные следует перемещать в `archived_fuzzies`.
• `docs`: документация и созданные справочники API Epydoc.
• `requests`: библиотека запросов Sulley. Каждая цель должна получить собственный файл, в котором можно хранить несколько запросов.
• `__REQUESTS__.html`: этот файл содержит описания сохранённых категорий запросов и перечисляет отдельные типы. Сохраняйте алфавитный порядок.
• `http.py`: различные запросы фаззинга веб-серверов.
• `trend.py`: содержит запросы, связанные с полным пошаговым руководством по фаззингу, обсуждаемым далее в документе.
• `sulley`: каркас фаззера. Если вы не хотите расширять каркас, вам не потребуется трогать эти файлы.
• `legos`: определённые пользователем сложные примитивы.
• `ber.py`: примитивы ASN.1/BER.
• `dcerpc.py`: примитивы Microsoft RPC NDR.
• `misc.py`: различные неклассифицированные сложные примитивы, такие как адреса электронной почты и имена узлов.
• `xdr.py`: типы XDR.
• `pgraph`: библиотека абстракции графов Python. Используется при построении сеансов.
• `utils`: различные вспомогательные процедуры.
• `dcerpc.py`: вспомогательные процедуры Microsoft RPC, например для привязки к интерфейсу и создания запроса.
• `misc.py`: различные неклассифицированные процедуры, например CRC-16 и процедуры манипулирования UUID.
• `scada.py`: вспомогательные процедуры для SCADA, включая кодировщик блоков DNP3.
• `__init__.py`: здесь определены различные псевдонимы `s_`, используемые при создании запросов.

• blocks.py: здесь определены блоки и вспомогательные средства блоков.
• pedrpc.py: этот файл определяет классы клиента и сервера, используемые Sulley для взаимодействия между различными агентами и главным фаззером.
• primitives.py: здесь определены различные примитивы фаззера, включая статические, случайные, строки и целые числа.
• sessions.py: функциональность построения и исполнения сеанса.
• sex.py: специализированный класс Sulley для обработки исключений.
• unit_tests: каркас модульного тестирования Sulley.
• utils: различные автономные утилиты.
• crashbin_explorer.py: утилита командной строки для исследования результатов, сохранённых в сериализованных файлах корзины аварий.
• pcap_cleaner.py: утилита командной строки для очистки каталога PCAP от всех записей, не связанных с дефектом.
• network_monitor.py: управляемый PedRPC агент мониторинга сети.
• process_monitor.py: управляемый PedRPC агент мониторинга цели на основе отладчика.
• unit_test.py: каркас модульного тестирования Sulley.
• vmcontrol.py: управляемый PedRPC агент управления VMWare.

Теперь, когда структура каталогов стала немного знакомее, давайте посмотрим, как Sulley обрабатывает представление данных. Это первый шаг построения фаззера.

Представление данных

Айтел был прав со SPIKE: мы внимательно рассмотрели все фаззеры, до которых смогли дотянуться, и блочный подход к представлению протокола возвышается над остальными, сочетая простоту и гибкость представления большинства протоколов. Sulley использует блочный подход для создания отдельных запросов, которые позднее связываются в сеанс. Для начала инициализируйте запрос новым именем:

```
s_initialize("new request")
```

Теперь вы начинаете добавлять в запрос примитивы, блоки и вложенные блоки. Каждый примитив можно отдельно отрисовать и изменить. Отрисовка примитива возвращает его содержимое в необработанном формате данных. Мутация примитива преобразует его внутреннее содержимое. Понятия отрисовки и мутации по большей части абстрагированы от разработчиков фаззеров, поэтому не беспокойтесь о них. Однако знайте, что каждый изменяемый примитив принимает значение по умолчанию, которое восстанавливается после исчерпания фаззимых значений.

Статические и случайные примитивы

Начнём с простейшего примитива s_static(), добавляющего в запрос статическое неизменяемое значение произвольной длины. Для вашего удобства по Sulley разбросаны различные псевдонимы: s_dunno(), s_raw() и s_unknown() являются псевдонимами s_static():

```
# все эти варианты эквивалентны:
s_static("pedram\x00was\x01here\x02")
s_raw("pedram\x00was\x01here\x02")
s_dunno("pedram\x00was\x01here\x02")
s_unknown("pedram\x00was\x01here\x02")
```

Примитивы, блоки и так далее принимают необязательный именованный аргумент name. Указание имени позволяет обращаться к именованному элементу непосредственно из запроса через request.names["name"], вместо того чтобы обходить структуру блоков до нужного элемента. С

предыдущим связан, но ему не эквивалентен примитив s_binary(), принимающий двоичные данные, представленные в нескольких форматах. Пользователи SPIKE узнают этот API, поскольку его функциональность эквивалентна, или, скорее, должна быть эквивалентна, уже знакомой вам:

```
# да, он может обрабатывать все эти форматы.
s_binary("0xde 0xad be ef \xca fe 00 01 02 0xba0xdd f0 0d")
```

Большинство примитивов Sulley управляется эвристиками фаззинга и потому имеет ограниченное число мутаций. Исключением является примитив s_random(), который можно использовать для создания случайных данных различной длины. Этот примитив принимает два обязательных аргумента, min_length и max_length, указывающих соответственно минимальную и максимальную длину случайных данных, создаваемых на каждой итерации. Он также принимает следующие необязательные именованные аргументы:

• num_mutations (целое, по умолчанию 25): число мутаций до возврата к значению по умолчанию.

• fuzzable (логическое, по умолчанию True): включить или отключить фаззинг этого примитива.
• name (строка, по умолчанию None): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Именованный аргумент num_mutations указывает, сколько раз этот примитив следует повторно отрисовать до того, как он будет считаться исчерпанным. Чтобы заполнить поле статического размера случайными данными, задайте одинаковые значения min_length и max_length.

Целые числа

И в двоичных, и в ASCII-протоколах повсюду встречаются целые числа различных размеров, например поле Content-Length в HTTP. Подобно большинству каркасов фаззинга, часть Sulley посвящена представлению этих типов:

• один байт: s_byte(), s_char();
• два байта: s_word(), s_short();
• четыре байта: s_dword(), s_long(), s_int();
• восемь байтов: s_qword(), s_double().

Каждый целочисленный тип принимает как минимум один параметр, значение целого по умолчанию. Дополнительно можно указать следующие необязательные именованные аргументы:

• endian (символ, по умолчанию '<'): порядок байтов битового поля. Укажите < для порядка от младшего к старшему и > для порядка от старшего к младшему.
• format (строка, по умолчанию "binary"): формат вывода, "binary" или "ascii", управляющий форматом отрисовки целочисленных примитивов. Например, значение 100 отрисовывается как "100" в ASCII и "\x64" в двоичном виде.
• signed (логическое, по умолчанию False): сделать размер знаковым вместо беззнакового, применимо только при format="ascii".
• full_range (логическое, по умолчанию False): если включён, примитив мутирует по всем возможным значениям, подробнее об этом далее.
• fuzzable (логическое, по умолчанию True): включить или отключить фаззинг этого примитива.
• name (строка, по умолчанию None): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Среди них особенно интересен модификатор full_range. Представьте, что вы хотите фаззить значение DWORD, это 4 294 967 295 возможных значений. При скорости 10 тестовых случаев в секунду фаззинг одного этого примитива занял бы 13 лет! Чтобы сократить столь огромное пространство входных данных, по умолчанию Sulley пробует только «умные» значения. Сюда

входят плюс и минус 10 граничных случаев вокруг 0, максимальное целое значение (MAX_VAL), MAX_VAL, делённое на 2, MAX_VAL, делённое на 3, MAX_VAL, делённое на 4, MAX_VAL, делённое на 8, MAX_VAL, делённое на 16, и MAX_VAL, делённое на 32. Исчерпание этого сокращённого пространства входных данных из 141 тестового случая занимает лишь секунды.

Строки и разделители

Строки встречаются повсюду. Адреса электронной почты, имена узлов, имена пользователей, пароли и многое другое являются примерами строковых компонентов, с которыми вы, несомненно, столкнётесь при фаззинге. Sulley предоставляет примитив s_string() для представления этих полей. Примитив принимает один обязательный аргумент, указывающий допустимое значение по умолчанию. Можно указать следующие дополнительные именованные аргументы:

• size (целое, по умолчанию -1): статический размер этой строки. Для динамического определения размера оставьте -1.
• padding (символ, по умолчанию ‘\x00’): если задан явный размер, а созданная строка меньше него, использовать это значение для дополнения поля до нужного размера.
• encoding (строка, по умолчанию “ascii”): кодировка строки. К допустимым вариантам относится всё, что способна принять процедура Python str.encode(). Для строк Microsoft Unicode укажите “utf_16_le”.
• fuzzable (логическое, по умолчанию True): включить или отключить фаззинг этого примитива.
• name (строка, по умолчанию None): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Строки часто разбираются на подполя посредством разделителей. Например, символ пробела используется как разделитель в запросе HTTP GET /index.html HTTP/1.0. Символы косой черты (/) и точки (.) в том же запросе тоже являются разделителями. При определении протокола в Sulley обязательно представляйте разделители с помощью примитива s_delim(). Как и у других примитивов, первый аргумент обязателен и используется для указания значения по умолчанию. Также, как другие примитивы, s_delim() принимает необязательные именованные аргументы fuzzable и name. Мутации разделителя включают повторение, замену и исключение. В качестве полного примера рассмотрим следующую последовательность примитивов для фаззинга тега body HTML.

```
# подвергает фаззингу строку: <BODY bgcolor=“black”>
s_delim(“<”)
s_string(“BODY”)
s_delim(“ ”)
s_string(“bgcolor”)
s_delim(“=”)

s_delim(“\””)
s_string(“black”)
s_delim(“\””)
s_delim(“>”)
```

Блоки

Освоив примитивы, далее давайте посмотрим, как их можно организовывать и вкладывать в блоки. Новые блоки определяются и открываются с помощью s_block_start(), а закрываются с помощью s_block_end(). Каждому блоку необходимо дать имя, указанное первым аргументом s_block_start(). Эта процедура также принимает следующие необязательные именованные аргументы:

• group (строка, по умолчанию None): имя группы, с которой следует связать этот блок, подробнее далее.
• encoder (указатель на функцию, по умолчанию None): указатель на функцию, которой передаются отрисованные данные перед их возвратом.
• dep (строка, по умолчанию None): необязательный примитив, от конкретного значения которого зависит этот блок.
• dep_value (смешанный тип, по умолчанию None): значение, которое должно содержать поле dep, чтобы блок был отрисован.
• dep_values (список смешанных типов, по умолчанию []): значения, которые может содержать поле dep, чтобы блок был отрисован.
• dep_compare (строка, по умолчанию "=="): метод сравнения, применяемый к зависимости. Допустимые варианты: ==, !=, >, >=, < и <=.

Группирование, кодирование и зависимости являются мощными возможностями, не встречающимися в большинстве других каркасов, и заслуживают дальнейшего разбора.

Группы

Группирование позволяет связать блок с групповым примитивом, указав, что блок должен перебрать все возможные мутации для каждого значения внутри группы. Групповой примитив полезен, например, для представления списка допустимых кодов операций или команд со сходными структурами аргументов. Примитив s_group() определяет группу и принимает два обязательных аргумента. Первый указывает имя группы, второй список возможных необработанных значений для перебора. В качестве простого примера рассмотрим следующий полный запрос Sulley, предназначенный для фаззинга веб-сервера:

```
# импортировать всю функциональность Sulley.
from sulley import *

# этот запрос предназначен для фаззинга: {GET,HEAD,POST,TRACE} /index.html HTTP/1.1

# определить новый блок с именем "HTTP BASIC".
s_initialize("HTTP BASIC")

# определить групповой примитив со списком команд HTTP для фаззинга.
s_group("verbs", values=["GET", "HEAD", "POST", "TRACE"])

# определить новый блок с именем "body" и связать его с группой выше.
if s_block_start("body", group="verbs"):
    # разбить остаток запроса HTTP на отдельные примитивы.
    s_delim(" ")
    s_delim("/")
    s_string("index.html")
    s_delim(" ")
    s_string("HTTP")
    s_delim("/")
    s_string("1")
    s_delim(".")
    s_string("1")

    # завершить запрос обязательной статической последовательностью.
    s_static("\r\n\r\n")

# закрыть открытый блок, аргумент имени здесь необязателен.
s_block_end("body")
```

Сценарий начинается с импорта всех компонентов Sulley. Далее инициализируется новый запрос с именем HTTP BASIC. Позднее на это имя можно ссылаться для прямого доступа к запросу. Затем определяется группа с именем verbs и возможными строковыми значениями GET, HEAD, POST и

TRACE. Запускается новый блок с именем body, связанный с ранее определённым групповым примитивом посредством необязательного именованного аргумента group. Обратите внимание, что s_block_start() всегда возвращает True, позволяя при необходимости оформлять содержащиеся в нём примитивы отступами с помощью простого условия if. Также обратите внимание, что аргумент имени для s_block_end() необязателен. Эти проектные решения каркаса приняты исключительно из эстетических соображений. Затем внутри границ блока body определяется ряд базовых примитивов разделителей и строк, после чего блок закрывается. Когда определённый запрос загружается в сеанс Sulley, фаззер создаёт и передаёт все возможные значения блока body по одному разу для каждой команды, определённой в группе.

Кодировщики

Кодировщики являются простым, но мощным модификатором блока. Функцию можно указать и подключить к блоку, чтобы изменять отрисованное содержимое этого блока до его возврата и передачи по сети. Лучше всего это объясняется реальным примером. Демон DcsProcessor.exe из Trend Micro Control Manager прослушивает TCP-порт 20901 и ожидает получения данных, отформатированных проприетарной процедурой кодирования XOR. Посредством обратной разработки декодировщика была создана следующая процедура кодирования XOR:

```
def trend_xor_encode (str):
    key = 0xA8534344
    ret = “”

    # дополнить до четырёхбайтной границы.
    pad = 4 - (len(str) % 4)
    if pad == 4:
        pad = 0

    str += “\x00” * pad

    while str:
        dword  = struct.unpack(“<L”, str[:4])[0]
        str    = str[4:]
        dword ^= key
        ret   += struct.pack(“<L”, dword)
        key    = dword

    return ret
```

Кодировщики Sulley принимают один параметр, данные для кодирования, и возвращают кодированные данные. Теперь определённый кодировщик можно подключить к блоку, содержащему фаззимые примитивы, позволяя разработчику фаззера продолжать так, будто этого небольшого препятствия никогда не существовало.

Зависимости

Зависимости позволяют применять условие к отрисовке целого блока. Это выполняется посредством первоначальной связи блока с примитивом, от которого он будет зависеть, с помощью необязательного именованного параметра dep. Когда Sulley приходит время отрисовать зависимый блок, он проверяет значение связанного примитива и ведёт себя соответствующим образом. Зависимое значение можно указать именованным параметром dep_value. В качестве альтернативы список зависимых значений можно указать именованным параметром dep_values.

Наконец, фактическое условное сравнение можно изменить посредством именованного параметра dep_compare. Например, рассмотрим ситуацию, когда в зависимости от значения целого числа ожидаются разные данные:

```
s_short("opcode", full_range=True)

# код операции 10 ожидает последовательность аутентификации.
if s_block_start("auth", dep="opcode", dep_value=10):
    s_string("USER")
    s_delim(" ")
    s_string("pedram")
    s_static("\r\n")
    s_string("PASS")
    s_delim(" ")
    s_delim("fuzzywuzzy")
s_block_end()

# коды операций 15 и 16 ожидают одну строку имени узла.
if s_block_start("hostname", dep="opcode", dep_values=[15, 16]):
    s_string("pedram.openrce.org")
s_block_end()

# остальные коды операций принимают строку с префиксом из двух подчёркиваний.
if s_block_start("something", dep="opcode", dep_values=[10, 15, 16],
                 dep_compare="!="):
    s_static("__")
    s_string("some string")
s_block_end()
```

Зависимости блоков можно соединять в цепочки любым числом способов, создавая мощные, и, к сожалению, сложные, сочетания.

Вспомогательные средства блоков

Важным аспектом генерации данных, с которым необходимо познакомиться для эффективного использования Sulley, являются вспомогательные средства блоков. В эту категорию входят средства размера, контрольные суммы и повторители.

Средства размера

Пользователи SPIKE будут знакомы со вспомогательным средством блока `s_sizer()`, или `s_size()`. Первым параметром оно принимает имя блока, размер которого следует измерить, и принимает следующие дополнительные именованные аргументы:

• `length` (целое, по умолчанию 4): длина поля размера.
• `endian` (символ, по умолчанию `'<'`): порядок байтов битового поля. Укажите `'<'` для порядка от младшего к старшему и `'>'` для порядка от старшего к младшему.
• `format` (строка, по умолчанию `"binary"`): формат вывода, `"binary"` или `"ascii"`, управляющий форматом отрисовки целочисленных примитивов.
• `inclusive` (логическое, по умолчанию `False`): должно ли средство размера учитывать собственную длину?
• `signed` (логическое, по умолчанию `False`): сделать размер знаковым вместо беззнакового, применимо только при `format="ascii"`.
• `fuzzable` (логическое, по умолчанию `False`): включить или отключить фаззинг этого примитива.
• `name` (строка, по умолчанию `None`): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Средства размера являются важнейшим компонентом генерации данных, позволяющим представлять сложные протоколы, такие как нотация XDR, ASN.1 и так далее. При отрисовке средства размера Sulley динамически вычисляет длину связанного блока. По умолчанию Sulley не фаззит поля размера. Во многих случаях это желательное поведение, однако если это не так,

включите флаг fuzzable.

Контрольные суммы

Подобно средствам размера, вспомогательное средство s_checksum() первым параметром принимает имя блока, для которого следует вычислить контрольную сумму. Можно также указать следующие необязательные именованные аргументы:

• algorithm (строка или указатель на функцию, по умолчанию “crc32”): алгоритм контрольной суммы, применяемый к целевому блоку (crc32, adler32, md5, sha1).
• endian (символ, по умолчанию ‘<’): порядок байтов битового поля. Укажите ‘<’ для порядка от младшего к старшему и ‘>’ для порядка от старшего к младшему.
• length (целое, по умолчанию 0): длина контрольной суммы, оставьте 0 для автоматического вычисления.
• name (строка, по умолчанию None): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Аргумент algorithm может иметь значение crc32, adler32, md5 или sha1. В качестве альтернативы для этого параметра можно указать указатель на функцию, чтобы применить специализированный алгоритм контрольной суммы.

Повторители

Вспомогательное средство s_repeat(), или s_repeater(), используется для повторения блока переменное число раз. Это полезно, например, при тестировании на переполнения во время разбора таблиц с несколькими элементами. Это средство принимает три обязательных аргумента: имя повторяемого блока, минимальное число повторений и максимальное число повторений. Дополнительно доступны следующие необязательные именованные аргументы:

• step (целое, по умолчанию 1): шаг между минимальным и максимальным числом повторений.
• fuzzable (логическое, по умолчанию False): включить или отключить фаззинг этого примитива.
• name (строка, по умолчанию None): как и для всех объектов Sulley, указание имени даёт прямой доступ к этому примитиву во всём запросе.

Рассмотрим следующий пример, связывающий все три представленных вспомогательных средства. Мы фаззим часть протокола, содержащую таблицу строк. Каждая запись таблицы состоит из двухбайтного поля типа строки, двухбайтного поля длины, строкового поля и, наконец, поля контрольной суммы CRC-32, вычисленной по строковому полю. Мы не знаем допустимых значений поля типа, поэтому будем фаззить его случайными данными. Вот как эта часть протокола может выглядеть в Sulley:

```
# запись таблицы: [тип][длина][строка][контрольная сумма]
if s_block_start(“table entry”):
    # допустимые типы неизвестны, поэтому заполним поле случайными данными.
    s_random(“\x00\x00”, 2, 2)

    # далее вставить средство размера длиной 2 для следующего строкового поля.
    s_size(“string field”, length=2)

    # вспомогательные средства блоков применимы только к блокам, поэтому заключить
    строковый примитив в блок.
    if s_block_start(“string field”):
        # строкой по умолчанию будет короткая последовательность символов C.
        s_string(“C” * 10)

    s_block_end()
```

```
    # добавить к записи таблицы контрольную сумму CRC-32 строки.
    s_checksum("string field")

s_block_end()

# повторять запись таблицы от 100 до 1000 раз с шагом 50 элементов на каждой
# итерации.
s_repeat("table entry", min_reps=100, max_reps=1000, step=50)
```

Этот сценарий Sulley будет фаззить не только разбор записей таблицы, но также может обнаружить дефект при обработке чрезмерно длинных таблиц.

Lego

Sulley использует lego для представления определённых пользователем компонентов, таких как адреса электронной почты, имена узлов и примитивы протоколов, используемые в Microsoft RPC, XDR, ASN.1 и других. В ASN.1 / BER строки представлены последовательностью `[0x04][0x84][длина dword][строка]`. При фаззинге протокола на основе ASN.1 добавление префиксов длины и типа перед каждой строкой может стать обременительным. Вместо этого можно определить lego и ссылаться на него:

```
s_lego("ber_string", "anonymous")
```

Каждый lego имеет сходный формат, за исключением необязательного именованного аргумента `options`, специфичного для отдельных lego. В качестве простого примера рассмотрим определение lego тега, полезного при фаззинге XML-подобных протоколов:

```
class tag (blocks.block):
    def __init__ (self, name, request, value, options={}):
        blocks.block.__init__(self, name, request, None, None, None, None)

        self.value   = value
        self.options = options

        if not self.value:
            raise sex.error("MISSING LEGO.tag DEFAULT VALUE")

        #
        # [разделитель][строка][разделитель]

        self.push(primitives.delim("<"))
        self.push(primitives.string(self.value))
        self.push(primitives.delim(">"))
```

Этот пример lego просто принимает желаемый тег как строку и заключает его в соответствующие разделители. Он делает это, расширяя класс блока и вручную добавляя разделители тега и предоставленную пользователем строку в блок посредством `self.push()`.

Ниже приведён другой пример, создающий простой lego для представления целых ASN.1 / BER[27] в Sulley. Для представления всех целых был выбран наименьший общий знаменатель, четырёхбайтные целые формы `[0x02][0x04][dword]`, где `0x02` указывает целочисленный тип, `0x04` указывает, что целое имеет длину четыре байта, а dword представляет фактическое передаваемое нами целое. Вот как выглядит определение из `sulley\legos\ber.py`:

```
class integer (blocks.block):
    def __init__ (self, name, request, value, options={}):
        blocks.block.__init__(self, name, request, None, None, None, None)

        self.value   = value
        self.options = options

        if not self.value:
            raise sex.error("MISSING LEGO.ber_integer DEFAULT VALUE")
```

```
        self.push(primitives.dword(self.value, endian=”>”))

    def render (self):
        # предоставить родительскому объекту выполнить первоначальную отрисовку.
        blocks.block.render(self)

        self.rendered = “\x02\x04” + self.rendered

        return self.rendered
```

Подобно предыдущему примеру, предоставленное целое добавляется в стек блока с помощью `self.push()`. В отличие от предыдущего примера, процедура `render()` перегружена, чтобы добавлять перед отрисованным содержимым статическую последовательность `\x02\x04`, удовлетворяя ранее описанным требованиям представления целого. Sulley растёт с созданием каждого нового фаззера. Разработанные блоки и запросы расширяют библиотеку запросов, и на них можно легко ссылаться и использовать при построении будущих фаззеров. Теперь пора посмотреть на построение сеанса.

Сеанс

После определения ряда запросов приходит время связать их в сеанс. Одним из главных преимуществ Sulley над другими каркасами фаззинга является способность фаззить глубоко внутри протокола. Это достигается связыванием запросов в граф. В следующем примере последовательность запросов связывается, а библиотека pgraph, от которой расширяются классы сеанса и запросов, используется для отрисовки графа в формате uDraw, как показано на рисунке 21.2:

```
from sulley import *

s_initialize(“helo”)
s_static(“helo”)

s_initialize(“ehlo”)
s_static(“ehlo”)

s_initialize(“mail from”)
s_static(“mail from”)

s_initialize(“rcpt to”)
s_static(“rcpt to”)

s_initialize(“data”)
s_static(“data”)

sess = sessions.session()

sess.connect(s_get(“helo”))
sess.connect(s_get(“ehlo”))
sess.connect(s_get(“helo”),      s_get(“mail from”))
sess.connect(s_get(“ehlo”),      s_get(“mail from”))
sess.connect(s_get(“mail from”), s_get(“rcpt to”))
sess.connect(s_get(“rcpt to”),   s_get(“data”))

fh = open(“session_test.udg”, “w+”)
fh.write(sess.render_graph_udraw())
fh.close()
```

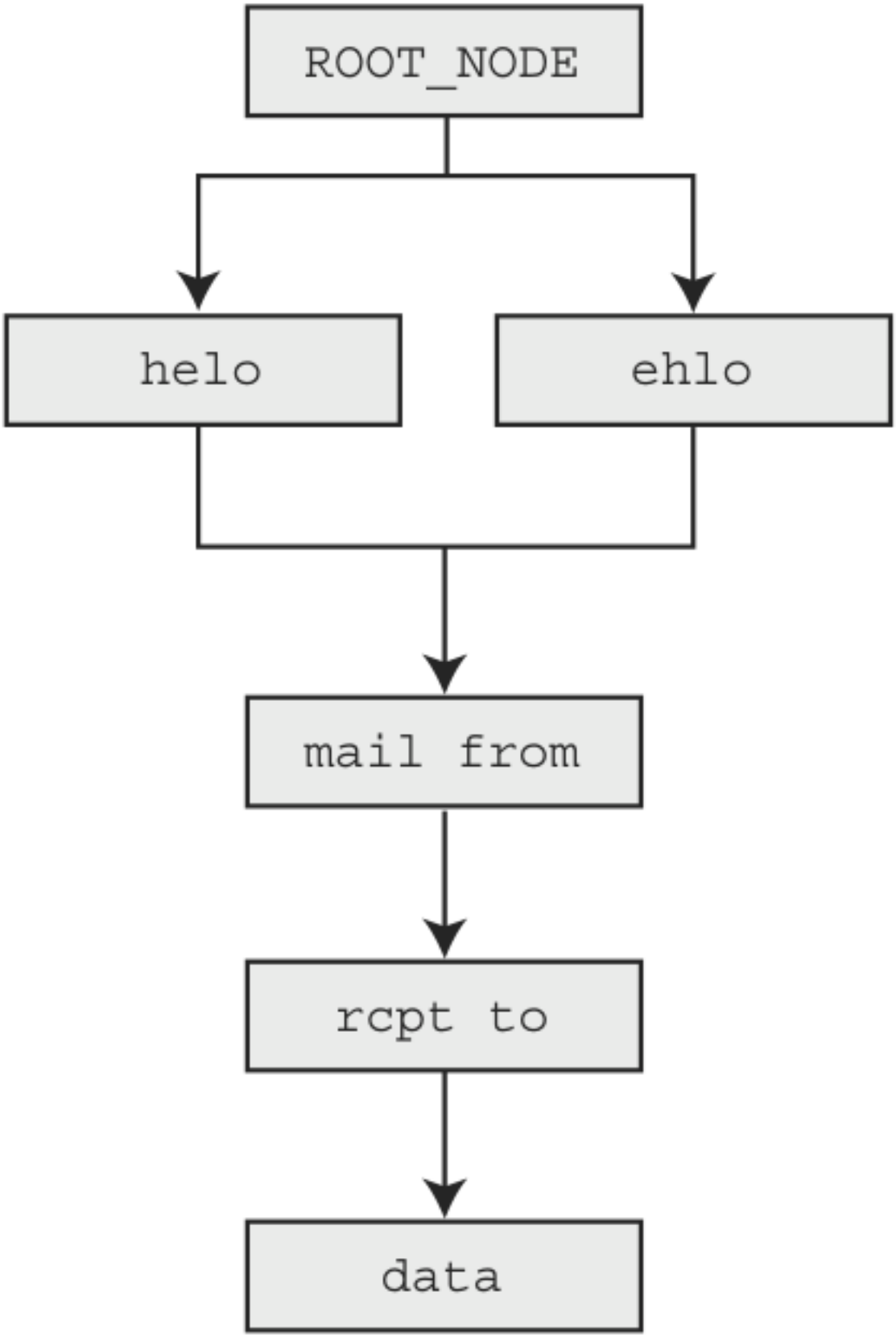


Рисунок 21.2. Пример структуры графа сеанса SMTP

Когда приходит время фаззинга, Sulley обходит структуру графа, начиная с корневого узла и фаззя каждый компонент по пути. В этом примере он начинает с запроса `helo`. После его завершения Sulley начнёт фаззить запрос `mail from`. Для этого перед каждым тестовым случаем он помещает

допустимый запрос helo. Далее Sulley переходит к фаззингу запроса rcpt to. И снова это выполняется посредством добавления перед каждым тестовым случаем допустимых запросов helo и mail from. Процесс продолжается через data, а затем начинается заново по пути ehlo. Возможность разбить протокол на отдельные запросы и фаззить все возможные пути по построенному графу протокола мощна. Рассмотрим, например, проблему, раскрытую в Ipswitch Collaboration Suite в сентябре 2006 года.[28] Дефект программы в данном случае представлял собой переполнение стека во время разбора длинных строк, содержащихся между символами @ и :. Этот случай интересен тем, что уязвимость доступна только по маршруту EHLO, но не HELO. Если фаззер не способен пройти все возможные пути протокола, такие проблемы могут быть упущены.

При создании экземпляра сеанса можно указать следующие необязательные именованные аргументы:

• session_filename (строка, по умолчанию None): имя файла для сериализации постоянных данных. Указание имени файла позволяет останавливать и возобновлять фаззер.
• skip (целое, по умолчанию 0): число пропускаемых тестовых случаев.
• sleep_time (число с плавающей точкой, по умолчанию 1.0): время ожидания между передачами тестовых случаев.
• log_level (целое, по умолчанию 2): уровень журнала, большее число означает больше сообщений журнала.
• proto (строка, по умолчанию “tcp”): протокол взаимодействия.
• timeout (число с плавающей точкой, по умолчанию 5.0): число секунд ожидания возврата send() или recv() до истечения времени ожидания.

Другой продвинутой возможностью Sulley является регистрация обратных вызовов на каждом ребре, определённом в структуре графа протокола. Это позволяет регистрировать функцию для вызова между передачами узлов, реализуя такую функциональность, как системы запрос-ответ. Метод обратного вызова должен соответствовать следующему прототипу:

```
def callback(node, edge, last_recv, sock)
```

Здесь node является узлом, который предстоит отправить, edge последним ребром по текущему пути фаззинга к node, last_recv содержит данные, возвращённые последней передачей через сокет, а sock является действующим сокетом. Обратный вызов полезен также в ситуациях, когда, например, размер следующего пакета указан в первом пакете. В качестве другого примера, если необходимо заполнить динамический IP-адрес цели, зарегистрируйте обратный вызов, получающий IP из sock.getpeername()[0]. Обратные вызовы рёбер можно также регистрировать посредством необязательного именованного аргумента callback метода session.connect().

Цели и агенты

Следующий шаг состоит в определении целей, связывании их с агентами и добавлении целей в сеанс. В следующем примере мы создаём новую цель, работающую внутри виртуальной машины VMWare, и связываем её с тремя агентами:

```
target = sessions.target(“10.0.0.1”, 5168)

target.netmon    = pedrpc.client(“10.0.0.1”,  26001)
target.procmon   = pedrpc.client(“10.0.0.1”,  26002)
target.vmcontrol = pedrpc.client(“127.0.0.1”, 26003)

target.procmon_options = \
{
    “proc_name”      : “SpntSvc.exe”,
    “stop_commands”  : [‘net stop “trend serverprotect”’],
    “start_commands” : [‘net start “trend serverprotect”’],
}
```

```
sess.add_target(target)
sess.fuzz()
```

Созданная цель привязана к TCP-порту 5168 на узле `10.0.0.1`. В целевой системе работает агент мониторинга сети, по умолчанию прослушивающий порт 26001. Сетевой монитор записывает все взаимодействия сокетов в отдельные файлы PCAP, помеченные номером тестового случая. В целевой системе также работает агент мониторинга процесса, по умолчанию прослушивающий порт 26002. Этот агент принимает дополнительные аргументы, указывающие имя процесса для присоединения, команду остановки целевого процесса и команду запуска целевого процесса. Наконец, в локальной системе работает агент управления VMWare, по умолчанию прослушивающий порт 26003. Цель добавляется в сеанс, и фаззинг начинается. Sulley способен фаззить несколько целей, каждая с уникальным набором связанных агентов. Это позволяет сэкономить время, разделив общее пространство тестов между различными целями.

Давайте подробнее рассмотрим функциональность каждого отдельного агента.

Агент: сетевой монитор (network_monitor.py)

Агент сетевого монитора отвечает за отслеживание сетевых взаимодействий и запись их в файлы PCAP на диске. В агенте жёстко запрограммирована привязка к TCP-порту 26001, он принимает соединения от сеанса Sulley по специализированному двоичному протоколу PedRPC. До передачи тестового случая цели Sulley связывается с этим агентом и запрашивает начало записи сетевого трафика. После успешной передачи тестового случая Sulley снова связывается с агентом, запрашивая сброс записанного трафика в файл PCAP на диске. Файлы PCAP именуются по номеру тестового случая для удобного извлечения. Этот агент не обязательно запускать в той же системе, что и целевое программное обеспечение. Однако он должен видеть отправляемый и получаемый сетевой трафик. Агент принимает следующие аргументы командной строки:

```
ERR> USAGE: network_monitor.py
<-d|--device DEVICE #>    device to sniff on (see list below)
[-f|--filter PCAP FILTER] BPF filter string
[-p|--log_path PATH]      log directory to store pcaps to
[-l|--log_level LEVEL]    log level (default 1), increase for more verbosity

Network Device List:
[0] \Device\NPF_GenericDialupAdapter
[1] {2D938150-427D-445F-93D6-A913B4EA20C0}  192.168.181.1
[2] {9AF9AAEC-C362-4642-9A3F-0768CDA60942}  0.0.0.0
[3] {9ADCDA98-A452-4956-9408-0968ACC1F482}  192.168.81.193
...
```

Агент: монитор процесса (process_monitor.py)

Агент монитора процесса отвечает за обнаружение дефектов, которые могут возникнуть в целевом процессе во время фаззингового тестирования. В агенте жёстко запрограммирована привязка к TCP-порту 26002, он принимает соединения от сеанса Sulley по специализированному двоичному протоколу PedRPC. После успешной передачи каждого отдельного тестового случая цели Sulley связывается с агентом, чтобы определить, был ли вызван дефект. Если да, высокоуровневые сведения о характере дефекта передаются обратно сеансу Sulley для отображения через внутренний веб-сервер, подробнее далее. Вызванные дефекты также записываются в сериализованную «корзину аварий» для посмертного анализа. Эта функциональность подробнее исследуется позднее. Агент принимает следующие аргументы командной строки:

```
ERR> USAGE: process_monitor.py
<-c|--crash_bin FILENAME> filename to serialize crash bin class to
```

```
[-p|--proc_name NAME]     process name to search for and attach to
[-i|--ignore_pid PID]     ignore this PID when searching for the target process
[-l|--log_level LEVEL]    log level (default 1), increase for more verbosity
```

Агент: управление VMWare (vmcontrol.py)

В агенте управления VMWare жёстко запрограммирована привязка к TCP-порту 26003, он принимает соединения от сеанса Sulley по специализированному двоичному протоколу PedRPC. Этот агент предоставляет API для взаимодействия с образом виртуальной машины, включая возможность запускать, останавливать, приостанавливать или сбрасывать образ, а также создавать, удалять и восстанавливать снимки. Если обнаружен дефект или цель недоступна, Sulley может связаться с агентом и вернуть виртуальную машину в заведомо исправное состояние. Инструмент уточнения последовательности тестов будет сильно полагаться на этого агента при выполнении задачи выявления точной последовательности тестовых случаев, вызывающей заданный сложный дефект. Агент принимает следующие аргументы командной строки:

```
ERR> USAGE: vmcontrol.py
<-x|--vmx FILENAME>    path to VMX to control
<-r|--vmrun FILENAME>  path to vmrun.exe
[-s|--snapshot NAME>   set the snapshot name
[-l|--log_level LEVEL] log level (default 1), increase for more verbosity
```

Веб-интерфейс мониторинга

Класс сеанса Sulley имеет встроенный минимальный веб-сервер с жёстко запрограммированной привязкой к порту 26000. После вызова метода `fuzz()` класса сеанса порождается поток веб-сервера, и можно видеть ход работы фаззера, включая промежуточные результаты. Пример снимка экрана показан на рисунке 21.3.

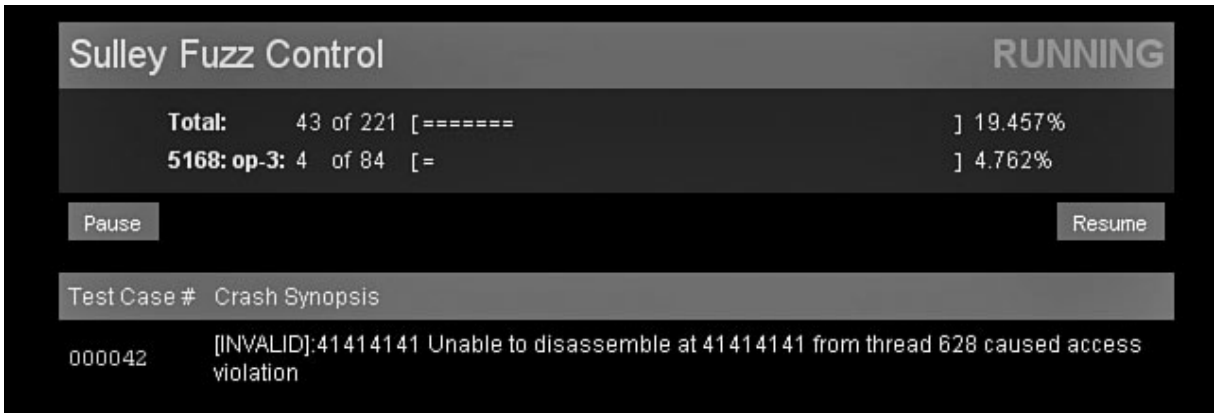


Рисунок 21.3. Веб-интерфейс мониторинга Sulley

Фаззер можно приостанавливать и возобновлять нажатием соответствующих кнопок. Краткое описание каждого обнаруженного дефекта отображается списком, а номер вызвавшего его тестового случая указан в первом столбце. Нажатие номера тестового случая загружает подробный дамп аварии на момент дефекта. Разумеется, эти сведения также доступны в файле корзины аварий и программно. После завершения сеанса приходит время перейти к этапу посмертного анализа и проанализировать результаты.

Посмертный анализ

После завершения сеанса фаззинга Sulley приходит время просмотреть результаты и перейти к этапу посмертного анализа. Встроенный веб-сервер сеанса даст ранние указания на потенциально обнаруженные проблемы, но именно сейчас вы фактически разберёте результаты. В этом процессе вам помогут несколько утилит. Первая называется `crashbin_explorer.py` и принимает следующие аргументы командной строки:

```
$ ./utils/crashbin_explorer.py
USAGE: crashbin_explorer.py <xxx.crashbin>
[-t|--test #]     dump the crash synopsis for a specific test case number
[-g|--graph name] generate a graph of all crash paths, save to ‘name’.udg
```

Например, эту утилиту можно использовать для просмотра каждого места обнаружения дефекта и, кроме того, перечисления номеров отдельных тестовых случаев, вызвавших дефект по этому адресу. Следующие результаты получены при реальном аудите анализатора протокола Jabber в Trillian:

```
$ ./utils/crashbin_explorer.py audits/trillian_jabber.crashbin
[3] ntdll.dll:7c910f29 mov ecx,[ecx] from thread 664 caused access violation
1415, 1416, 1417,
[2] ntdll.dll:7c910e03 mov [edx],eax from thread 664 caused access violation
3780, 9215,
[24] rendezvous.dll:4900c4f1 rep movsd from thread 664 caused access violation
1418, 1419, 1420, 1421, 1422, 1423, 1424, 1425, 3443, 3781, 3782, 3783,
3784, 3785, 3786, 3787, 9216, 9217, 9218, 9219, 9220, 9221, 9222, 9223,
[1] ntdll.dll:7c911639 mov cl,[eax+0x5] from thread 664 caused access violation
3442,
```

Ни одна из перечисленных точек дефекта может не выделяться как очевидно эксплуатируемая проблема. Мы можем глубже изучить конкретику отдельного дефекта, указав номер тестового случая ключом командной строки `-t`. Давайте рассмотрим тестовый случай 1416:

```
$ ./utils/crashbin_explorer.py audits/trillian_jabber.crashbin -t 1416
ntdll.dll:7c910f29 mov ecx,[ecx] from thread 664 caused access violation
when attempting to read from 0x263b7467
CONTEXT DUMP
EIP: 7c910f29 mov ecx,[ecx]
EAX: 039a0318 (  60424984) -> gt;&gt;&gt;...&gt;&gt;&gt;&gt;&gt;(heap)
EBX: 02f40000 (  49545216) ->    PP@ (heap)
ECX: 263b7467 ( 641430631) -> N/A
EDX: 263b7467 ( 641430631) -> N/A

EDI: 0399fed0 (  60423888) -> #e<root><message>&gt;&gt;&gt;...&gt;&gt;& (heap)
ESI: 039a0310 (  60424976) -> gt;&gt;&gt;...&gt;&gt;&gt;&gt;&gt;(heap)
EBP: 03989c38 (  60333112) -> \|gt;&t]IP”Ix;IXIox@ @x@PP8|p|Hg9I P (stack)
ESP: 03989c2c (  60333100) -> \|gt;&t]IP”Ix;IXIox@ @x@PP8|p|Hg9I (stack)
+00: 02f40000 (  49545216) ->    PP@ (heap)
+04: 0399fed0 (  60423888) -> #e<root><message>&gt;&gt;&gt;...&gt;&gt;& (heap)
+08: 00000000 (         0) -> N/A
+0c: 03989d0c (  60333324) -> Hg9I Pt]I@”ImI,IIpHsoIPnIX{ (stack)
+10: 7c910d5c (2089880924) -> N/A
+14: 02f40000 (  49545216) ->    PP@ (heap)
disasm around:
0x7c910f18 jnz 0x7c910fb0
0x7c910f1e mov ecx,[esi+0xc]
0x7c910f21 lea eax,[esi+0x8]
0x7c910f24 mov edx,[eax]
0x7c910f26 mov [ebp+0xc],ecx
0x7c910f29 mov ecx,[ecx]
0x7c910f2b cmp ecx,[edx+0x4]
0x7c910f2e mov [ebp+0x14],edx
0x7c910f31 jnz 0x7c911f21
stack unwind:
ntdll.dll:7c910d5c
rendezvous.dll:49023967
rendezvous.dll:4900c56d
```

```
kernel32.dll:7c80b50b
SEH unwind:
03989d38 -> ntdll.dll:7c90ee18
0398ffdc -> rendezvous.dll:49025d74
ffffffff -> kernel32.dll:7c8399f3
```

И снова ничего очевидного может не выделяться, но мы знаем, что влияем на это конкретное нарушение доступа, поскольку неправильно разыменовываемый регистр ECX содержит строку ASCII “&;tg”. Возможно, проблема расширения строки? Мы можем просматривать места аварий графически, что добавляет ещё одно измерение, отображая известные пути исполнения, с помощью ключа командной строки `-g`. Следующий созданный граф, рисунок 21.4, снова получен при реальном аудите анализатора Jabber в Trillian:

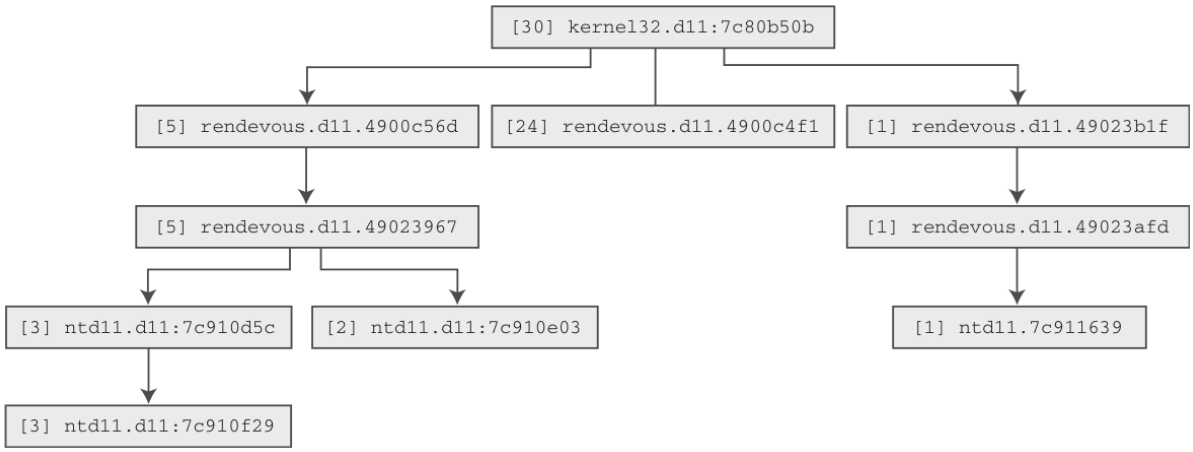


Рисунок 21.4. Графическое исследование корзины аварий Sulley

Мы видим, что, хотя обнаружены четыре разных места аварии, источник проблемы, по-видимому, один. Дальнейшее исследование показывает, что это действительно так. Конкретный дефект существует в подсистеме обмена сообщениями Rendezvous/Extensible Messaging and Presence Protocol (XMPP). Trillian находит находящихся поблизости пользователей посредством `_presence` службы mDNS, многоадресной DNS, на UDP-порту 5353. После регистрации пользователя через mDNS обмен сообщениями выполняется посредством XMPP по TCP-порту 5298. В `plugins\rendezvous.dll` к полученным сообщениям применяется следующая логика:

```
4900C470 str_len:
4900C470     mov cl, [eax]      ; *eax = message+1
4900C472     inc eax
4900C473     test cl, cl
4900C475     jnz short str_len
4900C477     sub eax, edx
4900C479     add eax, 128       ; strlen(message+1) + 128
4900C47E     push eax
4900C47F     call _malloc
```

Длина предоставленной строки сообщения вычисляется, и для хранения копии сообщения выделяется буфер кучи размером длина + 128, который затем передаётся через `expatxml.xmlComposeString()`, функцию, вызываемую со следующим прототипом:

```
plugin_send(MYGUID, “xmlComposeString”, struct xml_string_t *);

struct xml_string_t {
    unsigned int      struct_size;
    char              *string_buffer;
    struct xml_tree_t *xml_tree;
};
```

Процедура `xmlComposeString()` вызывает `expatxml.19002420()`, которая, среди прочего, кодирует для HTML символы &, > и < как `&`, `>` и `<` соответственно. Это поведение видно в

следующем фрагменте дизассемблированного кода:

```
19002492 push 0
19002494 push 0
19002496 push offset str_Amp       ; “&amp;”
1900249B push offset ampersand     ; “&”
190024A0 push eax
190024A1 call sub_190023A0
190024A6 push 0
190024A8 push 0
190024AA push offset str_Lt        ; “&lt;”
190024AF push offset less_than     ; “<”
190024B4 push eax
190024B5 call sub_190023A0
190024BA push
190024BC push
190024BE push offset str_Gt        ; “&gt;”
190024C3 push offset greater_than  ; “>”
190024C8 push eax
190024C9 call sub_190023A0
```

Поскольку первоначально вычисленная длина строки не учитывает это расширение строки, следующая последующая встроенная операция копирования памяти внутри `rendezvous.dll` может вызвать эксплуатируемое повреждение памяти:

```
4900C4EC mov ecx, eax
4900C4EE shr ecx, 2
4900C4F1 rep movsd
4900C4F3 mov ecx, eax
4900C4F5 and ecx, 3
4900C4F8 rep movsb
```

Каждый из дефектов, обнаруженных Sulley, был следствием этой логической ошибки. Отслеживание мест и путей дефектов позволило нам быстро выдвинуть предположение, что за них отвечает один источник. Последним шагом, который мы можем захотеть выполнить, является удаление всех файлов PCAP, не содержащих сведений о дефекте. Утилита `pcap_cleaner.py` написана именно для этой задачи:

```
$ ./utils/pcap_cleaner.py
USAGE: pcap_cleaner.py <xxx.crashbin> <path to pcaps>
```

Эта утилита откроет указанный файл корзины аварий, прочитает список номеров тестовых случаев, вызвавших дефект, и удалит все остальные файлы PCAP из указанного каталога. Чтобы лучше понять, как всё связывается воедино от начала до конца, мы пошагово рассмотрим полный пример реального аудита.

Полное пошаговое руководство

Этот пример затрагивает множество промежуточных и продвинутых понятий Sulley и, надеемся, закрепит ваше понимание каркаса. В руководстве пропущены многие подробности конкретики цели, поскольку главная цель этого раздела состоит в демонстрации использования ряда продвинутых возможностей Sulley. Выбранной целью является Trend Micro Server Protect, в частности конечная точка Microsoft DCE/RPC на TCP-порту 5168, к которой привязана служба `SpntSvc.exe`. Конечная точка RPC предоставляется из `TmRpcSrv.dll` со следующими сведениями заглушки языка определения интерфейсов (IDL):

```
// код операции: 0x00, адрес: 0x65741030
// UUID: 25288888-bd5b-11d1-9d53-0080c83a5c2c
// версия: 1.0

error_status_t rpc_opnum_0 (
    [in] handle_t arg_1,                        // не передаётся по сети
    [in] long trend_req_num,
```

```
    [in][size_is(arg_4)] byte some_string[],
    [in] long arg_4,
    [out][size_is(arg_6)] byte arg_5[],          // не передаётся по сети
    [in] long arg_6
);
```

Ни один из параметров `arg_1` и `arg_6` фактически не передаётся по сети. Это важный факт, который следует учесть позднее, когда мы будем писать собственно фаззинговые запросы. Дальнейшее исследование показывает, что параметр `trend_req_num` имеет особое значение. Верхняя и нижняя половины этого параметра управляют парой таблиц переходов, предоставляющих множество достижимых подпрограмм через эту единственную функцию RPC. Обратная разработка таблиц переходов выявляет следующие сочетания:

• Когда значение верхней половины равно `0x0001`, допустимыми значениями нижней половины являются числа от 1 до 21.
• Когда значение верхней половины равно `0x0002`, допустимыми значениями нижней половины являются числа от 1 до 18.
• Когда значение верхней половины равно `0x0003`, допустимыми значениями нижней половины являются числа от 1 до 84.
• Когда значение верхней половины равно `0x0005`, допустимыми значениями нижней половины являются числа от 1 до 24.
• Когда значение верхней половины равно `0x000A`, допустимыми значениями нижней половины являются числа от 1 до 48.
• Когда значение верхней половины равно `0x001F`, допустимыми значениями нижней половины являются числа от 1 до 24.

Далее необходимо создать специализированную процедуру кодировщика, отвечающую за инкапсуляцию определённых блоков в допустимый запрос DCE/RPC. Номер функции только один, поэтому это просто. Мы определяем базовую обёртку над `utisl.dcerpc.request()`, в которой параметр кода операции жёстко задан равным нулю:

```
# кодировщик запроса DCE/RPC для службы RPC Trend Server Protect 5168.
# opnum всегда равен нулю.
def rpc_request_encoder (data):
    return utils.dcerpc.request(0, data)
```

Построение запросов

Вооружившись этими сведениями и кодировщиком, мы можем начать определять запросы Sulley. Создаём файл `requests\trend.py`, который будет содержать все наши определения запросов и вспомогательных средств Trend, и начинаем писать код. Это превосходный пример преимущества построения запроса фаззера внутри языка, в отличие от специализированного языка, поскольку мы используем циклы Python для автоматического создания отдельного запроса для каждого допустимого верхнего значения из `trend_req_num`:

```
for op, submax in [(0x1, 22), (0x2, 19), (0x3, 85), (0x5, 25), (0xa, 49), (0x1f, 25)]:
    s_initialize("5168: op-%x" % op)

    if s_block_start("everything", encoder=rpc_request_encoder):
        # [in] long trend_req_num,
        s_group("subs", values=map(chr, range(1, submax)))
        s_static("\x00")                # subs фактически является словом с порядком от
    младшего к старшему
        s_static(struct.pack("<H", op))  # код операции

        # [in][size_is(arg_4)] byte some_string[],
        s_size("some_string")

        if s_block_start("some_string", group="subs"):
```

```
            s_static("A" * 0x5000, name="arg3")

        s_block_end()

        # [in] long arg_4,
        s_size("some_string")

        # [in] long arg_6
        s_static(struct.pack("<L", 0x5000)) # размер выходного буфера

    s_block_end()
```

Внутри каждого созданного запроса инициализируется новый блок и передаётся ранее определённому специализированному кодировщику. Далее примитив `s_group()` используется для определения последовательности с именем `subs`, представляющей ранее увиденное значение нижней половины `trend_req_num`. Затем значение слова верхней половины добавляется в поток запроса как статическое. Мы не будем фаззить `trend_req_num`, поскольку выполнили обратную разработку его допустимых значений; если бы мы этого не сделали, можно было бы включить фаззинг и этих полей. Далее в запрос добавляется префикс размера NDR для `some_string`. Здесь можно было бы использовать примитивы lego Sulley DCE/RPC NDR, но поскольку запрос RPC столь прост, мы решаем представить формат NDR вручную. Затем в запрос добавляется значение `some_string`. Строковое значение заключено в блок, чтобы можно было измерить его длину. В данном случае мы используем строку статического размера из символа A, приблизительно 20 тысяч. Обычно здесь вставлялся бы примитив `s_string()`, но поскольку мы знаем, что Trend аварийно завершится с любой длинной строкой, сокращаем набор тестов посредством статического значения. Длина строки снова добавляется в запрос для выполнения требования `size_is` для `arg_4`. Наконец, указываем произвольный статический размер выходного буфера и закрываем блок. Теперь запросы готовы, и мы можем перейти к созданию сеанса.

Создание сеанса

Для сеанса мы создаём в каталоге верхнего уровня Sulley новый файл с именем `fuzz_trend_server_protect_5168.py`. С тех пор этот файл был перемещён в каталог `archived_fuzzies`, поскольку завершил свою жизнь. Прежде всего импортируем Sulley и созданные запросы Trend из библиотеки запросов:

```
from sulley   import *
from requests import trend
```

Далее мы определим функцию предварительной отправки, отвечающую за установление соединения DCE/RPC до передачи любого отдельного тестового случая. Процедура предварительной отправки принимает единственный параметр, сокет для передачи данных. Благодаря доступности `utils.dcerpc.bind()`, вспомогательной процедуры Sulley, эту простую процедуру легко написать:

```
def rpc_bind (sock):
    bind = utils.dcerpc.bind("25288888-bd5b-11d1-9d53-0080c83a5c2c", "1.0")
    sock.send(bind)
    utils.dcerpc.bind_ack(sock.recv(1000))
```

Теперь пора инициировать сеанс и определить цель. Мы будем фаззить одну цель, установку Trend Server Protect внутри виртуальной машины VMWare с адресом `10.0.0.1`. Следуя рекомендациям каркаса, сохраним сериализованные сведения сеанса в каталог `audits`. Наконец, зарегистрируем для определённой цели агенты сетевого монитора, монитора процесса и управления виртуальной машиной:

```
sess   = sessions.session(session_filename="audits/trend_server_protect_5168.session")
target = sessions.target("10.0.0.1", 5168)
```

```
target.netmon    = pedrpc.client("10.0.0.1",  26001)
target.procmon   = pedrpc.client("10.0.0.1",  26002)
target.vmcontrol = pedrpc.client("127.0.0.1", 26003)
```

Поскольку присутствует агент управления VMWare, по умолчанию Sulley будет возвращаться к заведомо исправному снимку при каждом обнаружении дефекта или недоступности цели. Если агент управления VMWare недоступен, но имеется агент монитора процесса, Sulley пытается перезапустить целевой процесс, чтобы возобновить фаззинг. Это выполняется указанием параметров `stop_commands` и `start_commands` агенту монитора процесса:

```
target.procmon_options = \
{
    "proc_name"      : "SpntSvc.exe",
    "stop_commands"  : ['net stop "trend serverprotect"'],
    "start_commands" : ['net start "trend serverprotect"'],
}
```

Параметр `proc_name` обязателен при каждом использовании агента монитора процесса; он указывает имя процесса, к которому должен присоединиться отладчик и в котором следует искать дефекты. Если недоступны ни агент управления VMWare, ни агент монитора процесса, Sulley ничего не остаётся, кроме как просто предоставить цели время на восстановление, если передача данных завершится неудачно.

Далее мы приказываем цели запуститься, вызвав процедуру агента управления VMWare `restart_target()`. После запуска цель добавляется в сеанс, определяется процедура предварительной отправки, и каждый определённый запрос соединяется с корневым узлом фаззинга. Наконец, фаззинг начинается вызовом процедуры `fuzz()` класса сеанса.

```
# запустить цель.
target.vmcontrol.restart_target()
print "virtual machine up and running"

sess.add_target(target)

sess.pre_send = rpc_bind

sess.connect(s_get("5168: op-1"))
sess.connect(s_get("5168: op-2"))
sess.connect(s_get("5168: op-3"))
sess.connect(s_get("5168: op-5"))
sess.connect(s_get("5168: op-a"))
sess.connect(s_get("5168: op-1f"))

sess.fuzz()
```

Настройка среды

Последний шаг перед запуском сеанса фаззинга состоит в настройке среды. Мы выполняем его, запуская образ целевой виртуальной машины, а затем непосредственно внутри тестового образа агенты сетевого монитора и монитора процесса со следующими параметрами командной строки:

```
network_monitor.py -d 1 \
    -f "src or dst port 5168" \
    -p audits\trend_server_protect_5168

process_monitor.py -c audits\trend_server_protect_5168.crashbin \
    -p SpntSvc.exe
```

Оба агента исполняются с подключённого общего ресурса, соответствующего каталогу верхнего уровня Sulley, из которого работает сценарий сеанса. Строка фильтра Berkeley Packet Filter (BPF) передаётся сетевому монитору, чтобы записывались только интересующие нас пакеты. Также выбирается каталог внутри папки `audits`, где сетевой монитор будет создавать PCAP для каждого

тестового случая. Когда оба агента и целевой процесс работают, создаётся действующий снимок с именем `sulley ready and waiting`.

Далее мы выключаем VMWare и запускаем агент управления VMWare в системе-узле, системе фаззинга. Этому агенту требуются путь к исполняемому файлу `vmrun.exe`, путь к фактическому образу для управления и, наконец, имя снимка, к которому следует вернуться при обнаружении дефекта или сбое передачи данных:

```
vmcontrol.py -r “c:\\VMware\vmrun.exe”
    -x “v:\vmfarm\Trend\win_2000_pro.vmx”
    --snapshot “sulley ready and waiting”
```

На старт, внимание, марш! И посмертный анализ

Наконец, мы готовы. Просто запустите `fuzz_trend_server_protect_5168.py`, подключите веб-браузер к http://127.0.0.1:26000, чтобы отслеживать ход работы фаззера, откиньтесь назад, смотрите и наслаждайтесь.

Когда фаззер завершает прохождение списка из 221 тестового случая, мы обнаруживаем, что 19 из них вызвали дефекты. С помощью утилиты `crashbin_explorer.py` можно исследовать дефекты, классифицированные по адресу исключения:

```
$ ./utils/crashbin_explorer.py audits/trend_server_protect_5168.crashbin
[6] [INVALID]:41414141 Unable to disassemble at 41414141 from thread 568 caused
access violation
42, 109, 156, 164, 170, 198,
[3] LogMaster.dll:63272106 push ebx from thread 568 caused access violation
53, 56, 151,
[1] ntdll.dll:77fbb267 push dword [ebp+0xc] from thread 568 caused access
violation
195,
[1] Eng50.dll:6118954e rep movsd from thread 568 caused access violation
181,
[1] ntdll.dll:77facbbd push edi from thread 568 caused access violation
118,
[1] Eng50.dll:61187671 cmp word [eax],0x3b from thread 568 caused access violation
116,
[1] [INVALID]:0058002e Unable to disassemble at 0058002e from thread 568 caused
access violation
70,
[2] Eng50.dll:611896d1 rep movsd from thread 568 caused access violation
152, 182,
[1] StRpcSrv.dll:6567603c push esi from thread 568 caused access violation
106,

[1] KERNEL32.dll:7c57993a cmp ax,[edi] from thread 568 caused access violation
165,
[1] Eng50.dll:61182415 mov edx,[edi+0x20c] from thread 568 caused access violation
50,
```

Некоторые из них являются явно эксплуатируемыми проблемами, например тестовые случаи, приведшие к EIP `0x41414141`. Тестовый случай 70, по-видимому, также наткнулся на возможную проблему исполнения кода, переполнение Unicode, хотя после небольшого дополнительного исследования оно фактически может оказаться обычным переполнением. Утилита исследования корзины аварий также может создать графическое представление обнаруженных дефектов, рисуя пути на основании наблюдаемых обратных трассировок стека. Это способно помочь точно определить первопричину некоторых проблем. Утилита принимает следующие аргументы командной строки:

```
$ ./utils/crashbin_explorer.py
USAGE: crashbin_explorer.py <xxx.crashbin>
[-t|--test #]     dump the crash synopsis for a specific test case number
[-g|--graph name] generate a graph of all crash paths, save to ‘name’.udg
```

Например, мы можем подробнее исследовать состояние ЦП в момент дефекта, обнаруженного в ответ на тестовый случай 70:

```
$ ./utils/crashbin_explorer.py audits/trend_server_protect_5168.crashbin -t 70
[INVALID]:0058002e Unable to disassemble at 0058002e from thread 568 caused access
violation
when attempting to read from 0x0058002e
CONTEXT DUMP
EIP: 0058002e Unable to disassemble at 0058002e
EAX: 00000001 (         1) -> N/A
EBX: 0259e118 (  39444760) -> A.....AAAAA (stack)
ECX: 00000000 (         0) -> N/A
EDX: ffffffff (4294967295) -> N/A
EDI: 00000000 (         0) -> N/A
ESI: 0259e33e (  39445310) -> A.....AAAAA (stack)
EBP: 00000000 (         0) -> N/A
ESP: 0259d594 (  39441812) -> LA.XLT.......MPT.MSG.OFT.PPS.RT (stack)
+00: 0041004c (   4259916) -> N/A
+04: 0058002e (   5767214) -> N/A
+08: 0054004c (   5505100) -> N/A
+0c: 0056002e (   5636142) -> N/A
+10: 00530042 (   5439554) -> N/A
+14: 004a002e (   4849710) -> N/A
disasm around:
0x0058002e Unable to disassemble

SEH unwind:
0259fc58 -> StRpcSrv.dll:656784e3
0259fd70 -> TmRpcSrv.dll:65741820
0259fda8 -> TmRpcSrv.dll:65741820
0259ffdc -> RPCRT4.dll:77d87000
ffffffff -> KERNEL32.dll:7c5c216c
```

Здесь видно, что стек уничтожен строкой расширений файлов, которая выглядит как строка Unicode. Можно также открыть архивный файл PCAP для заданного тестового случая. На рисунке 21.5 показана выдержка снимка экрана Wireshark с исследованием содержимого одного из перехваченных файлов PCAP.

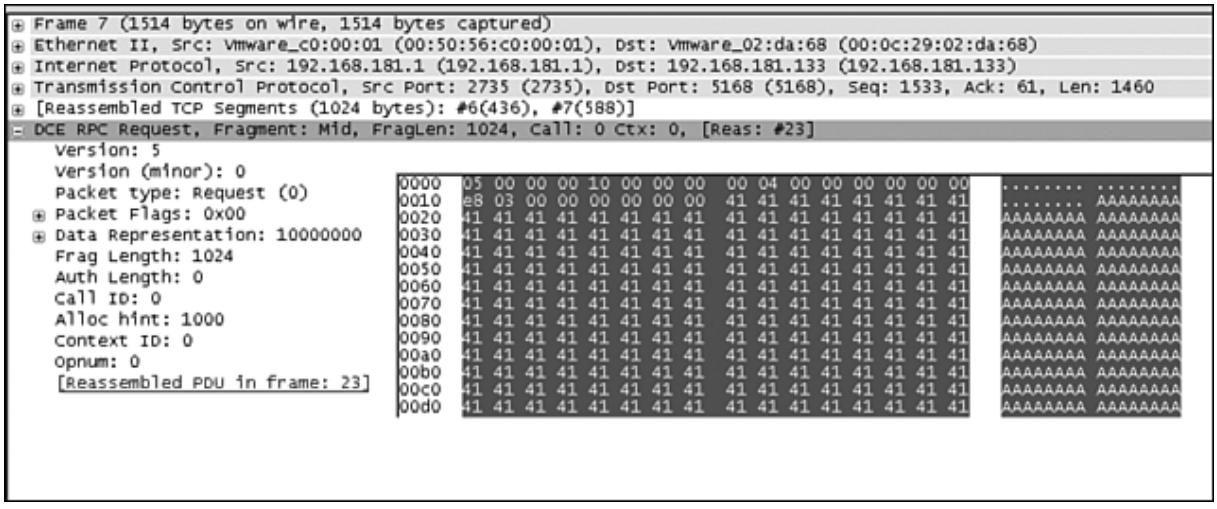


Рисунок 21.5. Анализатор DCE/RPC в Wireshark

Последним шагом, который мы можем захотеть выполнить, является удаление всех файлов PCAP, не содержащих сведений о дефекте. Утилита `pcap_cleaner.py` написана именно для этой задачи:

```
$ ./utils/pcap_cleaner.py
USAGE: pcap_cleaner.py <xxx.crashbin> <path to pcaps>
```

Эта утилита откроет указанный файл корзины аварий, прочитает список номеров тестовых случаев, вызвавших дефект, и удалит все остальные файлы PCAP из указанного каталога. Все

обнаруженные при этом фаззинге уязвимости исполнения кода были сообщены Trend и привели к следующим бюллетеням:

• TSRT-07-01: уязвимости переполнения стека в `StCommon.dll` Trend Micro ServerProtect.
• TSRT-07-02: уязвимости переполнения стека в `eng50.dll` Trend Micro ServerProtect.

Это не означает, что все возможные уязвимости в этом интерфейсе исчерпаны. Фактически это был самый элементарный возможный фаззинг интерфейса. Теперь может быть полезен вторичный фаззинг, действительно использующий примитив `s_string()`, а не просто длинную строку.

Итоги

Каркасы фаззинга предоставляют гибкую, повторно используемую и однородную среду разработки для охотников за ошибками и групп обеспечения качества. После рассмотрения списка доступных каркасов фаззинга в этой главе должно быть довольно очевидно, что явного победителя нет. Все они предоставляют разумный фундамент для построения, но выбор конкретного каркаса обычно будет определяться фаззимой целью и знакомством, или его отсутствием, с языком программирования, используемым конкретным каркасом. Практический пример формата файла SWF Adobe Macromedia Shockwave Flash показывает, что каркасы остаются незрелой технологией и вы всё ещё, вероятно, будете встречать ситуации, когда ни один существующий каркас не способен удовлетворить ваши потребности. Как показано, поэтому иногда может потребоваться разработать специализированное решение фаззинга, используя повторно применимые компоненты, созданные для текущей задачи. Надеемся, этот практический пример навёл вас на размышления и даст некоторые ориентиры, когда вы столкнётесь с собственными экзотическими задачами фаззинга.

В последнем разделе этой главы мы представили и исследовали новый каркас фаззинга Sulley. Были разобраны различные аспекты этого каркаса, чтобы ясно продемонстрировать достигнутые в этой работе усовершенствования. Sulley является последним в становящемся всё длиннее ряду каркасов фаззинга и активно сопровождается авторами этой книги. Актуальные обновления, дополнительную документацию и свежие примеры ищите по адресу http://www.fuzzing.org.

Сноски

[1] [назад] http://en.wikipedia.org/wiki/Asn.1
[2] [назад] http://en.wikipedia.org/wiki/Cyclic_redundancy_check
[3] [назад] http://www.dnp.org/
[4] [назад] http://antiparser.sourceforge.net/
[5] [назад] http://www.genexx.org/dfuz/
[6] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[7] [назад] http://www.gnu.org/copyleft/gpl.html
[8] [назад] http://www.immunitysec.com/downloads/advantages_of_block_based_analysis.pdf
[9] [назад] http://www.cygwin.com/
[10] [назад] http://peachfuzz.sourceforge.net
[11] [назад] http://en.wikipedia.org/wiki/Component_Object_Model
[12] [назад] http://www.appliedsec.com/resources.html
[13] [назад] http://labs.idefense.com/intelligence/vulnerabilities/display.php?id=287
[14] [назад] http://www.ethereal.com
[15] [назад] http://www.wireshark.org
[16] [назад] http://autodafe.sourceforge.net
[17] [назад] http://autodafe.sourceforge.net/docs/autodafe.pdf
[18] [назад] http://www.macromedia.com/software/flash/about/
[19] [назад] Это исследование в значительной степени и активно проводится Аароном Портноем из TippingPoint.
[20] [назад] http://www.adobe.com/licensing/developer/
[21] [назад] И вновь: код см. по адресу http://www.fuzzing.org.
[22] [назад] http://www.google.com/apis/index.html
[23] [назад] http://openrce.org/downloads/details/208/PaiMei

[24] [назад] http://en.wikipedia.org/wiki/ActionScript
[25] [назад] http://www.adobe.com/products/studio/
[26] [назад] http://www.pixar.com/featurefilms/inc/chars_pop1.html
[27] [назад] http://luca.ntop.org/Teaching/Appunti/asn1.html
[28] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-028.html

Глава 22. Автоматизированный разбор протокола

> «Я знаю, как трудно вам положить еду на свою семью».
>
> Джордж У. Буш, Большой Нашуа, штат Нью-Гэмпшир, 27 января 2000 года

В предыдущих главах мы внимательно рассмотрели широкий спектр фаззеров. От простых обобщённых фаззеров файлов, изменяющих байты, до самых развитых в настоящее время каркасов фаззинга, технологии фаззинга преодолевают ряд препятствий автоматизированного тестирования программного обеспечения. В этой главе мы представляем самые продвинутые формы фаззинга и попытки разрешить дилемму, общую для всех технологий фаззинга. В частности, мы берёмся за неукротимую задачу разбиения протокола на основные строительные блоки.

Глава начинается с обсуждения примитивных методов, нацеленных на автоматизацию разбора протокола, а затем переходит к применению биоинформатики и генетических алгоритмов. На переднем крае области автоматизированного тестирования программного обеспечения эти предметы в значительной мере теоретичны и сейчас активно исследуются рядом людей.

В чём проблема фаззинга?

Единственным самым болезненным аспектом фаззингового тестирования является порог входа, особенно при работе с недокументированными сложными двоичными протоколами, понимание которых требует значительного исследования. Представьте задачу фаззинга протокола Microsoft SMB до 1992 года, когда была выпущена первая версия Samba.[1] С тех пор проект Samba созрел в полноценный и совместимый с Windows клиент-сервер SMB. Это стало возможным благодаря бесчисленным человеко-часам добровольцев. Есть ли у вас сейчас столь большая группа и более десяти лет времени для фаззинга подобного недокументированного проприетарного протокола?

> **SAMBA**
>
> SAMBA зародилась 10 января 1992 года, когда Эндрю Триджелл из Австралийского национального университета опубликовал сообщение в группе новостей `vmsnet.networks.desktop.pathworks`.[2] В этом сообщении он объявил о доступности службы общего доступа к файлам UNIX, совместимой с Pathworks for DOS. Этот проект стал известен как nbserver, но в апреле 1994 года название изменили на Samba, когда Триджелл получил письмо от Syntax Corp. с жалобой на возможное нарушение товарного знака.[3] Полтора десятилетия спустя Samba превратилась в один из самых широко используемых проектов с открытым исходным кодом за всю историю.

Фаззить собственные протоколы легко. Вы знакомы с форматом, нюансами, сложностями и даже с тем, какие области могли программировать почти без сна и хотите проверить особенно тщательно. Фаззить документированный протокол, такой как HTTP, тоже сравнительно легко. Для справки доступны подробные RFC[4] и другие общедоступные документы. Создание эффективной и тщательной серии тестовых случаев является простой задачей ввода данных. Однако будьте осторожны: даже распространённость и документированность протокола не гарантируют, что

поставщик программного обеспечения строго следовал опубликованным стандартам. Рассмотрим реальный пример Ipswitch IMail. В сентябре 2006 года была открыто раскрыта удалённо эксплуатируемая уязвимость переполнения стека,[5] затрагивающая демон SMTP Ipswitch. Дефект существует из-за отсутствия проверки границ при разборе длинных строк, содержащихся между символами `@` и `:`. При просмотре нужной области двоичного файла проблему заметить несложно. Однако, поскольку уязвимая функциональность является проприетарной возможностью Ipswitch и не документирована ни в одном стандарте, вероятность её обнаружения фаззером меньше.

Фаззинг проприетарного протокола стороннего поставщика, каким бы простым он ни был, в лучшем случае может быть сложным. Чисто случайный фаззинг реализовать быстрее всего, но результаты далеки от интеллектуальных. Другие обобщённые методы, такие как инверсия байтов, обеспечивают лучшее тестирование, но требуют дополнительного шага: сначала необходимо наблюдать допустимый трафик между клиентом и сервером. К счастью, в большинстве случаев вызвать взаимодействие между произвольной парой клиент-сервер несложно. Однако невозможно с уверенностью утверждать, какой процент протокола остался ненаблюдавшимся. Например, возьмём HTTP без каких-либо предположений о предварительных знаниях. Кратковременное наблюдение за трафиком, вероятно, выявит действия GET и POST. Однако менее распространённые действия, такие как HEAD, OPTIONS и TRACE, наблюдаться будут с меньшей вероятностью. Как выявить реже используемые части произвольного протокола? Для лучшего разбора целевого протокола специалиста по обратной разработке можно натравить на двоичные файлы клиента и сервера. Это дорогостоящий шаг, поскольку опытные специалисты по обратной разработке обладают высокой квалификацией, и найти их трудно. В некоторых случаях, особенно для протоколов с обычным текстом, дополнительную функциональность иногда можно угадать. С двоичным протоколом такая роскошь доступна реже.

Прежде чем нести расходы на найм или назначение квалифицированного специалиста по обратной разработке, обязательно воспользуйтесь работой других. Мир велик, и, вероятно, если вам нужны подробности проприетарного протокола, кто-то ещё сталкивался с тем же разочарованием. Прежде чем погружаться слишком глубоко, обязательно спросите своего друга Google, что он знает о проблеме. Вы можете приятно удивиться, обнаружив неофициальную документацию, созданную другим человеком, уже принявшим тот же вызов. Wotsit.org также является прекрасным ресурсом по форматам файлов, как документированным, так и недокументированным. Для сетевых протоколов исходный код Wireshark и Ethereal также предоставляет хорошую отправную точку, поскольку включает определения протоколов для многих более известных проприетарных протоколов, которые уже были тщательно разобраны.

Когда бремя расшифровки проприетарного протокола ложится исключительно на ваши плечи, крайне желателен подход с меньшей интенсивностью ручного труда. Эта глава посвящена различным методологиям, способным помочь автоматизировать процесс обнаружения сигналов, то есть структуры, среди шума, то есть данных, внутри сетевых протоколов и форматов файлов.

Эвристические методы

Два метода, представленные в этом разделе, не являются истинными формами автоматизированного разбора протоколов. Эти основанные на эвристиках методы скорее можно применять для улучшения автоматизации и производительности фаззинга. Они представлены здесь в начале главы до углубления в более сложные методологии.

Фаззинг через прокси

Первый обсуждаемый нами основанный на эвристиках метод разбора реализован в частным образом разработанном[6] механизме фаззинга с подходящим названием ProxyFuzzer. В типичной сетевой модели клиент-сервер между клиентом и сервером происходит прямое взаимодействие, как показано на рисунке 22.1.



Рисунок 22.1. Типичный путь взаимодействия клиент-сервер

Как следует из названия, прокси-фаззер должен разорвать соединение между клиентом и сервером, чтобы расположиться в качестве ретранслятора. Для эффективного выполнения этой задачи и клиент, и сервер необходимо вручную настроить[7] искать друг друга по адресу прокси. Иными словами, клиент видит прокси как сервер, а сервер видит прокси как клиент. Новая сетевая схема со вставленным фаззером показана на рисунке 22.2.

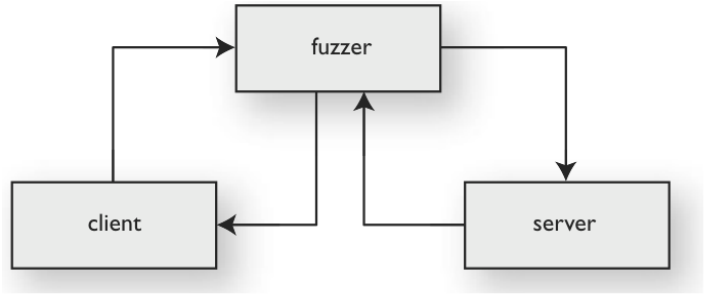


Рисунок 22.2. Прозрачный встроенный прокси

Сплошные соединительные линии между узлами на рисунке 22.2 представляют исходные неизменённые данные, которые на этом этапе просто перенаправлены через прокси. В режиме по умолчанию ProxyFuzzer вслепую ретранслирует полученный трафик на обоих концах, одновременно записывая транзакции в формате, подходящем для программного изменения или повторного воспроизведения. Это само по себе удобная возможность, поскольку позволяет исследователям пропустить промежуточные шаги, связанные с созданием и изменением файлов необработанного захвата пакетов (PCAP). В качестве отступления, изменение и повторное воспроизведение записанных транзакций является превосходным ручным подходом к обратной разработке неизвестных протоколов и методом, хорошо реализованным в Protocol Debugger (PDB) от Matasano.[8]

После успешной настройки ProxyFuzzer и подтверждения его работоспособности можно включить механизм автоматической мутации. В этом режиме ProxyFuzzer обрабатывает ретранслируемый трафик в поисках компонентов обычного текста. Он делает это, выявляя байты, попадающие в допустимый диапазон ASCII, в последовательностях длиннее указанного порога. После выявления обнаруженные строки либо случайно удлиняются, либо изменяются недопустимыми символами и токенами строки формата. На рисунке 22.3 изменённые потоки изображены штриховыми линиями.

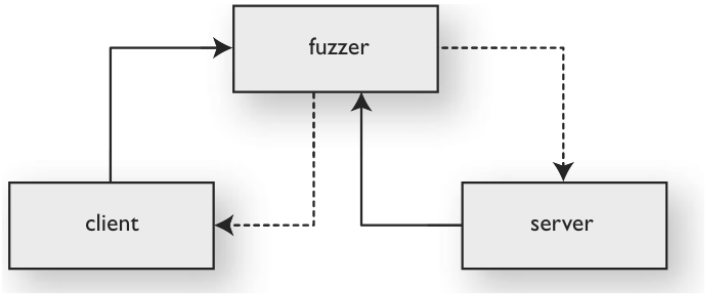


Рисунок 22.3. Автономная встроенная мутация

Несмотря на простоту, ProxyFuzzer чрезвычайно лёгок в использовании, помогает ручному анализу и даже уже обнаруживал уязвимости безопасности в нынешней форме! Возможно, величайшим преимуществом встроенного, прокси, фаззинга является то, что основная часть протокола остаётся нетронутой, пока изменяются отдельные поля. Сохранение большей части передачи данных неизменной позволяет успешно фаззить при малом объёме анализа. Рассмотрим, например, сложные протоколы, реализующие порядковые номера на уровне данных. Недопустимый порядковый номер приводит к немедленному обнаружению повреждения протокола, и оставшиеся данные, включая фаззенное поле, не разбираются. Расположившись внутри линии, фаззер может использовать допустимые транзакции и создавать мутации, достаточно допустимые, чтобы достичь анализатора.

Улучшенный фаззинг через прокси

Сейчас разрабатываются усовершенствования, призванные сделать ProxyFuzzer «умнее». Для помощи в автоматизированном обнаружении полей и последующей мутации рассматривались дополнительные эвристики. Текущий анализатор трафика способен изолировать поля обычного текста от остальной части двоичного протокола. Выявленные поля обычного текста следует обрабатывать дополнительно, ища распространённые разделители и ограничители полей, такие как пробелы, двоеточия и так далее. Кроме того, необработанные байты перед обнаруженными строками можно исследовать в поисках спецификаций длины и типа строки, то есть полей TLV. Возможные префиксы длины затем можно проверить перекрёстным исследованием ряда пакетов. Необработанные байты после строк также можно изучать для выявления дополнения статических полей. IP-адреса и клиента, и сервера часто появляются внутри потока данных, за пределами заголовков TCP/IP. Поскольку эти сведения известны, механизм анализа может сканировать сетевые данные в поисках и необработанного, и ASCII-представления.

Предполагая, что большинство простых протоколов можно описать как последовательность полей, относящихся к одной из категорий, показанных на рисунке 22.4, мы можем начать моделировать иерархию неизвестного протокола посредством ряда догадок и проверок.

РЕЗУЛЬТАТЫ PROXYFUZZER

ProxyFuzzer элементарен даже по сравнению с самыми базовыми фаззерами из предыдущих глав. Тем не менее он уже доказал эффективность против популярного, и явно плохо написанного, корпоративного программного обеспечения. Совершенно без настройки или помощи ProxyFuzzer обнаружил две удалённо эксплуатируемые уязвимости в программном обеспечении резервного копирования Computer Associates Brightstor.

Первая проблема проявляется во время транзакции после аутентификации, что ограничивает тяжесть воздействия. При стандартном взаимодействии клиент-сервер по TCP-порту 6050 длинное имя файла вызывает тривиально эксплуатируемую уязвимость

переполнения стека в `UnivAgent.exe`.

Вторая проблема существует в демоне HTTP `igateway.exe`, взаимодействующем по TCP-порту 5250. Она проявляется только при включённом флаге отладки, что ограничивает тяжесть воздействия. Запрошенное имя файла небезопасно передаётся через вызов `fprintf()`, предоставляя эксплуатируемую уязвимость строки формата.

Ни одна из обнаруженных проблем не особенно «сексуальна» на фоне множества элегантных уязвимостей. И всё же, учитывая минимальные усилия, потребовавшиеся для выявления этих ошибок, возможно, ProxyFuzzer всегда следует давать первую попытку.

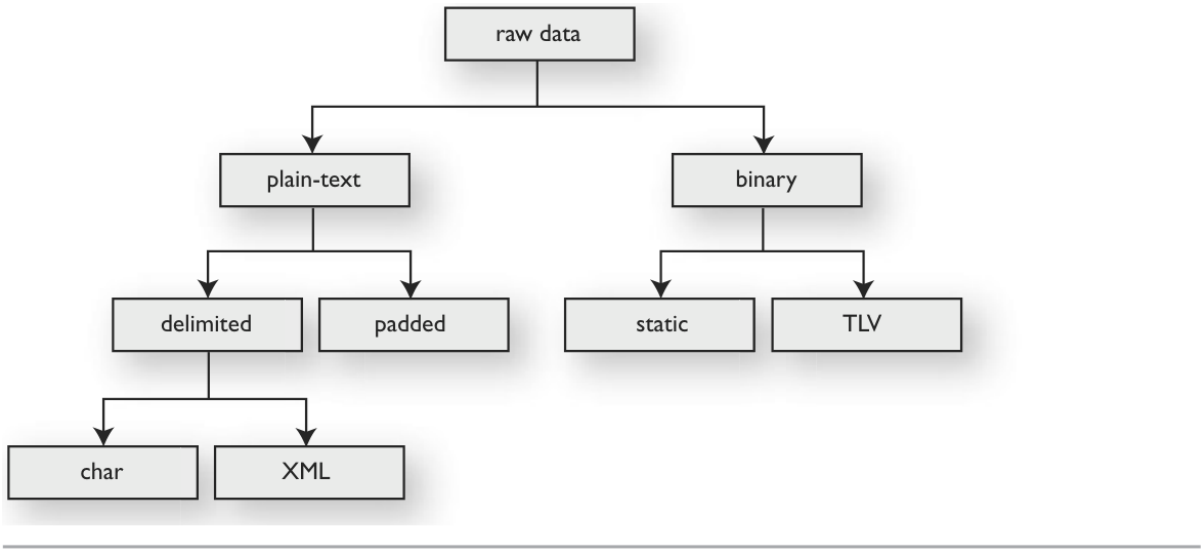


Рисунок 22.4. Иерархический разбор протокола

Этот элементарный метод, вероятно, принесёт мало пользы при разборе сложных двоичных протоколов, таких как SMB. Однако конечная цель здесь состоит не в проведении полного разбора протокола, а в расширении диапазона простых протоколов, подходящих для автоматического фаззинга. В качестве прикладного примера рассмотрим следующие искусственные сетевые данные, показанные по одной транзакции на строку, где необработанные шестнадцатеричные байты заключены между символами вертикальной черты (`|`):

```
|00 04|user|00 06|pedram|0a 0a 00 01|code rev 254|00 00 00 00 be ef|END
|00 04|user|00 04|cody|0a 0a 00 02|code rev 11|00 00 00 00 00 de ad|END
|00 04|user|00 05|aaron|0a 0a 00 03|code rev 31337|00 00 00 c0 1a|END
```

Человек, а возможно, даже дельфин, может легко бегло взглянуть на эти три транзакции и быстро сделать обоснованные догадки о конкретике протокола. Давайте посмотрим, что способен выявить улучшенный автоматический анализ, исследуя только первую транзакцию. Сначала быстро обнаруживается и закрепляется поле четырёхбайтного IP-адреса (`0a 0a 00 01` = частный IP-адрес `10.10.0.1`). Далее обнаруживаются четыре строки ASCII: `user`, `pedram`, `code rev 254` и `END`. Байтовые значения перед каждой строкой сканируются в поисках допустимого однобайтного значения длины, затем двухбайтного, трёхбайтного и, наконец, четырёхбайтного. Теперь процедура анализа может предположить, что протокол начинается с двух строк переменной длины, перед которыми стоит двухбайтная длина, а далее следует IP-адрес подключающегося клиента. Сканирование следующей строки в поисках префикса длины не даёт результатов. Предположив, что следующее поле ASCII имеет статический размер, механизм ищет и находит четыре завершающих нулевых байта. Следующее поле предположительно является полем ASCII статического размера 10 байт. Последняя строка, `END`, не содержит префикса длины, и за ней не следует дополнение. Предполагается, что это поле ASCII статического размера три байта.

О поле, содержащем оставшиеся два байта, `0xbeef`, предположить ничего нельзя. Оно может иметь статическую или переменную длину, поэтому требуется дальнейший анализ. Собрав всё вместе, механизм смог вывести из первой транзакции следующую структуру протокола:

• Двухбайтная длина строки, за которой следует строка.
• Двухбайтная длина строки, за которой следует строка.
• Четырёхбайтный IP-адрес клиента.
• Поле ASCII статической длины 10 байт с нулевым дополнением.
• Неизвестное двоичное поле статической или переменной длины.
• Поле ASCII статической длины три байта.

Теперь механизм переходит к последующим транзакциям, чтобы проверить догадки, сделанные при первоначальном анализе. Так одни предположения подтверждаются, другие опровергаются, и при этом точность смоделированного протокола продолжает повышаться. В отличие от других упрощённых примеров, этот основанный на эвристиках подход потерпит неудачу на более сложных протоколах. Помните, что цель заключалась не в полном понимании лежащего в основе протокола, а в улучшении фаззинга без помощи.

Эвристики дизассемблирования

Применение эвристик на уровне ассемблера, или дизассемблированного кода, для повышения эффективности фаззера является концепцией, которая ещё не использовалась ни одним общедоступным инструментом или каркасом фаззинга. Концепция проста: во время фаззинга отслеживать исполнение кода цели посредством инструмента динамического инструментирования, такого как отладчик. В отладчике искать сравнения со статическими строками и целыми числами. Передавать эти сведения обратно фаззеру для рассмотрения вопроса об их включении в будущие создаваемые тестовые случаи. Результаты будут различаться от случая к случаю, однако фаззер, несомненно, способен создавать более умные данные, правильно используя эту петлю обратной связи. Рассмотрим следующую выдержку кода из реального промышленного серверного программного обеспечения:

```
0040206C call ds:__imp__sscanf
00402072 mov eax, [esp+5DA4h+var_5CDC]
00402079 add esp, 0Ch
0040207C cmp eax, 3857106359       ; string prefix check
00402081 jz  short loc_40208D
00402083 push offset `string'      ; "access protocol error"
00402088 jmp loc_401D61
```

Необработанные сетевые данные преобразуются в целое с помощью API `sscanf()`. Полученное целое затем сравнивается со статическим целым значением `3857106359`, и, если значения не совпадают, анализатор протокола возвращается из-за «ошибки протокола доступа». Когда фаззер в первый раз проходит этот блок кода, компонент отладчика обнаруживает «магическое» значение и передаёт его через петлю обратной связи. Затем фаззер может включить значение в нескольких форматах в надежде правильно пощекотать цель и открыть дополнительное покрытие кода. Без петли обратной связи фаззер оказывается совершенно беспомощным.

Реализация для подтверждения концепции отладчика обратной связи фаззера, отслеживающего цель, была разработана чуть более чем за пару часов с помощью компонента PyDbg каркаса обратной разработки PaiMei.[9] Соответствующий исходный код ищите на http://www.fuzzing.org.

Рассмотрев более примитивные методы, далее мы изучим более продвинутый подход к автоматизации разбора, представив биоинформатику.

Биоинформатика

Wikipedia определяет биоинформатику, или вычислительную биологию, как широкий термин, обобщающий использование «прикладной математики, информатики, статистики, компьютерных наук, искусственного интеллекта, химии и биохимии для решения биологических задач, обычно на молекулярном уровне».[10] По существу, биоинформатика охватывает различные методы, применимые для обнаружения закономерностей в длинных последовательностях сложных, но структурированных данных, таких как последовательности генов. Сетевые протоколы также можно рассматривать как длинные последовательности сложных, но структурированных данных. Могут ли поэтому испытатели программного обеспечения заимствовать методы у биологов, чтобы упростить фаззинг?

Самый базовый анализ в биоинформатике представляет собой способность выровнять две последовательности независимо от длины, максимально повышая сходство. Во время выравнивания в любую последовательность можно вставлять пробелы для достижения этой цели. Рассмотрим следующее выравнивание последовательностей аминокислот:

```
Последовательность один: ACAT   TACAGGA
Последовательность два:  ACATTCCTACAGGA
```

В первую последовательность были вставлены три пробела, чтобы принудительно выровнять длину и максимально повысить сходство. Существует ряд алгоритмов для выполнения этой и других задач выравнивания последовательностей.

Один такой алгоритм, алгоритм Нидлмана-Вунша (Needleman-Wunsch, NW),[11] впервые был предложен в 1970 году Солом Нидлманом[12] и Кристианом Вуншем.[13] Алгоритм NW обычно применяется в биоинформатике для выравнивания двух последовательностей белков или нуклеотидов. Этот алгоритм является примером «динамического программирования», метода решения крупных задач посредством разбиения их на ряд меньших.

Возможно, первая публичная демонстрация применения теорем из биоинформатики к миру анализа сетевых протоколов состоялась в конце 2004 года на хакерской конференции ToorCon[14] в Сан-Диего, штат Калифорния. На конференции Маршалл Беддо представил экспериментальный каркас Python под названием Protocol Informatics (PI), который произвёл настоящий фурор и даже удостоился статьи в Wired.[15] Первоначальный веб-сайт с тех пор был закрыт, а каркас активно не сопровождался. Поскольку автор PI был нанят компанией анализа безопасности Mu Security,[16] некоторые предполагают, что технология больше не находится в открытом доступе, уступая место коммерческому предложению. К счастью, копия кода PI бета-качества всё ещё доступна для загрузки с Packet Storm.[17]

Цель каркаса PI заключается в автоматическом выведении границ полей произвольных протоколов посредством анализа больших объёмов наблюдавшихся данных. Было продемонстрировано, что PI успешно выявляет поля протоколов HTTP, ICMP и SMB. Он делает это с помощью алгоритма локального выравнивания последовательностей Смита-Уотермана[18] (Smith-Waterman, SW), алгоритма глобального выравнивания последовательностей NW, матриц сходства и филогенетических деревьев. Хотя тонкие подробности биоинформатических наук, применяемых каркасом PI, выходят за рамки этой книги, здесь представлены высокоуровневый обзор и как минимум названия используемых алгоритмов.

Сетевые протоколы могут содержать ряд резко различающихся типов сообщений. Попытка выровнять последовательности разных типов сообщений бесплодна. Для решения этой проблемы сначала к паре последовательностей применяется алгоритм SW локального выравнивания последовательностей, чтобы найти и сопоставить сходные подпоследовательности. Эти места и сопоставленные пары позднее используются для помощи глобальному выравниванию NW. Матрицы сходства используются для помощи алгоритму NW в оптимизации выравнивания

последовательностей. Две часто используемые матрицы, Percent Accepted Mutation (PAM) и Blocks Substitution Matrix (BLOSUM), применяются каркасом PI для более подходящего выравнивания последовательностей на основании типов данных. На практике это обобщается до выравнивания двоичных данных с другими двоичными данными, а данных ASCII с другими данными ASCII. Такое различение позволяет точнее выявлять поля переменной длины внутри структуры протокола. Каркас PI идёт ещё дальше, выполняя множественное выравнивание последовательностей для лучшего понимания целевого протокола. Чтобы обойти вычислительную неосуществимость прямого применения NW, алгоритм Unweighted Pairwise Mean by Arithmetic Averages (UPGMA) используется для создания филогенетического дерева, которое затем служит эвристическим руководством при выполнении множественных выравниваний.

Давайте посмотрим на каркас в действии, исследовав его способность разбирать простой протокол фиксированной длины, ICMP.[19] Ниже приведены основные шаги использования каркаса PI для разбора ICMP. Сначала необходимо собрать несколько пакетов ICMP для анализа:

```
# tcpdump -s 42 -c 100 -nl -w icmp.dump icmp
```

Далее перехваченный трафик можно пропустить через каркас PI:

```
# ./main.py -g -p ./icmp.dump
Protocol Informatics Prototype (v0.01 beta)
Written by Marshall Beddoe <mbeddoe@baselineresearch.net>
Copyright (c) 2004 Baseline Research

Found 100 unique sequences in '../dumps/icmp.out'

Creating distance matrix .. complete
Creating phylogenetic tree .. complete
Discovered 1 clusters using a weight of 1.00

Performing multiple alignment on cluster 1 .. complete

Output of cluster 1
0097 x08 x00 xad x4b x05 xbe x00 x60
0039 x08 x00 x30 x54 x05 xbe x00 x26
0026 x08 x00 xf7 xb2 x05 xbe x00 x19
0015 x08 x00 x01 xdb x05 xbe x00 x0e
0048 x08 x00 x4f xdf x05 xbe x00 x2f
0040 x08 x00 xf8 xa4 x05 xbe x00 x27
0077 x08 x00 xe8 x28 x05 xbe x00 x4c
0017 x08 x00 xe8 x6c x05 xbe x00 x10
0027 x08 x00 xc3 xa9 x05 xbe x00 x1a
0087 x08 x00 xdd xc1 x05 xbe x00 x56

0081 x08 x00 x88 x42 x05 xbe x00 x50
0058 x08 x00 xb0 x42 x05 xbe x00 x39
0013 x08 x00 x3e x38 x05 xbe x00
0067 x08 x00 x99 x36 x05 xbe x00 x42
0055 x08 x00 x0f x56 x05 xbe x00 x36
0004 x08 x00 xe6 xda x05 xbe x00 x03
0028 x08 x00 x83 xd9 x05 xbe x00 x1b
0095 x08 x00 xc1 xd9 x05 xbe x00 x5e
0093 x08 x00
     xb6 x05 xbe x00 x5c

[ output trimmed for sake of brevity ]

0010 x08 x00 xd1 xb6 x05 xbe x00
0024 x08 x00 x11 x8f x05 xbe x00 x17
0063 x08 x00 x11 x04 x05 xbe x00 x3e
0038 x08 x00 x37 x3b x05 xbe x00 x25

DT
BBB ZZZ BBB BBB BBB BBB ZZZ AAA

MT
000 000 081 089 000 000 000 100
```

```
Ungapped Consensus:
CONS x08 x00 x3f x18 x05 xbe x00 ???

DT
BBB ZZZ BBB BBB BBB BBB ZZZ AAA

MT
000 000 081 089 000 000 000 100

Step 3: Analyze Consensus Sequence
Pay attention to datatype composition and mutation rate.

Offset 0: Binary data, 0% mutation rate
Offset 1: Zeroed data, 0% mutation rate
Offset 2: Binary data, 81% mutation rate
Offset 3: Binary data, 89% mutation rate
Offset 4: Binary data, 0% mutation rate
Offset 5: Binary data, 0% mutation rate
Offset 6: Zeroed data, 0% mutation rate
Offset 7: ASCII data, 100% mutation rate
```

Хотя это не очевидно сразу, результаты созданного анализа преобразуются в следующую структуру протокола:

```
[ 1 байт ] [ 1 байт ] [ 2 байта ] [ 2 байта ] [ 1 байт ] [ 1 байт ]
```

Результаты недалеки от действительной структуры протокола:

```
[ 1 байт ] [ 1 байт ] [ 2 байта ] [ 2 байта ] [ 2 байта ]
```

Ошибка при выявлении последнего поля вызвана ограниченным числом пакетов ICMP, перехваченных для анализа. Ошибочно выявленное поле фактически является 16-битным порядковым номером. Поскольку использовалось лишь 100 пакетов, старший значащий байт поля ни разу не был увеличен.

Если бы PI передали больше данных, размер поля был бы выявлен правильно.

Применение биоинформатики к автоматизированному разбору протокола представляет собой очень интересный и передовой подход. В настоящее время результаты ограниченны, и ряд исследователей скептически относятся к общей пользе применения этих методов.[20] Тем не менее, поскольку каркас PI предоставил свидетельства успеха, можно надеяться увидеть дальнейшее развитие этой методологии.

Генетические алгоритмы

Генетический алгоритм (genetic algorithm, GA) - это метод приближенного поиска, используемый программами, которые имитируют эволюцию. GA выполняет мутации над исходной популяцией, сохраняя благоприятные наследственные признаки в будущих поколениях. Для выбора созданных образцов, которые являются более "приспособленными" к своей среде, применяются правила естественного отбора. Затем выбранные образцы скрещиваются и мутируют, и процесс продолжается. Для определения GA обычно требуются три компонента:

• Представление того, как могло бы выглядеть решение.
• Функция приспособленности для оценки того, насколько хорошим могло бы быть отдельное решение.
• Функция воспроизводства, отвечающая за мутацию и скрещивание двух решений.

Чтобы лучше проиллюстрировать эти понятия, рассмотрим простую задачу и ее решение с помощью GA. Революционная задача, которую мы хотели бы решить, состоит в максимизации числа единиц в двоичной строке длиной 10. Представление и функция приспособленности для этой задачи очевидны. Возможные решения представляются строкой двоичных цифр, а функция

приспособленности определяется как количество единиц в строке. В качестве функции воспроизводства, или скрещивания, мы произвольно решаем обменивать части двух строк в позициях 3 и 6 и случайно инвертировать один бит у потомка. Эта функция скрещивания может быть не самой эффективной, но она удовлетворяет нашим потребностям. Правила обмена позволяют родителям передавать наследственную информацию, а инверсия бита обеспечивает случайную мутацию. GA развивается следующим образом:

1. Начать со случайно созданной популяции решений.
2. Применить функцию приспособленности к каждому решению и выбрать наиболее "приспособленные".
3. Применить функцию скрещивания к выбранным решениям.
4. Полученные потомки заменяют исходную популяцию, и процесс повторяется.

Чтобы увидеть это в действии, начнем с популяции из четырех случайно созданных двоичных строк (решений) и вычислим приспособленность каждой из них:

```
0100100000        2
1000001010        3
1110100111        7
0000001000        1
```

Средняя пара (выделенная полужирным) вычисляется как наиболее "приспособленная" и поэтому выбирается для скрещивания (вот счастливчики). Обмен в позиции 3 создает одну пару потомков, а обмен в позиции 6 создает другую:

```
1000001010
3     100 + 0100111 -> 1000100111
1110100111
7     111 + 0001010 -> 1110001010

1000001010
3     100000 + 0111 -> 1000000111
1110100111
7     111010 + 1010 -> 1110101010
```

Затем созданные потомки подвергаются случайной мутации (выделенной полужирным), и функция приспособленности применяется снова:

```
1000100111  ->    1010100111        6
1110001010  ->    1110000010        4
1000000111  ->    1000001111        5
1110101010  ->    1110101110        7
```

Мы можем увидеть успех GA, поскольку средняя приспособленность новой популяции возросла. В этом конкретном примере мы использовали постоянную частоту мутаций. В более сложном примере можно было бы автоматически повышать частоту мутаций, если создаваемые потомки не эволюционируют в течение некоторого порогового периода времени. Обратите внимание, что генетические алгоритмы считаются стохастическими глобальными оптимизаторами. Иными словами, в алгоритме присутствует случайная составляющая, поэтому выходные данные постоянно меняются. Однако, хотя GA будет продолжать находить лучшие решения, он может никогда не найти наилучшее возможное решение (в нашем примере - строку из одних единиц), независимо от времени выполнения.

Применение генетических алгоритмов для улучшения фаззинга - предмет, который совсем недавно исследовала группа из Университета Центральной Флориды (University of Central Florida, UCF) и представила на конференции по безопасности BlackHat US 2006 USA.[21] Группа UCF продемонстрировала экспериментальный инструмент под названием Sidewinder, способный автоматически создавать входные данные, чтобы вынуждать выполнение идти по заданному пути. Решения GA в данном случае представляют собой созданные данные фаззинга, представленные в виде контекстно-свободных грамматик.[22] Затем для определения функции приспособленности

был выбран нетрадиционный подход к фаззингу. Вместо создания данных и отслеживания сбоев сначала статически обнаруживаются потенциально уязвимые участки кода, такие как вызовы небезопасных API (например, `strcpy`). Этот этап процесса похож на то, как Autodafé обнаруживает участки кода, чтобы придавать маркерам повышенный вес, как обсуждалось в предыдущей главе. Далее исследуется граф потока управления (control-flow graph, CFG; не путать с контекстно-свободной грамматикой) всего целевого двоичного файла, и извлекаются подграфы между точками поступления данных из сокета (вызовами `recv`) и потенциально уязвимыми участками кода. На рис. 22.5 изображен пример CFG, содержащего обе эти точки. Подробное определение и объяснение CFG приведено в главе 23 "Отслеживание фаззера".

На следующем этапе необходимо определить узлы на всех путях, соединяющих точку входа и целевой уязвимый код. На рис. 22.6 показан тот же CFG, где узлы вдоль соединяющего пути выделены черным.

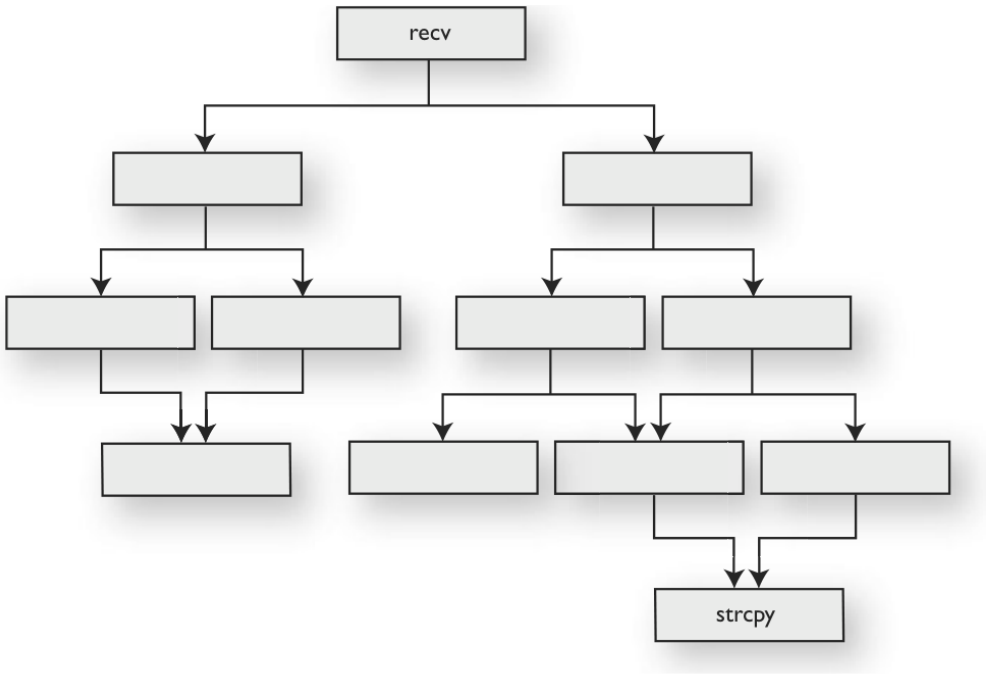


Рис. 22.5. Граф потока управления, содержащий потенциальную уязвимость

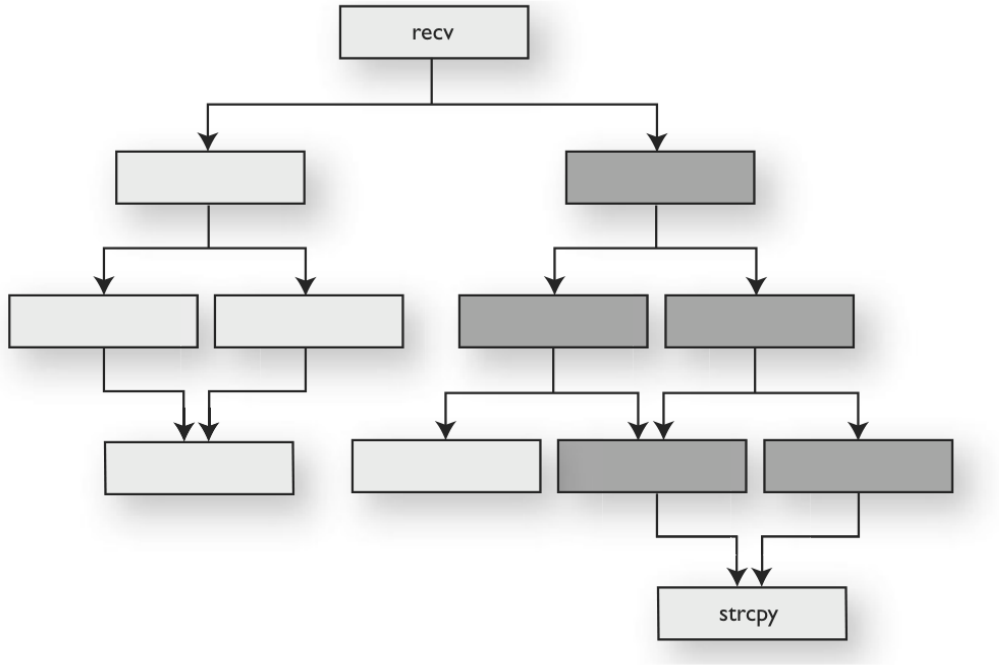


Рис. 22.6. Граф потока управления с выделенным соединяющим путем

Далее определяются выходные узлы в том же CFG. Выходной узел определяется как граничный узел непосредственно за пределами соединяющего пути. Если достигнут выходной узел, значит, выполнение отклонилось от любого из возможных путей к уязвимому коду. На рис. 22.7 показан тот же CFG, что и прежде, с выделенными черным узлами вдоль соединяющего пути. Кроме того, выходные узлы выделены белым.

Имея окончательно размеченный CFG, можно определить функцию приспособленности. Группа UCF применила к полученному CFG марковский процесс, чтобы вычислять приспособленность на основе вероятности прохождения определенных путей. На практике и, говоря проще, выполняются следующие действия. Отладчик используется для подключения к целевому процессу и установки точек останова на узле входа, целевом узле, выходных узлах и всех узлах вдоль соединяющего пути. Достижение этих точек останова отслеживается, чтобы оценивать продвижение по пути выполнения во время работы для заданных входных данных (решения). Выполнение отслеживается до достижения выходного узла. Фаззер создает входные данные и передает их целевому процессу. Входные данные с наибольшими значениями "приспособленности" выбираются для воспроизводства, и процесс продолжается, пока не будет успешно достигнут целевой узел, обозначающий потенциальную уязвимость.

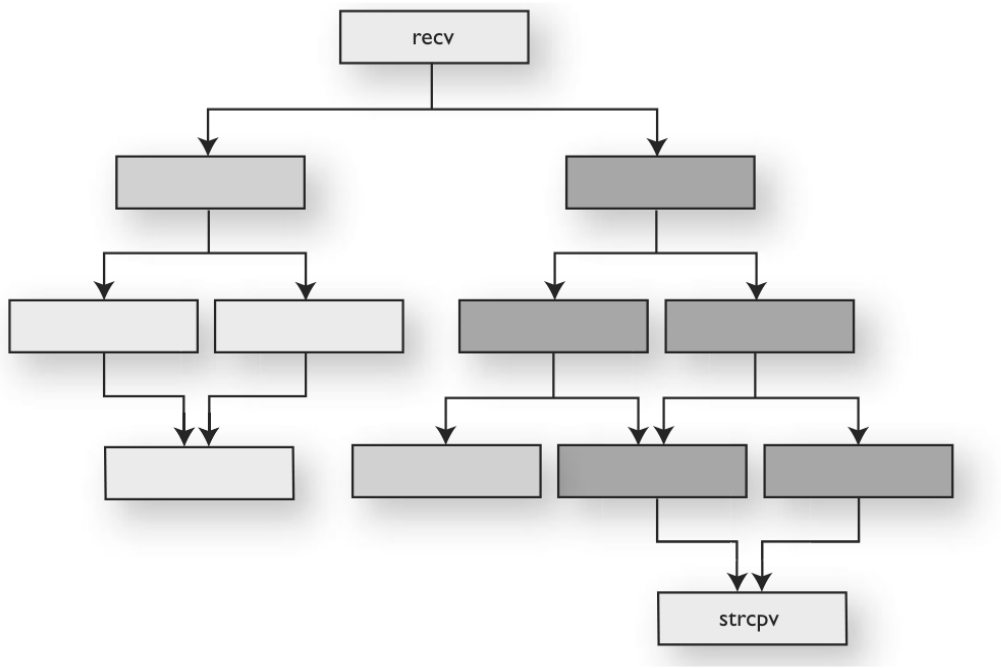


Рис. 22.7. Граф потока управления с выделенными выходными узлами

Sidewinder объединяет статический анализ, теорию графов и инструментированный отладчиком анализ времени выполнения, чтобы, по существу, методом полного перебора создать входные данные, способные достичь произвольных мест внутри целевого процесса. Хотя это мощная концепция, в настоящее время существует ряд ограничений, из-за которых данное решение далеко от совершенства. Во-первых, не все структуры CFG пригодны для обучения генетических алгоритмов. В частности, между структурой графа и разбором данных должна существовать некоторая зависимость. Кроме того, извлечение точных CFG посредством статического анализа возможно не всегда. Неточно извлеченный CFG, скорее всего, приведет к сбою всего процесса. Наконец, при таком подходе создание допустимых данных для протоколов, содержащих поля в стиле TLV, может занять непомерно много времени. Например, при фаззинге протокола с вычисляемым значением CRC наше Солнце, вероятно, станет сверхновой прежде, чем GA даст сколько-нибудь полезные результаты.

В целом этот метод интересен для изучения и, несомненно, в некоторых случаях приносит пользу. Возможно, лучше всего использовать Sidewinder для того, чтобы помочь фаззеру мутировать входные данные для прохождения небольшого подпути, когда необходимо увеличить покрытие кода, а не для поиска входных данных, способных пройти весь соединяющий путь от точки входа до цели.

Резюме

Эта глава началась с утверждения, что самый трудный аспект фаззинга - преодоление барьера для начала работы, связанного с пониманием и моделированием целевого протокола или формата файла. Были представлены три общих методологии, помогающие как при ручном, так и при автоматизированном разборе. Эвристические методы, биоинформатика и GA были описаны и рассмотрены в качестве возможных решений. Представленные в этой главе концепции находятся на переднем крае исследований фаззинга, где достижения появляются ежедневно. Более актуальные сведения и ресурсы можно найти на http://www.fuzzing.org.

Сноски

[1] [назад] http://www.samba.org
[2] [назад] http://groups.google.com/group/vmsnet.networks.desktop.pathworks/msg/7d939a9e7e419b9c
[3] [назад] http://www.samba.org/samba/docs/10years.html
[4] [назад] http://www.w3.org/Protocols/rfc2616/rfc2616.html
[5] [назад] http://www.zerodayinitiative.com/advisories/ZDI-06-028.html
[6] [назад] Ожидайте выпуска этого инструмента Коди Пирсом из TippingPoint.
[7] [назад] Этот процесс фактически можно автоматизировать некоторыми манипуляциями с IP, которые ради простоты не обсуждаются.
[8] [назад] http://www.matasano.com/log/399/pdb-blackhat-talk-materials-as-promised/
[9] [назад] http://openrce.org/downloads/details/208/PaiMei
[10] [назад] http://en.wikipedia.org/wiki/Bioinformatics
[11] [назад] http://en.wikipedia.org/wiki/Needleman-Wunsch_algorithm
[12] [назад] http://en.wikipedia.org/wiki/Saul_Needleman
[13] [назад] http://en.wikipedia.org/wiki/Christian_Wunsch
[14] [назад] http://www.toorcon.org
[15] [назад] http://www.wired.com/news/infostructure/0,1377,65191,00.html
[16] [назад] http://www.musecurity.com
[17] [назад] http://packetstormsecurity.org/sniffers/PI.tgz
[18] [назад] http://en.wikipedia.org/wiki/Smith-Waterman
[19] [назад] http://en.wikipedia.org/wiki/Internet_Control_Message_Protocol
[20] [назад] http://www.matasano.com/log/294/protocol-informatics/
[21] [назад] http://www.blackhat.com/html/bh-usa-06/bh-usa-06-speakers.html#Embleton
[22] [назад] http://en.wikipedia.org/wiki/Context-free_grammar

Глава 23. Отслеживание фаззера

> "Ну, я думаю, если вы говорите, что собираетесь что-то сделать, и не делаете этого, это надежность".
>
> - Джордж У. Буш в интернет-чате CNN, 30 августа 2000 года

В предыдущих главах мы дали определение фаззингу, раскрыли цели фаззинга, перечислили различные классы фаззинга и обсудили различные методы создания данных. Однако нам еще предстоит рассмотреть вопрос отслеживания хода работы наших разнообразных технологий фаззеров. Понятие отслеживания фаззера до сих пор не получило значительного внимания в сообществе специалистов по безопасности. Насколько нам известно, ни один из доступных в настоящее время коммерческих или свободно распространяемых фаззеров не затрагивает эту тему.

В этой главе мы дадим определение отслеживанию фаззера, также известному как покрытие кода. Мы обсудим его преимущества, исследуем различные подходы к реализации и, конечно же, в духе остальной части этой книги создадим рабочий прототип, который можно будет объединить с разработанными ранее фаззерами, чтобы обеспечить более мощный уровень анализа.

Что именно мы отслеживаем?

На самом низком программном уровне ЦП выполняет инструкции ассемблера, или команды, понятные ЦП. Доступный набор инструкций различается в зависимости от архитектуры, лежащей в основе вашей цели. Например, знакомая архитектура IA-32 / x86 представляет собой архитектуру компьютера с полным набором команд[1] (Complex Instruction Set Computer, CISC), которая предоставляет более 500 инструкций, включая, разумеется, возможность выполнять основные математические операции и манипулировать памятью. Сравните эту архитектуру с микропроцессором с сокращенным набором команд (Reduced Instruction Set Computer, RISC), таким как SPARC, который предоставляет менее 200 инструкций.[2]

CISC и RISC

Термин CISC фактически является ретроспективным определением, использованным для отличия этой конструкции от концепции RISC, которую в конце 1970-х годов разработали исследователи IBM. Основные философские различия между этими конструкциями состоят в том, что процессоры RISC, возможно, быстрее и дешевле в производстве, тогда как процессоры CISC реже обращаются к основной памяти и требуют меньше строк кода для выполнения одной и той же задачи. Спор о том, какая архитектура лучше, недавно достиг новых высот одержимости с появлением фанатичных пользователей Apple Mac, чье драгоценное божество до недавнего времени было доступно исключительно на процессоре PowerPC с архитектурой RISC.

Интересно, что, несмотря на огромное разнообразие сложных инструкций, доступных в процессоре x86, в среднем около дюжины инструкций выполняются более чем в половине случаев.[3]

Работаете ли вы с CISC или RISC, отдельные инструкции на самом деле реже пишутся человеком-программистом, а чаще компилируются или переводятся на этот низкоуровневый язык с языка более высокого уровня, такого как C или C++. В общем случае программы, с которыми вы регулярно взаимодействуете, содержат от десятков тысяч до миллионов инструкций. Каждое взаимодействие с программой вызывает выполнение инструкций. Такие простые действия, как щелчок мышью, обработка сетевого запроса или открытие файла, требуют выполнения многочисленных инструкций.

Если во время аудита с фаззингом нам удается вызвать сбой программного телефона (софтфона) Voice over IP (VoIP), как узнать, какой набор инструкций отвечал за сбой? Типичный подход, используемый исследователями сегодня, состоит в движении в обратном направлении посредством изучения журналов и подключения отладчика для точного определения места сбоя. Альтернативный подход состоит в движении вперед путем активного отслеживания того, какие инструкции выполняются в ответ на созданные нами запросы фаззинга. Некоторые отладчики, такие как OllyDbg, реализуют функции трассировки, позволяющие пользователям отслеживать все выполняемые инструкции. К сожалению, такая трассировка обычно требует очень много ресурсов и непрактична за пределами узконаправленных задач. Возвращаясь к нашему примеру: когда софтфон VoIP действительно дает сбой, мы можем просмотреть выполненные инструкции, чтобы определить, в какой точке процесса произошел сбой.

При написании программ один из самых простых способов их отладки - вставка операторов печати по всему коду. Таким образом, по мере выполнения программы разработчик узнает, где происходит выполнение и в каком порядке выполняются различные логические блоки. По существу, мы хотим решить ту же задачу, но на более низком уровне, где у нас нет возможности вставлять произвольные операторы печати.

Этот процесс наблюдения и записи точных инструкций, выполняемых при обработке различных запросов, мы определяем как "отслеживание фаззера". Более общий термин для этого процесса, который, возможно, вам уже знаком, - покрытие кода; на протяжении главы эти термины будут употребляться взаимозаменяемо. Отслеживание фаззера - довольно простой процесс, и, как мы вскоре увидим, он создает ряд возможностей для улучшения анализа фаззинга.

Визуализация двоичных файлов и базовые блоки

Важнейшее понятие для нашего обсуждения покрытия кода - понимание визуализации двоичных файлов, которая будет использоваться при проектировании и разработке нашего прототипа средства отслеживания фаззера. Дизассемблированные двоичные файлы можно визуализировать в виде графов, известных как графы вызовов. Для этого каждая функция представляется узлом, а вызовы между функциями - ребрами между узлами. Это полезная абстракция для рассмотрения структуры двоичного файла в целом, а также отношений между различными функциями. Рассмотрим искусственный пример на рис. 23.1.

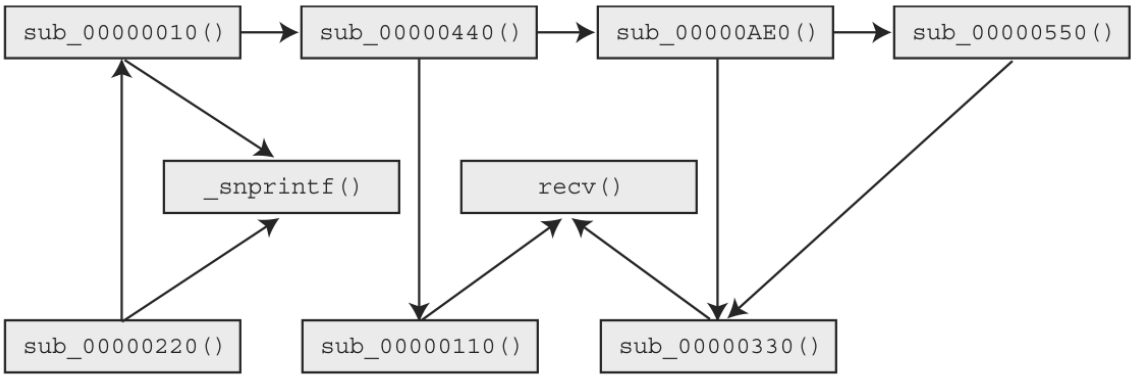


Рис. 23.1. Простой граф вызовов

Большинство узлов на рис. 23.1 содержат метку с префиксом `sub_`, за которым следует восьмизначное шестнадцатеричное число, представляющее адрес подпрограммы в памяти. Это соглашение об именовании используется дизассемблером DataRescue Interactive Disassembler Pro (IDA Pro)[4] для автоматического создания имен функций без символьной информации. Другие движки дизассемблирования используют похожие схемы именования. При анализе двоичных приложений с закрытым исходным кодом отсутствие символьной информации приводит к тому, что большинство функций получают имена в таком виде. Два узла на рис. 23.1 все же имеют символьные имена и были обозначены соответствующим образом: `snprintf()` и `recv()`. Беглого взгляда на этот граф вызовов достаточно, чтобы сразу определить, что безымянные подпрограммы `sub_00000110()` и `sub_00000330()` отвечают за обработку сетевых данных из-за их связи с `recv()`.

CFG

Дизассемблированные функции также можно визуализировать в виде графов, известных как CFG. Для этого каждый базовый блок представляется узлом, а переходы между базовыми блоками - ребрами между узлами. Отсюда естественным образом возникает вопрос: что такое базовый блок? Базовый блок определяется как последовательность инструкций, в которой после достижения первой инструкции базового блока гарантируется однократное выполнение по порядку каждой инструкции внутри блока. Если пояснить подробнее, базовый блок обычно начинается при следующих условиях:

• Начало функции.
• Цель инструкции перехода.
• Инструкция после инструкции перехода.

Базовые блоки обычно заканчиваются при следующих условиях:

• Инструкция перехода.

• Инструкция возврата.

Это упрощенный список точек начала и окончания базовых блоков. Более полный список учитывал бы вызовы функций, которые не возвращают управление, и обработку исключений. Однако для наших целей такое определение вполне подойдет.

CFG на иллюстрациях

Чтобы окончательно прояснить эту мысль, рассмотрим следующий фрагмент инструкций из воображаемой подпрограммы `sub_00000010`. Последовательности `...` на рис. 23.2 представляют произвольное число любых инструкций, не являющихся переходами. Такое представление дизассемблированной функции также известно как статический листинг; его пример показан на рис. 23.2.

```
00000010 sub_00000010
00000010 push ebp
00000011 mov ebp, esp
00000013 sub esp, 128h
…
00000025 jz 00000050
0000002B mov eax, 0Ah
00000030 mov ebx, 0Ah
…
00000050 xor eax, eax
00000052 xor ebx, ebx
…
```

Рис. 23.2. Дизассемблированный статический листинг `sub_00000010`

Разбиение статического листинга на базовые блоки - простая задача, если следовать правилам, изложенным ранее. Первый базовый блок начинается в верхней части функции по адресу `0x00000010` и заканчивается первой инструкцией перехода по адресу `0x00000025`. Следующая инструкция после перехода по адресу `0x0000002B`, а также цель перехода по адресу `0x00000050` отмечают начало новых базовых блоков. После разбиения на базовые блоки CFG подпрограммы, показанной на рис. 23.2, выглядит так, как на рис. 23.3.

Это полезная абстракция для рассмотрения различных путей потока управления через функцию, а также для обнаружения логических циклов. Главное, что следует помнить о CFG: каждый блок инструкций гарантированно выполняется как единое целое. Иными словами, если выполняется первая инструкция блока, то будут выполнены все инструкции в этом блоке. Позже мы увидим, как это знание может помочь нам воспользоваться необходимыми сокращениями при разработке средства отслеживания фаззера.

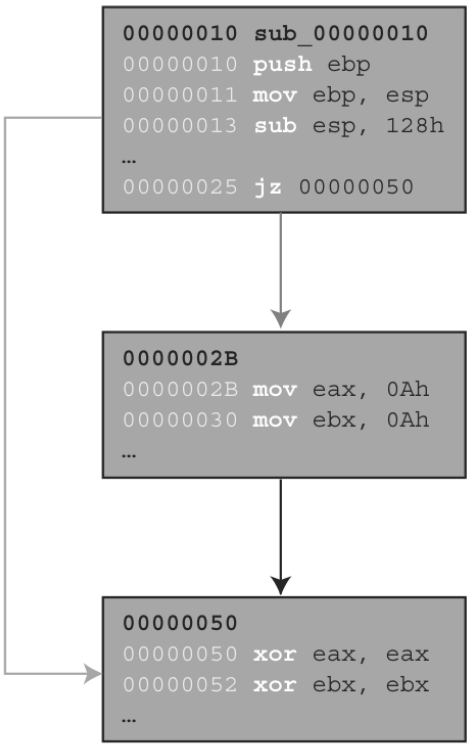


Рис. 23.3. Граф потока управления `sub_00000010`

Проектирование средства отслеживания фаззера

Для реализации собственного инструмента покрытия кода можно использовать несколько подходов. Если вы уже прочитали предыдущие главы, возможно, первый пришедший вам в голову подход - применить отладчик с функцией трассировки для отслеживания и регистрации каждой выполняемой инструкции. Именно этот подход использует OllyDbg для реализации функций "debug\trace into" и "debug\trace over". Мы могли бы воспроизвести эту функциональность на основе той же библиотеки PyDbg, которую использовали в главе 20 "Фаззинг в памяти: автоматизация", применив следующую логику:

1. Подключиться к целевому процессу, который мы подвергаем фаззингу, или загрузить его.
2. Зарегистрировать обработчик одиночного шага и установить флаг одиночного шага.
3. В обработчике одиночного шага зарегистрировать адрес текущей инструкции и снова установить флаг одиночного шага.
4. Продолжать до завершения процесса или сбоя.

Этот упрощенный подход можно реализовать поверх PyDbg менее чем в 50 строках Python. Однако он не является предпочтительным, поскольку инструментирование каждой инструкции внутри процесса с помощью отладчика вносит очень большую задержку, которая во многих случаях делает этот подход непригодным. Если вы не верите, попробуйте сами с соответствующим кодом, приведенным на боковой панели и также доступным в Интернете по адресу http://www.fuzzing.org. Более того, простое определение и регистрация выполненных инструкций охватывают лишь один аспект нашей общей цели. Для создания более качественных отчетов нам нужна дополнительная информация о нашей цели.

Мы реализуем средство отслеживания фаззера с помощью трехэтапного подхода:

• Сначала мы статически профилируем нашу цель.

• Второй этап - фактическое отслеживание в реальном времени и регистрация того, какие инструкции внутри нашей цели выполняются в ответ на действия нашего фаззера.
• Третий и последний этап предполагает перекрестное сопоставление созданных в реальном времени данных с данными профиля из этапа 1 для создания полезного отчета об отслеживании.

Профилирование

Если предположить, что у нас есть возможность определить, какие инструкции выполнялись внутри любого заданного приложения, какие еще сведения об этом приложении мы хотели бы узнать, чтобы лучше понять имеющийся набор данных? Например, как минимум мы хотели бы знать общее число инструкций в нашей цели. Тогда мы сможем определить, какой процент лежащего в основе кода целевого приложения отвечает за выполнение каждой конкретной задачи. В идеальном случае нам хотелось бы получить более подробную разбивку набора инструкций:

• Сколько инструкций находится в нашей цели?
• Сколько функций?
• Сколько базовых блоков?
• Какие инструкции внутри нашей цели относятся к стандартным вспомогательным библиотекам?
• Какие инструкции внутри нашей цели написали наши разработчики?
• Из каких строк исходного кода была получена каждая отдельная инструкция?
• Какие инструкции и данные достижимы и могут находиться под влиянием сетевых данных или данных файла?

Для ответа на эти вопросы можно использовать различные инструменты и методы. Большинство сред разработки способно создавать символьную отладочную информацию, которая также может помочь нам точнее ответить на эти вопросы. Для наших целей мы предполагаем, что исходный код не является легкодоступным.

Чтобы получить ответы на эти ключевые вопросы, мы строим наш профилировщик поверх дизассемблера IDA Pro. Из IDA можно легко получить статистические сведения об инструкциях и функциях. Реализовав простой алгоритм на основе правил для базовых блоков, мы также можем легко извлечь список всех базовых блоков, содержащихся в каждой функции.

Трассировка

Поскольку выполнение программы по одному шагу может занимать много времени и быть излишним, как еще мы можем отслеживать покрытие инструкций? Вспомним наше предыдущее определение: базовый блок - это просто последовательность инструкций, в которой гарантируется выполнение по порядку каждой инструкции внутри блока. Если мы можем отслеживать выполнение базовых блоков, то тем самым отслеживаем и выполнение инструкций. Эту задачу можно решить, объединив отладчик и информацию, полученную на этапе профилирования.

Используя библиотеку PyDbg, мы можем реализовать эту концепцию со следующей логикой:

1. Подключиться к целевому процессу, который мы подвергаем фаззингу, или загрузить его.
2. Зарегистрировать обработчик точки останова.
3. Установить точку останова в начале каждого базового блока.
4. В обработчике точки останова зарегистрировать адрес текущей инструкции и при необходимости восстановить точку останова.
5. Продолжать до завершения процесса или сбоя.

Регистрируя попадания в базовые блоки, а не отдельные инструкции, мы можем значительно повысить производительность по сравнению с традиционной функцией трассировки, такой как ограниченный подход OllyDbg, упомянутый ранее. Если нас интересует только то, какие базовые блоки выполняются, восстанавливать точки останова на этапе 4 не требуется. После попадания в базовый блок этот факт будет зарегистрирован в обработчике, и нас больше не интересует, будет ли и когда именно этот базовый блок когда-либо достигнут снова в будущем. Однако, если нас также интересует последовательность выполнения базовых блоков, мы должны восстанавливать точки останова на этапе 4. Если в один и тот же базовый блок попадают снова и снова, возможно, внутри цикла, мы зарегистрируем каждое повторное попадание. Восстановление точек останова предоставляет больше контекстной информации. Если позволить точкам останова исчезать, мы получим выигрыш в скорости. Решение о выборе одного или другого подхода, вероятно, будет зависеть от решаемой задачи, поэтому его следует сделать настраиваемым параметром нашего инструмента.

В ситуациях, когда мы отслеживаем большое число базовых блоков, скорость отслеживания может оказаться ниже желаемой. В таких ситуациях мы можем выполнить дальнейшее абстрагирование от уровня инструкций, отслеживая выполняемые функции, а не выполняемые базовые блоки. Для этого требуются минимальные изменения нашей логики. Просто установите точки останова на базовых блоках, с которых начинаются функции, вместо всех базовых блоков. И снова решение о выборе одного или другого подхода может меняться от запуска к запуску, поэтому его следует сделать настраиваемым параметром нашего инструмента.

Упрощенный инструмент покрытия кода PyDbg с одиночным шагом

Для любознательного читателя следующий сценарий PyDbg демонстрирует реализацию пошагового исполнителя. При первоначальном подключении и срабатывании точки останова сценарий переводит каждый существующий поток в режим одиночного шага. Также регистрируется обработчик создания новых потоков, чтобы и они переводились в режим одиночного шага.

Единственный "хитрый" момент при разработке пошагового исполнителя PyDbg - обработка переключений контекста в ядро. На каждом одиночном шаге сценарий проверяет, является ли текущая инструкция инструкцией `sysenter`. Эта инструкция служит шлюзом в ядро, используемым современными операционными системами Microsoft Windows. При обнаружении инструкции `sysenter` устанавливается точка останова по адресу, на который вернется ядро, а потоку разрешается выполняться обычным образом (то есть не в режиме одиночного шага).

Как только поток возвращается из ядра, выполнение одиночными шагами включается снова. Код для любознательных читателей показан далее.

```
from pydbg import *
from pydbg.defines import *

# обработчик точки останова.
def on_bp (dbg):
    ea     = dbg.exception_address
    disasm = dbg.disasm(ea)

    # перевести каждый поток в режим одиночного шага.
    if dbg.first_breakpoint:
        for tid in dbg.enumerate_threads():
            handle = dbg.open_thread(tid)
            dbg.single_step(True, handle)
            dbg.close_handle(handle)

    print "%08x: %s" % (ea, disasm)
    dbg.single_step(True)
```

```
    return DBG_CONTINUE

# обработчик одиночного шага.
def on_ss (dbg):
    ea     = dbg.exception_address
    disasm = dbg.disasm(ea)

    print "%08x: %s" % (ea, disasm)

    # выполнять одиночные шаги в пространстве ядра нельзя,
    # поэтому установить точку останова на возврате и продолжить
    if disasm == "sysenter":
        ret_addr = dbg.get_arg(0)
        dbg.bp_set(ret_addr)
    else:
        dbg.single_step(True)

    return DBG_CONTINUE

# обработчик создания потока.
def on_ct (dbg):
    # перевести вновь созданные потоки в режим одиночного шага.
    dbg.single_step(True)
    return DBG_CONTINUE

dbg = pydbg()
dbg.set_callback(EXCEPTION_BREAKPOINT,      on_bp)
dbg.set_callback(EXCEPTION_SINGLE_STEP,     on_ss)
dbg.set_callback(CREATE_THREAD_DEBUG_EVENT, on_ct)

try:
    dbg.attach(123)
    dbg.debug_event_loop()
except pdx, x:
    dbg.cleanup().detach()
    print x.__str__()
```

Еще одна удобная функция, которую следует реализовать в нашем средстве отслеживания фаззера, - возможность отфильтровывать предварительно записанное покрытие кода из новых записей. Эту функцию можно использовать для легкого исключения функций, базовых блоков и инструкций, отвечающих за обработку неинтересующих нас задач. Пример такого неинтересного кода - код, отвечающий за обработку событий графического интерфейса (щелчков мышью, выбора пунктов меню и так далее).

Позже, на боковой панели о покрытии кода для обнаружения уязвимостей, мы увидим, как это может пригодиться.

Перекрестное сопоставление

На третьем и последнем этапе мы объединяем информацию статического профилирования с выходными данными нашего средства отслеживания времени выполнения, чтобы создавать точные и полезные отчеты, отвечающие на следующие вопросы:

• Какие области кода отвечают за обработку наших входных данных?
• Какой процент нашей цели мы покрыли?
• Какой путь выполнения привел к конкретному сбою?
• Какие логические решения внутри нашей цели не были задействованы?
• Какие логические решения внутри нашей цели непосредственно зависят от предоставленных данных?

Последние два пункта этого списка относятся к принятию обоснованных решений о способах улучшить наш фаззер, а не к точному определению какого-либо конкретного дефекта в самом

целевом приложении. Мы подробнее рассмотрим эти пункты далее в главе при обсуждении различных преимуществ, которые может предоставить отслеживание фаззера.

Покрытие кода для обнаружения уязвимостей

14 июня 2005 года было опубликовано сообщение об удаленно эксплуатируемом переполнении буфера, затрагивающем Microsoft Outlook Express при разборе ответов протокола Network News Transfer Protocol (NNTP).[5] Рассмотрим преимущества анализа покрытия кода применительно к этой проблеме с точки зрения исследователя уязвимостей. Суть бюллетеня Microsoft гласит:

В Outlook Express существует уязвимость удаленного выполнения кода, когда он используется в качестве программы чтения групп новостей. Злоумышленник мог бы воспользоваться этой уязвимостью, создав вредоносный сервер групп новостей, который мог бы потенциально допустить удаленное выполнение кода, если бы пользователь запросил новости с этого сервера. Злоумышленник, успешно воспользовавшийся этой уязвимостью, мог бы получить полный контроль над затронутой системой. Однако для эксплуатации этой уязвимости требуется взаимодействие с пользователем.

Здесь не так много информации, на которую можно опереться, и ее, конечно, недостаточно, чтобы она принесла пользу, например, разработчикам сигнатур IDS или IPS. Все, что мы знаем, - ошибка проявляется при подключении Outlook Express к вредоносному серверу NNTP. Используя существовавший ранее инструмент покрытия кода с открытым исходным кодом Process Stalker,[6] попробуем выяснить, удастся ли нам несколько сузить проблему. Конкретные параметры командной строки и использование Process Stalker выходят за рамки этой книги. Конкретные подробности этого примера приведены в исходной статье OpenRCE.[7]

Установив доступное исправление и отслеживая, какие файлы в каталоге установки Outlook Express были изменены, мы можем определить, что уязвимый код должен находиться в `MSOE.DLL`. Анализ этого модуля с помощью подключаемого модуля Process Stalker для IDA Pro выявляет в общей сложности примерно 4800 функций и 58 000 базовых блоков. Ручной просмотр этих данных был бы весьма пугающей задачей. Чтобы сузить интересующую нас область, можно использовать Process Stalker для обнаружения путей кода, связанных с подключением Outlook Express к произвольному серверу NNTP. Хотя это, несомненно, эффективно сокращает пространство анализа, мы можем добиться большего. Вспомним предыдущее утверждение: каждое взаимодействие с нашей целью, в свою очередь, приводит к выполнению инструкций. В текущей ситуации это означает, что, когда мы, например, обращаемся к вредоносному серверу NNTP через обработчик URI `news://` в Internet Explorer, наш инструмент покрытия кода без необходимости зарегистрирует код, отвечающий за запуск Outlook Express, код, отвечающий за отрисовку таких экранных элементов, как окна и диалоговые окна, и код, отвечающий за обработку управляемых пользователем событий графического интерфейса, таких как щелчки по кнопкам. Например, при первом подключении к серверу NNTP появляется диалоговое окно, показанное на рис. 23.4.

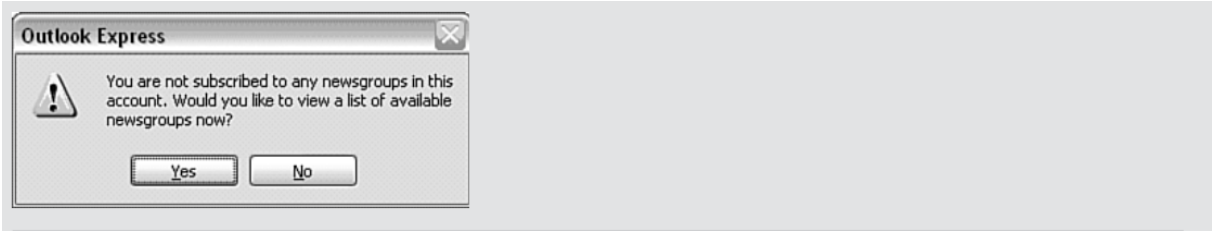


Рис. 23.4. Неудачное диалоговое окно перед эксплуатацией, снижающее степень серьезности

Создание, обработка и уничтожение этого диалогового окна совершенно не интересны для наших целей. Чтобы еще сильнее сузить область анализа, мы можем использовать Process Stalker для записи только кода, отвечающего за выполнение различных задач графического интерфейса. Для этого мы подключаемся к Outlook Express и взаимодействуем с ним, фактически не подключаясь к серверу NNTP. Затем мы удаляем перекрывающееся покрытие между нашими записями "только GUI" и "подключение к серверу", чтобы точно выделить код, отвечающий за разбор запроса сервера NNTP. В конечном счете нам удается сократить поле зрения до 91 функции. Из 1337 (клянемся, это число не выдумано) базовых блоков в этих 91 функциях в записи подключения к серверу фактически были посещены лишь 747. По сравнению с 58 000, с которых мы начали, это существенное и полезное сокращение!

Применение простого сценария для обнаружения базовых блоков, отвечающих за перемещение предоставленных сервером данных, дополнительно сокращает их число до жалких 26 базовых блоков, второй из которых раскрывает точные особенности уязвимости: произвольное количество предоставленных сервером данных, разделенных пробелами, копируется в статический 16-байтный буфер стека. Прекрасно, теперь мы можем быстренько состряпать эксплойт . . . э-э . . . фильтр защиты от уязвимости и вернуться домой как раз к последнему эпизоду "Клана Сопрано".[8]

Разбор инструмента покрытия кода

Библиотека PyDbg, с которой мы работали ранее, фактически является подкомпонентом более крупного каркаса обратного проектирования под названием PaiMei. PaiMei - активно разрабатываемый каркас обратного проектирования с открытым исходным кодом на Python, свободно доступный для загрузки.[9] По существу, PaiMei можно считать швейцарским армейским ножом специалиста по обратному проектированию; он может помочь в различных сложных задачах фаззинга, таких как интеллектуальное обнаружение и обработка исключений - тема, которую мы подробно рассматриваем в главе 24 "Интеллектуальное обнаружение сбоев".

В комплекте каркаса уже имеется инструмент покрытия кода под названием PAIMEIpstalker (также называемый Process Stalker или PStalker). Хотя в готовом виде этот инструмент может не создавать именно те отчеты, которые вам нужны, каркас и входящие в его комплект приложения имеют открытый исходный код, поэтому их можно изменять в соответствии с любыми конкретными потребностями данного проекта. Каркас разбит на следующие основные компоненты:

• *PyDbg*. Класс чистой абстракции отладки win32 на Python.

• *pGRAPH*. Уровень абстракции графа с отдельными классами для узлов, ребер и кластеров.

• *PIDA*. Построенный поверх pGRAPH, PIDA призван предоставить абстрактный и постоянный интерфейс к двоичным файлам (DLL и EXE) с отдельными классами для представления функций, базовых блоков и инструкций. Конечным результатом является создание переносимого файла, который после загрузки позволяет произвольно перемещаться по всему исходному двоичному

файлу.

Мы уже знакомы с интерфейсом PyDbg по предыдущему материалу. Создавать сценарии поверх PIDA еще проще. Интерфейс PIDA позволяет перемещаться по двоичному файлу как по графу графов, перечисляя узлы в виде функций или базовых блоков и ребра между ними. Рассмотрим следующий пример:

```
import pida

module = pida.load("target.exe.pida")

# последовательно перебрать каждую функцию модуля.
for func module.functions.values():
    print "%08x - %s" % (func.ea_start, func.name)

    # последовательно перебрать каждый базовый блок функции.
    for bb in func.basic_blocks.values():
        print "\t%08x" % bb.ea_start

        # последовательно перебрать каждую инструкцию
        # базового блока.
        for ins in bb.instructions.values():
            print "\t\t%08x %s" % (ins.ea, ins.disasm)
```

В этом фрагменте мы начинаем с загрузки ранее проанализированного и сохраненного файла PIDA в качестве модуля сценария. Затем последовательно перебираем каждую функцию модуля, попутно печатая начальный адрес и символьное имя. Далее последовательно перебираем каждый базовый блок, содержащийся в текущей функции. Для каждого базового блока мы идем дальше, последовательно перебирая содержащиеся инструкции и попутно печатая адрес инструкции и ее дизассемблированное представление. Исходя из нашего предыдущего плана, вам, возможно, уже очевидно, как можно объединить PyDbg и PIDA, чтобы легко создать прототип инструмента покрытия кода на уровне базовых блоков. Уровнем выше основных компонентов находится остальная часть каркаса PaiMei, разбитая на следующие всеобъемлющие компоненты:

• *Utilities*. Набор утилит для выполнения различных повторяющихся задач. Нас интересует конкретный класс утилит, `process_stalker.py`, написанный для предоставления абстрактного интерфейса к функциям покрытия кода.

• *Console*. Подключаемое приложение WxPython для быстрого и эффективного создания пользовательских интерфейсов GUI для недавно созданных инструментов. Именно через него мы взаимодействуем с модулем покрытия кода pstalker.
• *Scripts*. Отдельные сценарии для выполнения различных задач; один очень важный пример - сценарий IDA Python `pida_dump.py`, который запускается из IDA Pro для создания файлов `.PIDA`.

Взаимодействие с инструментом покрытия кода pstalker осуществляется через вышеупомянутый GUI WxPython, и, как мы вскоре увидим, он опирается на ряд компонентов, предоставляемых каркасом PaiMei. Исследовав различные подкомпоненты этого инструмента покрытия кода, мы рассмотрим его использование на примере конкретного случая.

Обзор компоновки PStalker

Модуль PStalker - один из многих модулей, доступных для использования в графической консоли PaiMei. На начальном экране PStalker, показанном на рис. 23.5, видно, что интерфейс разбит на три отдельных столбца.

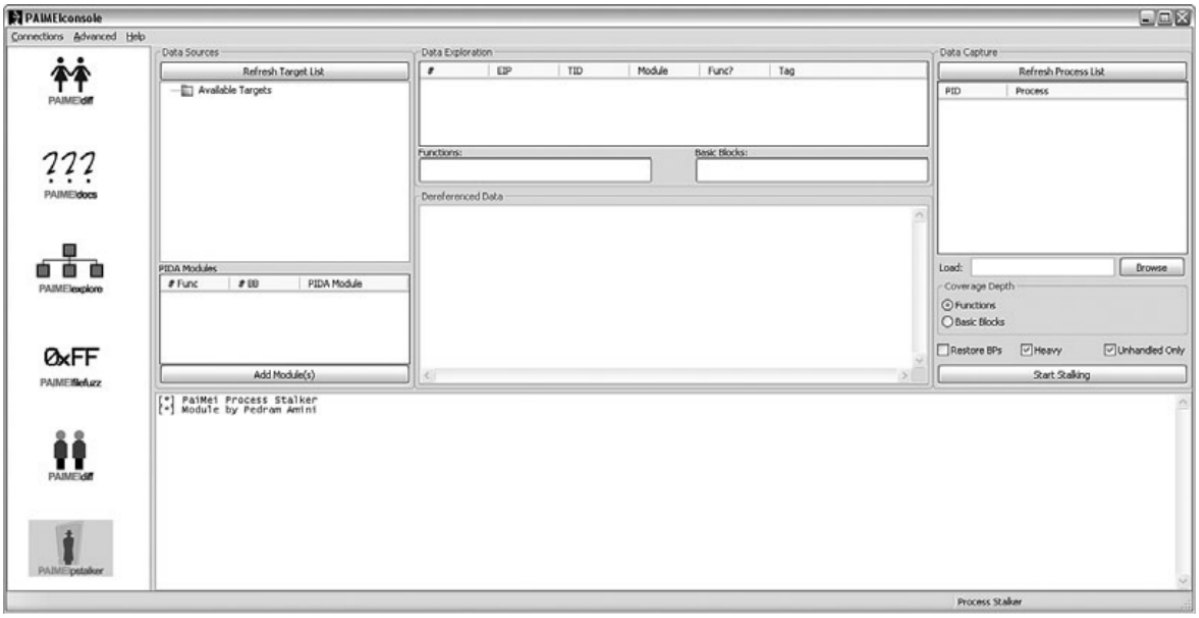


Рис. 23.5. Модуль PaiMei PStalker

Столбец Data Sources предоставляет функции, необходимые для создания и управления целями, за которыми будет вестись наблюдение, а также для загрузки соответствующих модулей PIDA. В столбце Data Exploration отображаются результаты каждого отдельного запуска покрытия кода, или наблюдения. Наконец, в столбце Data Capture указываются параметры конфигурации времени выполнения и целевой исполняемый файл. Общий рабочий процесс, связанный с этим инструментом, выглядит следующим образом (подробности далее):

• Создать новую цель, тег или и то и другое. Цели могут содержать несколько тегов, и каждый отдельный тег содержит собственные сохраненные данные покрытия кода.
• Выбрать тег для наблюдения.
• Загрузить модули PIDA для каждого файла `.DLL` и кода `.EXE`, для которых требуется информация о покрытии.
• Запустить целевой процесс.
• Обновить список процессов на панели захвата данных и выбрать целевой процесс для подключения либо найти целевой процесс для загрузки.
• Настроить параметры покрытия кода и подключиться к выбранной цели.
• Взаимодействовать с целью, пока записывается покрытие кода.
• В подходящий момент отключиться от цели и дождаться, пока PStalker экспортирует захваченные элементы в базу данных.
• Загрузить зарегистрированные попадания, чтобы начать изучение записанных данных.

Онлайновая документация[10] PaiMei содержит полнометражный видеоролик, демонстрирующий использование модуля PStalker. Если вы никогда его не видели, сейчас самое время посмотреть его, прежде чем продолжать.

Источники данных

После установления соединения всей консоли с MySQL через меню Connections выберите кнопку Retrieve Target List, чтобы заполнить дерево проводника источников данных. При первом запуске приложения этот список будет пуст. Новые цели можно создавать из контекстного меню корневого узла с меткой Available Targets. Используйте контекстное меню отдельных целей, чтобы удалить отдельную цель, добавить тег под целью и загрузить попадания из всех тегов под целью на панель исследования данных. Контекстное меню отдельных тегов предоставляет ряд функций:

• *Load hits*. Очистить панель исследования данных и загрузить попадания, связанные с выбранным тегом.
• *Append hits*. Добавить попадания, связанные с выбранным тегом, после всех существующих данных, уже находящихся на панели исследования.
• *Export to IDA*. Экспортировать попадания под выбранным тегом в виде сценария IDC, пригодного для импорта в IDA с целью выделения функций и блоков, в которые произошли попадания.
• *Sync with uDraw*. Синхронизировать панель исследования данных через общее для консоли соединение uDraw. Для этой функции очень полезны два монитора.
• *Use for stalking*. Сохранять все попадания, зарегистрированные на этапе захвата данных, в выбранном теге. Перед подключением к процессу необходимо выбрать тег для наблюдения.
• *Filter tag*. Не регистрировать в будущих сеансах наблюдения попадания в какие-либо узлы или базовые блоки под выбранным тегом. Для фильтрации можно отметить несколько тегов.
• *Clear tag*. Сохранить тег, но удалить все хранящиеся под ним попадания. PStalker не может вести наблюдение с записью в тег, который уже содержит данные. Чтобы перезаписать содержимое тега, сначала его необходимо очистить.
• *Expand tag*. Для каждой функции с попаданием в теге создать и добавить запись для каждого содержащегося в ней базового блока, даже если в этот базовый блок явного попадания не было.
• *Target/tag properties*. Изменить имя тега, просмотреть или изменить примечания к тегу либо изменить имя цели.
• *Delete tag*. Удалить тег и все хранящиеся под ним попадания.

Элемент управления списком модулей PIDA отображает список загруженных модулей PIDA, а также количество функций и базовых блоков в каждом из них. Перед подключением к целевому процессу необходимо загрузить по меньшей мере один модуль PIDA. На этапе покрытия кода к модулю PIDA обращаются для установки точек останова на функциях или базовых блоках. Модуль PIDA можно удалить из списка через контекстное меню.

Исследование данных

Панель исследования данных разбита на три раздела. Первый раздел заполняется при загрузке или добавлении попаданий из столбца Data Sources. Каждое попадание отображается в прокручиваемом списке. При выборе одного из элементов строки в текстовом элементе управления Dereferenced Data отображаются, если они доступны, контекстные данные, связанные с этим попаданием. Между списком и текстовыми элементами управления расположены две полосы выполнения, представляющие общее покрытие кода: число отдельно затронутых базовых блоков и функций, перечисленных на панели исследования, сравнивается с общим числом базовых блоков и функций во всех загруженных модулях PIDA.

Захват данных

После выбора тега для наблюдения, применения выбранных фильтров и загрузки целевых модулей PIDA следующим этапом будет фактическое подключение к целевому процессу и наблюдение за ним.

Щелкните кнопку Retrieve List, чтобы отобразить актуальный список процессов, в котором более новые процессы перечислены внизу. Выберите целевой процесс для подключения либо найдите целевой процесс для загрузки и выберите желаемую глубину покрытия. Выберите Functions, чтобы наблюдать и отслеживать, какие функции выполняются внутри целевого процесса, или Basic Blocks, чтобы более тщательно исследовать отдельные выполняемые базовые блоки.

Флажок Restore BPs определяет, следует ли восстанавливать точки останова после попадания в них. Чтобы определить, какой код выполнялся внутри цели, оставьте этот параметр отключенным. В качестве альтернативы, чтобы определить как то, что выполнялось, так и порядок выполнения, этот параметр следует включить.

Флажок Heavy определяет, будет ли PStalker сохранять контекстные данные при каждом зарегистрированном попадании. Отключите этот параметр для повышения скорости. Снимите флажок Unhandled Only, чтобы получать уведомления о событиях отладочных исключений, даже если отлаживаемая программа способна правильно их обрабатывать.

Наконец, установив все параметры и выбрав целевой процесс, щелкните кнопку Attach and Start Tracking, чтобы начать покрытие кода. Следите за окном журнала, где отображается информация времени выполнения.

Ограничения

У этого инструмента есть одно основное ограничение, о котором вам следует знать. В частности, инструмент зависит от статического анализа целевых двоичных файлов, предоставляемого IDA Pro компании DataRescue; поэтому инструмент даст сбой, когда IDA Pro допускает ошибки. Например, если данные ошибочно представлены как код, при установке PStalker точек останова на этих данных могут произойти неприятности. Это наиболее частая причина сбоя. Отключение параметра ядра анализа Make final pass в IDA Pro до автоматического анализа отключит агрессивную логику, которая допускает ошибки такого рода. Однако общий анализ также пострадает. Кроме того, в настоящее время невозможно трассировать упакованный или самомодифицирующийся код.

Хранение данных

Обычному пользователю, скорее всего, не потребуется знать, как данные организованы и хранятся за кулисами инструмента покрытия кода PStalker. Такие пользователи могут воспользоваться предоставляемой инструментом абстракцией и использовать его как есть. Однако более подробное знание схемы базы данных потребуется, если возникнет необходимость создавать пользовательские расширения или отчеты. Поэтому данный раздел посвящен разбору скрытого механизма хранения данных.

Все данные о целях и покрытии кода хранятся на сервере базы данных MySQL. Информация распределена по трем таблицам: `cc_hits`, `cc_tags` и `cc_targets`. Как показано в следующей структуре таблицы SQL, база данных `cc_targets` содержит простой список имен целей, автоматически создаваемый уникальный числовой идентификатор и текстовое поле для хранения любых относящихся к цели примечаний:

```
CREATE TABLE `paimei`.`cc_targets` (
  `id` int(10) unsigned NOT NULL auto_increment,
  `target` varchar(255) NOT NULL default '',
  `notes` text NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=MyISAM;
```

Следующая структура таблицы SQL относится к базе данных `cc_tags`, содержащей автоматически создаваемый уникальный числовой идентификатор, идентификатор цели, с которой связан тег, имя тега и, опять же, текстовое поле для хранения любых относящихся к тегу примечаний.

```
CREATE TABLE `paimei`.`cc_tags` (
  `id` int(10) unsigned NOT NULL auto_increment,
  `target_id` int(10) unsigned NOT NULL default '0',
  `tag` varchar(255) NOT NULL default '',
  `notes` text NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=MyISAM;
```

Наконец, как показано в следующей структуре таблицы SQL, база данных `cc_hits` содержит основную часть записанной информации о покрытии кода во время выполнения:

```
CREATE TABLE `paimei`.`cc_hits` (
  `target_id` int(10) unsigned NOT NULL default '0',
  `tag_id` int(10) unsigned NOT NULL default '0',
  `num` int(10) unsigned NOT NULL default '0',
  `timestamp` int(10) unsigned NOT NULL default '0',
  `eip` int(10) unsigned NOT NULL default '0',
  `tid` int(10) unsigned NOT NULL default '0',
  `eax` int(10) unsigned NOT NULL default '0',
  `ebx` int(10) unsigned NOT NULL default '0',
  `ecx` int(10) unsigned NOT NULL default '0',
  `edx` int(10) unsigned NOT NULL default '0',
  `edi` int(10) unsigned NOT NULL default '0',
  `esi` int(10) unsigned NOT NULL default '0',

  `ebp` int(10) unsigned NOT NULL default '0',
  `esp` int(10) unsigned NOT NULL default '0',
  `esp_4` int(10) unsigned NOT NULL default '0',
  `esp_8` int(10) unsigned NOT NULL default '0',
  `esp_c` int(10) unsigned NOT NULL default '0',
  `esp_10` int(10) unsigned NOT NULL default '0',
  `eax_deref` text NOT NULL,
  `ebx_deref` text NOT NULL,
  `ecx_deref` text NOT NULL,
  `edx_deref` text NOT NULL,
  `edi_deref` text NOT NULL,
  `esi_deref` text NOT NULL,
  `ebp_deref` text NOT NULL,
  `esp_deref` text NOT NULL,
  `esp_4_deref` text NOT NULL,
  `esp_8_deref` text NOT NULL,
  `esp_c_deref` text NOT NULL,
  `esp_10_deref` text NOT NULL,
  `is_function` int(1) unsigned NOT NULL default '0',
  `module` varchar(255) NOT NULL default '',
  `base` int(10) unsigned NOT NULL default '0',
  PRIMARY KEY (`target_id`,`tag_id`,`num`),
  KEY `tag_id` (`tag_id`),
  KEY `target_id` (`target_id`)
) ENGINE=MyISAM;
```

Поля таблицы `cc_hits` разбираются следующим образом:

• *target_id u tag_id*. Эти два поля связывают каждую отдельную строку этой таблицы с конкретным сочетанием цели и тега.
• *num*. В этом поле хранится порядковый номер, показывающий, когда представленный отдельной строкой блок кода выполнялся внутри указанного сочетания цели и тега.

• *timestamp*. Здесь хранится число секунд с начала эпохи UNIX (1 января 1970 года, 00:00:00 GMT) - формат, который легко преобразовать в другие представления.
• *eip*. В этом поле хранится абсолютный адрес выполненного блока кода, который представляет отдельная строка; это поле обычно форматируется как шестнадцатеричное число.
• *tid*. В этом поле хранится идентификатор потока, отвечавшего за выполнение блока кода по адресу `eip`. Идентификатор потока назначается ОС Windows и сохраняется, чтобы различать блоки кода, выполненные разными потоками многопоточного приложения.
• *eax, ebx, ecx, edx, edi, esi, ebp u esp*. Эти поля содержат числовые значения каждого из восьми регистров общего назначения в момент выполнения. Для каждого из регистров общего назначения и сохраненных смещений стека существует поле `deref`, содержащее ASCII-данные по конкретному регистру, рассматриваемому как указатель. В конце строки ASCII-данных находится тег - один из `(stack)`, `(heap)` или `(global)`, представляющий место, где были найдены данные, полученные разыменованием. Обозначение `N/A` используется в случаях, когда разыменование невозможно, поскольку соответствующий регистр не содержал адреса, пригодного для разыменования.
• *esp_4, esp_8, esp_c u esp_10*. Эти поля содержат числовые значения по соответствующим смещениям стека (`esp_4 = [esp+4]`, `esp_8 = [esp+8]` и так далее).
• *is_function*. Это логическое поле, в котором значение 1 означает, что блок данных с попаданием (по адресу `eip`) является началом функции.
• *module*. В этом поле хранится имя модуля, в котором произошло попадание.
• *base*. Это поле содержит числовой базовый адрес модуля, указанного в предыдущем поле. Эту информацию можно использовать совместно с `eip`, чтобы вычислить смещение внутри модуля, по которому произошло попадание в блок данных.

Открытое хранилище данных предоставляет опытным пользователям возможность создавать специальные и более сложные отчеты помимо тех, которые инструмент PStalker предоставляет по умолчанию. Испытаем средство отслеживания в полноценном разборе конкретного случая.

Разбор конкретного случая

В практическом разборе конкретного случая мы применим инструмент покрытия кода PStalker, чтобы определить относительную полноту аудита с фаззингом. Наше целевое программное обеспечение - Gizmo Project,[11] комплекс для связи через VoIP, обмена мгновенными сообщениями (IM) и телефонии. На рис. 23.6 приведен снабженный комментариями снимок экрана с веб-сайта Gizmo Project, подробно показывающий основные функции инструмента.

Gizmo реализует впечатляющее число функций, которые входят в число причин его стремительно растущей популярности. Многие считают Gizmo возможным "убийцей" Skype,[12] и от повсеместно распространенного Skype он отличается тем, что построен на открытых стандартах VoIP, таких как Session Initiation Protocol (SIP - RFC 3261), а не реализует закрытую проприетарную систему. Это решение позволяет Gizmo взаимодействовать с другими решениями, совместимыми с SIP. Оно также позволяет нам тестировать Gizmo стандартными инструментами проверки безопасности VoIP. Теперь, когда прицел наведен на целевое программное обеспечение, мы должны выбрать целевой протокол и подходящий фаззер.

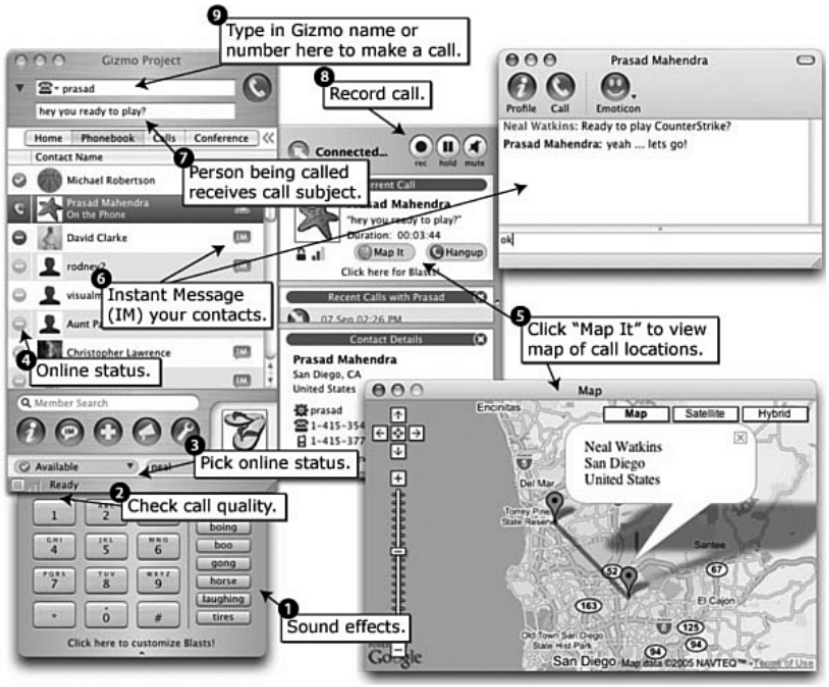


Рис. 23.6. Список функций Gizmo Project

Стратегия

Мы будем проверять способность Gizmo обрабатывать некорректно сформированные пакеты SIP. Хотя можно было бы выбрать в качестве цели несколько протоколов VoIP, SIP - хороший выбор, поскольку большая часть телефонной связи VoIP начинается с согласования сигнализации через этот протокол. Кроме того, уже существует несколько свободных фаззеров SIP. Для нашего теста мы будем использовать свободно распространяемый фаззер SIP, выпущенный кафедрой электротехники и информационной инженерии Университета Оулу, под названием PROTOS Test-Suite: c07-sip.[13] Набор тестов PROTOS содержит 4527 отдельных тестовых случаев, распространяемых в одном файле Java JAR. Хотя этот набор тестов ограничен проверкой только сообщений INVITE, он прекрасно подходит для наших нужд. Одна из причин выбора этого фаззера состоит в том, что группа, отвечающая за его разработку, выпустила коммерческую версию через компанию Codenomicon.[14] Коммерческая версия набора тестов SIP исследует протокол SIP глубже и включает 31 971 отдельный тестовый случай.

Наша цель такова: использовать PStalker во время выполнения полного набора тестовых случаев PROTOS, чтобы измерить достигнутое покрытие кода. Этот разбор конкретного случая можно расширить для выполнения таких практических задач, как сравнение решений для фаззинга или определение степени уверенности в данном аудите. Отдельные преимущества использования покрытия кода в качестве показателя подробнее обсуждаются далее в главе.

Набор тестов PROTOS запускается из командной строки и предоставляет почти 20 параметров для настройки его поведения во время выполнения, показанных здесь в выводе ключа справки:

```
$ java -jar c07-sip-r2.jar -h
Usage java -jar <jarfile>.jar [ [OPTIONS] | -touri <SIP-URI> ]
-touri  <addr>        Recipient of the request
Example: <addr> : you@there.com
-fromuri <addr>       Initiator of the request
Default: user@pamini.unity.local
-sendto <domain>      Send packets to <domain> instead of
                      domainname of -touri
```

```
-callid <callid>      Call id to start test-case call ids from
Default: 0
-dport <port>         Portnumber to send packets on host.
Default: 5060
-lport <port>         Local portnumber to send packets from
Default: 5060
-delay <ms>           Time to wait before sending new test-case
Defaults to 100 ms (milliseconds)
-replywait <ms>       Maximum time to wait for host to reply
Defaults to 100 ms (milliseconds)
-file <file>          Send file <file> instead of test-case(s)
-help                 Display this help
-jarfile <file>       Get data from an alternate bugcat
                      JAR-file <file>
-showreply            Show received packets
-showsent             Show sent packets
-teardown             Send CANCEL/ACK
-single <index>       Inject a single test-case <index>
-start <index>        Inject test-cases starting from <index>
-stop <index>         Stop test-case injection to <index>
-maxpdusize <int>     Maximum PDU size
Default to 65507 bytes
-validcase            Send valid case (case #0) after each
                      test-case and wait for a response. May
                      be used to check if the target is still
                      responding. Default: off
```

Ваши результаты могут отличаться, но мы обнаружили, что лучше всего работают следующие параметры (при этом обязательным является только параметр `touri`):

```
java -jar c07-sip-r2.jar -touri 17476624642@10.20.30.40 \
    -teardown                    \
    -sendto 10.20.30.40          \
    -dport 64064                 \
    -delay 2000                  \
    -validcase
```

Параметр задержки в две секунды дает Gizmo время освободить ресурсы, обновить GUI и иным образом восстановиться между тестовыми случаями. Параметр `validcase` указывает PROTOS проверять после каждого тестового случая и до продолжения, что допустимый обмен данными по-прежнему возможен. Этот примитивный метод позволяет фаззеру обнаружить, что с целью что-то пошло не так, и прекратить тестирование. В нашей ситуации это важный фактор, потому что Gizmo действительно дает сбой! К счастью для Gizmo, уязвимость срабатывает из-за разыменования нулевого указателя и потому не может быть эксплуатирована. Тем не менее проблема существует (подсказка: она возникает в пределах первых 250 тестовых случаев). Загрузите набор тестов и выясните сами.

```
Gizmo Context Dump at Time of Crash
[*] 0x004fd5d6 mov eax,[esi+0x38] from thread 196 caused access violation when
attempting to read from 0x00000038

CONTEXT DUMP
  EIP: 004fd5d6 mov eax,[esi+0x38]
  EAX: 0419fdfc (  68812284) -> <CCallMgr::IgnoreCall() (stack)
  EBX: 006ca788 (   7120776) -> e(elllllllllllllllllllll (PGPlsp.dll.data)
  ECX: 00000000 (         0) -> N/A
  EDX: 00be0003 (  12451843) -> N/A
  EDI: 00000000 (         0) -> N/A
  ESI: 00000000 (         0) -> N/A
  EBP: 00000000 (         0) -> N/A
  ESP: 0419fdd8 (  68812248) -> NR (stack)
  +00: 861c524e (2250003022) -> N/A
  +04: 0065d7fa (   6674426) -> N/A
  +08: 00000001 (         1) -> N/A
  +0c: 0419fe4c (  68812364) -> xN (stack)
  +10: 0419ff9c (  68812700) -> raOo|hoho||@ho0@*@b0zp (stack)
  +14: 0061cb99 (   6409113) -> N/A
disasm around:
```

```
  0x004fd5c7 xor eax,esp
  0x004fd5c9 push eax

  0x004fd5ca lea eax,[esp+0x24]
  0x004fd5ce mov fs:[0x0],eax
  0x004fd5d4 mov esi,ecx
  0x004fd5d6 mov eax,[esi+0x38]
  0x004fd5d9 push eax
  0x004fd5da lea eax,[esp+0xc]
  0x004fd5de push eax
  0x004fd5df call 0x52cc60
  0x004fd5e4 add esp,0x8

SEH unwind:
  0419ff9c -> 006171e8: mov edx,[esp+0x8]
  0419ffdc -> 006172d7: mov edx,[esp+0x8]
  ffffffff -> 7c839aa8: push ebp
```

Параметры `touri` и `dport` были получены посредством изучения пакетов, проходивших к порту 5060 или от него (стандартный порт SIP) во время запуска Gizmo. На рис. 23.7 показан соответствующий фрагмент с выделенными интересующими значениями.

Число 17476624642 - фактический телефонный номер, назначенный нашей учетной записи Gizmo, а порт 64064, по-видимому, является стандартным клиентским портом SIP для Gizmo. Наша стратегия готова. Целевое программное обеспечение, протокол и фаззер выбраны, а параметры фаззера определены. Теперь пора перейти к тактической части разбора конкретного случая.



Рис. 23.7. Декодированный захват пакета SIP при запуске Gizmo

Тактика

Мы подвергаем фаззингу SIP, поэтому нас интересует именно код обработки SIP. К счастью для нас, код легко изолировать, поскольку имя библиотеки `SIPPhoneAPI.dll` сразу выделяется как наша цель. Первым делом загрузите библиотеку в IDA Pro и запустите сценарий `pida_dump.py`, чтобы создать `SIPPhoneAPI.pida`. Поскольку нас интересует только отслеживание покрытия кода, указание "basic blocks" в качестве глубины анализа для сценария `pida_dump.py` повысит производительность. Далее запустим PaiMei и последовательно выполним необходимые предварительные действия, чтобы привести покрытие кода в рабочее состояние. Дополнительные сведения о конкретном использовании PaiMei приведены в онлайновой документации.[15] Первый этап, показанный на рис. 23.8, - создать цель, которую мы называем "Gizmo", и добавить тег с именем "Idle". Мы используем тег "Idle" для записи покрытия кода внутри Gizmo в состоянии "покоя". Это необязательный этап, но, поскольку для записи можно указать несколько тегов и, кроме того, выбирать отдельные теги для фильтрации, максимально возможная детализация никогда не повредит. Затем тег "Idle" необходимо выбрать для "Stalking", как показано на рис. 23.9, а файл PIDA SIPPhoneAPI необходимо загрузить, как показано на рис. 23.10.

После настройки источников данных и выбора тега мы выбираем целевой процесс и параметры захвата, после чего начинаем записывать покрытие кода. На рис. 23.11 показаны настроенные параметры, которые мы будем использовать. Кнопка Refresh Process List получает список выполняющихся в настоящее время процессов. В нашем примере мы решаем подключиться к уже выполняющемуся экземпляру Gizmo, а не загружать новый экземпляр с самого запуска.

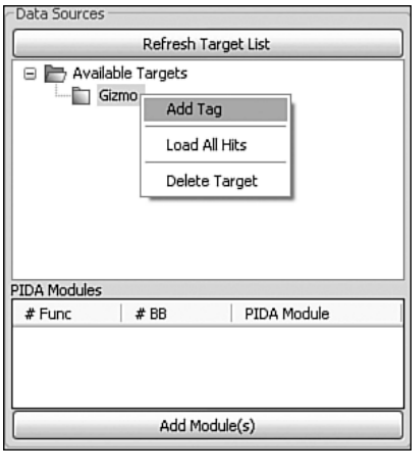


Рис. 23.8. Создание цели и тега PaiMei PStalker

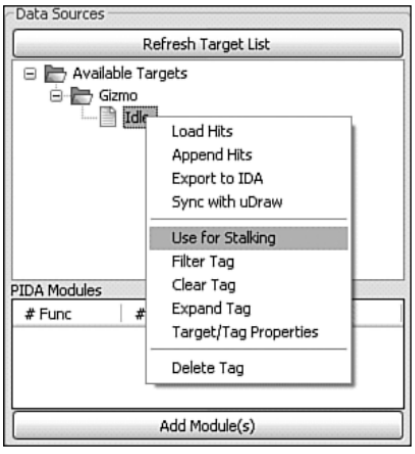


Рис. 23.9. Выбор тега "Idle" для наблюдения

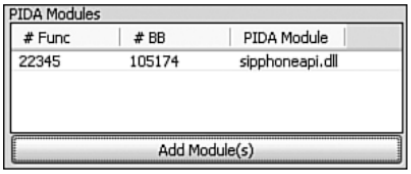


Рис. 23.10. Загруженный модуль PIDA SIPPhoneAPI

В разделе Coverage Depth мы выбираем Basic Blocks, чтобы получить наиболее детализированный уровень покрытия кода. Как вы ранее видели на рис. 23.10, наша целевая библиотека содержит чуть менее 25 000 функций и более чем в четыре раза больше базовых блоков. Поэтому, если для нас важна скорость, мы можем пожертвовать детализацией ради повышения производительности, выбрав вместо этого Functions в качестве глубины покрытия.

Флажок Restore BPs определяет, следует ли продолжать отслеживание базовых блоков после первого попадания в них. Нас интересует только то, куда привело нас выполнение, поэтому мы можем оставить этот параметр отключенным и повысить производительность.

Наконец, мы снимаем флажок Heavy. Этот флажок определяет, должен ли инструмент покрытия кода выполнять проверку контекста времени выполнения в каждой точке останова и сохранять обнаруженное содержимое. И снова, поскольку нас интересует только покрытие кода, дополнительные данные не нужны, поэтому мы избегаем снижения производительности из-за их бесполезного захвата.

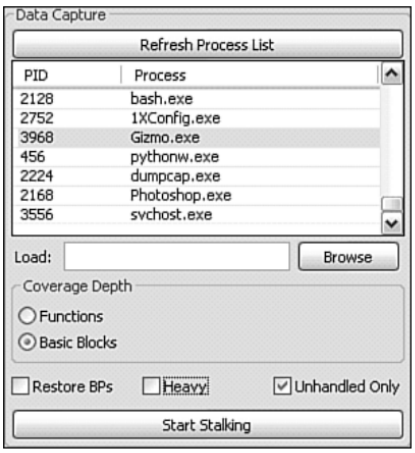


Рис. 23.11. Параметры захвата покрытия кода

После щелчка по "Start Stalking" инструмент покрытия кода подключается к Gizmo и начинает отслеживать выполнение. Если вас интересуют конкретные подробности того, как процесс протекает за кулисами, обратитесь к документации PaiMei. После активации сообщения журнала, указывающие действия и ход работы PStalker, записываются на панель журнала. Следующий фрагмент примера демонстрирует успешный запуск:

```
[*] Stalking module sipphoneapi.dll
[*] Loading 0x7c900000 \WINDOWS\system32\ntdll.dll
[*] Loading 0x7c800000 \WINDOWS\system32\kernel32.dll
[*] Loading 0x76b40000 \WINDOWS\system32\winmm.dll
[*] Loading 0x77d40000 \WINDOWS\system32\user32.dll
[*] Loading 0x77f10000 \WINDOWS\system32\gdi32.dll
[*] Loading 0x77dd0000 \WINDOWS\system32\advapi32.dll
[*] Loading 0x77e70000 \WINDOWS\system32\rpcrt4.dll
[*] Loading 0x76d60000 \WINDOWS\system32\iphlpapi.dll
[*] Loading 0x77c10000 \WINDOWS\system32\msvcrt.dll
[*] Loading 0x71ab0000 \WINDOWS\system32\ws2_32.dll
[*] Loading 0x71aa0000 \WINDOWS\system32\ws2help.dll
```

```
[*] Loading 0x10000000 \Internet\Gizmo Project\SipphoneAPI.dll
[*] Setting 105174 breakpoints on basic blocks in SipphoneAPI.dll
[*] Loading 0x16000000 \Internet\Gizmo Project\dnssd.dll
[*] Loading 0x006f0000 \Internet\Gizmo Project\libeay32.dll
[*] Loading 0x71ad0000 \WINDOWS\system32\wsock32.dll
[*] Loading 0x7c340000 \Internet\Gizmo Project\MSVCR71.DLL
[*] Loading 0x00340000 \Internet\Gizmo Project\ssleay32.dll
[*] Loading 0x774e0000 \WINDOWS\system32\ole32.dll
[*] Loading 0x77120000 \WINDOWS\system32\oleaut32.dll
[*] Loading 0x00370000 \Internet\Gizmo Project\IdleHook.dll

[*] Loading 0x61410000 \WINDOWS\system32\urlmon.dll
...
[*] debugger hit 10221d31 cc #1
[*] debugger hit 10221d4b cc #2
[*] debugger hit 10221d67 cc #3
[*] debugger hit 10221e20 cc #4
[*] debugger hit 10221e58 cc #5
[*] debugger hit 10221e5c cc #6
[*] debugger hit 10221e6a cc #7
[*] debugger hit 10221e6e cc #8
[*] debugger hit 10221e7e cc #9
[*] debugger hit 10221ea4 cc #10
[*] debugger hit 1028c2d0 cc #11
[*] debugger hit 1028c30d cc #12
[*] debugger hit 1028c369 cc #13
[*] debugger hit 1028c37b cc #14
...
```

Теперь мы можем активировать фаззер и отслеживать код, выполняемый во время простоя. Чтобы продолжить, мы ждем, пока список точек останова с попаданиями не начнет сходить на нет, что указывает на исчерпание кода в библиотеке SIP, который выполняется, пока Gizmo бездействует. Затем мы создаем новый тег и выбираем его в качестве активного через меню правой кнопки мыши, выбрав Use for Stalking. Наконец, мы щелкаем правой кнопкой мыши по созданному ранее тегу "Idle" и выбираем Filter Tag. В результате все базовые блоки, выполнявшиеся в состоянии простоя, фактически игнорируются. Теперь можно запустить фаззер и наблюдать за его работой. На рис. 23.12 показано покрытие кода, наблюдаемое в ответ на все 4527 тестовых случаев PROTOS.

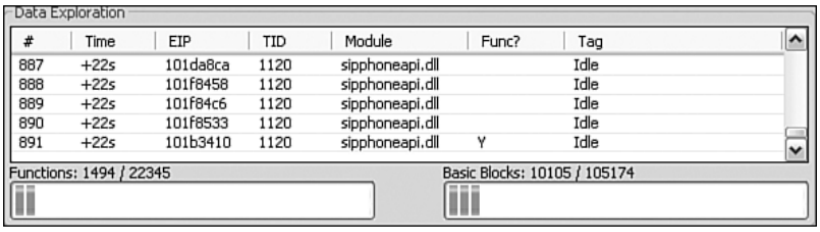


Рис. 23.12. Результаты покрытия кода

В заключение: набор тестов PROTOS смог достичь лишь примерно 6 процентов функций и 9 процентов базовых блоков внутри библиотеки SIPPhoneAPI. Для бесплатного набора тестов это неплохо, но результат показывает, почему запуск одного набора тестов не равнозначен комплексному аудиту. После выполнения набора тестов мы не проверили большую часть кода, и маловероятно, что мы всесторонне проверили функциональность внутри того кода, которого все же достигли. Однако эти результаты покрытия можно улучшить, добавив механизм перезапуска процесса между короткими интервалами тестовых случаев. Причина состоит в том, что, хотя Gizmo успешно отвечает на допустимые тестовые случаи, которые мы поручили PROTOS вставлять между некорректно сформированными тестами, спустя некоторое время Gizmo все же перестает работать правильно, и покрытие кода страдает. Перезапуск процесса сбрасывает внутреннее состояние и восстанавливает полную функциональность, позволяя нам точнее и тщательнее исследовать цель.

Учитывая, что PROTOS содержит лишь 1/7 числа тестовых случаев своей коммерческой сестры, можно ли ожидать от коммерческой версии семикратного увеличения покрытия кода? Вероятно, нет. Хотя ожидается увеличение покрытия кода, между числом тестовых случаев и полученным объемом задействованного кода не существует постоянного или предсказуемого соотношения. Точными могут быть только эмпирические результаты.

Итак, что мы получаем от отслеживания фаззеров таким образом и как можем усовершенствовать этот метод? Следующий раздел дает некоторые ответы на эти вопросы.

Преимущества и будущие усовершенствования

Традиционно фаззинг проводился не слишком научным образом. Часто исследователи безопасности и инженеры QA на скорую руку собирают примитивные сценарии для создания некорректно сформированных данных, позволяют своему творению выполняться произвольный период времени и на этом заканчивают. С недавним распространением коммерческих решений для фаззинга вы встретите организации, которые делают еще один шаг к научному тестированию: используют исчерпывающий список тестовых случаев, разработанный коммерческим поставщиком, таким как Codenomicon. Исследователям безопасности, которые просто ищут уязвимости, чтобы сообщить о них или продать, часто нет необходимости развивать процесс дальше. При нынешнем состоянии безопасности программного обеспечения существующих методологий достаточно для создания бесконечного длинного списка ошибок и уязвимостей. Однако по мере роста зрелости и осведомленности как в безопасной разработке, так и в тестировании возникнет потребность в более научном подходе. Разработчики программ и группы QA, заинтересованные в обнаружении всех уязвимостей, а не только низко висящих плодов, будут больше заботиться о применении более научного подхода к фаззингу. Отслеживание фаззера - необходимый шаг в этом направлении.

Продолжая использовать тот же пример, что и на протяжении всей главы, рассмотрим различные стороны, участвующие в разработке последнего выпуска софтфона VoIP, и преимущества, которые они могут получить от отслеживания фаззера. Для менеджера продукта знание конкретных областей проверенного кода позволяет точнее и с большей уверенностью определить уровень тестирования, которому подвергся их продукт. Например, рассмотрим следующее утверждение: "Последний выпуск нашего софтфона VoIP прошел 45 000 вредоносных тестовых случаев". Никаких доказательств в подтверждение этого заявления не предоставляется. Возможно, из 45 000 вредоносных случаев только 5000 охватывали функции, предоставляемые целевым софтфоном. Также возможно, что весь набор тестов задействовал лишь 10 процентов доступных функций, предоставляемых целевым софтфоном. Кто знает, возможно, тестовые случаи написали по-настоящему блестящие инженеры и они покрыли всю кодовую базу. Последний вариант был бы замечательным, но, опять же, без анализа покрытия кода невозможно определить достоверность первоначального заявления. Объединение средства отслеживания фаззера с тем же набором тестов могло бы привести к следующему, более содержательному утверждению: "Последний выпуск нашего софтфона VoIP прошел 45 000 вредоносных тестовых случаев с исчерпывающей проверкой более 90 процентов нашей кодовой базы".

Инженеру QA определение того, какие области софтфона не были проверены, позволяет создать более интеллектуальные тестовые случаи для следующего раунда фаззинга. Например, во время отслеживания фаззера при работе с софтфоном инженер мог заметить, что создаваемые фаззером тестовые случаи постоянно запускают процедуру с именем `parse_sip()`. Углубившись в эту процедуру, он замечает: хотя сама функция выполняется много раз, покрыты не все ветви внутри нее. Фаззер не исчерпывает логические возможности внутри процедуры разбора. Чтобы улучшить тестирование, инженеру следует исследовать непокрытые ветви и определить, какие

изменения можно внести в процедуры создания данных фаззера, чтобы задействовать эти пропущенные ветви.

Разработчику знание конкретного набора инструкций и пути, выполненного до любого обнаруженного сбоя, позволяет быстро и уверенно найти и исправить затронутую область кода. В настоящее время взаимодействие между инженерами QA и разработчиками может происходить на более высоком уровне, из-за чего обе стороны тратят время на проведение перекрывающихся исследований. Вместо получения от QA высокоуровневого отчета со списком серии тестовых случаев, вызывающих различные сбои, разработчики могут получить отчет с подробным описанием возможных мест сбоев вместе с тестовыми случаями.

Помните, что покрытие кода как отдельный показатель не подразумевает тщательного тестирования. Представьте, что операция копирования строки внутри исполняемого файла достигнута, но проверена только на строках малой длины. Код может быть уязвим для переполнения буфера, которое наш инструмент тестирования не обнаружил, поскольку через него была передана недостаточно длинная или неправильно отформатированная строка. Определить, что именно мы проверили, - половина дела; в идеале мы также предоставляли бы показатели того, насколько хорошо было проверено то, чего мы достигли.

Будущие усовершенствования

Одна удобная функция, которую стоит рассмотреть, - добавление возможности обмена данными между инструментом покрытия кода и фаззером. До и после передачи каждого отдельного тестового случая фаззер может уведомлять инструмент покрытия кода о транзакции, тем самым связывая и записывая покрытие кода, относящееся к каждому отдельному тестовому случаю. Альтернативный подход к реализации состоял бы в последующей обработке путем сопоставления временных меток, связанных с передачами данных, и временных меток, связанных с покрытием кода. Обычно это менее эффективно из-за задержки и неидеально синхронизированных часов, но также проще реализуется. В обоих случаях конечный результат позволяет аналитику углубиться в конкретные блоки кода, выполненные в ответ на отдельный тестовый случай.

Нынешнее требование перечислить местоположения всех базовых блоков и функций внутри цели представляет собой кропотливый и подверженный ошибкам процесс. Одно из решений этой проблемы - перечислять базовые блоки во время выполнения. Согласно руководству разработчика программного обеспечения для архитектуры Intel IA-32,[16] тому 3B,[17] разделу 18.5.2, процессоры Pentium 4 и Xeon на аппаратном уровне поддерживают "одиночный шаг по переходам":

> **Флаг BTF (одиночный шаг по переходам) (бит 1)**
>
> Когда он установлен, процессор рассматривает флаг TF в регистре EFLAGS как флаг "одиночного шага по переходам", а не как флаг "одиночного шага по инструкциям". Этот механизм позволяет выполнять процессор по одному шагу на предпринятых переходах, прерываниях и исключениях. См. раздел 18.5.5 "Одиночное выполнение по переходам, исключениям и прерываниям".

Использование этой функции позволяет разработать инструмент, способный трассировать целевой процесс на уровне базовых блоков без необходимости какого-либо априорного анализа. Также достигается выигрыш в скорости выполнения, поскольку больше не требуется устанавливать программные точки останова на каждом отдельном базовом блоке при запуске. Однако помните, что статический анализ и перечисление базовых блоков по-прежнему необходимы для определения полного масштаба кодовой базы и заявлений о проценте покрытия. Кроме того, хотя при таком подходе начало трассировки может быть быстрее, в долгосрочной перспективе он, вероятно, не будет быстрее описанного ранее метода. Причины - невозможность прекратить

отслеживание уже выполненных блоков, а также невозможность трассировать только конкретную модель. Тем не менее это интересный и мощный метод. Развивая показанный ранее в этой главе пошаговый исполнитель PyDbg, рабочий пошаговый исполнитель уровня переходов можно найти в записи блога "Branch Tracing with Intel MSR Registers"[18] на OpenRCE. Подробное исследование этого подхода оставлено читателю в качестве упражнения.

В качестве последнего замечания об усовершенствовании рассмотрим различие между покрытием кода и покрытием путей (или покрытием состояний процесса). То, что мы уже обсуждали в этой главе, относится к покрытию кода; иными словами, к тому, какие отдельные инструкции или блоки кода фактически выполнялись. Покрытие путей дополнительно учитывает различные возможные пути, по которым можно пройти через кластер блоков кода. Рассмотрим, например, кластер блоков кода, показанный на рис. 23.13.

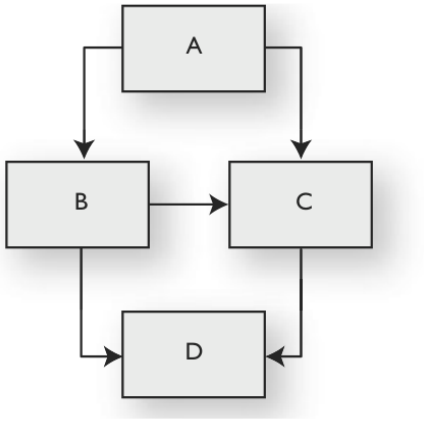


Рис. 23.13. Пример графа потока управления

Анализ покрытия кода может показать, что серия тестовых случаев задействовала все четыре блока A, B, C и D. Были ли задействованы все пути? Если да, какие пути задействовал каждый отдельный тестовый случай? Ответы на эти вопросы требуют покрытия путей - метода, который позволяет дополнительно выяснить, какие из следующих возможных сочетаний путей были покрыты:

• A -> B -> D
• A -> B -> C -> D
• A -> C -> D

Если, например, были задействованы только первый и третий пути в этом списке, анализ покрытия кода показал бы 100-процентное покрытие. Покрытие путей, напротив, показало бы, что покрыты лишь 66 процентов возможных путей. Эта дополнительная информация может позволить аналитику дополнительно настроить и улучшить список тестовых случаев.

Резюме

В этой главе мы представили концепцию покрытия кода применительно к фаззингу, начав с упрощенного решения, реализованного всего в нескольких строках Python. Изучив строительные блоки двоичного исполняемого кода, мы продемонстрировали более совершенный и ранее реализованный подход. Хотя прежде большая часть внимания общественности к фаззингу уделялась созданию данных, этот подход решает лишь половину проблемы. Нам не следует спрашивать только: "Как начать фаззинг?" Нужно также спросить: "Когда прекратить фаззинг?" Без отслеживания покрытия кода мы не можем осмысленно определить, когда наших усилий достаточно.

Был представлен и показан удобный для пользователя инструмент анализа покрытия кода с открытым исходным кодом PaiMei PStalker. Было исследовано применение этого инструмента для фаззинга, а также обсуждены области для усовершенствования. Применение этого инструмента и концепции покрытия кода, или "отслеживания фаззера", еще на один шаг приближает нас к более научному подходу к тестированию фаззингом. В следующей главе мы обсудим интеллектуальные методы обнаружения и обработки исключений, которые можно дополнительно включить в наш набор тестирования, выведя фаззинг на новый уровень.

Сноски

[1] [назад] http://www.intel.com/intelpress/chapter-scientific.pdf
[2] [назад] Точное число инструкций, предоставляемых этими архитектурами, может быть предметом споров, но общая идея остается в силе. Архитектуры CISC реализуют гораздо больше инструкций, чем их аналоги RISC.
[3] [назад] http://www.openrce.org/blog/view/575
[4] [назад] http://www.datarescue.com
[5] [назад] http://www.microsoft.com/technet/security/bulletin/MS05-030.mspx
[6] [назад] http://www.openrce.org/downloads/details/171/Process_Stalker
[7] [назад] https://www.openrce.org/articles/full_view/12
[8] [назад] http://www.hbo.com/sopranos/
[9] [назад] http://openrce.org/downloads/details/208/PaiMei
[10] [назад] http://pedram.openrce.org/PaiMei/docs/PAIMEIpstalker_flash_demo/
[11] [назад] http://www.gizmoproject.com/
[12] [назад] http://www.skype.com/
[13] [назад] http://www.ee.oulu.fi/research/ouspg/protos/testing/c07/sip/
[14] [назад] http://www.codenomicon.com/
[15] [назад] http://pedram.openrce.org/PaiMei/docs/
[16] [назад] http://www.intel.com/products/processor/manuals/index.htm
[17] [назад] ftp://download.intel.com/design/Pentium4/manuals/25366919.pdf
[18] [назад] http://www.openrce.org/blog/view/535

Глава 24. Интеллектуальное обнаружение сбоев

> "Никогда больше в залах Вашингтона, округ Колумбия, я не хочу давать объяснения, которые не могу объяснить".
>
> - Джордж У. Буш, Портленд, штат Орегон, 31 октября 2000 года

Мы знаем, как выбрать цель. Мы знаем, как создавать данные. Благодаря предыдущей главе мы знаем, как отслеживать, где и как обрабатываются наши данные. Следующая важная концепция, которой необходимо овладеть, - решение о том, когда наш фаззер успешно создал неприятности. Это не всегда очевидно само по себе, поскольку результат сбоя может быть нелегко обнаружить извне целевой системы. И снова эта тема не получила большого внимания в доступных сегодня фаззерах. Однако, в отличие от отслеживания фаззера, мы начинаем видеть успехи как в коммерческой сфере, так и в сфере открытого исходного кода.

В этой главе мы начнем с изучения самых примитивных решений для обнаружения сбоев, таких как простой разбор ответов. Затем перейдем на следующий, более сложный уровень, используя отладочные инструменты и технологии. Наконец, мы кратко коснемся наиболее совершенных методов обнаружения сбоев, исследовав динамическое инструментирование двоичных файлов (dynamic binary instrumentation, DBI). В целом инструменты, методы и концепции, показанные в этой главе, помогут нам определять, когда наш фаззер хорошо постарался, поступая плохо.

Примитивное обнаружение сбоев

Что вы получите, если вслепую отправите 50 000 некорректно сформированных запросов аутентификации IMAP, один из которых вызовет сбой сервера IMAP, и не будете проверять состояние сервера IMAP до последнего тестового случая? Все просто: вы не получите ничего. Не имея хотя бы общего представления о том, какой из тестовых случаев фактически вызвал сбой сервера IMAP, вы останетесь с таким же малым объемом информации, какой имели до запуска фаззера. Фаззер, который не имеет представления о состоянии своей цели, граничит с полной бесполезностью.

Существует ряд методов, которые фаззер может применять, чтобы определить, обнаружил ли отдельный тестовый случай какие-либо аномалии в цели; возможно, простейший из них - добавить проверку соединения между тестовыми случаями, как это делал PROTOS в предыдущей главе. Продолжая наш пример с IMAP, предположим, что мы работаем с фаззером, создающим следующие первые два тестовых случая:

```
x001 LOGIN AAAAAAAAAAAAAAAAAAAAAAAAAAAA...AAAA
x001 LOGIN %s%s%s%s%s%s%s%s%s%s...%s%s%s%s%s
```

Между каждым созданным тестовым случаем фаззер может пытаться установить TCP-соединение через порт 143 (порт сервера IMAP). В случае неудачи можно предположить, что последний тестовый случай вызвал сбой сервера. Коммерческие наборы тестов, выпускаемые Codenomicon, применяют этот подход: они включают функцию необязательной отправки допустимого, или заведомо хорошего, случая после каждого тестового случая. Полученный псевдокод для выполнения этой задачи очень прост:

```
for case in test_cases:
    fuzzer.send(case)
    if not fuzzer.tcp_connect(143):
        fuzzer.log_fault(case)
```

Мы можем немного улучшить эту логику, заменив проверку соединения тестом с заведомо допустимым ответом. Например, если мы знаем, что имя пользователя `paimei` и пароль `whiteeyebrow` являются допустимыми учетными данными на сервере IMAP, мы можем проверить не только доступность сервера, но и правильность его поведения, попытавшись успешно аутентифицироваться как пользователь `paimei`. В случае неудачи можно предположить, что последний тестовый случай вызвал неисправность сервера. И снова полученный псевдокод для выполнения этой задачи очень прост:

```
for case in test_cases:
    fuzzer.send(case)
    if not fuzzer.imap_login("paimei", "whiteeyebrow"):
        fuzzer.log_fault(case)
```

Ранее мы утверждали, что неудача проверки соединения или теста с заведомо допустимым ответом позволяет предположить, что последний тестовый случай вызвал неисправность внутри цели. Это не совсем точно. Представьте, например, что во время сеанса фаззинга сервера IMAP после тестового случая 500 мы получаем неудачу проверки соединения. Прекрасно! Мы обнаружили входные данные, способные вызвать сбой сервера. Однако прежде чем бежать по

коридору и сообщать о нашей великой находке, мы перезапускаем сервер IMAP, отдельно повторяем тестовый случай 500 и, к нашему огорчению, ничего не происходит. Что происходит? Вероятный сценарий состоит в том, что один или, возможно, сочетание десятков наших предыдущих тестовых случаев перевели сервер IMAP в состояние, которое позволило тестовому случаю 500 вывести сервер из строя. Однако сам по себе тестовый случай 500, по-видимому, обрабатывается правильно. В таких ситуациях для сужения круга возможностей мы можем использовать еще один простой метод, который называем пошаговым перебором фаззера.

Общая проблема такова: мы знаем, что где-то между тестовыми случаями 1 и 499 сервер IMAP был переведен в состояние, которое позволило тестовому случаю 500 вызвать его сбой, но не знаем, какое сочетание тестовых случаев отвечало за это. Можно использовать несколько алгоритмов пошагового перебора. Вот простой вариант для начала:

```
# найти верхнюю границу:
for i in xrange(1, 500):
    for j in xrange(1, i + 1):
        fuzzer.send(j)
    fuzzer.send(500)
    if not fuzzer.tcp_connect(143):
        upper_bound = i
        break
    fuzzer.restart_target()

# найти нижнюю границу:
for i in xrange(upper_bound, 0, -1):
    for j in xrange(i, upper_bound + 1):
        fuzzer.send(j)
    fuzzer.send(500)
    if fuzzer.tcp_connect(143):
        lower_bound = i
        break
    fuzzer.restart_target()
```

Первая половина логики отвечает за обнаружение верхней границы последовательности, создающей требуемое состояние. Для этого последовательно проверяются случаи с 1-го по n-й, после которых следует случай 500, причем n постоянно увеличивается на единицу. После проверки каждой последовательности цель фаззинга перезапускается, чтобы сбросить внутреннее состояние. Затем похожая логика применяется для обнаружения нижней границы. В совокупности наша процедура изолировала последовательность тестовых случаев, которая, если за ней следует случай 500, вызовет сбой цели. Восторг возвращается, и мы снова можем бежать по коридору. Разумеется, это сравнительно простой алгоритм, который не учитывает, что на самом деле сервер в уязвимое состояние может переводить пара тестовых случаев из выявленного диапазона. Например, мы можем обнаружить, что тестовые случаи с 15 по 20, за которыми следует тестовый случай 500, вызывают сбой, тогда как в действительности сбой вызовет последовательность из тестовых случаев 15, 20 и 500. Тем не менее алгоритм сузит круг возможностей.

Простые решения, которые мы только что рассмотрели, - самая основа отслеживания неисправностей. Чтобы еще лучше понять, что происходит внутри цели нашего фаззера, мы можем копнуть глубже и взглянуть на низкий уровень с помощью отладчика. Но прежде чем это делать, важно точно определить, что именно мы ищем.

Что мы ищем?

На самом низком уровне ЦП сообщает операционной системе о возникновении различных сбоев, исключений или прерываний (далее совместно именуемых событиями или исключениями), таких как нарушение доступа к памяти или деление на ноль. Многие из этих событий возникают естественным образом. Рассмотрим, например, что происходит, когда процесс пытается обратиться к памяти, которая в данный момент выгружена на диск. Создается страничное исключение, и операционная система обрабатывает событие, загружая соответствующую страницу из дискового кэша в основную память. Эта транзакция происходит совершенно прозрачно для процесса, который сделал исходный запрос на обращение к памяти. События, не обработанные операционной системой, передаются вниз на уровень процесса. Здесь и начинается наш интерес.

Пока наш фаззер с трудом продвигается по целевому процессу или устройству, он может вызвать целый ряд событий. Подключение отладчика к целям фаззера позволяет перехватывать эти события по мере их возникновения. Как только событие захвачено, целевой процесс замораживается в ожидании интерактивной команды. Если объединить это с изложенными ранее упрощенными подходами, то после того, как отладчик перехватит событие и приостановит цель фаззера, последующая проверка соединения или заведомо допустимых входных данных завершится неудачей. Такой объединенный подход позволяет потенциально обнаружить больше проблем в нашем примере сервера IMAP. Поскольку различные события исключений возникают на уровне ассемблера, именно мы должны осмысленно определить высокоуровневый источник события и, что важнее, определить, имеет ли событие значение с точки зрения безопасности. В следующем списке перечислены три основные категории, к которым относятся все уязвимости повреждения памяти в программах. Мы рассмотрим примеры из каждой категории, а также определим типы исключений, которые могут создаваться в ответ на каждую из них:

• Передача управления выполнением по адресу, по которому его не должно быть.
• Чтение данных с адреса, с которого их не следует читать.
• Запись данных по адресу, по которому их не следует записывать.

В качестве примера передачи управления выполнением по адресу, по которому его не должно быть, рассмотрим классическое переполнение стека и следующий фрагмент кода C из функции со стековым кадром, похожим на показанный на рис. 24.1.

```
void taboo (int arg1, char *arg2)
{
    int x;
    char buf[16];
    int y;

    strcpy(buf, arg2);
}
```

Эта искусственная функция принимает целочисленный и строковый аргументы, объявляет три локальные переменные, а затем копирует предоставленную строку в объявленный стековый буфер без учета размера исходной строки или буфера назначения. При вызове функции `taboo()` ее аргументы помещаются в стек в обратном порядке. Полученная ассемблерная инструкция `CALL` неявно помещает в стек адрес текущей инструкции (хранящийся в регистре EIP). Именно так ЦП узнает, куда передать управление обратно после завершения выполнения вызванной функции. Обычно затем пролог функции помещает в стек текущий указатель кадра (хранящийся в регистре EBP). Наконец, для трех объявленных локальных переменных выделяется место путем вычитания общего размера переменных из указателя стека (хранящегося в регистре ESP). Полученный стековый кадр выглядит в точности так, как показано на рис. 24.1.

Как показано на рис. 24.1, вспомните, что стек растет "вниз", от старших адресов к младшим. Однако запись данных происходит в противоположном направлении, "вверх". Если длина строки

arg2 превышает 16 байт (объявленный размер buf), strcpy() продолжит копирование за конец пространства, выделенного для buf, перезаписывая локальную целочисленную переменную x, затем сохраненный указатель кадра, затем сохраненный адрес возврата, затем первый аргумент и так далее. Если предположить, что arg2 - длинная строка из символов A, сохраненный EIP будет перезаписан значением 0x41414141 (0x41 - шестнадцатеричное значение ASCII-символа A).

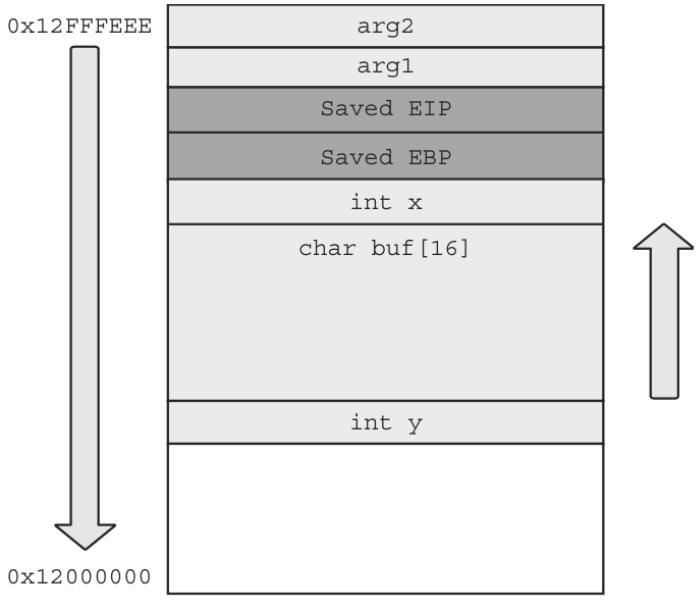


Рис. 24.1. Пример компоновки стека

Когда функция taboo() завершает выполнение, ассемблерная инструкция RETN восстанавливает перезаписанный сохраненный EIP и передает управление тем инструкциям, которые обнаружатся по адресу 0x41414141. В зависимости от содержимого памяти по адресу 0x41414141 возможны несколько разных сценариев. В большинстве случаев на этот конкретный адрес не будет отображено ни одной допустимой страницы памяти, и когда ЦП попытается прочитать следующую инструкцию по недоступному для чтения адресу 0x41414141, будет создано исключение ACCESS_VIOLATION. Обратите внимание: в этом конкретном примере, если длина arg2 превышает 16 байт, но меньше или равна 20 байтам, исключение не возникнет.

Если по этому адресу находятся допустимые данные, но не исполняемый код, а лежащий в основе процессор поддерживает разрешения страниц, запрещающие выполнение (nonexecutable, NX[1]), то, когда ЦП попытается выполнять код по неисполняемому адресу 0x41414141, снова будет создано исключение ACCESS_VIOLATION. Что интереснее всего, если по адресу находится допустимый исполняемый код, исключение создано не будет. Это ключевой момент, к которому мы вернемся далее в главе. Этот пример демонстрирует одну из самых прямолинейных форм перехвата потока управления.

В качестве примера чтения по адресам, по которым читать не следует, рассмотрим следующий фрагмент кода C из функции со стековым кадром, похожим на показанный на рис. 24.1:

```
void taboo_two (int arg1, char *arg2)
{
    int *x;
    char buf[] = "quick brown dog.";
    int y = 10;

    x = &y;

    for (int i = 0; i < arg1; i++)
        printf("%02x\n", buf[i]);
```

```
    strcpy(buf, arg2);
    printf("%d\n", *x)
}
```

Эта функция принимает целочисленный и строковый аргументы, объявляет три локальные переменные и устанавливает целочисленную переменную x как указатель на y. Затем функция продолжает печатать из buf переменное число шестнадцатеричных байтов, заданное целочисленным аргументом arg1. Поскольку проверка разумности аргумента, используемого в качестве условия завершения цикла, не выполняется, значение больше 16 заставит процедуру printf() обратиться к данным и вывести их, хотя делать этого ей не следует.

Как и прежде, данные читаются вверх, в сторону старших адресов, противоположную направлению роста стека. Поэтому цикл раскроет значение локальной целочисленной переменной x, затем сохраненный указатель кадра, затем сохраненный адрес возврата, затем первый аргумент и так далее. В зависимости от величины arg1 исключение ACCESS_VIOLATION при чтении может вообще не возникнуть, поскольку достаточно малые значения просто заставят цикл обратиться к данным, которые программист логически не хотел бы раскрывать, но которыми ЦП и операционная система охотно делятся, так как не знают об их конфиденциальности.

Далее выполняется неограниченное копирование строки, как и в предыдущем примере. Однако если предположить, что arg2 содержит только 20 символов A (на четыре байта больше размера buf), будет перезаписана лишь локальная переменная-указатель x. В такой ситуации переполнение стека не приведет к недопустимой передаче выполнения, как прежде. Однако во время вызова printf(), когда перезаписанная переменная разыменовывается как указатель, при попытке чтения по адресу 0x41414141, вероятно, возникнет ACCESS_VIOLATION.

В качестве более реалистичного примера чтения из места, откуда читать не следует, рассмотрим следующий фрагмент кода, иллюстрирующий раскрытие классической уязвимости форматной строки:

```
void syslog_wrapper (char *message)
{
    syslog(message);
}
```

Согласно прототипу API syslog(), ожидается форматная строка, за которой следует переменное число аргументов форматной строки. При прямой передаче пользовательской строки, содержащейся в аргументе message, функции syslog() все маркеры форматной строки, содержащиеся в аргументе, будут разобраны, а соответствующий аргумент форматной строки будет разыменован. Поскольку таких аргументов форматной строки не существует, будут использованы любые значения, которые в данный момент находятся в стеке функции. Например, если аргумент содержит %s%s%s%s%s, из стека читаются пять адресов и каждый разыменовывается как указатель на строку, пока не встретится нулевой байт. Передача достаточного числа маркеров форматной строки %s обычно приводит к тому, что недопустимый указатель рассматривается как строка, а это приводит к ACCESS_VIOLATION. В некоторых случаях указатель будет указывать на допустимую адресуемую память, но содержимое памяти не будет завершено NULL, из-за чего функция при поиске байта NULL прочитает данные за концом страницы. Обратите внимание: в зависимости от среды и числа предоставленных маркеров форматной строки нарушение может не возникнуть. И снова эта ключевая концепция рассматривается далее в главе.

В качестве примера записи по адресу, по которому записывать не следует, рассмотрим следующий фрагмент кода C из функции со стековым кадром, похожим на показанный на рис. 24.1:

```
void taboo_three (int arg1, char *arg2)
{
    int x;
    char buf[] = "quick brown dog.";
    int y;
    buf[arg1] = '\0';
```

```
}
```

Эта функция принимает целочисленный и строковый аргументы, объявляет три локальные переменные, а затем усекает строку, хранящуюся в `buf`, по индексу, заданному аргументом `arg1`. Поскольку проверка разумности предоставленного индекса не выполняется, значение больше 16 приведет к ошибочной записи нулевого байта по адресу, по которому в иных обстоятельствах записывать не следовало бы. Как мы видели в предыдущем примере, в зависимости от расположения этого адреса фактическое исключение может быть создано, а может и не быть. Даже если исключение не создается, при фаззинге мы хотели бы знать, когда это происходит.

В качестве более реалистичного примера записи в место, куда записывать не следует, рассмотрим следующий фрагмент кода и упрощенный пример, демонстрирующие раскрытие типичной уязвимости переполнения кучи:[2]

```
char *A = malloc(8);
char *B = malloc(16);
char *C = malloc(24);

strcpy(A, "PPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP");
free(B);
```

В этом фрагменте кода объявляются три символьных указателя, которые инициализируются так, чтобы указывать на динамически выделенную память, которая, как говорят, поступает из кучи. Помимо другой хранящейся внутри информации, каждый фрагмент кучи содержит прямую и обратную ссылки, образующие двусвязный список, показанный на рис. 24.2.

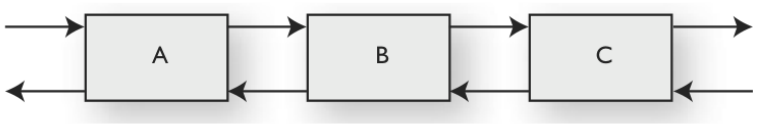


Рис. 24.2. Двусвязный список фрагментов кучи

При достижении вызова `strcpy()` выделенный буфер, на который ссылается A, переполняется, поскольку не может вместить длинную предоставленную строку из символов P. Поскольку переполняемый буфер хранится не в стеке, возможности затереть сохраненный адрес возврата нет. Так в чем же проблема? В следующей строке кода вызов `free(B)` должен удалить B из цепочки и соединить A и C друг с другом. Это выполняется посредством следующей базовой логики:

```
B->backward->forward = B->forward
B->forward->backward = B->backward
```

Поскольку прямой и обратный указатели хранятся в начале каждого фрагмента кучи, переполнение в результате вызова `strcpy()` перезапишет значения этих указателей во фрагменте B значением `0x50505050` (`0x50` - шестнадцатеричное значение ASCII-символа P). На уровне ассемблера это может превратиться как в недопустимые попытки записи, так и в недопустимые попытки чтения.

> **Печальная попытка создать решение безопасности**
>
> Хотя это непосредственно не связано с обсуждаемой темой, мы не можем не воспользоваться возможностью указать на забавную попытку создать решение безопасности, применяемое до компиляции. Некоторое время назад кафедра информатики Университета Дрекселя опубликовала исследовательскую работу под названием "Using Program Transformation to Secure C Programs Against Buffer Overflows".[3]
>
> В работе утверждается, что все эксплуатируемые переполнения буфера можно устранить, просто заменив использование всех стековых буферов буферами, выделенными из кучи.

> На 10 страницах работы рассматривается динамика преобразования, обсуждается эффективность преобразования, приводятся многочисленные ссылки и даже демонстрируется пример того, как их революционное преобразование может превратить искусственную уязвимость с эксплуатацией для выполнения кода в простой отказ в обслуживании. К несчастью для авторов, ни в один момент исследования им не встретился факт, что переполнения кучи также поддаются эксплуатации.

Замечание о выборе значений фаззинга

Теперь, когда мы закончили иллюстрировать, как ряд высокоуровневых фрагментов и концепций уязвимостей программ может преобразовываться в создаваемое низкоуровневое событие, рассмотрим ключевой вопрос, связанный с выбором значений фаззинга. Возможно, вы уже заметили, что в зависимости от различных условий значения фаззинга, которые в итоге используются как адрес памяти, могут фактически не привести к нарушению доступа и, следовательно, к созданию исключения. Это понятие необходимо учитывать при выборе значений для использования в подвергаемых фаззингу запросах. Плохо выбранные значения могут приводить к ложноотрицательным результатам - ситуациям, когда наш фаззер успешно прошел через уязвимый код, но не смог определить его как таковой.

Рассмотрим, например, различные ситуации, в которых четыре байта предоставленных фаззером данных используются при разыменовании памяти. Классическая строка символов A приводит к разыменованию адреса памяти `0x41414141`. Хотя в большинстве случаев на этот конкретный адрес не отображены допустимые страницы памяти, что приводит к ожидаемому нами `ACCESS_VIOLATION`, будут случаи, когда адрес действительно допустим и потому событие исключения не создается. Хотя, по всей вероятности, наш фаззер сделал достаточно, чтобы вызвать исключение еще дальше по ходу выполнения, мы хотели бы узнавать о неполадках как можно раньше. Если бы у вас была возможность выбрать четыре байта, используемые как недопустимый адрес памяти, что бы вы выбрали? Хорошо работает выбор адреса в адресном пространстве ядра (обычно `0x80000000-0xFFFFFFFF`), поскольку пространство ядра никогда не доступно из пользовательского режима.

Рассмотрим другой пример - случай уязвимости форматной строки. Мы видели: если предоставить недостаточное число маркеров `%s`, последующие разыменования указателей стека могут не вызвать никаких недопустимых обращений к памяти. Первым побуждением могло бы быть решить проблему, просто увеличив число маркеров `%s`, используемых в различных запросах фаззинга. Однако дальнейший анализ показал бы, что другие факторы, такие как установленные ограничения длины строки, могут сделать это решение бесполезным. Как обойти эту проблему более изящно? Вспомним: ключ к эксплуатации уязвимостей форматной строки - возможность записывать в стек с помощью маркера форматной строки `%n` и его производных. Маркер форматной строки `%n` записывает по соответствующему указателю стека число символов, которые должна была вывести форматная строка. Предоставив несколько маркеров `%n` вместо маркеров `%s`, мы с большей вероятностью можем вызвать сбой при ограничении более короткой входной строки. Сочетание перемежающихся маркеров `%n%s`, вероятно, будет самым разумным выбором с учетом ситуаций, когда маркер `%n` может быть отфильтрован или отключен (дополнительные сведения см. в главе 6 "Автоматизация и создание данных").

Короче говоря, осознание того, где могут оказаться выбранные вами данные фаззинга и как они могут использоваться, способно принести огромную пользу. Дополнительные сведения о выборе подходящих значений фаззинга приведены в главе 6.

Автоматизированное отслеживание с помощью отладчика

Недостаток отслеживания цели с помощью отладчика состоит в том, что, поскольку большинство отладчиков предназначены для интерактивного использования, об автоматическом извлечении полезной информации из самого отладчика и программном управлении им не может быть и речи. Меньше всего мы хотим вручную проверять наш фаззер и перезапускать процесс всякий раз, когда отладчик обнаруживает исключение. К счастью, мы снова можем воспользоваться помощью представленного ранее каркаса обратного проектирования PaiMei.[4] С помощью этого каркаса мы можем спроектировать систему фаззинга и отслеживания из двух частей, показанную на рис. 24.3. Следует отметить, что этот каркас обратного проектирования в настоящее время предназначен исключительно для использования на платформе Windows и что, как и в остальной части книги, в этом разделе мы сосредоточены на архитектуре Intel IA-32.

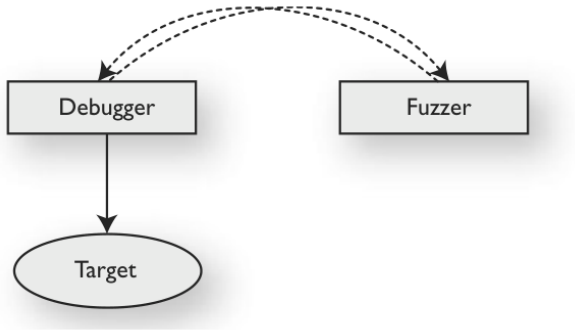


Рис. 24.3. Архитектура фаззера и средства отслеживания

Базовое средство отслеживания с отладчиком

Придерживаясь выбранного примера главы, представим, что наша цель - сервер IMAP. Цель можно либо подключить к отладчику, который мы вскоре создадим, либо загрузить непосредственно под его управлением. Отладчик имеет двусторонний обратный канал связи с нашим фаззером. Обратный канал можно реализовать несколькими способами. Вероятно, самым удобным выбором будут сокеты, поскольку они позволяют не только написать фаззер и отладчик на разных языках, но и потенциально запускать их на отдельных системах и даже разных платформах. Сразу погружаясь в работу, рассмотрим следующий код Python для компонента отладчика:

```
from pydbg import *
from pydbg.defines import *
import utils

def av_handler (dbg):
    crash_bin = utils.crash_binning.crash_binning()
    crash_bin.record_crash(dbg)

    # подать сигнал фаззеру.
    print crash_bin.crash_synopsis()
    dbg.terminate_process()

while 1:
    dbg = pydbg()
    dbg.set_callback(EXCEPTION_ACCESS_VIOLATON, av_handler)

    dbg.load(target_program, arguments)

    # подать сигнал фаззеру.
    dbg.run()
```

Возможно, это выглядит не слишком внушительно, но, следуя девизу каркаса PaiMei "коротко и ясно", этот фрагмент Python на самом деле делает очень многое. Возможно, вы узнаете первые две строки из предыдущих глав: они импортируют необходимую функциональность и определения из библиотеки отладчика Windows PyDbg.[5] Однако следующий импорт библиотеки мы еще не встречали. Модуль `PaiMei.utils`[6] содержит различные вспомогательные библиотеки обратного проектирования. В данном случае мы будем использовать модуль `crash_binning`, который объясняется далее. Если немного спуститься, вы найдете бесконечный цикл `while`. Цикл начинается с создания экземпляра класса PyDbg и регистрации функции `av_handler()` в качестве обработчика обратного вызова для обработки событий исключения `ACCESS_VIOLATION`. Всякий раз, когда происходит недопустимое обращение к памяти, как в любом из упомянутых ранее примеров, будет вызвана эта зарегистрированная функция. Затем отладчик загружает целевую программу со всеми необязательными аргументами. В этот момент необходимо подать фаззеру сигнал по обратному каналу, сообщив ему, что наша цель готова принять запрос фаззинга. В последней строке цикла отладчик позволяет отлаживаемой программе выполняться с помощью метода-члена `run()` (псевдонима `debug_event_loop()`).

В оставшихся строках нашей программы находится определение процедуры `av_handler()`. Как и любой зарегистрированный обратный вызов события, наша функция принимает один аргумент - текущий экземпляр объекта отладчика PyDbg. Функция начинается с создания экземпляра упомянутой ранее утилиты PaiMei `crash_binning`. Эта утилита существует для упрощения процесса сбора различных полезных атрибутов состояния текущего потока, который в данном случае вызвал обрабатываемое нами исключение `ACCESS_VIOLATION`. Следующая информация прозрачно записывается в следующей строке во время вызова `record_crash()`:

• *Адрес исключения*. Адрес, по которому возникло исключение; в данном случае адрес инструкции, вызвавшей `ACCESS_VIOLATION`.
• *Нарушение при записи*. Флаг, который в установленном состоянии указывает, что исключение возникло во время попытки записи в ячейку памяти. В противном случае исключение возникло во время попытки чтения из ячейки памяти.

• *Адрес нарушения*. Адрес, по которому была предпринята попытка чтения или записи, приведшая к созданию исключения.
• *Идентификатор потока нарушения*. Числовой идентификатор потока, в котором возникло исключение. Первые четыре атрибута в совокупности позволяют делать такие утверждения: "В потоке 1234 ассемблерная инструкция по адресу `0xDEADBEEF` не смогла прочитать данные по адресу `0xC0CAC01A` и вызвала завершение приложения".
• *Контекст*. Сохраненные значения регистров ЦП в момент возникновения исключения.
• *Дизассемблированное представление*. Дизассемблированное представление инструкции, вызвавшей исключение.
• *Окружающее дизассемблированное представление*. Дизассемблированное представление пяти инструкций до и после инструкции, вызвавшей исключение по указанному адресу исключения. Исследование окружающих инструкций в сочетании с контекстной информацией позволяет аналитикам лучше понять, почему возникло исключение.
• *Раскрутка стека*. Стек вызовов нарушившего потока в момент возникновения исключения. Хотя эта информация не всегда доступна на 32-битных платформах Windows, когда она доступна, то показывает аналитикам, как программа попала в текущее место. Альтернатива исследованию раскрутки стека - упреждающая запись действий цели, как обсуждалось в главе 23 "Отслеживание фаззера".
• *Раскрутка SEH*. Цепочка структурированных обработчиков исключений (Structured Exception Handler, SEH) в момент возникновения исключения. В зависимости от того, обрабатываем ли мы исключения первого шанса (подробнее об этом далее), аналитик может исследовать эту цепочку и увидеть, какие функции были зарегистрированы изнутри самой цели для обработки исключений.

В этот момент отладчику снова следует подать сигнал фаззеру, сообщив ему, что последний тестовый случай вызвал исключение, требующее дальнейшего исследования. Фаззер может записать этот факт, а затем ждать следующего сигнала от отладчика, сообщающего, что цель перезагружена и готова продолжить прием следующих запросов фаззинга. Подав сигнал фаззеру, отладчик печатает удобочитаемую сводку всех метаданных, захваченных при вызове `record_crash()`, и завершает отлаживаемую программу. Цикл `while` позаботится о повторной загрузке цели. Дополнительные сведения о лежащей в основе функциональности PyDbg приведены в главе 20 "Фаззинг в памяти: автоматизация".

Раскрутка стека в 64-битных версиях Windows

Неприятная реальность отладки на различных 32-битных платформах Windows состоит в том, что во многих случаях информация о раскрутке стека теряется. Общий подход к раскрутке стека прост. Текущий кадр начинается по смещению стека EBP. Адрес следующего кадра читается из указателя по EBP. Адрес возврата каждого данного кадра хранится по адресу EBP+4. Диапазон стека можно получить из блока среды потока (thread environment block, TEB), на который ссылается регистр FS:[7] FS[4] для адреса вершины стека и FS[8] для адреса дна стека. Этот процесс нарушается всякий раз, когда в цепочке вызовов существует функция, не использующая построение стекового кадра на основе EBP.

Отказ от использования указателя кадра - распространенная оптимизация компиляторов, поскольку она освобождает регистр EBP для использования в качестве регистра общего назначения. Однако функции с построением кадра на основе EBP могут обращаться к локальным переменным как к смещениям статического регистра EBP:

```
MOV EAX, [EBP-0xC]          ; EBP-based framing
MOV EAX, [ESP+0x44-0xC]     ; frame pointer omitted
```

Если эта оптимизация используется где-либо внутри раскручиваемой цепочки вызовов, информация об истинной цепочке теряется. Microsoft облегчила эту проблему на новых 64-битных платформах, заменив различные соглашения о вызовах единым соглашением о вызовах, полные подробности которого доступны в MSDN.[8] Кроме того, теперь каждая нелистовая функция (листовая функция не вызывает никаких других функций) имеет статическую информацию раскрутки, хранящуюся в файле Portable Executable (PE),[9] что позволяет в любой момент полностью и точно раскрутить стек.

Вот и все. Добавление этого сценария PyDbg в ваш арсенал поставит вас на передний край программного обеспечения отслеживания фаззеров, входящего сегодня в комплект коммерческих фаззеров. Однако мы можем добиться еще большего. При фаззинге программного обеспечения, особенно написанного не слишком хорошо, перед вами встанет пугающая задача каталогизации и сортировки потенциально тысяч тестовых случаев, вызвавших исключение. При дальнейшем анализе вы, вероятно, обнаружите, что многие случаи раскрывают одни и те же лежащие в основе проблемы. Иными словами, фаззер обнаружил несколько векторов к одной и той же уязвимости.

Более совершенное средство отслеживания с отладчиком

Возвращаясь к примеру IMAP из этой главы, снова рассмотрим эту проблему в реальных условиях. Сценарий таков: у нас есть специальный фаззер, способный создать 50 000 уникальных тестовых случаев. Мы запускаем фаззер совместно с недавно созданным сценарием отладчика и обнаруживаем, что 1000 тестовых случаев вызвали исключение. Теперь наша задача как исследователя - вручную тщательно изучить каждый из этих случаев и определить, что пошло не так. Взглянем на первых нескольких нарушителей спокойствия:

```
Test case 00005: x01 LOGIN %s%s%s%s%s%s%s%s%s%s...%s%s%s
EAX=11223300 ECX=FFFF7248...
EIP=0x00112233: REP SCASB

Test case 00017: x01 AUTHENTICATE %s%s%s%s%s%s%s%s%s%s...%s%s%s
EAX=00000000 ECX=FFFFFF70...
EIP=0x00112233: REP SCASB

Test case 00023: x02 SELECT %s%s%s%s%s%s%s%s%s%s...%s%s%s
EAX=47392700 ECX=FFFEEF44...
EIP=0x00112233: REP SCASB
```

Ради краткости в предыдущий фрагмент включен лишь частичный дамп контекста. В каждом из показанных случаев исключение возникло из-за попытки чтения с недопустимого адреса памяти (не показан). Обоснованная догадка, основанная исключительно на входных данных фаззинга, заставляет нас полагать, что лежащая в основе проблема вызвана либо неправильной способностью разбирать длинные строки, либо уязвимостью форматной строки. Заметив, что нарушение произошло по одному и тому же адресу при выполнении инструкции `REP SCASB`, мы подозреваем, что этот шаблон может присутствовать во многих из 1000 случаев. После нескольких беглых проверок наше подозрение подтверждается. Рассмотрим подробнее. Ассемблерная инструкция IA-32 `SCASB` сканирует строку в поисках байта, хранящегося в первом регистре AL (первом байте EAX). В каждом из показанных случаев AL=0. Префикс инструкции `REP` циклически повторяет инструкцию с префиксом и уменьшает регистр ECX, пока ECX не станет равен 0. В каждом из показанных случаев ECX имеет большое значение. Это типичный ассемблерный шаблон, используемый при определении длины строк. По-видимому, многие наши сбои происходят во время сканирования строки в поисках завершающего нулевого байта.

Как досадно. У нас есть 1000 тестовых случаев, которые, как мы знаем, приводят к проблеме, но значительная их часть, по-видимому, возникает в одном и том же месте или рядом с ним. Ручное отсеивание элементов списка - задача не для человека (хотя, возможно, она подойдет стажеру). Есть ли способ лучше? Конечно есть, иначе мы бы с самого начала не повели вас по этому пути.

Как можно предположить по названию, утилита сортировки сбоев PaiMei делает больше, чем просто отображает сведения о текущем контексте. Утилита в основном предназначена для использования как постоянный контейнер, хранящий относящуюся к делу контекстную информацию о многих и многих сбоях. Вызовы метода-члена `record_crash()` создают новую запись во внутренне поддерживаемом и организованном списке. Каждая созданная запись содержит собственные уникальные относящиеся к делу контекстные метаданные. Такой способ организации позволяет преобразовать предыдущее утверждение "1000 из 50 000 тестовых случаев приводят к исключению" в более полезное: "из 50 000 тестовых случаев 650 привели к исключению по адресу `0x00112233`, 300 привели к исключению по адресу `0x11335577`, 20 привели к исключению по адресу `0x22446688`" и так далее. Изменения исходного сценария просты и показаны здесь полужирным в оригинале:

```
from pydbg import *
from pydbg.defines import *
import utils

crash_bin = utils.crash_binning.crash_binning()
```

```
def av_handler (dbg):
    global crash_bin

    crash_bin.record_crash(dbg)

    # подать сигнал фаззеру.
    for ea in crash_bin.bins.keys():
        print "%d recorded crashes at %08x" % \
            (len(crash_bin.bins[ea]), ea)

    print crash_bin.crash_synopsis()
    dbg.terminate_process()

while 1:
    dbg = pydbg()
    dbg.set_callback(EXCEPTION_ACCESS_VIOLATON, av_handler)
    dbg.load(target_program, arguments)

    # подать сигнал фаззеру.
    dbg.run()
```

Чтобы обеспечить постоянство, объявление контейнера сбоев было перемещено в глобальное пространство имен. Также была добавлена логика, согласно которой при каждой записи нового исключения выполняется перебор известных адресов сбоев и отображается число отслеженных исключений по каждому адресу. Это не самый полезный способ отображения этой информации, но здесь он показан как пример. В качестве упражнения попробуйте внести в сценарий необходимые изменения, чтобы для каждого адреса нарушения создавался каталог. В каждом из этих каталогов для каждого зарегистрированного сбоя должны создаваться текстовые файлы, содержащие удобочитаемый вывод `crash_synopsis()`.

Для дальнейшего улучшения автоматизации и каталогизации мы можем расширить утилиту сортировки сбоев, чтобы она обращалась к информации раскрутки стека, доступной для каждого зарегистрированного сбоя. В этом смысле мы можем расширить структуру хранения данных от списка списков до списка деревьев. Это позволяет захватывать и группировать отдельные тестовые случаи как по пути, так и по адресу исключения. Рассмотрим рис. 24.4.

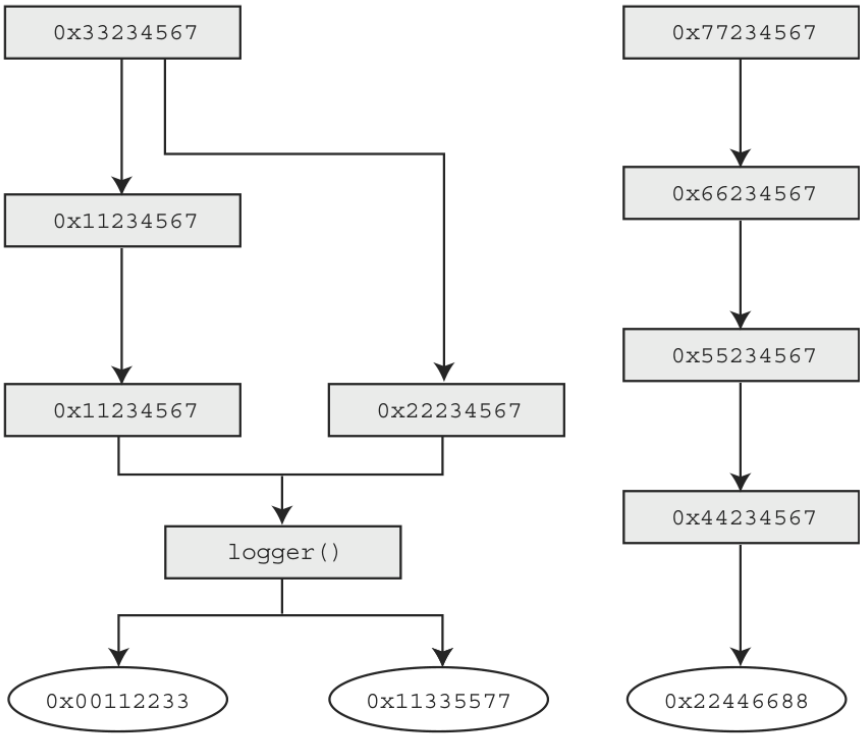


Рис. 24.4. Концепция каталогизации раскрутки стека при сортировке сбоев

Круглые точки данных на рис. 24.4 представляют прежние категории адресов исключений. Точки данных в прямоугольниках представляют адреса из раскрутки стека. Добавив точки данных раскрутки стека в наше представление, мы можем начать видеть пути, ведущие к различным адресам сбоев. По существу, мы способны улучшить наше последнее утверждение "из 50 000 тестовых случаев 650 привели к исключению по адресу `0x00112233`, 300 привели к исключению по адресу `0x11335577`" и так далее до утверждения "из 50 000 тестовых случаев 650 привели к исключению по адресу `0x00112233`, из которых 400 прошли по пути x, y, z, а 250 - по пути a, b, z". Что важнее, анализ такого представления может позволить определить лежащий в основе источник повторяющихся сбоев. В нашей текущей ситуации основная масса захваченных исключений произошла по адресам `0x00112233` и `0x11335577`. К обеим этим точкам ведут сотни путей, но ради краткости показаны лишь несколько. Все пути в этой ветви графа проходят через процедуру `logger()`. Может ли она быть источником основной массы наших проблем? Возможно, внутри этой процедуры существует уязвимость форматной строки. Поскольку эта процедура вызывается из многочисленных мест внутри нашего сервера IMAP, логично, что передача форматной строки нескольким командам IMAP могла привести к одной и той же проблеме. Хотя ответ может дать только более глубокий анализ, преимущества автоматизации анализа до вмешательства человека видны сразу. И снова изменения, необходимые для реализации этой дополнительной функциональности, оставлены читателю в качестве упражнения.

Исключения первого шанса и последнего шанса

Важная концепция, которую мы еще не рассмотрели применительно к обработке исключений, - понятие исключений первого шанса и последнего шанса. Когда внутри целевого процесса, загруженного под нашим отладчиком, возникает исключение, операционная система Microsoft Windows предоставляет отладчику так называемое уведомление первого шанса.[10] Отладчик сам решает, передавать ли исключение отлаживаемой программе. Если исключение передается отлаживаемой программе, возможен один из двух вариантов. Если отлаживаемая программа способна обработать исключение, она сделает это, и выполнение продолжится обычным образом. Если отлаживаемая программа неспособна обработать исключение, операционная система снова предоставляет исключение отладчику, на этот раз как уведомление последнего шанса. Является ли доставленное исключение исключением первого или последнего шанса, отражается значением в структуре `EXCEPTION_DEBUG_INFO`.[11] При работе с PyDbg это значение можно проверить из обработчика обратного вызова следующим образом:

```
def access_violation_handler (dbg):
    if dbg.dbg.u.Exception.dwFirstChance:
        # первый шанс
    else:
        # последний шанс
```

Ненулевое значение `dwFirstChance` означает, что отладчик встретил исключение впервые, а отлаживаемая программа еще не получила возможности увидеть и обработать его.

Что все это означает для наших целей? Это означает, что нам необходимо решить, будем ли мы игнорировать исключения первого шанса. Например, предположим, что мы снова проводим аудит нашего условного сервера IMAP и что за уязвимость форматной строки, раскрытую в процедуре `logger()`, отвечает следующий фрагмент кода:

```
void logger (char *message)
{
    ...
    try
    {
        // уязвимость форматной строки.
        fprintf(log_file, message);
    }
    except
    {
        fprintf(log_file, "Log entry failed!\n");
    }
    ...
}
```

Конструкция `try/except` вокруг уязвимого вызова `fprintf()` создает обработчик исключения. Если первый `fprintf()` завершится неудачей, будет вызван второй `fprintf()`. Если в данном случае мы решим игнорировать исключения первого шанса, то не сможем обнаружить уязвимость форматной строки, поскольку сервер IMAP корректно обрабатывает собственную ошибку. Это не означает, что уязвимость не существует или не поддается эксплуатации! Проще говоря, мы получим ложноотрицательный результат. С другой стороны, мы также будем оказываться в ситуациях, когда большинство обнаруженных исключений и корректно обрабатываются, и не раскрывают никаких проблем безопасности. Если мы решим обрабатывать исключения первого шанса, нам придется иметь дело с рядом ложноположительных результатов. Как независимый исследователь безопасности, вы не можете принять универсально правильное решение. Осознав этот вопрос, можно применить эмпирическое тестирование к каждой конкретной цели, чтобы определить, на какую сторону границы между ложноположительными и ложноотрицательными результатами вы хотели бы встать. Как разработчик программного обеспечения или инженер QA, проводящий тестирование для защиты собственного продукта, вы примете безопасное решение: отслеживать исключения первого шанса и уделять время изучению источника исключения в каждом

случае.

Динамическое инструментирование двоичных файлов

Отслеживание с помощью отладчика может дать очень многое. Однако, как упоминалось в главе 5 "Требования к эффективному фаззингу", панацея обнаружения ошибок заключается в DBI. Обсуждение этой темы с глубиной, необходимой для разработки прототипов инструментов, выходит за рамки этой книги. Однако информация, представленная в этом разделе, поможет вам яснее понять, как некоторые из доступных сегодня передовых инструментов отладки работают внутри. Движки DBI способны инструментировать целевой исполняемый файл на уровне наименьшего логического фрагмента - базового блока.

Вспомним из главы 23: базовый блок определяется как последовательность инструкций, в которой после достижения первой инструкции базового блока гарантируется выполнение по порядку каждой инструкции внутри блока. Системы DBI позволяют инструментировать отдельные инструкции внутри каждого базового блока посредством добавления, изменения или иного преобразования логики инструкций. Обычно это становится возможным благодаря перенаправлению потока управления выполнением от исходного набора инструкций в измененный кэш кода. В некоторых системах инструментирование на уровне инструкций может выполняться на более высокоуровневом RISC-подобном языке псевдоассемблера. Это позволяет разработчикам легче создавать инструменты DBI, а также пытаться обеспечить их работу на разных архитектурах. API, предоставляемый системой DBI, допускает самые разнообразные варианты использования: от профилирования и оптимизации до машинного перевода и отслеживания безопасности.

Сегодня существует ряд систем DBI, среди которых DynamoRIO,[12] DynInst[13] и Pin.[14] Например, система DynamoRIO была разработана как совместный проект Массачусетского технологического института и Hewlett-Packard. DynamoRIO реализована для архитектуры IA-32 и поддерживает как Microsoft Windows, так и Linux. DynamoRIO стабильна и обладает высокой производительностью, что подтверждается ее использованием в коммерческих программных решениях, таких как Memory Firewall[15] компании Determina.[16]

Дополнительные сведения об использовании DynamiRIO компанией Determina можно найти в исследовательской работе MIT под названием "Secure Execution Via Program Shepherding".[17] DBI Pin интересна тем, что, в отличие от большинства систем DBI, позволяет разрабатывать инструменты, способные подключаться к целевому процессу, а не ограничиваться загрузкой целевого процесса под управлением DBI.

Использование DBI позволяет разрабатывать инструменты отслеживания, чтобы дополнительно улучшать возможности обнаружения сбоев: от обнаружения возникновения сбоя до потенциального обнаружения его источника. Вернемся к использованному ранее примеру переполнения кучи:

```
char *A = malloc(8);
char *B = malloc(16);
char *C = malloc(24);

strcpy(A, "PPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP");
free(B);
```

При отслеживании на основе отладчика мы не могли обнаружить само переполнение в момент его возникновения во время вызова `strcpy()`. То же верно для показанных ранее переполнений стека. В случае переполнения стека исключение может быть создано и обнаружено при возврате из затронутой функции. В случае этого и других переполнений кучи исключение не будет создано до последующей операции с кучей, такой как вызов `free()`, показанный в предыдущем фрагменте кода. На основе DBI можно отслеживать и ограждать все выделения памяти по мере их возникновения во время выполнения. Термин "ограждать" означает возможность отметить или

иным образом запомнить начало и конец каждого фрагмента кучи. Посредством инструментирования всех инструкций записи в память можно добавить проверку, гарантирующую, что границы каждого отдельного фрагмента кучи не превышаются. Как только даже один байт выйдет за конец выделенного фрагмента, можно отправить уведомление. Успешная реализация такой технологии может сэкономить вам часы времени, анализа и, в конечном счете, деньги.

Кривая обучения, связанная с разработкой специальных инструментов DBI, отнюдь не полога. К счастью, другие до вас уже потрудились над созданием инструментов, которые можно расширить для интеграции с фаззером. В коммерческой сфере вы найдете такие продукты, как IBM Rational Purify,[18] Compuware DevPartner BoundsChecker,[19] OC Systems RootCause[20] и Parasoft Insure++.[21] Например, Purify фактически построен на Static Binary Instrumentation (SBI), а BoundsChecker построен на сочетании SBI и DBI. В любом случае эти продукты могут предоставить набор функций обнаружения ошибок и анализа производительности, которых можно было бы ожидать. В мире открытого исходного кода самое заметное решение, которое вам может встретиться, - Valgrind[22] (произносится "val" как в "value" и "grind" как в "grinned"). Valgrind предоставляет как систему DBI, так и ряд заранее разработанных инструментов, таких как Memcheck, который можно использовать для обнаружения утечек памяти и уязвимостей переполнения кучи. Для Valgrind также доступен ряд дополнений сторонних разработчиков. Для наших конкретных целей наибольшее значение имеет версия проверки границ Annelid.[23]

Для большинства использование автоматизации с помощью отладчика уже станет шагом вперед по сравнению с нынешней методологией фаззинга. Однако мы призываем вас исследовать некоторые из этих более совершенных путей обнаружения ошибок.

Резюме

В этой главе мы рассмотрели отслеживание цели фаззера: от самых базовых высокоуровневых методов до более совершенных методов с помощью отладчика. Мы также коснулись понятия DBI, представив различные системы DBI, а также несколько готовых решений, которые можно исследовать для более совершенного отслеживания цели.

С помощью знаний, полученных из этой главы, вы должны суметь объединить возможности специального инструмента отслеживания с отладчиком и высокоуровневого средства пошагового перебора фаззера. Это сочетание можно использовать универсальным образом, чтобы точно определять, какие отдельные тестовые случаи или, возможно, группы тестовых случаев заставили цель сбиться с пути. Благодаря дополнительным преимуществам автоматизации вы сможете переключить труд ваших стажеров с анализа после работы фаззера обратно на их традиционную задачу - приготовление для вас свежего кофе.

Сноски

[1] [назад] http://en.wikipedia.org/wiki/NX_bit
[2] [назад] http://doc.bughunter.net/buffer-overflow/free.html
[3] [назад] http://www.cs.drexel.edu/~spiros/research/papers/WCRE03a.pdf
[4] [назад] https://www.openrce.org/downloads/details/208/PaiMei
[5] [назад] http://pedram.redhive.com/PaiMei/docs/PyDbg/
[6] [назад] http://pedram.redhive.com/PaiMei/docs/Utilities/
[7] [назад] http://openrce.org/reference_library/files/reference/Windows%20Memory%20Layout,%20User-Kernel%20Address%20Spaces.pdf
[8] [назад] http://msdn2.microsoft.com/en-us/library/7kcdt6fy.aspx
[9] [назад] http://www.uninformed.org/?v=4&a=1&t=sumry
[10] [назад] http://msdn.microsoft.com/msdnmag/issues/01/09/hood/
[11] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/wcecoreos5/html/wce50lrfexceptiondebug-info.asp
[12] [назад] http://www.cag.lcs.mit.edu/dynamorio/

[13] [назад] http://www.dyninst.org/
[14] [назад] http://rogue.colorado.edu/pin/
[15] [назад] http://www.determina.com/products/memory_firewall.asp
[16] [назад] http://www.determina.com
[17] [назад] http://www.determina.com/products/memory_firewall.asp
[18] [назад] http://www-306.ibm.com/software/awdtools/purify/
[19] [назад] http://www.compuware.com/products/devpartner/visualc.htm
[20] [назад] http://www.ocsystems.com/prod_rootcause.html
[21] [назад] http://www.parasoft.com/jsp/products/home.jsp?product=Insure
[22] [назад] http://valgrind.org/
[23] [назад] http://valgrind.org/downloads/variants.html?njn

Часть IV. Взгляд вперед

Глава 25. Извлеченные уроки

> "Редко задается вопрос: учится ли наши дети?"
>
> - Джордж У. Буш, Флоренс, штат Южная Каролина, 11 января 2000 года

Мы надеемся, что к этому моменту смогли нарисовать ясную картину того, что такое фаззинг, почему он эффективен и как его можно реализовать для выявления скрытых сбоев в программном коде. В самом начале мы упомянули, что эта книга ориентирована на три разные целевые аудитории, которые могут воспользоваться мощью фаззинга: разработчиков, членов групп QA и исследователей безопасности. В этой главе мы разберем жизненный цикл разработки программного обеспечения (software development lifecycle, SDLC), чтобы определить, где каждая из этих групп может применять фаззинг для создания безопасного программного обеспечения.

Жизненный цикл разработки программного обеспечения

Когда-то фаззинг был методом, который почти исключительно использовали исследователи безопасности после того, как продукт становился "золотым" и проходил производственный цикл, но теперь разработчики программного обеспечения разумно учатся применять фаззинг, чтобы выявлять уязвимости на более ранних этапах SDLC. Microsoft приняла фаззинг в качестве ключевого компонента своего жизненного цикла разработки безопасного программного обеспечения в рамках Trustworthy Computing.[1] Эта методология открыто призывает разработчиков "применять инструменты тестирования безопасности, включая инструменты фаззинга", на этапе реализации того, что они называют жизненным циклом разработки безопасного программного обеспечения (Security Development Lifecycle, SDL). Microsoft разработала концепцию SDL, которая отражает этапы собственного SDLC; оба цикла показаны на рис. 25.1.

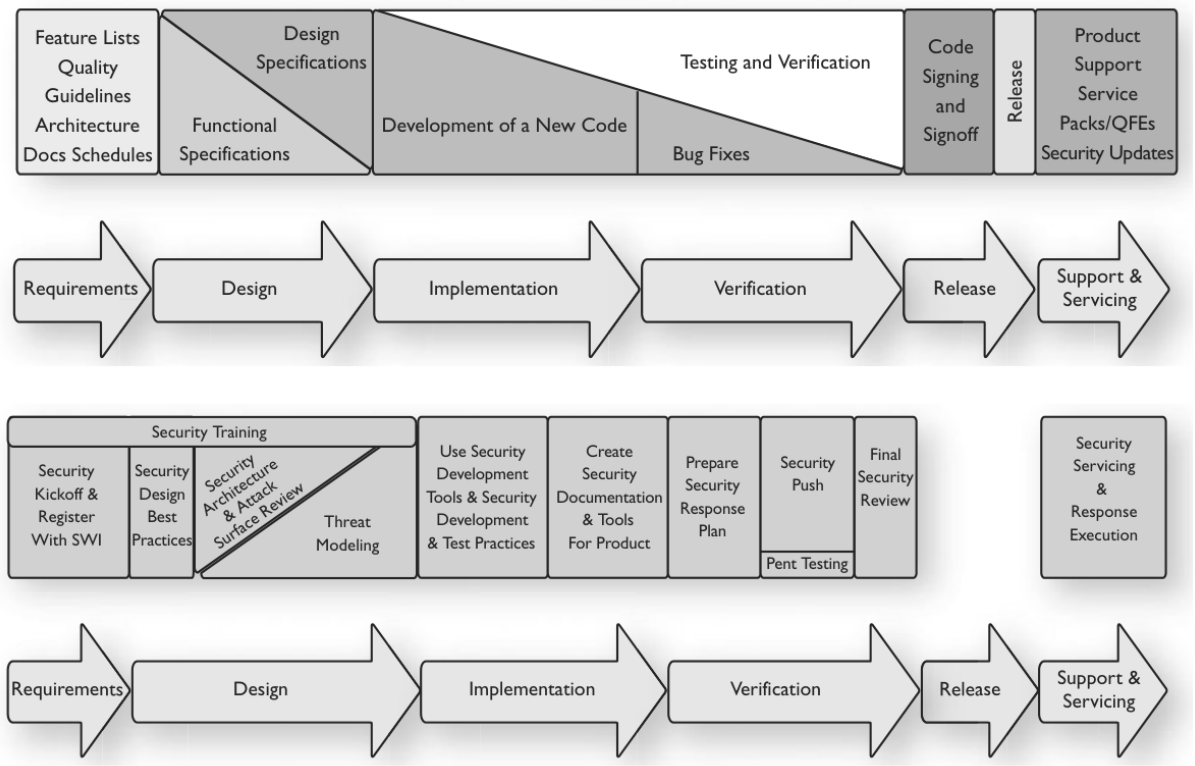


Рис. 25.1. SDLC Microsoft и SDL Microsoft

В этих параллельных процессах видно, что Microsoft дошла до определения надлежащего участия специалистов по безопасности на каждом этапе своего SDLC. Это важное признание неотъемлемой роли, которую безопасность должна играть на протяжении всего SDLC. Также необходимо реализовать то, что нельзя отразить на этих схемах: безопасность должна быть вплетена в саму ткань SDLC, а не существовать только как параллельный процесс. Разработчики, группы QA и исследователи безопасности должны работать сообща и координировать свои усилия ради общей цели - безопасного кода.

Существует множество методологий SDLC, но для наших целей мы используем исходную каскадную модель Уинстона Ройса[2] из-за ее простоты и широкого распространения. Каскадная модель применяет последовательный подход к разработке программного обеспечения, при котором разработка на протяжении жизненного цикла проходит пять отдельных этапов. Этот подход показан на рис. 25.2.

Внимание Microsoft к безопасности

Неудивительно, что Microsoft тесно вплетает безопасность в свой SDLC. Обладая доминирующей долей рынка, технологии Microsoft активно становятся целями исследователей безопасности. Хотя многие утверждают, что Microsoft предстоит долгий путь в области безопасности, нельзя отрицать, что компания уже добилась значительных успехов в повышении безопасности своего программного обеспечения.

В качестве примера рассмотрим Microsoft Internet Information Services (IIS),[3] веб-сервер Microsoft. В настоящее время публично раскрыты 14 уязвимостей, затрагивающих ветвь IIS 5.x.[4] Ветвь 6.x, представленная в начале 2003 года, затронута только тремя известными сбоями,[5] ни один из которых не имеет критической оценки.

Улучшения безопасности можно частично объяснить низкоуровневыми усовершенствованиями безопасности, такими как проверка безопасности буфера `/GS`,[6] предотвращение выполнения данных (Data Execution Prevention, DEP)[7] и безопасная

структурированная обработка исключений (Safe Structured Exception Handling, SafeSEH), а также одним из самых ожидаемых усовершенствований безопасности Windows Vista - рандомизацией размещения адресного пространства (Address Space Layout Randomization, ASLR).[8] В дополнение к этим усовершенствованиям безопасности Microsoft выделила кадровые ресурсы на безопасность посредством таких программ, как Secure Windows Initiative.[9] Фаззинг - один из многих методов, используемых сотрудниками этого подразделения для повышения безопасности программного обеспечения, которое вы видите развернутым сегодня.

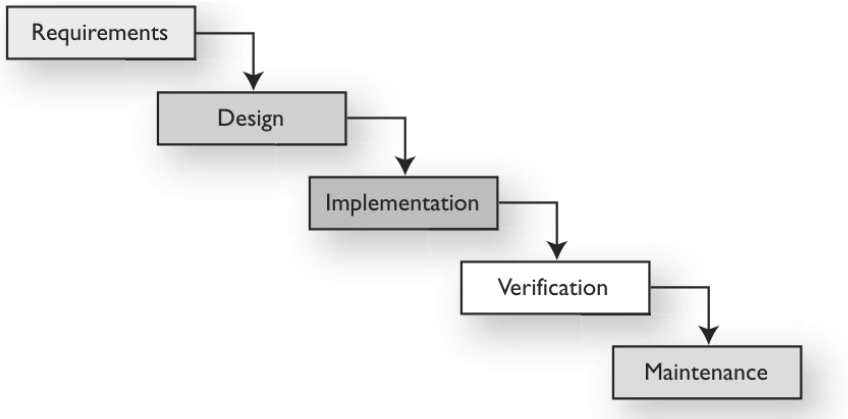


Рис. 25.2. Исходная каскадная модель Ройса

Далее мы рассмотрим, как фаззинг можно реализовать на каждом из этих этапов.

Анализ

Этап анализа включает весь сбор информации, необходимый до начала проекта. Этот этап включает работу с конечными пользователями, чтобы точно определить, что они ищут. Хотя фаззинг не играл бы непосредственной роли на этом этапе SDLC, важно, чтобы разработчики, группы QA и специалисты по безопасности начали думать о том, будет ли фаззинг подходящим инструментом тестирования для последующих этапов.

Проектирование

Проектирование приложения предполагает создание абстрактного представления проекта. Оно может включать различные методы моделирования в зависимости от моделируемой части общего решения, например аппаратных, программных компонентов или компонентов базы данных. Именно на этом этапе принимаются решения о языках программирования и программных библиотеках, которые будут использоваться на этапе написания кода.

По мере принятия проектных решений начинают проясняться осуществимость фаззинга и возможные подходы к нему. Именно на этом этапе определяются две важные детали, которые повлияют на общий подход. Во-первых, принимаются решения о реализуемых программных и аппаратных платформах. Это повлияет на инструменты фаззинга, которые можно использовать, и применимые классы фаззинга. Например, если проект представляет собой консольное приложение, предназначенное для использования в операционной системе Linux, подходит фаззинг переменных среды. Если проект предназначен для использования в операционной системе Windows, будут ли в нем элементы управления ActiveX, отмеченные как "безопасные для

сценариев" и предоставляющие функции внешним веб-сайтам? Если да, следует применить фаззинг объектов COM.

Сетевые протоколы для межпроцессного взаимодействия, вероятно, будут выбраны на этом этапе SDLC. Например, если требования конкретного проекта предусматривают встроенное решение для обмена мгновенными сообщениями, можно выбрать Extensible Messaging and Presence Protocol (XMPP). Этот протокол представляет собой открытую технологию обмена данными на основе XML, разработанную в 1999 году сообществом открытого исходного кода, стоящим за проектом Jabber.[10] Это подходящий этап SDLC, чтобы передать спецификацию протокола группе фаззинга. Также сейчас самое время выделить ресурсы на поиск и перечисление доступных в настоящее время инструментов тестирования XMPP и ранее обнаруженных уязвимостей разбора XMPP.

Другое важное решение, принимаемое на этапе проектирования, связано с векторами входных данных, которые будут разработаны в приложении. Помните, что фаззинг - это просто метод передачи приложению некорректно сформированных входных данных с наблюдением за тем, что произойдет. Поэтому крайне важно выявить все такие векторы входных данных и определить, как лучше всего подвергнуть их фаззингу. Также важно творчески подходить к определению того, что именно представляет собой вектор входных данных. Если что-то происходит в фоновом режиме и не предоставляется непосредственно конечному пользователю, это еще не означает, что злоумышленник не сможет создать специальный код для доступа к интерфейсу. Чем полнее список векторов входных данных, тем лучше общее покрытие кода, которого можно достичь с помощью фаззинга.

Написание кода

На этапе написания кода группы разработки работают над созданием необходимых компонентов общего проекта и постепенно объединяют их в единое приложение. Здесь важно помнить, что фаззинг может и должен применяться разработчиками во время разработки. Бремя тестирования сбоев не должны нести исключительно группы QA и специалисты по безопасности.

По мере компиляции и проверки правильной функциональности участков кода их также следует проверять на наличие надлежащих средств контроля безопасности. Фаззинг можно применять к любым возможным векторам входных данных, выявленным на этапе проектирования. Если предоставляемый элемент управления ActiveX готов, начинайте его фаззинг. Если сетевая служба завершена, начинайте ее фаззинг. Как мы видели ранее в главе 19 "Фаззинг в памяти" и главе 20 "Фаззинг в памяти: автоматизация", фаззинг в памяти также можно применять на уровне функций. Если данная функция предназначена для извлечения адреса электронной почты из предоставленной пользователем строки, попробуйте вызвать эту функцию тысячи раз с рядом случайных и умело выбранных строковых значений.

Плохая проверка входных данных часто является слабостью, приводящей к уязвимостям безопасности, а фаззинг поможет разработчикам лучше понять, почему проверка входных данных настолько важна.

Фаззинг предназначен для выявления входных данных, которые разработчик не учел и которые приложение неспособно обработать. Когда разработчики увидят это в действии, они окажутся в наилучшем положении для создания надлежащих средств проверки входных данных и обеспечения того, чтобы те же ошибки не повторялись в последующем коде. Что важнее, чем раньше в SDLC обнаруживаются уязвимости, тем ниже общая стоимость их исправления. Когда разработчик находит собственные ошибки, исправить их гораздо проще.

Тестирование

Когда программное обеспечение переходит на этап тестирования, к работе приступают группы QA. Группы безопасности также должны участвовать в этапе тестирования до того, как проект перейдет в эксплуатацию. Общие затраты всегда будут ниже, если дефекты обнаружены до выпуска программного обеспечения. Кроме того, можно уберечь компанию от ущерба корпоративной репутации, обеспечив внутреннее обнаружение уязвимостей, а не их первое появление в публичной рассылке.

Важно, чтобы группы QA и исследователи безопасности определили, как лучше всего работать совместно на этапе тестирования, чтобы использовать ресурсы друг друга. Привлечение специалистов по безопасности к SDLC может быть новым подходом для некоторых поставщиков программного обеспечения, но рост числа публично раскрытых уязвимостей продемонстрировал, насколько существенно их участие.

И группы QA, и исследователи безопасности должны знать о коммерческих инструментах фаззинга и инструментах с открытым исходным кодом, которые можно применять во время тестирования. Это область, где можно проводить учебные занятия между коллегами. Исследователи безопасности с большей вероятностью обладают актуальными знаниями о новейших и лучших доступных инструментах фаззинга, и эти знания можно использовать для обучения групп QA и разработчиков подходящим инструментам, которые можно применять для повышения безопасности. В качестве дополнения к этому обучению разработчики узнают о распространенных ошибках программирования, которых они могут избегать, и могут получить более глубокое понимание практик безопасного программирования.

Сопровождение

Несмотря на все усилия разработчиков, групп QA и исследователей безопасности, сбои, приводящие как к простым неудобствам, так и к критическим уязвимостям безопасности, попадут в эксплуатационный код. Крупные поставщики программного обеспечения вложили в безопасность миллиарды долларов, и, хотя ситуация улучшается, уязвимый код - реальность, с которой нам всегда придется иметь дело. Поэтому важно сохранять упреждающий подход к исследованию уязвимостей безопасности даже после перехода кода в эксплуатацию. Новые инструменты фаззинга, знающие исследователи и улучшенные методы фаззинга могут дать свежий взгляд на ранее проверенный код и выявить новые уязвимости. Более того, уязвимости могут зависеть от реализации и потому проявляться после выпуска кода в эксплуатацию из-за изменения конфигурации. Например, некоторые проблемы могут не проявляться на 32-битной платформе, но проявятся в 64-битной системе.

Помимо внутренних усилий по упреждающему выявлению уязвимого кода, важно иметь процессы для реагирования на сообщения независимых исследователей и работы с ними. Независимые исследователи отвечают за обнаружение большинства публичных уязвимостей безопасности, найденных в программах. Хотя мотивы исследователей при выявлении уязвимостей различаются, их усилия тем не менее важны для дальнейшего повышения безопасности эксплуатационного кода. Поставщики программного обеспечения должны позаботиться о наличии процессов, позволяющих независимым исследователям сообщать о своих находках группам, которые отвечают за связь между исследователями и разработчиками, в конечном счете исправляющими ошибки.

Реализация фаззинга в SDLC

Мы признаем, что каскадная модель предоставляет чрезмерно упрощенный обзор процессов, задействованных во всех проектах разработки, кроме самых простых. Большинство проектов включает несколько групп, работающих совместно, и этапы SDLC, которые происходят в повторяющихся циклах, а не в виде аккуратных последовательных этапов каскадной модели. Тем не менее любая модель разработки включает те же основные этапы в некотором сочетании. Старайтесь не сосредотачиваться на самой модели. Вместо этого используйте материал, обсуждавшийся в этой главе, чтобы внедрить фаззинг в собственный SDLC и лучше защитить код до того, как он попадет в эксплуатацию.

Разработчики

Разработчики - первая линия обороны при реализации безопасности в новых проектах, и им следует предоставить возможность подвергать фаззингу недавно разработанные функции, классы и библиотеки по мере их разработки. Предоставив разработчикам навыки распознавания основных проблем безопасности, можно немедленно устранить многие дорогостоящие проблемы безопасности, и они никогда не дойдут до групп QA или безопасности, не говоря уже о широкой публике.

Обычно разработчики выполняют большую часть программирования в конкретной интегрированной среде разработки (integrated development environment, IDE), в которой им удобно. Например, Microsoft Visual Studio - стандартная IDE для написания кода на C# и Visual Basic на платформе Windows, тогда как Eclipse - популярный выбор для разработки на Java и различных других языках. Один из рекомендуемых подходов, побуждающих разработчиков включать фаззинг в свои методы разработки, - создавать подключаемые модули IDE, а не требовать от разработчиков изучения совершенно нового набора инструментов тестирования безопасности.

Хотя DevInspect[11] компании SPI Dynamics не является непосредственно инструментом фаззинга, он служит примером такого подхода. DevInpect - инструмент, расширяющий функциональность Visual Studio и Eclipse, чтобы предоставлять разработчикам анализ исходного кода и тестирование методом черного ящика веб-приложений ASP.Net и Java.

Исследователи QA

Исторически QA была сосредоточена на тестировании функциональности, а не безопасности. К счастью, это меняется по мере того, как конечные пользователи все больше осознают опасности небезопасного программного обеспечения и требуют большего от его поставщиков. Хотя от групп QA не следует ожидать, что они станут экспертами по безопасности, от них следует ожидать базового понимания безопасности и, что важнее, знания того, когда необходимо привлечь эксперта по безопасности.

Фаззинг - процесс тестирования, идеально подходящий группам QA, поскольку его можно в высокой степени автоматизировать. Например, обратное проектирование не было бы подходящим навыком, которого можно требовать от исследователей QA, поскольку это узкоспециализированный навык, требующий серьезного обучения. С другой стороны, запуск инструментов фаззинга - вполне реалистичное ожидание от групп QA. Хотя для определения того, приводит ли конкретное исключение к эксплуатируемому состоянию, может потребоваться координация с группой безопасности, на этапе тестирования сбой любого рода заслуживает рассмотрения и, вероятно, должен быть устранен разработчиками. В этот момент различие между безопасностью и

функциональностью не так существенно. Существенно исправить имеющуюся проблему.

Исследователи безопасности

Для исследователей безопасности фаззинг не нов. Фаззинг - широко используемый подход к обнаружению уязвимостей безопасности благодаря его относительной простоте и высокой отдаче. В недавнем прошлом исследователи безопасности вообще не участвовали в SDLC. Во многих организациях это по-прежнему так. Если в вашем SDLC не задействованы специалисты по безопасности, пора пересмотреть процессы разработки программного обеспечения. Например, Microsoft проводит семинары BlueHat Security Briefings,[12] стремясь передать знания от известных исследователей безопасности своим сотрудникам.

Задача исследователей безопасности - сместить внимание на выявление уязвимостей на более ранних этапах SDLC. В исправлении дыры всегда есть ценность, независимо от того, когда она обнаружена, но, как отмечено на рис. 25.3, стоимость реализации исправлений возрастает по мере продвижения проекта разработки. Группы безопасности обладают специализированными навыками, которые можно использовать для адаптации процесса тестирования, чтобы постоянно улучшать применяемые инструменты и методы фаззинга. Хотя группы QA должны отвечать за запуск стандартных инструментов и сценариев фаззинга, именно исследователи безопасности должны создавать специальные инструменты или выявлять новые технологии и постоянно обучать группы QA и разработчиков их использованию.

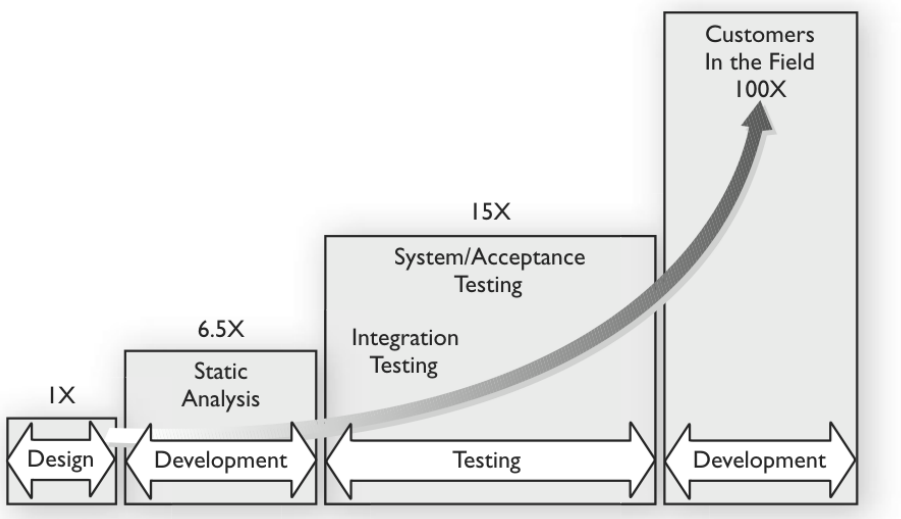


Рис. 25.3. Стоимость исправления дефектов программного обеспечения на протяжении SDLC

Резюме

Прошли те времена, когда кто-либо, участвующий в разработке программного обеспечения, мог сказать: "Безопасность - не моя проблема; у нас есть группа, которая этим занимается". Безопасность - ответственность каждого, включая разработчиков, группу QA и исследователей безопасности. Менеджеру продукта недостаточно просто игнорировать уязвимости и надеяться, что они волшебным образом исчезнут. Вашим людям также необходимо предоставить процессы и инструменты для встраивания безопасности в SDLC, а фаззинг представляет собой автоматизируемый процесс, которым могут воспользоваться все три группы. Фаззинг - не волшебная пуля, но это методология, которая хорошо подходит для разработки удобных в использовании инструментов безопасности, способных выявлять широкий спектр уязвимостей.

Сноски

[1] [назад] http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnsecure/html/sdl.asp
[2] [назад] http://en.wikipedia.org/wiki/Waterfall_process
[3] [назад] http://www.microsoft.com/WindowsServer2003/iis/default.mspx
[4] [назад] http://secunia.com/product/39/?task=advisories
[5] [назад] http://secunia.com/product/1438/?task=advisories
[6] [назад] http://msdn2.microsoft.com/en-US/library/8dbf701c.aspx
[7] [назад] http://en.wikipedia.org/wiki/Data_Execution_Prevention
[8] [назад] http://www.symantec.com/avcenter/reference/Address_Space_Layout_Randomization.pdf
[9] [назад] http://www.microsoft.com/technet/Security/bestprac/secwinin.mspx
[10] [назад] http://www.jabber.org/
[11] [назад] http://www.spidynamics.com/products/devinspect/
[12] [назад] http://www.microsoft.com/technet/security/bluehat/sessions/default.mspx

Глава 26. Взгляд вперед

> "Но в целом для нас с Лорой это был потрясающий год".
>
> - Джордж У. Буш, подводя итоги своего первого года на посту через три месяца после терактов 11 сентября, Вашингтон, округ Колумбия, 20 декабря 2001 года

Куда фаззинг пойдет дальше? Эта область уже начала переходить из академического и подпольного миров и встраиваться в корпоративные испытательные стенды и на всем протяжении SDLC. Учитывая растущее признание фаззинга среди разработчиков программного обеспечения, неудивительно видеть появление коммерческих инструментов, использующих преимущества этой мощной методологии. В этой главе мы рассмотрим, где фаззинг находится сегодня, и заглянем в хрустальный шар, чтобы попытаться определить, где он окажется в ближайшем будущем.

Коммерческие инструменты

Пожалуй, убедительный показатель созревания отрасли - появление в ней коммерческих игроков. Именно эту тенденцию мы сейчас наблюдаем в технологиях фаззинга. Поскольку крупные разработчики программного обеспечения, такие как Microsoft, приняли фаззинг в качестве средства выявления уязвимостей безопасности на более ранних этапах SDLC, открылись двери для появления новых продуктов и компаний, удовлетворяющих потребность в надежных и удобных фаззерах. В этом разделе мы рассмотрим некоторые появившиеся коммерческие организации, перечисленные в алфавитном порядке.

Beyond Security beSTORM[1]

beSTORM - инструмент тестирования методом черного ящика, использующий фаззинг для выявления уязвимостей в различных сетевых протоколах. Основатели Beyond Security первоначально участвовали в создании публичного веб-портала SecuriTeam,[2] распространяющего новости и ресурсы по безопасности. beSTORM охватывает следующие протоколы:

• HTTP - Hypertext Transfer Protocol.
• Расширения FrontPage.
• DNS - Domain Name System.

• FTP - File Transfer Protocol.
• TFTP - Trivial File Transfer Protocol.
• POP3 - Post Office Protocol v3.
• SIP - Session Initiation Protocol.
• SMB - Server Message Block.
• SMTP - Simple Mail Transfer Protocol.
• SSLv3 - Secure Sockets Layer v3.
• STUN - (Simple Traversal of User Datagram Protocol (UDP) Through Network Address Translators (NATs)).
• DHCP - Dynamics Host Configuration Protocol.

beSTORM работает на платформах Windows, UNIX и Linux и имеет отдельные компоненты тестирования и отслеживания.[3] Включение компонента отслеживания цели - функция, которую следует ожидать все чаще в следующем поколении коммерческих технологий фаззинга. В случае beSTORM компонент отслеживания построен как специальный отладчик и в настоящее время ограничен платформами UNIX и Linux. Снимок экрана beSTORM показан на рис. 26.1.

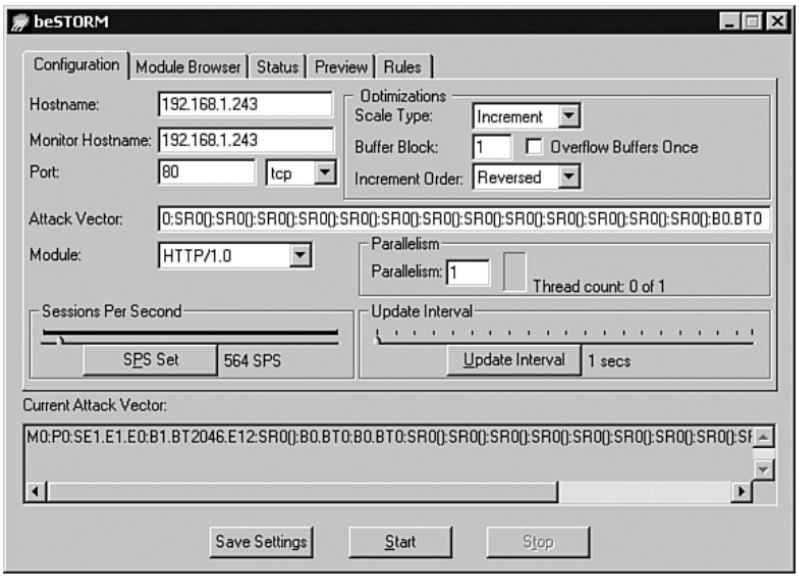


Рис. 26.1. Beyond Security beSTORM

BreakingPoint Systems BPS-1000[4]

Будучи самым новым игроком на поле, BreakingPoint заняла позицию на рынке с явным превосходством в чистой мощности создания трафика. BPS-1000 - специальное аппаратное решение, способное создавать более 5 миллионов сеансов TCP со скоростью 500 000 сеансов TCP в секунду. Хотя BPS-1000 не является фаззером в традиционном смысле, он способен мутировать создаваемый трафик для вызывания сбоев и обхода обнаружения.



Рис. 26.2. BreakingPoint Systems BPS-1000

Одна из наиболее интересных функций BPS-1000 - расположенный спереди адаптер переменного тока, предназначенный для отключения и включения питания целевых тестовых устройств при их сбое, чтобы продолжить тестирование.

Codenomicon[5]

Codenomicon предлагает, пожалуй, самые широко известные решения для фаззинга программного обеспечения. Корни основателей Codenomicon связаны с набором тестов PROTOS[6] - проектом, который первоначально спонсировал Университет Оулу в Линнанмаа, Финляндия. PROTOS впервые получил широкую известность еще в 2002 году, когда проект объявил об ошеломляющем списке уязвимостей в различных реализациях SNMPv1. Разработчики PROTOS разобрали протокол SNMPv1 и перечислили все возможные перестановки как пакетов запросов, так и пакетов ловушек. Затем они разработали значительную коллекцию транзакций, вводивших исключительные элементы - термин, который они определили как "входные данные, которые могли не быть должным образом учтены при реализации программного обеспечения".[7] В некоторых случаях эти исключительные элементы нарушали стандарт протокола, тогда как в других случаях они соответствовали спецификации, но включали содержимое, предназначенное для нарушения работы плохо написанных анализаторов. Результаты набора тестов PROTOS SNMPv1 привлекли значительное внимание, поскольку выявили многочисленные уязвимости, результатом которых стал выпуск исправлений десятками затронутых поставщиков аппаратного и программного обеспечения.[8] С тех пор проект PROTOS развился и сегодня состоит из наборов тестов для разнообразных сетевых протоколов и форматов файлов, включая следующие:

• WAP - Wireless Application Protocol.
• HTTP - Hypertext Transfer Protocol.
• LDAPv3 - Lightweight Directory Access Protocol v3.
• SNMPv1 - Simple Network Management Protocol v1.
• SIP - Session Initiation Protocol.
• H.323 - набор протоколов, широко используемых в видеоконференциях.
• ISAKMP - Internet Security Association and Key Management Protocol.
• DNS - Domain Name System.

Методология тестирования, лежащая в основе коммерческого продукта, не сильно отклонилась от своего начала в PROTOS. Самое заметное отличие состоит в том, что коммерческие наборы тестов охватывают большее число сетевых протоколов и форматов файлов и, кроме того, предлагают удобный GUI, показанный на рис. 26.3.

Codenomicon предлагает отдельный продукт для каждого отдельного протокола по высокой цене. Указанная розничная цена на момент написания составляла примерно 30 000 долларов США за протокол. Эта модель ценообразования предполагает, что компания ориентируется на

разработчиков программного обеспечения, которым для выпускаемых ими типов продуктов потребовалось бы лишь несколько наборов протоколов, а не на организацию, занимающуюся исключительно исследованиями безопасности и заинтересованную в тестировании более широкого спектра продуктов и протоколов. Самое большое ограничение продукта - отсутствие возможностей отслеживания цели. Codenomicon использует заведомо хорошие тестовые случаи для опроса состояния цели во время тестирования. Как обсуждалось ранее в главе 24 "Интеллектуальное обнаружение сбоев", это один из самых примитивных методов отслеживания.

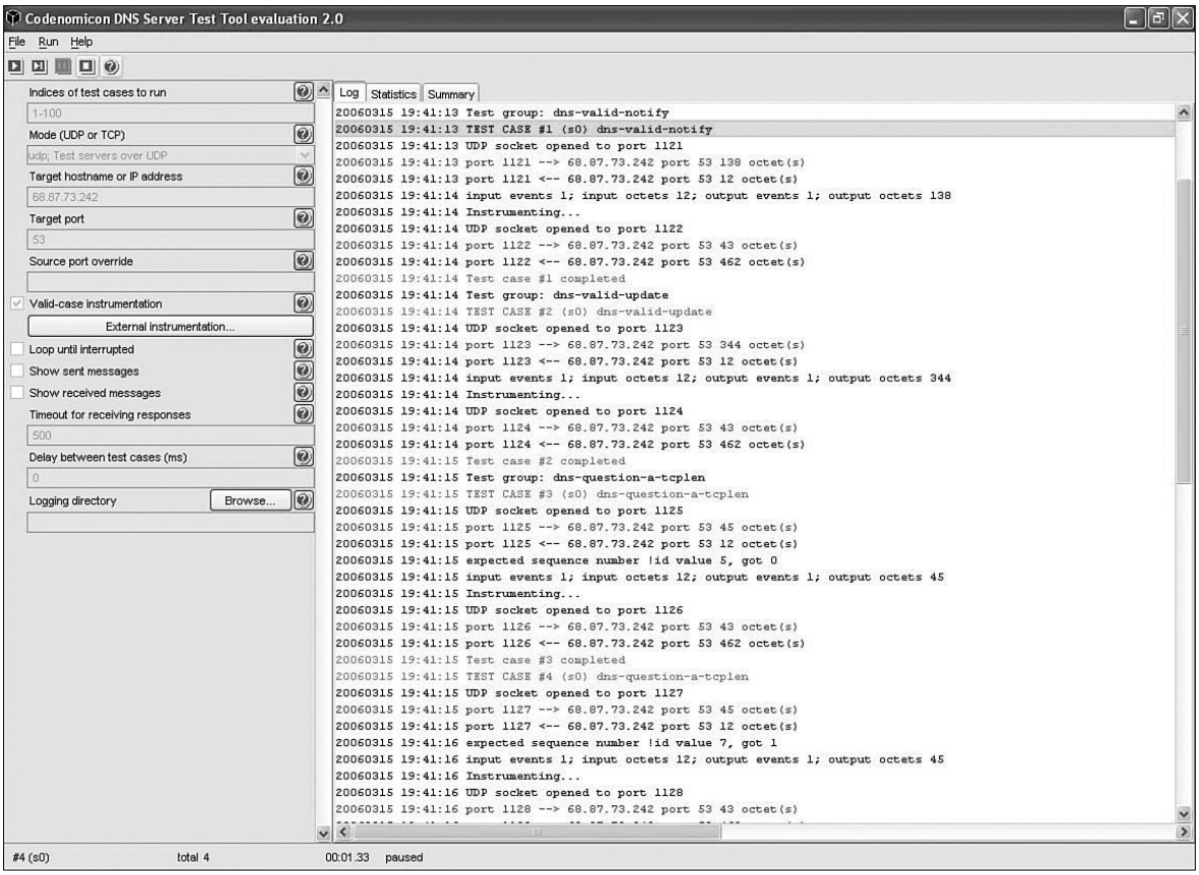


Рис. 26.3. Инструмент тестирования DNS-сервера Codenomicon

GLEG ProtoVer Professional[9]

GLEG - российская компания, которая была основана с предложением VulnDisco,[10] службы, состоящей из ежемесячных выпусков недавно обнаруженных уязвимостей в виде модулей эксплойтов для каркаса эксплуатации CANVAS[11] компании Immunity Inc. Позже GLEG расширила свое предложение, включив собственный фаззер, который она использовала для выявления многих уязвимостей, предлагаемых через продукт VulnDisco. Полученный продукт называется ProtoVer Professional и разработан на языке программирования Python. На момент написания этот фаззер поддерживал следующие протоколы:

- IMAP - Internet Message Access Protocol.
- LDAP - Lightweight Directory Access Protocol.
- SSL - Secure Sockets Layer.
- NFS - Network File System.

Набор тестов включает различные интерфейсы, в том числе командную строку, GUI и веб-интерфейс, показанный на рис. 26.4. ProtoVer написан исследователями безопасности и,

по-видимому, ориентирован на тот же рынок; в настоящее время годовая лицензия на все поддерживаемые протоколы стоит 4500 долларов США. В лицензионную плату также входит поддержка по электронной почте.

Mu Security Mu-4000[12]

Mu Security предлагает Mu-4000 как аппаратное устройство, способное как мутировать сетевой трафик, так и отслеживать сбои цели. Mu-4000 предназначен прежде всего для тестирования других сетевых устройств и, как решение BreakingPoint, включает возможность отключения и включения питания целевого устройства для восстановления после обнаруженного сбоя. Другая интересная функция Mu-4000 состоит в том, что при обнаружении сбоя он может автоматически создать экспериментальный эксплойт в виде исполняемого файла для Linux, который Mu называет внешним инициатором уязвимости. Mu утверждает, что ее устройство способно работать с любым протоколом на основе IP. Пример отчета с подробными сведениями об исключениях, обнаруженных в реализации DHCP, показан на рис. 26.5.

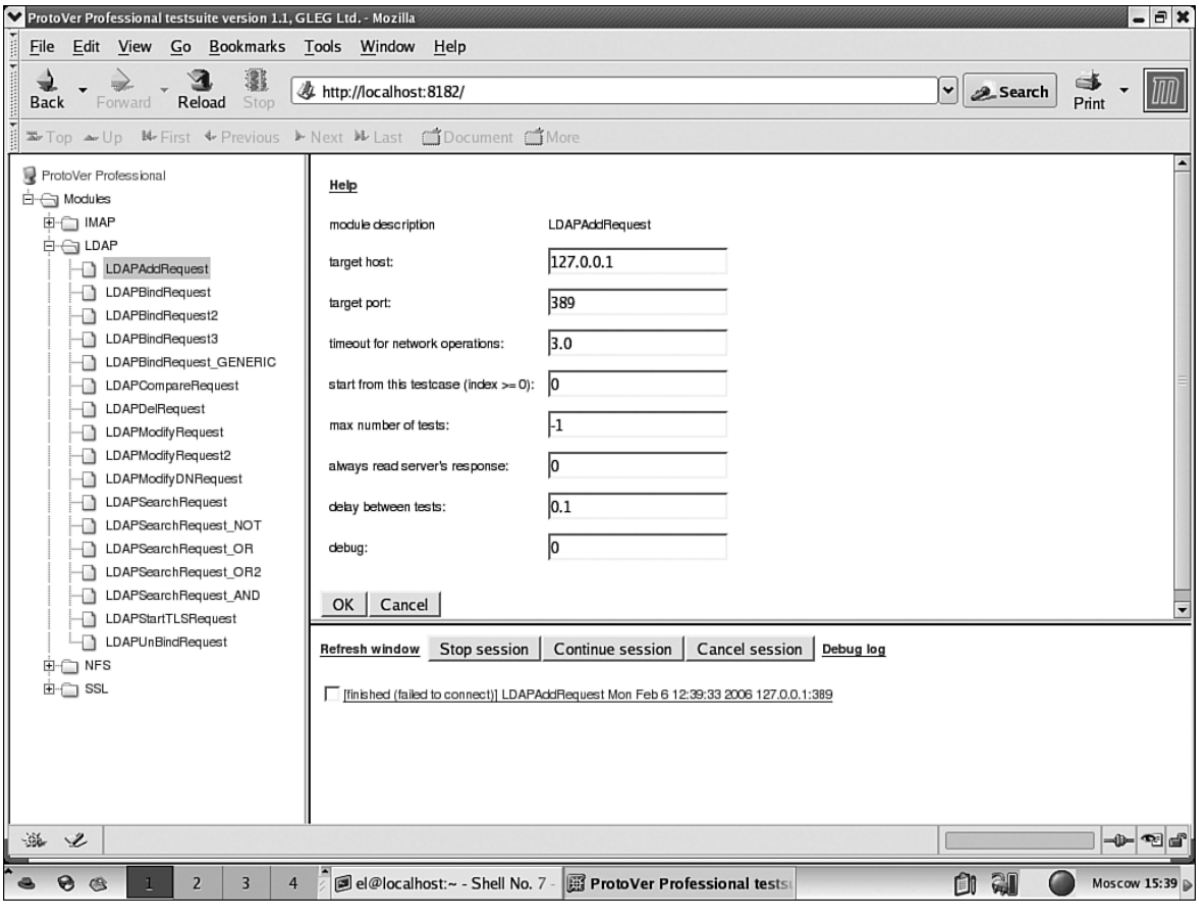
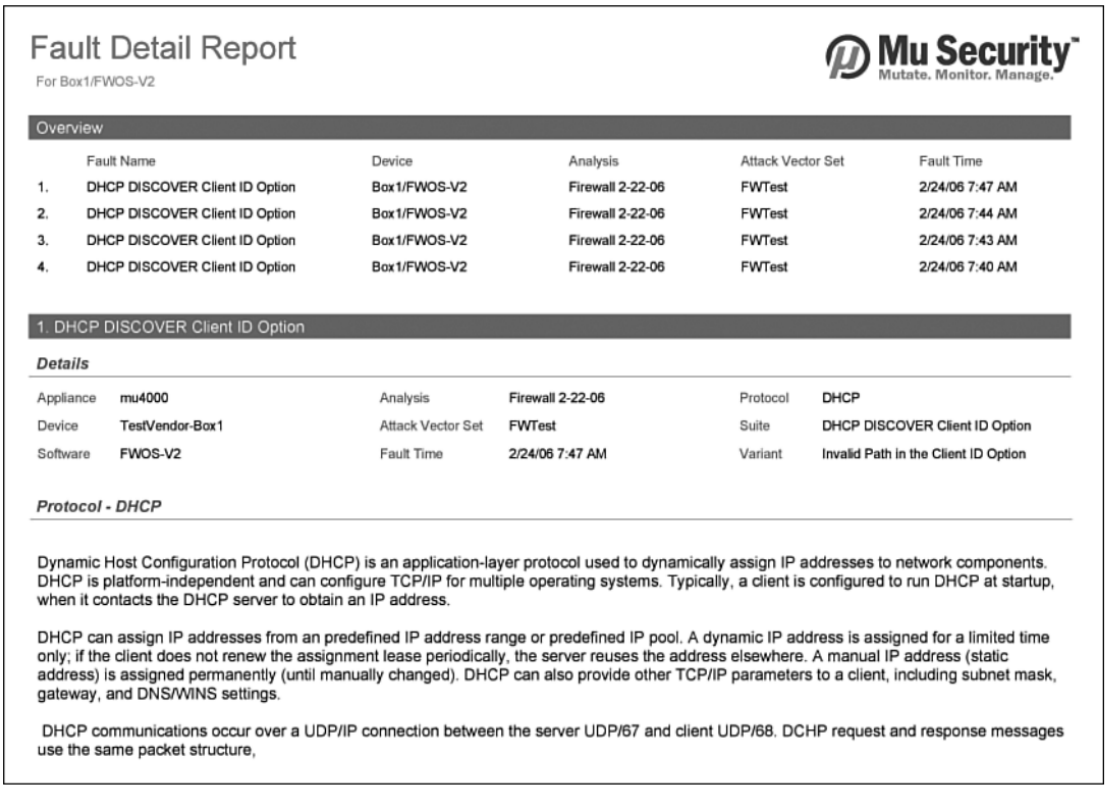


Рис. 26.4. Веб-интерфейс набора тестов GLEG ProtoVer Professional



Fault Detail Report

For Box1/FWOS-V2

Mu Security
Mutate. Monitor. Manage.

Overview

	Fault Name	Device	Analysis	Attack Vector Set	Fault Time
1.	DHCP DISCOVER Client ID Option	Box1/FWOS-V2	Firewall 2-22-06	FWTest	2/24/06 7:47 AM
2.	DHCP DISCOVER Client ID Option	Box1/FWOS-V2	Firewall 2-22-06	FWTest	2/24/06 7:44 AM
3.	DHCP DISCOVER Client ID Option	Box1/FWOS-V2	Firewall 2-22-06	FWTest	2/24/06 7:43 AM
4.	DHCP DISCOVER Client ID Option	Box1/FWOS-V2	Firewall 2-22-06	FWTest	2/24/06 7:40 AM

1. DHCP DISCOVER Client ID Option

Details

Appliance	mu4000	Analysis	Firewall 2-22-06	Protocol	DHCP
Device	TestVendor-Box1	Attack Vector Set	FWTest	Suite	DHCP DISCOVER Client ID Option
Software	FWOS-V2	Fault Time	2/24/06 7:47 AM	Variant	Invalid Path in the Client ID Option

Protocol - DHCP

Dynamic Host Configuration Protocol (DHCP) is an application-layer protocol used to dynamically assign IP addresses to network components. DHCP is platform-independent and can configure TCP/IP for multiple operating systems. Typically, a client is configured to run DHCP at startup, when it contacts the DHCP server to obtain an IP address.

DHCP can assign IP addresses from an predefined IP address range or predefined IP pool. A dynamic IP address is assigned for a limited time only; if the client does not renew the assignment lease periodically, the server reuses the address elsewhere. A manual IP address (static address) is assigned permanently (until manually changed). DHCP can also provide other TCP/IP parameters to a client, including subnet mask, gateway, and DNS/WINS settings.

DHCP communications occur over a UDP/IP connection between the server UDP/67 and client UDP/68. DCHP request and response messages use the same packet structure,

Рис. 26.5. Отчет Mu Security Mu-4000

Security Innovation Holodeck[13]

Holodeck компании Security Innovation - уникальное программное решение, позволяющее разработчикам эмулировать неожиданные сбои, чтобы исследовать надежность реализованных обработчиков безопасности и ошибок в программном обеспечении, разработанном для платформы Microsoft Windows. Продукт предоставляет типичные варианты поведения фаззинга, такие как возможность мутировать и файлы, и сетевые потоки. Среди более уникальных функций Holodeck - возможность вызывать следующие сбои:

• Сбои ресурсов при попытке обращения к определенным файлам, разделам и значениям реестра и объектам COM.
• Системные сбои, которые могут возникать в средах с ограниченным дисковым пространством, памятью и пропускной способностью сети.

Одна пользовательская лицензия Holodeck имеет разумную цену 1495 долларов США, а продукт предоставляет открытый API, позволяющий опытным пользователям создавать вокруг него инструменты автоматизации и расширения. Снимок главного представления Holodeck показан на рис. 26.6.

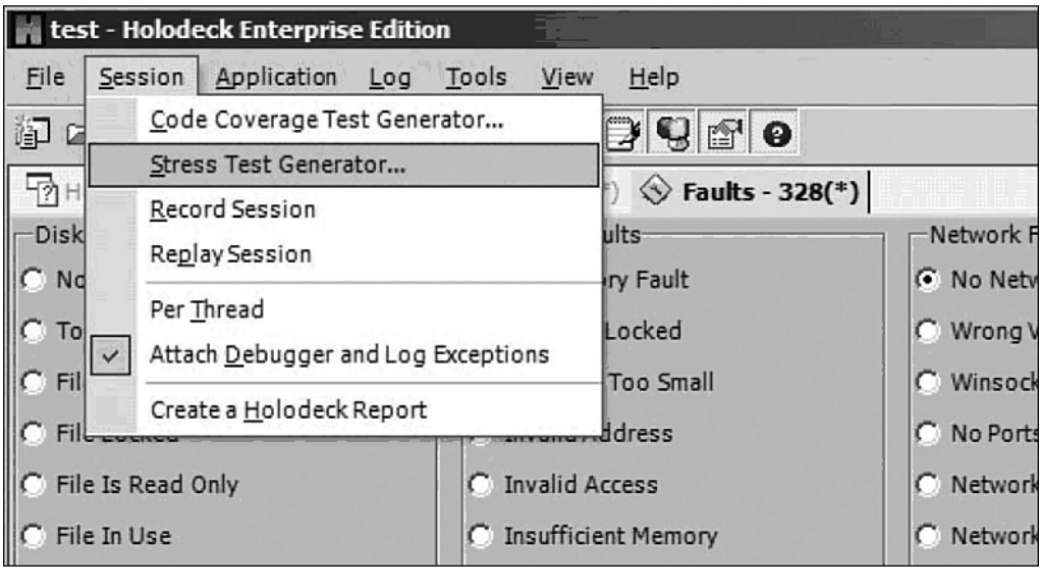


Рис. 26.6. Holodeck компании Security Innovation

Этот список компаний и продуктов ни в коем случае не является полным списком всех доступных коммерческих решений для фаззинга. Однако он дает представление о том, где отрасль находится сейчас. Большинство перечисленных решений - сравнительно новые технологии "версии 1". Отрасль технологий фаззинга, хотя и развивается, все еще сравнительно незрела.

Гибридные подходы к обнаружению уязвимостей

Обсуждавшиеся коммерческие фаззеры в основном представляют собой нишевые продукты, сосредоточенные на конкретном методе обнаружения уязвимостей. Если исследования уязвимостей чему-то нас научили, так это тому, что ни один подход не является волшебной пулей. У всех подходов есть преимущества и ограничения.

Зная это, мы ожидаем, что инструменты безопасности будут двигаться к форме гибридного анализа, объединяющей несколько подходов. Цель здесь - создать решение, в котором сумма частей больше целого. Возьмем, например, объединение анализа исходного кода с фаззингом. Одна из сложностей фаззинга - обеспечение достаточного покрытия кода. Как в сценарии чистого тестирования методом черного ящика можно быть уверенным, что фаззингу подверглись все возможные входные данные? В главе 23 "Отслеживание фаззера" мы обсуждали использование покрытия кода как показателя полноты усилий по фаззингу. На мгновение рассмотрим альтернативный подход с использованием гибридного анализа. Анализ начинался бы с автоматизированной проверки исходного кода. Эта традиционная форма тестирования методом белого ящика обычно выявляет многочисленные "потенциальные" уязвимости, но страдает от ложноположительных результатов, поскольку неспособна подтвердить результаты в окончательной форме приложения во время выполнения. Это ограничение можно устранить, используя результаты статического сканирования для помощи в создании случаев фаззинга, которые могут помочь доказать или опровергнуть эти потенциальные уязвимости на этапе динамического фаззинга аудита. Мы ожидаем, что фаззинг станет неотъемлемой частью гибридных технологий безопасности.

Интегрированные платформы тестирования

Новые и улучшенные разовые, автономные фаззеры всегда будут продолжать изобретаться и приниматься специалистами по безопасности. Однако группы QA и разработки с меньшей вероятностью найдут время и желание изучать тонкости передовых инструментов безопасности, учитывая, что тестирование безопасности всегда будет далеко отставать от необходимости выполнять требования и соблюдать сроки разработки. Чтобы получить широкое распространение в сообществах QA и разработки, технологии фаззинга должны войти в существующие платформы, с которыми эти группы уже знакомы и которые используют ежедневно. Ожидайте появления функций фаззинга, интегрированных в такие среды разработки, как Microsoft Visual Studio и Eclipse. Аналогично, популярные платформы QA, предлагаемые такими поставщиками, как IBM и Mercury, могли бы выиграть от возможностей фаззинга. Эти возможности могли бы предлагать поставщики или сопровождающие этих приложений, но первоначально их, скорее всего, будут разрабатывать сторонние организации и предлагать в виде подключаемых модулей приложений. Продукты DevInspect и QAInpect компании SPI Dynamics, хотя и не являются исключительно фаззерами, могли бы служить ранними примерами такого подхода.

Резюме

Куда отрасль движется дальше? Первые признаки указывают, что на горизонте более широкое внедрение технологий фаззинга. Мы приветствуем эту тенденцию и считаем фаззинг уникальной методологией в сообществе специалистов по безопасности по ряду причин. Что важнее всего, альтернативные методы, такие как обратное проектирование двоичных файлов и углубленный анализ исходного кода, требуют специализированных наборов навыков, которые нереалистичны для групп разработки и QA. Фаззинг, с другой стороны, можно реализовать как автоматизированный процесс, поэтому он доступен обеим группам.

Хотя независимые исследователи безопасности продолжат раздвигать границы и разрабатывать интересные методы фаззинга, для создания первой простой в использовании среды фаззинга, которая будет беспрепятственно интегрироваться в среды разработки, вероятно, потребуются усилия коммерческого поставщика. Одно из самых критических требований - усовершенствование обнаружения ошибок, как обсуждалось в главе 24. Методы отладки редко входят в комплект доступных сегодня фаззеров. Даже когда отладчики входят в комплект фаззеров, они, как правило, далеки от оптимальных. Для точного определения местоположения и автоматической классификации обнаруженных сбоев необходимо применять более совершенные методы. Поставщики средств обнаружения сбоев, такие как упомянутые в главе 24 IBM, Compuware, Parasoft и OC Systems, находятся в уникальном положении, позволяющем расширить их возможности и завершить комплект.

Исследуя историю похожих, но более зрелых технологий, можно ожидать нескольких ключевых событий. Во-первых, в эту область войдут более крупные, устоявшиеся компании, чтобы конкурировать с первыми первопроходцами. Поставщики могли бы разрабатывать собственные решения с нуля, но во многих случаях они приобретут некоторые из новаторских коммерческих технологий. Какие организации, вероятно, войдут в эту область? Трудно сказать с уверенностью, но ожидайте, что устоявшиеся поставщики программного обеспечения в областях безопасности и QA воспользуются возможностью отличиться от конкурентов посредством принятия технологий фаззинга.

Сноски

[1] [назад] http://www.beyondsecurity.com/
[2] [назад] http://www.securiteam.com/
[3] [назад] http://www.beyondsecurity.com/beSTORM_FAQ.pdf
[4] [назад] http://www.breakingpointsystems.com/
[5] [назад] http://www.codenomicon.com/
[6] [назад] http://www.ee.oulu.fi/research/ouspg/protos/
[7] [назад] http://www.ee.oulu.fi/research/ouspg/protos/testing/c06/snmpv1/index.html#h-ref2
[8] [назад] http://www.cert.org/advisories/CA-2002-03.html
[9] [назад] http://www.gleg.net/protover_pro.shtml
[10] [назад] http://www.gleg.net/products.shtml
[11] [назад] http://www.immunitysec.com/products-canvas.shtml
[12] [назад] http://www.musecurity.com/products/mu-4000.html
[13] [назад] http://www.securityinnovation.com/holodeck/